Advanced Algorithms for Discrete Optimization

Models, Paradigms, and Algorithms

Pierre Schaus

ICTEAM Institute
UCLouvain, Belgium



October 27, 2026

© 2026 Pierre Schaus

Advanced Algorithms for Discrete Optimization: Models, Paradigms, and Algorithms

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without prior written permission of the author.

Written with the assistance of artificial intelligence tools.

Published with LaTeX and biber.

First edition, October 27, 2026

Preface

Discrete optimization lies at the core of computer science, operations research, and applied mathematics. It addresses problems where decision variables take values from discrete domains, such as scheduling tasks, routing vehicles, allocating scarce resources, or configuring communication networks.

While many discrete optimization problems are computationally intractable in the worst case (**NP**-hard), real-world applications demand fast, robust, and often provably optimal solutions. To meet this challenge, several complementary algorithmic paradigms have been developed over the last several decades:

- **Dynamic Programming**: Decomposing problems into overlapping subproblems governed by the principle of optimality.
- **Branch and Bound**: Systematically enumerating the solution space while aggressively pruning non-promising regions through relaxation bounds.
- **Linear Programming & Duality**: Exploiting geometric and algebraic structures of polyhedra to find exact solutions or compute strong dual bounds.
- **Integer Linear Programming**: Formulating discrete problems as mixed-integer programs, exploiting total unimodularity, generating cutting planes, and executing LP-based branch-and-bound.
- **Lagrangian Relaxation**: Dualizing complicating constraints to decompose hard problems into tractable subproblems.
- **Network Flows**: Exploiting flow-cut duality, augmenting paths, scaling, and strongly polynomial combinatorial graph algorithms.
- **Local Search & Metaheuristics**: Exploring neighborhood structures to quickly uncover high-quality heuristic solutions in massive search spaces.
- **Constraint Programming**: Leveraging declarative modeling, domain filtering, and constraint propagation to prune infeasible regions efficiently.
- **Benchmarking Optimization Algorithms**: Rigorous empirical evaluation methodologies, shifted geometric means, and Dolan-Moré performance profiles.

Audience and Prerequisites

This book is designed for advanced graduate students (Master's and Ph.D.), researchers, and practitioners who want to master both the mathematical foundations and the practical implementation of modern discrete optimization techniques. Readers are expected to have a solid background in data structures, design and analysis of algorithms, graph theory, and basic linear algebra.

Pedagogical Approach and Software Framework

A distinguishing philosophy of this textbook is bridging mathematical rigor with high-performance algorithmic engineering. Theoretical formulations, polyhedral combinatorics, and optimality theorems are coupled with clean, idiomatic Java implementations. Rather than treating solvers as black boxes, students and practitioners are encouraged to examine, build, and extend core solver engines—including custom bounding procedures, branch-and-bound search strategies, and constraint propagators.

Structure of the Book

Each chapter is dedicated to one foundational paradigm. Chapters feature formal definitions, theoretical properties, pseudocode for key algorithms, concrete examples, and end-of-chapter exercises suitable for course assignments or self-study. The material can support a one-semester graduate course in discrete optimization or serve as a modular reference for specialized seminars in operations research and artificial intelligence.

Acknowledgments and AI Assistance

This textbook was written with the interactive assistance of artificial intelligence tools (specifically large language models). AI assistance was employed to support drafting, prose refinement, TikZ diagram typesetting, and Java code structuring under the continuous guidance, technical direction, and mathematical verification of the author. All pedagogical choices, theoretical formulations, proofs, and algorithmic designs remain the sole responsibility of the author.

Pierre Schaus
Louvain-la-Neuve, Belgium
October 27, 2026

Contents

Preface **iii**

1 Introduction to Discrete Optimization **1**
1.1 From Polynomial Algorithms to NP-Hardness . . . 1
1.2 Industrial Applications and Impact . . . 2
1.2.1 Vehicle Routing and Fleet Logistics (VRP) . . . 2
1.2.2 Workforce and Industrial Scheduling . . . 2
1.2.3 Cloud Computing and Resource Allocation . . . 3
1.2.4 Network Traffic Engineering . . . 3
1.3 Tackling NP-Hard Problems in Practice . . . 3
1.4 The Diagnostic Compass . . . 4
1.5 The Art of Practical Modeling . . . 4
1.5.1 High-Leverage Decision Variables . . . 5
1.5.2 Optimality with Imperfect Data . . . 5
1.6 Formal Problem Formulation . . . 6
1.7 A Taxonomy of Algorithmic Paradigms . . . 7
1.8 Generic Solvers and Practical Projects . . . 7
1.8.1 Generic Optimization Engines . . . 7
1.8.2 Hands-On Programming Projects . . . 8
1.8.3 Language Selection: Why Java? . . . 9
1.9 Exercises . . . 10

2 Dynamic Programming **11**
2.1 Computational Complexity of Knapsack . . . 11
2.1.1 The Polynomial-Time Reduction Technique . . . 12
2.2 Bellman Recurrence for Knapsack . . . 13
2.3 Naive Recursive Implementation . . . 14
2.3.1 Complexity Analysis of the Naive Recursion . . . 14
2.4 Memoization and Overlapping States . . . 14
2.4.1 Memoization with a Hash Map . . . 15
2.5 Tabulation with a 2D Array . . . 16
2.5.1 Iterative Algorithm (Bottom-Up) . . . 16
2.5.2 Subproblem Dependencies . . . 18
2.5.3 The Tabulation Matrix Illustrated . . . 19
2.5.4 Solution Reconstruction via Backtracking . . . 19
2.5.5 The Tabulation Grid as a DAG . . . 20
2.6 Complexity and Pseudo-Polynomial Time . . . 21
2.6.1 Is this Algorithm Polynomial? . . . 21

2.6.2 Weak vs. Strong NP-Hardness . . . 22
2.7 Case Study: Range Partitioning . . . 22
2.7.1 Dynamic Programming Formulation . . . 22
2.7.2 Prefix Sum Precomputation . . . 23
2.7.3 Java Implementation . . . 23
2.8 DP as Shortest Paths in a DAG . . . 24
2.9 A Generic Java Framework for DP . . . 25
2.9.1 The State Abstraction . . . 25
2.9.2 The Transition Class . . . 26
2.9.3 The Model Abstraction . . . 26
2.9.4 The Generic Solver Engine . . . 26
2.10 Knapsack in the Framework . . . 28
2.10.1 The Knapsack State . . . 28
2.10.2 The Knapsack Model . . . 28
2.11 Informed Search on the State DAG . . . 30
2.11.1 From Dijkstra to Heuristic Search . . . 30
2.11.2 The A^* Algorithm . . . 31
2.11.3 Admissibility and Optimality of A^* . . . 31
2.11.4 Consistent Heuristics . . . 33
2.12 Solving Dynamic Programs with A^* . . . 33
2.12.1 Knapsack as Minimization with LP Bound . . . 33
2.12.2 Negative Transition Weights and Robust A^* . . . 35
2.13 Weighted and Anytime Weighted A^* . . . 35
2.13.1 Weighted A^* Search . . . 36
2.13.2 Anytime Weighted A^* . . . 36
2.14 Java Implementation of A^* for DP . . . 38
2.14.1 Extending the Model and Solver Interfaces . . . 38
2.14.2 The A^* and Anytime Weighted A^* Solvers . . . 38
2.14.3 Case Study: The Traveling Salesperson Problem . . . 38
2.15 Exercises . . . 45

3 Branch and Bound **47**
3.1 Foundations of Branch and Bound . . . 47
3.1.1 The Three Core Mechanisms . . . 48
3.1.2 Maximization versus Minimization Duality . . . 49
3.2 Branch and Bound for 0/1 Knapsack . . . 50
3.2.1 Binary Search Tree Representation . . . 50
3.2.2 Bounding Procedure 1: Capacity Relaxation . . . 50
3.2.3 Bounding 2: Continuous LP Relaxation . . . 51
3.2.4 Worked Example and Search Tree . . . 53
3.3 Combinatorial Relaxations for the TSP . . . 56
3.3.1 TSP Relaxations and Formulations . . . 56
3.3.2 Comparison of the TSP Relaxations . . . 64
3.3.3 Branching Schemes and Partial Decisions . . . 64
3.4 Search Strategies and Optimality Gap . . . 70
3.4.1 Depth-First Search (DFS) . . . 70
3.4.2 Best-First Search (BFS) . . . 72
3.4.3 The Optimality Gap . . . 72
3.4.4 Hybrid Search: Diving (DFS-BFS) . . . 73

3.5 A Generic Java Framework for B&B . . . 73
3.5.1 Architecture and Core Interfaces . . . 73
3.5.2 Instantiating Knapsack in the Framework . . . 76
3.6 Practical Implementation: TSP in B&B . . . 78
3.7 Exercises . . . 79

4 Linear Programming **81**
4.1 Foundations of Linear Programming . . . 81
4.2 A Motivating Case: The Brewery Problem . . . 82
4.2.1 Problem Formulation . . . 82
4.2.2 Geometric Perspective in Two Dimensions . . . 83
4.3 Geometry in Higher Dimensions . . . 84
4.3.1 A Three-Dimensional Polytope Example . . . 84
4.3.2 Polyhedral Theory and Convexity . . . 84
4.3.3 Extreme Points and Vertex Optimality . . . 86
4.3.4 Combinatorial Explosion of Vertices . . . 87
4.4 Canonical and Standard Forms . . . 87
4.4.1 Motivation for Generic Formulations . . . 87
4.4.2 Canonical and Standard Equational Forms . . . 87
4.4.3 Form Transformations and Slack Variables . . . 88
4.5 The Simplex Algorithm . . . 89
4.5.1 Simplex by Example: The Brewery Problem . . . 89
4.5.2 Formalization of the Simplex Method . . . 92
4.5.3 Algorithmic Implementation in Java . . . 95
4.6 Initialization: Two-Phase and Big-M . . . 98
4.6.1 The Initial Basis Challenge . . . 98
4.6.2 The Two-Phase Simplex Method . . . 98
4.6.3 The Big-M Penalty Approach . . . 107
4.6.4 Comparison and Numerical Stability . . . 108
4.7 Duality Theory and Certificates . . . 108
4.7.1 The Primal-Dual Pair . . . 108
4.7.2 Weak and Strong Duality . . . 109
4.7.3 Complementary Slackness . . . 110
4.7.4 Dual Recovery and Shadow Prices . . . 110
4.8 Complexity, Cycling, and Solvers . . . 111
4.8.1 Degeneracy and Bland's Rule . . . 111
4.8.2 Worst-Case Exponential Complexity . . . 111
4.8.3 Polynomial-Time Interior Algorithms . . . 111
4.8.4 Warm-Starting and Solver Architecture . . . 112
4.9 Applications of Linear Programming . . . 112
4.9.1 The Nutritional Diet Problem . . . 112
4.9.2 From Continuous to Integer Programming . . . 112
4.10 Chapter Summary and Exercises . . . 113
4.10.1 Key Conceptual Takeaways . . . 113
4.10.2 Exercises . . . 113

5 Integer Linear Programming **117**
5.1 Foundations of Integer Linear Programming . . . 117
5.1.1 Mathematical Formulations . . . 117
5.1.2 The Geometry of the Integer Hull . . . 118
5.1.3 The Continuous Relaxation Bound . . . 118
5.2 Total Unimodularity and Polyhedral Integrality . . . 119
5.2.1 Network Flow and Bipartite Matching . . . 120
5.3 LP-Based Branch and Bound . . . 121
5.3.1 The Search Algorithm . . . 121
5.3.2 Branching Decisions: Variable Selection . . . 122
5.3.3 Updating the Simplex Tableau with Branching Constraints . . . 123
5.3.4 Dual Simplex Warm-Starting . . . 124
5.3.5 Dual Simplex Implementation in Java . . . 126
5.3.6 A Minimalistic Branch-and-Bound MIP Solver . . . 128
5.4 Cutting Planes and Branch-and-Cut . . . 132
5.4.1 Gomory Fractional Cuts . . . 132
5.4.2 The Branch-and-Cut Paradigm . . . 133
5.5 Mixed-Integer Linear Programming Modeling . . . 133
5.6 Chapter Summary and Exercises . . . 134
5.6.1 Key Conceptual Takeaways . . . 134
5.6.2 Exercises . . . 134

6 Lagrangian Relaxation **137**
6.1 The Lagrangian Relaxation Principle . . . 137
6.1.1 The Lagrangian Dual Problem . . . 138
6.2 Resource-Constrained Shortest Path . . . 139
6.2.1 Mathematical Formulation . . . 139
6.2.2 Lagrangian Dualization . . . 140
6.2.3 A Worked Example . . . 140
6.2.4 Geometry of the Lagrangian Dual . . . 142
6.3 Solving the Dual via Subgradients . . . 143
6.3.1 Subgradients and the Subdifferential . . . 143
6.3.2 The Subgradient Iteration Algorithm . . . 144
6.4 Lagrangian Relaxation for the TSP . . . 144
6.4.1 Structural Constraints of the TSP . . . 144
6.4.2 Dualizing Degree Constraints . . . 145
6.4.3 Modified Edge Costs and Multipliers . . . 146
6.4.4 Multiplier Invariance and Optimality . . . 146
6.4.5 A Worked Example on 4 Vertices . . . 147
6.5 Subgradient Optimization for Held–Karp . . . 149
6.5.1 Subgradients and Zero-Sum Property . . . 149
6.5.2 Adaptive Step Size Control . . . 149
6.5.3 The Held–Karp Subgradient Procedure . . . 150
6.6 Polyhedral Strength and Held–Karp . . . 152
6.6.1 The DFJ Formulation of the TSP . . . 152
6.6.2 Subtour LP and Held–Karp Equivalence . . . 154
6.6.3 Empirical Strength of the Bound . . . 155
6.7 Chapter Summary . . . 155
6.8 Exercises . . . 156

7 Network Flows **159**
7.1 Foundations and Problem Formulations . . . 159
7.1.1 Historical Context: Network Interdiction . . . 159
7.1.2 The Minimum Cut Problem . . . 160
7.1.3 The Maximum Flow Problem . . . 161
7.1.4 Case Study: Seminar Scheduling . . . 161
7.2 Linear Programming and Duality . . . 162
7.2.1 Primal and Dual Formulations . . . 162
7.2.2 Total Unimodularity and Integrality . . . 163
7.3 The Ford-Fulkerson Method . . . 164
7.3.1 Residual Graphs and Augmentation . . . 164
7.3.2 The Max-Flow Min-Cut Theorem . . . 167
7.4 Path Selection and Complexity . . . 169
7.4.1 Arbitrary Paths and Pseudo-Polynomiality . . . 169
7.4.2 The Capacity Scaling Algorithm . . . 170
7.4.3 Shortest Augmenting Path: Edmonds–Karp . . . 172
7.4.4 Maximum Capacity Augmenting Paths . . . 173
7.4.5 Comparison of Algorithmic Variants . . . 173
7.5 Object-Oriented Implementation . . . 173
7.5.1 The FlowEdge Data Structure . . . 173
7.5.2 The FlowNetwork Representation . . . 175
7.5.3 The Ford-Fulkerson Solver . . . 175
7.6 Modeling Reductions and Extensions . . . 177
7.6.1 Multiple Sources and Multiple Sinks . . . 177
7.6.2 Vertex Capacities and Node Splitting . . . 177
7.6.3 Lower Bounds and Edge Demands . . . 178
7.6.4 Bipartite Matching and Domain Filtering . . . 178
7.7 Minimum-Cost Flow . . . 179
7.7.1 Mathematical Formulation . . . 179
7.7.2 Residual Costs and Negative Cycles . . . 179
7.7.3 The Successive Shortest Path Algorithm . . . 181
7.8 Chapter Summary and Exercises . . . 181
7.8.1 Key Conceptual Takeaways . . . 182
7.8.2 Exercises . . . 182

8 Local Search and Metaheuristics **185**
8.1 Constructive Heuristics . . . 185
8.1.1 The Greedy Paradigm . . . 186
8.1.2 Greedy Heuristics for the TSP . . . 186
8.1.3 Beam Search and the Pilot Method . . . 191
8.2 Neighborhoods and Local Improvement . . . 193
8.2.1 Neighborhood Structures and Local Optima . . . 194
8.2.2 Pivot Rules: First vs. Best Improvement . . . 195
8.2.3 Design Principles for Neighborhoods . . . 196
8.3 Neighborhoods for the Traveling Salesman . . . 197
8.3.1 The 2-Opt Neighborhood . . . 197
8.3.2 Higher-Order Moves: 3-Opt and Or-Opt . . . 200
8.3.3 Reducing Asymmetric TSP to Symmetric TSP . . . 204
8.4 Neighborhood Extension and Ejection Chains . . . 207

8.4.1 Filter-and-Fan . . . 207
8.4.2 Lin–Kernighan and Ejection Chains . . . 208
8.5 Escaping Local Optima: Metaheuristics . . . 216
8.5.1 Large Neighborhood Search . . . 216
8.5.2 Tabu Search . . . 218
8.6 Constraint-Based Local Search . . . 221
8.6.1 Hard vs. Soft Constraints in CBLS . . . 222
8.6.2 Case Study: The Sudoku Puzzle . . . 222
8.6.3 Cumulative Scheduling: IFlat-IRelax . . . 223
8.7 Chapter Summary and Exercises . . . 227
8.7.1 Key Conceptual Takeaways . . . 230
8.7.2 Exercises . . . 231

9 Constraint Programming 235
9.1 The Declarative Programming Paradigm . . . 236
9.2 Modeling the n-Queens Problem . . . 237
9.2.1 Model 1: The Boolean Matrix Representation . . . 237
9.2.2 Model 2: The Column Decision Vector Representation . . . 238
9.3 Approach 1: Generate and Filter (DFS + Filter) . . . 239
9.3.1 Inversion of Control via the Hollywood Principle . . . 239
9.4 Approach 2: Prefix Pruning (DFS + Prune) . . . 241
9.4.1 Limitations of Hardcoded Prefix Pruning . . . 241
9.5 Architecture of a Generic Solver: `TinyCSP` . . . 243
9.5.1 Computational Paradigm: Search and Store . . . 243
9.5.2 The Fixpoint Propagation Engine . . . 244
9.5.3 Domain Representation with `java.util.BitSet` . . . 245
9.5.4 Variables and the `NotEqual` Propagator . . . 245
9.5.5 State Management and Search in `TinyCSP` . . . 247
9.6 Bound-Consistent Propagators . . . 247
9.6.1 Bound Pruning Operations on `Domain` . . . 251
9.6.2 The `LessOrEqual` Constraint ($x \leq y$) . . . 252
9.6.3 The Bound-Consistent `Sum` Constraint . . . 252
9.7 Performance and Data-Driven Propagation . . . 253
9.7.1 Profiling `TinyCSP`: Where Does the Time Go? . . . 255
9.7.2 From Naive Fixpoint to Data-Driven Propagation . . . 255
9.8 Modeling Expressiveness: CP vs. MIP on the TSP . . . 256
9.8.1 The Mixed-Integer Programming Formulation . . . 256
9.8.2 The Constraint Programming Formulation . . . 257
9.9 Combinatorial Applications in `TinyCSP` . . . 258
9.9.1 Magic Square and Symmetry Breaking . . . 258
9.9.2 The Killer Sudoku Problem . . . 260
9.9.3 Knight's Tour and Custom Constraints . . . 260
9.10 Exercises . . . 263

10 Benchmarking Optimization Algorithms 265
10.1 The Benchmarking Problem . . . 265
10.2 Pitfalls of Naive Aggregations . . . 267
10.2.1 Proposal 1: Arithmetic Mean of Runtimes . . . 267
10.2.2 Proposal 2: Counting Solved Instances . . . 267
10.3 Cactus Plots . . . 268

10.3.1 Strengths and Limitations of Cactus Plots . . . 269
10.4 Performance Profiles . . . 269
10.4.1 The Virtual Best Solver and Performance Ratios . . . 269
10.4.2 From Performance Ratios to the Cumulative Profile $\rho_s(\tau)$ . . . 270
10.4.3 Step-by-Step Interpretation of Performance Profiles . . . 270
10.4.4 Linear vs. Logarithmic Scale ($\log_2 \tau$) and Timeout Penalties . . . 272
10.5 Optimality Gaps and Gap Profiles . . . 273
10.5.1 Primal Bounds, Dual Bounds, and Relative Optimality Gap . . . 273
10.5.2 Joint Evaluation via Cactus Plots and Gap Profiles . . . 274
10.6 Anytime Behavior and Primal Integral . . . 275
10.6.1 Berthold's Primal Gap and Primal Integral . . . 276
10.7 Distributional and Pairwise Plots . . . 277
10.8 Aggregating Normalized Results . . . 278
10.8.1 The Paradox of the Arithmetic Mean of Normalized Results . . . 278
10.8.2 The Solution: The Geometric Mean . . . 279
10.9 Computing Benchmarking Metrics in Java . . . 280
10.10Exercises . . . 280

A Kruskal's Algorithm and Disjoint Sets **285**
A.1 The Minimum Spanning Tree Problem . . . 285
A.2 The Cut Property and Greedy Optimality . . . 286
A.3 Kruskal's Algorithm and Invariants . . . 287
A.4 The Disjoint-Set Data Structure . . . 288
A.4.1 Union by Rank and Path Compression . . . 288
A.5 Java Implementation . . . 289
A.6 Complexity and Comparison with Prim . . . 292
A.6.1 Kruskal vs. Prim: Trade-Offs . . . 292

References **293**

CHAPTER 1

Introduction to Discrete Optimization

Discrete optimization is the science and engineering of making optimal decisions from a finite or countably infinite set of possibilities. It stands at the intersection of computer science, operations research, and applied mathematics, providing the algorithmic backbone for managing complex modern infrastructure, logistics networks, manufacturing supply chains, and computing systems.

Unlike continuous optimization, where calculus, gradients, and smooth geometric structures guide search toward local or global extrema, discrete optimization operates on combinatorial landscapes. In these landscapes, moving a single discrete decision variable from 0 to 1 can drastically alter the feasibility and quality of an entire system.

1.1 From Polynomial Algorithms to NP-Hardness

In introductory courses on data structures and algorithms, students are introduced to a collection of celebrated polynomial-time algorithms:

- **Dijkstra's and Bellman-Ford's algorithms** for single-source shortest paths on graphs in $\mathcal{O}(|E| + |V| \log |V|)$ time.
- **Kruskal's and Prim's algorithms** for minimum spanning trees in $\mathcal{O}(|E| \log |V|)$ time.
- **Edmonds-Karp and push-relabel algorithms** for maximum network flow and bipartite matching in polynomial time.
- **Divide-and-conquer sorting** operating in $\mathcal{O}(n \log n)$ time.

These classical algorithms are elegant and practical. However, they share a common trait: they solve problems characterized by very specific algebraic or matroid structures that prevent combinatorial explosion.

When graduating from textbook abstractions to real-world industrial systems, a sobering truth emerges: **almost every real-world discrete optimization problem is NP-hard**. Consider what happens when we introduce a single realistic constraint into the classical polynomial problems:

- Find a shortest path between two nodes in a network: *Solvable in polynomial time* by Dijkstra's algorithm.
- Find a shortest path subject to a total transit budget or battery capacity limit (the Constrained Shortest Path Problem): *Immediately* **NP**-*hard* [GJ79].

- Find a minimum spanning tree connecting a set of terminals: *Solvable in polynomial time* by Prim's algorithm.
- Find a minimum spanning tree where the degree of each node cannot exceed 3 (the Degree-Constrained Spanning Tree Problem): *Immediately* **NP**-*hard*.
- Match jobs to identical machines to maximize throughput: *Solvable in polynomial time* via maximum bipartite matching.
- Match jobs to machines where tasks share disjunctive resources and strict deadlines: *Immediately* **NP**-*hard*.

The combinatorial explosion is not a rare mathematical curiosity; it is the default condition of real-world decision-making.

1.2 Industrial Applications and Impact

To appreciate the reach of discrete optimization, consider the critical industrial systems that rely on the algorithms developed in this book:

1.2.1 Vehicle Routing and Fleet Logistics (VRP)

Global e-commerce and logistics networks (such as Amazon, DHL, FedEx, and UPS) dispatch hundreds of thousands of delivery vehicles daily. The canonical *Vehicle Routing Problem (VRP)* requires determining an optimal set of routes for a fleet of vehicles based at one or more depots to service a set of customers with known demands:

- **Time Windows ($VRPTW$)**: Deliveries must arrive within customer-specified intervals (e.g., 10:00 AM – 11:30 AM).
- **Capacity and Compartment Constraints**: Vehicles have weight and volume thresholds, with refrigerated vs. non-refrigerated compartments.
- **Driver Rest and Regulations**: Routes must comply with legal driving hours and mandatory rest periods.

A fleet efficiency improvement of merely 1% in a major carrier translates into tens of millions of liters of fuel saved annually, hundreds of thousands of tons of reduced CO_2 emissions, and substantial cost reductions.

1.2.2 Workforce and Industrial Scheduling

Scheduling problems allocate scarce physical resources and human personnel over time:

- **Airline Crew Pairing and Rostering**: Major airlines (such as Air France-KLM, Delta, and Lufthansa) must assign tens of thousands of pilots and flight attendants to flights over monthly horizons. Feasible schedules must satisfy hundreds of regulatory requirements from civil aviation authorities (FAA, EASA), pilot certifications, rest periods, seniority privileges, and collective bargaining agreements.
- **Semiconductor Manufacturing**: In wafer fabrication plants (fabs), expensive cluster tools and photolithography machines process microchips through hundreds of sequential stages. Optimal job sequencing must account for sequence-dependent setup times, chemical re-entry constraints, and clean-room transport delays.
- **Operating Theater Scheduling**: In healthcare, hospitals must schedule elective surgeries across limited operating rooms, coordinating surgeon availability, post-operative

intensive care unit (ICU) beds, and specialized medical instrumentation under emergency disruption risks.

1.2.3 Cloud Computing and Resource Allocation

Hyper-scale cloud providers (such as Google Cloud, Microsoft Azure, and Amazon Web Services) host millions of virtual machines and containers on physical server racks across globally distributed data centers:

- **VM Placement and Consolidation**: Mapping customer workloads to physical servers to minimize total electricity and cooling power while avoiding CPU, RAM, and disk I/O bottlenecks.
- **High-Availability Constraints**: Preventing correlated failures by ensuring that primary and backup replicas of critical services are never hosted on the same physical server rack, power strip, or availability zone.

1.2.4 Network Traffic Engineering

Modern Internet backbones and Software-Defined Networks (SDN) deploy *Traffic Engineering (TE)* to route petabytes of data traffic across mesh topologies:

- Given a network topology graph $G = (V, E)$ with link capacities c_e and an origin-destination traffic matrix demand, compute packet paths (using Segment Routing or MPLS tunnels) that avoid link congestion, minimize end-to-end latency, and reserve failover capacity for link outages.

1.3 Tackling NP-Hard Problems in Practice

Recognizing that a problem is **NP**-hard is the beginning of the algorithm designer's task, not the end of it. **NP-hardness does not mean that a problem cannot be solved.**

Theoretical complexity analysis is inherently pessimistic: it characterizes the asymptotic computational effort required to solve the *pathological worst-case instance* among all conceivable inputs. However:

1. Real-world instances rarely resemble worst-case adversarial constructions. They exhibit geographic clustering, sparse interactions, metric triangle inequalities, and structured demand distributions.
2. Small- and medium-sized instances of **NP**-hard problems can routinely be solved to **proven mathematical optimality** using exact methods such as Branch-and-Bound, Branch-and-Cut, Dynamic Programming, and Constraint Programming.
3. Massive industrial instances with millions of variables can be solved to within fractions of a percent of optimality within minutes using decompositions, relaxations, and Large Neighborhood Search heuristics.

Example 1.1 *Exact Solvability at Scale: The Concorde TSP Solver*

A striking demonstration of exact optimization in action is the celebrated *Concorde TSP Solver*, developed by David Applegate, Robert Bixby, Vašek Chvátal, and William Cook [App+06].
Although the Traveling Salesperson Problem on n vertices has $(n-1)!/2$ possible tours—an astronomically vast search space even for modest n—Concorde routinely

solves instances of several hundred to a few thousand cities (such as the standard TSPLIB benchmarks with 1,000 to 2,000 cities) to **proven mathematical optimality in seconds or minutes** on an ordinary workstation.
Even more remarkably, Concorde holds the world record for the largest TSP instance solved to proven mathematical optimality: the benchmark instance `pla85900`, comprising **85,900 cities** derived from a programmable logic array circuit layout at Bell Laboratories.
Concorde achieves this through a state-of-the-art *Branch-and-Cut* framework: tightly coupling linear programming relaxations, iterative separation of facet-defining cutting planes (subtour elimination constraints, comb inequalities, and clique trees), primal metaheuristics (the Chained Lin-Kernighan algorithm) to uncover near-optimal upper bounds early, and branch-and-bound search. This achievement demonstrates that **NP**-hardness is not an insurmountable barrier: with deep polyhedral insight and algorithmic engineering, exact optimization routinely conquers instances of immense practical scale.

1.4 The Diagnostic Compass

When confronted with a new discrete optimization problem in academic research or industrial practice, the seasoned practitioner does not immediately open a solver or write code. Instead, they orient their algorithmic strategy by answering two fundamental questions:

The Two Diagnostic Questions of Discrete Optimization

1. **What size is your problem?**
   - How many discrete decision variables are involved? (Tens? Thousands? Millions?)
   - What is the planning horizon and constraint density? (A single warehouse with 10 vehicles, or a continental supply chain with 50,000 stops?)
2. **How much time do you give me to solve it?**
   - *Real-Time Online Operation*: Decisions must be returned in milliseconds or a few seconds (e.g., dynamic dispatch in ride-hailing, real-time packet rerouting).
   - *Operational Short-Turnaround*: Solutions needed within 1 to 15 minutes (e.g., flight gate re-assignment during an airport storm, morning truck dispatch).
   - *Tactical and Strategic Offline Planning*: Overnight or multi-hour computational budget (e.g., monthly pilot rostering, annual warehouse location).

The answers to these two questions determine which algorithmic paradigm—or combination of paradigms—should be deployed:

1.5 The Art of Practical Modeling

Solving a discrete optimization problem is not merely a mechanical application of algorithms; it is an **engineering art**. A sophisticated algorithm running on a poorly conceived model will fail, whereas a clean, insightful formulation can solve in seconds on standard solvers.

Table 1.1: Algorithmic Paradigm Selection Matrix based on Problem Scale and Time Budget.

Problem Scale	Generous Time Budget (Hours)	Tight Time Budget (Seconds to Minutes)
Small	**Exact Solvers**: Mixed-Integer Programming (MIP), Constraint Programming (CP), Branch-and-Bound. *Result: Provably optimal solution.*	**Dynamic Programming / Fast MIP**: Table lookup, prefix evaluation, or fast tree search with early cutoff. *Result: Optimal or near-optimal.*
Medium	**Decompositions**: Lagrangian Relaxation, Column Generation, Branch-and-Cut. *Result: Proven optimality or* $< 0.1\%$ *optimality gap.*	**Constraint Propagation + Heuristics**: CP with custom variable selection, restricted tree search. *Result: High-quality feasible solutions.*
Massive	**Hybrid Matheuristics**: Large Neighborhood Search (LNS) combined with MIP/CP subproblem repair. *Result: Deeply optimized solution with strong dual bounds.*	**Metaheuristics**: Tabu Search, Simulated Annealing, greedy randomized adaptive search (GRASP). *Result: Fast, robust heuristic solution.*

1.5.1 High-Leverage Decision Variables

The foremost task of the modeler is to discern which decisions provide high leverage and business value:

- Should we model the exact microsecond when a truck engine ignites, or simply the sequence of delivery stops and time windows?
- Should a railway dispatching model schedule every individual switch point and signal light, or model blocks of track between mainline stations?

There is an inevitable tension between **model fidelity** (capturing every operational nuance) and **computational tractability** (ensuring the model can be solved within the allotted time). The art of modeling lies in abstracting secondary details while rigorously preserving the constraints that dictate system feasibility.

1.5.2 Optimality with Imperfect Data

In academic theory, an algorithm designer strives to prove mathematical optimality down to an optimality gap of 0.00%. In real-world industrial practice, however, **obsessing over exact optimality on imperfect data is an illusion of precision**.

Consider two illustrative industrial realities:

1. **Industrial Manufacturing and Project Scheduling**: A mathematical model treats task durations as fixed integers (e.g., task A takes exactly 45 minutes). In a physical factory, however, task durations vary due to worker fatigue, micro-stoppages, machine calibration drifts, and raw material variations. A mathematically "optimal" schedule that packs jobs with zero buffer time will collapse the moment a task runs 3 minutes late. A slightly sub-optimal schedule with robust time buffers is practically superior to an optimal brittle schedule.
2. **Traffic Engineering in Computer Networks**: Network routing optimization computes paths assuming that the traffic demand between each origin and destination is known and summarized in a traffic matrix. In practice, a traffic matrix is only an *estimation* extrapolated from historical telemetry counters (such as NetFlow or IPFIX

samples). Actual network traffic fluctuates stochastically with video streaming bursts, software downloads, and unexpected link outages. Spending eight hours to find a routing scheme that is 0.05% better on yesterday's estimated traffic matrix is useless if today's traffic shifts by 10%.

When input data contains 5% uncertainty, spending hours of computational effort to close the final 0.1% optimality gap is an ineffective use of resources. What matters in practice is:

- **Robustness**: Solutions remain feasible and high-performing under modest real-world perturbations.
- **Speed of Response**: Delivering actionable solutions within operational decision cycles.
- **Transparent Constraints**: Providing operators with clear explanations of bottlenecks and trade-offs.

1.6 Formal Problem Formulation

Formally, a discrete optimization problem is defined as follows:

Definition 1.1 *Combinatorial Optimization Problem*

A *combinatorial optimization problem* is a tuple $(\mathcal{S}, \mathcal{X}, f)$, where:

- $\mathcal{S}$ is the configuration space, typically $\mathcal{S} \subseteq \mathbb{Z}^n$ or $\mathcal{S} \subseteq \{0,1\}^n$,
- $\mathcal{X} \subseteq \mathcal{S}$ is the set of *feasible solutions* satisfying a finite collection of constraints $g_j(\mathbf{x}) \leq 0$ for $j \in \{1, \ldots, m\}$,
- $f : \mathcal{S} \to \mathbb{R}$ is the *objective function*.

The goal is to find an optimal solution $\mathbf{x}^* \in \mathcal{X}$ such that:

$$f(\mathbf{x}^*) = \min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad \text{(for minimization)}, \quad \text{or} \quad f(\mathbf{x}^*) = \max_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad \text{(for maximization)}. \tag{1.1}$$

- If $\mathcal{X} = \emptyset$, the problem is **infeasible** (no configuration satisfies all constraints simultaneously).
- If there exist feasible solutions $\mathbf{x} \in \mathcal{X}$ with $f(\mathbf{x}) \to -\infty$ (for minimization), the problem is **unbounded**.
- Any configuration $\mathbf{x} \in \mathcal{X}$ is a **feasible solution**; its objective value $f(\mathbf{x})$ provides a valid *primal bound* on the global optimum $f(\mathbf{x}^*)$.

Theorem 1.1 *Combinatorial Explosion of Binary Spaces*

Let $\mathcal{S} = \{0,1\}^n$ be the space of binary decisions on n independent choices. The cardinality of the search space is:

$$|\mathcal{S}| = 2^n. \tag{1.2}$$

Adding a single variable doubles the size of the search space. For $n = 100$, $|\mathcal{S}| = 2^{100} \approx 1.27 \times 10^{30}$, a number exceeding the estimated number of grains of sand on Earth.

1.7 A Taxonomy of Algorithmic Paradigms

To navigate this combinatorial explosion, this book covers the foundational pillars of modern discrete optimization:

1. **Dynamic Programming (Chapter 2)**: Recursive decomposition into overlapping subproblems governed by Bellman's principle of optimality, DAG representations, and generic solver architectures.
2. **Branch and Bound (Chapter 3)**: Systematic state-space search over decision trees, employing mathematical relaxations and dominance bounds to prune non-promising subtrees.
3. **Linear Programming (Chapter 4)**: Continuous relaxations, polyhedral combinatorics, extreme point geometry, duality theory, and Simplex pivoting algorithms.
4. **Integer Linear Programming (Chapter 5)**: Polyhedral integrality, total unimodularity, LP-based branch-and-bound with dual simplex warm-starting, and cutting planes.
5. **Lagrangian Relaxation (Chapter 6)**: Decomposition of hard problems into tractable components by pricing out complicating constraints via subgradient optimization.
6. **Network Flows (Chapter 7)**: Maximum flow, minimum cut duality, Ford–Fulkerson, capacity scaling, Edmonds–Karp, and minimum-cost flows.
7. **Local Search & Metaheuristics (Chapter 8)**: Neighborhood structure exploration, trajectory methods (Simulated Annealing, Tabu Search), and Large Neighborhood Search (LNS) for rapid discovery of near-optimal solutions in massive spaces.
8. **Constraint Programming (Chapter 9)**: Declarative reasoning over finite variable domains, consistency notions, filtering algorithms for global constraints, and propagation engines.
9. **Benchmarking Optimization Algorithms (Chapter 10)**: Scientific methodology for empirical performance evaluation, shifted geometric means, Dolan-Moré performance profiles, and standard problem repositories.

1.8 Generic Solvers and Practical Projects

A defining philosophy of this course and textbook is that discrete optimization cannot be truly mastered solely through mathematical equations on paper or through one-off, hardcoded scripts. To develop the algorithmic intuition and problem-solving skills of an expert, one must bridge rigorous theoretical concepts with clean, robust, and extensible software architectures.

1.8.1 Generic Optimization Engines

In introductory computer science, dynamic programming or tree search algorithms are frequently presented as rigid, problem-specific routines—such as a hardcoded 2D array and nested loops for the 0/1 Knapsack problem. While such implementations demonstrate the recurrence relation, they fail to provide a reusable foundation for tackling new, unfamiliar combinatorial challenges.

In industrial operations research and advanced software engineering, practitioners do not reinvent search algorithms from scratch for each new application. Instead, they design and utilize **generic solver frameworks** governed by clean declarative abstractions:

- **Generic Dynamic Programming**: Decoupling the problem topology (states, transitions, rewards) from the search engine (memoization, topological traversal, and solution reconstruction). The same generic engine that solves Knapsack can solve the Traveling Salesperson Problem or shortest path problems without modifying a single line of search logic.
- **Generic Branch-and-Bound Engines**: Providing reusable search tree explorations, customizable node selection strategies, priority queues, and dual bounding interfaces independent of the concrete problem formulation.
- **Constraint Propagation Engines**: Implementing variable domain representations, dependency graphs, and event-driven filtering loops that execute independently of specific global constraints.
- **Metaheuristic and Local Search Architectures**: Designing modular neighborhood operators, accept/reject evaluators, and trajectory managers (e.g., Tabu Search and Large Neighborhood Search).

Throughout this textbook, every major algorithmic paradigm is presented alongside its generic software architecture, demonstrating how mathematical principles translate into elegant, production-grade tools.

1.8.2 Hands-On Programming Projects

To ground these theoretical abstractions in practice, the reader is invited to complete and extend some of these generic optimization tools across **six structured programming projects**:

1. **Project 1: Dynamic Programming**: Completing the core search and solution reconstruction engine (`getValueForState`, `getSolution`, `rebuildSolution`) in the generic parameterized DP framework (`Model<S>`, `State`, `Transition<S>`) in package `dynamicprogramming`. Validating the engine on 0/1 Knapsack, and then modeling and solving the Traveling Salesperson Problem (Held-Karp algorithm) using bitset state encodings (`TSPState`).
2. **Project 2: Branch-and-Bound and Lagrangian Relaxation**: Tackling the TSP with tree search and mathematical relaxations in package `branchandbound`. Implementing a simple 1-tree lower bound (`SimpleOneTree`), constructing a Branch-and-Bound solver (`BranchAndBoundTSP`), and deriving an enhanced dual bound reinforced by Lagrangian relaxation (`HeldKarpOneTree`) using subgradient optimization to penalize vertex degree violations.
3. **Project 3: Linear Programming, Simplex, and Maximum Flows**: Formulating combinatorial problems into standard linear programs ($\max \boldsymbol{cx}$ s.t. $\boldsymbol{Ax} \leq \boldsymbol{b}, \boldsymbol{x} \geq 0$) in package `linearprogramming`. Generating constraint matrices for Maximum Flow (`FlowMatrices`) and Maximum Bipartite Matching (`MatchingMatrices`), and implementing the Big-M Simplex method (`BigMSimplex`) to handle initial infeasibilities via lexicographic single-phase optimization, evaluated on Stigler's Diet problem.
4. **Project 4: Local Search and Invariants**: Developing a local search metaheuristic in `localsearch` for the Pigment Sequencing Problem (PSP), a discrete lot-sizing problem with sequence-dependent changeovers and deadlines. Implementing $\mathcal{O}(1)$ incremental cost invariants (`StockingCostInvariant`, `ChangeoverCostInvariant`), feasibility checks, restarts, and swap neighborhood exploration.
5. **Project 5: Constraint Programming**: Building core engine and filtering com-

ponents for the `TinyCSP` solver in package `constraintprogramming`. Implementing domain reduction operations (`removeAbove`, `removeBelow`) and bound-consistent filtering algorithms for global constraints (`Sum` and `LessOrEqual`), and modeling complex combinatorial puzzles such as the Magic Square and Killer Sudoku.

6. **Project 6: Dynamic Programming with A^* Search**: Unifying dynamic programming state spaces with heuristic search algorithms. Formulating a DP model for the Traveling Salesperson Problem and implementing admissible heuristic evaluation functions $h(s)$ inspired by Branch-and-Bound bounds. Implementing and benchmarking A^* search variants in package `astar`: standard A^*, Weighted A^* ($w_h \in \{1.25, 1.5, 2.0\}$) to trade bounded sub-optimality for search speed, and Anytime Weighted A^* to iteratively deliver and refine solutions under tight computational time limits.

All project starter code, automated unit test suites, benchmark problem instances, and validation harnesses are openly available in the accompanying GitHub repository:

`https://github.com/pschaus/linfo2266-students`

The internal mechanisms of these codebases are explained progressively throughout the chapters, directly accompanying the theoretical development of each topic.

1.8.3 Language Selection: Why Java?

A natural question arises: why is **Java** chosen as the primary programming language for this book and its projects, rather than C++ or Python?

In commercial operations research and high-performance solver development, low-level languages such as **C and C++** have historically been dominant. Commercial solvers (such as Gurobi, CPLEX, or Concorde) are predominantly implemented in C or C++ to exploit direct pointer manipulation, manual memory management, and bespoke cache layouts.

However, for a Master's level curriculum and for building deep algorithmic intuition, C++ presents significant pedagogical drawbacks:

- **Excessive Cognitive Overhead**: Complex template metaprogramming, manual memory management, dangling pointers, segmentation faults, and platform-dependent build configurations (Make, CMake, linker errors) divert the student's valuable cognitive energy away from discrete optimization principles toward systems-level debugging. The bar is unnecessarily high, making it harder to compile, debug, and rapidly experiment with algorithms.
- **Fragile Compilation and Toolchains**: Ensuring consistent compilation across macOS, Linux, and Windows environments when linking external C++ libraries can quickly become an unproductive hurdle.

At the opposite extreme, dynamically typed scripting languages such as **Python** offer rapid prototyping, but suffer from critical shortcomings for solver engineering:

- **Execution Speed**: The interpreted nature of Python incurs a performance penalty of $20\times$ to $100\times$ compared to compiled languages, making it impractical to execute deep tree searches with millions of nodes or high-frequency constraint propagation within acceptable time budgets.
- **Lack of Strict Static Typing**: Discrete optimization engines manipulate intricate, interconnected state graphs. Static typing is indispensable for defining clear contracts (such as `Model<S extends State>`) and catching conceptual and lifecycle bugs at compile time.

Java strikes the ideal balance between performance, safety, and readability:

1. **Strong Static Typing with Generics**: Java's robust type system and generic polymorphism enable clean, expressive abstractions for states, decisions, and models, allowing students to design generic solvers with ironclad compile-time safety.
2. **Sweet Spot Between Execution Speed and Readability**: Modern Java Virtual Machines (JVM) feature sophisticated HotSpot Just-In-Time (JIT) compilers, delivering execution throughput that is frequently within a factor of 1.5× to 2× of optimized C++ for memory-bound combinatorial algorithms, while maintaining clean, legible code.
3. **Frictionless Developer Experience**: Automatic garbage collection eliminates the danger of dangling pointers and memory leaks during complex tree traversals. Furthermore, Java guarantees seamless cross-platform portability, first-class IDE tooling, straightforward compilation, informative stack traces, and standardized unit testing with JUnit.

This pedagogical and engineering rationale mirrors the design philosophy of the seminal textbook *Algorithms* by Robert Sedgewick and Kevin Wayne [SW11], which famously adopted Java to teach foundational data structures and graph algorithms. By using Java, we ensure that students and practitioners spend their time thinking about state spaces, bounding functions, pruning theorems, and filtering logic—while still constructing production-grade tools that run with genuine industrial speed.

1.9 Exercises

Exercise 1.1 *Knapsack Formulation*

Formulate the 0/1 Knapsack problem with items $I = \{0, \ldots, n-1\}$, values $v_i > 0$, weights $w_i > 0$, and capacity $C > 0$ as a discrete optimization problem according to Definition 1.1. Explicitly specify the configuration space $\mathcal{S}$, the constraint defining $\mathcal{X}$, and the objective function f.

Exercise 1.2 *Diagnostic Compass for Industrial Problems*

For each of the following industrial scenarios, answer the two diagnostic questions (problem size and available time budget) and recommend an appropriate algorithmic paradigm from Table 1.1:

1. A ride-hailing platform matching 50 passengers to 50 nearby drivers every 3 seconds.
2. An airline creating a monthly crew schedule for 3,000 pilots across 15,000 flight legs over a weekend run.
3. An e-commerce warehouse routing 12 automated guided vehicles (AGVs) to pick 200 customer orders before a 2:00 PM shipping cutoff.

CHAPTER 2

Dynamic Programming

Dynamic Programming (DP) is one of the foundational algorithmic paradigms in discrete optimization, pioneered by Richard Bellman in the 1950s [Bel57]. It provides a systematic method for solving complex optimization problems by decomposing them into a collection of overlapping subproblems, solving each subproblem once, and storing their solutions in a table or cache to avoid redundant work.

Before diving into the mechanics of dynamic programming, it is crucial to understand the computational complexity of the problems we aim to solve. We begin with a rigorous reminder of the **NP**-completeness of the 0/1 Knapsack problem.

2.1 Computational Complexity of Knapsack

In discrete optimization, the *0/1 Knapsack Problem* is formulated as follows: given a set of n items $I = \{0, \dots, n-1\}$, where each item i has an integer profit $v_i > 0$ and an integer weight $w_i > 0$, and a knapsack with maximum weight capacity $C \in \mathbb{Z}^+$, select a subset of items to maximize the total profit without exceeding the capacity:

$$\begin{aligned} \text{maximize} \quad & \sum_{i \in I} v_i x_i \\ \text{subject to} \quad & \sum_{i \in I} w_i x_i \leq C \\ & x_i \in \{0,1\}, \quad \forall i \in I. \end{aligned} \tag{2.1}$$

To analyze its computational complexity from the perspective of complexity theory [GJ79], we consider its decision version:

Definition 2.1 *0/1 Knapsack Decision Problem (KNAPSACK)*

Given item values $v_0, \dots, v_{n-1} \in \mathbb{Z}^+$, weights $w_0, \dots, w_{n-1} \in \mathbb{Z}^+$, capacity $C \in \mathbb{Z}^+$, and a target profit threshold $K \in \mathbb{Z}^+$, does there exist a binary assignment $\mathbf{x} \in \{0,1\}^n$ such that:

$$\sum_{i=0}^{n-1} w_i x_i \leq C \quad \text{and} \quad \sum_{i=0}^{n-1} v_i x_i \geq K \ ? \tag{2.2}$$

2.1.1 The Polynomial-Time Reduction Technique

To prove that a decision problem B is **NP**-complete, one must follow the standard reduction methodology [Kar72]:

1. **Membership in NP**: Show that $B \in \mathbf{NP}$ by providing a polynomial-time verifier that takes an instance and a candidate certificate (witness) and checks its validity in polynomial time.
2. **Hardness by Reduction**: Choose a known **NP**-complete problem A and construct a function $f : \text{Instances}(A) \to \text{Instances}(B)$ such that:
   - f is computable in polynomial time in the size of the instance of A ($f \in \mathbf{P}$).
   - f preserves the answer: an instance x of A is a YES-instance if and only if $f(x)$ is a YES-instance of B:
   $$x \in A \iff f(x) \in B. \tag{2.3}$$

Definition 2.2 *The Subset-Sum Problem (SUBSET-SUM)*

Given a multiset of positive integers $A = \{a_0, a_1, \ldots, a_{n-1}\}$ and a target integer $T \in \mathbb{Z}^+$, does there exist a subset of indices $S \subseteq \{0, \ldots, n-1\}$ such that:

$$\sum_{i \in S} a_i = T \; ? \tag{2.4}$$

The Subset-Sum problem is one of Karp's 21 classic **NP**-complete problems [Kar72].

Theorem 2.1 *NP-Completeness of 0/1 Knapsack*

The Knapsack decision problem is **NP**-complete.

Proof. First, we show that Knapsack $\in \mathbf{NP}$. Given a candidate solution $\mathbf{x} = (x_0, \ldots, x_{n-1}) \in \{0,1\}^n$, a verifier computes $\sum_{i=0}^{n-1} w_i x_i$ and $\sum_{i=0}^{n-1} v_i x_i$, and checks whether the total weight is $\leq C$ and the total profit is $\geq K$. This requires $\mathcal{O}(n)$ additions and comparisons, which is polynomial in the input size. Hence, Knapsack $\in \mathbf{NP}$.

Second, we reduce Subset-Sum to Knapsack (Subset-Sum $\leq_P$ Knapsack). Let (A, T) be an arbitrary instance of Subset-Sum, where $A = \{a_0, \ldots, a_{n-1}\}$ and $T \in \mathbb{Z}^+$. We construct an instance of Knapsack as follows:

- We create n items, with item i having weight $w_i = a_i$ and value $v_i = a_i$ for all $i \in \{0, \ldots, n-1\}$.
- We set the knapsack capacity $C = T$.
- We set the target value threshold $K = T$.

This transformation requires copying n integers and setting $C = K = T$, which takes $\mathcal{O}(n)$ time.

We now establish correctness ($x \in$ Subset-Sum $\iff f(x) \in$ Knapsack):

- ($\Longrightarrow$) Suppose (A, T) is a YES-instance. Then there exists a subset $S \subseteq \{0, \ldots, n-1\}$ such that $\sum_{i \in S} a_i = T$. Setting $x_i = 1$ for $i \in S$ and $x_i = 0$ otherwise yields:
  $$\sum_{i=0}^{n-1} w_i x_i = \sum_{i \in S} a_i = T \leq C \quad \text{and} \quad \sum_{i=0}^{n-1} v_i x_i = \sum_{i \in S} a_i = T \geq K. \tag{2.5}$$
  Thus, the constructed knapsack instance is a YES-instance.

- ($\Longleftarrow$) Conversely, suppose the knapsack instance is a YES-instance. Then there exists $\mathbf{x} \in \{0,1\}^n$ such that $\sum_{i=0}^{n-1} w_i x_i \leq C = T$ and $\sum_{i=0}^{n-1} v_i x_i \geq K = T$. Because $w_i = v_i = a_i$ for each item:

$$T \leq \sum_{i=0}^{n-1} v_i x_i = \sum_{i=0}^{n-1} a_i x_i = \sum_{i=0}^{n-1} w_i x_i \leq T. \tag{2.6}$$

  Squeezing between T and T forces $\sum_{i=0}^{n-1} a_i x_i = T$. The set $S = \{i \mid x_i = 1\}$ is therefore a valid subset summing to T, proving (A, T) is a YES-instance.

Since the reduction is polynomial and sound, Knapsack is **NP**-complete. □

2.2 Bellman Recurrence for Knapsack

Since Knapsack is **NP**-hard, exhaustive search must evaluate all 2^n possible subsets of items. For $n = 50$, $2^{50} \approx 1.12 \times 10^{15}$ candidate solutions; testing one solution every microsecond would require more than 35,000 years.

Dynamic programming avoids this combinatorial explosion by formulating the problem recursively. Let the items be indexed $0, 1, \ldots, n-1$.

Definition 2.3 *Knapsack Subproblem State*

Let $O(j, k)$ denote the optimal (maximum) profit that can be obtained by selecting a subset from the prefix of items $\{0, 1, \ldots, j\}$ subject to a maximum capacity of k:

$$O(j,k) = \max_{\mathbf{x} \in \{0,1\}^{j+1}} \left\{ \sum_{i=0}^{j} v_i x_i \;\middle|\; \sum_{i=0}^{j} w_i x_i \leq k \right\}. \tag{2.7}$$

Our ultimate goal is to determine $O(n-1, C)$.

To compute $O(j, k)$, we consider the binary decision on item j:

1. **Do not select item** j ($x_j = 0$): The available capacity remains k, and the best achievable profit using the remaining items $\{0, \ldots, j-1\}$ is $O(j-1, k)$.
2. **Select item** j ($x_j = 1$): This decision is feasible only if $w_j \leq k$. If chosen, it consumes w_j units of capacity and provides profit v_j. The remaining capacity is $k - w_j$, and the optimal profit on the remaining items $\{0, \ldots, j-1\}$ is $O(j-1, k-w_j)$, yielding total profit $v_j + O(j-1, k-w_j)$.

Bellman's Principle of Optimality dictates that we simply take the maximum over these two options.

Theorem 2.2 *Bellman Recurrence for 0/1 Knapsack*

For any item index $j \in \{0, \ldots, n-1\}$ and capacity $k \in \{0, \ldots, C\}$:

$$O(j,k) = \begin{cases} 0 & \text{if } j < 0 \text{ or } k \leq 0, \\ O(j-1,k) & \text{if } w_j > k, \\ \max\Big(O(j-1,k),\ v_j + O(j-1,k-w_j)\Big) & \text{if } w_j \leq k. \end{cases} \tag{2.8}$$

2.3 Naive Recursive Implementation

Directly translating the recurrence relation (2.8) into Java yields the recursive procedure shown in Listing 2.1.

```java
public class KnapsackNaive {
    int[] value;
    int[] weight;
    int capa;
    int calls = 0; // counter to track recursive invocations

    public KnapsackNaive(int[] value, int[] weight,
                         int capa) {
        this.value = value;
        this.weight = weight;
        this.capa = capa;
    }

    public int solve() {
        return opt(value.length - 1, capa);
    }

    /**
     * Best value on items [0..j] with residual capacity k.
     */
    private int opt(int j, int k) {
        calls++;
        if (j < 0) {
            return 0;
        } else if (weight[j] > k) {
            return opt(j - 1, k);
        } else {
            int optWithout = opt(j - 1, k);
            int optWith = value[j]
                    + opt(j - 1, k - weight[j]);
            return Math.max(optWithout, optWith);
        }
    }
}
```

Listing 2.1: Naive recursive implementation of 0/1 Knapsack in Java (`KnapsackNaive.java`).

2.3.1 Complexity Analysis of the Naive Recursion

Let $T(j)$ denote the number of calls required to compute $\texttt{opt}(j, k)$ in the worst case (when $w_i \le k$ for all i):

$$T(j) = 2\,T(j-1) + \mathcal{O}(1), \quad T(0) = 1. \tag{2.9}$$

Solving this recurrence reveals that the recursion tree is a full binary tree of depth n:

$$T(n) = \sum_{\ell=0}^{n} 2^{\ell} = 2^{n+1} - 1 = \Theta(2^n). \tag{2.10}$$

The naive recursion re-computes identical subproblems an exponential number of times.

2.4 Memoization and Overlapping States

Why does the naive recursion take exponential time? The reason is *overlapping subproblems*.

Consider the following concrete instance:

$$\begin{aligned} \text{Values } v &= [1, 6, 18, 22, 28] \\ \text{Weights } w &= [2, 3, 5, 6, 7] \\ \text{Capacity } C &= 11. \end{aligned} \tag{2.11}$$

Let us observe two different decision paths from the root state ($j = 4, k = 11$):

- Path A: Do not take item 4, take item 3 ($w_3 = 6$), and do not take item 2 ($w_2 = 5$). The consumed weight is 6, leaving residual capacity $11 - 6 = 5$. The subproblem to solve is now $\texttt{opt}(1, 5)$.
- Path B: Do not take item 4, do not take item 3, but take item 2 ($w_2 = 5$). The consumed weight is again 5, leaving residual capacity $11 - 5 = 6$. Looking deeper, choosing items 0 and 1 ($w_0 + w_1 = 2 + 3 = 5$) also leaves residual capacity $11 - 5 = 6$.

Whenever two execution branches arrive at the same item index j with the **exact same remaining capacity** k, the subtrees rooted at those calls are identical. Solving them more than once is completely redundant.

2.4.1 Memoization with a Hash Map

The core idea of dynamic programming is **memoization**: storing the result of $\texttt{opt}(j, k)$ the first time it is computed, and looking it up in $\mathcal{O}(1)$ time whenever it is encountered again.

Listing 2.2 shows the implementation using a Java `HashMap` where the key is a pair (j, k).

```
1  import java.util.HashMap;
2  import util.Pair;
3
4  public class KnapsackHash {
5      int[] value;
6      int[] weight;
7      int capa;
8      HashMap<Pair<Integer, Integer>, Integer> cache;
9
10     public KnapsackHash(int[] value, int[] weight,
11                         int capa) {
12         this.value = value;
13         this.weight = weight;
14         this.capa = capa;
15         this.cache = new HashMap<>();
16     }
17
18     public int solve() {
19         return optCache(value.length - 1, capa);
20     }
21
22     private int optCache(int j, int k) {
23         Pair<Integer, Integer> entry = new Pair<>(j, k);
24         if (cache.containsKey(entry)) {
25             return cache.get(entry); // cache hit: O(1)
26         }
27         int res = opt(j, k);
28         cache.put(entry, res);       // cache store
29         return res;
30     }
31
32     private int opt(int j, int k) {
33         if (j < 0) return 0;
```

```
34          if (weight[j] > k) return optCache(j - 1, k);
35          int optWithout = optCache(j - 1, k);
36          int optWith = value[j]
37                  + optCache(j - 1, k - weight[j]);
38          return Math.max(optWithout, optWith);
39      }
40  }
```

Listing 2.2: Memoized Knapsack using a Hash Map (`KnapsackHash.java`).

Top-Down Sparsity: Reachable vs. Theoretical States. On our 5-item instance with $C = 11$, the Cartesian product of subproblem coordinates contains $(11 + 1) \times 5 = 60$ possible state pairs (j, k). However, top-down memoization visits and caches **only 30 distinct states**—evaluating exactly half of the theoretical state space by exploring only subproblems reachable along valid item selection paths, while completely bypassing the exponential recursion of the naive algorithm.

2.5 Tabulation with a 2D Array

While a `HashMap` provides an elegant top-down memoization strategy, it incurs object allocation overhead, auto-boxing, and hash collision costs. Because the state space is bounded by integer intervals $j \in \{0, \ldots, n-1\}$ and $k \in \{0, \ldots, C\}$, we can store the subproblem values in a simple **2D primitive integer array**:

$$\texttt{table}[k][i] \quad \text{where } k \in \{0, \ldots, C\},\ i \in \{0, \ldots, n-1\}. \tag{2.12}$$

Here, $\texttt{table}[k][i]$ stores the optimal profit achievable using a subset of items from $\{0, \ldots, i\}$ with maximum capacity k.

2.5.1 Iterative Algorithm (Bottom-Up)

Instead of recursion, we iteratively fill the table in topological order: from smaller item subsets to larger ones, and from smaller capacities to larger ones.

Algorithm 1: 0/1 Knapsack via 2D Array Tabulation

Input: Item values $v[0 \ldots n-1]$, weights $w[0 \ldots n-1]$, knapsack capacity C
Output: Optimal profit

Allocate 2D array $\texttt{table}[0 \ldots C][0 \ldots n-1] \leftarrow 0$;
`// Base case: item 0`
for $k \leftarrow w[0]$ **to** C **do**
  $\texttt{table}[k][0] \leftarrow v[0]$;
`// Fill table item by item`
for $i \leftarrow 1$ **to** $n-1$ **do**
  for $k \leftarrow 0$ **to** $w[i]-1$ **do**
    $\texttt{table}[k][i] \leftarrow \texttt{table}[k][i-1]$;
  for $k \leftarrow w[i]$ **to** C **do**
    $\texttt{optWithout} \leftarrow \texttt{table}[k][i-1]$;
    $\texttt{optWith} \leftarrow v[i] + \texttt{table}[k-w[i]][i-1]$;
    $\texttt{table}[k][i] \leftarrow \max(\texttt{optWithout}, \texttt{optWith})$;
return $\texttt{table}[C][n-1]$;

The Java implementation is given in Listing 2.3.

```java
public class KnapsackTable {
    int[] value;
    int[] weight;
    int capa;
    int[][] table;

    public KnapsackTable(int[] value, int[] weight,
                         int capa) {
        this.value = value;
        this.weight = weight;
        this.capa = capa;
        // table[k][i]: best value on items [0..i], capa k
        this.table = new int[capa + 1][value.length];
    }

    public int solve() {
        // Base case: item 0
        for (int k = weight[0]; k <= capa; k++) {
            table[k][0] = value[0];
        }
        // Fill table for items 1 to n-1
        for (int i = 1; i < value.length; i++) {
            for (int k = 0; k < weight[i]; k++) {
                table[k][i] = table[k][i - 1];
            }
            for (int k = weight[i]; k <= capa; k++) {
                int optWithout = table[k][i - 1];
                int optWith = value[i]
                        + table[k - weight[i]][i - 1];
                table[k][i] = Math.max(optWith, optWithout);
            }
        }
        return table[capa][value.length - 1];
    }

    /**
     * Reconstructs optimal subset by backtracking.
     * @return boolean array; true indicates item selected
     */
    public boolean[] reconstructSolution() {
        boolean[] selected = new boolean[value.length];
        int k = capa;
        for (int i = value.length - 1; i > 0; i--) {
            // Differing value means item i was taken
            if (table[k][i] != table[k][i - 1]) {
                selected[i] = true;
                k -= weight[i];
            }
        }
        // Base case: item 0
        if (k >= weight[0] && table[k][0] > 0) {
            selected[0] = true;
        }
        return selected;
    }
}
```

Listing 2.3: Iterative 2D table implementation of 0/1 Knapsack (`KnapsackTable.java`).

2.5.2 Subproblem Dependencies

A fundamental observation about the 2D dynamic programming grid is that the computation of any cell $\texttt{table}[k][i]$ depends *exclusively* on values in the immediately preceding column $i-1$. Specifically, every entry has **at most two incoming candidate edges**:

1. **The Horizontal Transition (Exclude Item i)**: If item i is not selected ($x_i = 0$), the knapsack capacity remains k and no value is added:

$$\texttt{table}[k][i-1] \xrightarrow{\text{reward } 0} \texttt{table}[k][i]. \tag{2.13}$$

2. **The Diagonal Transition (Include Item i)**: If item i is selected ($x_i = 1$), which is feasible only when $k \geq w_i$, item i consumes w_i units of capacity and contributes profit v_i:

$$\texttt{table}[k-w_i][i-1] \xrightarrow{\text{reward } +v_i} \texttt{table}[k][i]. \tag{2.14}$$

The stored value $\texttt{table}[k][i]$ is simply the maximum between these two incoming transitions. When the capacity is insufficient to accommodate item i ($k < w_i$), the diagonal edge is infeasible, and the cell trivially inherits its value from the horizontal predecessor $\texttt{table}[k][i-1]$.

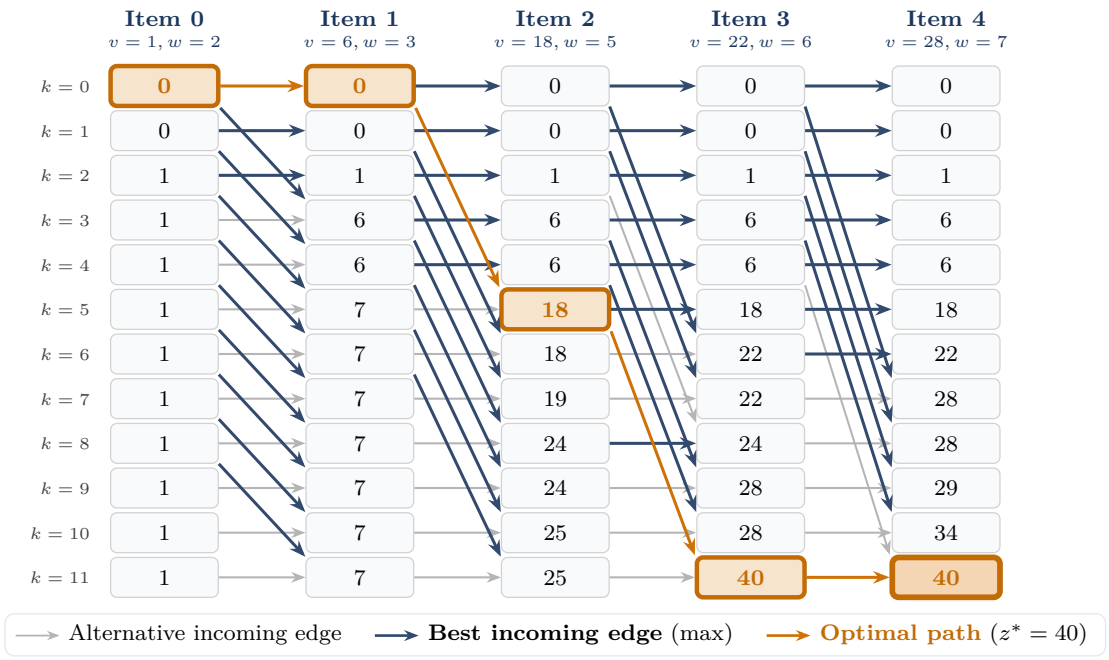


Figure 2.1: Complete 0/1 Knapsack dynamic programming table displaying all subproblem transitions across all 12×5 states. Faint gray arrows represent alternative candidate transitions; bold navy arrows indicate the **best incoming edge** (max) that determines each cell's value; the prominent amber trajectory highlights the **optimal solution path** originating at $(0, 0)$ and culminating at the optimal profit $\texttt{table}[11][4] = \mathbf{40}$.

Figure 2.1 visualizes this complete transition structure:

- **Faint gray arrows**: Transitions evaluated that did not achieve the maximum.
- **Bold navy arrows**: Winning transitions determining cell values (e.g., cell $(11, 3)$ selects the diagonal arrow from $(5, 2)$ since $18 + 22 = 40 > 25$).
- **Amber trajectory**: Tracing the winning arrows backwards from $\texttt{table}[11][4]$ identifies the exact **optimal path** and packed items.

2.5.3 The Tabulation Matrix Illustrated

On our running instance ($v = [1, 6, 18, 22, 28]$, $w = [2, 3, 5, 6, 7]$, $C = 11$), Table 2.1 displays the completed numerical values of the DP table. The optimal profit of **40** is found at the bottom-right corner `table`$[11][4]$.

Table 2.1: Dynamic Programming Table `table`$[k][i]$ for $C = 11$. Bold values with asterisks (*) mark the subproblems visited along the optimal solution path.

Capacity k	**Item 0** $(1, 2)$	**Item 1** $(6, 3)$	**Item 2** $(18, 5)$	**Item 3** $(22, 6)$	**Item 4** $(28, 7)$
0	**0***	**0***	0	0	0
1	0	0	0	0	0
2	1	1	1	1	1
3	1	6	6	6	6
4	1	6	6	6	6
5	1	7	**18***	18	18
6	1	7	18	22	22
7	1	7	19	22	28
8	1	7	24	24	28
9	1	7	24	28	29
10	1	7	25	28	34
11	1	7	25	**40***	**40***

2.5.4 Solution Reconstruction via Backtracking

While the matrix entry `table`$[C][n-1]$ computes the optimal numerical objective value ($z^* = 40$), practical applications require the actual binary decision vector:

$$\boldsymbol{x}^* = [x_0^*, x_1^*, \ldots, x_{n-1}^*] \in \{0, 1\}^n. \tag{2.15}$$

Because the table was filled in forward topological order, the optimal decisions can be recovered in reverse order by **backtracking** from cell $(C, n-1)$ back to stage 0, as formalized in Algorithm 2.

Algorithm 2: Knapsack Solution Reconstruction via Backtracking

Input: Completed DP table `table`$[0 \ldots C][0 \ldots n-1]$, values v, weights w, capacity C
Output: Binary decision vector $\boldsymbol{x} \in \{0, 1\}^n$

Allocate array $x[0 \ldots n-1] \leftarrow 0$;
$k \leftarrow C$;
for $i \leftarrow n-1$ **to** 1 ***step*** -1 **do**
  if *table*$[k][i] \neq$ *table*$[k][i-1]$ **then**
    $x[i] \leftarrow 1$; // Item i was selected
    $k \leftarrow k - w[i]$; // Deduct capacity consumed by item i
  else
    $x[i] \leftarrow 0$; // Item i was skipped
// Base case: evaluate item 0
if $k \geq w[0]$ ***and*** *table*$[k][0] > 0$ **then**
  $x[0] \leftarrow 1$;
return $\boldsymbol{x}$;

Backtracking Walkthrough

Applying Algorithm 2 to our running example reveals the exact sequence of optimal decisions:

1. **Start at** $(k = 11, i = 4)$: We compare $\texttt{table}[11][4] = 40$ with $\texttt{table}[11][3] = 40$. Because the values are identical, item 4 was **not** needed to achieve profit 40. We set $x_4^* = 0$, keep capacity $k = 11$, and step to $(11, 3)$.
2. **At** $(k = 11, i = 3)$: We compare $\texttt{table}[11][3] = 40$ with $\texttt{table}[11][2] = 25$. Because $40 \neq 25$, the value *must* have originated from including item 3! We set $x_3^* = 1$, deduct weight $w_3 = 6$ $(k \leftarrow 11 - 6 = 5)$, and step to $(5, 2)$.
3. **At** $(k = 5, i = 2)$: We compare $\texttt{table}[5][2] = 18$ with $\texttt{table}[5][1] = 7$. Because $18 \neq 7$, item 2 was selected! We set $x_2^* = 1$, deduct weight $w_2 = 5$ $(k \leftarrow 5 - 5 = 0)$, and step to $(0, 1)$.
4. **At** $(k = 0, i = 1)$: Here $k = 0 < w_1 = 3$. Item 1 cannot be packed. We set $x_1^* = 0$ and step to $(0, 0)$.
5. **At** $(k = 0, i = 0)$: Capacity is exhausted $(k = 0 < w_0 = 2)$. We set $x_0^* = 0$.

The reconstructed optimal solution is:

$$\boldsymbol{x}^* = [0, 0, \mathbf{1}, \mathbf{1}, 0], \tag{2.16}$$

selecting items 2 and 3 with total weight $5 + 6 = 11 \leq 11$ and total profit $18 + 22 = \mathbf{40}$.

Complexity of Solution Reconstruction. While computing the full dynamic programming table requires $\Theta(C \cdot n)$ time, reconstructing the optimal solution vector via backtracking requires only n comparisons and subtractions:

$$\text{Reconstruction Time} = \mathcal{O}(n), \qquad \text{Auxiliary Space} = \mathcal{O}(1). \tag{2.17}$$

Solution recovery is computationally negligible compared to the table construction phase.

2.5.5 The Tabulation Grid as a DAG

The observation that each cell in Figure 2.1 has incoming transitions with associated weights leads directly to a unifying theoretical concept: **the dynamic programming grid is a Directed Acyclic Graph (DAG)**, and finding the optimal solution is mathematically equivalent to finding the **Longest Path** in this DAG.

Let us define the knapsack state DAG $G = (V, E)$ explicitly:

- **Vertices** V: Every state (k, i) for $k \in \{0, \ldots, C\}$ and $i \in \{0, \ldots, n-1\}$, augmented with an initial source vertex $(0, \text{Start})$.
- **Edges** E: Directed edges connecting states at stage $i - 1$ to states at stage i:
  - *Skip edge*: $(k, i-1) \to (k, i)$ with edge weight 0 $(x_i = 0)$.
  - *Take edge*: $(k - w_i, i - 1) \to (k, i)$ with edge weight $+v_i$ $(x_i = 1$, valid for $k \geq w_i)$.

Because all directed edges point strictly from stage $i - 1$ to stage i, the graph cannot contain cycles: it is strictly a *layered acyclic graph*.

Notice that the dynamic programming recurrence (2.8) can be rewritten directly in graph-theoretic terms:

$$\texttt{table}[k][i] = \max_{(u,v) \in E,\, v=(k,i)} \left(\texttt{table}[u] + \text{weight}(u, v)\right). \tag{2.18}$$

Notice that this is *precisely* the Bellman-Ford recurrence for computing the **longest path in a Directed Acyclic Graph (DAG)** using topological ordering! Every path from the source $(0, \text{Start})$ to a terminal state represents a feasible item selection, its cumulative edge weight is the total profit, and the dynamic programming solution is simply the maximum-weight trajectory through the grid.

Keep This in Mind for Our Generic DP Solver. We invite the reader to remember this observation for later. In Section 2.8, we will formalize how *every* discrete dynamic program reduces to a shortest or longest path problem on a DAG. Furthermore, when we design our **generic Dynamic Programming solver framework in Java** in Section 2.9, we will capitalize directly on this graph abstraction: by viewing states as graph vertices and decisions as weighted transitions, we will construct an elegant, reusable solver engine that solves DP problems without requiring hand-crafted table dimensions or hardcoded loops.

2.6 Complexity and Pseudo-Polynomial Time

The 2D table contains $(C + 1) \times n$ cells. Computing the value in each cell requires $\mathcal{O}(1)$ operations (one table lookup, one addition, and one comparison). Therefore:

$$\text{Time Complexity} = \Theta(C \cdot n), \qquad \text{Space Complexity} = \Theta(C \cdot n). \tag{2.19}$$

2.6.1 Is this Algorithm Polynomial?

At first glance, $\Theta(C \cdot n)$ might appear polynomial because it is the product of C and n. **However, it is not polynomial in the theoretical sense.**

In computational complexity theory, the running time of an algorithm is measured as a function of the **length of the input**, denoted $|I|$, defined as the number of bits required to encode the instance:

- The number of items n requires $\mathcal{O}(\log n)$ bits.
- The weights and profits require $\mathcal{O}\left(\sum \log w_i + \sum \log v_i\right)$ bits.
- The integer capacity C is encoded in binary using:

$$L = \lceil \log_2(C + 1) \rceil \quad \text{bits.} \tag{2.20}$$

Notice that the numerical value C is exponentially related to its bit length L:

$$C \approx 2^L. \tag{2.21}$$

Expressing the runtime $\Theta(C \cdot n)$ in terms of the input bit-length L reveals:

$$\text{Runtime} = \Theta(n \cdot 2^L). \tag{2.22}$$

The algorithm is **exponential in the length of the input representing** C!

Definition 2.4 *Pseudo-Polynomial Time*

An algorithm for a problem whose inputs include numerical values is said to run in *pseudo-polynomial time* if its execution time is polynomial in the number of input elements n and the **numeric magnitudes** of the numbers involved, but exponential in the number of bits required to represent them.

2.6.2 Weak vs. Strong NP-Hardness

This distinction partitions **NP**-hard problems into two classes:

- **Weakly NP-Hard**: Problems (such as 0/1 Knapsack and Subset-Sum) that admit a pseudo-polynomial time algorithm. When data numbers are bounded by a polynomial in n (or small in practice, e.g., $C \leq 10^5$), dynamic programming solves them efficiently.
- **Strongly NP-Hard**: Problems (such as the Traveling Salesperson Problem, Vertex Cover, and Bin Packing) that remain **NP**-hard even if all numbers in the input are bounded by a polynomial in n. Strongly **NP**-hard problems cannot admit pseudo-polynomial time algorithms unless $\mathbf{P} = \mathbf{NP}$.

Practical Advice for Knapsack. If the capacity C and weights w_i share a greatest common divisor $g = \gcd(C, w_0, \ldots, w_{n-1}) > 1$, dividing all weights and C by g scales down the table size without altering the optimal solution. When C is exceptionally large (e.g., $C = 10^{12}$), dynamic programming on the profit space or fully polynomial-time approximation schemes (FPTAS) can be employed.

2.7 Case Study: Range Partitioning

To reinforce the dynamic programming methodology, we now examine an optimization problem where the objective is to minimize a maximum bottleneck cost: the *Range Partitioning Problem*.

Definition 2.5 *Range Partitioning Problem*

Given:

1. A sequence $S = [s_0, s_1, \ldots, s_{n-1}]$ of n non-negative numbers,
2. An integer $k \geq 1$,

partition S into at most k contiguous sub-ranges without reordering the elements, such that the **maximum sum over all ranges is minimized**.

Example 2.1 *Range Partitioning Example*

Let $S = [1, 2, 3, 4, 5, 6, 7, 8, 9]$ and $k = 3$. A feasible partition into 3 contiguous ranges is:

$$[1, 2, 3, 4, 5], \quad [6, 7], \quad [8, 9]. \tag{2.23}$$

The sums of these ranges are 15, 13, and 17, respectively. The maximum range sum is $\max(15, 13, 17) = \mathbf{17}$, which is the optimal solution.

2.7.1 Dynamic Programming Formulation

Let $O(i, \ell)$ denote the optimal objective value on the prefix sequence $[s_0, \ldots, s_i]$ using at most ℓ partitions (with $i \in \{0, \ldots, n-1\}$ and $\ell \in \{1, \ldots, k\}$).

To determine $O(i, \ell)$, we make a decision: *where to place the split* between the $(\ell - 1)$-th partition and the final (ℓ-th) partition. Suppose the last partition covers elements from index $j + 1$ to i, namely $[s_{j+1}, \ldots, s_i]$:

- The sum of the last partition is $\sum_{m=j+1}^{i} s_m$.
- The preceding prefix $[s_0, \ldots, s_j]$ is partitioned into at most $\ell - 1$ sub-ranges, yielding optimal bottleneck cost $O(j, \ell - 1)$.

- The bottleneck cost for this choice is $\max\left(O(j, \ell-1), \sum_{m=j+1}^{i} s_m\right)$.

Since we wish to minimize the bottleneck, we minimize over all valid split indices $j \in \{0, \ldots, i-1\}$:

$$O(i, \ell) = \min_{0 \le j \le i-1} \max\left(O(j, \ell-1), \sum_{m=j+1}^{i} s_m\right). \tag{2.24}$$

Base Cases:

1. With only $\ell = 1$ partition:

$$O(i, 1) = \sum_{m=0}^{i} s_m, \quad \forall i \in \{0, \ldots, n-1\}. \tag{2.25}$$

2. For a sequence of length 1 ($i = 0$):

$$O(0, \ell) = s_0, \quad \forall \ell \in \{1, \ldots, k\}. \tag{2.26}$$

The optimal global solution is given by $O(n-1, k)$.

2.7.2 Prefix Sum Precomputation

Computing $\sum_{m=j+1}^{i} s_m$ naively takes $\mathcal{O}(n)$ time. By precomputing a *prefix sum array*:

$$\texttt{prefixSum}[i] = \sum_{m=0}^{i} s_m, \quad \text{with } \texttt{prefixSum}[i] = \texttt{prefixSum}[i-1] + s_i, \tag{2.27}$$

any range sum is obtained in $\mathcal{O}(1)$ time:

$$\sum_{m=j+1}^{i} s_m = \texttt{prefixSum}[i] - \texttt{prefixSum}[j]. \tag{2.28}$$

2.7.3 Java Implementation

The complete Java implementation using 2D tabulation is presented in Listing 2.4.

```
1  import java.util.Arrays;
2
3  class RangePartitioningTable {
4      int[] input;
5      int[] prefixSum;
6      int n;
7      int nPartitions;
8      int[][] cache;
9
10     public RangePartitioningTable(int[] input, int k) {
11         this.input = input;
12         this.n = input.length;
13         this.nPartitions = k;
14         // cache[k][i]: best value for k parts on [0..i]
15         this.cache = new int[k + 1][n];
16
17         // Precompute prefix sums for O(1) interval queries
18         prefixSum = new int[n];
19         prefixSum[0] = input[0];
20         for (int i = 1; i < n; i++) {
```

```
            prefixSum[i] = prefixSum[i - 1] + input[i];
        }
    }

    private int sumInterval(int start, int end) {
        return prefixSum[end] - prefixSum[start - 1];
    }

    int solve() {
        // Base cases
        for (int k = 1; k <= nPartitions; k++) {
            // k partitions on sequence of 1 element
            cache[k][0] = input[0];
        }
        for (int i = 1; i < n; i++) {
            // 1 partition up to index i
            cache[1][i] = cache[1][i - 1] + input[i];
        }

        // Fill table: k in 2..nPartitions, i in 1..n-1
        for (int k = 2; k <= nPartitions; k++) {
            for (int i = 1; i < n; i++) {
                int min = Integer.MAX_VALUE;
                for (int j = 0; j <= i - 1; j++) {
                    int res = Math.max(
                        sumInterval(j + 1, i),
                        cache[k - 1][j]);
                    if (res < min) {
                        min = res;
                    }
                }
                cache[k][i] = min;
            }
        }
        return cache[nPartitions][n - 1];
    }
}
```

Listing 2.4: Range Partitioning with 2D Tabulation (`RangePartitioningTable.java`).

Theorem 2.3 *Complexity of Range Partitioning DP*

The dynamic program fills a table of size $(k+1) \times n$. For each cell (k, i), testing all split points $j \in \{0, \ldots, i-1\}$ takes $\mathcal{O}(i) = \mathcal{O}(n)$ time using prefix sums. Thus, the total time complexity is $\mathcal{O}(k \cdot n^2)$ and the space complexity is $\mathcal{O}(k \cdot n)$.

2.8 DP as Shortest Paths in a DAG

A unifying insight in discrete optimization is that **every discrete dynamic program can be reduced to a shortest or longest path problem in a Directed Acyclic Graph (DAG)**, specifically a *layered acyclic graph*.

Consider the state graph $G = (V, E)$ defined by the dynamic program:

- **Vertices** V: Each node corresponds to a reachable state $s \in \mathcal{S}$ (e.g., (j, k) for Knapsack).
- **Edges** E: A directed edge $(u, v) \in E$ corresponds to a feasible transition from state u to successor state v, with edge weight equal to the transition cost or value.

- **Source**: The root state $r \in V$ representing the unassigned problem (e.g., $(n-1, C)$).
- **Sink**: The terminal base case state $t \in V$ (e.g., $j < 0$ or $k = 0$).

Because decisions progress strictly through stages (e.g., item indices decrease monotonically $j \to j-1$), the graph contains no cycles: it is strictly a DAG.

Theorem 2.4 *DP Equivalence to DAG Paths*

- Any **maximization** DP problem is equivalent to finding a **Longest Path** from the root r to a sink in the state DAG.
- Any **minimization** DP problem is equivalent to finding a **Shortest Path** from the root r to a sink in the state DAG.

Why DAGs are Special. Finding the longest simple path in an arbitrary directed graph is a notorious **NP**-hard problem (it contains the Hamiltonian Path problem as a special case). However, on a **Directed Acyclic Graph**, topological sorting guarantees that longest and shortest paths can be solved in **linear time**:

$$\mathcal{O}(|V| + |E|). \tag{2.29}$$

Dynamic programming is precisely this linear-time traversal over the state DAG!

2.9 A Generic Java Framework for DP

Recognizing that every dynamic program is fundamentally a path search over a state DAG opens up an elegant software engineering opportunity: rather than writing ad-hoc loops and arrays for every new problem, we can build a **reusable, generic Dynamic Programming solver framework**.

We now present a generic framework, located in package `dynamicprogramming`. The framework is composed of four clean abstractions:

1. `State`: The nodes of the DAG.
2. `Transition<S>`: The directed edges of the DAG.
3. `Model<S>`: The declarative specification of the problem (root, transitions, base cases).
4. `DynamicProgramming<S>`: The generic solver engine that performs memoized search and solution reconstruction.

2.9.1 The State Abstraction

Every state must be hashable so that the generic solver can store it in a global hash table. In Java, this means providing implementations of `hashCode()` and `equals()`, as shown in Listing 2.5.

```
1  public abstract class State {
2      abstract int hash();
3      abstract boolean isEqual(State state);
4
5      @Override
6      public int hashCode() {
7          return this.hash();
8      }
9
10     @Override
```

```
    public boolean equals(Object o) {
        if (o instanceof State) {
            return isEqual((State) o);
        }
        return false;
    }
}
```

Listing 2.5: The `State` abstract class (`DynamicProgramming.java`).

2.9.2 The Transition Class

A transition represents a directed edge pointing to a successor state, carrying the decision identifier and the transition value (weight):

```
public class Transition<S extends State> {
    private S successor;
    private int decision;
    private double value;

    public Transition(S successor, int decision,
                      double value) {
        this.successor = successor;
        this.decision  = decision;
        this.value     = value;
    }

    public S getSuccessor()   { return successor; }
    public int getDecision()  { return decision; }
    public double getValue()  { return value; }
}
```

Listing 2.6: The `Transition` class (`DynamicProgramming.java`).

2.9.3 The Model Abstraction

The problem model defines the topology of the DAG: what the root state is, how to generate outgoing transitions, when a base case is reached, and whether the problem is a maximization or minimization:

```
import java.util.List;

public abstract class Model<S extends State> {
    abstract S getRootState();
    abstract boolean isBaseCase(S state);
    abstract double getBaseCaseValue(S state);
    abstract List<Transition<S>> getTransitions(S state);
    abstract boolean isMaximization();
}
```

Listing 2.7: The `Model` abstract class (`DynamicProgramming.java`).

2.9.4 The Generic Solver Engine

The generic solver class `DynamicProgramming<S>` implements top-down memoization using a hash map (`HashMap<State, Double>`) and reconstructs the optimal decision sequence:

```java
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import util.Solution;

public class DynamicProgramming<S extends State> {
    Model<S> model;
    HashMap<State, Double> table; // memoization cache

    public DynamicProgramming(Model<S> model) {
        this.model = model;
        this.table = new HashMap<>();
    }

    public Solution getSolution() {
        getValueForState(model.getRootState());
        return rebuildSolution();
    }

    private double getValueForState(S state) {
        if (model.isBaseCase(state)) {
            return model.getBaseCaseValue(state);
        }
        if (table.containsKey(state)) {
            return table.get(state);
        }

        double best = model.isMaximization()
                ? -Double.MAX_VALUE : Double.MAX_VALUE;
        for (Transition<S> transition :
                model.getTransitions(state)) {
            double succVal = getValueForState(
                    transition.getSuccessor());
            double value = succVal + transition.getValue();
            if (model.isMaximization()) {
                if (value > best) best = value;
            } else {
                if (value < best) best = value;
            }
        }
        table.put(state, best);
        return best;
    }

    private Solution rebuildSolution() {
        S currentState = model.getRootState();
        double best = table.get(currentState);
        List<Integer> decisions = new LinkedList<>();
        double currentValue = best;

        while (!model.isBaseCase(currentState)) {
            for (Transition<S> t :
                    model.getTransitions(currentState)) {
                double succVal = getValueForState(
                        t.getSuccessor());
                double value = succVal + t.getValue();
                if (value == currentValue) {
                    currentState = t.getSuccessor();
                    decisions.add(t.getDecision());
                    currentValue -= t.getValue();
```

```
                        break;
                    }
                }
            }
            return new Solution(best, decisions);
        }
    }
```

Listing 2.8: Core evaluation logic of `DynamicProgramming<S>`.

2.10 Knapsack in the Framework

With the generic solver in place, implementing a new dynamic program requires only two concise classes:

1. A problem-specific state class extending `State`.
2. A problem model class extending `Model<S>`.

2.10.1 The Knapsack State

The state encapsulates the current item index and residual capacity. It computes its hash code using `Objects.hash(item, capacity)`:

```
import java.util.Objects;

public class KnapsackState extends State {
    int item, capacity;

    public KnapsackState(int index, int capacity) {
        this.item = index;
        this.capacity = capacity;
    }

    @Override
    int hash() {
        return Objects.hash(item, capacity);
    }

    @Override
    boolean isEqual(State s) {
        if (s instanceof KnapsackState) {
            KnapsackState other = (KnapsackState) s;
            return this.item == other.item
                && this.capacity == other.capacity;
        }
        return false;
    }
}
```

Listing 2.9: Knapsack State implementation (`KnapsackState.java`).

2.10.2 The Knapsack Model

The `Knapsack` model defines the transitions corresponding to leaving or taking each item:

```
import java.util.LinkedList;
import java.util.List;
```

```java
import util.knapsack.KnapsackInstance;

public class Knapsack extends Model<KnapsackState> {
    KnapsackInstance instance;
    KnapsackState root;

    public Knapsack(KnapsackInstance instance) {
        this.instance = instance;
        this.root = new KnapsackState(0, instance.capacity);
    }

    @Override
    boolean isBaseCase(KnapsackState state) {
        return state.item == instance.n
            || state.capacity == 0;
    }

    @Override
    double getBaseCaseValue(KnapsackState state) {
        return 0.0;
    }

    @Override
    List<Transition<KnapsackState>> getTransitions(
            KnapsackState state) {
        List<Transition<KnapsackState>> transitions =
                new LinkedList<>();

        // Decision 0: Do not take the item
        transitions.add(new Transition<>(
            new KnapsackState(state.item + 1,
                              state.capacity),
            0,
            0.0
        ));

        // Decision 1: Take item if capacity allows
        if (instance.weight[state.item] <= state.capacity) {
            int remCapa = state.capacity
                    - instance.weight[state.item];
            transitions.add(new Transition<>(
                new KnapsackState(state.item + 1, remCapa),
                1,
                instance.value[state.item]
            ));
        }

        return transitions;
    }

    @Override
    KnapsackState getRootState() { return root; }

    @Override
    boolean isMaximization()     { return true; }
}
```

Listing 2.10: Knapsack Model implementation (`Knapsack.java`).

To solve the knapsack instance, the user simply runs:

```java
Knapsack model = new Knapsack(instance);
```

```
DynamicProgramming<KnapsackState> solver =
        new DynamicProgramming<>(model);
Solution solution = solver.getSolution();
```

The generic framework executes top-down memoized traversal, handles base cases, and reconstructs the optimal set of selected items seamlessly.

2.11 Informed Search on the State DAG

Top-down memoization (Section 2.9) and bottom-up tabulation (Section 2.5) share a fundamental characteristic: to determine the optimal solution at the root state r, they evaluate *every* state in the dynamic programming DAG that is reachable from r. When the reachable state space is modest, this exhaustive exploration is fast and simple. However, for strongly **NP**-hard problems such as the Traveling Salesperson Problem ($\mathcal{O}(n^2 2^n)$ states) or large-capacity resource allocation problems, the state DAG grows exponentially, causing exhaustive memoization to exhaust both time and memory.

Because solving a minimization dynamic program is equivalent to finding a shortest path from the root state $s = r$ to a terminal state t in the state DAG (Theorem 2.4), we can avoid expanding unpromising regions of the state space by guiding the traversal with *informed graph search*.

2.11.1 From Dijkstra to Heuristic Search

Given a directed graph $G = (V, E)$ with non-negative edge weights $w : E \to \mathbb{R}^+$ and a source vertex $s \in V$, *Dijkstra's algorithm* [Dij59] computes the shortest-path distance $d[v]$ from s to all vertices $v \in V$, as stated in Algorithm 3.

Algorithm 3: Dijkstra's Shortest Path Algorithm [Dij59]

Input: Graph $G = (V, E)$, edge weights $w : E \to \mathbb{R}^+$, source vertex $s \in V$
Output: Shortest-path distances $d[v]$ and predecessors $\pi[v]$ from s

foreach *vertex* $v \in V$ **do**
  $d[v] \leftarrow +\infty$; $\pi[v] \leftarrow$ NIL;
$d[s] \leftarrow 0$; $C \leftarrow \emptyset$; // Closed set of expanded vertices
$O \leftarrow \emptyset$; Insert$(O, s, d[s])$; // Open set: min-priority queue
while $O \neq \emptyset$ **do**
  $u \leftarrow$ ExtractMin(O); $C \leftarrow C \cup \{u\}$;
  foreach *vertex* $v \in Adj[u]$ **do**
    if $d[v] > d[u] + w(u, v)$ **then**
      $d[v] \leftarrow d[u] + w(u, v)$; $\pi[v] \leftarrow u$;
      InsertOrUpdateKey$(O, v, d[v])$;

The algorithm partitions discovered vertices into two sets:

- **The Open Set** O: A min-priority queue containing discovered states that remain to be expanded, ordered by their best known distance $d[v]$ from s.
- **The Closed Set** C: The set of states that have already been extracted from O and expanded.

Implemented with a binary heap, Dijkstra's algorithm runs in $\mathcal{O}(|E| \log |V|)$ time. However, as illustrated in Figure 2.2a, Dijkstra's algorithm is *uninformed* regarding the location of the

target t: because it orders the priority queue solely by the accumulated distance $g(u) = d[u]$ from s, its search frontier expands radially in an equidistant ball around s. A state n_0 moving in the opposite direction of t with $g^*(n_0) = r$ is treated as equally promising as a state n_1 lying directly on the optimal path to t with $g^*(n_1) = r$.

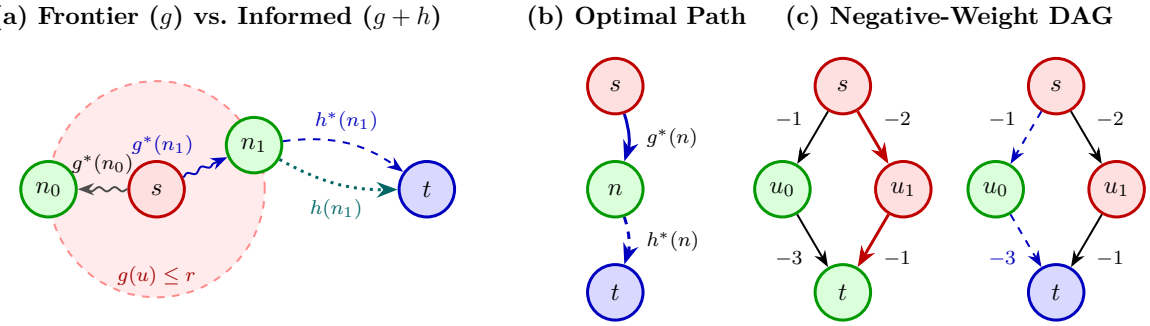


Figure 2.2: Principles of informed search on state graphs. (a) Dijkstra's algorithm expands states in order of their distance $g(u)$ from the source s, treating n_0 and n_1 equally on the equidistant frontier ball, whereas A^* adds a lower-bound heuristic estimate $h(n_1) \le h^*(n_1)$ of the remaining cost to guide expansion toward the target t. (b) Decomposition of an optimal $s \rightsquigarrow t$ trajectory through an unexpanded open state n with exact prefix cost $g^*(n)$ and remaining optimal cost $h^*(n)$. (c) On a DAG with negative edge weights, greedy extraction without re-opening or incumbent checks expands u_1 ($g = -2$) and prematurely terminates at t ($g = -3 < -1$) before expanding u_0 along the true optimal path of cost -4.

2.11.2 The A^* Algorithm

To focus the search toward the goal t, Hart, Nilsson, and Raphael [HNR68] introduced the A^* *algorithm*. For any state $u \in V$, let:

- $g^*(u)$ denote the true optimal path cost from the start state s to u, and $g(u) \ge g^*(u)$ denote the cost of the best path from s to u discovered so far;
- $h^*(u)$ denote the true optimal path cost from u to a terminal goal state t, and $h(u)$ denote a *heuristic function* that estimates $h^*(u)$ (for instance, the straight-line Euclidean distance in geometric routing, or a linear programming relaxation bound in combinatorial optimization).

Instead of ordering the open priority queue O by $g(u)$, A^* orders states by the *evaluation function*:

$$f(u) = g(u) + h(u), \tag{2.30}$$

which represents the estimated total cost of a complete path from s to t passing through u. The complete procedure is formalized in Algorithm 4.

2.11.3 Admissibility and Optimality of A^*

Under what condition on the heuristic h is Algorithm 4 guaranteed to return an optimal solution?

Algorithm 4: The A^* Search Algorithm [HNR68]

Input: Graph $G = (V, E)$, edge weights $w : E \to \mathbb{R}^+$, source s, goal t, heuristic $h : V \to \mathbb{R}$
Output: Optimal distance $g[t]$ and predecessor map π

foreach *vertex* $v \in V$ **do**
  $g[v] \leftarrow +\infty$; $f[v] \leftarrow +\infty$; $\pi[v] \leftarrow$ NIL;
$g[s] \leftarrow 0$; $f[s] \leftarrow h(s)$; $C \leftarrow \emptyset$;
$O \leftarrow \emptyset$; Insert$(O, s, f[s])$; `// Min-priority queue ordered by` $f[v]$
while $O \neq \emptyset$ **do**
  $u \leftarrow$ ExtractMin(O); $C \leftarrow C \cup \{u\}$;
  if $u = t$ **then**
    return $g[t]$, π ; `// Goal reached`
  foreach *vertex* $v \in Adj[u]$ **do**
    if $v \notin C$ ***and*** $g[u] + w(u, v) < g[v]$ **then**
      $g[v] \leftarrow g[u] + w(u, v)$;
      $f[v] \leftarrow g[v] + h(v)$; `// Score = path cost + heuristic`
      $\pi[v] \leftarrow u$;
      InsertOrUpdateKey$(O, v, f[v])$;
return No path to goal;

Definition 2.6 *Admissible Algorithm and Admissible Heuristic*

A search algorithm is said to be *admissible* if it is guaranteed to return an optimal solution whenever a path to a goal state exists. A heuristic function $h : V \to \mathbb{R}$ is *admissible* if it never overestimates the true minimum remaining cost to reach a goal state:

$$h(u) \leq h^*(u), \quad \forall u \in V, \tag{2.31}$$

with $h(t) = 0$ at any terminal goal state t. In minimization terminology, an admissible heuristic is a valid *lower bound* on the optimal completion cost from state u.

Theorem 2.5 *Optimality of A^* Search*

If $G = (V, E)$ has non-negative edge weights (or no negative-cost cycles and nodes are re-opened when improved) and the heuristic h is admissible ($h(u) \leq h^*(u)$ for all $u \in V$), then A^* is admissible: the first time the goal state t is extracted from O, its cost $g[t]$ equals the optimal path cost C^*.

Proof. We proceed by contradiction (Figure 2.2b). Let C^* denote the optimal path cost from s to t, and suppose that A^* terminates by extracting t from O with a suboptimal path cost $C = g[t] = f[t] > C^*$.

Because an optimal path from s to t exists and A^* did not find it, there must exist at least one vertex n along an optimal $s \rightsquigarrow t$ path that remains unexpanded in the open set ($n \in O$) at the moment t is extracted, and whose ancestors along the optimal path have all been expanded with their exact optimal prefix costs, so that $g(n) = g^*(n)$.

1. Because A^* extracted t from the min-priority queue O instead of n, we must have:
$$f(n) \geq f(t) = g(t) = C > C^*. \tag{2.32}$$
2. On the other hand, since n has already been reached along the optimal prefix from s ($g(n) = g^*(n)$) and h is admissible ($h(n) \leq h^*(n)$), the evaluation function at n satisfies:
$$f(n) = g(n) + h(n) = g^*(n) + h(n) \leq g^*(n) + h^*(n) = C^*, \tag{2.33}$$
where the final equality $g^*(n) + h^*(n) = C^*$ holds because n lies on an optimal path from s to t.

Combining (2.32) and (2.33) yields $C^* < f(n) \leq C^*$, a contradiction. Therefore, $C = C^*$. □

2.11.4 Consistent Heuristics

Notice that in Algorithm 4, once a state u is added to the closed set C, it is never expanded again ($v \notin C$). This single-expansion guarantee relies on a structural property of the heuristic known as *consistency* (or *monotonicity*).

Definition 2.7 *Consistent (Monotone) Heuristic*

A heuristic $h : V \to \mathbb{R}$ is *consistent* if $h(t) = 0$ at every terminal state t and, for every state $u \in V$ and every successor $v \in \text{Adj}[u]$, h satisfies the triangle inequality:
$$h(u) \leq w(u, v) + h(v). \tag{2.34}$$

Adding $g(u)$ to both sides of (2.34) shows that along any transition (u, v) with $g(v) = g(u) + w(u, v)$, the evaluation function f is monotonically non-decreasing:
$$f(u) = g(u) + h(u) \leq g(u) + w(u, v) + h(v) = g(v) + h(v) = f(v). \tag{2.35}$$

Consequently, when h is consistent, A^* extracts states from O in non-decreasing order of f-values, and the very first time any state $u \in V$ is extracted from O, its path cost is guaranteed to be optimal ($g(u) = g^*(u)$). It therefore never needs to be re-inserted into O or have its priority decreased after closing. In practice, natural relaxation-based heuristics (such as linear programming or minimum spanning tree relaxations) are almost always both admissible and consistent.

2.12 Solving Dynamic Programs with A^*

Let us now apply A^* directly to solve dynamic programming models by searching their state transition DAGs.

2.12.1 Knapsack as Minimization with LP Bound

Recall the 0/1 Knapsack problem from Section 2.1 over items $I = \{0, 1, \ldots, n-1\}$ with profits $v_i > 0$, weights $w_i > 0$, and capacity C. Because A^* is formulated as a shortest-path (minimization) algorithm, we convert the maximization objective $\max \sum_{i=0}^{n-1} v_i x_i$ into an equivalent minimization objective by negating the item profits:
$$\text{minimize} \quad \sum_{i=0}^{n-1} (-v_i) x_i \qquad \text{subject to} \quad \sum_{i=0}^{n-1} w_i x_i \leq C, \quad x_i \in \{0, 1\}. \tag{2.36}$$

In the state DAG, each state is a pair (i, k) representing the current item index $i \in \{0, \ldots, n\}$ and the residual capacity $k \in \{0, \ldots, C\}$, starting from the root state $s = (0, C)$:

- **Skip transition ($x_i = 0$):** $(i, k) \to (i + 1, k)$ with edge weight $w = 0$.
- **Take transition ($x_i = 1$, valid if $w_i \leq k$):** $(i, k) \to (i + 1, k - w_i)$ with edge weight $w = -v_i$.
- **Admissible heuristic** $h(i, k)$: To obtain a fast, tight lower bound on the minimum (most negative) remaining cost from state (i, k), we solve Dantzig's *linear programming relaxation* [Dan57] on the remaining items $\{i, \ldots, n-1\}$ with residual capacity k. We sort the remaining items in non-increasing order of efficiency ratio v_j/w_j, pack them greedily until the first item j^* that exceeds the remaining capacity k', and add the fractional profit $\lfloor (v_{j^*}/w_{j^*}) \cdot k' \rfloor$. Negating this upper bound on profit yields an admissible heuristic $h(i, k) = -\lfloor z_{\mathrm{LP}}(i, k) \rfloor \leq h^*(i, k)$.

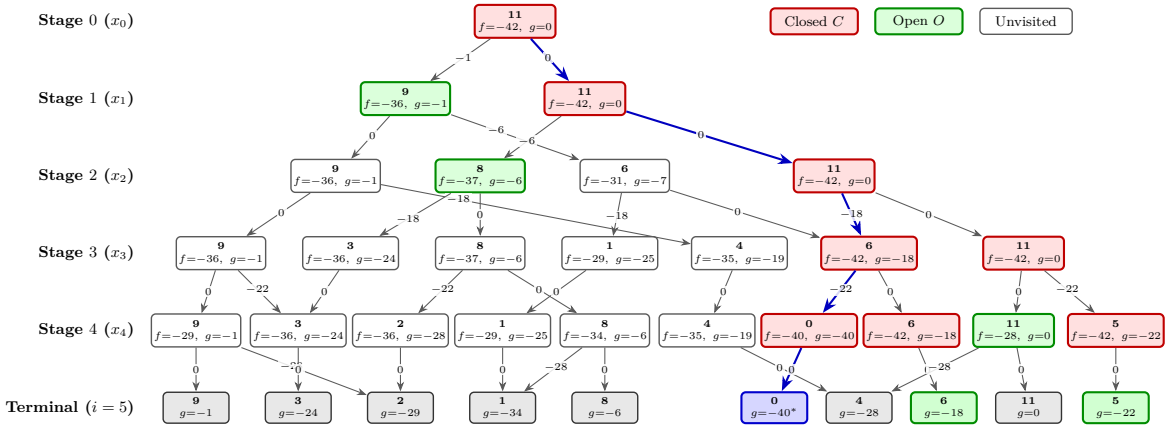


Figure 2.3: Resolution of the 5-item 0/1 Knapsack state DAG ($v = [1, 6, 18, 22, 28]$, $w = [2, 3, 5, 6, 7]$, $C = 11$) using A^* with the negated linear programming relaxation heuristic $h(i, k)$. Each node displays its residual capacity $\mathbf{k}$ on the top line and its scores $f = g + h$ and g on the bottom line. Red nodes belong to the Closed Set C (expanded states), green nodes belong to the Open Set O (frontier states in the priority queue), and white nodes are never visited by A^*. The blue trajectory highlights the optimal path reaching terminal state $(k = 0, g^* = -40)$.

Example 2.2 *Tracing A^* on the 5-Item Knapsack DAG*

Consider our running 5-item instance with $v = [1, 6, 18, 22, 28]$, $w = [2, 3, 5, 6, 7]$, and capacity $C = 11$. Figure 2.3 illustrates the complete state DAG and the exact state of the Open Set O and Closed Set C when A^* terminates:

1. At the root state $(i = 0, k = 11)$, the LP relaxation packs item 4 $(v_4 = 28, w_4 = 7)$ and a fraction 4/6 of item 3 $(v_3 = 22, w_3 = 6)$, giving $\lfloor 28 + 22 \cdot (4/6) \rfloor = \lfloor 42.67 \rfloor = 42$. Thus $g = 0$, $h = -42$, and $f = -42$.
2. Extracting $(i = 0, k = 11)$ generates $(i = 1, k = 9)$ with $g = -1, f = -36$ (taking item 0) and $(i = 1, k = 11)$ with $g = 0, f = -42$ (skipping item 0).
3. Because $f = -42 < -36$, A^* leaves $(i = 1, k = 9)$ unexpanded in O and immediately expands $(i = 1, k = 11)$, followed by states with $f = -42$ at stages 2, 3, and 4: $(i = 2, k = 11)$, $(i = 3, k = 11)$, $(i = 4, k = 5)$, $(i = 3, k = 6)$, and

$(i = 4, k = 6)$.

4. Next, the minimum f-value in O is $f = -40$ at state $(i = 4, k = 0, g = -40)$ (corresponding to taking items 2 and 3 with $w_2+w_3 = 5+6 = 11$ and $v_2+v_3 = 18+22 = 40$). Expanding it inserts the terminal state $(i = 5, k = 0, g = -40, f = -40)$, which is immediately extracted next to prove optimality!

Observe the dramatic pruning achieved by A^*: out of the 35 nodes in the state DAG, A^* expands **only** 8 **internal states** (red), keeps 6 states on the open frontier (green), and **completely bypasses** 20 **states** (white) whose subtrees are never even generated!

2.12.2 Negative Transition Weights and Robust A^*

When converting a maximization dynamic program into a minimization problem via negated rewards $w(u, v) = -v_i < 0$, we must be careful: standard Dijkstra's algorithm and standard A^* (Algorithm 4) assume non-negative edge weights (or a consistent heuristic satisfying $f(u) \leq f(v)$ along all transitions) [Ste+14].

Why can negative edge weights cause standard Dijkstra or A^* (without re-opening) to fail, even on a DAG? Consider the 4-vertex DAG in Figure 2.2c with $h(v) = 0$:

- Expanding s inserts u_0 with $g[u_0] = -1$ and u_1 with $g[u_1] = -2$ into O.
- Because $-2 < -1$, u_1 is extracted first and relaxes edge (u_1, t) of weight -1, inserting the goal t into O with $g[t] = -3$.
- At the next iteration, $O = \{t, u_0\}$ with $g[t] = -3$ and $g[u_0] = -1$. Standard Dijkstra/A^* extracts t first (since $-3 < -1$) and halts, returning the suboptimal cost -3 instead of expanding u_0 to discover the optimal path $s \to u_0 \to t$ of cost $-1 + (-3) = -4$!

In Figure 2.3, A^* succeeded without re-opening because the negated LP relaxation heuristic $h(i, k)$ is *consistent* even under negative transition costs: committing to a decision at stage i can only restrict the LP relaxation at stage $i + 1$, so $g(u) + h(u) \leq g(u) + w(u, v) + h(v)$ $(f(u) \leq f(v))$ holds along every edge.

When h is merely admissible $(h(u) \leq h^*(u))$ but not consistent (so $f(v) < f(u)$ can occur along negative-cost edges), we obtain a provably exact solver called *Robust* A^* (Algorithm 5) by introducing two modifications:

1. **Incumbent Tracking Instead of Immediate Termination**: Reaching the terminal state t updates the best known solution cost $g_{\text{inc}} \leftarrow g[t]$ rather than halting immediately. Search terminates only when $O = \emptyset$ or when the minimum f-value extracted from O satisfies $f[u] \geq g_{\text{inc}}$.
2. **Re-Opening Improved Closed States**: Whenever a transition (u, v) discovers a strictly shorter path $g[u] + w(u, v) < g[v]$ to a state $v \in C$, v is removed from C $(C \leftarrow C \setminus \{v\})$ and re-inserted into O. Because G is a finite DAG, no cycle exists and every state can be re-opened only a finite number of times.

2.13 Weighted and Anytime Weighted A^*

On large-scale **NP**-hard problems, even an informed A^* search with a tight admissible heuristic h may expand millions of states before reaching the bottom of the state DAG, because many distinct partial paths share very similar lower bounds $f(u) = g(u) + h(u)$. If a practitioner needs a high-quality feasible solution within a strict time limit, standard A^* suffers from poor

Algorithm 5: Robust A^* for DAGs with Negative Transition Weights

Input: DAG $G = (V, E)$, weights $w : E \to \mathbb{R}$, source s, goal t, admissible heuristic h
Output: Optimal cost g_{inc} and predecessor map π

foreach *vertex* $v \in V$ **do**
  $g[v] \leftarrow +\infty$; $f[v] \leftarrow +\infty$; $\pi[v] \leftarrow \text{NIL}$;
$g[s] \leftarrow 0$; $f[s] \leftarrow h(s)$; $g_{\text{inc}} \leftarrow +\infty$; $C \leftarrow \emptyset$;
$O \leftarrow \emptyset$; $\text{Insert}(O, s, f[s])$;
while $O \neq \emptyset$ **do**
  $u \leftarrow \text{ExtractMin}(O)$; $C \leftarrow C \cup \{u\}$;
  if $f[u] \geq g_{inc}$ **then**
    return g_{inc}, π ; `// Open frontier cannot improve incumbent`
  foreach *vertex* $v \in Adj[u]$ **do**
    if $g[u] + w(u, v) + h(v) \geq g_{inc}$ **then**
      continue ; `// Prune transition by incumbent bound`
    else if $g[u] + w(u, v) < g[v]$ **then**
      $\pi[v] \leftarrow u$; $g[v] \leftarrow g[u] + w(u, v)$; $f[v] \leftarrow g[v] + h(v)$;
      if $v = t$ **then**
        $g_{\text{inc}} \leftarrow g[t]$; `// Update incumbent solution`
      else
        $\text{InsertOrUpdateKey}(O, v, f[v])$; $C \leftarrow C \setminus \{v\}$;
return g_{inc}, π;

anytime behavior: it produces no feasible solution at all until a terminal state is finally popped from O.

2.13.1 Weighted A^* Search

To accelerate the search toward terminal states, Pohl [Poh70] introduced *Weighted* A^*. Given an inflation weight $w_h \geq 1$, Weighted A^* orders the open priority queue O using the weighted evaluation function:

$$f'(u) = g(u) + w_h \cdot h(u). \tag{2.37}$$

- When $w_h = 1$, $f'(u) = f(u)$ and the algorithm reduces to standard A^*.
- As $w_h > 1$ increases, the heuristic estimate of the remaining cost is amplified relative to the accumulated path cost $g(u)$, biasing the priority queue to expand states that are closer to the terminal stage (in the limit $w_h \to \infty$, it becomes pure greedy best-first search).

While inflating h by $w_h > 1$ sacrifices exact admissibility ($w_h \cdot h(u)$ can overestimate $h^*(u)$), it typically reduces the number of expanded nodes before reaching a first terminal solution by orders of magnitude, and guarantees that the first solution found has cost at most $w_h \cdot C^*$ whenever h is admissible and consistent.

2.13.2 Anytime Weighted A^*

How can we combine the rapid initial solution discovery of Weighted A^* with the eventual optimality guarantee of exact A^*? A first idea is to run a sequence of Weighted A^* searches

from scratch with decreasing weights $w_h^{(0)} > w_h^{(1)} > \cdots > 1$, but restarting discards the open frontier and expanded states.

A far more effective approach, due to Hansen and Zhou [HZ07], is *Anytime Weighted A^** (Algorithm 6), which transforms Weighted A^* into a single continuous anytime search via three synergistic mechanisms:

1. **Inflated Priority Ordering**: States in the open priority queue O are ordered by the non-admissible weighted score $f'(u) = g(u) + w_h \cdot h(u)$ with $w_h \geq 1$, driving the search to dive rapidly to a first complete solution and repeatedly discover improved incumbents.
2. **Continued Search with Incumbent Upper Bound**: Instead of stopping when a terminal state t is reached, the algorithm records the new best solution cost as a primal upper bound $g_{\text{inc}} \leftarrow g[t]$ (notifying the user immediately) and continues extracting states from O.
3. **Admissible Lower-Bound Pruning (Branch-and-Bound Integration)**: Even though O is ordered by $f'(u) = g(u) + w_h \cdot h(u)$, the unweighted score $f(u) = g(u) + h(u)$ remains a valid admissible lower bound on any solution extending u. Therefore, any state u (or transition $u \to v$) satisfying:

$$g[u] + h(u) \geq g_{\text{inc}} \qquad \text{or} \qquad g[u] + w(u,v) + h(v) \geq g_{\text{inc}} \tag{2.38}$$

cannot possibly lead to a solution strictly better than g_{inc} and is immediately pruned!

Algorithm 6: Anytime Weighted A^* [HZ07]

Input: Graph $G = (V, E)$, weights $w : E \to \mathbb{R}$, source s, goal t, admissible heuristic h, weight $w_h \geq 1$
Output: Best solution cost g_{inc} and predecessor map π

foreach *vertex* $v \in V$ **do**
  $g[v] \leftarrow +\infty$; $f'[v] \leftarrow +\infty$; $\pi[v] \leftarrow \text{NIL}$;
$g[s] \leftarrow 0$; $f'[s] \leftarrow w_h \cdot h(s)$; $g_{\text{inc}} \leftarrow +\infty$; $C \leftarrow \emptyset$;
$O \leftarrow \emptyset$; Insert$(O, s, f'[s])$; `// Ordered by weighted score` $f'[v]$
while $O \neq \emptyset$ ***and*** *not interrupted* **do**
  $u \leftarrow \text{ExtractMin}(O)$; $C \leftarrow C \cup \{u\}$;
  if $g[u] + h(u) \geq g_{inc}$ **then**
    continue ; `// Prune state using admissible lower bound`
  foreach *vertex* $v \in Adj[u]$ **do**
    if $g[u] + w(u,v) + h(v) \geq g_{inc}$ **then**
      continue ; `// Prune transition using admissible lower bound`
    else if $g[u] + w(u,v) < g[v]$ **then**
      $\pi[v] \leftarrow u$; $g[v] \leftarrow g[u] + w(u,v)$;
      $f'[v] \leftarrow g[v] + w_h \cdot h(v)$;
      if $v = t$ **then**
        $g_{\text{inc}} \leftarrow g[t]$; `// New incumbent solution found`
      else
        InsertOrUpdateKey$(O, v, f'[v])$; $C \leftarrow C \setminus \{v\}$;
return g_{inc}, π ; `// Optimal if` $O = \emptyset$`, heuristic if interrupted`

Dual Role of f and f' in Anytime Weighted A^*. Notice how Anytime Weighted A^* bridges

Dynamic Programming and Branch-and-Bound (Chapter 3): state dominance is enforced via the hash table $g[v]$ (if an alternative path reaches the same DP state v with $g[u] + w(u, v) \geq g[v]$, it is discarded by Bellman's Principle of Optimality), while g_{inc} acts as the global Branch-and-Bound primal upper bound and $f(v) = g(v) + h(v)$ acts as the admissible dual lower bound. When the priority queue O becomes empty ($O = \emptyset$), every unpruned path has been explored and g_{inc} is **proven optimal**!

Table 2.2: Comparison of graph-search paradigms on a Dynamic Programming state DAG.

Algorithm	**Queue Order**	**Stop Condition**	**Anytime?**	**Guarantee**
Tabulation / Memo DP	Topological / DFS	Full DAG explored	No	Exact (C^*)
Dijkstra (Algorithm 3)	$g(u)$	First goal t	No	Exact (C^*, $w \geq 0$)
Standard A^* (Algorithm 4)	$g(u) + h(u)$	First goal t	No	Exact (C^*, h consistent)
Robust A^* (Algorithm 5)	$g(u) + h(u)$	$f[u] \geq g_{\text{inc}}$ or $O = \emptyset$	Yes	Exact (C^*, h admissible)
Weighted A^*	$g(u) + w_h h(u)$	First goal t	No	w_h-Approximation
Anytime Weighted A^*	$g(u) + w_h h(u)$	$O = \emptyset$ (or timeout)	Yes	Exact (C^* when $O = \emptyset$)

2.14 Java Implementation of A^* for DP

We now show how the generic Dynamic Programming framework of Section 2.9 extends naturally to support A^*, Weighted A^*, and Anytime Weighted A^*.

2.14.1 Extending the Model and Solver Interfaces

Compared to the basic DP `Model<S>` in Listing 2.7, an A^* model (Listing 2.11) simply adds the heuristic evaluation method `h(S state)` (and renames the base-case methods to `isTerminalState` and `getTerminalStateValue`). A lightweight `Node<S>` wrapper pairs each state with its priority score f for insertion into Java's `PriorityQueue`, and the `Solver` interface accepts a callback `BiConsumer<Solution, SearchStatistics>` that is invoked whenever a new incumbent solution is discovered.

2.14.2 The A^* and Anytime Weighted A^* Solvers

In Listings 2.13 and 2.14, we present the generic `Astar<S>` and `AnytimeWeightedAstar<S>` solver implementations. Both classes operate solely on the abstract `Model<S>` interface without any problem-specific logic.

2.14.3 Case Study: The Traveling Salesperson Problem

To demonstrate the versatility of this architecture on a strongly **NP**-hard routing problem, let us formulate the *Traveling Salesperson Problem* (Held–Karp dynamic program) for n cities $\{0, 1, \ldots, n-1\}$ starting and ending at depot city 0:

- **State (`TSPState`)**: Encapsulates the salesperson's current city `position` $\in \{0, \ldots, n-1\}$ and the subset of unvisited cities `toVisit` represented as a `BitSet`. At the root state, `position` $= 0$ and `toVisit` $= \{1, \ldots, n-1\}$.
- **Terminal State and Value**: A state is terminal when `toVisit.isEmpty()`, at which point the salesperson must return from `state.position` to the depot city 0, incurring terminal cost `distanceMatrix`[`state.position`][0].
- **Admissible Heuristic `h(TSPState state)`**: In any valid completion of the tour from `state.position` visiting all cities in $U =$ `toVisit` and returning to depot 0, every

```java
import java.util.List;
import java.util.function.BiConsumer;
import util.Solution;

public abstract class Model<S extends State> {
    public abstract S getRootState();
    public abstract boolean isTerminalState(S state);
    public abstract double getTerminalStateValue(S state);
    public abstract List<Transition<S>> getTransitions(
            S state);
    public abstract double h(S state);
}

class Node<S> implements Comparable<Node<S>> {
    final S state;
    final double f;

    public Node(S state, double f) {
        this.state = state;
        this.f = f;
    }

    @Override
    public int compareTo(Node<S> o) {
        return Double.compare(this.f, o.f);
    }
}

public interface Solver {
    void solve(BiConsumer<Solution, SearchStatistics>
                onSolution);
}
```

Listing 2.11: The `Model<S>`, `Node<S>`, and `Solver` abstractions for A^*-based dynamic programming.

```java
import java.util.*;

public class Knapsack extends Model<KnapsackState> {
    KnapsackInstance instance;
    KnapsackState root;

    public Knapsack(KnapsackInstance instance) {
        this.instance = instance;
        this.root = new KnapsackState(0, instance.capacity);
    }

    @Override
    public boolean isTerminalState(KnapsackState state) {
        return state.item == instance.n
            || state.capacity == 0;
    }

    @Override
    public double getTerminalStateValue(KnapsackState s) {
        return 0.0;
    }

    @Override
    public KnapsackState getRootState() { return root; }

    @Override
    public double h(KnapsackState state) {
        int rem = instance.n - state.item;
        Integer[] idx = new Integer[rem];
        double[] ratio = new double[instance.n];
        for (int i = state.item; i < instance.n; i++) {
            ratio[i] = (double) instance.value[i]
                     / instance.weight[i];
            idx[i - state.item] = i;
        }
        Arrays.sort(idx, (a, b) ->
            Double.compare(ratio[b], ratio[a]));
        int cap = state.capacity, maxProfit = 0;
        for (int item : idx) {
            if (cap <= 0) break;
            if (cap >= instance.weight[item]) {
                maxProfit += instance.value[item];
                cap -= instance.weight[item];
            } else {
                maxProfit += (int) Math.floor(
                    ratio[item] * cap);
                cap = 0;
            }
        }
        return -maxProfit;
    }
    // getTransitions(state) identical to Listing 2.9
    // with negated transition value -instance.value[item]
}
```

Listing 2.12: Knapsack model with negated LP relaxation heuristic (`Knapsack.java`).

```java
public class Astar<S extends State> implements Solver {
    Model<S> model;
    Map<S, S> pred = new HashMap<>();
    Map<S, Integer> prevDec = new HashMap<>();
    Map<S, Double> g = new HashMap<>();
    Set<S> closed = new HashSet<>();

    public Astar(Model<S> model) { this.model = model; }

    @Override
    public void solve(
            BiConsumer<Solution, SearchStatistics> onSol) {
        long t0 = System.currentTimeMillis();
        int expanded = 0;
        PriorityQueue<Node<S>> pq = new PriorityQueue<>();
        S root = model.getRootState();
        pred.put(root, null); prevDec.put(root, -1);
        g.put(root, 0.0);
        pq.add(new Node<>(root, model.h(root)));
        while (!pq.isEmpty()) {
            S cur = pq.poll().state;
            expanded++;
            closed.add(cur);
            if (model.isTerminalState(cur)) {
                long dt = System.currentTimeMillis() - t0;
                onSol.accept(rebuild(cur, g.get(cur)),
                    new SearchStatistics(dt, expanded));
                return;
            }
            for (Transition<S> t :
                    model.getTransitions(cur)) {
                S succ = t.getSuccessor();
                if (!closed.contains(succ)) {
                    double newG = g.get(cur) + t.getValue();
                    if (model.isTerminalState(succ))
                        newG += model.getTerminalStateValue(
                            succ);
                    if (newG < g.getOrDefault(
                            succ, Double.MAX_VALUE)) {
                        g.put(succ, newG);
                        pred.put(succ, cur);
                        prevDec.put(succ, t.getDecision());
                        pq.add(new Node<>(
                            succ, newG + model.h(succ)));
                    }
                }
            }
        }
    }
}
```

Listing 2.13: Generic A^* solver for Dynamic Programming models (`Astar.java`).

```java
public class AnytimeWeightedAstar<S extends State>
        implements Solver {
    final Model<S> model;
    final double hWeight;
    final int timeLimit; // time limit in seconds
    Map<S, S> pred = new HashMap<>();
    Map<S, Integer> prevDec = new HashMap<>();
    Map<S, Double> g = new HashMap<>();

    public AnytimeWeightedAstar(Model<S> model,
            double hWeight, int timeLimit) {
        this.model = model;
        this.hWeight = hWeight;
        this.timeLimit = timeLimit;
    }

    @Override
    public void solve(
            BiConsumer<Solution, SearchStatistics> onSol) {
        long t0 = System.currentTimeMillis();
        int expanded = 0;
        PriorityQueue<Node<S>> pq = new PriorityQueue<>();
        S root = model.getRootState();
        pred.put(root, null); prevDec.put(root, -1);
        g.put(root, 0.0);
        pq.add(new Node<>(root, hWeight * model.h(root)));
        double gInc = Double.MAX_VALUE;
        while (!pq.isEmpty() && System.currentTimeMillis()
                - t0 < timeLimit * 1000L) {
            S cur = pq.poll().state;
            if (g.get(cur) + model.h(cur) >= gInc) continue;
            expanded++;
            for (Transition<S> t :
                    model.getTransitions(cur)) {
                S succ = t.getSuccessor();
                double newG = g.get(cur) + t.getValue();
                if (model.isTerminalState(succ))
                    newG += model.getTerminalStateValue(succ);
                if (newG + model.h(succ) >= gInc) continue;
                if (newG < g.getOrDefault(
                        succ, Double.MAX_VALUE)) {
                    g.put(succ, newG); pred.put(succ, cur);
                    prevDec.put(succ, t.getDecision());
                    if (model.isTerminalState(succ)) {
                        gInc = newG;
                        long dt = System.currentTimeMillis()
                                - t0;
                        onSol.accept(rebuild(succ, newG),
                            new SearchStatistics(
                                dt, expanded));
                    } else pq.add(new Node<>(succ,
                        newG + hWeight * model.h(succ)));
                }
            }
        }
    }
}
```

Listing 2.14: Generic Anytime Weighted A^* solver (`AnytimeWeightedAstar.java`).

unvisited city $i \in U$ must be entered at least once from either `state.position` or another unvisited city $j \in U \setminus \{i\}$, and the depot 0 must be entered from some city $i \in U$. Summing the minimum possible incoming edge cost for each $i \in U$ plus the minimum return edge to 0:

$$h(\text{pos}, U) = \sum_{i \in U} \min_{j \in (\{\text{pos}\} \cup U) \setminus \{i\}} c_{j,i} \; + \; \min_{i \in U} c_{i,0} \tag{2.39}$$

yields a fast $\mathcal{O}(|U|^2)$ admissible lower bound (Listings 2.15 and 2.16).

```java
import java.util.*;

public class TSPState extends State {
    int position;
    BitSet toVisit;

    public TSPState(int position, BitSet toVisit) {
        this.position = position;
        this.toVisit = toVisit;
    }

    @Override
    public int hash() {
        return Objects.hash(position, toVisit);
    }

    @Override
    public boolean isEqual(State s) {
        if (s instanceof TSPState) {
            TSPState o = (TSPState) s;
            return position == o.position
                && toVisit.equals(o.toVisit);
        }
        return false;
    }
}

// Helper in Astar<S> and AnytimeWeightedAstar<S>:
private Solution rebuild(S state, double value) {
    LinkedList<Integer> dec = new LinkedList<>();
    while (prevDec.get(state) != -1) {
        dec.addFirst(prevDec.get(state));
        state = pred.get(state);
    }
    return new Solution(value, dec);
}
```

Listing 2.15: TSP state and solution reconstruction (`TSPState.java`, `Astar.java`).

```java
public class TSP extends Model<TSPState> {
    TSPInstance instance;
    TSPState root;

    public TSP(TSPInstance instance) {
        this.instance = instance;
        BitSet toVisit = new BitSet(instance.n);
        toVisit.flip(1, instance.n);
        this.root = new TSPState(0, toVisit);
    }

    @Override
    public boolean isTerminalState(TSPState s) {
        return s.toVisit.isEmpty();
    }
    @Override
    public double getTerminalStateValue(TSPState s) {
        return instance.distanceMatrix[s.position][0];
    }
    @Override
    public TSPState getRootState() { return root; }

    @Override
    public List<Transition<TSPState>> getTransitions(
            TSPState s) {
        List<Transition<TSPState>> trans =
            new LinkedList<>();
        for (int i = s.toVisit.nextSetBit(0); i >= 0;
                i = s.toVisit.nextSetBit(i + 1)) {
            BitSet rem = (BitSet) s.toVisit.clone();
            rem.flip(i);
            trans.add(new Transition<>(new TSPState(i, rem),
                i, instance.distanceMatrix[s.position][i]));
        }
        return trans;
    }

    @Override
    public double h(TSPState s) {
        if (isTerminalState(s)) return 0.0;
        double lb = 0.0, minToDepot = Double.MAX_VALUE;
        for (int i = s.toVisit.nextSetBit(0); i >= 0;
                i = s.toVisit.nextSetBit(i + 1)) {
            double minIn =
                instance.distanceMatrix[s.position][i];
            for (int j = s.toVisit.nextSetBit(0); j >= 0;
                    j = s.toVisit.nextSetBit(j + 1)) {
                if (j != i) minIn = Math.min(minIn,
                    instance.distanceMatrix[j][i]);
            }
            lb += minIn;
            minToDepot = Math.min(minToDepot,
                instance.distanceMatrix[i][0]);
        }
        return lb + minToDepot;
    }
}
```

Listing 2.16: TSP A^* model and cheapest-incident heuristic (`TSP.java`).

2.15 Exercises

Exercise 2.1 *Prefix Sum Boundary Conditions*

In Listing 2.4, method `sumInterval(start, end)` reads `prefixSum[start - 1]`. Explain what happens when `start` $= 0$. How should the method or prefix array be structured to handle `start` $= 0$ cleanly without throwing an `ArrayIndexOutOfBoundsException`?

Exercise 2.2 *Memory Optimization for Knapsack DP*

Notice that computing column i in `KnapsackTable.java` requires only values from column $i-1$. Show how to reduce the space complexity of the tabulation approach from $\mathcal{O}(C \cdot n)$ to $\mathcal{O}(C)$ using only two 1D arrays (or a single 1D array traversed backwards).

Exercise 2.3 *Partitioning with Non-Contiguous Ranges*

Show how to modify the 0/1 Knapsack recurrence equation to solve the Subset-Sum problem with boolean values `true`/`false` indicating whether a target sum is achievable.

Exercise 2.4 *Consistency Implies Admissibility*

Let $G = (V, E)$ be a state transition graph with goal state t, and let $h : V \to \mathbb{R}$ be a consistent heuristic (Definition 2.7) satisfying $h(t) = 0$ and $h(u) \leq w(u, v) + h(v)$ for every directed edge $(u, v) \in E$.

1. Using induction on the number of edges along an optimal path $u = v_0 \to v_1 \to \cdots \to v_k = t$, prove that h is necessarily admissible ($h(u) \leq h^*(u)$ for all $u \in V$).
2. Prove that in the 0/1 Knapsack state DAG with negated profits $w((i,k),(i+1,k')) \in \{0, -v_i\}$, the negated continuous LP relaxation heuristic $h_{\text{LP}}(i,k) = -z_{\text{LP}}(i,k)$ is consistent even though the transition weights are negative.

Exercise 2.5 *Suboptimality Bound of Weighted A^**

Suppose Weighted A^* orders the open priority queue O using $f'(u) = g(u) + w_h \cdot h(u)$ with weight $w_h \geq 1$, where h is an admissible heuristic ($h(u) \leq h^*(u)$). Adapt the proof by contradiction of Theorem 2.5 to show that if every node n along an optimal $s \rightsquigarrow t$ path satisfies $g(n) = g^*(n)$ when $n \in O$, then the first path to t extracted by Weighted A^* has cost at most $w_h \cdot C^*$.

Exercise 2.6 *Strengthening the TSP Heuristic with 1-Trees*

In Listing 2.16, the heuristic `h(TSPState state)` sums the cheapest incoming edge for each unvisited city $i \in U =$ `state.toVisit` plus the cheapest edge from U back to depot 0.

1. Show that the subgraph of minimum incoming edges selected by (2.39) may contain disconnected cycles among the unvisited cities U.
2. Explain how replacing the sum of cheapest incoming edges inside U by the weight

of a *Minimum Spanning Tree* (MST) over U (connected to `state.position` and to depot 0 by their respective shortest edges to U) yields a strictly dominating, consistent heuristic $h_{\text{MST}}(\texttt{pos}, U) \geq h(\texttt{pos}, U)$ in $\mathcal{O}(|U|^2)$ time.

CHAPTER 3

Branch and Bound

In combinatorial optimization, many fundamental problems—including the 0/1 Knapsack problem, the Traveling Salesperson Problem (TSP), and general Mixed-Integer Linear Programs (MILP)—are **NP**-hard. Exhaustive enumeration of all feasible candidate solutions is computationally prohibitive: a knapsack instance with $n = 60$ items entails $2^{60} \approx 1.15 \times 10^{18}$ configurations, while a symmetric TSP on $n = 30$ cities requires evaluating $\frac{(n-1)!}{2} \approx 4.42 \times 10^{30}$ distinct tours. Even at the rate of one billion evaluations per second, brute-force enumeration would span millions of years.

To solve such problems to provable optimality without inspecting every candidate, we turn to *Branch and Bound* (B&B), an implicit enumeration paradigm pioneered in the foundational work of Ailsa Land and Alison Doig Harcourt [LD60], George Dantzig [Dan57], and John Little et al. [Lit+63]. Rather than examining every individual leaf of the combinatorial search tree, Branch and Bound constructs a hierarchical decomposition of the solution space and systematically prunes entire subtrees whose solutions are mathematically guaranteed to be suboptimal or infeasible.

At the core of Branch and Bound lies an indispensable principle that guides all modern integer programming:

"Combinatorial optimization is the art of relaxing."

When an optimization problem is too constrained or combinatorial to solve directly in polynomial time, we relax one or more difficult constraints. The resulting relaxed subproblem can be solved efficiently, yielding an optimistic bound that proves whether exploring a region of the search space is worthwhile.

3.1 Foundations of Branch and Bound

Consider an abstract discrete optimization problem P_0 formulated as:

$$z^* = \max_{\mathbf{x} \in X} f(\mathbf{x}), \tag{3.1}$$

where $X \subseteq \mathbb{Z}^n \times \mathbb{R}^m$ represents the discrete or mixed-integer feasible region, and $f : X \to \mathbb{R}$ denotes the objective function to be maximized. Branch and Bound solves P_0 by coordinating three algorithmic mechanisms: *branching*, *bounding*, and *fathoming* (pruning).

3.1.1 The Three Core Mechanisms

Definition 3.1 *Branching (Space Partitioning)*

Let $X_s \subseteq X$ represent the feasible domain of a subproblem P_s. *Branching* partitions X_s into $k \geq 2$ non-empty, mutually exclusive (or covering) subsets $X_{s,1}, X_{s,2}, \ldots, X_{s,k}$ such that:

$$X_s = \bigcup_{i=1}^{k} X_{s,i}. \tag{3.2}$$

Consequently, the optimal objective over X_s satisfies:

$$\max_{\mathbf{x} \in X_s} f(\mathbf{x}) = \max_{i \in \{1,\ldots,k\}} \left(\max_{\mathbf{x} \in X_{s,i}} f(\mathbf{x}) \right). \tag{3.3}$$

In binary optimization, branching typically splits a subproblem by selecting an undecided binary variable $x_j \in \{0, 1\}$ and generating two children: one setting $x_j = 1$ and the other setting $x_j = 0$.

Definition 3.2 *Bounding Procedure (Relaxation)*

For any subproblem P_s with feasible domain X_s, a *bounding procedure* computes an optimistic estimate $UB(X_s)$ on the optimal value achievable within X_s:

$$UB(X_s) \geq \max_{\mathbf{x} \in X_s} f(\mathbf{x}). \tag{3.4}$$

In a maximization context, $UB(X_s)$ is an *upper bound*. Symmetrically, for minimization problems, the procedure computes a *lower bound* $LB(X_s) \leq \min_{\mathbf{x} \in X_s} f(\mathbf{x})$.

An optimistic bound is obtained by solving a *relaxation* $\mathcal{R}(P_s)$ of P_s, formed by expanding the feasible space ($X_s \subseteq X_s^{\text{rel}}$) or by replacing the objective function with an over-estimator. For the bound to be useful in practice, $\mathcal{R}(P_s)$ must satisfy two conflicting criteria:

1. **Tractability**: The relaxation must be solvable in polynomial (or pseudo-polynomial) time, typically via linear programming, greedy heuristics, or polynomial-time graph algorithms.
2. **Tightness**: The bound $UB(X_s)$ must be as close as possible to $\max_{\mathbf{x} \in X_s} f(\mathbf{x})$. A loose bound fails to prune unpromising branches, degenerating the search into exhaustive enumeration.

Throughout the search, the algorithm maintains an *incumbent solution* $\mathbf{x}^* \in X$, representing the best feasible solution discovered so far, with objective value:

$$LB^* = f(\mathbf{x}^*). \tag{3.5}$$

Because $\mathbf{x}^*$ is feasible for the original problem P_0, LB^* serves as a global lower bound on the true optimum ($LB^* \leq z^*$). If no feasible solution is known initially, we set $LB^* = -\infty$.

Theorem 3.1 *The Three Fathoming (Pruning) Rules*

During the exploration of the search tree, a subproblem P_s with domain X_s can be immediately *fathomed* (pruned without further branching) if and only if at least one of

the following three conditions holds:

1. **Pruning by Bound (Domination)**:

$$UB(X_s) \leq LB^*. \tag{3.6}$$

   Since no candidate solution in X_s can yield an objective value strictly exceeding $UB(X_s)$, X_s cannot contain a solution strictly better than the current incumbent $\mathbf{x}^*$.

2. **Pruning by Infeasibility**:

$$X_s^{\text{rel}} = \emptyset \implies X_s = \emptyset. \tag{3.7}$$

   If the relaxed problem admits no feasible solution, the restricted original subproblem is guaranteed to be empty.

3. **Pruning by Optimality (Integrality)**: Let $\mathbf{x}_s^*$ be an optimal solution to the relaxation $\mathcal{R}(P_s)$. If $\mathbf{x}_s^*$ satisfies all constraints of the original problem ($\mathbf{x}_s^* \in X_s$), then $\mathbf{x}_s^*$ is optimal for P_s, and $f(\mathbf{x}_s^*) = UB(X_s)$. If $f(\mathbf{x}_s^*) > LB^*$, we update the incumbent:

$$\mathbf{x}^* \leftarrow \mathbf{x}_s^*, \quad LB^* \leftarrow f(\mathbf{x}_s^*). \tag{3.8}$$

   Since the optimum of P_s is fully determined, no further branching on X_s is required.

3.1.2 Maximization versus Minimization Duality

Optimization problems can be formulated interchangeably as maximization or minimization through the fundamental sign-reversal identity:

$$\min_{\mathbf{x}\in X} c(\mathbf{x}) \iff \max_{\mathbf{x}\in X} \big(-c(\mathbf{x})\big). \tag{3.9}$$

While theoretical expositions often adopt a maximization perspective, general-purpose optimization solvers and algorithmic frameworks (including the generic Java solver developed in this chapter) conventionally standardize on *minimization*. The duality between the two paradigms is summarized in Table 3.1. In minimization, relaxations provide *lower bounds* $LB(X_s) \leq c(\mathbf{x})$ for all $\mathbf{x} \in X_s$, while the incumbent provides an *upper bound* $UB^* = c(\mathbf{x}^*)$. A node is pruned by domination whenever $LB(X_s) \geq UB^*$.

Table 3.1: Duality between Maximization and Minimization in Branch and Bound.

Role / Property	**Maximization** ($\max f(\mathbf{x})$)	**Minimization** ($\min c(\mathbf{x})$)
Relaxation bound	$UB(X_s) \geq \max_{\mathbf{x}\in X_s} f(\mathbf{x})$	$LB(X_s) \leq \min_{\mathbf{x}\in X_s} c(\mathbf{x})$
Incumbent value	$LB^* = f(\mathbf{x}^*)$	$UB^* = c(\mathbf{x}^*)$
Initial incumbent	$LB^* = -\infty$	$UB^* = +\infty$
Incumbent update	$LB^* \leftarrow f(\mathbf{x})$ (if $f(\mathbf{x}) > LB^*$)	$UB^* \leftarrow c(\mathbf{x})$ (if $c(\mathbf{x}) < UB^*$)
Fathoming by bound	$UB(X_s) \leq LB^*$	$LB(X_s) \geq UB^*$
Fathoming by optimality	$LB^* \leftarrow f(\mathbf{x}_s^*)$ (if $\mathbf{x}_s^* \in X$)	$UB^* \leftarrow c(\mathbf{x}_s^*)$ (if $\mathbf{x}_s^* \in X$)

3.2 Branch and Bound for 0/1 Knapsack

We now illustrate the mechanics of Branch and Bound on the 0/1 Knapsack problem. Recall from Chapter 2 that given n items with values $v_0, \dots, v_{n-1} \in \mathbb{Z}^+$, weights $w_0, \dots, w_{n-1} \in \mathbb{Z}^+$, and a total knapsack capacity $C \in \mathbb{Z}^+$, the problem is formulated as:

$$\begin{aligned} \text{maximize} \quad & \sum_{i=0}^{n-1} v_i x_i \\ \text{subject to} \quad & \sum_{i=0}^{n-1} w_i x_i \leq C, \\ & x_i \in \{0,1\}, \quad \forall i \in \{0, \dots, n-1\}. \end{aligned} \tag{3.10}$$

3.2.1 Binary Search Tree Representation

We construct a binary search tree by deciding the inclusion of items sequentially. At depth $k \in \{0, \dots, n-1\}$, the search branches on variable x_k:

- **Left Branch** ($x_k = 1$): The item is included in the knapsack. This decision is feasible only if the remaining knapsack capacity satisfies $C_{\text{residual}} \geq w_k$. If included, accumulated value increases by v_k and residual capacity becomes $C_{\text{residual}} - w_k$.
- **Right Branch** ($x_k = 0$): The item is excluded. The accumulated value and residual capacity remain unchanged.

A node in this tree is defined by the decision prefix $(x_0, \dots, x_{k-1})$, the accumulated value $V_{\text{fixed}} = \sum_{i=0}^{k-1} v_i x_i$, and the residual capacity $C_{\text{residual}} = C - \sum_{i=0}^{k-1} w_i x_i$. Unconstrained brute-force search would expand $2^{n+1} - 1$ nodes. To prune this tree effectively, we examine two candidate bounding procedures.

3.2.2 Bounding Procedure 1: Capacity Relaxation

The simplest relaxation consists in dropping the knapsack capacity constraint entirely for all undecided items $\{k, \dots, n-1\}$:

$$\begin{aligned} \text{maximize} \quad & V_{\text{fixed}} + \sum_{i=k}^{n-1} v_i x_i \\ \text{subject to} \quad & x_i \in \{0,1\}, \quad \forall i \in \{k, \dots, n-1\}. \end{aligned} \tag{3.11}$$

Since capacity is unconstrained, the optimal assignment trivially sets $x_i = 1$ for all remaining items:

$$UB_{\text{capa}} = V_{\text{fixed}} + \sum_{i=k}^{n-1} v_i. \tag{3.12}$$

While UB_{capa} can be computed in $\mathcal{O}(1)$ time after an $\mathcal{O}(n)$ prefix-sum precomputation, it is extremely loose because it completely ignores weights. It only prunes branches when the sum of *all* remaining item values cannot beat the incumbent.

3.2.3 Bounding 2: Continuous LP Relaxation

A vastly superior bound is obtained by preserving the capacity constraint while relaxing the integrality restriction $x_i \in \{0,1\}$ to the continuous interval $x_i \in [0,1]$:

$$\begin{aligned} \text{maximize} \quad & V_{\text{fixed}} + \sum_{i=k}^{n-1} v_i x_i \\ \text{subject to} \quad & \sum_{i=k}^{n-1} w_i x_i \le C_{\text{residual}}, \\ & 0 \le x_i \le 1, \quad \forall i \in \{k, \dots, n-1\}. \end{aligned} \tag{3.13}$$

This continuous knapsack problem was solved by George Dantzig [Dan57] via a greedy efficiency rule.

Definition 3.3 *Value-to-Weight Ratio (Item Efficiency)*

For each item i, its *efficiency* r_i is defined as the ratio of its value to its weight:

$$r_i = \frac{v_i}{w_i}. \tag{3.14}$$

Without loss of generality, we assume the items are sorted in non-increasing order of efficiency:

$$\frac{v_0}{w_0} \ge \frac{v_1}{w_1} \ge \cdots \ge \frac{v_{n-1}}{w_{n-1}}. \tag{3.15}$$

Definition 3.4 *Critical (Split) Item*

Given items sorted by non-increasing efficiency and an available capacity C_{residual}, let j denote the index of the *critical item* (or split item), defined as the first item that cannot be added in its entirety without violating capacity:

$$j = \min \left\{ i \in \{k, \dots, n-1\} \;\middle|\; \sum_{\ell=k}^{i} w_\ell > C_{\text{residual}} \right\}. \tag{3.16}$$

If $\sum_{\ell=k}^{n-1} w_\ell \le C_{\text{residual}}$, all remaining items fit, and no critical item exists (all $x_i = 1$).

Theorem 3.2 *Optimality of the Greedy Linear Relaxation*

Suppose the undecided items $\{k, \dots, n-1\}$ are sorted by non-increasing efficiency $r_i = v_i/w_i$. Let j be the critical item defined in Definition 3.4. The optimal solution $\mathbf{x}^*$ to the continuous knapsack problem (3.13) is given by:

$$x_i^* = \begin{cases} 1 & \text{for } k \le i < j, \\ \dfrac{C_{\text{residual}} - \sum_{\ell=k}^{j-1} w_\ell}{w_j} & \text{for } i = j, \\ 0 & \text{for } j < i \le n-1. \end{cases} \tag{3.17}$$

The corresponding continuous upper bound is:

$$UB_{\text{LP}} = V_{\text{fixed}} + \sum_{i=k}^{j-1} v_i + x_j^* \cdot v_j. \tag{3.18}$$

Proof. Suppose for contradiction that another feasible solution $\mathbf{x}'$ exists with strictly higher objective value: $\sum_{i=k}^{n-1} v_i x_i' > \sum_{i=k}^{n-1} v_i x_i^*$. Since $\mathbf{x}' \neq \mathbf{x}^*$, there must exist at least one index $a < j$ where $x_a' < x_a^* = 1$ and at least one index $b > j$ where $x_b' > x_b^* = 0$ (or a fractional imbalance at j).

Consider the difference vector $\boldsymbol{\delta} = \mathbf{x}^* - \mathbf{x}'$. We can decompose the difference into indices where $\mathbf{x}^*$ takes more than $\mathbf{x}'$ ($i \leq j$, where $\delta_i > 0$) and indices where $\mathbf{x}^*$ takes less than $\mathbf{x}'$ ($i \geq j$, where $\delta_i < 0$). Because both solutions are feasible and $\mathbf{x}^*$ exhausts the capacity:

$$\sum_{i \leq j} w_i \delta_i \geq \sum_{i \geq j} w_i(-\delta_i) > 0. \tag{3.19}$$

Now consider the net difference in objective value:

$$\sum_{i=k}^{n-1} v_i x_i^* - \sum_{i=k}^{n-1} v_i x_i' = \sum_{i=k}^{n-1} v_i \delta_i = \sum_{i \leq j} \frac{v_i}{w_i} w_i \delta_i - \sum_{i \geq j} \frac{v_i}{w_i} w_i(-\delta_i). \tag{3.20}$$

Because the items are sorted in non-increasing ratio order, $r_i \geq r_j$ for all $i \leq j$, and $r_i \leq r_j$ for all $i \geq j$. Therefore:

$$\sum_{i \leq j} r_i(w_i \delta_i) \geq r_j \sum_{i \leq j} w_i \delta_i \geq r_j \sum_{i \geq j} w_i(-\delta_i) \geq \sum_{i \geq j} r_i(w_i(-\delta_i)). \tag{3.21}$$

Hence, $\sum_{i=k}^{n-1} v_i x_i^* - \sum_{i=k}^{n-1} v_i x_i' \geq 0$, which contradicts the assumption that $\mathbf{x}'$ achieves a strictly higher profit. Thus, $\mathbf{x}^*$ is optimal. □

Example 3.1 *Continuous Relaxation Computation*

Consider a knapsack instance with capacity $C = 14$ and 5 items already sorted by non-increasing efficiency:

Item Index i	0	1	2	3	4
Value v_i	28	22	18	6	1
Weight w_i	7	6	5	2	1
Ratio $r_i = v_i/w_i$	4.00	3.67	3.60	3.00	1.00
Cumulative Weight $\sum_{\ell=0}^{i} w_\ell$	7	13	18	20	21

At the root node ($k = 0$, $C_{\text{residual}} = 14$):

- Item 0 is taken completely ($x_0^* = 1$), consuming $w_0 = 7$; residual capacity becomes $14 - 7 = 7$.
- Item 1 is taken completely ($x_1^* = 1$), consuming $w_1 = 6$; residual capacity becomes $7 - 6 = 1$.
- Item 2 has weight $w_2 = 5 > 1$, making $j = 2$ the critical item. We take fraction $x_2^* = \frac{1}{5}$.
- Items 3 and 4 are excluded: $x_3^* = x_4^* = 0$.

The continuous upper bound is:

$$UB_{\text{LP}} = v_0 + v_1 + \frac{1}{5}v_2 = 28 + 22 + 0.2 \times 18 = 50 + 3.6 = 53.6. \tag{3.22}$$

Since all item values are integers, any valid integer solution must satisfy $\sum v_i x_i \leq \lfloor 53.6 \rfloor = 53$.

3.2.4 Worked Example and Search Tree

To see how Branch and Bound operates in practice, consider the following 0/1 knapsack instance:

$$\begin{aligned} \text{maximize} \quad & 28x_0 + 30x_1 + 20x_2 \\ \text{subject to} \quad & 4x_0 + 6x_1 + 4x_2 \leq 9, \\ & x_0, x_1, x_2 \in \{0, 1\}. \end{aligned} \tag{3.23}$$

The instance parameters and item efficiencies are:

- Item 0: $v_0 = 28$, $w_0 = 4 \implies r_0 = \frac{28}{4} = 7.0$.
- Item 1: $v_1 = 30$, $w_1 = 6 \implies r_1 = \frac{30}{6} = 5.0$.
- Item 2: $v_2 = 20$, $w_2 = 4 \implies r_2 = \frac{20}{4} = 5.0$.

The items are already sorted in non-increasing efficiency order ($7.0 \geq 5.0 \geq 5.0$). We explore the tree using Depth-First Search (DFS), branching on x_0, then x_1, then x_2, prioritizing the inclusion branch ($x_i = 1$) before the exclusion branch ($x_i = 0$). At each node, we track the tuple:

$$(\text{Accumulated Value},\ \text{Residual Capacity},\ \text{Upper Bound } UB,\ \text{Incumbent } LB^*). \tag{3.24}$$

Trace Under LP Relaxation

We now walk through every step of the search tree under the continuous LP relaxation, as depicted in Figure 3.1:

1. **Node 1 (Root)**: No decisions made ($V_{\text{fixed}} = 0, C_{\text{residual}} = 9, LB^* = -\infty$). The LP relaxation takes Item 0 fully ($w_0 = 4, v_0 = 28$), leaving $9 - 4 = 5$ units of capacity. Item 1 has weight $w_1 = 6 > 5$, so it becomes the critical item ($j = 1$). We take fraction $\frac{5}{6}$ of Item 1, yielding fractional profit $\frac{5}{6} \times 30 = 25$. Thus:

   $$UB = 28 + 25 = 53. \tag{3.25}$$

   Since $53 > LB^* = -\infty$, the node is active. We branch on $x_0 = 1$.

2. **Node 2 ($x_0 = 1$)**: Item 0 is included ($V_{\text{fixed}} = 28, C_{\text{residual}} = 9 - 4 = 5, LB^* = -\infty$). The remaining capacity is 5. For the remaining items $\{1, 2\}$, Item 1 is the critical item with fraction $\frac{5}{6}$, giving bound $UB = 28 + \frac{5}{6} \times 30 = 53$. Since $53 > -\infty$, we branch on $x_1 = 1$.

3. **Node 3 ($x_0 = 1, x_1 = 1$)**: Including Item 1 requires $w_1 = 6$, but $C_{\text{residual}} = 5$. The required total weight is $4 + 6 = 10 > 9$. This node violates capacity:

   $$C_{\text{residual}} = 5 - 6 = -1 < 0 \implies \textbf{Pruned by Infeasibility}. \tag{3.26}$$

   The algorithm backtracks to Node 2 and explores the right branch $x_1 = 0$.

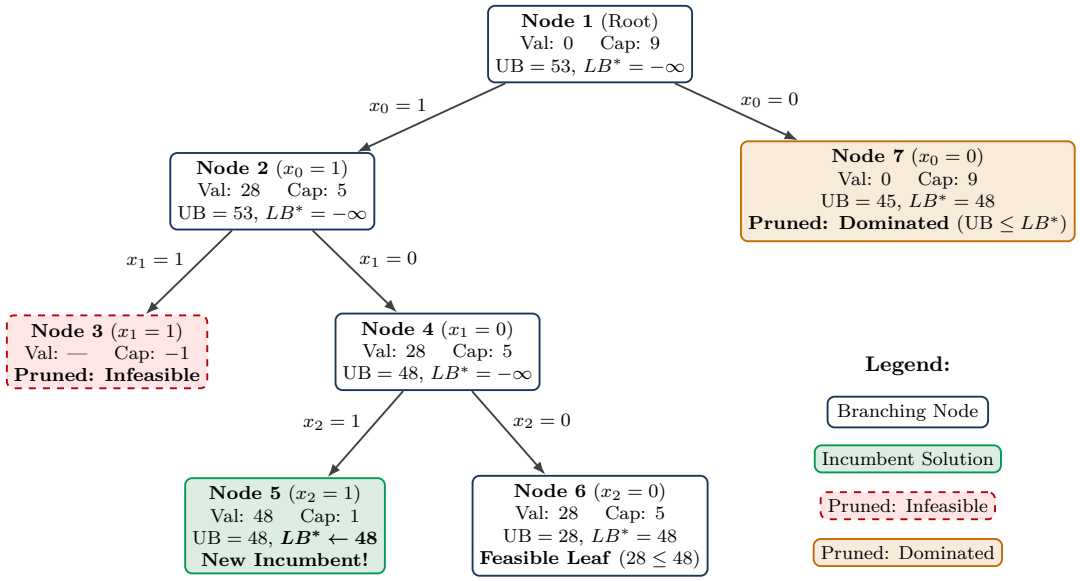


Figure 3.1: Branch and Bound search tree for the 0/1 knapsack instance (3.23) using the continuous LP relaxation. The search explores left branches first ($x_i = 1$). When Node 5 establishes the incumbent $LB^* = 48$, Node 7 on the right branch is evaluated. Its upper bound $UB = 45 \leq 48$ allows pruning the entire right subtree without exploring x_1 and x_2.

4. **Node 4 ($x_0 = 1, x_1 = 0$)**: Item 1 is excluded ($V_{\text{fixed}} = 28, C_{\text{residual}} = 5, LB^* = -\infty$). The only remaining item is Item 2 ($w_2 = 4, v_2 = 20$). Since $w_2 \leq C_{\text{residual}}$, it fits completely. The upper bound is $UB = 28 + 20 = 48$. We branch on $x_2 = 1$.
5. **Node 5 ($x_0 = 1, x_1 = 0, x_2 = 1$)**: Item 2 is included ($V_{\text{fixed}} = 28 + 20 = 48, C_{\text{residual}} = 5 - 4 = 1$). All items have been assigned, so this is a complete, feasible integer solution. We update the incumbent:

$$\mathbf{x}^* = (1, 0, 1), \quad LB^* \leftarrow 48. \tag{3.27}$$

   Being a leaf, the node terminates. The algorithm backtracks to Node 4 and tests $x_2 = 0$.
6. **Node 6 ($x_0 = 1, x_1 = 0, x_2 = 0$)**: Item 2 is excluded ($V_{\text{fixed}} = 28, C_{\text{residual}} = 5$). This leaf represents candidate solution $(1, 0, 0)$ with value 28. Since $28 \leq LB^* = 48$, it does not improve the incumbent. The search backtracks all the way to Root (Node 1).
7. **Node 7 ($x_0 = 0$)**: Item 0 is excluded ($V_{\text{fixed}} = 0, C_{\text{residual}} = 9, LB^* = 48$). We solve the continuous relaxation on undecided items $\{1, 2\}$ with available capacity 9:
   - Item 1 is taken fully ($w_1 = 6, v_1 = 30$), leaving $9 - 6 = 3$ units of capacity.
   - Item 2 has weight $w_2 = 4 > 3$, so it is the critical item. We take fraction $\frac{3}{4}$ of Item 2, contributing $\frac{3}{4} \times 20 = 15$.

   The resulting continuous upper bound is:

$$UB = 30 + 15 = 45. \tag{3.28}$$

   We now evaluate the pruning condition against the incumbent $LB^* = 48$:

$$UB = 45 \leq LB^* = 48 \implies \textbf{Pruned by Bound (Dominated)}! \tag{3.29}$$

   The entire subtree beneath $x_0 = 0$ (which contains candidate solutions $(0, 1, 1)$, $(0, 1, 0)$, $(0, 0, 1)$, and $(0, 0, 0)$) is discarded in a single bounding test.

The search terminates. The optimal solution is $\mathbf{x}^* = (1, 0, 1)$ with optimal profit $z^* = 48$. Exactly **7 nodes** were visited, out of 15 in the full binary tree.

Comparison with Capacity Relaxation

To appreciate the decisive role of bound tightness, let us examine the execution of Branch and Bound on the identical instance using the naive *Capacity Relaxation* (3.12), illustrated in Figure 3.2:

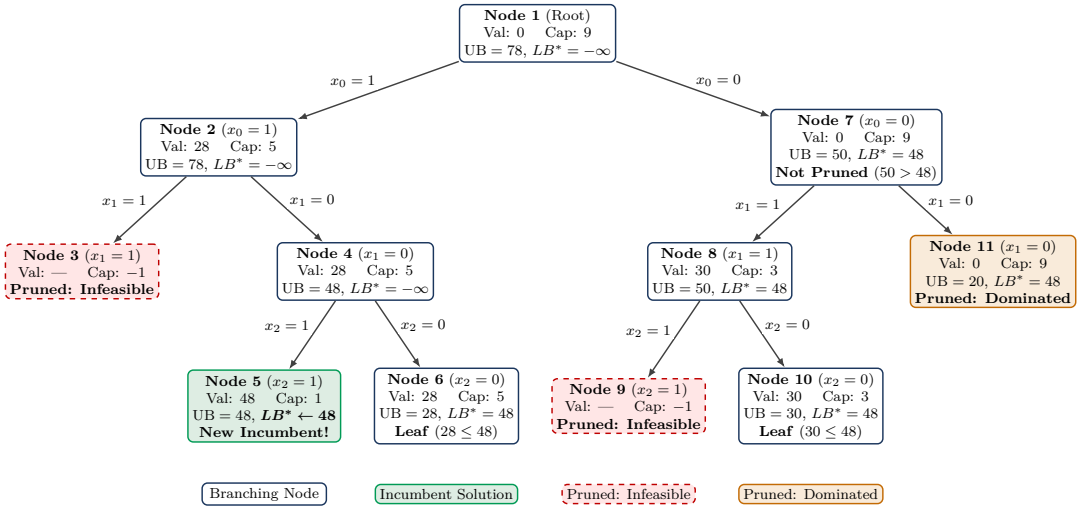


Figure 3.2: Branch and Bound search tree for instance (3.23) using the weak Capacity Relaxation. Because UB_{capa} ignores item weights, Node 7 produces an upper bound of $50 > 48$. Consequently, the branch cannot be pruned, forcing the algorithm to explore 4 additional nodes (Nodes 8, 9, 10, 11).

Under capacity relaxation:

- At Root (Node 1), $UB = 28 + 30 + 20 = 78$.
- At Node 2 ($x_0 = 1$), $UB = 28 + 30 + 20 = 78$.
- At Node 4 ($x_0 = 1, x_1 = 0$), $UB = 28 + 20 = 48$.
- At Node 5, the incumbent is updated to $LB^* = 48$.
- Now consider **Node 7 ($x_0 = 0$)**: The accumulated value is 0, and remaining items are $\{1, 2\}$. The capacity relaxation bound is:

$$UB_{\text{capa}} = 0 + v_1 + v_2 = 30 + 20 = 50. \tag{3.30}$$

  Because $50 > LB^* = 48$, **Node 7 cannot be pruned!**
- The search is forced to expand Node 8 ($x_0 = 0, x_1 = 1$, $UB = 50$), evaluate Node 9 ($x_0 = 0, x_1 = 1, x_2 = 1$, infeasible), evaluate Node 10 ($x_0 = 0, x_1 = 1, x_2 = 0$, leaf with value 30), and expand Node 11 ($x_0 = 0, x_1 = 0$, $UB = 20 \leq 48$, pruned).

A quantitative comparison between the two bounding procedures is summarized in Table 3.2.

The Cost-Benefit Tradeoff of Bounding. Computing the LP relaxation requires sorting items once in $\mathcal{O}(n \log n)$ time and evaluating fractional allocations in $\mathcal{O}(n)$ time per node (or $\mathcal{O}(1)$ incrementally). While slightly more expensive per node than the $\mathcal{O}(1)$ capacity sum, the dramatic reduction in tree size saves exponential amounts of work as n increases. Tight bounds are the lifeblood of efficient Branch and Bound.

Table 3.2: Quantitative comparison of Branch and Bound under Capacity and LP Relaxations on instance (3.23).

Metric	Capacity Relaxation	Continuous LP Relaxation
Root Upper Bound (UB_0)	78	53
Nodes Visited	11	7
Infeasible Nodes Detected	2 (Nodes 3, 9)	1 (Node 3)
Subtrees Pruned by Bound	1 (Node 11)	1 (Node 7)
Search Space Eliminated by Bound	2 leaves (25%)	4 leaves (50%)
Optimal Solution Found	$(1, 0, 1)$, Value 48	$(1, 0, 1)$, Value 48

3.3 Combinatorial Relaxations for the TSP

We now turn from the 0/1 Knapsack problem to one of the most celebrated and historically significant problems in combinatorial optimization: the *Traveling Salesperson Problem* (TSP). The TSP is **NP**-hard and exhibits factorial growth in its naive search space: on a complete graph with n vertices, there are $\frac{(n-1)!}{2}$ distinct undirected tours. Solving the TSP to provable optimality for non-trivial instances requires designing bounding procedures that compute strong, admissible lower bounds by relaxing hard combinatorial constraints.

3.3.1 TSP Relaxations and Formulations

Let $G = (V, E)$ be a complete undirected weighted graph with vertex set $V = \{0, 1, \ldots, n-1\}$, edge set $E = \{\{i, j\} : i, j \in V, i < j\}$, and non-negative symmetric edge costs $c_e \geq 0$ for each $e \in E$. A *tour* (or Hamiltonian cycle) is a simple closed cycle that visits every vertex in V exactly once and returns to the starting vertex. The objective is to find a tour of minimum total cost.

To solve the TSP via Branch and Bound, we require strong, admissible lower bounds that can be evaluated quickly at each search node. Rather than relying on classical algebraic integer linear programming models with an exponential family of subtour elimination constraints (such as the Dantzig–Fulkerson–Johnson formulation [DFJ54], which requires dynamic separation algorithms in branch-and-cut solvers), we characterize a Hamiltonian cycle through three complementary graph-theoretic formulations. Each formulation isolates different topological properties of a tour and immediately suggests natural, polynomial-time combinatorial relaxations to derive lower bounds:

1. **Formulation 1**: Unique connected component and degree-2 regularity, yielding the Assignment (2-factor) and Cheapest Incident Edges relaxations.
2. **Formulation 2**: Spanning tree on V plus one edge, yielding the Minimum Spanning Tree plus Cheapest Edge relaxation.
3. **Formulation 3**: 1-tree rooted at vertex 0 with degree-2 regularity, yielding the celebrated Held–Karp 1-tree relaxation.

Formulation 1: Connected 2-Factor

Rather than specifying an exponential family of subset inequalities, we can characterize a Hamiltonian cycle purely in terms of graph topology. Let $G(x) = (V, E(x))$ denote the spanning subgraph induced by the edge selection vector $\boldsymbol{x} \in \{0, 1\}^{|E|}$, where $E(x) = \{e \in E : x_e = 1\}$.

We state the optimization problem directly on the structural properties of $G(x)$:

$$\begin{aligned} \text{minimize} \quad & \sum_{e \in E} c_e x_e \\ \text{subject to} \quad & \sum_{e \in \delta(i)} x_e = 2, \quad \forall i \in V \quad \text{(degree constraints)}, \\ & \kappa(G(x)) = 1 \quad \text{(unique connected component)}, \\ & x_e \in \{0,1\}, \quad \forall e \in E, \end{aligned} \tag{3.31}$$

where $\kappa(G(x))$ denotes the number of connected components in the subgraph $G(x)$.

Proposition 3.3 *Equivalence of the Connected 2-Regular Subgraph Formulation*

A spanning subgraph $G' = (V, E')$ is a valid Hamiltonian cycle on n vertices if and only if it satisfies:

1. Every vertex has degree exactly 2: $\deg_{G'}(i) = 2$ for all $i \in V$.
2. The subgraph consists of a unique connected component: $\kappa(G') = 1$.

Proof. ($\Rightarrow$) If G' is a Hamiltonian cycle, it is a single simple cycle visiting every vertex in V exactly once. By definition, every vertex has degree 2, and the cycle is path-connected across all n vertices; hence, $\kappa(G') = 1$.

($\Leftarrow$) Suppose G' satisfies both conditions. In any graph where every vertex has degree exactly 2, every connected component must itself be a 2-regular connected graph. The only connected 2-regular graphs are simple cycles. Therefore, G' is a disjoint union of simple cycles (a 2-factor). Because $\kappa(G') = 1$, there exists exactly one connected component, which means G' consists of a single simple cycle. Since G' spans all n vertices of V, this unique cycle has length n, which is by definition a Hamiltonian cycle. □

Formulation (3.31) isolates the two orthogonal dimensions of the TSP:

- **A local condition**: each vertex must satisfy 2-regularity ($\deg(i) = 2$).
- **A global topological condition**: the subgraph must form a single connected component ($\kappa = 1$).

Deriving Relaxations from Formulation 1. By relaxing one or both of these two conditions, Formulation (3.31) directly yields two combinatorial lower bounds:

Relaxation 1A: Minimum Weight Assignment (Bipartite Matching / 2-Factor) Relaxation. What if we drop the global connectivity condition ($\kappa = 1$) while strictly preserving degree-2 regularity across all vertices? In directed terms, every vertex in a tour must have in-degree 1 and out-degree 1. Dropping the unique connected component constraint

yields the classical *Linear Assignment Problem* (AP):

$$\begin{aligned} \text{minimize} \quad & \sum_{i=0}^{n-1} \sum_{j=0, j\neq i}^{n-1} c_{ij} x_{ij} \\ \text{subject to} \quad & \sum_{j=0, j\neq i}^{n-1} x_{ij} = 1, \quad \forall i \in \{0, \ldots, n-1\}, \\ & \sum_{i=0, i\neq j}^{n-1} x_{ij} = 1, \quad \forall j \in \{0, \ldots, n-1\}, \\ & x_{ij} \in \{0, 1\}, \quad \forall i \neq j. \end{aligned} \tag{3.32}$$

The first constraint ensures each city has exactly one successor (out-degree 1), while the second ensures each city has exactly one predecessor (in-degree 1). Setting $c_{ii} = \infty$ explicitly prevents self-loops ($x_{ii} = 0$).

Definition 3.5 *2-Factor (Cycle Cover)*

A *2-factor* (or cycle cover) of a graph G is a spanning subgraph in which every vertex has degree exactly 2. In a directed graph, a 2-factor is a set of vertex-disjoint directed cycles that collectively visit every vertex in V exactly once.

Theorem 3.4 *Assignment Relaxation Bound*

Every valid TSP tour is a 2-factor consisting of a single cycle of length n. Consequently, the optimal solution to the Assignment Problem (3.32) provides an admissible lower bound on the TSP:

$$LB_{\text{AP}} \leq z^*_{\text{TSP}}. \tag{3.33}$$

If the optimal assignment solution forms a single connected cycle, it is a feasible TSP tour and solves the problem to global optimality.

Proof. Any valid TSP tour corresponds to an assignment matrix x_{ij} where each city has exactly one successor and one predecessor, with no subtours. Since the Assignment Problem minimizes total cost over all 2-factors (with or without subtours), its feasible domain strictly contains all TSP tours. Thus, $LB_{\text{AP}} \leq z^*_{\text{TSP}}$. If the solution contains only one cycle, it satisfies global connectivity, making it feasible and optimal for the original TSP. □

- **Constraints Relaxed**: Global connectivity ($\kappa = 1$) is completely dropped. The solution may decompose into multiple disconnected cycles (subtours).
- **Constraints Preserved**: 2-regularity across all vertices is strictly enforced ($\deg(i) = 2$ for all $i \in V$).
- **Algorithmic Complexity**: The Assignment Problem can be solved in $\mathcal{O}(n^3)$ time using the Hungarian algorithm (Kuhn-Munkres [Kuh55]) or the Jonker-Volgenant shortest augmenting path algorithm [JV87].
- **Bound Quality**: Strong. Because degree 2 is enforced at every vertex, LB_{AP} is often close to z^*_{TSP}. However, solving an $\mathcal{O}(n^3)$ matching problem at every search node is computationally intensive.

Example 3.2 *Computing the Assignment (2-Factor) Bound*

Solving the Linear Assignment Problem on our canonical 7-vertex instance (Figure 3.3(c)) enforces in-degree 1 and out-degree 1 at every vertex. The optimal 2-factor decomposes into two disconnected directed subtours:

- Subtour 1 (5 vertices): $0 \to 5 \to 4 \to 6 \to 1 \to 0$ with directed edge costs $1 + 2 + 2 + 3 + 4 = 12$.
- Subtour 2 (2 vertices): $2 \leftrightarrow 3$ (directed 2-cycle $2 \to 3 \to 2$) with directed edge costs $3 + 3 = 6$.

The resulting assignment lower bound is:

$$LB_{\text{AP}} = 12 + 6 = 18. \tag{3.34}$$

Although every vertex strictly satisfies 2-regularity ($\deg(i) = 2$), the solution is not a feasible tour because it violates the unique connected component constraint by forming two disjoint cycles.

Relaxation 1B: Cheapest Incident Edges Relaxation. What if we relax Formulation (3.31) even further by dropping global connectivity AND uncoupling the coordinated degree choices? In an undirected graph, every tour vertex must have degree 2. A very fast lower bound is obtained by independently selecting the two cheapest available incident edges for each vertex $i \in V$ and summing their weights. Because every edge $e = \{u, v\}$ is incident to two endpoints, this sum counts each edge twice:

$$LB_{\text{incident}} = \frac{1}{2} \sum_{i \in V} \min_{\substack{j_1, j_2 \in V \setminus \{i\} \\ j_1 \neq j_2}} (c_{i,j_1} + c_{i,j_2}). \tag{3.35}$$

In directed or asymmetric models where each city must have in-degree 1, the bound simplifies to selecting the single cheapest incoming edge into each vertex:

$$LB_{\text{in}} = \sum_{i \in V} \min_{j \in V \setminus \{i\}} c_{ji}. \tag{3.36}$$

- **Constraints Relaxed**: Global connectivity ($\kappa = 1$) is entirely dropped. Degree-2 regularity is relaxed to independent local edge choices; the union of edges need not form valid cycles.
- **Constraints Preserved**: Local incident coverage (each vertex has at least two incident edge choices, or in-degree at least 1).
- **Algorithmic Complexity**: Scanning each row of the distance matrix requires $\mathcal{O}(n^2)$ time total (or $\mathcal{O}(n)$ incrementally).
- **Bound Quality**: Very weak, but computationally inexpensive.

Formulation 2: Spanning Tree Plus Edge

A second, profoundly influential graph-theoretic characterization models a Hamiltonian cycle through tree connectivity:

$$\begin{aligned} \text{minimize} \quad & \sum_{e \in E} c_e x_e \\ \text{subject to} \quad & \{e \in E : x_e = 1\} \text{ forms a spanning tree on } V \text{ plus exactly 1 edge}, \\ & \sum_{e \in \delta(i)} x_e = 2, \quad \forall i \in V \quad \text{(degree constraints)}, \\ & x_e \in \{0,1\}, \quad \forall e \in E. \end{aligned} \tag{3.37}$$

Proposition 3.5 *Equivalence of the Spanning Tree Formulation*

Every valid TSP tour on n vertices consists of a spanning tree on V plus exactly one additional edge, with every vertex having degree exactly 2.

Proof. Let T be a valid Hamiltonian cycle on n vertices. By definition, T contains exactly n vertices and n edges, and every vertex has degree 2. If we remove any single edge $e^* = \{u, v\}$ from T, the remaining subgraph $T \setminus \{e^*\}$ is a simple path visiting all n vertices. A path on n vertices has $n-1$ edges, is connected, and contains no cycles; hence, $T \setminus \{e^*\}$ is a spanning tree on V. Reinserting e^* yields a spanning tree plus one edge that closes the cycle, where every vertex maintains degree 2. □

Deriving Relaxation from Formulation 2: Minimum Spanning Tree plus Cheapest Edge. What happens if we drop the degree-2 constraint on all vertices in Formulation (3.37), while strictly preserving tree connectivity on V plus one edge?

Definition 3.6 *Spanning Tree plus Cheapest Edge Bound*

The *Minimum Spanning Tree plus Cheapest Edge* relaxation drops the requirement that $\sum_{e \in \delta(i)} x_e = 2$ for all $i \in V$, while retaining the constraint that $\{e : x_e = 1\}$ forms a spanning tree on V plus one additional edge:

$$LB_{\text{MST+1}} = w(\text{MST}(V)) + \min_{e \in E \setminus \text{MST}(V)} c_e, \tag{3.38}$$

where $\text{MST}(V)$ denotes a Minimum Spanning Tree on the complete graph $G = (V, E)$, and $w(\text{MST}(V)) = \sum_{e \in \text{MST}(V)} c_e$.

Theorem 3.6 *Validity of the MST plus Cheapest Edge Bound*

For any TSP instance with non-negative edge costs, $LB_{\text{MST+1}} \leq z^*_{\text{TSP}}$.

Proof. Let T^* be an optimal TSP tour with cost z^*_{TSP}. By Theorem 3.5, T^* can be decomposed into a spanning tree $T_{\text{span}} \subseteq T^*$ on V ($n-1$ edges) and a single remaining edge $e^* \in T^* \setminus T_{\text{span}}$. By definition of the Minimum Spanning Tree, $w(\text{MST}(V)) \leq w(T_{\text{span}})$. Furthermore, e^* is an edge in $E \setminus T_{\text{span}}$, so $\min_{e \notin \text{MST}(V)} c_e \leq c_{e^*}$. Summing these two inequalities yields:

$$LB_{\text{MST+1}} = w(\text{MST}(V)) + \min_{e \notin \text{MST}(V)} c_e \leq w(T_{\text{span}}) + c_{e^*} = z^*_{\text{TSP}}. \tag{3.39}$$

□

- **Constraints Relaxed**: Degree-2 regularity on all vertices is completely relaxed. Vertices in MST(V) can be leaves ($\deg(i) = 1$) or branching hubs ($\deg(i) \geq 3$).
- **Constraints Preserved**: Global connectivity is strictly enforced (the spanning tree connects all n vertices in a single component), and exactly n edges are chosen.
- **Algorithmic Complexity**: An MST on n vertices can be computed in $\mathcal{O}(n^2)$ time via Prim's algorithm. Identifying the cheapest non-tree edge takes $\mathcal{O}(n^2)$ time (or $\mathcal{O}(|E|)$). Total time is strictly $\mathcal{O}(n^2)$.
- **Bound Quality**: Noticeably tighter than cheapest incident edges because global connectivity is preserved. However, relaxing degree 2 everywhere leaves many leaves and hubs.

Example 3.3 *Computing the MST plus Cheapest Edge Bound*

Consider the canonical 7-vertex weighted graph depicted in Figure 3.3 with vertex set $V = \{0, 1, 2, 3, 4, 5, 6\}$ and 12 symmetric edge costs:

$$\begin{aligned} &c_{0,1} = 4, \quad c_{0,5} = 1, \quad c_{0,2} = 5, \quad c_{1,6} = 3, \quad c_{1,5} = 2, \quad c_{6,5} = 6, \\ &c_{6,4} = 2, \quad c_{5,4} = 2, \quad c_{5,2} = 9, \quad c_{4,2} = 1, \quad c_{4,3} = 2, \quad c_{2,3} = 3. \end{aligned} \tag{3.40}$$

To evaluate $LB_{\text{MST+1}}$:

1. The Minimum Spanning Tree on all 7 vertices (MST(V)) selects 6 edges: $\{0, 5\}$ (cost 1), $\{4, 2\}$ (cost 1), $\{1, 5\}$ (cost 2), $\{5, 4\}$ (cost 2), $\{6, 4\}$ (cost 2), and $\{4, 3\}$ (cost 2). The total tree cost is:

$$w(\text{MST}(V)) = 1 + 1 + 2 + 2 + 2 + 2 = 10. \tag{3.41}$$

2. The remaining non-tree edges are $\{0, 1\}$ (cost 4), $\{0, 2\}$ (cost 5), $\{1, 6\}$ (cost 3), $\{6, 5\}$ (cost 6), $\{5, 2\}$ (cost 9), and $\{2, 3\}$ (cost 3). The cheapest non-tree edge is $\{2, 3\}$ with cost 3 (tied with $\{1, 6\}$).

Adding this cheapest edge yields the lower bound:

$$LB_{\text{MST+1}} = 10 + 3 = 13. \tag{3.42}$$

As illustrated in Figure 3.3(a), this relaxed solution leaves leaf vertices 0, 1, and 6 with degree 1, while vertex 4 has degree 4 and vertex 5 has degree 3.

Formulation 3: Rooted 1-Tree

Notice the primary weakness of Formulation 2's relaxation: it relaxes the degree constraint on *every* vertex. In their seminal work, Held and Karp [HK70] observed that we can anchor the tree connectivity by designating a specific root vertex, say vertex $0 \in V$. This yields the *1-tree* formulation:

$$\begin{aligned} \text{minimize} \quad & \sum_{e \in E} c_e x_e \\ \text{subject to} \quad & \{e \in E : x_e = 1\} \text{ forms a 1-tree rooted at vertex 0,} \\ & \sum_{e \in \delta(i)} x_e = 2, \quad \forall i \in V \quad \text{(degree constraints),} \\ & x_e \in \{0, 1\}, \quad \forall e \in E. \end{aligned} \tag{3.43}$$

Definition 3.7 *1-Tree*

Let $G = (V, E)$ be a connected graph and let $0 \in V$ be a designated root vertex. A *1-tree rooted at vertex* 0 consists of:

1. A spanning tree on the subgraph induced by the remaining $n-1$ vertices $V \setminus \{0\} = \{1, 2, \ldots, n-1\}$.
2. Exactly two distinct edges connecting root vertex 0 to the induced subgraph $V \setminus \{0\}$.

Notice that a 1-tree on n vertices contains exactly $(n-2)+2 = n$ edges, is connected, and contains *exactly one simple cycle*, which necessarily passes through root vertex 0.

Proposition 3.7 *Equivalence of the 1-Tree Formulation*

A spanning subgraph $T = (V, E')$ is a valid Hamiltonian cycle on n vertices if and only if T is a 1-tree rooted at vertex 0 and every vertex in V has degree exactly 2.

Proof. ($\Rightarrow$) Let T be a valid Hamiltonian cycle on n vertices. By definition, root vertex 0 has degree exactly 2 in T. Removing vertex 0 and its two incident edges leaves a simple path spanning all remaining $n-1$ vertices $V \setminus \{0\}$. A path on $n-1$ vertices has $n-2$ edges, is connected, and contains no cycles; hence, it is a spanning tree on $V \setminus \{0\}$. Thus, T is by definition a 1-tree rooted at vertex 0, where $\deg(i) = 2$ for all $i \in V$.

($\Leftarrow$) Suppose T is a 1-tree rooted at vertex 0 and satisfies $\deg_T(i) = 2$ for all $i \in V$. A 1-tree on n vertices contains $(n-2)+2 = n$ edges and is connected, so it has a unique connected component ($\kappa(T) = 1$). Since every vertex has degree exactly 2, by Theorem 3.3, T must be a single simple cycle spanning all n vertices—a valid Hamiltonian cycle. □

Deriving Relaxations from Formulation 3. Formulation (3.43) is structurally superior to Formulation (3.37) because the degree-2 constraint at root vertex 0 ($\deg(0) = 2$) is *built directly into the topological definition of the 1-tree*!

Relaxation 3: The Minimum Weight 1-Tree (Held–Karp) Relaxation. What happens if we strictly enforce the 1-tree structure (guaranteeing tree connectivity on $V \setminus \{0\}$ and $\deg(0) = 2$) while relaxing the degree-2 constraints on the remaining $n-1$ vertices $V \setminus \{0\}$?

Theorem 3.8 *The 1-Tree Bound on the TSP*

Every valid TSP tour is a 1-tree in which every vertex has degree exactly 2. Consequently, the minimum weight 1-tree provides an admissible lower bound on the optimal TSP tour cost:

$$LB_{\text{1-tree}} = w(\text{MST}(V \setminus \{0\})) + c_{0,j_1} + c_{0,j_2} \leq z^*_{\text{TSP}}, \tag{3.44}$$

where $j_1, j_2 \in V \setminus \{0\}$ are the two distinct vertices with the smallest edge costs to root vertex 0.

Proof. Let T be any valid TSP tour. By Theorem 3.7, T is a 1-tree rooted at vertex 0 with $\deg(i) = 2$ for all $i \in V$. Since the set of all valid TSP tours is a strict subset of all possible 1-trees on G, the minimum weight 1-tree cannot exceed the cost of the optimal tour. □

Proposition 3.9 *Dominance of the 1-Tree over MST plus Cheapest Edge*

For any choice of root vertex 0, the 1-tree bound is at least as tight as the MST plus cheapest edge bound:

$$LB_{\text{1-tree}} \geq LB_{\text{MST+1}}. \tag{3.45}$$

Proof. The MST plus cheapest edge relaxation allows vertex 0 to have degree 1, 3, or more in the selected n edges. The 1-tree explicitly restricts the feasible set of relaxed subgraphs by enforcing $\sum_{e \in \delta(0)} x_e = 2$. Restricting the feasible region of a minimization problem can never decrease the optimal objective value. Hence, $LB_{\text{1-tree}} \geq LB_{\text{MST+1}}$. □

- **Constraints Relaxed**: Degree-2 regularity on the $n-1$ vertices $V \setminus \{0\}$ is relaxed.
- **Constraints Preserved**: Degree-2 regularity at root vertex 0 is strictly enforced ($\deg(0) = 2$), global connectivity is preserved, and the graph contains exactly one cycle passing through vertex 0.
- **Algorithmic Complexity**: Computing the 1-tree requires finding an MST on $n-1$ vertices via Prim's algorithm in $\mathcal{O}(n^2)$ time, followed by an $\mathcal{O}(n)$ scan to locate the two cheapest edges incident to root 0. Total time is strictly $\mathcal{O}(n^2)$.
- **Bound Quality**: Excellent balance of speed and tightness. It forms the backbone of exact TSP solvers.

Example 3.4 *Computing the Minimum 1-Tree Bound*

We evaluate the 1-tree relaxation on the same 7-vertex instance, choosing vertex 0 as the designated root (Figure 3.3(b)):

1. An MST on the remaining 6 vertices $V \setminus \{0\} = \{1, 2, 3, 4, 5, 6\}$ selects 5 edges: $\{4, 2\}$ (cost 1), $\{1, 5\}$ (cost 2), $\{5, 4\}$ (cost 2), $\{6, 4\}$ (cost 2), and $\{4, 3\}$ (cost 2). The total weight of this tree is:

$$w(\text{MST}(V \setminus \{0\})) = 1 + 2 + 2 + 2 + 2 = 9. \tag{3.46}$$

2. The three edges incident to root vertex 0 in the original graph are $\{0, 5\}$ (cost 1), $\{0, 1\}$ (cost 4), and $\{0, 2\}$ (cost 5). The two cheapest incident edges are $\{0, 5\}$ and $\{0, 1\}$, with cost:

$$c_{0,5} + c_{0,1} = 1 + 4 = 5. \tag{3.47}$$

Summing these two components yields the 1-tree lower bound:

$$LB_{\text{1-tree}} = 9 + 5 = 14. \tag{3.48}$$

Notice that $LB_{\text{1-tree}} = 14 > LB_{\text{MST+1}} = 13$, confirming that enforcing degree 2 at vertex 0 tightens the relaxation (Theorem 3.9).

Connecting to Lagrangian Relaxation: Strengthening the 1-Tree Bound. Formulation (3.43) provides the foundation for Lagrangian relaxation. Since the 1-tree relaxation already enforces connectivity and $\deg(0) = 2$, the degree constraints $\deg(i) = 2$ for the remaining $n-1$ vertices $i \in V \setminus \{0\}$ can be dualized using Lagrange multipliers $\boldsymbol{\pi} \in \mathbb{R}^n$. In Chapter 6, we will explain how subgradient optimization iteratively adjusts these multipliers

to penalize degree violations, strengthening the 1-tree bound to obtain significantly tighter lower bounds (the celebrated Held–Karp bound).

3.3.2 Comparison of the TSP Relaxations

The four relaxations represent complementary projections of the Traveling Salesperson polytope. Their theoretical properties and computational trade-offs are summarized in Table 3.3.

Table 3.3: Comparison of Combinatorial Relaxations for the Traveling Salesperson Problem.

Relaxation	Parent Formulation	Degree Constraints	Connectivity	Complexity	Relative Tightness
Cheapest Incident	Formulation 1	Relaxed ($\deg \geq 1$)	Dropped	$\mathcal{O}(n^2)$	Very Weak
Assignment (2-Factor)	Formulation 1	$\deg(i) = 2 \quad \forall i \in V$	Dropped	$\mathcal{O}(n^3)$	Strong
MST + Cheapest Edge	Formulation 2	Relaxed ($\forall i \in V$)	Preserved	$\mathcal{O}(n^2)$	Moderate
Minimum 1-Tree	Formulation 3	$\deg(0) = 2$, others relaxed	Preserved	$\mathcal{O}(n^2)$	Strong

Example 3.5 *Optimal Tour and Bounding Hierarchy*

Figure 3.3(d) displays the unique optimal Hamiltonian tour on this 7-vertex graph:

$$0 - 5 - 1 - 6 - 4 - 3 - 2 - 0, \tag{3.49}$$

with total optimal cost:

$$z^*_{\text{TSP}} = 1 + 2 + 3 + 2 + 2 + 3 + 5 = 18. \tag{3.50}$$

Comparing all bounds across Figure 3.3 illustrates the bounding hierarchy:

$$LB_{\text{MST+1}}\ (13) \quad \leq \quad LB_{\text{1-tree}}\ (14) \quad \leq \quad LB_{\text{AP}}\ (18) \quad = \quad z^*_{\text{TSP}}\ (18). \tag{3.51}$$

Remarkably, while the assignment bound attains the optimal value ($LB_{\text{AP}} = 18 = z^*_{\text{TSP}}$), it cannot certify optimality on its own because its relaxed solution is infeasible for the TSP (disconnected into two subtours). Within Branch and Bound, branching would still be required to eliminate the subtour, or tree connectivity must be imposed.

Orthogonal Relaxations. Notice the striking structural orthogonality between the *1-Tree* and the *Assignment* relaxations:

- The 1-Tree relaxation (derived from Formulation 3 (3.43)) **preserves connectivity** but **relaxes degrees** (except at vertex 0).
- The Assignment relaxation (derived from Formulation 1 (3.31)) **preserves degrees** everywhere but **relaxes connectivity**.

In Chapter 6, we will explain how to strengthen the 1-tree bound using Lagrangian relaxation to obtain significantly tighter lower bounds.

3.3.3 Branching Schemes and Partial Decisions

When Branch and Bound explores the search space of the TSP, every subproblem P_s corresponds to a node in the search tree where a *partial set of decisions* has already been committed along the path from the root node to P_s. Formally, the decision variables $\boldsymbol{x} \in \{0,1\}^{|E|}$ are partitioned into three subsets:

- **Mandatory (Fixed) Edges** $E_{\text{fixed}} = \{e \in E : x_e = 1\}$: edges that *must* be included in any tour completed from this node.

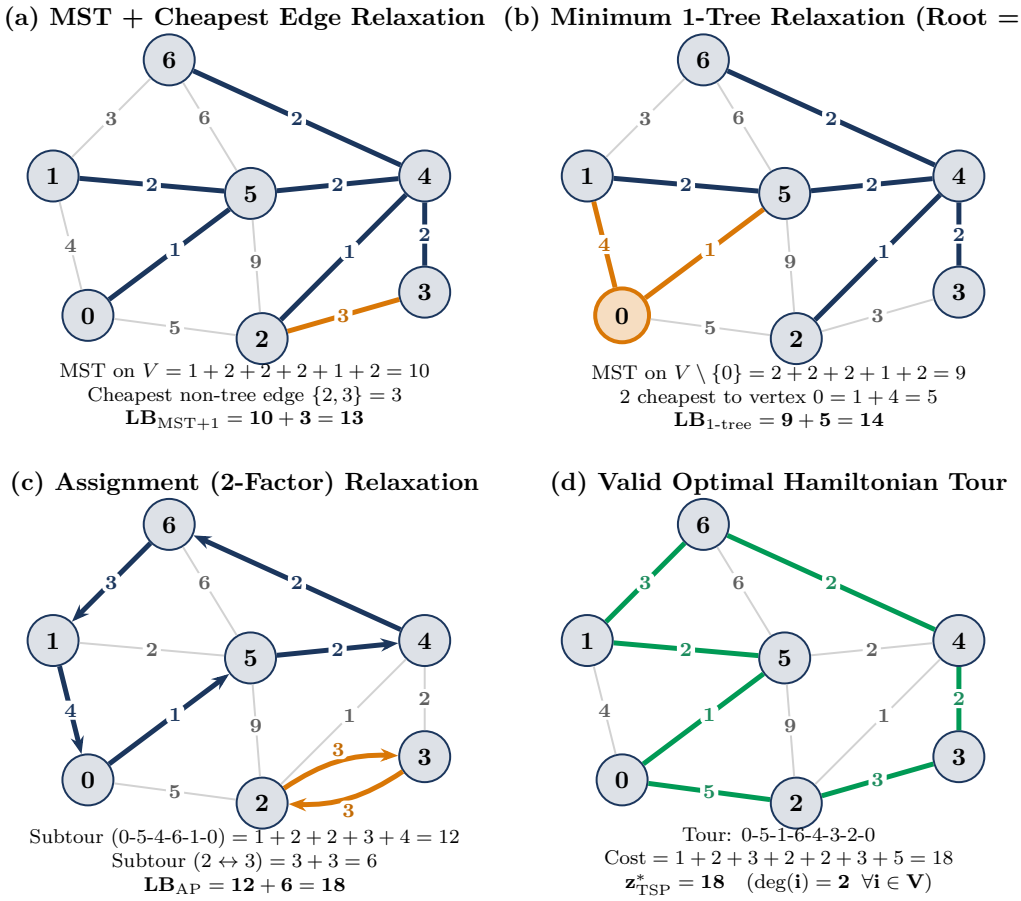


Figure 3.3: Comparison of combinatorial relaxations and the optimal Hamiltonian tour on the canonical 7-vertex graph. Inactive graph edges are shown in light gray with their costs; active edges are highlighted. (a) Minimum Spanning Tree on V (cost 10) plus cheapest non-tree edge $\{2,3\}$ (cost 3) yields $LB_{\text{MST+1}} = 13$ (from Formulation 2). (b) Minimum 1-Tree with root 0: MST on $V \setminus \{0\}$ (cost 9) plus two cheapest edges to vertex 0 (cost 5) yields $LB_{\text{1-tree}} = 14$ (from Formulation 3). (c) Assignment (2-factor) relaxation enforces in-degree 1 and out-degree 1 at every vertex, decomposing into two subtours of cost $12+6 = 18$ (from Formulation 1). (d) Optimal Hamiltonian tour of cost $z^*_{\text{TSP}} = 18$. Across all relaxations, the bounding hierarchy $13 \leq 14 \leq 18 = z^*_{\text{TSP}}$ holds.

- **Excluded (Forbidden) Edges** $E_{\text{excluded}} = \{e \in E : x_e = 0\}$: edges that *cannot* be selected in any completion.
- **Free (Undecided) Edges** $E_{\text{free}} = E \setminus (E_{\text{fixed}} \cup E_{\text{excluded}})$: candidate edges available for selection.

Conditioning Lower Bounds on Ancestor Decisions. To enable effective pruning during search, the lower-bounding procedure at any node P_s must strictly take into account all branching decisions committed along the path from the root node down to P_s. Conditioning the relaxation on E_{fixed} and E_{excluded} restricts the feasible space of completions, forcing the relaxed solution to incorporate all mandatory edges and avoid all forbidden ones. Consequently, the computed lower bound is monotonically non-decreasing along every branch of the search tree:

$$LB(P_0) \leq LB(P_1) \leq \cdots \leq LB(P_s). \tag{3.52}$$

As search descends deeper into the tree, the accumulated restrictions force the relaxation to adopt increasingly constrained, higher-cost substructures. As soon as $LB(P_s) \geq UB^*$, the entire subtree rooted at P_s is fathomed and pruned immediately, preventing the exploration of exponential numbers of suboptimal completions.

Partitioning the Search Space

Before examining how relaxations incorporate committed edges, we must address the branching operator itself: how do we divide an active subproblem into mutually exclusive, smaller subproblems? Two primary branching paradigms exist for the Traveling Salesperson Problem:

Scheme 1: Sequential Path Extension (Tour Construction). In this intuitive, constructive scheme, the search tree builds a Hamiltonian cycle sequentially as a growing path. Fixing origin vertex 0 as the starting point, each search node at depth k represents an ordered partial path:

$$P = [0, v_1, v_2, \ldots, v_k], \tag{3.53}$$

together with the set of unvisited candidate vertices:

$$V_{\text{remaining}} = V \setminus \{0, v_1, \ldots, v_k\}. \tag{3.54}$$

The branching step generates $|V_{\text{remaining}}|$ child subproblems, each corresponding to choosing a distinct candidate vertex $v_{k+1} \in V_{\text{remaining}}$ to append to the path, committing edge $\{v_k, v_{k+1}\}$ to E_{fixed}.

The sequential path-extension scheme possesses clear operational advantages: the maximum search depth is strictly bounded by n, and detecting a leaf solution is straightforward (when $k = n - 1$, the path visits all cities and is closed by appending edge $\{v_{n-1}, 0\}$). However, it imposes an arbitrary chronological ordering on tour construction: early decisions must choose between local paths even when the global topology of the remaining tour is still completely unconstrained.

Scheme 2: Conflict-Driven Edge Branching (Relaxation-Guided on the 1-Tree). Rather than forcing edges to form a single continuous path from the start, a more flexible and mathematically elegant paradigm allows edges to be fixed anywhere across the graph, letting the violations of the relaxation directly dictate the branching choices.

Suppose we evaluate the minimum 1-tree relaxation at search node P_s, respecting all currently committed edges E_{fixed} and E_{excluded}:

1. **Exact Optimality Detection**: If every vertex in the resulting 1-tree T happens to have degree exactly 2 ($\deg_T(i) = 2$ for all $i \in V$), then T is a connected 2-regular spanning graph. By Theorem 3.3, T is a valid Hamiltonian cycle! The relaxation has solved the subproblem to global optimality without further branching: we update the incumbent upper bound $UB^* \leftarrow \min(UB^*, w(T))$ and fathom the node immediately.
2. **Degree-Conflict Identification**: If T is not a valid tour, it cannot satisfy 2-regularity everywhere. Because the total degree sum in a 1-tree is $\sum_{i \in V} \deg_T(i) = 2n$ and root vertex 0 strictly satisfies $\deg_T(0) = 2$, there must exist at least one vertex $v \in V \setminus \{0\}$ that has degree strictly greater than 2:

$$\deg_T(v) \geq 3. \tag{3.55}$$

   We refer to such a vertex as a *conflict hub*. In any valid tour, vertex v can have at most 2 incident edges. Therefore, among the $d = \deg_T(v) \geq 3$ incident edges chosen by the 1-tree, at least one edge *cannot* belong to any feasible tour completion!
3. **Resolving the Conflict by Branching**: We can directly resolve this conflict by branching on the incident edges of vertex v:
   - *Binary Edge Branching*: Select an edge $e \in \delta_T(v) \cap E_{\text{free}}$ incident to conflict vertex v in the 1-tree (for example, the edge with the highest cost or the greatest degree violation). Partition into two child subproblems:
     - **Exclude Edge** e: enforce $x_e = 0$ ($e \in E_{\text{excluded}}$). This branch directly eliminates the conflict edge, forcing the 1-tree in this subtree to find an alternative, more costly connection. This almost always produces an immediate increase in the lower bound ($LB_{\text{child}} > LB_{\text{parent}}$).
     - **Include Edge** e: enforce $x_e = 1$ ($e \in E_{\text{fixed}}$). This branch commits edge e to the tour, which consumes one unit of the degree quota at vertex v and triggers constraint propagation to exclude other edges.
   - *Multi-Way Exclusion Branching*: Alternatively, let $\{e_1, e_2, \ldots, e_d\}$ be the $d \geq 3$ edges incident to vertex v in the 1-tree. Since any valid tour can contain at most 2 of these edges, at least one must be excluded. One can create mutually disjoint child branches by excluding e_1 in the first branch, fixing e_1 and excluding e_2 in the second branch, and so forth.

Conflict-driven branching is widely considered the gold standard for relaxation-based Branch and Bound because every branch is purposeful: it directly targets and eliminates the structural flaw (the degree violation) that prevented the relaxation from being a feasible solution.

Conditioning Bounds with Fixed Edges

Regardless of whether path-extension or conflict-driven branching is used, the lower-bounding engine must correctly enforce the sets E_{excluded} and E_{fixed} inherited from all ancestors up to the current node.

1. Handling Excluded Edges ($x_e = 0$). An edge $e = \{u, v\} \in E_{\text{excluded}}$ cannot appear in any completion of the current subproblem:

- **Cost Masking**: In the distance matrix D, excluded edges are effectively eliminated by setting $c_{uv} \leftarrow \infty$ (or tracked via a fast boolean mask `excluded`$[u][v] =$ `true`). Any graph algorithm (Prim, Kruskal, Dijkstra, Hungarian) simply skips masked edges.

- **Logical Deductions (Constraint Propagation)**: Committing edges to E_{fixed} logically forces additional edges into E_{excluded} through constraint propagation:

  1. *Degree-2 Exhaustion*: As soon as any vertex $v_i \in V$ has two incident edges fixed in E_{fixed}, its degree quota is completely exhausted. To maintain 2-regularity, every other incident edge must be excluded:

  $$\forall u \notin \mathcal{N}_{\text{fixed}}(v_i), \quad \text{mark edge } \{v_i, u\} \text{ as } \textbf{excluded}. \tag{3.56}$$

  In sequential path extension $P = [0, v_1, \ldots, v_k]$, every internal vertex v_i $(1 \leq i < k)$ already has its incoming edge (v_{i-1}, v_i) and outgoing edge (v_i, v_{i+1}) fixed; hence, all other edges incident to v_i are immediately excluded.

  2. *Premature Subtour Prevention*: If a set of mandatory edges forms a path connecting vertices u and w of length $\ell < n$, adding edge $\{u, w\}$ would close an isolated cycle of length $\ell < n$. Therefore:

  $$\text{if } |E_{\text{fixed}}| < n - 1, \quad \text{mark edge } \{u, w\} \text{ as } \textbf{excluded}. \tag{3.57}$$

  In sequential path extension $P = [0, v_1, \ldots, v_k]$, this rule dictates that return edge $\{v_k, 0\}$ is strictly forbidden as long as unvisited cities remain $(k < n - 1)$.

  3. *Anti-Parallel Exclusion*: In directed models, selecting directed arc (u, v) immediately excludes reverse arc (v, u) to prevent forming an invalid 2-cycle.

2. Handling Mandatory (Fixed) Edges ($x_e = 1$). When edges E_{fixed} are committed to the tour, the relaxation must be constrained to include every edge in E_{fixed}:

- **In Spanning Tree and 1-Tree Relaxations**: The most efficient way to incorporate fixed edges into an MST or 1-tree is via Kruskal's algorithm equipped with a Disjoint-Set (Union-Find) data structure (see Appendix A for a comprehensive review of Kruskal's algorithm, the Cut Property, and the Union-Find data structure):

  1. Initialize a Union-Find partition where each vertex is in its own singleton set.
  2. *Pre-insert all mandatory edges* $e = \{u, v\} \in E_{\text{fixed}}$:
     - If any vertex has more than 2 incident fixed edges ($\deg_{E_{\text{fixed}}}(v) > 2$), the subproblem violates 2-regularity and is **pruned immediately by infeasibility** ($LB \leftarrow \infty$).
     - If find(u) == find(v), edge e connects two vertices already in the same connected component, forming a cycle! If $|E_{\text{fixed}}| < n$, this constitutes a premature subtour, and the subproblem is **pruned immediately by infeasibility** ($LB \leftarrow \infty$).
     - Otherwise, call union(u, v) and accumulate the cost c_e into the baseline bound.
  3. *Greedy Completion on Free Edges*: Once all mandatory edges are merged, iterate through the remaining non-excluded free edges E_{free} in non-decreasing order of cost, adding each edge whose endpoints belong to different sets until the spanning tree on $V \setminus \{0\}$ is complete.
  4. *Root Connections for the 1-Tree*: For root vertex 0, if $m \in \{0, 1, 2\}$ edges incident to vertex 0 are already in E_{fixed}, they are retained; the remaining $2 - m$ connections are selected greedily as the cheapest available non-excluded edges incident to 0.

- **In Assignment Relaxations**: If directed arc (u, v) is fixed $(x_{uv} = 1)$:

  - Remove row u and column v from the cost matrix, reducing the assignment problem from dimension m to $m-1$.
  - Add the constant cost c_{uv} directly to the accumulated baseline lower bound.
  - Alternatively, set $c_{uv} \leftarrow -M$ (or 0) while setting all other entries in row u and column v to ∞.
- **In Cheapest Incident Edges**: If an edge incident to vertex v is fixed to u, the first incident cost for v is locked to c_{uv}, and only the second incident edge is chosen greedily among non-excluded neighbors.

Example 3.6 *Branching Decisions and Bound Conditioning on a 7-Vertex Graph*

We illustrate both branching paradigms and the bound-conditioning mechanics on the canonical 7-vertex TSP instance from Figure 3.3, whose edge weights are given in (3.40).

1. Conflict-Driven 1-Tree Branching. Recall from Example 3.4 that the unconstrained 1-tree at root node P_0 (with root vertex 0) selects the 7 edges highlighted in Figure 3.3(b), achieving lower bound $LB(P_0) = 14$:

- Vertex degrees in the 1-tree: $\deg(0) = 2$, $\deg(1) = 2$, leaves $\deg(2) = 1$, $\deg(3) = 1$, $\deg(6) = 1$, and *conflict hubs* $\deg(5) = 3$ and $\deg(4) = 4$.

Because vertex 4 has degree 4 (incident to $\{4,2\}$, $\{4,3\}$, $\{6,4\}$, and $\{5,4\}$), it violates the degree-2 requirement of a valid tour. We choose incident edge $e = \{5,4\}$ (cost 2) and perform binary edge branching:

1. **Left Branch (Exclude $\{5,4\}$)**: We commit $x_{5,4} = 0$, so $E_{\text{excluded}} = \{\{5,4\}\}$ and $E_{\text{fixed}} = \emptyset$. When Kruskal's algorithm computes the MST on $V \setminus \{0\} = \{1,2,3,4,5,6\}$, edge $\{5,4\}$ is masked with $c_{5,4} \leftarrow \infty$. The algorithm selects:
   - $\{4,2\}$ (cost 1), $\{1,5\}$ (cost 2), $\{6,4\}$ (cost 2), $\{4,3\}$ (cost 2).
   - At this stage, the forest consists of two disconnected components: $\{1,5\}$ and $\{2,3,4,6\}$. Since edge $\{5,4\}$ is excluded, the cheapest available edge bridging these components is $\{1,6\}$ with cost 3 (cheaper than $\{6,5\}$ of cost 6 or $\{5,2\}$ of cost 9).

   The resulting MST on $V \setminus \{0\}$ has weight $1+2+2+2+3 = 10$ (increased by 1). Adding the two cheapest incident edges to root 0 ($\{0,5\}$ of cost 1 and $\{0,1\}$ of cost 4, sum 5) yields the updated lower bound:

$$LB(P_{\text{left}}) = 10 + 5 = 15. \tag{3.58}$$

   Notice that conditioning on the single exclusion $x_{5,4} = 0$ has **strictly strengthened the lower bound** from 14 to 15 ($LB(P_{\text{left}}) > LB(P_0)$) while reducing the degree of conflict vertex 4 from 4 to 3!
2. **Right Branch (Include $\{5,4\}$)**: We commit $x_{5,4} = 1$, so $E_{\text{fixed}} = \{\{5,4\}\}$. Edge $\{5,4\}$ is pre-inserted into the Union-Find structure, and Kruskal completes the tree using the remaining cheapest free edges. The lower bound remains $LB(P_{\text{right}}) = 14$, but vertex 4 has now consumed 1 of its 2 available degrees, allowing subsequent branching on its remaining edges to rapidly resolve the conflict or trigger infeasibility pruning.

2. Sequential Path Extension Branching. In the path-extension scheme, search starts at root $P_0 = [0]$:

1. Branching from $[0]$ chooses the next city. Suppose we branch to child $P_1 = [0, 5]$ (edge $\{0, 5\}$ fixed, cost 1). Return edge $\{5, 0\}$ is marked excluded to prevent premature 2-cycle $0 - 5 - 0$.
2. From $[0, 5]$, branching on candidate city 4 yields node $P_2 = [0, 5, 4]$ with fixed path edges $\{0, 5\}$ and $\{5, 4\}$ (accumulated path cost $1 + 2 = 3$):
   - *Degree-2 Exhaustion*: Vertex 5 is now an internal path vertex with its two tour edges committed (incoming from 0, outgoing to 4). Constraint propagation immediately marks all other edges incident to 5 as **excluded**:
   $$E_{\text{excluded}} \leftarrow E_{\text{excluded}} \cup \{\{5, 1\}, \{5, 6\}, \{5, 2\}\}. \tag{3.59}$$
   - *Subtour Prevention*: Because unvisited cities $\{1, 2, 3, 6\}$ remain, edge $\{4, 0\}$ is marked **excluded**.

   When computing the completion lower bound on remaining vertices $\{1, 2, 3, 6\}$ (which must connect to path endpoints 0 and 4), the excluded edges $\{5, 1\}$, $\{5, 6\}$, $\{5, 2\}$, and $\{4, 0\}$ are completely inaccessible. This forces the relaxation to use more expensive detours (such as $\{1, 6\}$ of cost 3), directly driving the conditioned lower bound upward and accelerating bound pruning.

3.4 Search Strategies and Optimality Gap

Regardless of the problem under consideration, every Branch and Bound solver can be conceptualized as an exploration engine operating over a dynamic collection of active, unexpanded subproblems called the *search frontier*, *fringe*, or *open nodes* $\mathcal{F}$.

Algorithm 7 outlines the abstract Branch and Bound algorithm using this open-nodes formulation (standardized on minimization). The algorithm maintains the frontier $\mathcal{F}$ alongside the incumbent solution $\mathbf{x}^*$ and its objective upper bound UB^*. At each iteration, a node n is extracted from $\mathcal{F}$, tested against the current bound, and either fathomed or expanded into child subproblems.

The critical observation is that the exploration order of the search tree is entirely determined by line 6:

$$n \leftarrow \mathcal{F}.\text{remove}(). \tag{3.60}$$

By substituting different data structure implementations for the open-nodes collection $\mathcal{F}$, we obtain fundamentally distinct exploration paradigms with dramatically different performance characteristics.

3.4.1 Depth-First Search (DFS)

Depth-First Search implements $\mathcal{F}$ as a Last-In, First-Out (LIFO) stack. The algorithm immediately expands the child of the most recently visited node, diving down a single branch until a leaf is reached or fathomed.

- **Memory Efficiency**: The maximum number of nodes residing simultaneously on the stack is bounded by $\mathcal{O}(b \cdot d)$, where b is the branching factor ($b = 2$ for binary trees)

Algorithm 7: Generic Branch and Bound Framework with Open Nodes.

Input: Root subproblem P_0; lower-bounding procedure $LB(\cdot)$; branching operator children$(\cdot)$
Output: Globally optimal solution $\mathbf{x}^*$ with optimal cost UB^*

$\mathcal{F} \leftarrow$ OpenNodes();
`// Initialize search frontier with root`
$\mathcal{F}$.add(P_0);
$UB^* \leftarrow \infty$;
`// Incumbent upper bound`
$\mathbf{x}^* \leftarrow$ null;
while $\mathcal{F}$ *is not empty* **do**
  $n \leftarrow \mathcal{F}$.remove();
  `// Node selection dictated by queue policy`
  if n.*lowerBound*() $\geq UB^*$ **then**
    continue;
    `// Pruned by bound (incumbent improved since insertion)`
  if n.*isSolutionCandidate*() **then**
    if n.*objectiveFunction*() $< UB^*$ **then**
      $UB^* \leftarrow n$.objectiveFunction();
      `// Update incumbent`
      $\mathbf{x}^* \leftarrow n$.getSolution();
  else
    for ***each*** *child* $\in n$.*children*() **do**
      if *child*.*lowerBound*() $< UB^*$ **then**
        $\mathcal{F}$.add(child);
        `// Filter and enqueue promising subproblems`
return $\mathbf{x}^*$

and d is the maximum search depth ($d \leq n$). Memory consumption is strictly linear in problem size.

- **Rapid Incumbent Discovery**: By diving directly to leaves, DFS uncovers full feasible solutions early in the search. This quickly raises LB^*, enabling aggressive pruning of subsequent branches.
- **Susceptibility to Poor Branching Decisions**: If the search takes a suboptimal branch near the root of the tree, DFS can spend billions of iterations exhaustively exploring an unproductive subtree before backtracking.

3.4.2 Best-First Search (BFS)

Best-First Search (also called Best-Bound Search) implements $\mathcal{F}$ as a priority queue ordered by the optimistic bound of each node:

$$n^* = \arg\max_{n \in \mathcal{F}} UB(n) \quad (\text{or } \arg\min_{n \in \mathcal{F}} LB(n) \text{ in minimization}). \tag{3.61}$$

The algorithm always expands the globally most promising node in the entire search frontier.

- **Minimal Node Exploration**: Best-First Search is provably optimal in the number of nodes expanded: it never expands any node whose optimistic bound is worse than the optimal objective value z^* [Wol20].
- **Exponential Memory Consumption**: In the worst case, the frontier $\mathcal{F}$ can grow exponentially with tree depth. For large problems, BFS frequently exhausts available RAM long before reaching an optimal leaf, terminating with an out-of-memory failure.
- **Delayed Incumbent Updates**: BFS tends to expand nodes in breadth-like layers across the top of the tree, often discovering its first feasible integer solution very late in the search.

3.4.3 The Optimality Gap

In practical industrial optimization, one rarely has the computational budget to run Branch and Bound to absolute completion. An essential feature of Branch and Bound is that it operates as an *anytime algorithm*: it provides both a valid feasible solution and a mathematical guarantee on how far that solution could possibly be from the global optimum.

Definition 3.8 *Global Dual Bound and Optimality Gap*

Let $\mathcal{F}$ be the set of open nodes in the search frontier at any instant. For a minimization problem with incumbent upper bound $UB^* = c(\mathbf{x}^*)$, the *global lower bound* across the entire unexplored search space is:

$$LB_{\text{global}} = \min_{n \in \mathcal{F}} LB(n). \tag{3.62}$$

The *absolute optimality gap* and *relative optimality gap* are defined respectively as:

$$\text{Gap}_{\text{abs}} = UB^* - LB_{\text{global}}, \qquad \text{Gap}_{\text{rel}} = \frac{UB^* - LB_{\text{global}}}{UB^*}. \tag{3.63}$$

Symmetrically, for a maximization problem with incumbent LB^* and open nodes providing upper bounds:

$$UB_{\text{global}} = \max_{n \in \mathcal{F}} UB(n), \qquad \text{Gap}_{\text{rel}} = \frac{UB_{\text{global}} - LB^*}{LB^*}. \tag{3.64}$$

When $\text{Gap}_{\text{rel}} = 0$, the incumbent is proven to be mathematically optimal. In commercial practice, solvers typically terminate when $\text{Gap}_{\text{rel}} \leq \varepsilon$ (for example, $\varepsilon = 10^{-4}$ or 0.01%), guaranteeing that the discovered solution is within a fraction of a percent of the theoretical optimum.

3.4.4 Hybrid Search: Diving (DFS-BFS)

To combine the lean memory footprint and rapid incumbent discovery of DFS with the global guidance of BFS, state-of-the-art solvers employ *diving heuristics* (hybrid BFS-DFS).

As shown in Algorithm 8, the algorithm extracts the globally best node n from a Best-First priority queue. Instead of returning immediately to the queue after expanding n, the solver performs a greedy depth-first *dive*: at each step, it selects the child with the best local lower bound, pushes all other sibling children into the global priority queue, and continues diving down that single trajectory until reaching a leaf or encountering a node pruned by bound. This discovers high-quality feasible solutions rapidly while maintaining a globally bounded frontier.

Algorithm 8: Hybrid Best-First / Depth-First Diving Strategy.

Input: Initial open nodes priority queue $\mathcal{F}$ containing the root node n_0; incumbent $UB^* \leftarrow \infty$
Output: Optimal or proven ε-optimal solution $\mathbf{x}^*$

```
while F is not empty do
    n ← F.extractMin();
    // Greedy dive from node n until a leaf or pruned state
    while not n.isSolutionCandidate() and n.lowerBound() < UB* do
        children ← n.children();
        if children is empty then
            break;
        Sort children in ascending order of lowerBound();
        n ← children[0];
        // Follow the best child
        for i ← 1 to |children| − 1 do
            F.add(children[i]);
            // Store siblings in global frontier
    if n.isSolutionCandidate() then
        if n.objectiveFunction() < UB* then
            UB* ← n.objectiveFunction();
            x* ← n.getSolution();
return x*
```

3.5 A Generic Java Framework for B&B

We now present a clean, generic, and extensible Java framework for Branch and Bound. The framework cleanly decouples the core search engine from problem-specific state representations and frontier queuing strategies.

3.5.1 Architecture and Core Interfaces

The architecture is structured around three core abstractions:

1. `Node<T>`: Represents an active node in the search tree, encapsulating problem state T, depth, bound computation, and branching logic.
2. `OpenNodes<T>`: Represents the search frontier (fringe) ADT.
3. `BranchAndBound`: The generic solver engine executing the exploration loop.

To standardize across all combinatorial problems, the framework formulates every problem as **minimization**. For maximization problems (such as Knapsack), the implementation maps $\max f(\mathbf{x}) \iff \min(-f(\mathbf{x}))$, returning negated upper bounds: lowerBound() $= -UB$.

Listing 3.1 presents the generic `Node<T>` interface.

```java
import java.util.List;

/**
 * Node interface for Branch and Bound.
 * Formulated for minimization problems.
 * @param <T> Domain state associated with this node.
 */
public interface Node<T> {
    /** True if node is complete, feasible solution. */
    boolean isSolutionCandidate();

    /** Objective value (valid if isSolutionCandidate). */
    double objectiveFunction();

    /** Lower bound on completions of this node. */
    double lowerBound();

    /** Depth in search tree (root has depth 0). */
    int depth();

    /** Generates children of this node via branching. */
    List<Node<T>> children();

    /** Returns domain state associated with this node. */
    T getState();
}
```

Listing 3.1: The generic `Node<T>` interface (`Node.java`).

Listing 3.2 defines the `OpenNodes<T>` interface alongside its standard concrete implementations: Best-First (priority queue), Depth-First (stack), and Depth-First with Best-First tie-breaking.

```java
import java.util.Comparator;
import java.util.PriorityQueue;
import java.util.Stack;

public interface OpenNodes<T> {
    void add(Node<T> n);
    Node<T> remove();
    boolean isEmpty();
    int size();
}

/** Best-First Search: extracts node with minimal bound. */
public class BestFirstOpenNodes<T> implements OpenNodes<T> {
    private final PriorityQueue<Node<T>> queue;

    public BestFirstOpenNodes() {
        queue = new PriorityQueue<>(
```

```
            Comparator.comparingDouble(Node::lowerBound)
        );
    }
    public void add(Node<T> n) { queue.add(n); }
    public Node<T> remove()    { return queue.remove(); }
    public boolean isEmpty()   { return queue.isEmpty(); }
    public int size()          { return queue.size(); }
}

/** Depth-First Search: LIFO exploration using stack. */
public class DepthFirstOpenNodes<T>
        implements OpenNodes<T> {
    private final Stack<Node<T>> stack = new Stack<>();

    public void add(Node<T> n) { stack.push(n); }
    public Node<T> remove()    { return stack.pop(); }
    public boolean isEmpty()   { return stack.isEmpty(); }
    public int size()          { return stack.size(); }
}

/** DFS with Best-First tie-breaking on equal depth. */
public class DepthFirstBestFirstOpenNodes<T>
        implements OpenNodes<T> {
    private final PriorityQueue<Node<T>> queue;

    public DepthFirstBestFirstOpenNodes() {
        queue = new PriorityQueue<>((o1, o2) -> {
            if (o1.depth() != o2.depth()) {
                // deeper first
                return Integer.compare(
                        o2.depth(), o1.depth());
            }
            return Double.compare(
                    o1.lowerBound(), o2.lowerBound());
        });
    }
    public void add(Node<T> n) { queue.add(n); }
    public Node<T> remove()    { return queue.remove(); }
    public boolean isEmpty()   { return queue.isEmpty(); }
    public int size()          { return queue.size(); }
}
```

Listing 3.2: Open nodes interface and concrete search strategies.

Listing 3.3 shows the core `BranchAndBound.minimize` solver method. Notice the direct correspondence with the abstract algorithm presented in Algorithm 7: the polymorphic `OpenNodes<T>` collection instantiates the search frontier $\mathcal{F}$, while the `while` loop executes node selection, candidate evaluation, and child expansion identically.

```
import java.util.function.Consumer;

public class BranchAndBound {
    /**
     * Explores search space using Branch and Bound.
     * @param openNodes Collection containing root node.
     * @param onSolution Callback on improving solution.
     */
    public static <T> void minimize(
            OpenNodes<T> openNodes,
            Consumer<Node<T>> onSolution) {
        double upperBound = Double.MAX_VALUE;
```

```
13
14          while (!openNodes.isEmpty()) {
15              Node<T> n = openNodes.remove();
16
17              if (n.isSolutionCandidate()) {
18                  double objective = n.objectiveFunction();
19                  if (objective < upperBound) {
20                      upperBound = objective;
21                      onSolution.accept(n);
22                  }
23              } else if (n.lowerBound() < upperBound) {
24                  for (Node<T> child : n.children()) {
25                      openNodes.add(child);
26                  }
27              }
28          }
29      }
30  }
```

Listing 3.3: The generic Branch and Bound exploration loop (`BranchAndBound.java`).

3.5.2 Instantiating Knapsack in the Framework

To instantiate the 0/1 Knapsack problem in our framework, we define the `NodeKnapsack` class presented in Listing 3.4.

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class NodeKnapsack implements Node<NodeKnapsack> {
5      final int[] value;
6      final int[] weight;
7      final int selectedValue;
8      final int capaLeft;
9      final int index;
10     final boolean selected;
11     final NodeKnapsack parent;
12     final double ub;
13     final int depth;
14
15     public NodeKnapsack(NodeKnapsack parent,
16                         int[] value, int[] weight,
17                         int selectedValue, int capaLeft,
18                         int index, boolean selected) {
19         this.parent = parent;
20         this.value = value;
21         this.weight = weight;
22         this.selectedValue = selectedValue;
23         this.capaLeft = capaLeft;
24         this.index = index;
25         this.selected = selected;
26         this.depth = (parent == null) ? 0
27                 : parent.depth + 1;
28         this.ub = lpRelaxUBound();
29     }
30
31     /** Continuous LP relaxation on undecided items. */
32     private double lpRelaxUBound() {
33         double valueUb = selectedValue;
34         int c = capaLeft;
```

```
35          for (int i = index + 1; i < value.length; i++) {
36              if (weight[i] <= c) {
37                  valueUb += value[i];
38                  c -= weight[i];
39              } else {
40                  double frac = (double) c / weight[i];
41                  valueUb += frac * value[i];
42                  break;
43              }
44          }
45          return valueUb;
46      }
47
48      @Override
49      public boolean isSolutionCandidate() {
50          return index == value.length - 1;
51      }
52
53      @Override
54      public double objectiveFunction() {
55          return -selectedValue; // negated for min
56      }
57
58      @Override
59      public double lowerBound() {
60          return -ub; // negated upper bound is lower bound
61      }
62
63      @Override
64      public int depth() {
65          return depth;
66      }
67
68      @Override
69      public List<Node<NodeKnapsack>> children() {
70          List<Node<NodeKnapsack>> children =
71                  new ArrayList<>(2);
72          int nextIndex = index + 1;
73
74          // Right child: do not select item at nextIndex
75          children.add(new NodeKnapsack(
76              this, value, weight, selectedValue,
77              capaLeft, nextIndex, false));
78
79          // Left child: select item at nextIndex
80          if (capaLeft >= weight[nextIndex]) {
81              children.add(new NodeKnapsack(
82                  this, value, weight,
83                  selectedValue + value[nextIndex],
84                  capaLeft - weight[nextIndex],
85                  nextIndex, true));
86          }
87          return children;
88      }
89
90      @Override
91      public NodeKnapsack getState() { return this; }
92  }
```

Listing 3.4: Knapsack node implementation (`BranchAndBoundKnapsack.java`).

Executing the solver requires only pre-sorting items by ratio and instantiating the root node:

```
// Pre-sort items in non-increasing ratio order
sortValueOverWeight(value, weight);

OpenNodes<NodeKnapsack> openNodes =
        new BestFirstOpenNodes<>();
NodeKnapsack root = new NodeKnapsack(
        null, value, weight, 0, capacity, -1, false);
openNodes.add(root);

BranchAndBound.minimize(openNodes, solNode -> {
    System.out.println("New best knapsack profit: "
            + (-solNode.lowerBound()));
});
```

3.6 Practical Implementation: TSP in B&B

To synthesize the theoretical and algorithmic foundations developed in this chapter, readers are invited to implement an exact Branch and Bound solver for the Traveling Salesperson Problem by bridging the combinatorial relaxations from Section 3.3 with the generic search engine established in Section 3.5.

The implementation involves three core components:

1. **Search Node State and Branching**: Design a search node class (`NodeTSP`) implementing the generic `Node<NodeTSP>` interface. The node encapsulates the partial decisions accumulated along the branch from the root node (such as the growing partial path in sequential path extension or the sets E_{fixed} and E_{excluded} in edge-based branching; see Section 3.3.3). When generating child nodes, perform the necessary constraint propagation:
    - Exclude all non-tour edges incident to vertices whose degree-2 quota has been reached.
    - Forbid edges that would prematurely close a subtour before all n cities have been visited.
2. **Conditioned Lower-Bounding Procedures**: Implement the lower-bounding procedures developed in Section 3.3:
    - *Cheapest Incident Edges*: A fast baseline bound scanning non-excluded incident edges.
    - *Minimum 1-Tree Relaxation*: Utilizing Kruskal's algorithm and the Union-Find data structure (reviewed in Appendix A) to compute the conditioned 1-tree on $V \setminus \{0\}$ respecting all mandatory and excluded edges.
3. **Search Execution and Benchmarking**: Integrate the search node and bounding routines into the generic `BranchAndBound.minimize` exploration engine. Compare the impact of search strategies (Depth-First Search vs. Best-First Search) and bound quality (cheapest incident vs. 1-tree) on the number of expanded nodes and total CPU time across representative TSP instances.

3.7 Exercises

Exercise 3.1 *Branch and Bound Tree under Variable Reordering*

Consider the following 0/1 knapsack instance:

$$\begin{aligned} \text{maximize} \quad & 25x_0 + 18x_1 + 36x_2 \\ \text{subject to} \quad & 5x_0 + 6x_1 + 4x_2 \leq 11, \\ & x_0, x_1, x_2 \in \{0, 1\}. \end{aligned} \tag{3.65}$$

1. Compute the value-to-weight efficiency ratios r_0, r_1, r_2. Is the given ordering x_0, x_1, x_2 sorted by non-increasing efficiency?
2. Draw the complete Branch and Bound search tree under Depth-First Search with the continuous LP relaxation using the original variable ordering x_0, x_1, x_2.
3. Now sort the variables by non-increasing efficiency: x_2, x_0, x_1. Recompute the Branch and Bound search tree.
4. Compare the number of nodes evaluated and branches pruned under both orderings. Explain why ordering variables by efficiency improves pruning performance.

Exercise 3.2 *Optimality of the Split Item in Continuous Knapsack*

Let $\mathbf{x}^*$ be the optimal solution to the continuous knapsack relaxation with critical item j. Prove that if the capacity C is an integer and all item weights w_i are integers, the fractional variable x_j^* satisfies $x_j^* = 0$ if and only if the optimal solution to the continuous knapsack relaxation is strictly integer feasible.

Exercise 3.3 *Proof of the 1-Tree Cycle Property*

Let $G = (V, E)$ be a connected undirected graph with $|V| = n$. Prove graph-theoretically that any 1-tree on G contains:

1. Exactly n edges.
2. Exactly one simple cycle, and that this cycle necessarily contains the designated vertex 0.

Exercise 3.4 *Edge-Based Branching for the Traveling Salesperson Problem*

In Section 3.6, branching is performed by extending a partial tour vertex by vertex. An alternative branching scheme branches on edges: selecting an undecided edge $e = \{u, v\}$ and generating two children:

- Child 1: Force edge e to be included in the tour ($x_e = 1$).
- Child 2: Forbid edge e from being included in the tour ($x_e = 0$).

1. How should edge e be chosen to maximize pruning efficiency?
2. What logical deductions can be made immediately when $x_e = 1$ is posted?
3. Compare the depth and branching factor of this edge-branching scheme with the vertex-extension scheme.

Exercise 3.5 *Memory Complexity of Best-First Search*

Consider a binary Branch and Bound search tree of uniform depth d. Suppose that at depth $k \leq d$, all nodes have identical upper bounds, forcing Best-First Search to explore the tree in breadth-first layers. Derive the exact number of nodes residing in the priority queue when the search reaches depth k. What does this imply about the scalability of pure Best-First Search on large-scale combinatorial problems?

CHAPTER 4

Linear Programming

Linear programming (LP) is the cornerstone of continuous and discrete optimization. Developed by George Dantzig in 1947 [Dan63] alongside the revolutionary Simplex algorithm, linear programming addresses the problem of optimizing a linear objective function subject to linear equality and inequality constraints. Beyond its immense direct utility in industrial operations, logistics, and telecommunications, linear programming serves as the algorithmic engine of discrete optimization. As established in Chapter 3, relaxing integrality constraints in mixed-integer linear programs produces continuous linear relaxations that yield rigorous dual bounds and prune combinatorial search trees.

In this chapter, we develop the geometric, algebraic, and computational foundations of linear programming. We begin with a concrete motivating case study—the Brewery Problem—and examine its geometric perspective in two and three dimensions. We formalize polyhedral theory, prove the fundamental theorem of vertex optimality, and analyze why naive vertex enumeration fails. We then introduce canonical and standard forms, derive the Simplex tableau mechanics, and provide clean Java implementations. Finally, we explore initialization techniques (Two-Phase and Big-M), duality theory, complexity, total unimodularity, and network flow modeling.

4.1 Foundations of Linear Programming

A linear program optimizes a linear objective function over a continuous domain restricted by linear equations or linear inequalities.

Definition 4.1 *Linear Program*

Let $n \in \mathbb{Z}^+$ denote the number of continuous decision variables, and let $m \in \mathbb{Z}^+$ denote the number of constraints. A *linear program* optimizes a linear function of

$\mathbf{x} = (x_0, x_1, \ldots, x_{n-1})^\top \in \mathbb{R}^n$ subject to linear constraints:

$$\begin{aligned} \text{maximize} \quad & \mathbf{c}^\top \mathbf{x} = \sum_{j=0}^{n-1} c_j x_j \\ \text{subject to} \quad & \sum_{j=0}^{n-1} a_{i,j} x_j \leq b_i, \quad \forall i \in M_{\leq}, \\ & \sum_{j=0}^{n-1} a_{i,j} x_j = b_i, \quad \forall i \in M_{=}, \\ & \sum_{j=0}^{n-1} a_{i,j} x_j \geq b_i, \quad \forall i \in M_{\geq}, \\ & x_j \geq 0, \quad \forall j \in N_{\geq}, \qquad x_j \in \mathbb{R} \text{ (free)}, \quad \forall j \in N_{\text{free}}, \end{aligned} \tag{4.1}$$

where $M_{\leq} \cup M_{=} \cup M_{\geq} = \{0, 1, \ldots, m-1\}$ partitions the constraints and $N_{\geq} \cup N_{\text{free}} = \{0, 1, \ldots, n-1\}$ partitions the decision variables.

In vector-matrix notation, when all constraints are less-than-or-equal-to inequalities and all variables are non-negative, the problem is concisely stated as:

$$\max \left\{ \mathbf{c}^\top \mathbf{x} \;\middle|\; \mathbf{A}\mathbf{x} \leq \mathbf{b},\ \mathbf{x} \geq \mathbf{0} \right\}, \tag{4.2}$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the constraint matrix, $\mathbf{b} \in \mathbb{R}^m$ is the right-hand-side vector, and $\mathbf{c} \in \mathbb{R}^n$ is the objective cost vector.

4.2 A Motivating Case: The Brewery Problem

4.2.1 Problem Formulation

To build geometric and algorithmic intuition, we examine a classic manufacturing optimization model: *The Brewery Problem* [SW11].

A small brewery produces two handcrafted beverages: ale and beer. Production is strictly limited by scarce agricultural resources: corn, hops, and malt. The production recipes and economic yields are defined as follows:

- One barrel of ale requires 5 units of corn, 4 units of hops, and 35 units of malt, generating a profit of 13 monetary units.
- One barrel of beer requires 15 units of corn, 4 units of hops, and 20 units of malt, generating a profit of 23 monetary units.
- Available ingredient supplies are capped at 480 units of corn, 160 units of hops, and 1190 units of malt.

Adhering to our zero-based indexing convention, let $x_0 \geq 0$ denote the number of barrels of ale produced, and let $x_1 \geq 0$ denote the number of barrels of beer produced. The linear

programming formulation is:

$$\begin{aligned} \text{maximize} \quad & Z = 13x_0 + 23x_1 & \\ \text{subject to} \quad & 5x_0 + 15x_1 \leq 480 & \text{(Corn inventory)} \\ & 4x_0 + 4x_1 \leq 160 & \text{(Hops inventory)} \\ & 35x_0 + 20x_1 \leq 1190 & \text{(Malt inventory)} \\ & x_0 \geq 0,\ x_1 \geq 0. & \end{aligned} \tag{4.3}$$

4.2.2 Geometric Perspective in Two Dimensions

Because Equation (4.3) involves only two decision variables (x_0 and x_1), we can visualize its entire mathematical structure in the Cartesian plane, as illustrated in Figure 4.1.

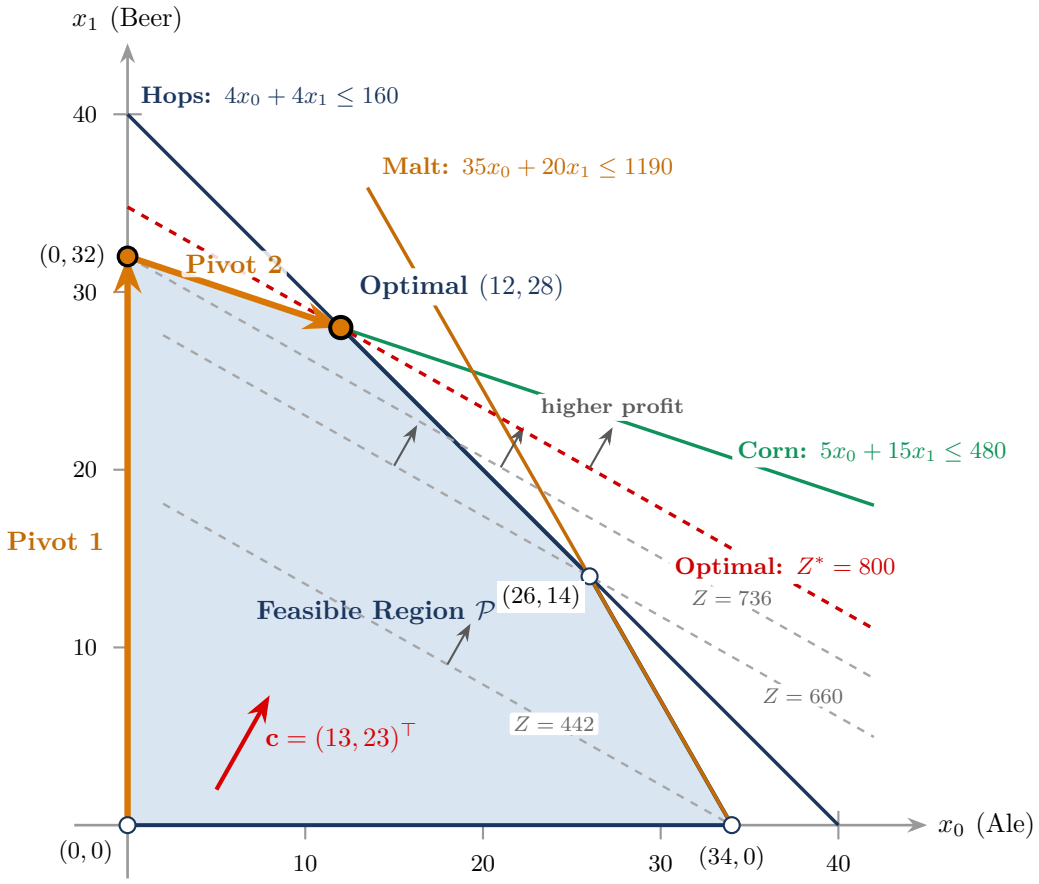


Figure 4.1: Feasible region of the Brewery Problem (Equation (4.3)). The shaded polygon $\mathcal{P}$ is the intersection of five closed halfspaces. Dashed lines depict iso-profit contours at the extreme points ($Z = 442, 660, 736$) up to the optimal contour ($Z^* = 800$), with orthogonal arrows indicating the gradient direction of increasing profit. The Simplex algorithm moves from the origin $(0, 0)$ to $(0, 32)$ along Pivot 1, and then to the optimal vertex $(12, 28)$ along Pivot 2.

Each linear inequality defines a closed *halfspace*:

$$H_i = \left\{ \mathbf{x} \in \mathbb{R}^2 \;\middle|\; \mathbf{a}_i^\top \mathbf{x} \leq b_i \right\}. \tag{4.4}$$

The set of feasible production plans is the intersection of these five halfspaces:

$$\mathcal{P} = \left\{ \mathbf{x} \in \mathbb{R}^2_{\geq 0} \;\middle|\; 5x_0 + 15x_1 \leq 480,\ 4x_0 + 4x_1 \leq 160,\ 35x_0 + 20x_1 \leq 1190 \right\}. \tag{4.5}$$

The boundary of $\mathcal{P}$ is a closed, convex polygon with five corners, called *vertices* or *extreme points*:

- $V_0 = (0, 0)$ with profit $Z = 13(0) + 23(0) = 0$.
- $V_1 = (0, 32)$ with profit $Z = 13(0) + 23(32) = 736$.
- $V_2 = (12, 28)$ with profit $Z = 13(12) + 23(28) = 156 + 644 = 800$.
- $V_3 = (26, 14)$ with profit $Z = 13(26) + 23(14) = 338 + 322 = 660$.
- $V_4 = (34, 0)$ with profit $Z = 13(34) + 23(0) = 442$.

The objective function $Z = 13x_0 + 23x_1$ defines a family of parallel lines with normal vector $\mathbf{c} = (13, 23)^\top$, called *iso-profit contour lines*. As profit increases, the line moves in the direction of the gradient $\mathbf{c}$. The last point of contact between the expanding contour line and the feasible region $\mathcal{P}$ is the vertex $V_2 = (12, 28)$, establishing that the optimal production plan yields $Z^* = 800$.

4.3 Geometry in Higher Dimensions

4.3.1 A Three-Dimensional Polytope Example

The geometric principles observed in two dimensions generalize naturally to three and higher dimensions. Consider the following 3-dimensional linear program with variables $x_0, x_1, x_2 \geq 0$:

$$\begin{aligned} \text{maximize} \quad & Z = x_0 + x_1 \\ \text{subject to} \quad & x_0 + x_1 + x_2 \leq 4 \\ & x_0 \leq 2 \\ & x_2 \leq 3 \\ & 3x_1 + x_2 \leq 6 \\ & x_0, x_1, x_2 \geq 0. \end{aligned} \tag{4.6}$$

As shown in Figure 4.2, each linear inequality defines a halfspace bounded by a 2-dimensional plane in $\mathbb{R}^3$. The intersection of these halfspaces forms a 3-dimensional bounded polytope whose extreme points are:

$$(0, 0, 0),\ (2, 0, 0),\ (2, 2, 0),\ (0, 2, 0),\ (0, 0, 3),\ (1, 0, 3),\ (0, 1, 3),\ (2, 0, 2).$$

For the objective function $\max(x_0 + x_1)$, evaluating the objective across all vertices reveals that the maximum value $Z^* = 4$ is attained at the vertex $\mathbf{x}^* = (2, 2, 0)$. For an alternative objective such as $\max(x_0 + x_1 + x_2)$, the maximum value $Z^* = 4$ is simultaneously attained at multiple vertices: $(2, 2, 0)$, $(1, 0, 3)$, $(0, 1, 3)$, and $(2, 0, 2)$, meaning the entire face connecting these vertices is optimal.

4.3.2 Polyhedral Theory and Convexity

We now formalize the mathematics governing polyhedral geometry in $\mathbb{R}^n$ [BT97].

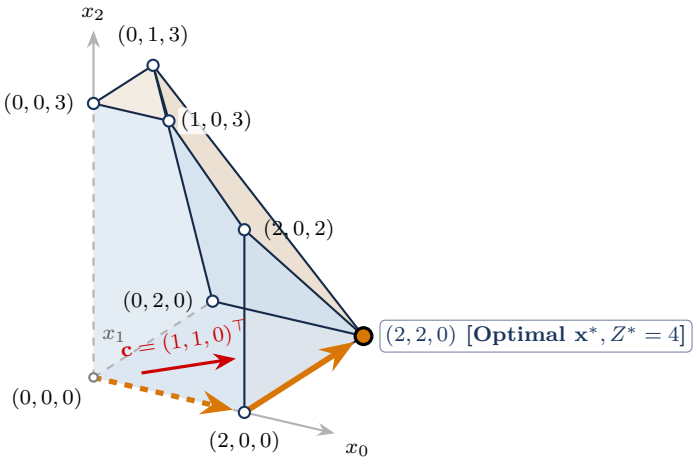


Figure 4.2: Feasible 3D polytope for Equation (4.6). The feasible region is bounded by planar facets. Vertices correspond to basic solutions where three independent planes intersect. Edges connect adjacent vertices, and Simplex walks along improving edges to reach the optimal vertex $\mathbf{x}^* = (2, 2, 0)$.

Definition 4.2 *Convex Set and Convex Combination*

A set $S \subseteq \mathbb{R}^n$ is *convex* if, for any two points $\mathbf{x}, \mathbf{y} \in S$, the entire line segment connecting them belongs to S:

$$\alpha\mathbf{x} + (1 - \alpha)\mathbf{y} \in S, \quad \forall \alpha \in [0, 1]. \tag{4.7}$$

A point $\mathbf{x} \in \mathbb{R}^n$ is a *convex combination* of points $\mathbf{v}^{(0)}, \mathbf{v}^{(1)}, \ldots, \mathbf{v}^{(k-1)}$ if there exist non-negative weights $\alpha_0, \ldots, \alpha_{k-1} \geq 0$ such that $\sum_{i=0}^{k-1} \alpha_i = 1$ and:

$$\mathbf{x} = \sum_{i=0}^{k-1} \alpha_i \mathbf{v}^{(i)}. \tag{4.8}$$

Definition 4.3 *Polyhedron and Polytope*

A *polyhedron* $\mathcal{P} \subseteq \mathbb{R}^n$ is the intersection of a finite number of closed halfspaces:

$$\mathcal{P} = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{A}\mathbf{x} \leq \mathbf{b}\}, \tag{4.9}$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$. A bounded polyhedron is called a *polytope*.

Proposition 4.1 *Convexity of Polyhedra*

Every polyhedron $\mathcal{P} = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{A}\mathbf{x} \leq \mathbf{b}\}$ is convex.

Proof. Let $\mathbf{x}, \mathbf{y} \in \mathcal{P}$ and let $\alpha \in [0, 1]$. By definition, $\mathbf{A}\mathbf{x} \leq \mathbf{b}$ and $\mathbf{A}\mathbf{y} \leq \mathbf{b}$. Consider $\mathbf{z} = \alpha\mathbf{x} + (1 - \alpha)\mathbf{y}$. By linearity of matrix multiplication:

$$\mathbf{A}\mathbf{z} = \mathbf{A}(\alpha\mathbf{x} + (1 - \alpha)\mathbf{y}) = \alpha(\mathbf{A}\mathbf{x}) + (1 - \alpha)(\mathbf{A}\mathbf{y}) \leq \alpha\mathbf{b} + (1 - \alpha)\mathbf{b} = \mathbf{b}. \tag{4.10}$$

Hence, $\mathbf{z} \in \mathcal{P}$, proving that $\mathcal{P}$ is convex. □

4.3.3 Extreme Points and Vertex Optimality

Definition 4.4 *Extreme Point and Vertex*

Let $\mathcal{P} \subseteq \mathbb{R}^n$ be a polyhedron.

1. A point $\mathbf{x} \in \mathcal{P}$ is an *extreme point* if it cannot be expressed as a strict convex combination of two distinct points in $\mathcal{P}$:

$$\mathbf{x} = \alpha\mathbf{y} + (1-\alpha)\mathbf{z}, \quad \mathbf{y}, \mathbf{z} \in \mathcal{P},\ \alpha \in (0,1) \implies \mathbf{x} = \mathbf{y} = \mathbf{z}. \tag{4.11}$$

2. A point $\mathbf{x} \in \mathcal{P}$ is a *vertex* if there exists a cost vector $\mathbf{c} \in \mathbb{R}^n$ such that $\mathbf{x}$ is the unique maximizer of $\mathbf{c}^\top\mathbf{y}$ over all $\mathbf{y} \in \mathcal{P}$:

$$\mathbf{c}^\top\mathbf{x} > \mathbf{c}^\top\mathbf{y}, \quad \forall \mathbf{y} \in \mathcal{P} \setminus \{\mathbf{x}\}. \tag{4.12}$$

Theorem 4.2 *Minkowski-Weyl Representation Theorem*

Every point in a bounded polytope $\mathcal{P} \subset \mathbb{R}^n$ is a convex combination of its finite set of extreme points (vertices) $\{\mathbf{v}^{(0)}, \mathbf{v}^{(1)}, \ldots, \mathbf{v}^{(k-1)}\}$:

$$\forall \mathbf{x} \in \mathcal{P}, \quad \exists \boldsymbol{\alpha} \in \mathbb{R}^k_{\geq 0} \text{ with } \sum_{i=0}^{k-1} \alpha_i = 1 \quad \text{such that} \quad \mathbf{x} = \sum_{i=0}^{k-1} \alpha_i \mathbf{v}^{(i)}. \tag{4.13}$$

Theorem 4.3 *Fundamental Theorem of Linear Programming*

Let $\mathcal{P} \subset \mathbb{R}^n$ be a non-empty bounded polytope. For any linear objective function $\mathbf{c}^\top\mathbf{x}$, at least one optimal solution $\mathbf{x}^*$ is an extreme point (vertex) of $\mathcal{P}$.

Proof. Let $\{\mathbf{v}^{(0)}, \mathbf{v}^{(1)}, \ldots, \mathbf{v}^{(k-1)}\}$ denote the finite collection of extreme points of $\mathcal{P}$. Because $\mathcal{P}$ is compact and the objective function $\mathbf{c}^\top\mathbf{x}$ is continuous, the maximum of $\mathbf{c}^\top\mathbf{x}$ over $\mathcal{P}$ is attained at some point $\mathbf{x}^* \in \mathcal{P}$.

By the Minkowski-Weyl Representation Theorem (Theorem 4.2), $\mathbf{x}^*$ can be represented as a convex combination of the vertices:

$$\mathbf{x}^* = \sum_{i=0}^{k-1} \alpha_i \mathbf{v}^{(i)}, \quad \text{where } \alpha_i \geq 0, \quad \sum_{i=0}^{k-1} \alpha_i = 1. \tag{4.14}$$

Applying the linear objective function $\mathbf{c}^\top$ yields:

$$\mathbf{c}^\top\mathbf{x}^* = \mathbf{c}^\top \left(\sum_{i=0}^{k-1} \alpha_i \mathbf{v}^{(i)} \right) = \sum_{i=0}^{k-1} \alpha_i \left(\mathbf{c}^\top \mathbf{v}^{(i)} \right). \tag{4.15}$$

Let $\mathbf{v}^{(j)}$ be a vertex that achieves the maximum objective value among all vertices: $\mathbf{c}^\top\mathbf{v}^{(j)} = \max_{0 \leq i < k} \mathbf{c}^\top\mathbf{v}^{(i)}$. Then:

$$\mathbf{c}^\top\mathbf{x}^* = \sum_{i=0}^{k-1} \alpha_i \left(\mathbf{c}^\top \mathbf{v}^{(i)} \right) \leq \sum_{i=0}^{k-1} \alpha_i \left(\mathbf{c}^\top \mathbf{v}^{(j)} \right) = \left(\mathbf{c}^\top \mathbf{v}^{(j)} \right) \sum_{i=0}^{k-1} \alpha_i = \mathbf{c}^\top \mathbf{v}^{(j)}. \tag{4.16}$$

Since $\mathbf{v}^{(j)} \in \mathcal{P}$ and $\mathbf{x}^*$ is optimal over all points in $\mathcal{P}$, we also have $\mathbf{c}^\top\mathbf{v}^{(j)} \leq \mathbf{c}^\top\mathbf{x}^*$. Therefore:

$$\mathbf{c}^\top\mathbf{x}^* = \mathbf{c}^\top\mathbf{v}^{(j)}. \tag{4.17}$$

Thus, the vertex $\mathbf{v}^{(j)}$ attains the optimal objective value, proving the theorem. □

4.3.4 Combinatorial Explosion of Vertices

Theorem 4.3 implies that solving a continuous optimization problem over infinitely many feasible points reduces to searching over a finite set of vertices. However, exhaustive vertex enumeration is computationally intractable because the number of vertices grows exponentially with the dimension:

- In 1 dimension, the unit interval $[0, 1]$ has $2 = 2^1$ vertices (defined by 2 inequalities).
- In 2 dimensions, the unit square $[0, 1]^2$ has $4 = 2^2$ vertices (defined by 4 inequalities).
- In 3 dimensions, the unit cube $[0, 1]^3$ has $8 = 2^3$ vertices (defined by 6 inequalities).
- In n dimensions, the unit hypercube $[0, 1]^n$ has 2^n vertices (defined by $2n$ linear inequalities).

For $n = 50$, $2^{50} \approx 1.12 \times 10^{15}$ vertices; evaluating one vertex per microsecond would require more than 35,000 years. For a general system with m constraints and $n + m$ variables, the number of potential vertices can reach $\binom{n+m}{m}$.

This combinatorial explosion demonstrates that brute-force enumeration is impractical. We require an algorithm that navigates through the vertices intelligently: the Simplex algorithm.

4.4 Canonical and Standard Forms

4.4.1 Motivation for Generic Formulations

In two and three dimensions, geometry allows us to visualize halfspaces, draw contour lines, and identify optimal vertices visually. In real-world discrete optimization, however, models routinely involve tens of thousands of variables and constraints. To solve arbitrary linear programs on a computer, we cannot rely on geometric visualization; we must design a generic, purely algebraic algorithm.

Just as the Gauss-Jordan elimination algorithm operates on arbitrary systems of linear equations $\mathbf{Ax} = \mathbf{b}$, an algebraic linear programming solver requires a standardized mathematical formulation. Standardizing the problem allows a single solver engine to process any optimization instance regardless of its original domain-specific structure.

4.4.2 Canonical and Standard Equational Forms

Two canonical representations dominate the optimization literature:

Definition 4.5 *Canonical Form*

A linear program is in *canonical form* if it is a maximization problem where all functional constraints are less-than-or-equal-to inequalities ($\leq$) and all variables are non-negative:

$$\begin{aligned} \text{maximize} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{Ax} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned} \tag{4.18}$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$, and $\mathbf{c} \in \mathbb{R}^n$.

Definition 4.6 *Standard Form*

A linear program is in *standard form* (or equational form) if the objective is maximization, all functional constraints are linear equations (=), and all variables are non-negative:

$$\begin{aligned} \text{maximize} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned} \tag{4.19}$$

where $\mathbf{b} \geq \mathbf{0}$. Standard form is the direct operational format for the Simplex algorithm.

4.4.3 Form Transformations and Slack Variables

Every linear program can be converted into standard form through the following transformations:

1. **Minimization to Maximization**: Negate the cost coefficients:

$$\min \mathbf{d}^\top \mathbf{x} \iff -\max(-\mathbf{d}^\top \mathbf{x}). \tag{4.20}$$

2. **Inequalities to Equalities (Slack Variables)**: For each less-than-or-equal-to constraint $\sum_{j=0}^{n-1} a_{i,j} x_j \leq b_i$, introduce a non-negative *slack variable* $x_{n+i} \geq 0$:

$$\sum_{j=0}^{n-1} a_{i,j} x_j + x_{n+i} = b_i, \quad x_{n+i} \geq 0. \tag{4.21}$$

   If $x_{n+i} = 0$, the constraint holds with strict equality and is said to be *tight* or *active*.

3. **Surplus Variables**: For a greater-than-or-equal-to constraint $\sum_{j=0}^{n-1} a_{i,j} x_j \geq b_i$, subtract a non-negative *surplus variable* $s_i \geq 0$:

$$\sum_{j=0}^{n-1} a_{i,j} x_j - s_i = b_i, \quad s_i \geq 0. \tag{4.22}$$

4. **Equalities to Inequalities**: An equality $\mathbf{a}_i^\top \mathbf{x} = b_i$ can be converted into two inequalities: $\mathbf{a}_i^\top \mathbf{x} \leq b_i$ and $-\mathbf{a}_i^\top \mathbf{x} \leq -b_i$.
5. **Unconstrained (Free) Variables**: A decision variable $x_j \in \mathbb{R}$ with no sign restriction is split into two non-negative variables:

$$x_j = x_j^+ - x_j^-, \quad \text{with } x_j^+ \geq 0,\ x_j^- \geq 0. \tag{4.23}$$

Example 4.1 *Converting the Brewery Problem to Standard Form*

The Brewery Problem (Equation (4.3)) has $n = 2$ original decision variables (x_0, x_1) and $m = 3$ resource constraints. We introduce three slack variables: $x_2 \geq 0$ (unused

corn), $x_3 \geq 0$ (unused hops), and $x_4 \geq 0$ (unused malt):

$$\begin{aligned} \text{maximize} \quad & Z = 13x_0 + 23x_1 + 0x_2 + 0x_3 + 0x_4 \\ \text{subject to} \quad & 5x_0 + 15x_1 + x_2 = 480 \\ & 4x_0 + 4x_1 + x_3 = 160 \\ & 35x_0 + 20x_1 + x_4 = 1190 \\ & x_0, x_1, x_2, x_3, x_4 \geq 0. \end{aligned} \tag{4.24}$$

The resulting system has $n' = n + m = 5$ variables and $m = 3$ equations.

4.5 The Simplex Algorithm

4.5.1 Simplex by Example: The Brewery Problem

To understand how Dantzig's Simplex algorithm systematically navigates the boundary of a polytope, we return to the standard-form Brewery Problem established in Equation (4.24):

$$\begin{aligned} \text{maximize} \quad & Z = 13x_0 + 23x_1 + 0x_2 + 0x_3 + 0x_4 & & \\ \text{subject to} \quad & 5x_0 + 15x_1 + x_2 = 480 & \text{(Corn)} & \\ & 4x_0 + 4x_1 + x_3 = 160 & \text{(Hops)} & \\ & 35x_0 + 20x_1 + x_4 = 1190 & \text{(Malt)} & \\ & x_0, x_1, x_2, x_3, x_4 \geq 0. & & \end{aligned} \tag{4.25}$$

This system comprises $m = 3$ equations and $n' = n + m = 5$ non-negative variables, leaving $5 - 3 = 2$ degrees of freedom. Because there are more variables than equations, the system cannot be uniquely inverted without fixing two variables.

The core insight of the Simplex method is to partition the variables into two complementary sets:

- **Non-basic variables** ($n = 2$ variables): set temporarily to zero.
- **Basic variables** ($m = 3$ variables): uniquely solved in terms of the non-basic variables.

Expressing the basic variables and the objective function Z exclusively in terms of the non-basic variables produces an algebraic representation known as a *dictionary*.

Step 0: Initial Basic Feasible Solution (BFS) We designate the original decision variables as non-basic: $x_0 = 0$ (ale) and $x_1 = 0$ (beer). Isolating the three slack variables $\{x_2, x_3, x_4\}$ as basic variables yields the initial dictionary:

$$x_2 = 480 - 5x_0 - 15x_1, \tag{4.26}$$
$$x_3 = 160 - 4x_0 - 4x_1, \tag{4.27}$$
$$x_4 = 1190 - 35x_0 - 20x_1, \tag{4.28}$$
$$Z = 0 + 13x_0 + 23x_1. \tag{4.29}$$

Setting $x_0 = 0$ and $x_1 = 0$ immediately determines the basic solution:

$$x_2 = 480, \quad x_3 = 160, \quad x_4 = 1190, \quad Z = 0. \tag{4.30}$$

Because all variables are non-negative ($x \geq \mathbf{0}$), this basic solution is feasible—a *basic feasible solution* (BFS). Geometrically, this initial point corresponds to the origin vertex $V_0 = (0, 0)$ in Figure 4.1.

Step 1: First Pivot (Moving from V_0 to V_1) To increase the profit $Z = 13x_0 + 23x_1$, we observe that both non-basic variables have positive coefficients. Increasing either variable improves the objective:

- **Entering Variable Selection**: Producing beer (x_1) increases profit at a rate of \$23 per barrel, while ale (x_0) yields \$13 per barrel. Following Dantzig's largest-coefficient rule, we choose x_1 as the *entering variable* to enter the basis. We hold the other non-basic variable at $x_0 = 0$.
- **Leaving Variable Selection (Minimum Ratio Rule)**: As x_1 increases, the basic slack variables decrease according to Equations (4.26) to (4.28):

$$\begin{aligned} x_2 = 480 - 15x_1 \geq 0 &\implies x_1 \leq \frac{480}{15} = 32, \\ x_3 = 160 - 4x_1 \geq 0 &\implies x_1 \leq \frac{160}{4} = 40, \\ x_4 = 1190 - 20x_1 \geq 0 &\implies x_1 \leq \frac{1190}{20} = 59.5. \end{aligned}$$

  The strictest bound is imposed by the corn constraint ($x_2 \geq 0$): x_1 cannot exceed 32. At $x_1 = 32$, corn is fully exhausted ($x_2 = 0$). Increasing x_1 further would make $x_2 < 0$, violating feasibility. Hence, x_2 is the *leaving variable*: it leaves the basis and becomes non-basic.
- **Dictionary Substitution**: We invert the corn equation to express the new basic variable x_1 in terms of the new non-basic variables (x_0, x_2):

$$15x_1 = 480 - 5x_0 - x_2 \implies x_1 = 32 - \frac{1}{3}x_0 - \frac{1}{15}x_2. \tag{4.31}$$

  Substituting Equation (4.31) into the remaining dictionary equations yields:

$$x_1 = 32 - \frac{1}{3}x_0 - \frac{1}{15}x_2, \tag{4.32}$$

$$x_3 = 160 - 4x_0 - 4\left(32 - \frac{1}{3}x_0 - \frac{1}{15}x_2\right) = 32 - \frac{8}{3}x_0 + \frac{4}{15}x_2, \tag{4.33}$$

$$x_4 = 1190 - 35x_0 - 20\left(32 - \frac{1}{3}x_0 - \frac{1}{15}x_2\right) = 550 - \frac{85}{3}x_0 + \frac{4}{3}x_2, \tag{4.34}$$

$$Z = 13x_0 + 23\left(32 - \frac{1}{3}x_0 - \frac{1}{15}x_2\right) = 736 + \frac{16}{3}x_0 - \frac{23}{15}x_2. \tag{4.35}$$

Setting the non-basic variables $x_0 = 0$ and $x_2 = 0$ produces the new BFS:

$$x_1 = 32, \quad x_3 = 32, \quad x_4 = 550, \quad x_0 = 0, \quad x_2 = 0, \quad Z = 736. \tag{4.36}$$

Geometrically, this pivot traversed the edge connecting $V_0(0, 0)$ to $V_1(0, 32)$ in Figure 4.1, boosting profit from \$0 to \$736.

Step 2: Second Pivot (Moving from V_1 to V_2) We inspect the updated objective formula Equation (4.35):

$$Z = 736 + \frac{16}{3}x_0 - \frac{23}{15}x_2. \tag{4.37}$$

- **Entering Variable Selection**: The coefficient of x_2 is $-23/15 < 0$, confirming that leaving corn idle reduces profit. However, the coefficient of x_0 is $+16/3 > 0$. Even though ale initially appeared less profitable than beer (\$13 vs. \$23), producing ale is

now beneficial because it consumes fewer resources per barrel than beer (only 5 kg of corn vs. 15 kg), allowing resource reallocation. We select x_0 to enter the basis (holding $x_2 = 0$).

- **Leaving Variable Selection**: With $x_2 = 0$, increasing x_0 impacts basic variables according to:

$$x_1 = 32 - \frac{1}{3}x_0 \geq 0 \implies x_0 \leq \frac{32}{1/3} = 96,$$
$$x_3 = 32 - \frac{8}{3}x_0 \geq 0 \implies x_0 \leq \frac{32}{8/3} = 12,$$
$$x_4 = 550 - \frac{85}{3}x_0 \geq 0 \implies x_0 \leq \frac{550}{85/3} \approx 19.41.$$

  The minimum ratio is $\theta^* = 12$, reached by x_3 (hops). At $x_0 = 12$, available hops are fully exhausted ($x_3 = 0$). Thus, x_3 leaves the basis and x_0 enters.

- **Dictionary Substitution**: Isolating x_0 from the hops equation Equation (4.33):

$$\frac{8}{3}x_0 = 32 + \frac{4}{15}x_2 - x_3 \implies x_0 = 12 + \frac{1}{10}x_2 - \frac{3}{8}x_3. \tag{4.38}$$

  Substituting Equation (4.38) into Equations (4.32), (4.34) and (4.35):

$$x_1 = 32 - \frac{1}{3}\left(12 + \frac{1}{10}x_2 - \frac{3}{8}x_3\right) - \frac{1}{15}x_2 = 28 - \frac{1}{10}x_2 + \frac{1}{8}x_3, \tag{4.39}$$
$$x_0 = 12 + \frac{1}{10}x_2 - \frac{3}{8}x_3, \tag{4.40}$$
$$x_4 = 550 - \frac{85}{3}\left(12 + \frac{1}{10}x_2 - \frac{3}{8}x_3\right) + \frac{4}{3}x_2 = 210 - \frac{3}{2}x_2 + \frac{85}{8}x_3, \tag{4.41}$$
$$Z = 736 + \frac{16}{3}\left(12 + \frac{1}{10}x_2 - \frac{3}{8}x_3\right) - \frac{23}{15}x_2 = 800 - x_2 - 2x_3. \tag{4.42}$$

Setting the non-basic variables $x_2 = 0$ and $x_3 = 0$ produces the new BFS:

$$x_0 = 12, \quad x_1 = 28, \quad x_4 = 210, \quad x_2 = 0, \quad x_3 = 0, \quad Z = 800. \tag{4.43}$$

Geometrically, this pivot traversed the edge from $V_1(0, 32)$ to $V_2(12, 28)$ in Figure 4.1.

Step 3: Stopping Criterion and Optimality Certificate Consider the final objective expression Equation (4.42):

$$Z = 800 - x_2 - 2x_3. \tag{4.44}$$

Every feasible production plan must satisfy non-negativity: $x_2 \geq 0$ and $x_3 \geq 0$. Because the coefficients of both non-basic variables are strictly negative (-1 and -2), any feasible point satisfies:

$$-x_2 \leq 0 \quad \text{and} \quad -2x_3 \leq 0 \implies Z = 800 - x_2 - 2x_3 \leq 800. \tag{4.45}$$

This inequality holds for *all* feasible points in the entire polytope $\mathcal{P}$. Because our current basic solution achieves $Z = 800$, no feasible plan can possibly surpass it. The dictionary thus provides an irrefutable, self-verifying **certificate of global optimality**. Pivoting terminates with the optimal solution:

$$x_0^* = 12 \text{ (ale)}, \quad x_1^* = 28 \text{ (beer)}, \quad x_4^* = 210 \text{ (unused malt)}, \quad x_2^* = x_3^* = 0, \tag{4.46}$$

yielding maximal profit $Z^* = 800$.

Step 4: Simplex Tableaux Representation Carrying out symbolic variable substitutions by hand is tedious. The numerical coefficients can be recorded compactly in a 2D matrix called a *simplex tableau*. By rewriting the objective equation as:

$$Z - 13x_0 - 23x_1 + 0x_2 + 0x_3 + 0x_4 = 0, \tag{4.47}$$

the pivots correspond directly to elementary Gauss-Jordan row operations.

Initial Tableau (T_0): Basis $\{x_2, x_3, x_4\}$, $Z = 0$:

$$T_0 = \left[\begin{array}{ccccc|c} x_0 & x_1 & x_2 & x_3 & x_4 & \text{RHS} \\ \hline 5 & \mathbf{15} & 1 & 0 & 0 & 480 \\ 4 & 4 & 0 & 1 & 0 & 160 \\ 35 & 20 & 0 & 0 & 1 & 1190 \\ \hline 13 & 23 & 0 & 0 & 0 & 0 \end{array}\right] \tag{4.48}$$

- Entering column: $q = 1$ (x_1, beer), since $a[3][1] = 23 > 0$.
- Minimum ratio test: Row 0: $480/15 = 32$; Row 1: $160/4 = 40$; Row 2: $1190/20 = 59.5$. Minimum ratio is $\theta^* = 32$ at row $p = 0$. Pivot element is $a[0][1] = \mathbf{15}$.

Tableau 1 (T_1): Basis $\{x_1, x_3, x_4\}$, $Z = 736$:

$$T_1 = \left[\begin{array}{ccccc|c} x_0 & x_1 & x_2 & x_3 & x_4 & \text{RHS} \\ \hline 1/3 & 1 & 1/15 & 0 & 0 & 32 \\ \mathbf{8/3} & 0 & -4/15 & 1 & 0 & 32 \\ 85/3 & 0 & -4/3 & 0 & 1 & 550 \\ \hline 16/3 & 0 & -23/15 & 0 & 0 & -736 \end{array}\right] \tag{4.49}$$

- Entering column: $q = 0$ (x_0, ale), since $a[3][0] = 16/3 > 0$.
- Minimum ratio test: Row 0: $32/(1/3) = 96$; Row 1: $32/(8/3) = 12$; Row 2: $550/(85/3) \approx 19.41$. Minimum ratio is $\theta^* = 12$ at row $p = 1$. Pivot element is $a[1][0] = \mathbf{8/3}$.

Tableau 2 (T_2): Basis $\{x_1, x_0, x_4\}$, $Z = 800$:

$$T_2 = \left[\begin{array}{ccccc|c} x_0 & x_1 & x_2 & x_3 & x_4 & \text{RHS} \\ \hline 0 & 1 & 1/10 & -1/8 & 0 & 28 \\ 1 & 0 & -1/10 & 3/8 & 0 & 12 \\ 0 & 0 & 3/2 & -85/8 & 1 & 210 \\ \hline 0 & 0 & -1 & -2 & 0 & -800 \end{array}\right] \tag{4.50}$$

All reduced costs in row 3 are non-positive ($\bar{\mathbf{c}} = (0, 0, -1, -2, 0) \leq \mathbf{0}$). Optimality is reached at $Z^* = 800$. Notice how this algebraic sequence exactly mirrors the vertex trajectory $V_0(0,0) \to V_1(0,32) \to V_2(12,28)$ identified geometrically in Figure 4.1.

4.5.2 Formalization of the Simplex Method

Having witnessed the mechanics of the Simplex algorithm on the Brewery Problem, we now establish its formal mathematical theory for an arbitrary linear program.

Bases and Matrix Partitions

Consider a standard-form linear program with m equality constraints and $n' = n + m$ non-negative variables:

$$\text{maximize} \quad \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{A}\mathbf{x} = \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0}, \tag{4.51}$$

where $\mathbf{A} \in \mathbb{R}^{m \times n'}$ has full row rank $\operatorname{rank}(\mathbf{A}) = m \leq n'$. We partition the columns of $\mathbf{A}$ into an $m \times m$ invertible basis matrix $\mathbf{A}_B$ and an $m \times (n' - m)$ non-basis matrix $\mathbf{A}_N$:

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_B & \mathbf{A}_N \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} \mathbf{x}_B \\ \mathbf{x}_N \end{bmatrix}, \qquad \mathbf{c} = \begin{bmatrix} \mathbf{c}_B \\ \mathbf{c}_N \end{bmatrix}. \tag{4.52}$$

The constraint equation $\mathbf{A}_B\mathbf{x}_B + \mathbf{A}_N\mathbf{x}_N = \mathbf{b}$ determines the basic variables $\mathbf{x}_B$ as a linear function of the non-basic variables $\mathbf{x}_N$:

$$\mathbf{x}_B = \mathbf{A}_B^{-1}\mathbf{b} - \mathbf{A}_B^{-1}\mathbf{A}_N\mathbf{x}_N. \tag{4.53}$$

A *basic solution* sets all non-basic variables to zero: $\mathbf{x}_N = \mathbf{0}$, yielding $\mathbf{x}_B = \mathbf{A}_B^{-1}\mathbf{b}$. If $\mathbf{x}_B \geq \mathbf{0}$, the solution is a *basic feasible solution* (BFS), which corresponds algebraically to an extreme point of the polytope $\mathcal{P} = \{\mathbf{x} \geq \mathbf{0} \mid \mathbf{A}\mathbf{x} = \mathbf{b}\}$.

Substituting Equation (4.53) into the objective function yields:

$$\begin{aligned} Z &= \mathbf{c}_B^\top \mathbf{x}_B + \mathbf{c}_N^\top \mathbf{x}_N \\ &= \mathbf{c}_B^\top \left(\mathbf{A}_B^{-1}\mathbf{b} - \mathbf{A}_B^{-1}\mathbf{A}_N\mathbf{x}_N \right) + \mathbf{c}_N^\top \mathbf{x}_N \\ &= \underbrace{\mathbf{c}_B^\top \mathbf{A}_B^{-1}\mathbf{b}}_{Z_0} + \underbrace{\left(\mathbf{c}_N^\top - \mathbf{c}_B^\top \mathbf{A}_B^{-1}\mathbf{A}_N \right)}_{\bar{\mathbf{c}}_N^\top} \mathbf{x}_N. \end{aligned} \tag{4.54}$$

The scalar Z_0 represents the objective value at the current basic solution, and $\bar{\mathbf{c}}_N^\top$ is the vector of *reduced costs*. The reduced cost $\bar{c}_j$ quantifies the net rate of increase in Z per unit increase in the non-basic variable x_j.

The complete equational system is stored in a 2D tableau `a[i][j]` of dimensions $(m + 1) \times (n + m + 1)$:

Basis	$\mathbf{x}_N$	$\mathbf{x}_B$	RHS
$\mathbf{x}_B$	$\mathbf{A}_B^{-1}\mathbf{A}_N$	$\mathbf{I}_m$	$\mathbf{A}_B^{-1}\mathbf{b}$
Obj	$-(\mathbf{c}_N^\top - \mathbf{c}_B^\top \mathbf{A}_B^{-1}\mathbf{A}_N)$	$\mathbf{0}^\top$	$-Z_0$

(4.55)

In array coordinates:

- Rows $0 \leq i < m$ store the constraint equations transformed by $\mathbf{A}_B^{-1}$.
- Row m stores the objective row (reduced costs in columns $0 \leq j < n + m$, and $-Z_0$ in column $n + m$).
- Column $n + m$ holds the current right-hand-side values $\mathbf{A}_B^{-1}\mathbf{b}$.

Pivoting and the Minimum Ratio Rule

A single iteration of the Simplex method executes a *pivot*, which swaps one non-basic variable into the basis (the *entering variable*) and one basic variable out (the *leaving variable*).

Step 1: Selecting the Entering Variable To increase the objective function in a maximization problem, we must pick a non-basic column q with a positive reduced cost: $a[m][q] > 0$.

- **Dantzig's Rule**: Choose $q = \arg\max_j a[m][j]$. This maximizes the immediate per-unit rate of ascent.
- **Bland's Smallest-Subscript Rule**: Choose $q = \min\{j \mid a[m][j] > 0\}$. This rule provably prevents cycling (Section 4.8).

If $a[m][j] \leq 0$ for all columns $j \in \{0, \ldots, n + m - 1\}$, the current basic feasible solution is optimal.

Step 2: Selecting the Leaving Variable (Minimum Ratio Rule) Increasing x_q causes the basic variables to adjust according to $x_{B(i)} = \bar{b}_i - \bar{a}_{i,q}x_q$. To maintain non-negativity $x_{B(i)} \geq 0$, x_q can increase only until the first basic variable drops to zero:

$$\theta^* = \min_{0 \leq i < m \,:\, a[i][q] > 0} \frac{a[i][n+m]}{a[i][q]}. \tag{4.56}$$

The row p that attains this minimum ratio is selected as the pivot row:

$$p = \arg\min_{0 \leq i < m \,:\, a[i][q] > 0} \frac{a[i][n+m]}{a[i][q]}. \tag{4.57}$$

If $a[i][q] \leq 0$ for all rows $i \in \{0, \ldots, m-1\}$, x_q can be increased indefinitely without violating any constraint. The linear program is *unbounded* ($Z^* = +\infty$).

Step 3: Gauss-Jordan Elimination The tableau is updated using elementary row operations:

$$\begin{aligned} a'[i][j] &= a[i][j] - a[p][j] \cdot \frac{a[i][q]}{a[p][q]}, && \forall i \neq p,\ \forall j, \\ a'[p][j] &= \frac{a[p][j]}{a[p][q]}, && \forall j. \end{aligned} \tag{4.58}$$

Optimality Certificate in the Tableau

Theorem 4.4 *Simplex Optimality Condition*

If every reduced cost in row m of the simplex tableau is non-positive:

$$\bar{c}_j = a[m][j] \leq 0, \quad \forall j \in \{0, \ldots, n+m-1\}, \tag{4.59}$$

then the current basic feasible solution is globally optimal.

Proof. For any feasible solution $\mathbf{x} \geq \mathbf{0}$, Equation (4.54) states:

$$Z(\mathbf{x}) = Z_0 + \sum_{j \in N} \bar{c}_j x_j. \tag{4.60}$$

Because $x_j \geq 0$ and $\bar{c}_j \leq 0$ for every $j \in N$, each term $\bar{c}_j x_j \leq 0$. Hence:

$$Z(\mathbf{x}) = Z_0 + \sum_{j \in N} \underbrace{\bar{c}_j}_{\leq 0} \underbrace{x_j}_{\geq 0} \leq Z_0. \tag{4.61}$$

Since the current basic solution achieves $Z = Z_0$, no feasible solution can yield a larger objective value. □

4.5.3 Algorithmic Implementation in Java

Tableau Representation and Memory Layout As illustrated in Figure 4.3, the tableau is stored as a single two-dimensional array `a` of dimensions $(m+1) \times (n+m+1)$. This layout directly mirrors the partitioned block dictionary of Equation (4.55):

- **Constraint Matrix A** $(m \times n)$: Rows $0 \leq i < m$ and columns $0 \leq j < n$ hold the original constraint coefficients $A_{i,j}$.
- **Slack Variables** $\mathbf{I}_m$ $(m \times m)$: Rows $0 \leq i < m$ and columns $n \leq j < n+m$ contain an identity matrix. Setting the diagonal entries $\texttt{a}[i][n+i] = 1.0$ introduces m non-negative slack variables $x_n, \ldots, x_{n+m-1}$.
- **Right-Hand Side b** $(m \times 1)$: Column $n+m$ stores the resource limits b_i for each constraint row $0 \leq i < m$.
- **Objective Row** $\mathbf{c}^\top$ $(1 \times (n+m+1))$: Row m stores the objective function coefficients. Columns $0 \leq j < n$ hold the costs c_j (the initial reduced costs). Columns $n \leq j < n+m$ represent zero costs for slack variables, and column $n+m$ stores the negated current objective value $-Z_0 = 0.0$.
- **Basis Tracker `basis`**: An integer array of size m tracking which variable is currently basic in each constraint row i. Because all $b_i \geq 0$, the slacks form the initial basic feasible solution: $\texttt{basis}[i] = n+i$.

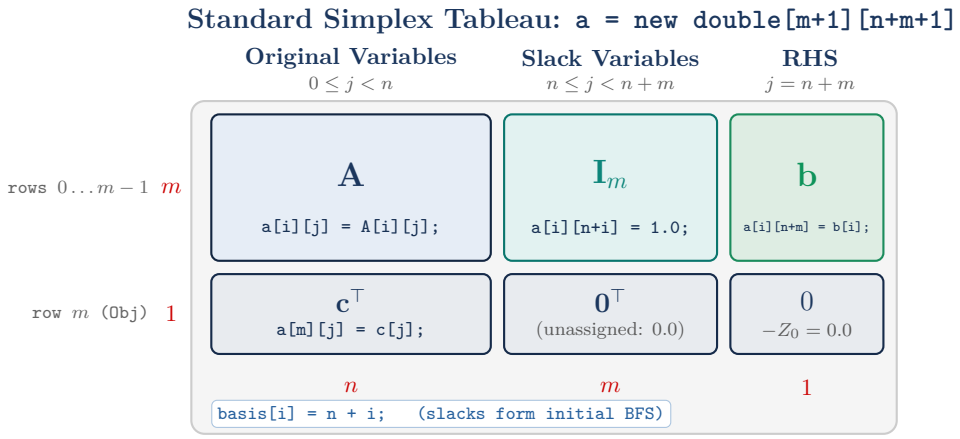


Figure 4.3: Standard Simplex tableau 2D array memory layout and Java initialization (`LinearProgramming.java`) storing m constraints and 1 objective row across n original variables, m slacks, and 1 RHS column in an array of size $(m+1) \times (n+m+1)$.

Listing 4.1 presents the complete Java implementation of the Simplex solver.

```java
public class LinearProgramming {
    private static final double EPSILON = 1.0E-10;
    private double[][] a;   // (m+1) x (n+m+1) tableau
    private int m, n;
    private int[] basis;    // basis[i] = basic var for row i

    public LinearProgramming(double[][] A, double[] b,
                             double[] c) {
        m = b.length; n = c.length;
        for (int i = 0; i < m; i++)
            if (b[i] < 0) throw new IllegalArgumentException("RHS < 0");

        a = new double[m + 1][n + m + 1];
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++) a[i][j] = A[i][j];

        basis = new int[m];
        for (int i = 0; i < m; i++) {
            a[i][n + i] = 1.0;
            a[i][m + n] = b[i];
            basis[i] = n + i;
        }
        for (int j = 0; j < n; j++) a[m][j] = c[j];

        solve();
    }

    private void solve() {
        while (true) {
            int q = bland();
            if (q == -1) break; // optimal
            int p = minRatioRule(q);
            if (p == -1) throw new ArithmeticException("LP is unbounded");
            pivot(p, q);
            basis[p] = q;
        }
    }

    // Bland: smallest index with positive reduced cost
    private int bland() {
        for (int j = 0; j < m + n; j++)
            if (a[m][j] > EPSILON) return j;
        return -1;
    }

    // Dantzig: largest positive reduced cost
    private int dantzig() {
        int q = -1;
        double maxCost = EPSILON;
        for (int j = 0; j < m + n; j++)
            if (a[m][j] > maxCost) { maxCost = a[m][j]; q = j; }
        return q;
    }
```

```
54
55      private int minRatioRule(int q) {
56          int p = -1;
57          for (int i = 0; i < m; i++) {
58              if (a[i][q] <= EPSILON) continue;
59              if (p == -1 ||
60                  a[i][m+n] / a[i][q] < a[p][m+n] / a[p][q]) p = i;
61          }
62          return p;
63      }
64
65      private void pivot(int p, int q) {
66          for (int i = 0; i <= m; i++)
67              for (int j = 0; j <= m + n; j++)
68                  if (i != p && j != q)
69                      a[i][j] -= a[p][j] * (a[i][q] / a[p][q]);
70          for (int i = 0; i <= m; i++)
71              if (i != p) a[i][q] = 0.0;
72          for (int j = 0; j <= m + n; j++)
73              if (j != q) a[p][j] /= a[p][q];
74          a[p][q] = 1.0;
75      }
76
77      public double value() { return -a[m][m + n]; }
78
79      public double[] primal() {
80          double[] x = new double[n];
81          for (int i = 0; i < m; i++)
82              if (basis[i] < n) x[basis[i]] = a[i][m + n];
83          return x;
84      }
85
86      public double[] dual() {
87          double[] y = new double[m];
88          for (int i = 0; i < m; i++) y[i] = -a[m][n + i];
89          return y;
90      }
91
92      public double[][] tableau() { return a; }
93      public int[] basis() { return basis; }
94  }
```

Listing 4.1: Self-contained Simplex solver using Bland's rule.

Pivot Selection Strategies Method `bland()` implements Bland's smallest-subscript rule, scanning row m from left to right and returning the first index j satisfying `a[m][j] > EPSILON`. Alternatively, method `dantzig()` selects the column with the maximum positive reduced cost. The threshold `EPSILON` $= 10^{-10}$ protects against numerical inaccuracies: a tiny residual such as $+10^{-16}$ caused by roundoff error will not trigger an extraneous pivot.

Minimum Ratio Test and Unboundedness Method `minRatioRule(q)` determines the leaving row p by computing the ratio `a[i][m+n]`/`a[i][q]` across all rows where `a[i][q]` > `EPSILON`. Non-positive entries are strictly ignored because they do not constrain an increase in x_q. If all entries in column q are non-positive, no row constrains the growth of x_q; `p` remains -1, signaling that the linear program is unbounded and triggering an exception.

In-Place Gauss-Jordan Pivoting Method `pivot(p, q)` updates the entire tableau in $\mathcal{O}(m(n+m))$ operations:

1. For all rows $i \neq p$ and columns $j \neq q$, the entry is updated by subtracting the scaled pivot row: $a[i][j] \leftarrow a[i][j] - a[p][j] \cdot \frac{a[i][q]}{a[p][q]}$.
2. Column q is zeroed out for all rows $i \neq p$.
3. The pivot row p is normalized by dividing each entry by $a[p][q]$, setting $a[p][q] = 1$.
4. Finally, `basis[p]` is updated to record that variable q is now basic for row p.

Extracting Primal and Dual Solutions Upon termination:

- `value()` returns the optimal objective value $-a[m][n+m]$.
- `primal()` extracts the primal decision variables $x_0, \ldots, x_{n-1}$. For each row i, if `basis[i]` $< n$, the corresponding variable is assigned the RHS value $a[i][n+m]$; all non-basic variables remain zero.
- `dual()` extracts the optimal dual variables $y_i = -a[m][n+i]$ directly from the reduced costs of the slack variables, a property formalized by Linear Programming Duality in Section 4.7.

4.6 Initialization: Two-Phase and Big-M

4.6.1 The Initial Basis Challenge

In the Brewery Problem, finding an initial basic feasible solution was immediate because $\mathbf{b} \geq \mathbf{0}$, allowing slack variables to serve as an initial identity basis.

However, when constraints have negative right-hand sides ($b_i < 0$), setting original variables to zero violates non-negativity. Recall that canonical form (Definition 4.5) expresses all functional constraints as less-than-or-equal-to inequalities ($\leq$). For instance, if an application requires $2x_0 - x_1 \geq 2$, multiplying by -1 transforms it into the canonical $\leq$ inequality $-2x_0 + x_1 \leq -2$. Consider:

$$\begin{aligned} \text{maximize} \quad & Z = 2x_0 - x_1 && (4.62)\\ \text{subject to} \quad & -2x_0 + x_1 \leq -2, && (4.63)\\ & x_0 - 5x_1 \leq -4, && (4.64)\\ & x_0, x_1 \geq 0. && (4.65) \end{aligned}$$

In canonical form $\mathbf{Ax} \leq \mathbf{b}$, the right-hand side vector $\mathbf{b} = (-2, -4)^\top$ has negative entries for both constraint (4.63) ($b_0 = -2 < 0$) and constraint (4.64) ($b_1 = -4 < 0$). At the origin $(x_0, x_1) = (0, 0)$, both constraints are violated: $0 \not\leq -2$ and $0 \not\leq -4$. Introducing standard slack variables $s_0, s_1 \geq 0$ yields $-2x_0 + x_1 + s_0 = -2$ and $x_0 - 5x_1 + s_1 = -4$; setting $x_0 = x_1 = 0$ requires $s_0 = -2 < 0$ and $s_1 = -4 < 0$, violating non-negativity.

4.6.2 The Two-Phase Simplex Method

Geometric Intuition and Principles

The Two-Phase Simplex method resolves the initial basis challenge through a two-stage geometric strategy, illustrated conceptually in Figure 4.4:

1. **Phase I (Feasibility via Embedding)**: When the origin $\mathbf{x} = \mathbf{0}$ lies outside the feasible polytope $\mathcal{P}$, the standard identity basis formed by slack variables is infeasible because it violates non-negativity. Phase I overcomes this by embedding the n-dimensional space into an augmented $(n + m)$-dimensional space. For each constraint i, we introduce a non-negative *artificial variable* $x_{a,i} \geq 0$, which acts as an extra coordinate measuring the degree of constraint violation at the origin.

   Setting each artificial variable equal to the non-negative right-hand side ($x_{a,i} = b_i \geq 0$) provides a trivial, valid basic feasible solution in this higher-dimensional artificial polytope. We then optimize an auxiliary Phase I objective that penalizes the sum of artificial violations:

$$\text{maximize} \quad W = -\sum_{i=0}^{m-1} x_{a,i} \quad \Longleftrightarrow \quad \text{minimize} \quad \sum_{i=0}^{m-1} x_{a,i}. \tag{4.66}$$

   As depicted by the dashed green trajectory in Figure 4.4, the Simplex algorithm pivots along the edges of the artificial polytope, systematically contracting the artificial dimensions toward zero.

   - If $\max W = 0$, every artificial variable has been driven to zero ($x_{a,i} = 0$ for all i). At this exact instant, the algorithm has eliminated the artificial violation and "landed" directly on an extreme point $\mathbf{x}^{(0)}$ on the boundary of the original feasible polytope $\mathcal{P}$. The active basic variables constitute a certified basic feasible solution for the original problem.
   - If $\max W < 0$, at least one artificial variable remains strictly positive at optimality ($\sum x_{a,i} > 0$). This provides an irrefutable certificate that no combination of original variables can satisfy all constraints simultaneously: the original linear program is **infeasible**.

2. **Phase II (Optimality on the Original Polytope)**: Once Phase I delivers a valid vertex $\mathbf{x}^{(0)} \in \mathcal{P}$, the artificial scaffolding is discarded. We drop the auxiliary objective row W and the artificial variable columns, reinstate the true objective function $\mathbf{c}^\top \mathbf{x}$, and express it in terms of the current non-basic variables. Phase II then resumes standard Simplex pivoting, traversing adjacent vertices along improving edges of $\mathcal{P}$ (illustrated by the solid orange trajectory $\mathbf{x}^{(0)} \to \mathbf{x}^{(1)} \to \cdots \to \mathbf{x}^*$ in Figure 4.4) until an optimal vertex is certified or an unbounded ray is identified.

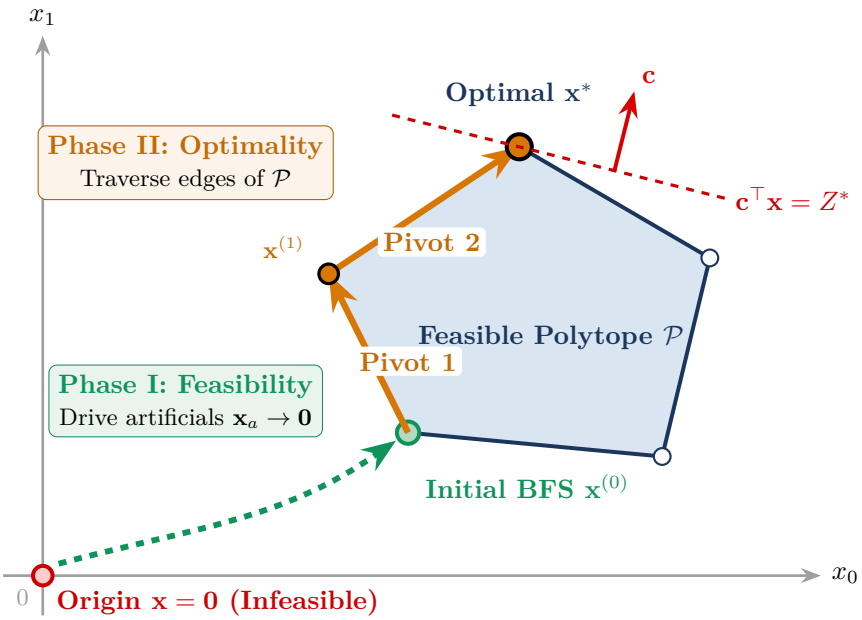


Figure 4.4: Two-Phase Simplex geometry. Phase I introduces artificial variables to create an artificial feasible space, driving them to zero along the dashed green trajectory to reach an extreme point $\mathbf{x}^{(0)}$ of the original polytope $\mathcal{P}$. Phase II then traverses the edges of $\mathcal{P}$ (solid orange path) to the optimal vertex $\mathbf{x}^*$.

Step-by-Step Worked Example

To illustrate both the algebraic transformations and tableau mechanics of the Two-Phase method, we trace the solution of the canonical-form linear program from Equation (4.62):

$$\begin{aligned} \text{maximize} \quad & Z = 2x_0 - x_1 && (4.67)\\ \text{subject to} \quad & -2x_0 + x_1 \leq -2, && (4.68)\\ & x_0 - 5x_1 \leq -4, && (4.69)\\ & x_0, x_1 \geq 0. && (4.70) \end{aligned}$$

Step 1: Adding Slacks and Row Sign Normalization The problem is in canonical form $\mathbf{Ax} \leq \mathbf{b}$ with constraint matrix $\mathbf{A} = \begin{pmatrix} -2 & 1 \\ 1 & -5 \end{pmatrix}$ and right-hand side vector $\mathbf{b} = \begin{pmatrix} -2 \\ -4 \end{pmatrix}$.

Converting constraints (4.68) and (4.69) into standard equality form by adding non-negative slack variables $s_0, s_1 \geq 0$ yields:

$$\begin{aligned} -2x_0 + x_1 + s_0 &= -2, && (4.71)\\ x_0 - 5x_1 + s_1 &= -4. && (4.72) \end{aligned}$$

Setting $x_0 = 0$ and $x_1 = 0$ assigns basic values $s_0 = -2$ and $s_1 = -4$. Both violate the non-negativity requirement ($s_0 \geq 0, s_1 \geq 0$); thus, the standard slack variables cannot serve as an initial basic feasible solution.

To restore non-negative right-hand sides, we multiply any constraint row where $b_i < 0$ by -1. Because both $b_0 = -2 < 0$ in Equation (4.71) and $b_1 = -4 < 0$ in Equation (4.72) are strictly negative, multiplying both equations by -1 normalizes the constants to $+2$ and $+4$:

$$\begin{aligned} 2x_0 - x_1 - s_0 &= 2, && (4.73)\\ -x_0 + 5x_1 - s_1 &= 4. && (4.74) \end{aligned}$$

This row-sign normalization matches the logic of `TwoPhaseSimplex.java`, where row i is negated whenever $b_i < 0$. In Equations (4.73) and (4.74), the slack variables now appear with coefficient -1.0 (functioning as surplus variables).

Step 2: Injecting Artificial Variables and Phase I Objective Because the surplus variables in Equations (4.73) and (4.74) have coefficients -1.0, they cannot provide an initial identity basis (setting non-basic variables to 0 would yield $s_0 = -2 < 0, s_1 = -4 < 0$). We therefore introduce non-negative artificial variables $a_0, a_1 \geq 0$ with coefficients $+1.0$:

$$\begin{aligned} 2x_0 - x_1 - s_0 + a_0 &= 2, && (4.75)\\ -x_0 + 5x_1 - s_1 + a_1 &= 4. && (4.76) \end{aligned}$$

Setting $x_0 = x_1 = s_0 = s_1 = 0$ gives $a_0 = 2 \geq 0$ and $a_1 = 4 \geq 0$, establishing an initial basic feasible solution in the augmented space with basis $\{a_0, a_1\}$.

The Phase I objective maximizes $W = -a_0 - a_1$. Expressing a_0 and a_1 from Equations (4.75) and (4.76):

$$a_0 = 2 - 2x_0 + x_1 + s_0, \qquad a_1 = 4 + x_0 - 5x_1 + s_1,$$

which yields the Phase I objective in non-basic variables:

$$W = -(2 - 2x_0 + x_1 + s_0) - (4 + x_0 - 5x_1 + s_1) = -6 + x_0 + 4x_1 - s_0 - s_1. \quad (4.77)$$

In dictionary form, Equation (4.77) directly provides the reduced costs for row Ph1 ($\bar{w}_{x_0} = 1$, $\bar{w}_{x_1} = 4$, $\bar{w}_{s_0} = -1$, $\bar{w}_{s_1} = -1$), with initial baseline value $W_0 = -6$.

Step 3: Initial Canonicalized Tableau Following the memory layout of Figure 4.5, the tableau has $m+2=4$ rows and $n+2m+1=7$ columns. Row $m=2$ stores the original objective ($Z=2x_0-x_1$), while row $m+1=3$ stores the Phase I objective (Equation (4.77)):

$$T_0 = \begin{array}{c|cccccc|c} \text{Basis} & x_0 & x_1 & s_0 & s_1 & a_0 & a_1 & \text{RHS} \\ \hline a_0 & 2 & -1 & -1 & 0 & 1 & 0 & 2 \\ a_1 & -1 & 5 & 0 & -1 & 0 & 1 & 4 \\ \hline \text{Obj } (Z) & 2 & -1 & 0 & 0 & 0 & 0 & 0 \\ \text{Ph1 } (W) & 1 & 4 & -1 & -1 & 0 & 0 & 6 \end{array} \tag{4.78}$$

The right-hand side of row Ph1 stores the negated value $-W=6$, corresponding to $W=-6$.

Step 4: Phase I Pivoting

- **Pivot 1**: Scanning row Ph1 using Bland's rule, the first column with a positive reduced cost is column $q=0$ (x_0, with $\bar{w}_0=1>0$). We apply the minimum ratio rule among rows where $a[i][0]>0$:

$$\text{Row 0 } (a_0): \frac{2}{2}=1, \qquad \text{Row 1 } (a_1): a[1][0]=-1\le 0 \text{ (ineligible)}.$$

  The minimum ratio is 1, so row $p=0$ is selected: x_0 enters the basis and artificial variable a_0 leaves. Pivoting on entry $a[0][0]=2$ yields:

$$T_1 = \begin{array}{c|cccccc|c} \text{Basis} & x_0 & x_1 & s_0 & s_1 & a_0 & a_1 & \text{RHS} \\ \hline x_0 & 1 & -\frac{1}{2} & -\frac{1}{2} & 0 & \frac{1}{2} & 0 & 1 \\ a_1 & 0 & \frac{9}{2} & -\frac{1}{2} & -1 & \frac{1}{2} & 1 & 5 \\ \hline \text{Obj } (Z) & 0 & 0 & 1 & 0 & -1 & 0 & -2 \\ \text{Ph1 } (W) & 0 & \frac{9}{2} & -\frac{1}{2} & -1 & -\frac{1}{2} & 0 & 5 \end{array} \tag{4.79}$$

  The current Phase I objective is $W=-5$, reflecting that artificial variable $a_1=5$ remains.

- **Pivot 2**: In row Ph1, column $q=1$ (x_1) has positive reduced cost $\bar{w}_1=9/2>0$. Testing the ratio rule:

$$\text{Row 0 } (x_0): a[0][1]=-\frac{1}{2}\le 0 \text{ (ineligible)}, \qquad \text{Row 1 } (a_1): \frac{5}{9/2}=\frac{10}{9}.$$

  Row $p=1$ is the unique candidate: x_1 enters and the remaining artificial variable a_1 leaves. Pivoting on entry $a[1][1]=9/2$ yields:

$$T_2 = \begin{array}{c|cccccc|c} \text{Basis} & x_0 & x_1 & s_0 & s_1 & a_0 & a_1 & \text{RHS} \\ \hline x_0 & 1 & 0 & -\frac{5}{9} & -\frac{1}{9} & \frac{5}{9} & \frac{1}{9} & \frac{14}{9} \\ x_1 & 0 & 1 & -\frac{1}{9} & -\frac{2}{9} & \frac{1}{9} & \frac{2}{9} & \frac{10}{9} \\ \hline \text{Obj } (Z) & 0 & 0 & 1 & 0 & -1 & 0 & -2 \\ \text{Ph1 } (W) & 0 & 0 & 0 & 0 & -1 & -1 & 0 \end{array} \tag{4.80}$$

Step 5: Phase I Conclusion and Feasibility Certificate In tableau T_2, all reduced costs in row Ph1 for the non-artificial variables $\{x_0, x_1, s_0, s_1\}$ are $0\le 0$. The Phase I objective value is:

$$W^* = -a[3][6] = -0 = 0.$$

Because $\max W = 0$, both artificial variables have been driven to zero ($a_0 = 0, a_1 = 0$). The basic variables are:

$$x_0 = \frac{14}{9}, \qquad x_1 = \frac{10}{9}.$$

Because $x_0 \geq 0$ and $x_1 \geq 0$, $\mathbf{x}^{(0)} = (14/9, 10/9)^\top$ is a certified basic feasible solution (extreme point) of the original polyhedron $\mathcal{P}$. Phase I has successfully concluded.

Step 6: Phase II Transition and Tableau We discard the Phase I objective row and the artificial columns a_0, a_1. Because elementary row operations were simultaneously applied to the original objective row Z during the Phase I pivots (as implemented in `TwoPhaseSimplex.java`), row Z is already in canonical form with respect to the feasible basis $\{x_0, x_1\}$, expressing Z purely in terms of the non-basic surplus variables:

$$T_{\text{II}} = \begin{array}{c|cccc|c} \text{Basis} & x_0 & x_1 & s_0 & s_1 & \text{RHS} \\ \hline x_0 & 1 & 0 & -\frac{5}{9} & -\frac{1}{9} & \frac{14}{9} \\ x_1 & 0 & 1 & -\frac{1}{9} & -\frac{2}{9} & \frac{10}{9} \\ \hline \text{Obj } (Z) & 0 & 0 & 1 & 0 & -2 \end{array} \tag{4.81}$$

The current objective value is $Z = -(-2) = 2$, which coincides with $2x_0 - x_1 = 2(14/9) - (10/9) = 18/9 = 2$.

Step 7: Phase II Execution and Unboundedness Certificate In the objective row of T_{II}, surplus variable s_0 has a positive reduced cost:

$$\bar{c}_{s_0} = 1 > 0.$$

To increase Z, we select s_0 as the entering column ($q = 2$). We inspect the constraint entries in column s_0:

$$a[0][2] = -\frac{5}{9} \leq 0, \qquad a[1][2] = -\frac{1}{9} \leq 0.$$

Every coefficient in the entering column is non-positive ($a[i][2] \leq 0$ for all i). Expressing the basic variables as functions of s_0 (holding $s_1 = 0$):

$$x_0 = \frac{14}{9} + \frac{5}{9}s_0,$$
$$x_1 = \frac{10}{9} + \frac{1}{9}s_0.$$

Because the coefficients $+5/9$ and $+1/9$ are strictly positive, increasing s_0 causes both x_0 and x_1 to increase. Neither basic variable decreases, so neither variable will ever reach zero. The surplus s_0 can be increased toward $+\infty$ without violating feasibility:

$$Z = 2 + 1 \cdot s_0 \xrightarrow{s_0 \to \infty} +\infty.$$

The minimum ratio rule returns $p = -1$, certifying that the linear program is **unbounded**.

Pedagogical Significance of the Example. This trace demonstrates the dual power of the Simplex method: Phase I resolves the challenge of an infeasible origin by navigating through an augmented artificial space to locate a valid initial vertex $\mathbf{x}^{(0)} = (14/9, 10/9)^\top$ on $\mathcal{P}$. Phase II then detects that the feasible region extends infinitely along the ray direction $(5/9, 1/9)^\top$ with an increasing objective, providing an exact algorithmic certificate of unboundedness. If the objective had instead been bounded—for instance, maximizing $-2x_0 + x_1$ (the negation of the original objective)—then the reduced costs in T_{II} would all be non-positive ($\bar{c}_{s_0} = -1 \leq 0, \bar{c}_{s_1} = 0 \leq 0$), certifying that $\mathbf{x}^{(0)}$ is immediately the unique global optimum.

Two-Phase Implementation in Java

Tableau Setup and Phase I Canonicalization As depicted in Figure 4.5, the constructor of `TwoPhaseSimplex` configures the 2D array `a` of dimensions $(m+2) \times (n+2m+1)$, adding m artificial variable columns and an auxiliary Phase I objective row:

- **Row Sign Normalization**: If any right-hand side constant is negative ($b_i < 0$), the entire constraint row i is multiplied by -1, ensuring that all entries in the RHS column $n+2m$ are non-negative ($b_i \geq 0$). The slack variable coefficient is correspondingly set to -1.0.
- **Artificial Variables $\mathbf{I}_m$** (columns $n+m \leq j < n+2m$): An $m \times m$ identity block introduces artificial variables $x_{a,0}, \ldots, x_{a,m-1}$ to supply an immediate starting basis.
- **Phase I Auxiliary Objective** (row $m+1$): Stores the auxiliary objective $W = -\sum_{i=0}^{m-1} x_{a,i}$, initialized with coefficients -1.0 in the artificial columns.
- **Canonicalization via Pivoting**: Because the artificial variables are declared basic (`basis`$[i] = n+m+i$), their reduced costs in the Phase I objective row $m+1$ must be eliminated before Phase I begins. The constructor accomplishes this by executing `pivot(i, n + m + i)` for each row i. This Gauss-Jordan elimination clears the -1.0 entries in row $m+1$ and computes the initial value $W = -\sum b_i$, placing the Phase I tableau into canonical form.

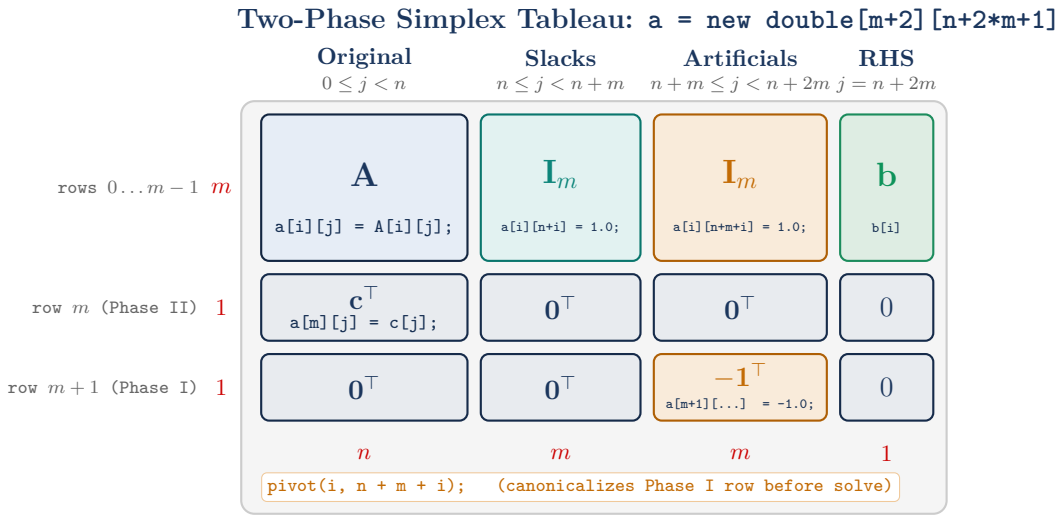


Figure 4.5: Two-Phase Simplex tableau 2D array memory layout and Java initialization (`TwoPhaseSimplex.java`) augmented to size $(m+2) \times (n+2m+1)$ with m artificial variable columns and an auxiliary Phase I objective row.

Listing 4.2 displays the complete implementation of `TwoPhaseSimplex`.

```java
public class TwoPhaseSimplex {
    private static final double EPSILON = 1.0E-8;
    private double[][] a; // tableaux: (m+2) x (n+2*m+1)
    private int m, n;
    private int[] basis;

    public TwoPhaseSimplex(double[][] A, double[] b, double[] c) {
        m = b.length;
        n = c.length;
        a = new double[m + 2][n + m + m + 1];

        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++) a[i][j] = A[i][j];

        for (int i = 0; i < m; i++) a[i][n + i] = 1.0;
        for (int i = 0; i < m; i++) a[i][n + m + m] = b[i];
        for (int j = 0; j < n; j++) a[m][j] = c[j];

        // Ensure non-negative RHS
        for (int i = 0; i < m; i++) {
            if (b[i] < 0) {
                a[i][n + m + m] = -b[i];
                for (int j = 0; j < n; j++) a[i][j] = -a[i][j];
                a[i][n + i] = -1.0;
            }
        }

        // Artificial variables form initial basis
        for (int i = 0; i < m; i++) {
            a[i][n + m + i] = 1.0;
            a[m + 1][n + m + i] = -1.0; // minimize sum(x_a)
        }
        for (int i = 0; i < m; i++) pivot(i, n + m + i);

        basis = new int[m];
        for (int i = 0; i < m; i++) basis[i] = n + m + i;

        phase1();
        phase2();
    }

    private void phase1() {
        while (true) {
            int q = -1;
            for (int j = 0; j < n + m; j++)
                if (a[m + 1][j] > EPSILON) { q = j; break; }
            if (q == -1) break; // Phase I optimal

            int p = minRatioRule(q);
            pivot(p, q);
            basis[p] = q;
        }
        if (a[m + 1][n + m + m] > EPSILON)
            throw new ArithmeticException("LP is infeasible");
    }
```

```
56
57      private void phase2() {
58          while (true) {
59              int q = -1;
60              for (int j = 0; j < n + m; j++)
61                  if (a[m][j] > EPSILON) { q = j; break; }
62              if (q == -1) break; // Phase II optimal
63
64              int p = minRatioRule(q);
65              if (p == -1)
66                  throw new ArithmeticException("LP is unbounded");
67              pivot(p, q);
68              basis[p] = q;
69          }
70      }
71
72      private int minRatioRule(int q) {
73          int p = -1;
74          for (int i = 0; i < m; i++) {
75              if (a[i][q] <= EPSILON) continue;
76              if (p == -1 ||
77                  a[i][n+m+m] / a[i][q] < a[p][n+m+m] / a[p][q]) p = i;
78          }
79          return p;
80      }
81
82      private void pivot(int p, int q) {
83          for (int i = 0; i <= m + 1; i++)
84              for (int j = 0; j <= n + m + m; j++)
85                  if (i != p && j != q)
86                      a[i][j] -= a[p][j] * (a[i][q] / a[p][q]);
87          for (int i = 0; i <= m + 1; i++)
88              if (i != p) a[i][q] = 0.0;
89          for (int j = 0; j <= n + m + m; j++)
90              if (j != q) a[p][j] /= a[p][q];
91          a[p][q] = 1.0;
92      }
93
94      public double value() { return -a[m][n + m + m]; }
95
96      public double[] primal() {
97          double[] x = new double[n];
98          for (int i = 0; i < m; i++)
99              if (basis[i] < n) x[basis[i]] = a[i][n + m + m];
100         return x;
101     }
102 }
```

Listing 4.2: Two-Phase Simplex implementation in Java.

4.6.3 The Big-M Penalty Approach

An alternative initialization is the *Big-M method*. Instead of an auxiliary objective, artificial variables $\mathbf{x}_a$ are penalized directly in the original objective function using a large positive constant $M \gg 0$:

$$\begin{aligned} \text{maximize} \quad & \mathbf{c}^\top \mathbf{x} - M \sum_{i=0}^{m-1} x_{a,i} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} + \mathbf{s} + \mathbf{x}_a = \mathbf{b} \\ & \mathbf{x}, \mathbf{s}, \mathbf{x}_a \geq \mathbf{0}. \end{aligned} \tag{4.82}$$

Because the problem is a maximization, any positive value of an artificial variable heavily degrades the objective. If a feasible solution exists, the algorithm automatically drives all artificial variables to zero during standard iterations.

Listing 4.3 shows how the tableau is initialized in `BigMSimplex`.

```
public class BigMSimplex {
    private static final double EPSILON = 1.0E-8;
    private double[][] a;
    private int m, n;
    private int[] basis;

    public BigMSimplex(double[][] A, double[] b,
                       double[] c, double BigM) {
        m = b.length; n = c.length;
        a = new double[m + 1][n + m + m + 1];

        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++) a[i][j] = A[i][j];

        for (int i = 0; i < m; i++) a[i][n + i] = 1.0;
        for (int i = 0; i < m; i++) a[i][n + m + m] = b[i];
        for (int j = 0; j < n; j++) a[m][j] = c[j];
        for (int j = n + m; j < n + m + m; j++)
            a[m][j] = -BigM; // penalize artificials

        // Multiply rows with b[i] < 0 by -1
        for (int i = 0; i < m; i++) {
            if (b[i] < 0) {
                a[i][n + m + m] = -b[i];
                for (int j = 0; j < n; j++) a[i][j] = -a[i][j];
                a[i][n + i] = -1.0;
            }
        }

        for (int i = 0; i < m; i++) a[i][n + m + i] = 1.0;
        for (int i = 0; i < m; i++) pivot(i, n + m + i);

        basis = new int[m];
        for (int i = 0; i < m; i++) basis[i] = n + m + i;

        solve();
    }
}
```

Listing 4.3: Big-M Simplex constructor and initialization.

4.6.4 Comparison and Numerical Stability

Numerical Instability of Big-M. While conceptually straightforward, the Big-M method introduces significant numerical instability in floating-point computation:

- **Catastrophic Cancellation**: When M is very large (e.g., $M = 10^9$), arithmetic operations like $c_j - M \cdot a_{i,j}$ cause significant roundoff loss in IEEE 754 floating-point representations. Small genuine cost differences are erased by truncation errors.
- **Ill-Conditioning**: Large disparate magnitudes degrade the condition number of the basis matrix.
- **False Conclusions**: If M is too small, an artificial variable may remain in the basis at optimality, leading to incorrect claims of feasibility.

Consequently, commercial solvers (such as Gurobi, CPLEX, and HiGHS) almost universally implement the Two-Phase Simplex method or dual simplex initialization rather than Big-M.

Table 4.1 summarizes the trade-offs between the two initialization schemes.

Criterion	Two-Phase Simplex	Big-M Method
Tableau Dimension	$(m+2) \times (n+2m+1)$	$(m+1) \times (n+2m+1)$
Parameter Sensitivity	None (parameter-free)	High (sensitive to choice of M)
Numerical Stability	High (avoids ill-conditioned rows)	Low (prone to catastrophic cancellation)
Infeasibility Detection	Exact ($W^* < 0$)	Requires testing if $x_{a,i} > \epsilon$
Implementation Complexity	Requires two objective loops	Single objective loop
Industrial Use	Universal standard	Primarily pedagogical

Table 4.1: Comparison of Two-Phase Simplex versus the Big-M method.

4.7 Duality Theory and Certificates

Every linear program has an associated linear program called its *dual*. Duality is among the most profound principles in mathematical optimization, providing certificates of optimality and economic sensitivities.

4.7.1 The Primal-Dual Pair

Definition 4.7 *Primal and Dual Linear Programs*

Given a primal linear program in canonical form:

$$\begin{aligned} \text{maximize} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned} \tag{4.83}$$

its *dual linear program* is defined as:

$$\begin{aligned} \text{minimize} \quad & \mathbf{b}^\top \mathbf{y} \\ \text{subject to} \quad & \mathbf{A}^\top \mathbf{y} \geq \mathbf{c} \\ & \mathbf{y} \geq \mathbf{0}, \end{aligned} \tag{4.84}$$

where $\mathbf{y} = (y_0, y_1, \ldots, y_{m-1})^\top \in \mathbb{R}^m$ is the vector of dual decision variables.

Each dual variable y_i corresponds to the i-th primal constraint, and each dual constraint corresponds to the j-th primal variable.

4.7.2 Weak and Strong Duality

Theorem 4.5 *Weak Duality Theorem*

If $\mathbf{x}$ is a primal feasible solution and $\mathbf{y}$ is a dual feasible solution, then:

$$\mathbf{c}^\top \mathbf{x} \leq \mathbf{b}^\top \mathbf{y}. \tag{4.85}$$

Proof. Because $\mathbf{x} \geq \mathbf{0}$ and $\mathbf{A}^\top \mathbf{y} \geq \mathbf{c}$, taking the inner product with $\mathbf{x}$ preserves the inequality:

$$\mathbf{c}^\top \mathbf{x} \leq (\mathbf{A}^\top \mathbf{y})^\top \mathbf{x} = \mathbf{y}^\top \mathbf{A}\mathbf{x}. \tag{4.86}$$

Because $\mathbf{y} \geq \mathbf{0}$ and $\mathbf{A}\mathbf{x} \leq \mathbf{b}$, we similarly have:

$$\mathbf{y}^\top (\mathbf{A}\mathbf{x}) \leq \mathbf{y}^\top \mathbf{b} = \mathbf{b}^\top \mathbf{y}. \tag{4.87}$$

Combining both inequalities yields $\mathbf{c}^\top \mathbf{x} \leq \mathbf{b}^\top \mathbf{y}$. □

Weak duality establishes that any dual feasible solution provides an upper bound on the optimal value of the primal. Conversely, any primal feasible solution provides a lower bound on the dual.

Theorem 4.6 *Strong Duality Theorem*

If a linear program has an optimal solution $\mathbf{x}^*$, then its dual linear program also has an optimal solution $\mathbf{y}^*$, and their optimal objective values are identical:

$$\mathbf{c}^\top \mathbf{x}^* = \mathbf{b}^\top \mathbf{y}^*. \tag{4.88}$$

4.7.3 Complementary Slackness

Theorem 4.7 *Complementary Slackness Theorem*

Let $\mathbf{x}$ be primal feasible and $\mathbf{y}$ be dual feasible. Then $\mathbf{x}$ and $\mathbf{y}$ are simultaneously optimal for their respective problems if and only if:

$$x_j \left(\sum_{i=0}^{m-1} a_{i,j} y_i - c_j \right) = 0, \quad \forall j \in \{0, \dots, n-1\}, \tag{4.89}$$

and

$$y_i \left(b_i - \sum_{j=0}^{n-1} a_{i,j} x_j \right) = 0, \quad \forall i \in \{0, \dots, m-1\}. \tag{4.90}$$

Proof. Subtracting the primal objective from the dual objective:

$$\mathbf{b}^\top \mathbf{y} - \mathbf{c}^\top \mathbf{x} = \mathbf{b}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{A}\mathbf{x} + \mathbf{y}^\top \mathbf{A}\mathbf{x} - \mathbf{c}^\top \mathbf{x} = \mathbf{y}^\top (\mathbf{b} - \mathbf{A}\mathbf{x}) + (\mathbf{A}^\top \mathbf{y} - \mathbf{c})^\top \mathbf{x}. \tag{4.91}$$

Since $\mathbf{x}$ and $\mathbf{y}$ are feasible, $\mathbf{y} \geq \mathbf{0}$, $\mathbf{b} - \mathbf{A}\mathbf{x} \geq \mathbf{0}$, $\mathbf{A}^\top \mathbf{y} - \mathbf{c} \geq \mathbf{0}$, and $\mathbf{x} \geq \mathbf{0}$. Every term in the sum is non-negative:

$$\sum_{i=0}^{m-1} \underbrace{y_i}_{\geq 0} \underbrace{(b_i - \mathbf{a}_{i,\cdot}^\top \mathbf{x})}_{\geq 0} + \sum_{j=0}^{n-1} \underbrace{x_j}_{\geq 0} \underbrace{(\mathbf{a}_{\cdot,j}^\top \mathbf{y} - c_j)}_{\geq 0} = \mathbf{b}^\top \mathbf{y} - \mathbf{c}^\top \mathbf{x} \geq 0. \tag{4.92}$$

By Strong Duality (Theorem 4.6), $\mathbf{x}$ and $\mathbf{y}$ are optimal if and only if $\mathbf{b}^\top \mathbf{y} - \mathbf{c}^\top \mathbf{x} = 0$. Since a sum of non-negative terms equals zero if and only if each term is zero, Equations (4.89) and (4.90) must hold. □

4.7.4 Dual Recovery and Shadow Prices

Proposition 4.8 *Dual Extraction from Tableau*

At optimality, the reduced costs of the slack variables $s_i = x_{n+i}$ in row m of the simplex tableau satisfy:

$$y_i^* = -a[m][n+i], \quad \forall i \in \{0, \dots, m-1\}. \tag{4.93}$$

In the Brewery Problem (Section 4.5.1), the final row of Tableau 2 has slack reduced costs $a[3][2] = -1$ (corn slack), $a[3][3] = -2$ (hops slack), and $a[3][4] = 0$ (malt slack). The optimal dual variables are therefore:

$$y_0^* = 1, \quad y_1^* = 2, \quad y_2^* = 0. \tag{4.94}$$

Evaluating the dual objective confirms strong duality:

$$\mathbf{b}^\top \mathbf{y}^* = 480(1) + 160(2) + 1190(0) = 480 + 320 = 800 = Z^*. \tag{4.95}$$

Economic Interpretation (Shadow Prices) The dual variable y_i^* represents the *shadow price* or marginal value of resource i:

$$y_i^* = \frac{\partial Z^*}{\partial b_i}. \tag{4.96}$$

- $y_0^* = 1$: Having one additional unit of corn increases maximum profit by 1 unit.
- $y_1^* = 2$: Having one additional unit of hops increases maximum profit by 2 units.
- $y_2^* = 0$: Malt is in excess (slack $x_4^* = 210 > 0$). Adding more malt has zero marginal value. By complementary slackness, since the malt constraint is not tight, its shadow price is strictly zero ($y_2^* = 0$).

4.8 Complexity, Cycling, and Solvers

4.8.1 Degeneracy and Bland's Rule

A basic feasible solution is *degenerate* if at least one basic variable has value zero: $x_{B(i)} = 0$. Geometrically, degeneracy occurs when more than n hyperplanes intersect at a single vertex in $\mathbb{R}^n$.

In the presence of degeneracy, a pivot may yield a minimum ratio of $\theta^* = 0$. In this case, the entering variable enters at value zero, the basis changes, but the geometric point $\mathbf{x}$ and the objective value Z remain unchanged. This is known as *stalling*.

If a sequence of degenerate pivots cycles through a recurring set of bases without altering the solution, the Simplex algorithm can enter an infinite loop. This phenomenon is called *cycling*.

Theorem 4.9 *Bland's Anticycling Theorem*

Under Bland's smallest-subscript rule (always selecting the entering column $q = \min\{j \mid \bar{c}_j > 0\}$ and tie-breaking the leaving row by choosing the smallest index p), the Simplex algorithm never cycles and terminates in a finite number of iterations [Bla77].

4.8.2 Worst-Case Exponential Complexity

Despite its extraordinary practical speed, the Simplex algorithm with Dantzig's rule is not a polynomial-time algorithm in the worst case. In 1972, Victor Klee and George Minty constructed a celebrated family of perturbed hypercubes [KM72]:

$$\begin{aligned} \text{maximize} \quad & \sum_{j=0}^{n-1} 10^{n-1-j} x_j \\ \text{subject to} \quad & 2\sum_{k=0}^{j-1} 10^{j-k} x_k + x_j \le 100^j, \quad \forall j \in \{0, \dots, n-1\}, \\ & x_j \ge 0, \quad \forall j \in \{0, \dots, n-1\}. \end{aligned} \tag{4.97}$$

The feasible polytope of the *Klee-Minty cube* has 2^n vertices. Dantzig's pivot rule visits every single vertex before reaching the optimum, requiring $2^n - 1$ pivots! For $n = 50$, the algorithm would require $2^{50} \approx 10^{15}$ pivots.

4.8.3 Polynomial-Time Interior Algorithms

The theoretical question of whether linear programming could be solved in polynomial time was resolved positively by Leonid Khachiyan in 1979 using the *Ellipsoid Method* [Kha79]. In 1984, Narendra Karmarkar introduced the first practical polynomial-time *Interior Point Method* [Kar84].

Interior point methods traverse the interior of the feasible polytope following the central path, rather than walking along boundary edges. They solve linear programs with n variables in $\mathcal{O}(n^{3.5}L)$ operations, where L is the bit length of the input data.

4.8.4 Warm-Starting and Solver Architecture

If interior point methods possess polynomial worst-case guarantees, why does the Simplex algorithm remain dominant in discrete optimization?

The answer lies in *warm-starting* (re-optimization). In branch-and-bound and cutting-plane algorithms, solvers evaluate thousands of linear programming relaxations that differ by only a single bounding constraint ($x_j \leq \lfloor v \rfloor$ or $x_j \geq \lceil v \rceil$). The dual simplex method can restart directly from the optimal basis of the parent node and restore feasibility in just 2–5 pivots! In contrast, interior point methods cannot be warm-started effectively and must solve each relaxation almost from scratch.

Modern commercial solvers (such as Gurobi and CPLEX) and high-performance open-source solvers (such as HiGHS and SCIP) combine primal simplex, dual simplex, and barrier solvers, employing presolve transformations, sparse LU factorizations, and steepest-edge pricing.

4.9 Applications of Linear Programming

4.9.1 The Nutritional Diet Problem

Pioneered by Nobel laureate George Stigler in 1945 [Sti45], the diet problem seeks the least-cost combination of foods satisfying daily nutritional requirements:

$$\begin{aligned} \text{minimize} \quad & \sum_{j=0}^{n-1} \text{cost}_j \cdot x_j \\ \text{subject to} \quad & \sum_{j=0}^{n-1} \text{nutrient}_{i,j} \cdot x_j \geq \text{minReq}_i, \quad \forall i \in \{0, \ldots, m-1\}, \\ & \sum_{j=0}^{n-1} \text{nutrient}_{i,j} \cdot x_j \leq \text{maxReq}_i, \quad \forall i \in \{0, \ldots, m-1\}, \\ & x_j \geq 0, \quad \forall j \in \{0, \ldots, n-1\}. \end{aligned} \tag{4.98}$$

Because the constraints have positive lower bounds ($\text{minReq}_i > 0$), setting all food quantities to zero ($x_j = 0$) violates nutritional adequacy. As implemented in `DietProblem.java`, this formulation requires Two-Phase Simplex or Big-M to establish an initial feasible diet before optimizing cost.

Furthermore, Duality Theory (Section 4.7) provides profound economic insights into this model: the optimal dual variables y_i^* represent the *marginal shadow price* of nutrient i. If a health authority increases the minimum daily requirement minReq_i by 1 unit, the total grocery expenditure rises by exactly y_i^* monetary units. Conversely, non-binding nutritional constraints exhibit zero dual prices by complementary slackness ($\bar{s}_i > 0 \implies y_i = 0$).

4.9.2 From Continuous to Integer Programming

Throughout this chapter, decisions have been assumed to be continuously divisible ($\mathbf{x} \geq \mathbf{0}$, $\mathbf{x} \in \mathbb{R}^n$). This continuous hypothesis underpins all classical polyhedral results:

- The feasible region is convex, guaranteeing that local optimality implies global optimality.
- Extreme points can be systematically reached and traversed along polyhedral edges in polynomial average time via the Simplex algorithm.
- Linear programming duality provides exact certificates of optimality and unboundedness.

However, in combinatorial optimization, decisions are rarely divisible. Decisions typically represent indivisible entities: selecting an item ($x_j \in \{0, 1\}$), dispatching an integral number of transport vehicles ($x_j \in \mathbb{Z}_{\geq 0}$), or ordering tasks. Restricting variables to integer values breaks convexity and moves the problem into the realm of **NP**-hard discrete optimization.

In Chapter 5, we build directly upon the foundations established here. We explore when continuous linear programming solves integer problems directly (via total unimodularity), and how modern solvers seamlessly combine the Simplex algorithm with the tree search of Chapter 3 through LP-based Branch-and-Bound, dual simplex warm-starting, and cutting planes.

4.10 Chapter Summary and Exercises

4.10.1 Key Conceptual Takeaways

Table 4.2 summarizes the theoretical and algorithmic principles established in this chapter.

Concept	Key Result and Algorithmic Significance
Polyhedral Convexity	Feasible sets are convex polyhedra; local optimality implies global optimality.
Extreme Points	An optimal solution always exists at an extreme point (vertex) of the polytope.
Simplex Algorithm	Traverses adjacent 1-dimensional edges with non-decreasing objective values.
Pivoting	Minimum ratio rule maintains primal feasibility; Bland's rule provably prevents cycling.
Initialization	Two-Phase Simplex and Big-M establish feasibility when the origin is infeasible.
Duality	Primal and dual optimal values coincide ($\mathbf{c}^\top \mathbf{x}^* = \mathbf{b}^\top \mathbf{y}^*$); dual variables yield shadow prices.
Sensitivity	Shadow prices and allowable ranges quantify marginal resource valuations.

Table 4.2: Core foundations of linear programming and polyhedral theory.

4.10.2 Exercises

Exercise 4.1 *Tableau Pivot and Optimality Verification*

Consider the following simplex tableau for a maximization problem with non-negative variables $x_0, x_1, x_2, x_3, x_4, x_5 \geq 0$:

x_0	x_1	x_2	x_3	x_4	x_5	RHS
-1	1	-3	1	0	0	25
2	3	4	0	1	0	12
3	-1	4	0	0	1	15
2	-1	-1	0	0	0	-20

1. Identify the current basic feasible solution, its basic variable indices, and its objective value.

2. Identify the entering column using Bland's rule.
3. Compute the minimum ratio for each eligible row and identify the leaving row.
4. Perform the pivot operation and state the updated tableau.
5. Is the resulting basic feasible solution optimal? Justify your answer.

Exercise 4.2 *Form Transformation and Two-Phase Initialization*

Consider the linear program:

$$\begin{aligned} \text{maximize} \quad & 3x_0 - x_1 + 2x_2 \\ \text{subject to} \quad & x_0 + 2x_1 - x_2 \geq 4 \\ & 2x_0 - x_1 + x_2 \leq 2 \\ & x_0 - 3x_1 \leq -3 \\ & x_0, x_1, x_2 \geq 0. \end{aligned}$$

1. Transform the problem into standard form with non-negative right-hand sides.
2. Formulate the Phase I linear program by introducing artificial variables.
3. Set up the initial Phase I simplex tableau and perform the algebraic operations needed to canonicalize the artificial objective row.
4. Determine whether the original problem possesses a basic feasible solution.

Exercise 4.3 *Duality and Shadow Price Analysis*

A manufacturing facility produces three components x_0, x_1, x_2 using machining, assembly, and testing resources with capacities 100, 80, and 60 hours, respectively. The optimal simplex tableau yields slack variable reduced costs $\bar{c}_{s_0} = -4$, $\bar{c}_{s_1} = 0$, and $\bar{c}_{s_2} = -2.5$, with optimal profit $Z^* = 1450$.

1. State the optimal dual vector $\mathbf{y}^* = (y_0^*, y_1^*, y_2^*)^\top$.
2. An external contractor offers to lease 10 additional hours of machining capacity for \$35. Should the plant manager accept the offer?
3. What is the marginal value of expanding testing capacity?
4. Verify complementary slackness for the assembly constraint.

Exercise 4.4 *Total Unimodularity of Bipartite Graphs*

Let $G = (U \cup V, E)$ be an undirected bipartite graph. Let $\mathbf{A} \in \{0,1\}^{(|U|+|V|)\times|E|}$ denote its vertex-edge incidence matrix.

1. Show that every column of $\mathbf{A}$ contains exactly two non-zero entries, each equal to 1.
2. Prove by induction on the submatrix size that $\det(\mathbf{B}) \in \{-1, 0, 1\}$ for every square submatrix $\mathbf{B}$ of $\mathbf{A}$.
3. Conclude that the maximum cardinality matching problem on bipartite graphs can be solved via continuous linear programming.

Exercise 4.5 *The Klee-Minty Cube*

Consider the 3-dimensional Klee-Minty cube (Equation (4.97)) with $n = 3$:

$$\begin{aligned} \text{maximize} \quad & 100x_0 + 10x_1 + x_2 \\ \text{subject to} \quad & x_0 \leq 1 \\ & 20x_0 + x_1 \leq 100 \\ & 200x_0 + 20x_1 + x_2 \leq 10000 \\ & x_0, x_1, x_2 \geq 0. \end{aligned}$$

1. Trace the sequence of basic feasible solutions visited by the Simplex algorithm using Dantzig's largest-coefficient pricing rule, starting from $\mathbf{x} = (0, 0, 0)$.
2. Count the total number of pivots executed before reaching the optimal vertex.
3. Now solve the same problem using Bland's smallest-subscript rule. Does Bland's rule visit fewer vertices on this instance? Explain why.

Exercise 4.6 *Network Flow Modeling and Dual Interpretation*

Consider a directed graph $G = (V, E)$ with source $s = 0$ and sink $t = |V| - 1$.

1. Formulate the maximum (s, t)-flow problem as a linear program in canonical form.
2. Formulate its dual linear program.
3. Show that any dual feasible solution $\mathbf{y}$ with binary variables $p_v \in \{0, 1\}$ corresponds to an (s, t)-cut in G.
4. Using Strong Duality, deduce the Max-Flow Min-Cut Theorem.

CHAPTER 5

Integer Linear Programming

Linear programming provides a polynomial-time computational framework for continuous decisions. However, in combinatorial optimization, decisions are frequently discrete and indivisible: items cannot be partially selected, facilities cannot be fractionally opened, and sequencing decisions are strictly binary. Restricting decision variables to integer values transforms tractable continuous polyhedra into computationally hard (**NP**-hard) discrete search spaces.

This chapter bridges continuous linear programming (Chapter 4) with the systematic tree search of branch and bound (Chapter 3). We investigate the geometric structure of integer polyhedra, identify structural conditions (total unimodularity) under which continuous linear programming naturally yields integer solutions, and develop the modern *LP-based Branch-and-Bound* engine, showing how the Dual Simplex algorithm enables rapid warm-starting when adding branching constraints and cutting planes.

5.1 Foundations of Integer Linear Programming

5.1.1 Mathematical Formulations

An *Integer Linear Program* (ILP) restricts all decision variables to integer values:

$$\begin{aligned} \text{maximize} \quad & Z = \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \in \mathbb{Z}^n_{\geq 0}, \end{aligned} \tag{5.1}$$

where $\mathbf{A} \in \mathbb{Z}^{m \times n}$, $\mathbf{b} \in \mathbb{Z}^m$, and $\mathbf{c} \in \mathbb{R}^n$. When variables are further restricted to binary choices $\mathbf{x} \in \{0,1\}^n$, the formulation is a *0-1 Integer Linear Program* (or Binary Integer Program).

In real-world applications, discrete decisions frequently interact with continuous physical quantities (e.g., binary facility opening decisions paired with continuous commodity flows). Such problems are formulated as *Mixed-Integer Linear Programs* (MILP):

$$\begin{aligned} \text{maximize} \quad & Z = \mathbf{c}^\top \mathbf{x} + \mathbf{d}^\top \mathbf{y} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} \leq \mathbf{b} \\ & \mathbf{x} \in \mathbb{Z}^n_{\geq 0}, \quad \mathbf{y} \in \mathbb{R}^p_{\geq 0}. \end{aligned} \tag{5.2}$$

5.1.2 The Geometry of the Integer Hull

To analyze an integer linear program geometrically, we define two fundamental geometric objects:

Definition 5.1 *Continuous Relaxation and Integer Hull*

Let $S = \{\mathbf{x} \in \mathbb{Z}^n_{\geq 0} \mid \mathbf{Ax} \leq \mathbf{b}\}$ be the set of integer feasible solutions.

1. The *continuous LP relaxation polyhedron* is:
$$\mathcal{P} = \{\mathbf{x} \in \mathbb{R}^n_{\geq 0} \mid \mathbf{Ax} \leq \mathbf{b}\}. \tag{5.3}$$
2. The *integer hull* $\mathcal{P}_I$ is the convex hull of all integer feasible points:
$$\mathcal{P}_I = \text{conv}(S) = \text{conv}(\mathcal{P} \cap \mathbb{Z}^n). \tag{5.4}$$

By construction, the integer hull is a convex polyhedron whose vertices are guaranteed to be integral, and it satisfies the inclusion:

$$\mathcal{P}_I \subseteq \mathcal{P}. \tag{5.5}$$

Theorem 5.1 *Fundamental Theorem of Integer Programming*

Let $\mathcal{P} = \{\mathbf{x} \in \mathbb{R}^n_{\geq 0} \mid \mathbf{Ax} \leq \mathbf{b}\}$ be a bounded polyhedron (polytope). Then optimizing a linear objective over the discrete set $S = \mathcal{P} \cap \mathbb{Z}^n$ is equivalent to optimizing over its integer hull $\mathcal{P}_I$:
$$\max_{\mathbf{x} \in \mathcal{P} \cap \mathbb{Z}^n} \mathbf{c}^\top \mathbf{x} = \max_{\mathbf{x} \in \mathcal{P}_I} \mathbf{c}^\top \mathbf{x}. \tag{5.6}$$

Proof. Because $\mathcal{P}$ is bounded, the set of integer points $S = \mathcal{P} \cap \mathbb{Z}^n$ is finite. Every point $\mathbf{x} \in \mathcal{P}_I = \text{conv}(S)$ can be expressed as a convex combination of points in S: $\mathbf{x} = \sum_k \lambda_k \mathbf{x}^{(k)}$ with $\lambda_k \geq 0$, $\sum_k \lambda_k = 1$, and $\mathbf{x}^{(k)} \in S$. By linearity:
$$\mathbf{c}^\top \mathbf{x} = \sum_k \lambda_k (\mathbf{c}^\top \mathbf{x}^{(k)}) \leq \max_k (\mathbf{c}^\top \mathbf{x}^{(k)}). \tag{5.7}$$
Hence, the maximum over $\mathcal{P}_I$ is attained at an extreme point of $\mathcal{P}_I$, which is an element of S. □

The Intractability of the Integer Hull. Theorem 5.1 reveals a profound theoretical insight: if an explicit linear inequality description $\mathcal{P}_I = \{\mathbf{x} \mid \tilde{\mathbf{A}}\mathbf{x} \leq \tilde{\mathbf{b}}\}$ were readily available, solving an **NP**-hard integer program would reduce to a polynomial-time continuous linear program! However, describing $\mathcal{P}_I$ explicitly requires an exponential number of facet-defining linear inequalities in general. The algorithmic goal of cutting plane methods and branch-and-cut is to approximate $\mathcal{P}_I$ *locally* near the optimal relaxation point.

5.1.3 The Continuous Relaxation Bound

Because $\mathcal{P}_I \subseteq \mathcal{P}$, optimizing over the continuous relaxation $\mathcal{P}$ expands the feasible region. For a maximization problem, this relaxation provides an upper bound on the integer optimum:

$$Z^*_{\text{ILP}} = \max_{\mathbf{x} \in \mathcal{P} \cap \mathbb{Z}^n} \mathbf{c}^\top \mathbf{x} \leq \max_{\mathbf{x} \in \mathcal{P}} \mathbf{c}^\top \mathbf{x} = Z^*_{\text{LP}}. \tag{5.8}$$

The non-negative difference:

$$\Delta_{\text{gap}} = Z^*_{\text{LP}} - Z^*_{\text{ILP}} \geq 0 \tag{5.9}$$

is the *integrality gap*. If the continuous relaxation optimum $\mathbf{x}^*$ happens to satisfy $\mathbf{x}^* \in \mathbb{Z}^n$, then $\mathbf{x}^*$ is immediately certified as an optimal integer solution, and $\Delta_{\text{gap}} = 0$.

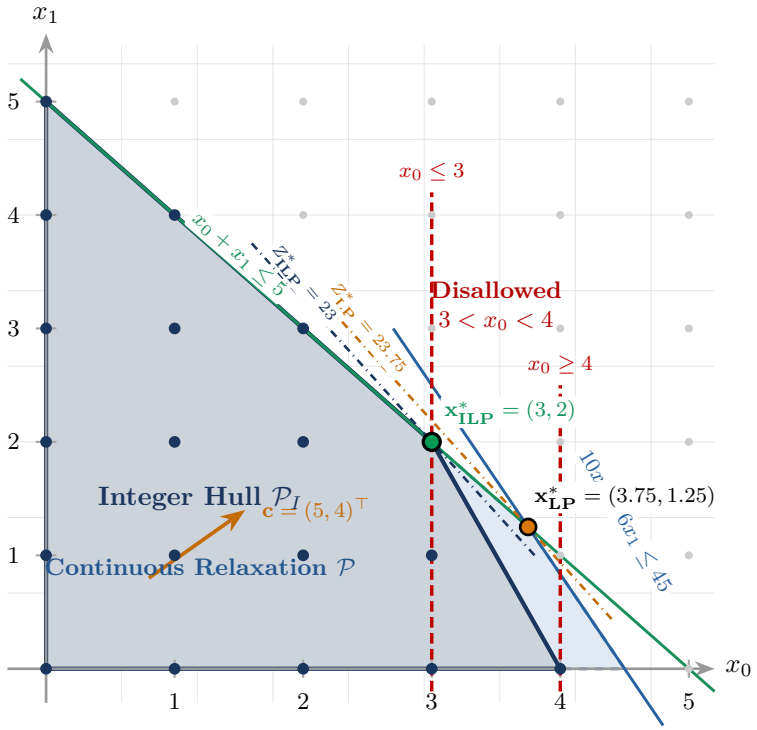


Figure 5.1: Geometric relationship between the continuous LP relaxation $\mathcal{P}$, the integer hull $\mathcal{P}_I = \text{conv}(\mathcal{P} \cap \mathbb{Z}^2)$, and LP-based branching. Discrete integer feasible solutions (navy dots) lie inside $\mathcal{P}$. The continuous relaxation optimum $\mathbf{x}^*_{\text{LP}} = (3.75, 1.25)$ achieves $Z^*_{\text{LP}} = 23.75$ on an exterior fractional corner. Branching on x_0 creates two subproblems ($x_0 \leq 3$ and $x_0 \geq 4$), carving away the non-integral strip $3 < x_0 < 4$ and isolating the integer optimum $\mathbf{x}^*_{\text{ILP}} = (3, 2)$ with $Z^*_{\text{ILP}} = 23$.

Figure 5.1 illustrates this fundamental geometric relationship for the two-variable problem analyzed in Example 5.1. The continuous relaxation polyhedron $\mathcal{P}$ contains points not in the integer hull $\mathcal{P}_I$. The objective function $\mathbf{c}^\top \mathbf{x}$ drives the continuous Simplex algorithm to the fractional vertex $\mathbf{x}^*_{\text{LP}} = (3.75, 1.25)$. Branching on x_0 introduces the disjunction $x_0 \leq 3 \vee x_0 \geq 4$, eliminating the fractional strip $3 < x_0 < 4$ without removing any integer feasible points, and guiding the search toward $\mathbf{x}^*_{\text{ILP}} = (3, 2)$.

5.2 Total Unimodularity and Polyhedral Integrality

Under specific algebraic matrix structures, the continuous LP relaxation is guaranteed to yield an integer solution without branching.

Definition 5.2 *Totally Unimodular Matrix*

A matrix $\mathbf{A} \in \mathbb{Z}^{m \times n}$ is *totally unimodular* (TUM) if the determinant of every square submatrix of $\mathbf{A}$ belongs to $\{-1, 0, 1\}$.

In particular, every individual entry of a totally unimodular matrix must belong to $\{-1, 0, 1\}$, since each entry is a 1×1 submatrix.

Theorem 5.2 *Hoffman-Kruskal Integrality Theorem*

Let $\mathbf{A} \in \mathbb{Z}^{m \times n}$ be an integral matrix. The polyhedron:

$$\mathcal{P} = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{A}\mathbf{x} \leq \mathbf{b},\ \mathbf{x} \geq \mathbf{0}\} \tag{5.10}$$

has only integral extreme points for every integer vector $\mathbf{b} \in \mathbb{Z}^m$ for which $\mathcal{P}$ is non-empty if and only if $\mathbf{A}$ is totally unimodular.

Proof. Let $\mathbf{A}_B$ be any basis matrix corresponding to an extreme point $\mathbf{x}$. In standard equational form with slack variables $[\mathbf{A} \mid \mathbf{I}_m]$, any basis matrix is an $m \times m$ invertible submatrix. The basic variables are given by Cramer's rule:

$$x_{B(i)} = \frac{\det(\mathbf{A}_B^{(i)})}{\det(\mathbf{A}_B)}, \quad \forall i \in \{0, \ldots, m-1\}, \tag{5.11}$$

where $\mathbf{A}_B^{(i)}$ is obtained by replacing column i of $\mathbf{A}_B$ with the integer right-hand side $\mathbf{b}$. Because $\mathbf{A}$ is TUM and $\mathbf{A}_B$ is non-singular, $\det(\mathbf{A}_B) \in \{-1, 1\}$. Expanding $\det(\mathbf{A}_B^{(i)})$ along the integer column $\mathbf{b}$ produces an integer. Dividing an integer by ± 1 yields an integer. Hence, every basic feasible solution $\mathbf{x}$ is entirely integral. □

5.2.1 Network Flow and Bipartite Matching

A cornerstone application of total unimodularity is network optimization:

Proposition 5.3 *Node-Arc Incidence Matrices are TUM*

Let $G = (V, E)$ be a directed graph with $V = \{0, \ldots, |V| - 1\}$. The node-arc incidence matrix $\mathbf{M} \in \{-1, 0, 1\}^{|V| \times |E|}$, defined by:

$$M_{v,e} = \begin{cases} +1, & \text{if arc } e \text{ leaves vertex } v, \\ -1, & \text{if arc } e \text{ enters vertex } v, \\ 0, & \text{otherwise,} \end{cases} \tag{5.12}$$

is totally unimodular.

Consequently, for any minimum-cost network flow problem with integer capacities $u_e \in \mathbb{Z}$ and integer node supplies/demands $b_v \in \mathbb{Z}$:

$$\begin{aligned} \text{minimize} \quad & \sum_{e \in E} c_e f_e \\ \text{subject to} \quad & \sum_{e \in \delta^+(v)} f_e - \sum_{e \in \delta^-(v)} f_e = b_v, \quad \forall v \in V, \\ & 0 \leq f_e \leq u_e, \quad \forall e \in E, \end{aligned} \tag{5.13}$$

solving the continuous linear relaxation with the Simplex algorithm guarantees an integer optimal flow $\boldsymbol{f}^* \in \mathbb{Z}^{|E|}$.

Similarly, in a bipartite graph $G = (U \cup V, E)$, the vertex-edge incidence matrix is totally unimodular. Solving the continuous LP relaxation:

$$\max \sum_{e \in E} x_e \quad \text{s.t.} \quad \sum_{e \in \delta(w)} x_e \leq 1 \; (\forall w \in U \cup V), \quad x_e \geq 0 \tag{5.14}$$

automatically yields optimal binary matching variables $x_e^* \in \{0, 1\}$.

Listing 5.1 shows how a directed network is encoded as a linear program in Java.

```java
public class FlowMatrices {
    final double[][] A;
    final double[] b;
    final double[] c;

    public FlowMatrices(FlowNetwork network) {
        int V = network.V(), E = network.E();
        int nRows = E + 2 * V, nCols = E;
        A = new double[nRows][nCols];
        b = new double[nRows];
        c = new double[nCols];

        int source = network.source();
        for (int e : network.edgesFrom(source)) c[e] = 1.0;

        // Capacity constraints: f_e <= capacity_e
        for (int e = 0; e < E; e++) {
            A[e][e] = 1.0;
            b[e] = network.edge(e).capacity();
        }

        // Flow conservation: sum_in - sum_out = 0
        int row = E;
        for (int v = 0; v < V; v++) {
            if (v == source || v == network.sink()) continue;
            for (int e : network.inEdges(v))  A[row][e] =  1.0;
            for (int e : network.outEdges(v)) A[row][e] = -1.0;
            b[row++] = 0.0;
            // Opposite inequality: -(sum_in - sum_out) <= 0
            for (int e : network.inEdges(v))  A[row][e] = -1.0;
            for (int e : network.outEdges(v)) A[row][e] =  1.0;
            b[row++] = 0.0;
        }
    }
}
```

Listing 5.1: Encoding Network Flow constraint matrices in Java.

5.3 LP-Based Branch and Bound

When the constraint matrix $\mathbf{A}$ is not totally unimodular, continuous LP relaxations generally yield fractional extreme points. To eliminate fractional solutions, modern solvers integrate the Simplex algorithm into the tree search framework of Branch and Bound (Chapter 3).

5.3.1 The Search Algorithm

Consider the integer linear program $\max\{\mathbf{c}^\top \mathbf{x} \mid \mathbf{A}\mathbf{x} \leq \mathbf{b},\ \mathbf{x} \in \mathbb{Z}^n_{\geq 0}\}$. The solver maintains a global lower bound $\underline{Z}$ (the objective value of the best known integer feasible solution, initialized to $-\infty$), and a queue $\mathcal{Q}$ of active subproblems (nodes).

At each node $u \in \mathcal{Q}$:

1. **Relaxation Solve**: Solve the continuous linear programming relaxation $LP(u)$ via the Simplex algorithm, obtaining an optimal basic solution $\mathbf{x}^*$ with objective value $Z_{LP}(u)$.
2. **Pruning Checks**:
   - *Pruning by Infeasibility*: If $LP(u)$ is infeasible, prune node u (the subproblem contains no feasible integer points).
   - *Pruning by Bound*: If $Z_{LP}(u) \leq \underline{Z}$, prune node u (no feasible completion in this subtree can strictly improve upon the incumbent).
   - *Pruning by Integrality*: If all integer-constrained variables are integral ($x_j^* \in \mathbb{Z}$ for all j), then $\mathbf{x}^*$ is integer feasible. Update the incumbent $\underline{Z} \leftarrow \max(\underline{Z}, Z_{LP}(u))$ and prune node u.
3. **Branching**: If $Z_{LP}(u) > \underline{Z}$ and at least one variable x_k^* is fractional ($x_k^* \notin \mathbb{Z}$), choose a fractional variable x_k and partition the feasible space into two disjoint subproblems:

$$\text{Left child (Down-branch): } x_k \leq \lfloor x_k^* \rfloor, \qquad \text{Right child (Up-branch): } x_k \geq \lceil x_k^* \rceil. \tag{5.15}$$

5.3.2 Branching Decisions: Variable Selection

When multiple basic variables are fractional at an optimal node solution $\mathbf{x}^*$, the choice of the branching variable x_k exerts an immense impact on the size of the search tree. Solvers employ several heuristics:

1. Most Fractional Rule Select the variable whose fractional part $f_j = x_j^* - \lfloor x_j^* \rfloor$ is closest to 0.5:

$$k = \arg\min_{j\,:\,f_j > 0} |f_j - 0.5|. \tag{5.16}$$

The intuition is that variables furthest from an integer impose the greatest disruption to the continuous relaxation, maximizing bound degradation in the child nodes. While computationally trivial ($\mathcal{O}(n)$), this myopic rule ignores objective coefficients and constraint geometry.

2. Strong Branching Before committing to a branch, the solver tests a candidate subset of fractional variables. For each candidate x_j, the solver tentatively applies both bounds $x_j \leq \lfloor x_j^* \rfloor$ and $x_j \geq \lceil x_j^* \rceil$, executes a limited number of Dual Simplex pivots (typically 10–50 iterations), and measures the objective degradations:

$$\Delta_j^- = Z^* - Z_{LP}(x_j \leq \lfloor x_j^* \rfloor), \qquad \Delta_j^+ = Z^* - Z_{LP}(x_j \geq \lceil x_j^* \rceil). \tag{5.17}$$

The solver then selects the variable that maximizes a combined score, such as the product score:

$$\text{score}(j) = \max(\Delta_j^-, 10^{-6}) \times \max(\Delta_j^+, 10^{-6}). \tag{5.18}$$

Strong branching produces very small search trees, but the computational cost of testing multiple candidates at every node is high.

3. Pseudo-Cost Branching To obtain the quality of strong branching without its computational overhead, solvers track historical performance. The *pseudo-costs* Ψ_j^- and Ψ_j^+ record the average objective degradation per unit change observed whenever variable x_j was branched upon in ancestor nodes:

$$\Psi_j^- = \frac{1}{|K_j^-|} \sum_{u \in K_j^-} \frac{Z_{\text{parent}} - Z_{\text{child}}}{\lfloor x_j^* \rfloor - x_j^*}, \qquad \Psi_j^+ = \frac{1}{|K_j^+|} \sum_{u \in K_j^+} \frac{Z_{\text{parent}} - Z_{\text{child}}}{x_j^* - \lceil x_j^* \rceil}. \tag{5.19}$$

At subsequent nodes, the degradations are estimated cheaply as $\Delta_j^- \approx \Psi_j^- \cdot f_j$ and $\Delta_j^+ \approx \Psi_j^+ \cdot (1 - f_j)$.

4. Reliability Branching State-of-the-art MIP solvers (such as HiGHS, SCIP, and Gurobi) combine strong branching and pseudo-costs via *reliability branching*. For the first η times a variable is considered (typically $\eta \in [4, 8]$), strong branching evaluates the variable directly to initialize reliable pseudo-costs. Once a variable has been evaluated η times, the solver switches permanently to fast pseudo-cost estimates.

5.3.3 Updating the Simplex Tableau with Branching Constraints

The decisive reason LP-based Branch-and-Bound is computationally viable is that child LP relaxations *do not need to be solved from scratch*. When a branching constraint is added to a child node, the parent's optimal Simplex tableau is updated directly.

Algebraic Tableau Representation Recall from Section 4.5 that an optimal simplex tableau partitions variables into basic indices B and non-basic indices N. The canonical dictionary equations express the basic variables and the objective in terms of non-basic variables:

$$x_{B(i)} = \bar{b}_i - \sum_{j \in N} \bar{a}_{i,j} x_j, \quad \forall i \in \{0, \ldots, m-1\}, \tag{5.20}$$

$$Z = Z_0 + \sum_{j \in N} \bar{c}_j x_j. \tag{5.21}$$

Because the parent tableau is optimal for a maximization problem, every reduced cost is non-positive:

$$\bar{c}_j \leq 0, \quad \forall j \in N. \tag{5.22}$$

Adding a Down-Branch Constraint Suppose basic variable $x_{B(k)}$ is fractional with value $\bar{b}_k = x_{B(k)}^* \notin \mathbb{Z}$. Let $f_k = \bar{b}_k - \lfloor \bar{b}_k \rfloor \in (0, 1)$ denote its fractional part. The down-branch imposes:

$$x_{B(k)} \leq \lfloor \bar{b}_k \rfloor. \tag{5.23}$$

We convert this inequality into standard equality form by introducing a non-negative slack variable $s_{\text{branch}} \geq 0$:

$$x_{B(k)} + s_{\text{branch}} = \lfloor \bar{b}_k \rfloor. \tag{5.24}$$

Crucially, $x_{B(k)}$ is already a basic variable! To maintain the canonical tableau form, basic variables cannot appear with non-zero coefficients in other constraint rows. We must substitute the dictionary expression of $x_{B(k)}$ from Equation (5.20) into Equation (5.24):

$$\left(\bar{b}_k - \sum_{j \in N} \bar{a}_{k,j} x_j \right) + s_{\text{branch}} = \lfloor \bar{b}_k \rfloor. \tag{5.25}$$

Rearranging to express the new basic slack variable s_{branch} yields:

$$s_{\text{branch}} - \sum_{j \in N} \bar{a}_{k,j} x_j = \lfloor \bar{b}_k \rfloor - \bar{b}_k = -f_k. \tag{5.26}$$

In standard tableau row notation with basic variable s_{branch} in row m:

$$s_{\text{branch}} + \sum_{j \in N} (-\bar{a}_{k,j}) x_j = -f_k. \tag{5.27}$$

Adding an Up-Branch Constraint Symmetrically, the up-branch imposes $x_{B(k)} \geq \lceil \bar{b}_k \rceil$. Multiplying by -1 to convert to canonical $\leq$ form and introducing a non-negative slack variable $s_{\text{branch}} \geq 0$ gives:

$$-x_{B(k)} + s_{\text{branch}} = -\lceil \bar{b}_k \rceil. \tag{5.28}$$

Substituting $-x_{B(k)} = -\bar{b}_k + \sum_{j \in N} \bar{a}_{k,j} x_j$ from Equation (5.20) into Equation (5.28) yields the augmented row:

$$s_{\text{branch}} + \sum_{j \in N} (+\bar{a}_{k,j}) x_j = \bar{b}_k - \lceil \bar{b}_k \rceil = -(1 - f_k). \tag{5.29}$$

Notice that both (5.27) and (5.29) follow the exact same template: the non-basic coefficients of row k are scaled by sign $\in \{-1, +1\}$, the new slack variable s_{branch} enters the basis with coefficient $+1$, and the new right-hand side is strictly negative ($-f_k < 0$ on the down-branch and $-(1 - f_k) < 0$ on the up-branch).

The Fundamental Duality Asymmetry Inspecting the augmented tableau with row (5.27) reveals two fundamental properties:

1. **Primal Infeasibility**: Setting all non-basic variables $x_j = 0$ ($j \in N$) assigns $s_{\text{branch}} = -f_k < 0$. The current basic solution violates non-negativity and is *primal infeasible*.
2. **Dual Feasibility (Optimality)**: The objective row (5.21) is completely unchanged! The new slack variable s_{branch} has an initial cost of 0, so its reduced cost is zero. All existing non-basic variables retain their reduced costs $\bar{c}_j \leq 0$. The tableau is **100% dual feasible**!

5.3.4 Dual Simplex Warm-Starting

Because the augmented tableau is dual feasible but primal infeasible, applying the Primal Simplex algorithm would require discarding the tableau and executing Two-Phase Simplex from scratch.

Instead, the **Dual Simplex Algorithm** (Lemke, 1954 [Lem54]) is applied directly! The Dual Simplex algorithm maintains dual feasibility ($\bar{c}_j \leq 0$) while systematically eliminating negative right-hand sides until primal feasibility is restored.

Step 1: Dual Leaving Row Selection (Primal Feasibility Rule) Select a row p with a negative right-hand side constant:

$$p = \underset{i \,:\, a[i][n+m] < 0}{\arg\min} \; a[i][n+m]. \tag{5.30}$$

For a newly added branching row, $p = m$ (the branching row itself, where $a[m][n+m] = -f_k < 0$).

Step 2: Dual Ratio Test (Dual Optimality Rule) To ensure that all reduced costs in row $m+1$ remain non-positive ($\bar{c}'_j \leq 0$) after Gauss-Jordan elimination, the entering column $q \in N$ must be chosen among columns where the pivot row entry is strictly negative:

$$q = \underset{j \in N \,:\, a[p][j]<0}{\arg\min} \frac{a[\text{obj}][j]}{a[p][j]} = \underset{j \in N \,:\, a[p][j]<0}{\arg\min} \left|\frac{\bar{c}_j}{a[p][j]}\right|. \tag{5.31}$$

- If no entry in row p is negative ($a[p][j] \geq 0$ for all $j \in N$), then no non-basic variable can increase the negative right-hand side. The subproblem is certified to be **infeasible**, and the child node is immediately pruned!
- Otherwise, column q enters the basis, and row p leaves.

Step 3: Gauss-Jordan Pivot Perform the standard row operations on pivot element $a[p][q] < 0$. After typically only **1 to 3 dual pivots**, all basic right-hand sides become non-negative, reaching the optimal solution of the child node!

Example 5.1 *Updating the Tableau via Dual Simplex*

Consider an integer linear program whose continuous relaxation has two non-negative variables $x_0, x_1 \geq 0$:

$$\begin{aligned} \text{maximize} \quad & Z = 5x_0 + 4x_1 \\ \text{subject to} \quad & x_0 + x_1 \leq 5 \\ & 10x_0 + 6x_1 \leq 45 \\ & x_0, x_1 \in \mathbb{Z}_{\geq 0}. \end{aligned}$$

Introducing slack variables $s_0, s_1 \geq 0$, the optimal continuous simplex tableau is:

$$T_0 = \begin{array}{c|cccc|c} \text{Basis} & x_0 & x_1 & s_0 & s_1 & \text{RHS} \\ \hline x_1 & 0 & 1 & \frac{5}{2} & -\frac{1}{4} & \frac{5}{4} \\ x_0 & 1 & 0 & -\frac{3}{2} & \frac{1}{4} & \frac{15}{4} \\ \hline \text{Obj } (Z) & 0 & 0 & -\frac{5}{2} & -\frac{1}{4} & -\frac{95}{4} \end{array}$$

The continuous optimal solution is fractional: $x_0^* = 15/4 = 3.75$, $x_1^* = 5/4 = 1.25$, with $Z^* = 23.75$.
We branch on x_0: its fractional part is $f_0 = 0.75$. The down-branch imposes $x_0 \leq 3$, or $x_0 + s_2 = 3$. Substituting $x_0 = \frac{15}{4} - \left(-\frac{3}{2}s_0 + \frac{1}{4}s_1\right)$ into $x_0 + s_2 = 3$ gives:

$$s_2 = 3 - \frac{15}{4} + \left(-\frac{3}{2}s_0 + \frac{1}{4}s_1\right) \implies s_2 + \frac{3}{2}s_0 - \frac{1}{4}s_1 = -\frac{3}{4}.$$

Appending this row with basic variable s_2 yields the augmented tableau:

$$T_{\text{branch}} = \begin{array}{c|ccccc|c} \text{Basis} & x_0 & x_1 & s_0 & s_1 & s_2 & \text{RHS} \\ \hline x_1 & 0 & 1 & \frac{5}{2} & -\frac{1}{4} & 0 & \frac{5}{4} \\ x_0 & 1 & 0 & -\frac{3}{2} & \frac{1}{4} & 0 & \frac{15}{4} \\ s_2 & 0 & 0 & \frac{3}{2} & -\frac{1}{4} & 1 & -\frac{3}{4} \\ \hline \text{Obj } (Z) & 0 & 0 & -\frac{5}{2} & -\frac{1}{4} & 0 & -\frac{95}{4} \end{array}$$

All reduced costs are non-positive ($\bar{c}_{s_0} = -5/2$, $\bar{c}_{s_1} = -1/4$), confirming dual feasibility. However, the RHS in row s_2 is $-3/4 < 0$. We perform a Dual Simplex step:

- **Leaving row**: Row $p = s_2$ (the only row with negative RHS).

- **Entering column**: Inspect negative entries in row s_2: the only negative entry is column s_1 with $a[s_2][s_1] = -1/4$. Column $q = s_1$ must enter.
- **Pivot on** (s_2, s_1): Multiplying row s_2 by -4 normalizes the pivot entry to 1, and eliminating s_1 from the other rows produces:

$T_{\text{dual}} =$

Basis	x_0	x_1	s_0	s_1	s_2	RHS
x_1	0	1	1	0	-1	2
x_0	1	0	0	0	1	3
s_1	0	0	-6	1	-4	3
Obj (Z)	0	0	-4	0	-1	-23

In exactly **one dual pivot**, all RHS entries are non-negative! The new optimal basic solution is:

$$x_0 = 3, \quad x_1 = 2, \quad Z = 23.$$

Because both x_0 and x_1 are integers, this child node immediately yields an integer feasible solution with value $Z = 23$, establishing a powerful global lower bound!

5.3.5 Dual Simplex Implementation in Java

Listing 5.2 implements the complete Dual Simplex re-optimization algorithm in Java. When a branching constraint or cutting plane is appended to the tableau, the constructor receives the augmented dictionary matrix. Method `solve()` drives out negative right-hand sides via dual pivots: `selectLeavingRow()` identifies the leaving row ($a[p][\text{RHS}] < 0$), and `dualRatioTest(p)` selects the entering column q minimizing $|\bar{c}_j / a[p][j]|$. If no negative pivot exists in row p, `solve()` returns `false`, immediately certifying child node infeasibility.

```
public class DualSimplex {
    private static final double EPS = 1e-10;
    private final double[][] a;
    private final int[] basis;
    private final int m, nCols;

    public DualSimplex(double[][] a, int[] basis,
                       int m, int nCols) {
        this.a = a;
        this.basis = basis;
        this.m = m;
        this.nCols = nCols;
    }

    /** Dual Simplex loop: returns false if infeasible. */
    public boolean solve() {
        while (true) {
            int p = selectLeavingRow();
            if (p == -1) return true; // Primal feasible!
            int q = dualRatioTest(p);
            if (q == -1) return false; // Infeasible node
            pivot(p, q);
            basis[p] = q;
        }
    }

    /** Select row with the most negative right-hand side. */
    private int selectLeavingRow() {
        int p = -1;
```

```
30          double minRHS = -EPS;
31          for (int i = 0; i < m; i++) {
32              if (a[i][nCols] < minRHS) {
33                  minRHS = a[i][nCols];
34                  p = i;
35              }
36          }
37          return p;
38      }
```

```
39
40      /** Dual ratio test: argmin |c_j / a[p][j]| for a[p][j] < -EPS. */
41      private int dualRatioTest(int p) {
42          int q = -1;
43          double minRatio = Double.POSITIVE_INFINITY;
44          for (int j = 0; j < nCols; j++) {
45              if (a[p][j] < -EPS) {
46                  double ratio = Math.abs(a[m][j] / a[p][j]);
47                  if (ratio < minRatio) {
48                      minRatio = ratio;
49                      q = j;
50                  }
51              }
52          }
53          return q;
54      }
55
56      private void pivot(int p, int q) {
57          for (int i = 0; i <= m; i++)
58              for (int j = 0; j <= nCols; j++)
59                  if (i != p && j != q)
60                      a[i][j] -= a[p][j] * (a[i][q] / a[p][q]);
61          for (int i = 0; i <= m; i++)
62              if (i != p) a[i][q] = 0.0;
63          for (int j = 0; j <= nCols; j++)
64              if (j != q) a[p][j] /= a[p][q];
65          a[p][q] = 1.0;
66      }
67
68      public double value() { return -a[m][nCols]; }
69
70      public double[] primal(int nOriginal) {
71          double[] x = new double[nOriginal];
72          for (int i = 0; i < m; i++)
73              if (basis[i] < nOriginal)
74                  x[basis[i]] = a[i][nCols];
75          return x;
76      }
77  }
```

Listing 5.2: Compact Dual Simplex warm-starting in Java.

5.3.6 A Minimalistic Branch-and-Bound MIP Solver

We now assemble the Primal Simplex solver (`LinearProgramming` from Chapter 4), the tableau augmentation rules (5.27)–(5.29), and the Dual Simplex warm-start engine (`DualSimplex` in Listing 5.2) into a complete, self-contained solver `BranchAndBoundMIP` for Mixed-Integer Linear Programs (Listings 5.3 and 5.4):

$$\text{maximize } \mathbf{c}^\top \mathbf{x} \quad \text{subject to} \quad \mathbf{A}\mathbf{x} \le \mathbf{b},\ \ \mathbf{x} \ge \mathbf{0},\ \ x_j \in \mathbb{Z}\ \ (\forall j : \texttt{isInteger}[j] = \texttt{true}). \tag{5.32}$$

The boolean mask `isInteger` of length n allows the exact same engine to solve both pure Integer Linear Programs (when $\texttt{isInteger}[j] = \texttt{true}$ for all $j \in \{0, \dots, n-1\}$) and Mixed-Integer Linear Programs where only a subset of variables is restricted to integers.

Why Every Fractional Variable is Basic. Observe how `selectMostFractionalRow()` in Listing 5.4 identifies a fractional integer-constrained variable: rather than scanning all n decision variables, it scans only the m basic rows $i \in \{0, \dots, m-1\}$ where $\texttt{basis}[i] < n$ and $\texttt{isInteger}[\texttt{basis}[i]]$ holds. Why can a non-basic variable never be fractional? Because in standard equality form $\mathbf{A}\mathbf{x} + \boldsymbol{s} = \mathbf{b}$

with $\mathbf{x}, \boldsymbol{s} \geq \mathbf{0}$, every non-basic variable is fixed to zero ($x_j = 0 \in \mathbb{Z}$ for all $j \in N$). Consequently, any variable with a fractional value $x_k^* \notin \mathbb{Z}$ is strictly positive ($x_k^* > 0$) and is guaranteed to reside in the basis of some unique constraint row r, whose tableau entries $\bar{a}_{r,j}$ are immediately available to construct the child branching rows (5.27) and (5.29)!

Let us trace `BranchAndBoundMIP` on two illustrative examples: first on the pure two-variable ILP of Example 5.1, and second on a Mixed-Integer Linear Program that combines discrete equipment decisions with a continuous operational variable.

Example 5.2 *Complete Branch-and-Bound Trace on a Pure ILP*

We invoke `new BranchAndBoundMIP(A, b, c)` on the integer program of Figure 5.1 and Example 5.1:

$$\text{maximize } Z = 5x_0 + 4x_1 \quad \text{subject to} \quad x_0 + x_1 \leq 5, \;\; 10x_0 + 6x_1 \leq 45, \;\; x_0, x_1 \in \mathbb{Z}_{\geq 0}.$$

The best-first priority queue explores the search tree in the following sequence:

1. **Node 0 (Root LP)**: `LinearProgramming` finds $\mathbf{x}^{(0)} = (3.75, 1.25)$ with $Z_{\text{LP}}(N_0) = 23.75$. Both variables have fractional distance $|f_j - 0.5| = 0.25$; breaking ties by smallest variable index selects x_0 and generates two warm-started children via `DualSimplex`:
   - *Down-child* N_1 ($x_0 \leq 3$): 1 dual pivot yields $\mathbf{x}^{(1)} = (3, 2)$ with $Z_{\text{LP}}(N_1) = 23.0$ (enqueued).
   - *Up-child* N_2 ($x_0 \geq 4$):Appending $s_2 - \frac{3}{2}s_0 + \frac{1}{4}s_1 = -\frac{1}{4}$ and pivoting on s_0 yields $\mathbf{x}^{(2)} = (4, 5/6)$ with $Z_{\text{LP}}(N_2) = 23\frac{1}{3} \approx 23.33$ (enqueued).
2. **Node 2 ($x_0 \geq 4$, bound 23.33)**: Popped first by best-first order because $23.33 > 23.0$. Variable $x_1 = 5/6$ is fractional, generating two children:
   - *Down-child* N_3 ($x_0 \geq 4, x_1 \leq 0$): 1 dual pivot yields $\mathbf{x}^{(3)} = (4.5, 0)$ with $Z_{\text{LP}}(N_3) = 22.5$ (enqueued).
   - *Up-child* N_4 ($x_0 \geq 4, x_1 \geq 1$): `DualSimplex.solve()` returns `false` because $10(4) + 6(1) = 46 > 45$; N_4 is immediately pruned by infeasibility!
3. **Node 1 ($x_0 \leq 3$, bound 23.0)**: Popped next. Both $x_0 = 3$ and $x_1 = 2$ are integers (`selectMostFractionalRow` returns -1), updating the incumbent to $\underline{Z} = 23.0$ and $\mathbf{x}^* = (3, 2)$.
4. **Node 3 ($x_0 \geq 4, x_1 \leq 0$, bound 22.5)**: Popped from the queue and immediately pruned by bound since $22.5 \leq \underline{Z} = 23.0$.

The solver terminates after expanding `nodesExplored()` $= 3$ nodes, certifying the optimal integer solution $\mathbf{x}^* = (3, 2)$ with $Z^* = 23.0$.

Example 5.3 *Solving a Mixed-Integer Linear Program (MILP vs. Pure ILP)*

Suppose a manufacturer decides how many modular assembly lines $x_0, x_1 \in \mathbb{Z}_{\geq 0}$ to install at two plants, together with a continuous volume $x_2 \in \mathbb{R}_{\geq 0}$ (in tons) of outsourced sub-assembly, subject to a capital budget of 10 million EUR and 17 units of specialist

```java
public class BranchAndBoundMIP {
    private static final double EPS = 1e-6;
    private final int n;
    private final boolean[] isInteger;
    private double bestObj = Double.NEGATIVE_INFINITY;
    private double[] bestPrimal = null;
    private int nodesExplored = 0;

    private record SearchNode(
            double[][] a, int[] basis, int m, int nCols,
            double bound, double[] primal)
            implements Comparable<SearchNode> {
        @Override
        public int compareTo(SearchNode other) {
            return Double.compare(other.bound, this.bound);
        }
    }

    public BranchAndBoundMIP(double[][] A, double[] b,
                             double[] c) {
        this(A, b, c, allTrue(c.length));
    }

    public BranchAndBoundMIP(double[][] A, double[] b,
                             double[] c, boolean[] isInt) {
        this.n = c.length;
        this.isInteger = isInt.clone();
        solve(A, b, c);
    }

    private void solve(double[][] A, double[] b, double[] c) {
        LinearProgramming rootLP =
            new LinearProgramming(A, b, c);
        int m0 = b.length, nCols0 = n + m0;
        PriorityQueue<SearchNode> open =
            new PriorityQueue<>();
        open.add(new SearchNode(
            rootLP.tableau(), rootLP.basis(), m0, nCols0,
            rootLP.value(), rootLP.primal()));

        while (!open.isEmpty()) {
            SearchNode node = open.poll();
            if (node.bound <= bestObj + EPS) continue;
            nodesExplored++;
            int r = selectMostFractionalRow(node);
            if (r == -1) {
                bestObj = node.bound;
                bestPrimal = node.primal;
            } else {
                double val = node.a[r][node.nCols];
                SearchNode down = branchChild(
                    node, r, -1.0, Math.floor(val) - val);
                if (down != null && down.bound > bestObj + EPS)
                    open.add(down);
                SearchNode up = branchChild(
                    node, r, +1.0, val - Math.ceil(val));
                if (up != null && up.bound > bestObj + EPS)
                    open.add(up);
            }
        }
    }
```

Listing 5.3: Best-first LP-based Branch-and-Bound MIP solver (`BranchAndBoundMIP.java`, Part 1).

```
    private int selectMostFractionalRow(SearchNode node) {
        int bestRow = -1, bestVar = Integer.MAX_VALUE;
        double minDist = Double.POSITIVE_INFINITY;
        for (int i = 0; i < node.m; i++) {
            int var = node.basis[i];
            if (var < n && isInteger[var]) {
                double val = node.a[i][node.nCols];
                double frac = val - Math.floor(val);
                if (frac > EPS && frac < 1.0 - EPS) {
                    double dist = Math.abs(frac - 0.5);
                    if (dist < minDist - EPS ||
                        (Math.abs(dist - minDist) <= EPS
                         && var < bestVar)) {
                        minDist = dist;
                        bestRow = i;
                        bestVar = var;
                    }
                }
            }
        }
        return bestRow;
    }

    private SearchNode branchChild(
            SearchNode parent, int r,
            double sign, double rhs) {
        int m = parent.m, nCols = parent.nCols;
        int newM = m + 1, newNCols = nCols + 1;
        double[][] nextA = new double[newM + 1][newNCols + 1];
        for (int i = 0; i < m; i++) {
            System.arraycopy(
                parent.a[i], 0, nextA[i], 0, nCols);
            nextA[i][newNCols] = parent.a[i][nCols];
        }
        System.arraycopy(
            parent.a[m], 0, nextA[newM], 0, nCols);
        nextA[newM][newNCols] = parent.a[m][nCols];

        int basicVar = parent.basis[r];
        for (int j = 0; j < nCols; j++)
            if (j != basicVar)
                nextA[m][j] = sign * parent.a[r][j];
        nextA[m][nCols] = 1.0;
        nextA[m][newNCols] = rhs;

        int[] nextBasis = Arrays.copyOf(parent.basis, newM);
        nextBasis[m] = nCols;
        DualSimplex ds = new DualSimplex(
            nextA, nextBasis, newM, newNCols);
        if (!ds.solve()) return null;
        return new SearchNode(nextA, nextBasis, newM,
            newNCols, ds.value(), ds.primal(n));
    }

    public double value() { return bestObj; }
    public double[] primal() { return bestPrimal.clone(); }
    public int nodesExplored() { return nodesExplored; }
}
```

Listing 5.4: Variable selection and Dual Simplex child warm-starting (`BranchAndBoundMIP.java`, Part 2).

labor:

$$\begin{aligned} \text{maximize} \quad & Z = 9x_0 + 10x_1 + 3x_2 \\ \text{subject to} \quad & 2x_0 + 3x_1 + x_2 \leq 10, \\ & 6x_0 + 4x_1 + 2x_2 \leq 17, \\ & x_0, x_1 \in \mathbb{Z}_{\geq 0}, \quad x_2 \in \mathbb{R}_{\geq 0}. \end{aligned}$$

At the root continuous LP relaxation, $x_0 = 1.1, x_1 = 2.6, x_2 = 0$ with $Z^*_{\text{LP}} = 35.9$. Using `BranchAndBoundMIP`, we can compare the Mixed-Integer formulation (where x_2 is continuous) against the pure Integer formulation (where x_2 must also be an integer):

```
double[][] A = {{2, 3, 1}, {6, 4, 2}};
double[] b = {10, 17}, c = {9, 10, 3};

// 1. Mixed-Integer (x0, x1 in Z; x2 in R):
BranchAndBoundMIP milp = new BranchAndBoundMIP(
    A, b, c, new boolean[]{true, true, false});
// -> milp.value() = 33.5, milp.primal() = [1.0, 2.0, 1.5]

// 2. Pure Integer (x0, x1, x2 in Z):
BranchAndBoundMIP ilp = new BranchAndBoundMIP(A, b, c);
// -> ilp.value() = 33.0,  ilp.primal()  = [0.0, 3.0, 1.0]
```

In the MILP, when $x_0 = 1$ and $x_1 = 2$, the second constraint $6(1) + 4(2) + 2x_2 \leq 17$ allows the continuous variable to take the fractional value $x_2^* = 1.5$, achieving $Z^*_{\text{MILP}} = 9(1) + 10(2) + 3(1.5) = 33.5$ (since `isInteger`$[2] =$ `false`, the solver does not branch on $x_2 = 1.5$). When x_2 is also restricted to $\mathbb{Z}_{\geq 0}$, $x_2 = 1.5$ is forbidden and the optimal objective drops to $Z^*_{\text{ILP}} = 33.0$ at $\mathbf{x}^*_{\text{ILP}} = (0, 3, 1)$.

5.4 Cutting Planes and Branch-and-Cut

Branch and bound partitions the search space geometrically. An orthogonal approach, pioneered by Ralph Gomory in 1958 [Gom58], is to *tighten* the continuous relaxation by adding valid inequalities that cut off fractional vertices without removing any integer feasible points.

5.4.1 Gomory Fractional Cuts

Gomory cuts possess a remarkable property: they can be derived directly from the optimal simplex tableau of the continuous relaxation without requiring problem-specific structural knowledge.

Theorem 5.4 *Gomory Fractional Cut Validity*

Let $x_i + \sum_{j \in N} \bar{a}_{i,j} x_j = \bar{b}_i$ be a row of an optimal simplex tableau where $\bar{b}_i \notin \mathbb{Z}$. Decompose each coefficient into its integer floor and fractional part:

$$\bar{a}_{i,j} = \lfloor \bar{a}_{i,j} \rfloor + f_{i,j}, \qquad \bar{b}_i = \lfloor \bar{b}_i \rfloor + f_i, \tag{5.33}$$

where $0 \leq f_{i,j} < 1$ and $0 < f_i < 1$. Then the *Gomory fractional cut*:

$$\sum_{j \in N} f_{i,j} x_j \geq f_i \tag{5.34}$$

is satisfied by every integer feasible solution $\mathbf{x} \in \mathcal{P} \cap \mathbb{Z}^n$, but strictly violates the current fractional LP solution $\mathbf{x}^*$.

Proof. Rewriting the tableau row equation:

$$x_i + \sum_{j \in N} \lfloor \bar{a}_{i,j} \rfloor x_j - \lfloor \bar{b}_i \rfloor = f_i - \sum_{j \in N} f_{i,j} x_j. \tag{5.35}$$

For any integer feasible solution $\mathbf{x} \in \mathbb{Z}^n_{\geq 0}$, the left-hand side is an integer because all variables and floor terms are integers. Hence, the right-hand side $f_i - \sum_{j \in N} f_{i,j} x_j$ must also be an integer. Because $f_i < 1$ and $f_{i,j} \geq 0, x_j \geq 0$, the right-hand side is strictly less than 1:

$$f_i - \sum_{j \in N} f_{i,j} x_j < 1. \tag{5.36}$$

An integer strictly less than 1 cannot exceed 0:

$$f_i - \sum_{j \in N} f_{i,j} x_j \leq 0 \iff \sum_{j \in N} f_{i,j} x_j \geq f_i. \tag{5.37}$$

Finally, at the current LP relaxation solution $\mathbf{x}^*$, every non-basic variable is zero ($x^*_j = 0$ for all $j \in N$). Substituting $\mathbf{x}^*$ yields $0 \geq f_i > 0$, a contradiction. Thus, $\mathbf{x}^*$ is cut off! □

Adding a slack variable $s_{\text{cut}} \geq 0$ transforms Equation (5.34) into:

$$s_{\text{cut}} - \sum_{j \in N} f_{i,j} x_j = -f_i. \tag{5.38}$$

Notice that, exactly like a branching constraint, the cut row has a negative right-hand side ($-f_i < 0$) while preserving dual feasibility ($\bar{c}_j \leq 0$). Thus, Gomory cuts are resolved using the exact same **Dual Simplex** engine!

5.4.2 The Branch-and-Cut Paradigm

Pure cutting plane methods historically suffered from slow numerical convergence due to ill-conditioning. Modern commercial and open-source solvers combine cutting planes with tree search in the **Branch-and-Cut** framework:

1. At node u, solve the continuous LP relaxation.
2. **Cut Generation Loop**: Generate valid cutting planes (Gomory cuts, knapsack cover cuts, clique cuts, flow covers). Re-optimize using Dual Simplex. Repeat until bound improvement diminishes (the "tailing-off" effect).
3. **Branch**: If the solution remains fractional, branch on a fractional variable using reliability branching.

5.5 Mixed-Integer Linear Programming Modeling

Binary variables ($x \in \{0, 1\}$) provide expressive modeling power for discrete logic:

1. Logical Implications and Indicator Constraints To enforce the logical implication "if $x = 1$, then $y = 1$", where $x, y \in \{0, 1\}$:

$$x \leq y. \tag{5.39}$$

To enforce "at most k out of n options can be selected":

$$\sum_{j=0}^{n-1} x_j \leq k. \tag{5.40}$$

2. Big-M Formulations and Numerical Pitfalls Consider linking a continuous activity level $y \in [0, U]$ to a binary decision $x \in \{0, 1\}$ (e.g., if a machine is turned off, $x = 0$, its output must be zero, $y = 0$):

$$y \leq Mx, \tag{5.41}$$

where M is a sufficiently large upper bound ($M \geq U$).

The Hazard of Large M. Choosing an unnecessarily large constant M (e.g., $M = 10^9$) causes two severe deficiencies:

1. *Weak LP Relaxation*: At fractional nodes where $x = 10^{-6}$, the constraint allows $y = 10^3$, giving a very loose upper bound and causing massive search trees.
2. *Catastrophic Numerical Cancellation*: When M exceeds solver tolerance ratios ($1/\epsilon \approx 10^8$), roundoff errors trigger false feasibility claims. Always set M to the tightest valid physical upper bound ($M = U$).

3. Strong vs. Weak Formulations: Facility Location In the Uncapacitated Facility Location Problem (UFLP), client $i \in \{0, \ldots, m-1\}$ can be served by facility $j \in \{0, \ldots, n-1\}$ ($x_{i,j} \in [0, 1]$) only if facility j is open ($y_j \in \{0, 1\}$).

- **Weak Formulation**: $\sum_{i=0}^{m-1} x_{i,j} \leq m \cdot y_j$.
- **Strong Formulation**: $x_{i,j} \leq y_j$ for all $i \in \{0, \ldots, m-1\}$.

Although the weak formulation has fewer constraints, the strong formulation defines a much tighter polyhedron: its continuous LP relaxation bound is significantly closer to the integer optimum, reducing branch-and-bound runtimes by orders of magnitude.

5.6 Chapter Summary and Exercises

5.6.1 Key Conceptual Takeaways

Table 5.1 summarizes the core principles of Integer Linear Programming.

5.6.2 Exercises

Exercise 5.1 *Total Unimodularity Proof*

Prove that the constraint matrix of the Maximum Bipartite Matching problem is totally unimodular. Use the criterion that every column contains at most two non-zero entries (± 1), and that vertices can be partitioned into sets U and V.

Concept	Key Result and Algorithmic Significance
Integer Hull $\mathcal{P}_I$	The convex hull of integer feasible points; optimizing over $\mathcal{P}_I$ is linear but **NP**-hard.
LP Relaxation	Provides an upper bound $Z^*_{\text{LP}} \geq Z^*_{\text{ILP}}$; bounding quality determines tree size.
Total Unimodularity	If $\mathbf{A}$ is TUM and $\mathbf{b} \in \mathbb{Z}^m$, all extreme points of the continuous LP are integer.
Branching Split	Partitions fractional space via $x_k \leq \lfloor x_k^* \rfloor$ and $x_k \geq \lceil x_k^* \rceil$.
Tableau Augmentation	Adding a branch constraint leaves the tableau dual feasible but primal infeasible.
Dual Simplex	Restores feasibility in 1–3 pivots without restarting from scratch (warm-starting).
Gomory Cuts	Valid inequalities derived directly from fractional tableau rows; resolved via Dual Simplex.
Branch-and-Cut	State-of-the-art MIP engine combining cut generation at search nodes with tree branching.

Table 5.1: Foundations of Integer Linear Programming and Branch-and-Cut.

Exercise 5.2 *Tableau Augmentation and Dual Simplex Pivot*

An optimal continuous simplex tableau for maximizing $Z = 3x_0 + 4x_1$ with slacks s_0, s_1 has row:

$$x_0 + \frac{2}{3}s_0 - \frac{1}{3}s_1 = \frac{7}{3}.$$

1. Formulate the down-branch constraint $x_0 \leq \lfloor 7/3 \rfloor = 2$ using a non-negative slack $s_2 \geq 0$.
2. Substitute the dictionary expression of x_0 to express the new row with basic variable s_2.
3. Identify the leaving row and apply the dual ratio test to determine the entering column.
4. Perform the dual pivot and verify that primal feasibility is restored.

Exercise 5.3 *Derivation of a Gomory Fractional Cut*

For the tableau row in Exercise 5.2:

$$x_0 + \frac{2}{3}s_0 - \frac{1}{3}s_1 = \frac{7}{3} :$$

1. Write the coefficients of $-1/3$ and $7/3$ in terms of their integer floors and fractional parts ($a = \lfloor a \rfloor + f$ with $0 \leq f < 1$).
2. Derive the corresponding Gomory fractional cut $\sum_j f_{i,j} s_j \geq f_i$.
3. Introduce slack s_{cut} and express the cut as an augmented tableau row.

Exercise 5.4 *Formulation Tightness: Facility Location*

Consider a facility location problem with 3 clients and 2 candidate facilities. Write out the constraints for both the weak formulation ($\sum_{i=0}^{2} x_{i,j} \leq 3y_j$) and the strong

formulation ($x_{i,j} \leq y_j$). Show a fractional point (x, y) that is feasible for the weak formulation relaxation but strictly infeasible for the strong formulation relaxation.

CHAPTER 6

Lagrangian Relaxation

In discrete optimization, NP-hard combinatorial problems often possess an underlying dual structure: they can be viewed as an *easy* core subproblem (such as a shortest path or a minimum spanning tree, solvable in polynomial time) entangled with a collection of *complicating* or *hard* side constraints (such as budget limits, resource capacities, or degree balance).

In Chapter 3, we observed that simply ignoring complicating constraints yields valid lower bounds for minimization problems. However, outright omission frequently produces bounds that are too loose to prune search trees effectively. *Lagrangian relaxation* provides an elegant and mathematically profound bridge between computational tractability and bound tightness. Instead of enforcing complicating constraints strictly (which induces NP-hardness) or discarding them entirely (which degrades bound quality), Lagrangian relaxation *dualizes* them into the objective function, penalizing their violation via a vector of *Lagrange multipliers*.

Originally conceived in continuous calculus by Joseph-Louis Lagrange and extended to discrete and integer programming by Everett [Eve63], Geoffrion [Geo74], and Fisher [Fis81], Lagrangian relaxation represents one of the most powerful paradigms in operational research. In this chapter, we develop the theoretical foundations of Lagrangian duality and subgradient optimization. We first illustrate the mechanics of the approach on the *Resource-Constrained Shortest Path Problem* (RCSPP). We then study its application to the Traveling Salesperson Problem (TSP), leading to the celebrated *Held–Karp lower bound* [HK70; HK71].

6.1 The Lagrangian Relaxation Principle

Consider a generic discrete minimization problem formulated as:

$$z^* = \min_{\mathbf{x}} \quad \mathbf{c}^\top \mathbf{x} \tag{6.1}$$

$$\text{subject to} \quad \mathbf{A}\mathbf{x} \leq \mathbf{b}, \tag{6.2}$$

$$\mathbf{x} \in X, \tag{6.3}$$

where $X \subseteq \mathbb{Z}^n$ represents a discrete set with special combinatorial structure (for example, the set of incidence vectors of all directed s-t paths or all spanning trees in a graph). We assume that optimizing a linear objective over X alone (without constraint (6.2)) can be performed in polynomial time by a specialized combinatorial algorithm. Conversely, the system of linear inequalities $\mathbf{A}\mathbf{x} \leq \mathbf{b}$ (with $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$) represents the *complicating constraints* that render the joint problem (6.1)–(6.3) NP-hard.

Definition 6.1 *Lagrangian Relaxation and Dual Function*

Let $\boldsymbol{\lambda} \in \mathbb{R}^m_+$ be a non-negative vector of *Lagrange multipliers* associated with the complicating constraints $\mathbf{Ax} \leq \mathbf{b}$. The *Lagrangian relaxation* of problem (6.1)–(6.3) is defined as:

$$\mathcal{L}(\boldsymbol{\lambda}) = \min_{\mathbf{x} \in X} \left\{ \mathbf{c}^\top \mathbf{x} + \boldsymbol{\lambda}^\top (\mathbf{Ax} - \mathbf{b}) \right\}. \tag{6.4}$$

The function $\mathcal{L}(\boldsymbol{\lambda})\colon \mathbb{R}^m_+ \to \mathbb{R}$ is called the *Lagrangian dual function*.

Notice that when $\boldsymbol{\lambda} \geq \mathbf{0}$, any candidate solution $\mathbf{x} \in X$ that violates the i-th complicating constraint ($(\mathbf{Ax})_i > b_i$) incurs a positive penalty $\lambda_i((\mathbf{Ax})_i - b_i) > 0$, inflating the objective value. Conversely, solutions that satisfy the constraint with strict slack receive a non-positive adjustment.

Theorem 6.1 *Weak Duality of Lagrangian Relaxation*

For any vector of non-negative multipliers $\boldsymbol{\lambda} \geq \mathbf{0}$, the value of the Lagrangian dual function provides a valid lower bound on the optimal primal objective value:

$$\mathcal{L}(\boldsymbol{\lambda}) \leq z^*. \tag{6.5}$$

Proof. Let $\mathbf{x}^*$ be an optimal solution to the primal problem (6.1)–(6.3), so that $z^* = \mathbf{c}^\top \mathbf{x}^*$. Because $\mathbf{x}^*$ is feasible for the primal problem, it satisfies $\mathbf{x}^* \in X$ and $\mathbf{Ax}^* \leq \mathbf{b}$, which implies $\mathbf{Ax}^* - \mathbf{b} \leq \mathbf{0}$. Since $\boldsymbol{\lambda} \geq \mathbf{0}$, the scalar product satisfies:

$$\boldsymbol{\lambda}^\top (\mathbf{Ax}^* - \mathbf{b}) = \sum_{i=0}^{m-1} \lambda_i ((\mathbf{Ax}^*)_i - b_i) \leq 0. \tag{6.6}$$

Therefore:

$$\mathbf{c}^\top \mathbf{x}^* + \boldsymbol{\lambda}^\top (\mathbf{Ax}^* - \mathbf{b}) \leq \mathbf{c}^\top \mathbf{x}^* = z^*. \tag{6.7}$$

By definition, $\mathcal{L}(\boldsymbol{\lambda})$ minimizes $\mathbf{c}^\top \mathbf{x} + \boldsymbol{\lambda}^\top (\mathbf{Ax} - \mathbf{b})$ over all $\mathbf{x} \in X$. Since $\mathbf{x}^* \in X$, it immediately follows that:

$$\mathcal{L}(\boldsymbol{\lambda}) = \min_{\mathbf{x} \in X} \left\{ \mathbf{c}^\top \mathbf{x} + \boldsymbol{\lambda}^\top (\mathbf{Ax} - \mathbf{b}) \right\} \leq \mathbf{c}^\top \mathbf{x}^* + \boldsymbol{\lambda}^\top (\mathbf{Ax}^* - \mathbf{b}) \leq z^*. \tag{6.8}$$

□

Handling Equality Constraints. If the complicating constraints are formulated as equalities $\mathbf{Ax} = \mathbf{b}$, the corresponding multipliers $\boldsymbol{\lambda} \in \mathbb{R}^m$ are unconstrained in sign ($\boldsymbol{\lambda} \in \mathbb{R}^m$). For any feasible solution $\mathbf{x}^*$, $\mathbf{Ax}^* - \mathbf{b} = \mathbf{0}$, so $\boldsymbol{\lambda}^\top (\mathbf{Ax}^* - \mathbf{b}) = 0$ holds regardless of the sign of $\boldsymbol{\lambda}$. Thus, $\mathcal{L}(\boldsymbol{\lambda}) \leq z^*$ holds unconditionally for all $\boldsymbol{\lambda} \in \mathbb{R}^m$.

6.1.1 The Lagrangian Dual Problem

Since $\mathcal{L}(\boldsymbol{\lambda}) \leq z^*$ for every $\boldsymbol{\lambda} \geq \mathbf{0}$, our natural goal is to identify the multiplier vector that delivers the tightest possible lower bound. This leads to the *Lagrangian dual problem*:

$$\mathcal{L}^* = \max_{\boldsymbol{\lambda} \geq \mathbf{0}} \mathcal{L}(\boldsymbol{\lambda}) = \max_{\boldsymbol{\lambda} \geq \mathbf{0}} \left\{ \min_{\mathbf{x} \in X} \left\{ \mathbf{c}^\top \mathbf{x} + \boldsymbol{\lambda}^\top (\mathbf{Ax} - \mathbf{b}) \right\} \right\}. \tag{6.9}$$

The difference $z^* - \mathcal{L}^* \geq 0$ is known as the *duality gap*. In discrete optimization, because X is non-convex, the duality gap is generally non-zero ($z^* > \mathcal{L}^*$), unlike in continuous linear programming where strong duality holds.

Proposition 6.2 *Concavity and Non-Smoothness of the Dual Function*

The Lagrangian dual function $\mathcal{L}(\boldsymbol{\lambda})$ is concave over its domain $\mathbb{R}^m_+$. Furthermore, if X is a finite discrete set, $\mathcal{L}(\boldsymbol{\lambda})$ is piecewise linear and non-differentiable at points where multiple solutions $\mathbf{x} \in X$ tie for the minimum.

Proof. Since X is finite, let $X = \{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(K-1)}\}$. For each fixed $\mathbf{x}^{(k)} \in X$, the function $f_k(\boldsymbol{\lambda}) = \mathbf{c}^\top \mathbf{x}^{(k)} + \boldsymbol{\lambda}^\top (\mathbf{A}\mathbf{x}^{(k)} - \mathbf{b})$ is affine (linear plus a constant) in $\boldsymbol{\lambda}$. The Lagrangian dual function is the pointwise minimum of this finite collection of affine functions:

$$\mathcal{L}(\boldsymbol{\lambda}) = \min_{k \in \{0,\ldots,K-1\}} f_k(\boldsymbol{\lambda}). \tag{6.10}$$

The pointwise minimum (or infimum) of any collection of affine (concave) functions is concave. At any point $\boldsymbol{\lambda}$ where two or more distinct solutions $\mathbf{x}^{(j)}, \mathbf{x}^{(k)} \in X$ achieve the minimum with different constraint residual vectors $\mathbf{A}\mathbf{x}^{(j)} \neq \mathbf{A}\mathbf{x}^{(k)}$, the function $\mathcal{L}(\boldsymbol{\lambda})$ exhibits a slope discontinuity (a "kink") and is therefore non-differentiable. □

Because $\mathcal{L}(\boldsymbol{\lambda})$ is non-differentiable, traditional gradient ascent cannot be employed to solve the dual problem (6.9). Before presenting subgradient optimization to resolve this challenge, we examine how Lagrangian relaxation operates on a concrete network optimization problem.

6.2 Resource-Constrained Shortest Path

To develop concrete intuition, consider the *Resource-Constrained Shortest Path Problem* (RCSPP). Let $G = (V, A)$ be a directed graph with vertex set $V = \{0, 1, \ldots, n-1\}$, origin node $s = 0$, and destination node $t = n - 1$. Each directed arc $(i, j) \in A$ has:

- a traversal monetary cost $c_{ij} \geq 0$, and
- a resource consumption or traversal time $t_{ij} \geq 0$.

The traveler is allotted a maximum resource budget $T > 0$. The objective is to find a directed path from origin 0 to destination $n - 1$ that minimizes total traversal cost without exceeding the budget T.

6.2.1 Mathematical Formulation

Let $x_{ij} \in \{0, 1\}$ be binary decision variables indicating whether arc $(i, j) \in A$ is traversed. The problem is formulated as an integer linear program:

$$z^* = \min_{\mathbf{x}} \quad \sum_{(i,j)\in A} c_{ij} x_{ij} \tag{6.11}$$

$$\text{subject to} \quad \sum_{j:\,(i,j)\in A} x_{ij} - \sum_{j:\,(j,i)\in A} x_{ji} = \begin{cases} 1 & \text{if } i = 0, \\ -1 & \text{if } i = n-1, \\ 0 & \text{if } i \in V \setminus \{0, n-1\}, \end{cases} \tag{6.12}$$

$$\sum_{(i,j)\in A} t_{ij} x_{ij} \leq T, \tag{6.13}$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A. \tag{6.14}$$

Constraints (6.12) are standard flow conservation equations, defining the set $\mathcal{P}$ of all directed $0 \to (n-1)$ paths. Without constraint (6.13), the problem reduces to the classical

single-source Shortest Path Problem, solvable in $\mathcal{O}(|A| + |V| \log |V|)$ time via Dijkstra's algorithm.

However, the addition of the scalar budget constraint (6.13) renders RCSPP **NP**-hard (specifically, weakly **NP**-hard by a direct reduction from the 0–1 Knapsack or Subset-Sum problem; see Ahuja et al. [AMO93]). Constraint (6.13) is therefore the complicating constraint.

6.2.2 Lagrangian Dualization

Following the Lagrangian relaxation recipe, we associate a non-negative multiplier $\lambda \geq 0$ with the budget constraint (6.13) and dualize it into the objective function:

$$\begin{aligned} \mathcal{L}(\lambda) &= \min_{\mathbf{x} \in \mathcal{P}} \left\{ \sum_{(i,j) \in A} c_{ij} x_{ij} + \lambda \left(\sum_{(i,j) \in A} t_{ij} x_{ij} - T \right) \right\} \\ &= \min_{\mathbf{x} \in \mathcal{P}} \left\{ \sum_{(i,j) \in A} (c_{ij} + \lambda t_{ij}) x_{ij} \right\} - \lambda T. \end{aligned} \tag{6.15}$$

The Penalized Shortest Path Subproblem. For any fixed multiplier value $\lambda \geq 0$, the quantity $-\lambda T$ is a constant offset. We can define modified arc costs:

$$c'_{ij}(\lambda) = c_{ij} + \lambda t_{ij} \quad \forall (i,j) \in A. \tag{6.16}$$

Because $c_{ij} \geq 0$, $t_{ij} \geq 0$, and $\lambda \geq 0$, the modified costs are strictly non-negative: $c'_{ij}(\lambda) \geq 0$. Consequently, evaluating $\mathcal{L}(\lambda)$ for any given λ requires solving a standard unconstrained Shortest Path Problem on G with arc weights $c'_{ij}(\lambda)$, which Dijkstra's algorithm solves in polynomial time!

The multiplier λ has a transparent economic interpretation: it represents the *internal shadow price* (or toll) per unit of resource consumed. When $\lambda = 0$, the algorithm ignores resource usage and finds the cheapest path, which may severely exceed the budget T. As λ increases, resource-heavy arcs become penalizingly expensive, steering the shortest path toward faster, resource-frugal trajectories.

6.2.3 A Worked Example

To trace the mechanics of Lagrangian relaxation numerically, consider the 6-vertex directed network illustrated in Figure 6.1(a). The vertices are $V = \{0, 1, 2, 3, 4, 5\}$, with origin $s = 0$, destination $t = 5$, and a resource budget $T = 14$.

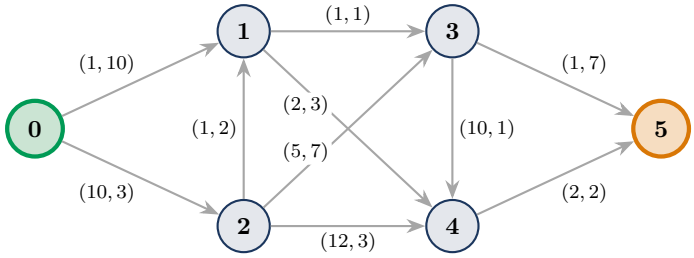
The network contains 9 directed paths connecting vertex 0 to vertex 5. Table 6.1 enumerates each path P, its original traversal cost $c(P) = \sum_{e \in P} c_e$, its resource consumption $t(P) = \sum_{e \in P} t_e$, and its Lagrangian objective value:

$$f_P(\lambda) = c(P) + \lambda(t(P) - T) = c(P) + \lambda(t(P) - 14). \tag{6.17}$$

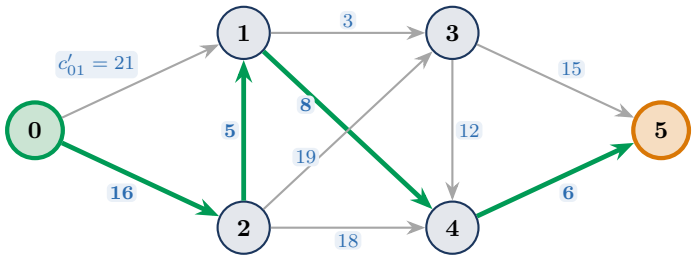
Let us analyze the behavior of $\mathcal{L}(\lambda)$ across different multiplier values:

- **At** $\lambda = 0$: The relaxation completely disregards time constraints. Dijkstra's algorithm selects path $P_0 = (0 \to 1 \to 3 \to 5)$ with unpenalized cost $\mathcal{L}(0) = 3$. However, $t(P_0) = 18 > 14$, so P_0 is severely infeasible. The lower bound 3 is valid, but very loose compared to the optimal feasible solution ($z^* = 13$, achieved by P_3).
- **At** $\lambda = 1$: Traversal time is penalized at a rate of 1 per unit. The modified cost of P_0 rises to $3 + 1 \times (18 - 14) = 7$. Path $P_1 = (0 \to 1 \to 4 \to 5)$ now wins with modified cost $5 + 1 \times (15 - 14) = 6$. The lower bound tightens from 3 to 6.

(a) Original Network: Arc Weights (c_{ij}, t_{ij}), Budget $T = 14$



(b) Lagrangian Subproblem: Modified Costs $c'_{ij} = c_{ij} + 2t_{ij}$



Shortest Path at $\lambda = 2$: $P^* = (0 \to 2 \to 1 \to 4 \to 5)$, Cost $c'(P^*) = 16 + 5 + 8 + 6 = 35$
Lagrangian Lower Bound: $\mathcal{L}(\lambda = 2) = c'(P^*) - \lambda T = 35 - 2 \times 14 = \mathbf{7}$
Original Metrics: $c(P^*) = 10 + 1 + 2 + 2 = 15$, $t(P^*) = 3 + 2 + 3 + 2 = 10 \leq 14$ (Primal Feasible!)

Figure 6.1: The Resource-Constrained Shortest Path network. Panel (a) shows the original arc parameters (c_{ij}, t_{ij}) with resource budget $T = 14$. Panel (b) shows the modified arc costs $c'_{ij} = c_{ij} + 2t_{ij}$ for multiplier $\lambda = 2$. The optimal shortest path under modified weights (highlighted in green) yields the optimal dual lower bound $\mathcal{L}(2) = 7$.

Table 6.1: All directed paths from origin 0 to destination 5 with traversal metrics and Lagrangian evaluation across multipliers $\lambda \in \{0, 1, 2, 3\}$. Feasible paths satisfying $t(P) \leq 14$ are highlighted in bold.

Path P	**Cost** $c(P)$	**Time** $t(P)$	$\lambda = 0$	$\lambda = 1$	$\lambda = 2$	$\lambda = 3$
$P_0 : 0 \to 1 \to 3 \to 5$	3	18	**3**	7	11	15
$P_1 : 0 \to 1 \to 4 \to 5$	5	15	5	**6**	**7**	8
$\mathbf{P_2 : 0 \to 1 \to 3 \to 4 \to 5}$	**14**	**14**	14	14	14	14
$\mathbf{P_3 : 0 \to 2 \to 1 \to 3 \to 5}$	**13**	**13**	13	12	11	10
$\mathbf{P_4 : 0 \to 2 \to 1 \to 4 \to 5}$	**15**	**10**	15	11	**7**	**3**
$\mathbf{P_5 : 0 \to 2 \to 1 \to 3 \to 4 \to 5}$	**24**	**9**	24	19	14	9
$P_6 : 0 \to 2 \to 3 \to 5$	16	17	16	19	22	25
$\mathbf{P_7 : 0 \to 2 \to 3 \to 4 \to 5}$	**27**	**13**	27	26	25	24
$\mathbf{P_8 : 0 \to 2 \to 4 \to 5}$	**24**	**8**	24	18	12	6
Dual Lower Bound $\mathcal{L}(\lambda)$			**3**	**6**	**7**	**3**

- **At $\lambda = 2$:** As shown in Figure 6.1(b), the modified costs on arcs change significantly: $c'_{01} = 21$, while $c'_{02} = 16$. Paths P_1 and P_4 tie for the minimum:

$$f_{P_1}(2) = 5 + 2 \times (15 - 14) = 7, \tag{6.18}$$
$$f_{P_4}(2) = 15 + 2 \times (10 - 14) = 7. \tag{6.19}$$

  The lower bound reaches $\mathcal{L}(2) = 7$. Furthermore, path P_4 consumes $t(P_4) = 10 \leq 14$ time units, providing our first primal feasible path with actual cost $c(P_4) = 15$!

- **At $\lambda = 3$:** Time consumption is penalized so aggressively that the fast, high-cost path P_4 ($t = 10$) dominates, but its dual value drops to $15+3\times(10-14) = 3$. Over-penalizing resource consumption causes the dual bound to decrease!

6.2.4 Geometry of the Lagrangian Dual

Plotting $\mathcal{L}(\lambda)$ as a function of $\lambda \geq 0$ reveals the characteristic structure of Lagrangian duality, as shown in Figure 6.2. Each path P_k corresponds to a straight line of slope $t(P_k) - T$. The Lagrangian dual function $\mathcal{L}(\lambda)$ is the lower envelope (pointwise minimum) of these lines:

$$\mathcal{L}(\lambda) = \min_{k \in \{0,\ldots,8\}} \{c(P_k) + \lambda(t(P_k) - 14)\}. \tag{6.20}$$

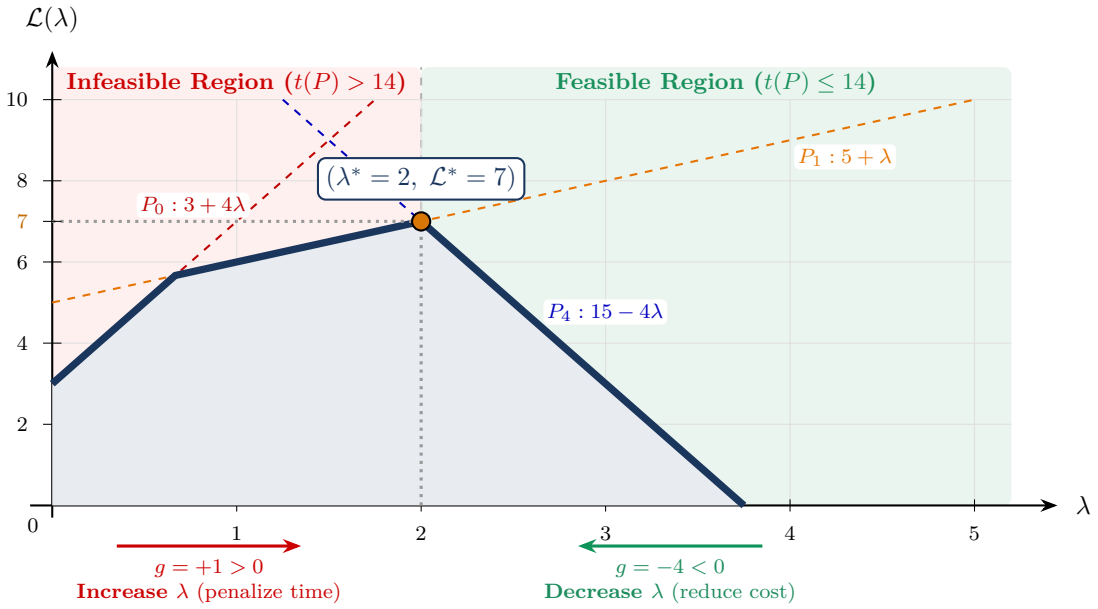


Figure 6.2: The concave piecewise-linear Lagrangian dual function $\mathcal{L}(\lambda)$ for the RCSPP example. The optimal multiplier $\lambda^* = 2$ achieves the peak lower bound $\mathcal{L}^* = 7$. For $\lambda < 2$, the optimal path violates the budget ($t(P) > T$), yielding a positive subgradient that increases λ. For $\lambda > 2$, the subgradient is negative, steering λ back toward the optimum.

The lower envelope is concave and non-smooth. Its maximum occurs at the intersection of the line for P_1 (slope $15 - 14 = +1$) and the line for P_4 (slope $10 - 14 = -4$):

$$5 + 1 \cdot (\lambda - 0) = 15 - 4\lambda \implies 5\lambda = 10 \implies \lambda^* = 2. \tag{6.21}$$

The optimal Lagrangian bound is $\mathcal{L}^* = \mathcal{L}(2) = 7$.

6.3 Solving the Dual via Subgradients

To find the optimal multiplier $\boldsymbol{\lambda}^*$ that maximizes the concave dual function $\mathcal{L}(\boldsymbol{\lambda})$, we cannot compute standard gradients because $\mathcal{L}(\boldsymbol{\lambda})$ has non-differentiable vertices ("kinks") at every point where multiple primal solutions tie for the minimum.

6.3.1 Subgradients and the Subdifferential

Non-smooth convex and concave optimization relies on the concept of a *subgradient* (see Shor [Sho85]).

Definition 6.2 *Subgradient of a Concave Function*

Let $f\colon \mathbb{R}^m \to \mathbb{R}$ be a concave function. A vector $\mathbf{g} \in \mathbb{R}^m$ is called a *subgradient* of f at point $\boldsymbol{\lambda}$ if for all $\boldsymbol{\lambda}' \in \mathbb{R}^m$:

$$f(\boldsymbol{\lambda}') \leq f(\boldsymbol{\lambda}) + \mathbf{g}^\top(\boldsymbol{\lambda}' - \boldsymbol{\lambda}). \tag{6.22}$$

The set of all subgradients of f at $\boldsymbol{\lambda}$ is called the *subdifferential*, denoted by $\partial f(\boldsymbol{\lambda})$.

Condition (6.22) states that the affine hyper-plane $\boldsymbol{\lambda}' \mapsto f(\boldsymbol{\lambda}) + \mathbf{g}^\top(\boldsymbol{\lambda}' - \boldsymbol{\lambda})$ lies everywhere above (or tangent to) the concave function f. A fundamental theorem of non-smooth optimization states that $\boldsymbol{\lambda}^*$ is a global maximizer of a concave function f if and only if $\mathbf{0} \in \partial f(\boldsymbol{\lambda}^*)$.

Theorem 6.3 *Subgradient of the Lagrangian Dual Function*

Let $\boldsymbol{\lambda} \geq \mathbf{0}$, and let $\mathbf{x}(\boldsymbol{\lambda}) \in X$ be any optimal primal solution to the Lagrangian subproblem (6.4), so that $\mathcal{L}(\boldsymbol{\lambda}) = \mathbf{c}^\top\mathbf{x}(\boldsymbol{\lambda}) + \boldsymbol{\lambda}^\top(\mathbf{A}\mathbf{x}(\boldsymbol{\lambda}) - \mathbf{b})$. Then the constraint residual vector:

$$\mathbf{g}(\boldsymbol{\lambda}) = \mathbf{A}\mathbf{x}(\boldsymbol{\lambda}) - \mathbf{b} \tag{6.23}$$

is a valid subgradient of $\mathcal{L}$ at $\boldsymbol{\lambda}$, that is, $\mathbf{g}(\boldsymbol{\lambda}) \in \partial\mathcal{L}(\boldsymbol{\lambda})$.

Proof. Let $\boldsymbol{\lambda}' \geq \mathbf{0}$ be any arbitrary multiplier vector. By definition of the dual function:

$$\begin{aligned}\mathcal{L}(\boldsymbol{\lambda}') &= \min_{\mathbf{x}\in X}\left\{\mathbf{c}^\top\mathbf{x} + (\boldsymbol{\lambda}')^\top(\mathbf{A}\mathbf{x} - \mathbf{b})\right\} \\ &\leq \mathbf{c}^\top\mathbf{x}(\boldsymbol{\lambda}) + (\boldsymbol{\lambda}')^\top(\mathbf{A}\mathbf{x}(\boldsymbol{\lambda}) - \mathbf{b}),\end{aligned} \tag{6.24}$$

where the inequality holds because $\mathbf{x}(\boldsymbol{\lambda}) \in X$ is a feasible (though not necessarily optimal) candidate for the subproblem under $\boldsymbol{\lambda}'$. Subtracting $\mathcal{L}(\boldsymbol{\lambda}) = \mathbf{c}^\top\mathbf{x}(\boldsymbol{\lambda}) + \boldsymbol{\lambda}^\top(\mathbf{A}\mathbf{x}(\boldsymbol{\lambda}) - \mathbf{b})$ from both sides yields:

$$\begin{aligned}\mathcal{L}(\boldsymbol{\lambda}') - \mathcal{L}(\boldsymbol{\lambda}) &\leq (\boldsymbol{\lambda}')^\top(\mathbf{A}\mathbf{x}(\boldsymbol{\lambda}) - \mathbf{b}) - \boldsymbol{\lambda}^\top(\mathbf{A}\mathbf{x}(\boldsymbol{\lambda}) - \mathbf{b}) \\ &= (\boldsymbol{\lambda}' - \boldsymbol{\lambda})^\top(\mathbf{A}\mathbf{x}(\boldsymbol{\lambda}) - \mathbf{b}) = \mathbf{g}(\boldsymbol{\lambda})^\top(\boldsymbol{\lambda}' - \boldsymbol{\lambda}).\end{aligned} \tag{6.25}$$

Rearranging gives $\mathcal{L}(\boldsymbol{\lambda}') \leq \mathcal{L}(\boldsymbol{\lambda}) + \mathbf{g}(\boldsymbol{\lambda})^\top(\boldsymbol{\lambda}' - \boldsymbol{\lambda})$, confirming that $\mathbf{g}(\boldsymbol{\lambda}) \in \partial\mathcal{L}(\boldsymbol{\lambda})$. □

Theorem 6.3 delivers an extraordinarily practical result: *we obtain a valid subgradient for free whenever we solve the Lagrangian subproblem!* For the RCSPP, if P_k is the shortest path computed under multiplier λ_k, the subgradient is simply the scalar constraint violation:

$$g_k = t(P_k) - T. \tag{6.26}$$

- If $t(P_k) > T$, the path exceeds the budget, so $g_k > 0$. The subgradient indicates that λ should be *increased* to penalize time more heavily.
- If $t(P_k) < T$, the path consumes less than the allotted budget, so $g_k < 0$. The subgradient indicates that λ should be *decreased* to seek lower monetary costs.

6.3.2 The Subgradient Iteration Algorithm

The subgradient optimization algorithm iteratively updates the multiplier vector in the direction of the subgradient, projecting the result back onto the non-negative orthant $\mathbb{R}^m_+$:

$$\boldsymbol{\lambda}^{(k+1)} = \max\left\{\mathbf{0},\ \boldsymbol{\lambda}^{(k)} + \mu_k \mathbf{g}^{(k)}\right\}, \tag{6.27}$$

where $\mu_k > 0$ is a scalar *step size* at iteration k.

Unlike smooth gradient ascent, the sequence of dual values $\mathcal{L}(\boldsymbol{\lambda}^{(k)})$ is *not guaranteed to increase monotonically* at every iteration (see Figure 6.2). However, if the step size sequence satisfies the classical convergence conditions:

$$\lim_{k\to\infty} \mu_k = 0 \quad \text{and} \quad \sum_{k=0}^{\infty} \mu_k = \infty, \tag{6.28}$$

then the sequence $\boldsymbol{\lambda}^{(k)}$ is guaranteed to converge to an optimal multiplier $\boldsymbol{\lambda}^*$ (Shor [Sho85]). A common step size rule meeting these criteria is the harmonic series $\mu_k = \mu_0/(k+1)$.

Algorithm 9 outlines the complete subgradient optimization framework for RCSPP. In addition to computing the dual bound $\underline{L}$, the algorithm tracks the best primal feasible path $\bar{P}$ discovered during execution, maintaining an optimality gap $(\mathbf{c}(\bar{P}) - \underline{L})/\mathbf{c}(\bar{P})$.

Binary Search for Single Complicating Constraints. When an optimization problem has only a *single* scalar multiplier $\lambda \in \mathbb{R}_+$ (as in RCSPP), $\mathcal{L}(\lambda)$ is a one-dimensional concave function. In that special case, one can alternatively apply binary search, golden section search, or ternary search on the interval $[0, \lambda_{\max}]$ to find λ^* rapidly without tuning step sizes. However, subgradient optimization remains indispensable for combinatorial problems involving hundreds or thousands of multipliers, such as the Traveling Salesperson Problem.

6.4 Lagrangian Relaxation for the TSP

We now apply Lagrangian relaxation to the Traveling Salesperson Problem on an undirected complete graph $G = (V, E)$ with edge costs $w_e \geq 0$ and $|V| = n$, where vertices are indexed from 0 to $n - 1$.

6.4.1 Structural Constraints of the TSP

In Chapter 3, we recognized that any valid Hamiltonian cycle corresponds to an edge subset $T \subseteq E$ of cardinality $|T| = n$ that simultaneously satisfies two distinct structural conditions:

1. **Subtour-Free Connectivity**: The selected edges must connect all vertices into a single cycle without isolated sub-circuits. As established in Theorem 3.8, this is equivalent to requiring the edge set to form a *1-tree* (a spanning tree on $V \setminus \{0\}$ combined with two distinct edges incident to vertex 0).
2. **Degree Regularity**: Every vertex $i \in V$ must have degree exactly 2:

$$\deg(i) = \sum_{e \in \delta(i)} x_e = 2 \quad \forall i \in V. \tag{6.29}$$

Algorithm 9: Subgradient Optimization for the RCSPP

Input: Directed graph $G = (V, A)$, costs c_{ij}, traversal times t_{ij}, budget T, tolerance ϵ, max iterations K

Output: Best lower bound $\underline{L}$, best feasible path $\bar{P}$

$\underline{L} \leftarrow -\infty$
$\bar{P} \leftarrow$ null, $\quad \bar{z} \leftarrow +\infty$
$\lambda_0 \leftarrow 0, \quad k \leftarrow 0$
while $k < K$ ***and*** *step size* $\mu_k > \epsilon$ **do**
  Define modified costs $c'_{ij} = c_{ij} + \lambda_k t_{ij}$ for all $(i, j) \in A$
  $P_k \leftarrow \text{ShortestPath}(G, c', 0, n-1)$ `// Dijkstra's algorithm`
  $\mathcal{L}_k \leftarrow c(P_k) + \lambda_k (t(P_k) - T)$
  if $\mathcal{L}_k > \underline{L}$ **then**
    $\underline{L} \leftarrow \mathcal{L}_k$ `// Update best dual lower bound`
  if $t(P_k) \leq T$ ***and*** $c(P_k) < \bar{z}$ **then**
    $\bar{P} \leftarrow P_k, \quad \bar{z} \leftarrow c(P_k)$ `// Update best primal feasible solution`
  if $\bar{z} - \underline{L} \leq \epsilon$ **then**
    break `// Certified optimality within tolerance`
  $g_k \leftarrow t(P_k) - T$ `// Compute subgradient`
  $\mu_k \leftarrow 1.0/(k+1)$ `// Divergent step size`
  $\lambda_{k+1} \leftarrow \max(0,\ \lambda_k + \mu_k g_k)$
  $k \leftarrow k + 1$
return $\underline{L}, \bar{P}$

The minimum 1-tree problem satisfies the connectivity requirement and can be solved in polynomial time $\mathcal{O}(|E| \log |V|)$ using Kruskal's or Prim's algorithm (Appendix A). However, an unconstrained minimum 1-tree frequently violates the degree regularity constraints: some vertices may end up as leaves ($\deg(i) = 1$), while others become hubs with degree 3, 4, or higher.

6.4.2 Dualizing Degree Constraints

To eliminate these degree violations without sacrificing the polynomial tractability of 1-tree computations, Michael Held and Richard Karp [HK70; HK71] proposed dualizing the n degree constraints (6.29).

Let $\boldsymbol{\pi} = (\pi_0, \pi_1, \ldots, \pi_{n-1}) \in \mathbb{R}^n$ be a vector of Lagrange multipliers, where each π_i is associated with the degree constraint of vertex $i \in V$. Because degree constraints are *equalities* ($\deg(i) = 2$), the multipliers π_i are *unconstrained in sign*: $\pi_i \in \mathbb{R}$.

Formally, the primal TSP is stated as:

$$z^* = \min_{\mathbf{x}} \quad \sum_{e \in E} w_e x_e \tag{6.30}$$

$$\text{subject to} \quad \sum_{e \in \delta(i)} x_e = 2 \quad \forall i \in V, \tag{6.31}$$

$$\mathbf{x} \text{ is the incidence vector of a 1-tree.} \tag{6.32}$$

Dualizing the degree equations (6.31) yields the Lagrangian dual function:

$$\mathcal{L}(\boldsymbol{\pi}) = \min_{\mathbf{x}\in 1\text{-Tree}} \left\{ \sum_{e\in E} w_e x_e + \sum_{i\in V} \pi_i \left(2 - \sum_{e\in\delta(i)} x_e \right) \right\}. \tag{6.33}$$

We now rewrite the dualized penalty term. Since each edge $e = \{i, j\}$ is incident to exactly two endpoints i and j, summing over all vertices distributes the multiplier contributions across edges:

$$\sum_{i\in V} \pi_i \sum_{e\in\delta(i)} x_e = \sum_{\{i,j\}\in E} (\pi_i + \pi_j) x_{ij}. \tag{6.34}$$

Substituting this identity into (6.33) produces:

$$\begin{aligned} \mathcal{L}(\boldsymbol{\pi}) &= \min_{\mathbf{x}\in 1\text{-Tree}} \left\{ \sum_{\{i,j\}\in E} w_{ij} x_{ij} - \sum_{\{i,j\}\in E} (\pi_i + \pi_j) x_{ij} + 2 \sum_{i\in V} \pi_i \right\} \\ &= \min_{\mathbf{x}\in 1\text{-Tree}} \left\{ \sum_{\{i,j\}\in E} (w_{ij} - \pi_i - \pi_j) x_{ij} \right\} + 2 \sum_{i\in V} \pi_i. \end{aligned} \tag{6.35}$$

6.4.3 Modified Edge Costs and Multipliers

Formula (6.35) reveals the brilliance of the Held–Karp relaxation. For any fixed multiplier vector $\boldsymbol{\pi}$, the Lagrangian subproblem is simply a *Minimum 1-Tree Problem* evaluated with *modified edge weights*:

$$w'_{ij} = w_{ij} - \pi_i - \pi_j \quad \forall \{i, j\} \in E, \tag{6.36}$$

to which the constant offset $2\sum_{i\in V} \pi_i$ is added.

The mechanical intuition is immediately apparent:

- If vertex i has too high a degree ($\deg(i) > 2$) in the current 1-tree, it is an over-utilized hub. To discourage edges incident to i, we decrease π_i. Because $w'_{ij} = w_{ij} - \pi_i - \pi_j$, decreasing π_i *increases* the cost of all edges incident to i, making them less attractive in subsequent MST selections!
- Conversely, if vertex i has too low a degree ($\deg(i) = 1$), it is an under-utilized leaf. Increasing π_i *decreases* the modified cost of incident edges, making them cheaper and encouraging the 1-tree to incorporate them.

6.4.4 Multiplier Invariance and Optimality

A remarkable mathematical property of the modified weights (6.36) is that they preserve the exact relative costs of all valid Hamiltonian tours.

Theorem 6.4 *Tour Length Invariance under Held–Karp Multipliers*

Let $H \subset E$ be any Hamiltonian cycle in G. For any multiplier vector $\boldsymbol{\pi} \in \mathbb{R}^n$, the total modified weight of H plus the dual offset equals its exact original weight:

$$\sum_{\{i,j\}\in H} w'_{ij} + 2 \sum_{i\in V} \pi_i = \sum_{\{i,j\}\in H} w_{ij}. \tag{6.37}$$

Furthermore, if the multipliers satisfy $\sum_{i \in V} \pi_i = 0$, then:

$$\sum_{\{i,j\} \in H} w'_{ij} = \sum_{\{i,j\} \in H} w_{ij}. \tag{6.38}$$

Proof. Because H is a Hamiltonian cycle, every vertex $i \in V$ has degree exactly 2 ($\deg_H(i) = 2$). Summing the modified edge weights over H:

$$\begin{aligned} \sum_{\{i,j\} \in H} w'_{ij} &= \sum_{\{i,j\} \in H} (w_{ij} - \pi_i - \pi_j) \\ &= \sum_{\{i,j\} \in H} w_{ij} - \sum_{i \in V} \deg_H(i) \pi_i \\ &= \sum_{\{i,j\} \in H} w_{ij} - 2 \sum_{i \in V} \pi_i. \end{aligned} \tag{6.39}$$

Adding $2 \sum_{i \in V} \pi_i$ to both sides cancels the multiplier terms completely, establishing the theorem. When $\sum_{i \in V} \pi_i = 0$, the offset is zero, so the modified tour cost equals the original tour cost identically. □

Theorem 6.5 *Held–Karp Optimality Certificate*

Suppose that for some multiplier vector $\boldsymbol{\pi} \in \mathbb{R}^n$, the minimum 1-tree T^* computed under modified weights w' has degree $\deg_{T^*}(i) = 2$ for all $i \in V$. Then T^* is an optimal TSP tour, and $\mathcal{L}(\boldsymbol{\pi}) = z^*$.

Proof. If $\deg_{T^*}(i) = 2$ for all $i \in V$, then T^* is a connected 2-regular graph on n vertices, which is by definition a Hamiltonian cycle. By Theorem 6.4, the cost of T^* in the original graph is:

$$w(T^*) = \sum_{e \in T^*} w'_e + 2 \sum_{i \in V} \pi_i = \mathcal{L}(\boldsymbol{\pi}). \tag{6.40}$$

Since T^* is a valid TSP tour, its cost satisfies $w(T^*) \geq z^*$. On the other hand, by weak duality (Theorem 6.1), $\mathcal{L}(\boldsymbol{\pi}) \leq z^*$. Combining these inequalities yields:

$$z^* \leq w(T^*) = \mathcal{L}(\boldsymbol{\pi}) \leq z^* \implies \mathcal{L}(\boldsymbol{\pi}) = z^* = w(T^*). \tag{6.41}$$

Thus, T^* is provably optimal. □

6.4.5 A Worked Example on 4 Vertices

Figure 6.3 visualizes how Lagrangian multipliers tighten the 1-tree bound on a complete 4-vertex graph K_4 with $V = \{0, 1, 2, 3\}$. The original edge weights are:

$$w_{01} = 7, \quad w_{02} = 4, \quad w_{03} = 9, \quad w_{12} = 5, \quad w_{13} = 8, \quad w_{23} = 6.$$

Example 6.1 *Tightening the 1-Tree Bound via Multipliers*

We trace two iterations of the Held–Karp bounding procedure:

1. **Iteration 0 ($\boldsymbol{\pi} = \mathbf{0}$)**:

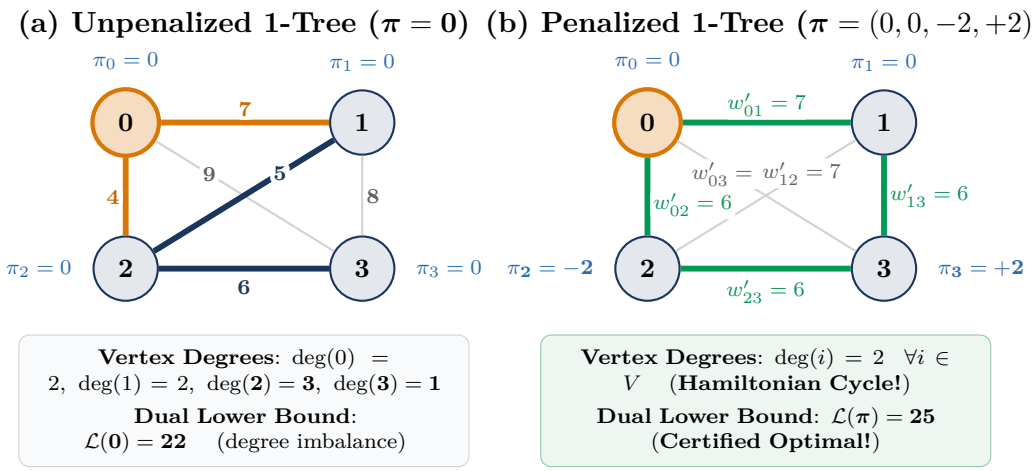


Figure 6.3: Lagrangian relaxation and multiplier transformation for a 4-vertex TSP. Panel (a): unpenalized 1-tree ($\boldsymbol{\pi} = \mathbf{0}$) with degree imbalance at vertices 2 and 3 ($\mathcal{L} = 22$). Panel (b): after introducing multipliers $\boldsymbol{\pi} = (0, 0, -2, +2)$, the minimum 1-tree under modified weights $w'_{ij} = w_{ij} - \pi_i - \pi_j$ becomes 2-regular, yielding a certified optimal Hamiltonian cycle of cost $\mathcal{L} = 25$.

- Two cheapest edges incident to root vertex 0: $\{0, 2\}$ ($w = 4$) and $\{0, 1\}$ ($w = 7$).
- Minimum Spanning Tree on $V \setminus \{0\} = \{1, 2, 3\}$: edges $\{1, 2\}$ ($w = 5$) and $\{2, 3\}$ ($w = 6$) are chosen (edge $\{1, 3\}$ with $w = 8$ is omitted).
- The unpenalized 1-tree contains edges $\{\{0, 2\}, \{0, 1\}, \{1, 2\}, \{2, 3\}\}$ with cost:

$$\mathcal{L}(\mathbf{0}) = 4 + 7 + 5 + 6 = 22.$$

- Vertex degrees in this 1-tree: $\deg(0) = 2$, $\deg(1) = 2$, $\deg(2) = 3$, $\deg(3) = 1$.
- Degree violations: vertex 2 has an excess of edges ($\deg(2) - 2 = +1$), while vertex 3 has a deficit ($\deg(3) - 2 = -1$).

2. **Iteration 1 ($\boldsymbol{\pi} = (0, 0, -2, +2)$)**: Applying a step size $\mu = 2$ in the subgradient direction $g_i = 2 - \deg(i)$ yields:

$$\pi_0 = 0, \quad \pi_1 = 0, \quad \pi_2 = 0 + 2(2 - 3) = -2, \quad \pi_3 = 0 + 2(2 - 1) = +2.$$

Notice that $\sum_{i=0}^{3} \pi_i = 0 + 0 - 2 + 2 = 0$. The modified edge weights $w'_{ij} = w_{ij} - \pi_i - \pi_j$ become:

$$\begin{aligned} w'_{01} &= 7 - 0 - 0 = 7, & w'_{02} &= 4 - 0 - (-2) = 6, \\ w'_{03} &= 9 - 0 - 2 = 7, & w'_{12} &= 5 - 0 - (-2) = 7, \\ w'_{13} &= 8 - 0 - 2 = 6, & w'_{23} &= 6 - (-2) - 2 = 6. \end{aligned}$$

We recompute the minimum 1-tree under w':

- Two cheapest edges from vertex 0: $\{0, 2\}$ ($w' = 6$) and $\{0, 1\}$ ($w' = 7$) [or $\{0, 3\}$ ($w' = 7$)].
- MST on $\{1, 2, 3\}$: edges $\{1, 3\}$ ($w' = 6$) and $\{2, 3\}$ ($w' = 6$).

- Total modified cost: $6 + 7 + 6 + 6 + 2\sum_i \pi_i = 25 + 2(0) = 25$.

The lower bound jumped from 22 to 25! Furthermore, the resulting 1-tree has edges $\{\{0,1\},\{1,3\},\{2,3\},\{0,2\}\}$, which forms a simple cycle of length 4: $0-1-3-2-0$. By Theorem 6.5, this tour is certified optimal with original length $7+8+6+4=25$.

6.5 Subgradient Optimization for Held–Karp

To maximize $\mathcal{L}(\boldsymbol{\pi})$ across all multiplier vectors $\boldsymbol{\pi} \in \mathbb{R}^n$, we deploy subgradient optimization.

6.5.1 Subgradients and Zero-Sum Property

By Theorem 6.3, for any current multiplier vector $\boldsymbol{\pi}$, the subgradient $\mathbf{g} \in \mathbb{R}^n$ of $\mathcal{L}(\boldsymbol{\pi})$ is the degree residual:

$$g_i = 2 - \deg_T(i) \quad \forall i \in V, \tag{6.42}$$

where $\deg_T(i)$ is the degree of vertex i in the minimum 1-tree T computed under modified weights $w'_{ij} = w_{ij} - \pi_i - \pi_j$.

Lemma 6.6 *Conservation of Multiplier Zero-Sum*

In any 1-tree T on n vertices, the sum of all vertex degrees is identically equal to $2n$. Consequently:

$$\sum_{i \in V} g_i = \sum_{i \in V} (2 - \deg_T(i)) = 2n - \sum_{i \in V} \deg_T(i) = 2n - 2n = 0. \tag{6.43}$$

Therefore, if the initial multiplier vector satisfies $\sum_{i \in V} \pi_i^{(0)} = 0$, then under the subgradient update rule:

$$\boldsymbol{\pi}^{(k+1)} = \boldsymbol{\pi}^{(k)} + \mu_k \mathbf{g}^{(k)}, \tag{6.44}$$

the zero-sum property is strictly preserved for all iterations $k \geq 0$:

$$\sum_{i \in V} \pi_i^{(k+1)} = \sum_{i \in V} \pi_i^{(k)} + \mu_k \sum_{i \in V} g_i^{(k)} = 0 + \mu_k(0) = 0. \tag{6.45}$$

Theorem 6.6 is remarkable: it guarantees that by simply initializing $\boldsymbol{\pi}^{(0)} = \mathbf{0}$, the multipliers remain zero-sum throughout all iterations automatically. No projection or normalization step is ever required!

6.5.2 Adaptive Step Size Control

In practice, subgradient convergence depends critically on the step size μ_k. While the theoretical harmonic step $\mu_k = \mu_0/(k+1)$ guarantees asymptotic convergence, its practical progress is often slow. For the Held–Karp bound, Polyak's adaptive step size rule [Pol87], as refined by Held and Karp [HK71] and Valenzuela and Jones [VJ97], provides vastly superior numerical performance:

$$\mu_k = \frac{\alpha_k(\mathrm{UB} - \mathcal{L}(\boldsymbol{\pi}^{(k)}))}{\sum_{i \in V}(2 - \deg_T(i))^2}, \tag{6.46}$$

where:

- UB is a known primal upper bound (the length of any valid TSP tour, obtained quickly via a heuristic such as nearest neighbor or 2-opt; see Chapter 8). If no upper bound is available, one can replace $(\mathrm{UB} - \mathcal{L}_k)$ by $\mathcal{L}_k$ as in Valenzuela and Jones [VJ97].
- $\sum_{i \in V} (2 - \deg_T(i))^2 = \|\mathbf{g}^{(k)}\|_2^2$ is the squared Euclidean norm of the subgradient vector.
- $\alpha_k \in (0, 2]$ is a scalar parameter initialized to $\alpha_0 = 2.0$ (or 1.0). If the lower bound fails to strictly improve over a predetermined window of iterations (e.g., 5 or 10 iterations), α is decayed: $\alpha_{k+1} \leftarrow 0.9 \cdot \alpha_k$ or $\alpha_{k+1} \leftarrow 0.5 \cdot \alpha_k$.

Figure 6.4 illustrates the convergence trajectory of subgradient optimization. Notice how the instantaneous bound $\mathcal{L}_k = \mathcal{L}(\boldsymbol{\pi}^{(k)})$ oscillates non-monotonically, while the best lower bound $\underline{\mathcal{L}}_k = \max_{j \le k} \mathcal{L}_j$ monotonically ascends in discrete jumps toward the optimal Held–Karp bound $\mathcal{L}^*$.

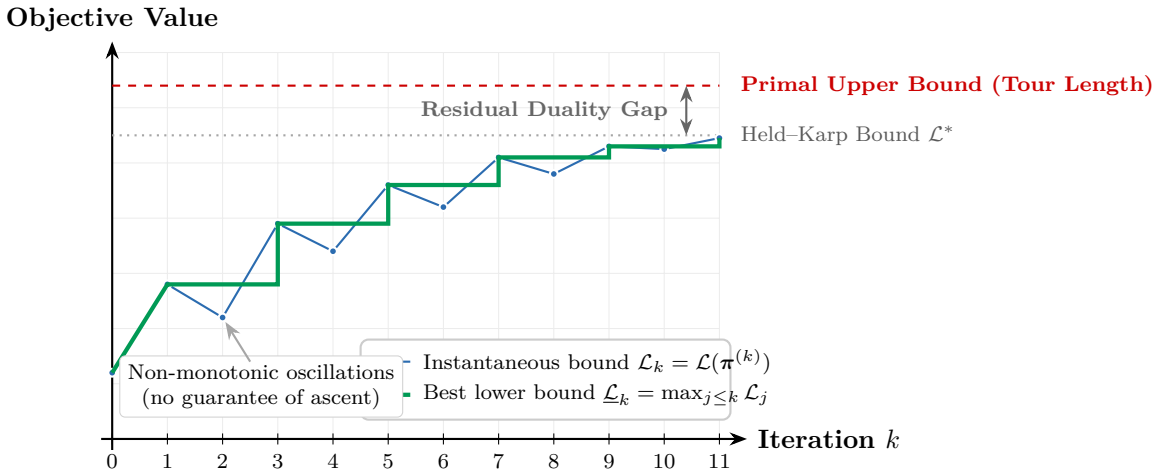


Figure 6.4: Subgradient optimization convergence for the Held–Karp TSP lower bound. The instantaneous dual value $\mathcal{L}_k$ oscillates non-monotonically due to slope discontinuities at vertices of the 1-tree polyhedron. The running best bound $\underline{\mathcal{L}}_k$ ascends monotonically toward $\mathcal{L}^*$, closing the gap against the primal upper bound.

6.5.3 The Held–Karp Subgradient Procedure

The subgradient optimization framework for the Held–Karp 1-tree relaxation proceeds iteratively. Starting from the unpenalized baseline ($\boldsymbol{\pi}^{(0)} = \mathbf{0}$), it repeatedly evaluates the minimum 1-tree under penalized costs and steers the multipliers along the subgradient direction.

The complete algorithmic procedure unfolds in the following systematic steps:

1. **Initialization**:
   - Set initial multiplier vector $\boldsymbol{\pi}^{(0)} = (0, 0, \ldots, 0) \in \mathbb{R}^n$.
   - Initialize the best recorded lower bound $\underline{\mathcal{L}} \leftarrow -\infty$.
   - Set an initial step-size scalar $\alpha_0 \in (0, 2]$ (typically $\alpha_0 = 2.0$), a decay factor $\gamma \in (0, 1)$ (e.g., $\gamma = 0.7$ or 0.5), and a non-improvement window parameter p (e.g., $p = 5$ or 10).
   - Compute or retrieve a heuristic upper bound UB (such as via nearest-neighbor followed by 2-opt local search; see Chapter 8).

2. **Edge Weight Transformation**: At iteration $k \geq 0$, modify the edge costs using the current multiplier vector $\boldsymbol{\pi}^{(k)}$:

$$w'_{ij} = w_{ij} - \pi_i^{(k)} - \pi_j^{(k)} \quad \forall \{i, j\} \in E. \tag{6.47}$$

3. **Minimum 1-Tree Subproblem Resolution**:
   - Select designated root vertex $r \in V$ (conventionally $r = 0$).
   - Compute a Minimum Spanning Tree T_{MST} on the induced subgraph $G[V \setminus \{0\}]$ under modified weights w' using Kruskal's algorithm with a disjoint-set (Union-Find) data structure (as detailed in Appendix A) or Prim's algorithm in $\mathcal{O}(n^2)$ time.
   - Identify the two distinct edges $e_1 = \{0, u\}$ and $e_2 = \{0, v\}$ $(u \neq v)$ incident to root vertex 0 that minimize w'.
   - Form the minimum 1-tree $T^{(k)} = T_{\text{MST}} \cup \{e_1, e_2\}$.
4. **Dual Lower Bound Evaluation**: Evaluate the Lagrangian dual function:

$$\mathcal{L}(\boldsymbol{\pi}^{(k)}) = \sum_{e \in T^{(k)}} w'_e + 2 \sum_{i \in V} \pi_i^{(k)}. \tag{6.48}$$

   Update the running best lower bound:

$$\underline{\mathcal{L}} \leftarrow \max\left(\underline{\mathcal{L}},\ \mathcal{L}(\boldsymbol{\pi}^{(k)})\right). \tag{6.49}$$

5. **Optimality Test**: Compute the degree $\deg_{T^{(k)}}(i)$ of every vertex $i \in V$ in $T^{(k)}$.
   - If $\deg_{T^{(k)}}(i) = 2$ for all $i \in V$, the 1-tree $T^{(k)}$ is connected and 2-regular, hence a valid Hamiltonian cycle! By Theorem 6.5, $T^{(k)}$ is an optimal TSP tour, and the algorithm halts immediately with an optimality certificate ($\mathcal{L}(\boldsymbol{\pi}^{(k)}) = z^*$).
6. **Subgradient Computation and Multiplier Update**: Evaluate the subgradient vector $\mathbf{g}^{(k)} \in \mathbb{Z}^n$:

$$g_i^{(k)} = 2 - \deg_{T^{(k)}}(i) \quad \forall i \in V. \tag{6.50}$$

   If $\|\mathbf{g}^{(k)}\|_2 = 0$, all degrees are satisfied; terminate. Otherwise, compute the Polyak step size:

$$\mu_k = \frac{\alpha_k \left(\text{UB} - \mathcal{L}(\boldsymbol{\pi}^{(k)})\right)}{\sum_{i \in V} \left(2 - \deg_{T^{(k)}}(i)\right)^2}. \tag{6.51}$$

   Update the multipliers:

$$\pi_i^{(k+1)} = \pi_i^{(k)} + \mu_k \left(2 - \deg_{T^{(k)}}(i)\right) \quad \forall i \in V. \tag{6.52}$$

7. **Zero-Sum Multiplier Centering**: While $\sum_{i \in V} g_i^{(k)} = 0$ theoretically guarantees that zero-sum multipliers remain zero-sum (Theorem 6.6), floating-point rounding errors can induce numerical drift over hundreds of iterations. In practice, one recenters the multipliers after each update:

$$\bar{\pi} = \frac{1}{n} \sum_{i \in V} \pi_i^{(k+1)}, \qquad \pi_i^{(k+1)} \leftarrow \pi_i^{(k+1)} - \bar{\pi} \quad \forall i \in V. \tag{6.53}$$

8. **Step Size Decay and Termination**: If the best lower bound $\underline{\mathcal{L}}$ fails to strictly improve over p consecutive iterations, reduce the step parameter:

$$\alpha_{k+1} \leftarrow \gamma \cdot \alpha_k. \tag{6.54}$$

Terminate when the maximum iteration limit $K_{\max}$ is reached, when $\alpha_k < \epsilon$, or when $\lceil \underline{\mathcal{L}} \rceil = \mathrm{UB}$ for integer edge weights.

Computational Complexity of the Subgradient Step. Each iteration requires:

- Computing modified edge weights: $\mathcal{O}(n^2)$ arithmetic operations.
- Minimum Spanning Tree on $n-1$ vertices: $\mathcal{O}(n^2)$ using Prim's algorithm or $\mathcal{O}(n^2 \log n)$ via Kruskal's algorithm on complete graphs.
- Finding the two cheapest incident edges to vertex 0: $\mathcal{O}(n)$ comparisons.
- Degree evaluation and subgradient update: $\mathcal{O}(n)$ operations.

Consequently, each subgradient iteration runs in $\mathcal{O}(n^2)$ time. In practice, 50 to 200 iterations suffice to capture over 99% of the asymptotic Held–Karp bound, executing in mere milliseconds even for graphs with hundreds of vertices.

6.6 Polyhedral Strength and Held–Karp

Why is the Held–Karp bound so celebrated in combinatorial optimization? To understand its power, we must examine the classical integer programming formulation of the TSP and its polyhedral relaxation.

6.6.1 The DFJ Formulation of the TSP

In their landmark 1954 paper, George Dantzig, Ray Fulkerson, and Selmer Johnson [DFJ54] introduced the first integer linear programming formulation of the TSP. Consider an undirected complete graph $G = (V, E)$ with edge weights $w_e \geq 0$. For each edge $e \in E$, let $x_e \in \{0, 1\}$ be a binary decision variable indicating whether edge e is traversed in the tour:

$$x_e = \begin{cases} 1 & \text{if edge } e \text{ belongs to the tour,} \\ 0 & \text{otherwise.} \end{cases} \tag{6.55}$$

Degree Constraints (2-Factor)

Because every vertex in a valid Hamiltonian cycle must have exactly one incoming edge and one outgoing edge, the degree of every vertex in the tour subgraph must equal 2:

$$\sum_{e \in \delta(i)} x_e = 2 \quad \forall i \in V, \tag{6.56}$$

where $\delta(i) = \{e \in E \mid i \in e\}$ denotes the set of edges incident to vertex i.

However, the degree constraints (6.56) alone are *insufficient* to define a Hamiltonian cycle. Any collection of mutually disjoint cycles that collectively visit every vertex in V (known as a *2-factor* or *subtour*) satisfies (6.56). For instance, in a 6-vertex graph with $V = \{0, 1, 2, 3, 4, 5\}$, two disconnected triangles $(0, 1, 2)$ and $(3, 4, 5)$ satisfy $\deg(i) = 2$ for all $i \in V$, yet fail to form a connected Hamiltonian tour.

Subtour Elimination Constraints (SECs)

To eliminate disconnected subtours, Dantzig, Fulkerson, and Johnson introduced constraints requiring that every proper, non-empty vertex subset $S \subset V$ cannot contain an isolated cycle.

For any vertex subset $S \subset V$, let $E(S) = \{\{u, v\} \in E \mid u, v \in S\}$ denote the set of edges whose both endpoints lie entirely within S. If the tour were to form an isolated cycle on the vertices of S, it would select exactly $|S|$ edges from $E(S)$. To forbid such a cycle, the number of selected edges within S cannot exceed $|S| - 1$:

$$\sum_{e \in E(S)} x_e \leq |S| - 1 \quad \forall S \subset V,\ 2 \leq |S| \leq n - 2. \tag{6.57}$$

Equivalently, let $\delta(S) = \{\{u, v\} \in E \mid u \in S,\ v \in V \setminus S\}$ denote the cut separating S from the rest of the graph. Any connected Hamiltonian cycle that enters S must eventually exit S, meaning it must cross the cut $\delta(S)$ at least twice:

$$\sum_{e \in \delta(S)} x_e \geq 2 \quad \forall S \subset V,\ 2 \leq |S| \leq n - 2. \tag{6.58}$$

Proposition 6.7 *Equivalence of SEC Formulations*

Under the degree constraints $\sum_{e \in \delta(i)} x_e = 2$ for all $i \in V$, the subset edge-capacity constraint (6.57) and the cut constraint (6.58) are mathematically equivalent for every vertex subset $S \subset V$.

Proof. Summing the degree equations over all vertices $i \in S$:

$$\sum_{i \in S} \sum_{e \in \delta(i)} x_e = 2|S|. \tag{6.59}$$

In this summation, every internal edge $e \in E(S)$ has both endpoints in S and is counted exactly twice. Every cut edge $e \in \delta(S)$ has exactly one endpoint in S and is counted once. Therefore:

$$2 \sum_{e \in E(S)} x_e + \sum_{e \in \delta(S)} x_e = 2|S| \iff \sum_{e \in \delta(S)} x_e = 2|S| - 2 \sum_{e \in E(S)} x_e. \tag{6.60}$$

Substituting this identity demonstrates that:

$$\sum_{e \in E(S)} x_e \leq |S| - 1 \iff 2|S| - 2 \sum_{e \in E(S)} x_e \geq 2 \iff \sum_{e \in \delta(S)} x_e \geq 2. \tag{6.61}$$

□

The complete Dantzig–Fulkerson–Johnson (DFJ) integer linear program is:

$$\min_{\boldsymbol{x}} \quad \sum_{e \in E} w_e x_e \tag{6.62}$$

$$\text{subject to} \quad \sum_{e \in \delta(i)} x_e = 2 \quad \forall i \in V, \tag{6.63}$$

$$\sum_{e \in E(S)} x_e \leq |S| - 1 \quad \forall S \subset V,\ 2 \leq |S| \leq n - 2, \tag{6.64}$$

$$x_e \in \{0, 1\} \quad \forall e \in E. \tag{6.65}$$

Notice that the number of subtour elimination constraints (6.64) is $2^{n-1} - n - 1$, which grows exponentially with n. For $n = 50$, there are more than 10^{14} constraints, precluding any static enumeration in an LP solver!

6.6.2 Subtour LP and Held–Karp Equivalence

The linear programming relaxation of the DFJ model, denoted *DFJ-LP*, is obtained by relaxing the integrality restriction $x_e \in \{0,1\}$ to $0 \le x_e \le 1$:

$$z_{\text{DFJ-LP}} = \min_{\boldsymbol{x}} \quad \sum_{e \in E} w_e x_e \tag{6.66}$$

$$\text{subject to} \quad \sum_{e \in \delta(i)} x_e = 2 \quad \forall i \in V, \tag{6.67}$$

$$\sum_{e \in E(S)} x_e \le |S| - 1 \quad \forall S \subset V,\ 2 \le |S| \le n-2, \tag{6.68}$$

$$0 \le x_e \le 1 \quad \forall e \in E. \tag{6.69}$$

In two seminal papers, Michael Held and Richard Karp [HK70; HK71] established the theoretical equivalence between the Lagrangian dual of the 1-tree relaxation and this exponential LP relaxation.

Theorem 6.8 *Held–Karp Equivalence Theorem*

The maximum value of the Lagrangian dual function $\mathcal{L}(\boldsymbol{\pi})$ over all node multiplier vectors $\boldsymbol{\pi} \in \mathbb{R}^n$ equals the optimal objective value of the linear programming relaxation of the DFJ subtour elimination formulation:

$$\max_{\boldsymbol{\pi} \in \mathbb{R}^n} \mathcal{L}(\boldsymbol{\pi}) = z_{\text{DFJ-LP}}. \tag{6.70}$$

Polyhedral Intuition and Outline. By Geoffrion's foundational theorem on Lagrangian duality [Geo74], the optimal Lagrangian dual value for an integer program with dualized linear constraints $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ and base combinatorial set X is equal to the optimal value of the linear program that optimizes over the convex hull of X:

$$\max_{\boldsymbol{\lambda}} \mathcal{L}(\boldsymbol{\lambda}) = \min \left\{ \boldsymbol{c}^{\top} \boldsymbol{x} \mid \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b},\ \boldsymbol{x} \in \operatorname{conv}(X) \right\}. \tag{6.71}$$

In the Held–Karp relaxation, X is the set of all 1-trees on G. A 1-tree consists of a spanning tree on $V \setminus \{0\}$ combined with two edges incident to root vertex 0. By Jack Edmonds' polyhedral characterization of the spanning tree polytope [Edm71], the convex hull $\operatorname{conv}(X)$ is completely described by the non-negativity bounds, the total edge-count equality $\sum_{e \in E} x_e = n$, and the tree-subtour inequalities $\sum_{e \in E(S)} x_e \le |S| - 1$ for all $S \subseteq V \setminus \{0\}$.

When we constrain $\boldsymbol{x} \in \operatorname{conv}(X)$ by the dualized degree equations $\sum_{e \in \delta(i)} x_e = 2$ for all $i \in V$, the resulting polytope is precisely the DFJ-LP feasible region defined by (6.67)–(6.69). Hence, the dual optimum $\max_{\boldsymbol{\pi}} \mathcal{L}(\boldsymbol{\pi})$ coincides with $z_{\text{DFJ-LP}}$. □

The practical significance of Theorem 6.8 is profound:

- **Linear Programming Approach**: Computing $z_{\text{DFJ-LP}}$ directly with an LP solver requires delayed constraint generation (cutting planes): one begins with the degree equations, solves the LP, solves a minimum s-t cut problem (via maximum flow) to detect violated subtour constraints, adds them dynamically to the simplex tableau, and re-optimizes until no subtours are violated.
- **Lagrangian Relaxation Approach**: Subgradient optimization converges to the *exact same polyhedral bound* $z_{\text{DFJ-LP}}$ without any LP solver or cutting planes, relying solely on $\mathcal{O}(n^2)$ Minimum Spanning Tree computations!

6.6.3 Empirical Strength of the Bound

Beyond its polyhedral equivalence, the true justification for the prominence of the Held–Karp bound is its extraordinary tightness on practical and benchmark instances.

In their original computational investigations, Held and Karp [HK70; HK71] made the remarkable empirical observation that on instances up to 64 cities, the gap between the optimal tour length z^* and the Held–Karp bound $\mathcal{L}^*$ was frequently less than 1%, and often closed completely to 0.0% at the root node.

Subsequent extensive computational studies over thousands of instances—most notably the definitive studies by Johnson and McGeoch [JM97], Valenzuela and Jones [VJ97], and Applegate, Bixby, Chvátal, and Cook [App+06]—have confirmed this behavior:

- **Uniform Random Euclidean Instances**: For points distributed uniformly at random in the unit square, the expected optimality gap of the Held–Karp lower bound against the true optimal tour is empirically between **0.7% and 0.9%**, across instance sizes ranging from $n = 100$ to $n = 100,000$ [JM97; VJ97].
- **TSPLIB Benchmark Library**: On the standard TSPLIB library of geographic and printed circuit board instances, the Held–Karp bound is almost always within **1.0% to 1.5%** of the certified optimum [App+06].
- **Comparison with Unpenalized Bounds**: In stark contrast, the static unpenalized 1-tree ($\boldsymbol{\pi} = \mathbf{0}$) typically exhibits an optimality gap of **15% to 25%**, while the naive 2-cheapest incident edges bound typically yields gaps of **30% to 45%**.

The Cornerstone of Exact TSP Solvers. This near-zero optimality gap explains why the Held–Karp bound is the cornerstone of state-of-the-art exact TSP solvers, such as the renowned *Concorde* solver [App+06]. In a Branch and Bound or Branch and Cut framework, reducing the root gap from 20% to less than 1% reduces the size of the search tree from hundreds of millions of nodes to a manageable handful of branches, making exact optimization tractable for instances with thousands of vertices.

6.7 Chapter Summary

Key Takeaways: Lagrangian Relaxation.

- **Dualizing Complicating Constraints**: Lagrangian relaxation extracts the easy combinatorial core of an NP-hard problem (X) and dualizes the complicating constraints into the objective function via penalty multipliers ($\boldsymbol{\lambda}$ or $\boldsymbol{\pi}$).
- **Guaranteed Lower Bounds**: By weak duality, evaluating the relaxed subproblem under any valid multiplier vector yields a rigorous lower bound on the primal minimization problem.
- **Subgradient Optimization**: The dual function $\mathcal{L}$ is concave and piecewise linear. Subgradients are obtained for free from constraint violations ($\mathbf{Ax} - \mathbf{b}$). Updating multipliers along the subgradient direction drives them toward the optimal dual bound.
- **Zero-Sum Multiplier Conservation**: In the TSP, because every 1-tree has exactly n edges, the sum of degree violations $\sum_{i \in V}(2 - \deg(i)) = 0$ is conserved at every iteration. This guarantees that initial zero-sum multipliers remain zero-sum throughout the search.
- **Polyhedral Strength**: The Held–Karp bound matches the linear programming relaxation of the exponential DFJ subtour elimination formulation, yet requires only simple minimum spanning tree calculations.

Table 6.2 summarizes the structural components of Lagrangian relaxation across the two problem classes studied in this chapter.

Table 6.2: Systematic comparison of Lagrangian relaxation formulations for RCSPP and the TSP.

Component	RCSPP	Traveling Salesperson Problem
Primal Objective	Minimize path cost $\mathbf{c}^\top \mathbf{x}$	Minimize tour weight $\mathbf{w}^\top \mathbf{x}$
Easy Core Subproblem	Directed s-t paths (Dijkstra)	Minimum 1-Trees (Kruskal/Prim)
Complicating Constraints	Resource budget $\sum_e t_e x_e \leq T$	Degree regularity $\deg(i) = 2\ \forall i \in V$
Multipliers	Scalar $\lambda \geq 0$ (inequality)	Vector $\boldsymbol{\pi} \in \mathbb{R}^n$, $\sum_i \pi_i = 0$ (equality)
Modified Edge Weights	$c'_{ij} = c_{ij} + \lambda t_{ij}$	$w'_{ij} = w_{ij} - \pi_i - \pi_j$
Subgradient Vector	$g = \sum_e t_e x_e - T$	$g_i = 2 - \deg_T(i) \quad \forall i \in V$
Polyhedral Bound	Equals LP relaxation of RCSPP	Equals DFJ Subtour Elimination LP

6.8 Exercises

Exercise 6.1 *Lagrangian Relaxation of the 0–1 Knapsack Problem*

Consider the 0–1 Knapsack Problem:

$$\max_{\mathbf{x}} \quad \sum_{i=0}^{n-1} v_i x_i \quad \text{subject to} \quad \sum_{i=0}^{n-1} w_i x_i \leq W, \quad x_i \in \{0,1\}\ \forall i \in \{0,\dots,n-1\},$$

where $v_i > 0$ and $w_i > 0$.

1. Formulate the Lagrangian relaxation obtained by dualizing the capacity constraint with multiplier $\lambda \geq 0$.
2. Show that for a fixed λ, the objective decomposes across items into n independent single-variable subproblems:

$$\mathcal{L}(\lambda) = \lambda W + \sum_{i=0}^{n-1} \max_{x_i \in \{0,1\}} (v_i - \lambda w_i) x_i.$$

   State the optimal decision rule for $x_i(\lambda)$ in terms of the ratio v_i/w_i and the multiplier λ.
3. **Guiding the Dual Equivalence Proof**: Prove that the optimal Lagrangian dual bound $\min_{\lambda \geq 0} \mathcal{L}(\lambda)$ equals the continuous LP relaxation bound z_{LP} by following these steps:
   (a) Assume items are indexed in non-increasing ratio order: $v_0/w_0 \geq v_1/w_1 \geq \cdots \geq v_{n-1}/w_{n-1}$. Let s denote the critical (split) item in the greedy continuous knapsack relaxation:

$$\sum_{i=0}^{s-1} w_i \leq W < \sum_{i=0}^{s} w_i.$$

   (b) Choose the specific candidate multiplier $\lambda^* = v_s/w_s$. Evaluate $\mathcal{L}(\lambda^*)$ explicitly using the decision rule from Part 2.
   (c) Show algebraically that $\mathcal{L}(\lambda^*)$ is identically equal to the continuous knapsack upper bound:

$$\mathcal{L}(\lambda^*) = \sum_{i=0}^{s-1} v_i + \frac{W - \sum_{i=0}^{s-1} w_i}{w_s} v_s = z_{\text{LP}}.$$

(d) Conclude by weak duality ($\mathcal{L}(\lambda) \geq z_{\text{LP}}$ for all $\lambda \geq 0$ due to the integrality property of $\mathcal{X} = \{0,1\}^n$) that $\min_{\lambda \geq 0} \mathcal{L}(\lambda) = z_{\text{LP}}$.

Exercise 6.2 *Step-by-Step Held–Karp Calculation*

Consider a complete TSP graph with 5 vertices $V = \{0,1,2,3,4\}$ and the symmetric distance matrix:

$$\boldsymbol{D} = \begin{pmatrix} 0 & 9 & 3 & 8 & 11 \\ 9 & 0 & 7 & 12 & 6 \\ 3 & 7 & 0 & 5 & 10 \\ 8 & 12 & 5 & 0 & 4 \\ 11 & 6 & 10 & 4 & 0 \end{pmatrix}.$$

1. Compute the minimum 1-tree rooted at vertex 0 with initial multipliers $\boldsymbol{\pi}^{(0)} = \mathbf{0}$. Report the selected edges, the vertex degrees, and the initial lower bound $\mathcal{L}(\mathbf{0})$.
2. Identify the degree violations and compute the subgradient vector $\mathbf{g}^{(0)} \in \mathbb{R}^5$, where $g_i^{(0)} = 2 - \deg_{T_0}(i)$. Verify that $\sum_{i=0}^{4} g_i^{(0)} = 0$.
3. Using a simple fixed step size $\theta_0 = 1.0$, compute the updated multiplier vector:

$$\boldsymbol{\pi}^{(1)} = \boldsymbol{\pi}^{(0)} + \theta_0 \, \mathbf{g}^{(0)}.$$

   Verify that the zero-sum condition $\sum_{i=0}^{4} \pi_i^{(1)} = 0$ is preserved.
4. Compute the modified distance matrix $D'_{ij} = D_{ij} - \pi_i^{(1)} - \pi_j^{(1)}$ under $\boldsymbol{\pi}^{(1)}$ and evaluate the updated 1-tree lower bound:

$$\mathcal{L}(\boldsymbol{\pi}^{(1)}) = c(T_{\boldsymbol{\pi}^{(1)}}) + 2\sum_{i=0}^{4} \pi_i^{(1)}.$$

   Show that the lower bound strictly improves from 26 to 27.

Exercise 6.3 *Root Independence of the 1-Tree Relaxation*

In our presentation of the Held–Karp bound, vertex 0 was arbitrarily designated as the special root vertex for the 1-tree computation. Let $r \in V$ be any choice of root vertex, and let $\mathcal{L}_r(\boldsymbol{\pi})$ denote the corresponding Lagrangian dual function.

1. Prove that for any Hamiltonian cycle $H \subset E$, H is a 1-tree with respect to root r for *every* $r \in V$.
2. Prove that the optimal Held–Karp dual value is invariant to the choice of root vertex:

$$\max_{\boldsymbol{\pi} \in \mathbb{R}^n} \mathcal{L}_r(\boldsymbol{\pi}) = \max_{\boldsymbol{\pi} \in \mathbb{R}^n} \mathcal{L}_{r'}(\boldsymbol{\pi}) \quad \forall r, r' \in V.$$

3. In a practical finite-iteration subgradient search, does the choice of root vertex affect the rate of convergence? Discuss why practitioners sometimes cycle through different root vertices during search.

Exercise 6.4 *Held–Karp Lagrangian Heuristic*

Subgradient optimization generates a sequence of 1-trees $T_0, T_1, \ldots, T_k$. Although most of these 1-trees are not Hamiltonian cycles, they often share many edges with optimal or near-optimal TSP tours.

1. Design a greedy tour construction heuristic (a *Lagrangian heuristic*) that takes a 1-tree T_k and converts it into a feasible Hamiltonian cycle by removing redundant edges and reconnecting disconnected components.
2. Explain how running this heuristic periodically during subgradient optimization helps dynamically update the upper bound UB in Polyak's step size formula (6.46).

CHAPTER 7

Network Flows

Network flow theory occupies a foundational position in combinatorial optimization and operational research. It bridges the structural elegance of graph theory with the polyhedral rigor of linear programming duality. Many practical decision problems—ranging from freight logistics and telecommunication routing to project scheduling, bipartite matching, and domain filtering in constraint satisfaction—can be cast as finding optimal commodity flows through capacitated networks.

In this chapter, we develop the theoretical foundations and algorithmic engines of network flow optimization. We begin by formalizing the dual pair formed by the *Maximum Flow Problem* and the *Minimum Cut Problem*, establishing their equivalence through linear programming duality and the combinatorial *Max-Flow Min-Cut Theorem*. We study the classical augmenting path method of Ford and Fulkerson, uncover the pathological behavior of unguided path choices, and analyze polynomial-time path selection strategies: capacity scaling (Δ-scaling) and the shortest augmenting path rule of Edmonds and Karp. We provide clean, idiomatic Java implementations of the underlying data structures and algorithms, explore practical modeling transformations (multiple sources and sinks, vertex capacities, lower bounds, and bipartite matching), and introduce the *Minimum-Cost Flow Problem* alongside the Successive Shortest Path algorithm.

7.1 Foundations and Problem Formulations

7.1.1 Historical Context: Network Interdiction

The mathematical formulation of network flows originated during the early Cold War. In a 1955 classified RAND Corporation memorandum, Theodore E. Harris and Frank S. Ross evaluated the capacity of the Soviet railway network connecting the Soviet Union to its satellite nations in Eastern Europe [HR55]. Their goal was twofold: from a logistical perspective, to estimate the maximum tonnage of supplies that could be transported from origins to destinations; and from an interdiction perspective, to identify the critical rail segments whose destruction would disrupt the transit network at minimum cost.

Harris and Ross recognized that the maximum throughput of the rail network was inextricably bound to the capacity of its most restrictive bottleneck. Lester R. Ford and Delbert R. Fulkerson formalized this observation mathematically in 1956, proving the celebrated *Max-Flow Min-Cut Theorem* [FF56; FF62]. In 1999, the Pentagon declassified the original Harris–Ross memorandum, revealing that modern network flow theory arose directly from this duality between flow throughput and network cut capacity.

7.1.2 The Minimum Cut Problem

We represent a capacitated network as a directed graph $G = (V, E)$, where $V = \{0, 1, \ldots, n-1\}$ is a finite set of n vertices and $E \subseteq V \times V$ is a set of m directed edges. Two distinguished vertices are specified: a *source* $s \in V$ and a *sink* (or target) $t \in V$ with $s \neq t$. Each directed edge $(u, v) \in E$ is endowed with a non-negative capacity $c(u, v) \geq 0$, representing the maximum rate at which commodity can traverse the edge from u to v. When $(u, v) \notin E$, we adopt the convention $c(u, v) = 0$.

Definition 7.1 *s-t Cut and Cut Capacity*

An *s-t cut* (or simply a *cut*) in a network $G = (V, E)$ is a partition of the vertex set V into two disjoint subsets A and $B = V \setminus A$ such that $s \in A$ and $t \in B$.
The *capacity* of the cut (A, B), denoted $\mathrm{cap}(A, B)$, is the sum of the capacities of all directed edges originating in A and terminating in B:

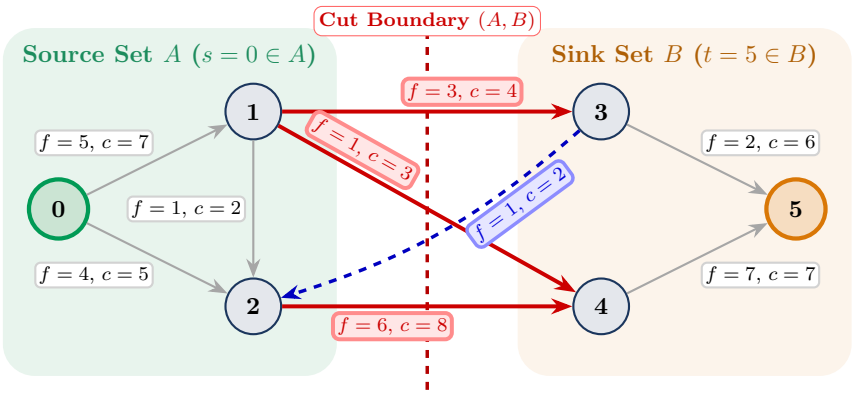
$$\mathrm{cap}(A, B) = \sum_{u \in A} \sum_{v \in B:\ (u,v) \in E} c(u, v). \tag{7.1}$$

A fundamental structural property of cut capacity is its *asymmetry*: edges originating in B and terminating in A do *not* contribute to $\mathrm{cap}(A, B)$. This reflects the physical principle that reverse edges cannot transport commodity forward from the source set A to the destination set B (Figure 7.1).

Definition 7.2 *Minimum s-t Cut Problem*

Given a capacitated network $G = (V, E)$ with source s, sink t, and non-negative edge capacities $c\colon E \to \mathbb{R}_{\geq 0}$, the *Minimum s-t Cut Problem* is to find an s-t cut (A, B) of minimum capacity:

$$\min_{(A,B)} \{\mathrm{cap}(A, B) \mid s \in A,\ t \in B,\ A \cup B = V,\ A \cap B = \emptyset\}. \tag{7.2}$$



Cut Capacity: $\mathrm{cap}(A, B) = c(1,3) + c(1,4) + c(2,4) = 4 + 3 + 8 = \mathbf{15}$ (*forward arcs only*)
Net Flow across Cut: $\mathrm{netflow}(A, B) = [f(1,3) + f(1,4) + f(2,4)] - f(3,2) = (3 + 1 + 6) - 1 = \mathbf{9} = v(f)$

Figure 7.1: An s-t cut (A, B) separating source $s = 0$ from sink $t = 5$. Only forward directed arcs crossing from A to B (solid red) contribute to the cut capacity $\mathrm{cap}(A, B) = 4+3+8 = 15$. The reverse arc $(3, 2)$ from B to A (dashed blue) carries flow back into A and is excluded from $\mathrm{cap}(A, B)$, but subtracts from the net flow across the cut.

7.1.3 The Maximum Flow Problem

We now formalize the dual primal concept: routing commodity through the network.

Definition 7.3 *s-t Flow and Flow Conservation*

An *s-t flow* (or simply a *flow*) is a function $f\colon E \to \mathbb{R}_{\geq 0}$ that assigns a non-negative scalar value $f(u,v)$ to each directed edge $(u,v) \in E$ satisfying two conditions:

1. **Capacity Constraints**: The flow along each edge cannot exceed its capacity:
$$0 \leq f(u,v) \leq c(u,v), \quad \forall (u,v) \in E. \tag{7.3}$$
2. **Flow Conservation (Local Equilibrium)**: At every intermediate vertex $v \in V \setminus \{s,t\}$, the total inflow equals the total outflow:
$$\sum_{u\colon (u,v)\in E} f(u,v) = \sum_{w\colon (v,w)\in E} f(v,w), \quad \forall v \in V \setminus \{s,t\}. \tag{7.4}$$

The *value* of the flow f, denoted $v(f)$, is the net commodity leaving the source s:
$$v(f) = \sum_{v\colon (s,v)\in E} f(s,v) - \sum_{u\colon (u,s)\in E} f(u,s). \tag{7.5}$$

Without loss of generality, we may assume that no directed edges enter s and no directed edges leave t. Under this standard assumption, the flow value simplifies to the total outflow from s, which by global conservation matches the total inflow into t:
$$v(f) = \sum_{v\colon (s,v)\in E} f(s,v) = \sum_{u\colon (u,t)\in E} f(u,t). \tag{7.6}$$

Definition 7.4 *Maximum s-t Flow Problem*

Given a capacitated network $G = (V,E)$ with source s, sink t, and capacities c, the *Maximum s-t Flow Problem* is to find a feasible flow f that maximizes the flow value $v(f)$:
$$\max_{f\colon E\to\mathbb{R}_{\geq 0}} \{v(f) \mid \text{constraints (7.3) and (7.4) hold}\}. \tag{7.7}$$

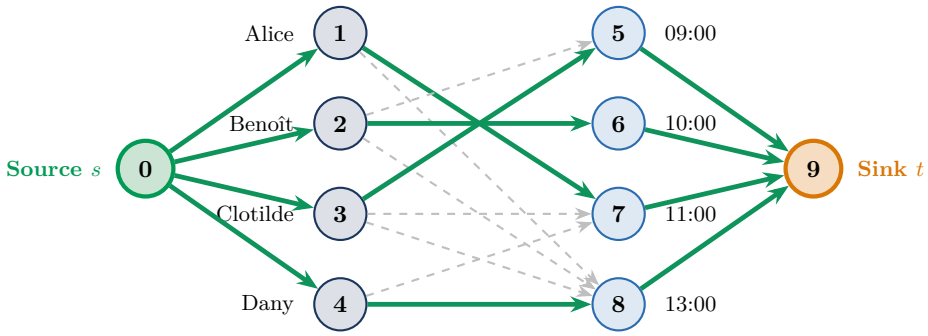
7.1.4 Case Study: Seminar Scheduling

To illustrate how discrete assignment problems naturally reduce to maximum flow formulations, consider an academic scheduling instance. Four speakers—Alice, Benoît, Clotilde, and Dany—must each deliver a one-hour presentation in a single lecture room. Four available time slots are proposed: 09:00, 10:00, 11:00, and 13:00. Each speaker indicates their availability:

- Alice: 11:00, 13:00.
- Benoît: 09:00, 10:00, 13:00.
- Clotilde: 09:00, 11:00, 13:00.
- Dany: 11:00, 13:00.

Can all four speakers be assigned to distinct, compatible time slots? We model this as a maximum flow problem in a bipartite network $G = (V,E)$ with $n = 10$ vertices indexed from 0 to 9 (Figure 7.2):

1. Source $s = 0$ connects to each speaker vertex $u \in \{1, 2, 3, 4\}$ with capacity $c(0, u) = 1$.
2. Each speaker connects to their available slot vertices $w \in \{5, 6, 7, 8\}$ with capacity $c(u, w) = 1$.
3. Each time slot connects to the sink $t = 9$ with capacity $c(w, 9) = 1$.



All capacities $c(e) = 1$. Solid green arcs indicate the maximum flow $v(f^*) = 4$ (matching).
Schedule: Clotilde → 09:00, Benoît → 10:00, Alice → 11:00, Dany → 13:00.

Figure 7.2: The seminar scheduling problem modeled as a maximum flow problem. Unit capacities ensure that each speaker delivers at most one lecture and each time slot hosts at most one presentation. The maximum flow value $v(f^*) = 4$ proves that a complete feasible schedule exists: Clotilde → 09:00, Benoît → 10:00, Alice → 11:00, and Dany → 13:00.

Because every edge capacity is an integer ($c_e = 1$), the Integrality Theorem (Theorem 7.7) guarantees that an optimal flow is integral: $f(u, w) \in \{0, 1\}$. An edge with $f(u, w) = 1$ corresponds to assigning speaker u to time slot w. If the maximum flow achieves $v(f^*) = 4$, every speaker is scheduled; if $v(f^*) < 4$, the minimum cut identifies the bottleneck preventing a complete schedule.

7.2 Linear Programming and Duality

7.2.1 Primal and Dual Formulations

Because the objective function and all constraints of the maximum flow problem are linear, it can be cast directly as a continuous Linear Program (LP). Let variable $f_{u,v} \geq 0$ denote the flow on edge $(u, v) \in E$.

$$\begin{aligned} \text{maximize} \quad & \sum_{v\colon (s,v)\in E} f_{s,v} \\ \text{subject to} \quad & \sum_{u\colon (u,v)\in E} f_{u,v} - \sum_{w\colon (v,w)\in E} f_{v,w} = 0, \quad \forall v \in V \setminus \{s, t\}, \\ & f_{u,v} \leq c_{u,v}, \quad \forall (u, v) \in E, \\ & f_{u,v} \geq 0, \quad \forall (u, v) \in E. \end{aligned} \tag{7.8}$$

To derive the dual linear program, we assign dual variables to each constraint of the primal problem:

- Dual potential $p_v \in \mathbb{R}$ associated with the flow conservation equation at vertex $v \in V \setminus \{s, t\}$. For convenience of notation, we fix $p_s = 1$ and $p_t = 0$.

- Dual variable $z_{u,v} \geq 0$ associated with the capacity constraint $f_{u,v} \leq c_{u,v}$ for each edge $(u, v) \in E$.

Formulating the Lagrangian of the primal maximization problem and applying standard LP duality principles yields the dual linear program:

$$\begin{aligned} \text{minimize} \quad & \sum_{(u,v)\in E} c_{u,v}\, z_{u,v} \\ \text{subject to} \quad & z_{u,v} - p_u + p_v \geq 0, \quad \forall (u,v) \in E, \\ & p_s = 1, \quad p_t = 0, \\ & z_{u,v} \geq 0, \quad \forall (u,v) \in E. \end{aligned} \tag{7.9}$$

The constraint $z_{u,v} \geq p_u - p_v$ combined with $z_{u,v} \geq 0$ implies that in any optimal dual solution, $z_{u,v} = \max\{0, p_u - p_v\}$.

Proposition 7.1 *Dual Problem as Minimum Cut Relaxation*

If the potential variables p_v are restricted to binary values $p_v \in \{0, 1\}$, the dual problem (7.9) is an exact formulation of the Minimum s-t Cut Problem.

Proof. Let $A = \{v \in V \mid p_v = 1\}$ and $B = \{v \in V \mid p_v = 0\}$. Since $p_s = 1$ and $p_t = 0$, $s \in A$ and $t \in B$, so (A, B) is a valid s-t cut. For any edge $(u, v) \in E$:

$$z_{u,v} = \max\{0, p_u - p_v\} = \begin{cases} 1 & \text{if } u \in A \text{ and } v \in B, \\ 0 & \text{otherwise.} \end{cases} \tag{7.10}$$

Thus $z_{u,v} = 1$ if and only if edge (u, v) originates in A and terminates in B. The dual objective function evaluates to:

$$\sum_{(u,v)\in E} c_{u,v} z_{u,v} = \sum_{u\in A} \sum_{v\in B\colon (u,v)\in E} c_{u,v} = \text{cap}(A, B). \tag{7.11}$$

Minimizing this objective over binary choices $p_v \in \{0, 1\}$ is therefore identical to finding an s-t cut of minimum capacity. □

7.2.2 Total Unimodularity and Integrality

The dual formulation (7.9) is a continuous linear relaxation over $p_v \in [0, 1]$. Why does solving this continuous relaxation guarantee binary cut indicators $p_v \in \{0, 1\}$, and why does the primal LP (7.8) yield integer flows whenever capacities $c_{u,v}$ are integers?

The answer lies in the algebraic concept of *Total Unimodularity* (TUM) introduced in Chapter 5. Recall that a matrix $\mathbf{A}$ is totally unimodular if the determinant of every square submatrix belongs to $\{-1, 0, 1\}$.

Theorem 7.2 *Integrality of Network Matrices*

Let $\mathbf{M} \in \{-1, 0, 1\}^{|V|\times|E|}$ be the vertex-edge incidence matrix of a directed graph

$G = (V, E)$, defined by:

$$M_{v,e} = \begin{cases} +1 & \text{if edge } e \text{ leaves vertex } v, \\ -1 & \text{if edge } e \text{ enters vertex } v, \\ 0 & \text{otherwise.} \end{cases} \tag{7.12}$$

Then $\mathbf{M}$ is totally unimodular.

Proof. We prove that every $k \times k$ submatrix $\mathbf{B}$ of $\mathbf{M}$ satisfies $\det(\mathbf{B}) \in \{-1, 0, 1\}$ by induction on k.

- **Base case ($k = 1$)**: Every entry of $\mathbf{M}$ is $+1$, -1, or 0. Thus $\det(\mathbf{B}) \in \{-1, 0, 1\}$.
- **Inductive step**: Assume the property holds for all submatrices of size $(k-1) \times (k-1)$. Consider a $k \times k$ submatrix $\mathbf{B}$. Each column of $\mathbf{B}$ corresponds to a directed edge in G and can contain at most one $+1$ and at most one -1:
  1. If any column of $\mathbf{B}$ contains only zeros, then $\det(\mathbf{B}) = 0$.
  2. If any column contains exactly one non-zero entry (± 1), expand $\det(\mathbf{B})$ along that column. By the Laplace expansion formula, $\det(\mathbf{B}) = \pm 1 \cdot \det(\mathbf{B}')$, where $\mathbf{B}'$ is a $(k-1) \times (k-1)$ submatrix. By the inductive hypothesis, $\det(\mathbf{B}') \in \{-1, 0, 1\}$, so $\det(\mathbf{B}) \in \{-1, 0, 1\}$.
  3. If every column contains exactly two non-zero entries (one $+1$ and one -1), then summing all row vectors of $\mathbf{B}$ yields the zero vector $\mathbf{0}^\top$. Hence the rows of $\mathbf{B}$ are linearly dependent, and $\det(\mathbf{B}) = 0$.

In all cases, $\det(\mathbf{B}) \in \{-1, 0, 1\}$, establishing total unimodularity. □

By the Hoffman–Kruskal theorem (Chapter 5), optimizing a linear objective over a polyhedron defined by a totally unimodular constraint matrix and integer bounds guarantees that every extreme point is integral. Consequently, solving the maximum flow linear program via the Simplex algorithm naturally yields an optimal integer flow and an optimal binary cut whenever all capacities are integers.

Simplex versus Graph Algorithms. Although the Simplex algorithm solves maximum flow problems with guaranteed integrality, the worst-case time complexity of the Simplex method on general polyhedra is exponential ($\mathcal{O}(2^n)$ pivots on Klee–Minty configurations; see Chapter 4). Specialized combinatorial graph algorithms exploit the underlying topology of the residual network directly, achieving strongly polynomial running times such as $\mathcal{O}(|V| \cdot |E|^2)$ without performing matrix inversions or simplex pivots.

7.3 The Ford-Fulkerson Method

7.3.1 Residual Graphs and Augmentation

A natural greedy heuristic for finding a maximum flow would be to repeatedly find a directed path from s to t in G that has spare capacity, push as much flow as possible along this path, and repeat until all paths are saturated. However, this naive greedy approach can become permanently trapped in a sub-optimal flow assignment, because it provides no mechanism to undo an earlier sub-optimal routing choice.

To overcome this fundamental limitation, Lester Ford and Delbert Fulkerson introduced the concept of an *augmenting path* with forward and backward edges [FF56]. A forward

edge allows increasing flow on an unsaturated edge, while a backward edge allows *decreasing* flow on an edge carrying positive flow. Decreasing flow on an edge (v, u) is mathematically equivalent to *redirecting* commodity along an alternate path, effectively undoing an earlier suboptimal routing decision.

This mechanism is encapsulated by the *residual network*:

Definition 7.5 *Residual Graph and Residual Capacity*

Let f be a feasible flow in a capacitated network $G = (V, E)$ with capacities c. The *residual graph* $G_f = (V, E_f)$ has the same vertex set V as G. For each pair of vertices $u, v \in V$, the residual capacity $c_f(u, v)$ is defined as:

$$c_f(u,v) = \begin{cases} c(u,v) - f(u,v) & \text{if } (u,v) \in E \text{ (forward edge)}, \\ f(v,u) & \text{if } (v,u) \in E \text{ (backward edge)}, \\ 0 & \text{otherwise.} \end{cases} \tag{7.13}$$

The residual edge set E_f consists of all ordered pairs (u, v) with strictly positive residual capacity:

$$E_f = \{(u,v) \in V \times V \mid c_f(u,v) > 0\}. \tag{7.14}$$

An edge $(u, v) \in E$ generates:

- A **forward residual edge** $(u, v) \in E_f$ with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$ whenever (u, v) is not saturated.
- A **backward residual edge** $(v, u) \in E_f$ with capacity $c_f(v, u) = f(u, v) > 0$ whenever (u, v) carries positive flow.

Figure 7.3 illustrates the exact correspondence between a flow assignment in G and its residual network G_f.

Definition 7.6 *Augmenting Path and Bottleneck Capacity*

An *augmenting path* P is a simple directed path from source s to sink t in the residual graph G_f.
The *bottleneck capacity* (or residual capacity) of P, denoted $\delta(P)$, is the minimum residual capacity among all edges along the path:

$$\delta(P) = \min_{(u,v) \in P} c_f(u,v) > 0. \tag{7.15}$$

Lemma 7.3 *Flow Augmentation Lemma*

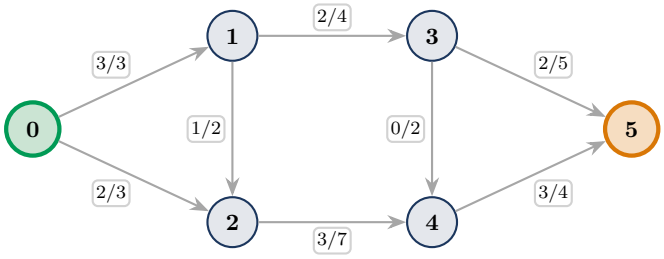
Let f be a feasible flow in G, and let P be an augmenting path in G_f with bottleneck capacity $\delta = \delta(P)$. Define an updated flow function $f' \colon E \to \mathbb{R}_{\geq 0}$ by:

$$f'(u,v) = \begin{cases} f(u,v) + \delta & \text{if } (u,v) \text{ is a forward edge on } P, \\ f(u,v) - \delta & \text{if } (v,u) \text{ is a backward edge on } P, \\ f(u,v) & \text{if neither } (u,v) \text{ nor } (v,u) \text{ appears on } P. \end{cases} \tag{7.16}$$

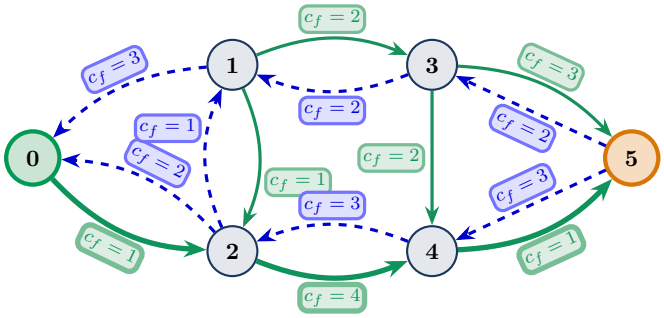
Then f' is a feasible flow in G, and its value is strictly increased by δ:

$$v(f') = v(f) + \delta. \tag{7.17}$$

(a) Flow Network G with Flow Assignment $f(e)/c(e)$, Current Value $v(f) = 5$



(b) Residual Network G_f (Solid green: Forward $c_f = c - f$; Dashed blue: Backward $c_f = f$)



Augmenting Path in G_f (bold green): $P = (0 \to 2 \to 4 \to 5)$, bottleneck $\delta = \min\{1, 4, 1\} = \mathbf{1}$
Pushing $\delta = 1$ along P yields updated flow $v(f') = 5 + 1 = \mathbf{6}$.

Figure 7.3: A flow assignment f in G and its corresponding residual network G_f. Saturated edges in G (such as $(0, 1)$ with $f = 3, c = 3$) have zero forward residual capacity and appear only as backward edges in G_f. Partially saturated edges (such as $(0, 2)$ with $f = 2, c = 3$) generate both a forward residual arc with $c_f(0, 2) = 1$ and a backward residual arc with $c_f(2, 0) = 2$.

Proof. We verify that f' satisfies capacity constraints and conservation laws:

1. **Capacity constraints**: For forward edges on P, $f'(u,v) = f(u,v) + \delta \leq f(u,v) + c_f(u,v) = f(u,v) + (c(u,v) - f(u,v)) = c(u,v)$. Non-negativity holds since $\delta > 0$. For backward edges on P, $f'(u,v) = f(u,v) - \delta \geq f(u,v) - c_f(v,u) = f(u,v) - f(u,v) = 0$.
2. **Flow conservation**: Consider any intermediate vertex $v \in V \setminus \{s,t\}$. If path P does not visit v, inflow and outflow remain unchanged. If P traverses v, it enters v along one residual edge and leaves v along another. There are four cases depending on whether incoming/outgoing residual edges correspond to forward or backward edges in G; in all four cases, the net change in total inflow to v exactly balances the net change in total outflow from v.
3. **Flow value**: Path P originates at s and terminates at t. The net flow leaving s increases by exactly δ, so $v(f') = v(f) + \delta$.

□

7.3.2 The Max-Flow Min-Cut Theorem

To establish the optimality conditions of the Ford–Fulkerson method, we first analyze the relationship between the flow through a network and the capacity of an arbitrary cut.

Definition 7.7 *Net Flow Across a Cut*

Let f be a flow in G, and let (A,B) be an s-t cut. The *net flow across the cut* (A,B), denoted $\text{netflow}(A,B)$, is the total flow on edges from A to B minus the total flow on edges from B to A:

$$\text{netflow}(A,B) = \sum_{u \in A} \sum_{v \in B \colon (u,v) \in E} f(u,v) - \sum_{u \in B} \sum_{v \in A \colon (u,v) \in E} f(u,v). \tag{7.18}$$

Lemma 7.4 *Flow-Value Lemma*

Let f be any feasible flow in G, and let (A,B) be any s-t cut. The net flow across (A,B) equals the value of the flow:

$$\text{netflow}(A,B) = v(f). \tag{7.19}$$

Proof. We sum the flow conservation equations over all vertices $u \in A$. For $u = s$, the net outflow is $v(f)$. For every other vertex $u \in A \setminus \{s\}$, flow conservation guarantees that net outflow is 0. Therefore:

$$v(f) = \sum_{u \in A} \left(\sum_{w \colon (u,w) \in E} f(u,w) - \sum_{w \colon (w,u) \in E} f(w,u) \right). \tag{7.20}$$

Any edge (u,w) where both $u \in A$ and $w \in A$ appears twice in this double summation: once with a positive sign (as an outflow from u) and once with a negative sign (as an inflow to w). These internal terms cancel out completely. The only surviving terms are edges crossing the cut boundary:

$$v(f) = \sum_{u \in A} \sum_{v \in B \colon (u,v) \in E} f(u,v) - \sum_{u \in B} \sum_{v \in A \colon (u,v) \in E} f(u,v) = \text{netflow}(A,B). \tag{7.21}$$

□

Lemma 7.5 *Weak Duality of Network Flows*

For any feasible flow f and any s-t cut (A, B):

$$v(f) \leq \text{cap}(A, B). \tag{7.22}$$

Proof. By the Flow-Value Lemma (Theorem 7.4) and the non-negativity of flows:

$$v(f) = \text{netflow}(A, B) = \sum_{u \in A, v \in B} f(u, v) - \sum_{u \in B, v \in A} f(u, v) \tag{7.23}$$

$$\leq \sum_{u \in A, v \in B} f(u, v) \leq \sum_{u \in A, v \in B} c(u, v) = \text{cap}(A, B). \tag{7.24}$$

□

Theorem 7.5 immediately implies that the capacity of *any* cut provides an upper bound on the value of *every* feasible flow. Consequently:

$$\max_f v(f) \leq \min_{(A,B)} \text{cap}(A, B). \tag{7.25}$$

We are now ready to state and prove the central theorem of network flow theory:

Theorem 7.6 *Max-Flow Min-Cut Theorem*

Let f be a feasible flow in a capacitated network $G = (V, E)$ with source s and sink t. The following three statements are equivalent:

(i) There exists an s-t cut (A, B) such that $\text{cap}(A, B) = v(f)$.

(ii) f is a maximum flow.

(iii) There is no augmenting path from s to t in the residual graph G_f.

Proof. We establish equivalence through the circular chain of implications (i) $\implies$ (ii) $\implies$ (iii) $\implies$ (i):

- **(i) $\implies$ (ii)**: Suppose there exists a cut (A, B) with $\text{cap}(A, B) = v(f)$. By Weak Duality (Theorem 7.5), for any other feasible flow f', $v(f') \leq \text{cap}(A, B) = v(f)$. Thus no flow can achieve a value strictly greater than $v(f)$, meaning f is a maximum flow.
- **(ii) $\implies$ (iii)**: Suppose, for the sake of contradiction, that f is a maximum flow but there exists an augmenting path P from s to t in G_f. By Theorem 7.3, pushing the bottleneck capacity $\delta(P) > 0$ produces a feasible flow f' with $v(f') = v(f) + \delta(P) > v(f)$. This contradicts the maximality of f.
- **(iii) $\implies$ (i)**: Suppose there is no augmenting path from s to t in G_f. We construct an explicit cut (A, B) as follows: define A to be the set of all vertices reachable from s via directed paths in the residual graph G_f, and let $B = V \setminus A$.

  1. Clearly $s \in A$ (a path of zero length). Since no augmenting path exists from s to t in G_f, $t \notin A$, which implies $t \in B$. Hence (A, B) is a valid s-t cut.
  2. For every edge $(u, v) \in E$ with $u \in A$ and $v \in B$, the edge (u, v) cannot belong to E_f (otherwise v would be reachable from s in G_f, placing $v \in A$). Therefore, $c_f(u, v) = 0$, which implies $f(u, v) = c(u, v)$ (all forward edges are fully saturated).

3. For every edge $(v, u) \in E$ with $v \in B$ and $u \in A$, the backward residual edge (u, v) cannot belong to E_f (otherwise v would be reachable from $u \in A$). Therefore, $c_f(u, v) = f(v, u) = 0$ (all backward edges carry zero flow).

Evaluating the net flow across this cut (A, B):

$$\text{netflow}(A, B) = \sum_{u \in A, v \in B} f(u, v) - \sum_{v \in B, u \in A} f(v, u) \tag{7.26}$$

$$= \sum_{u \in A, v \in B} c(u, v) - 0 = \text{cap}(A, B). \tag{7.27}$$

By the Flow-Value Lemma, $v(f) = \text{netflow}(A, B) = \text{cap}(A, B)$. This completes the proof.

□

Theorem 7.7 *Integrality Theorem*

If all edge capacities $c(u, v)$ in a network $G = (V, E)$ are integers, then there exists an optimal maximum flow f^* such that $f^*(u, v) \in \mathbb{Z}_{\geq 0}$ for every edge $(u, v) \in E$. Furthermore, the Ford–Fulkerson algorithm initialized with zero flow maintains integer flow values at every augmentation and terminates with an integral maximum flow.

Proof. We proceed by induction on the number of augmentations. The initial flow $f \equiv 0$ is trivially integer-valued. In the residual graph G_f, all forward capacities $c(u, v) - f(u, v)$ and backward capacities $f(v, u)$ are differences of integers, and therefore integers. The bottleneck capacity $\delta(P) = \min_{e \in P} c_f(e)$ is the minimum over a finite set of positive integers, so $\delta(P) \geq 1$ is an integer. Augmenting the flow along P adds or subtracts $\delta(P)$, preserving integrality. Since each augmentation increases the flow value by at least 1, the algorithm must terminate in a finite number of steps on any finite-capacity network. □

Extracting the Minimum Cut. Theorem 7.6 provides a direct, constructive method to compute a minimum cut once a maximum flow f^* is obtained: perform a Breadth-First Search (BFS) or Depth-First Search (DFS) starting from source s in the residual graph G_{f^*}. The set of all vertices marked as reachable from s constitutes set A; the set of unreachable vertices constitutes set $B = V \setminus A$. The cut (A, B) is guaranteed to be a minimum cut.

7.4 Path Selection and Complexity

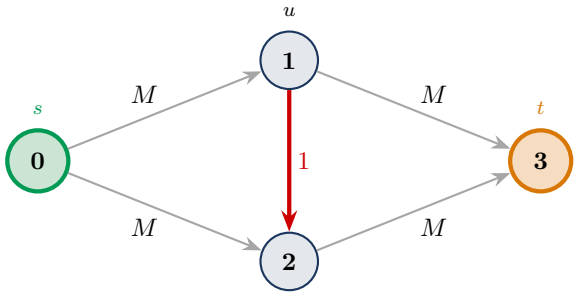
7.4.1 Arbitrary Paths and Pseudo-Polynomiality

The classical Ford–Fulkerson method does not specify how augmenting paths should be chosen in G_f. If augmenting paths are chosen arbitrarily (e.g., using an arbitrary Depth-First Search traversal), the number of augmentations can depend directly on the numerical magnitudes of the edge capacities, leading to pseudo-polynomial behavior.

Consider the pathological 4-node network illustrated in Figure 7.4. The network has vertices $V = \{0, 1, 2, 3\}$ with source $s = 0$ and sink $t = 3$. Edges $(0, 1)$, $(0, 2)$, $(1, 3)$, and $(2, 3)$ have a large capacity $M \in \mathbb{Z}^+$, while the central cross-edge $(1, 2)$ has unit capacity 1.

Suppose the DFS search alternates between two augmenting paths:

1. **Odd iterations**: Path $P_1 = (0 \to 1 \to 2 \to 3)$ with bottleneck $\delta = \min\{M, 1, M\} = 1$. This pushes 1 unit of flow, saturating $(1, 2)$ and creating reverse residual edge $(2, 1)$ with capacity 1.



Worst-Case Alternating Augmentations ($M = 10^6$):
Iteration 1: Path $0 \to 1 \to 2 \to 3$, Bottleneck $\delta = 1 \implies$ Flow pushes 1 unit across $(1, 2)$.
Iteration 2: Residual arc $(2, 1)$ has capacity 1. Path $0 \to 2 \to 1 \to 3$, Bottleneck $\delta = 1$.
... Alternating sequence requires $2M = 2 \times 10^6$ augmentations!
(*Edmonds–Karp or Capacity Scaling terminates in 2 augmentations.*)

Figure 7.4: The pathological 4-node diamond network. If an unguided DFS search repeatedly selects the augmenting path using the central cross-arc $(1, 2)$ and its reverse residual arc $(2, 1)$, each iteration pushes only $\delta = 1$ unit of flow. The algorithm requires $2M$ augmentations to reach the maximum flow value of $2M$.

2. **Even iterations**: Path $P_2 = (0 \to 2 \to 1 \to 3)$ with bottleneck $\delta = \min\{M, 1, M\} = 1$. This pushes 1 unit of flow, canceling the flow on $(1, 2)$ and recreating forward residual edge $(1, 2)$ with capacity 1.

Each augmentation increases the flow value by only 1. To reach the optimal maximum flow $v(f^*) = 2M$, the algorithm executes exactly $2M$ augmentations! If $M = 10^9$, the algorithm performs 2×10^9 graph searches. Because the input size required to encode M is $\mathcal{O}(\log M)$ bits, the complexity is exponential in the input size, rendering unguided Ford–Fulkerson a *pseudo-polynomial* algorithm:

$$\text{Running Time (Generic Ford–Fulkerson)} = \mathcal{O}(|E| \cdot v(f^*)) = \mathcal{O}(|V| \cdot |E| \cdot U), \tag{7.28}$$

where $U = \max_{e \in E} c(e)$ is the maximum edge capacity. Even worse, if capacities are allowed to be irrational numbers, an unguided Ford–Fulkerson algorithm can fail to terminate entirely and can converge to a value strictly smaller than the true maximum flow.

7.4.2 The Capacity Scaling Algorithm

To eliminate pseudo-polynomiality, we need path selection strategies that prioritize large augmentations. The *Capacity Scaling Algorithm* (or Δ-scaling) implements this intuition: it systematically searches for augmenting paths with bottleneck capacities of at least Δ, where Δ is a power-of-two threshold that is halved in successive scaling phases.

Algorithm 10: Capacity Scaling Max-Flow Algorithm

Input: Network $G = (V, E)$, source s, sink t, capacities c
Output: Maximum flow f^*

```
Initialize f(u, v) ← 0 for all (u, v) ∈ E;
U ← max_{e∈E} c(e);
Δ ← 2^⌊log₂ U⌋;
while Δ ≥ 1 do
    while there exists an augmenting path P in G_f with c_f(e) ≥ Δ for all e ∈ P do
        δ ← min_{e∈P} c_f(e);
        Augment flow along P by δ via (7.16);
    Δ ← Δ/2;
return f;
```

Let $G_f(\Delta)$ denote the subgraph of the residual network consisting solely of edges with residual capacity at least Δ: $E_f(\Delta) = \{e \in E_f \mid c_f(e) \geq \Delta\}$. A Δ-scaling phase repeatedly discovers augmenting paths in $G_f(\Delta)$ using standard BFS or DFS.

Lemma 7.8 *Residual Cut Bound in Scaling Phases*

At the end of a Δ-scaling phase, the difference between the optimal maximum flow value $v(f^*)$ and the current flow value $v(f)$ satisfies:

$$v(f^*) - v(f) \leq |E| \cdot \Delta. \tag{7.29}$$

Proof. At the termination of the Δ-scaling phase, there is no directed path from s to t in $G_f(\Delta)$. Let S be the set of vertices reachable from s in $G_f(\Delta)$, and let $T = V \setminus S$. Since $t \notin S$, (S, T) is an s-t cut.

By definition of S, no edge in G_f originating in S and terminating in T can have residual capacity $\geq \Delta$. Hence, for every directed edge $(u, v) \in E$ with $u \in S, v \in T$, the residual capacity satisfies $c_f(u, v) < \Delta$. The total residual capacity of the cut (S, T) in G_f is bounded by:

$$c_f(S, T) = \sum_{u \in S, v \in T} c_f(u, v) \leq |E| \cdot \Delta. \tag{7.30}$$

By the Flow-Value Lemma, the additional flow that can be routed from s to t in G_f cannot exceed the residual capacity of any cut, so $v(f^*) - v(f) \leq c_f(S, T) \leq |E| \cdot \Delta$. □

Theorem 7.9 *Complexity of Capacity Scaling*

The Capacity Scaling Algorithm performs at most $2|E|$ augmentations per scaling phase. Its total running time is $\mathcal{O}(|E|^2 \log U)$, which is weakly polynomial in the size of the network.

Proof. In the next phase, the threshold is $\Delta' = \Delta/2$, and each augmentation carries a bottleneck capacity of at least $\Delta' = \Delta/2$. Since the total remaining deficit between $v(f^*)$ and $v(f)$ at the start of this new phase was at most $|E| \cdot \Delta$, the number of augmentations performed in this phase cannot exceed:

$$\frac{|E| \cdot \Delta}{\Delta/2} = 2|E|. \tag{7.31}$$

The initial threshold is $\Delta \leq U$, and the algorithm terminates when $\Delta < 1$. The number of scaling phases is therefore $1 + \lfloor \log_2 U \rfloor = \mathcal{O}(\log U)$. In each phase, finding an augmenting path in $G_f(\Delta)$ via BFS or DFS takes $\mathcal{O}(|E|)$ time. The overall time complexity is:

$$\mathcal{O}(\log U) \times 2|E| \times \mathcal{O}(|E|) = \mathcal{O}(|E|^2 \log U). \tag{7.32}$$

□

7.4.3 Shortest Augmenting Path: Edmonds–Karp

In 1972, Jack Edmonds and Richard M. Karp discovered that choosing augmenting paths with the *minimum number of edges* guarantees a *strongly polynomial* running time, completely independent of edge capacities [EK72]. (This result was independently discovered by Efim A. Dinitz in 1970 [Din70].)

The Edmonds–Karp algorithm simply implements the Ford–Fulkerson method where each augmenting path is found via a **Breadth-First Search** (BFS) in G_f, treating each residual edge as having unit length 1.

Lemma 7.10 *Monotonicity of Residual Distances*

Let $d_{G_f}(s, v)$ denote the shortest path distance (measured in number of edges) from source s to vertex v in the residual graph G_f. For every vertex $v \in V$, the distance $d_{G_f}(s, v)$ increases monotonically after each flow augmentation:

$$d_{G_{f'}}(s, v) \geq d_{G_f}(s, v), \quad \forall v \in V, \tag{7.33}$$

where f' is the flow obtained after augmenting along a shortest augmenting path P.

Proof. Suppose, for the sake of contradiction, that there exists a non-empty set of vertices whose shortest distance from s decreases after augmenting along P. Let v be a vertex in this set that minimizes $d_{G_{f'}}(s, v)$, and let P' be a shortest path from s to v in $G_{f'}$. Let u be the predecessor of v on P', so that $(u, v) \in E_{f'}$ and:

$$d_{G_{f'}}(s, v) = d_{G_{f'}}(s, u) + 1. \tag{7.34}$$

Because $d_{G_{f'}}(s, u) < d_{G_{f'}}(s, v)$, by our minimal choice of v, the distance to u did not decrease:

$$d_{G_{f'}}(s, u) \geq d_{G_f}(s, u). \tag{7.35}$$

Now consider the status of edge (u, v) in G_f before the augmentation:

1. **Case 1:** $(u, v) \in E_f$. Then $d_{G_f}(s, v) \leq d_{G_f}(s, u) + 1$. Combining with (7.34) and (7.35) gives $d_{G_f}(s, v) \leq d_{G_{f'}}(s, u) + 1 = d_{G_{f'}}(s, v)$, contradicting the assumption that the distance to v decreased.
2. **Case 2:** $(u, v) \notin E_f$. Since $(u, v) \in E_{f'}$, the augmentation along P must have created (u, v) as a reverse residual edge. This implies that path P must have traversed (v, u) in the forward direction. Since P was a shortest path in G_f:

   $$d_{G_f}(s, u) = d_{G_f}(s, v) + 1. \tag{7.36}$$

   Substituting into (7.34) and (7.35):

   $$d_{G_{f'}}(s, v) = d_{G_{f'}}(s, u) + 1 \geq d_{G_f}(s, u) + 1 = d_{G_f}(s, v) + 2. \tag{7.37}$$

   This contradicts $d_{G_{f'}}(s, v) < d_{G_f}(s, v)$.

In both cases, we reach a contradiction. Hence $d_{G_f}(s, v)$ never decreases. □

Theorem 7.11 *Edmonds–Karp Bound on Augmentations*

The Edmonds–Karp algorithm performs at most $\frac{1}{2}|V||E|$ augmentations. Since each BFS takes $\mathcal{O}(|E|)$ time, the total running time is $\mathcal{O}(|V| \cdot |E|^2)$, which is strongly polynomial.

Proof. An edge (u, v) on an augmenting path P is called *critical* if it achieves the bottleneck capacity: $c_f(u, v) = \delta(P)$. Every augmentation saturates at least one critical edge, causing (u, v) to disappear from the residual network.

For (u, v) to become critical again in a subsequent augmentation, flow must first be pushed in the opposite direction along (v, u), which requires (v, u) to appear on some intermediate augmenting path. Let f be the flow when (u, v) was first critical, and let f' be the flow when (v, u) was subsequently augmented. By Theorem 7.10:

$$d_{G_f}(s, v) = d_{G_f}(s, u) + 1, \tag{7.38}$$

$$d_{G_{f'}}(s, u) = d_{G_{f'}}(s, v) + 1 \geq d_{G_f}(s, v) + 1 = d_{G_f}(s, u) + 2. \tag{7.39}$$

Thus, between two consecutive occurrences of (u, v) being critical, the distance from s to u must increase by at least 2. Because the maximum shortest-path distance in G_f from s to any vertex cannot exceed $|V| - 1$ without creating a cycle, edge (u, v) can become critical at most:

$$\frac{|V| - 1}{2} \leq \frac{|V|}{2} \text{ times.} \tag{7.40}$$

Since each of the $2|E|$ directed edges (forward and backward) can be critical at most $|V|/2$ times, and every augmentation saturates at least one edge, the total number of augmentations is at most:

$$2|E| \times \frac{|V|}{4} = \frac{1}{2}|V||E|. \tag{7.41}$$

Each augmenting path is discovered in $\mathcal{O}(|E|)$ time via BFS, yielding an overall running time of $\mathcal{O}(|V| \cdot |E|^2)$. □

7.4.4 Maximum Capacity Augmenting Paths

Another intuitive strategy is the *Maximum Capacity Augmenting Path* (or *fattest path*) rule: at each iteration, choose the path that maximizes the bottleneck capacity $\delta(P) = \min_{e \in P} c_f(e)$.

This path can be discovered using a priority queue in $\mathcal{O}(|E| \log |V|)$ time by modifying Dijkstra's algorithm (replacing the additive path length with the min-operator). A standard volume-doubling argument shows that the number of augmentations is at most $\mathcal{O}(|E| \log U)$, yielding a total running time of $\mathcal{O}(|E|^2 \log |V| \log U)$.

7.4.5 Comparison of Algorithmic Variants

Table 7.1 summarizes the theoretical characteristics of maximum flow algorithms.

7.5 Object-Oriented Implementation

7.5.1 The FlowEdge Data Structure

In modern algorithm engineering, the residual network G_f is not constructed as an explicit separate graph. Instead, each directed edge (u, v) stores both its capacity $c(u, v)$ and its

Algorithm	Path Selection Strategy	Time Complexity	Bound Type
Generic Ford–Fulkerson	Arbitrary path (DFS)	$\mathcal{O}(\lvert V\rvert \cdot \lvert E\rvert \cdot U)$	Pseudo-polynomial
Capacity Scaling	Bottleneck capacity $\geq \Delta$	$\mathcal{O}(\lvert E\rvert^2 \log U)$	Weakly polynomial
Fattest Path	Maximum bottleneck (Dijkstra)	$\mathcal{O}(\lvert E\rvert^2 \log \lvert V\rvert \log U)$	Weakly polynomial
Edmonds–Karp	Shortest path in edges (BFS)	$\mathcal{O}(\lvert V\rvert \cdot \lvert E\rvert^2)$	Strongly polynomial
Dinitz Algorithm	Blocking flows in layered networks	$\mathcal{O}(\lvert V\rvert^2 \cdot \lvert E\rvert)$	Strongly polynomial
Goldberg–Tarjan	Push-relabel with FIFO queue	$\mathcal{O}(\lvert V\rvert^3)$	Strongly polynomial

Table 7.1: Comparison of maximum flow algorithmic paradigms and their worst-case complexities.

current flow $f(u, v)$. Methods determine residual capacities and augment flows symmetrically from the perspective of either endpoint.

Listing 7.1 presents the `FlowEdge` class.

```
public class FlowEdge {
    private final int v;            // from vertex
    private final int w;            // to vertex
    private final double capacity; // forward capacity
    private double flow;            // current flow

    public FlowEdge(int v, int w, double capacity) {
        if (v < 0 || w < 0) {
            throw new IllegalArgumentException("Negative vertex");
        }
        if (capacity < 0.0) {
            throw new IllegalArgumentException("Negative capacity");
        }
        this.v = v;
        this.w = w;
        this.capacity = capacity;
        this.flow = 0.0;
    }

    public int from() { return v; }
    public int to() { return w; }
    public double capacity() { return capacity; }
    public double flow() { return flow; }

    public int other(int vertex) {
        if (vertex == v) return w;
        if (vertex == w) return v;
        throw new IllegalArgumentException("Invalid endpoint");
    }

    public double residualCapacityTo(int vertex) {
        if (vertex == v) return flow;              // backward edge
        if (vertex == w) return capacity - flow;  // forward edge
        throw new IllegalArgumentException("Invalid endpoint");
    }

    public void addResidualFlowTo(int vertex, double delta) {
```

```
        if (delta < 0.0) {
            throw new IllegalArgumentException("Negative delta");
        }
        if (vertex == v) flow -= delta;           // backward edge
        else if (vertex == w) flow += delta;      // forward edge
        else throw new IllegalArgumentException("Invalid endpoint");
    }
}
```

Listing 7.1: The capacitated edge data structure `FlowEdge.java`.

7.5.2 The FlowNetwork Representation

The `FlowNetwork` class maintains an adjacency list for each vertex. Crucially, when an edge $e = (v, w)$ is added, reference e is inserted into *both* `adj[v]` and `adj[w]`. When traversing from v, e represents a forward edge to w; when traversing from w, e represents a backward edge to v. This allows the residual network to be queried implicitly in $\mathcal{O}(1)$ time per edge.

```
import java.util.LinkedList;
import java.util.Queue;

public class FlowNetwork {
    private final int V;
    private int E;
    private Queue<FlowEdge>[] adj;

    @SuppressWarnings("unchecked")
    public FlowNetwork(int V) {
        if (V < 0) throw new IllegalArgumentException("Negative V");
        this.V = V;
        this.E = 0;
        this.adj = (Queue<FlowEdge>[]) new LinkedList[V];
        for (int v = 0; v < V; v++) {
            adj[v] = new LinkedList<FlowEdge>();
        }
    }

    public int V() { return V; }
    public int E() { return E; }

    public void addEdge(FlowEdge e) {
        int v = e.from();
        int w = e.to();
        adj[v].add(e); // forward reference
        adj[w].add(e); // backward reference
        E++;
    }

    public Iterable<FlowEdge> adj(int v) {
        return adj[v];
    }
}
```

Listing 7.2: Adjacency-list network representation `FlowNetwork.java`.

7.5.3 The Ford-Fulkerson Solver

Listing 7.3 implements the Edmonds–Karp algorithm. Method `hasAugmentingPath` performs a Breadth-First Search using a `Queue<Integer>`. It tracks visited vertices via `marked[]` and

remembers the discovering edge in `edgeTo[]`. Once an augmenting path is confirmed, the constructor computes its bottleneck capacity, pushes flow along the path, and updates the total flow value.

```
import java.util.LinkedList;
import java.util.Queue;

public class FordFulkerson {
    private final int V;
    private boolean[] marked;
    private FlowEdge[] edgeTo;
    private double value;

    public FordFulkerson(FlowNetwork G, int s, int t) {
        this.V = G.V();
        if (s == t || s < 0 || s >= V || t < 0 || t >= V) {
            throw new IllegalArgumentException("Invalid s or t");
        }
        this.value = 0.0;

        while (hasAugmentingPath(G, s, t)) {
            // Compute bottleneck capacity along path
            double bottle = Double.POSITIVE_INFINITY;
            for (int v = t; v != s; v = edgeTo[v].other(v)) {
                bottle = Math.min(bottle,
                    edgeTo[v].residualCapacityTo(v));
            }

            // Augment residual flow along path
            for (int v = t; v != s; v = edgeTo[v].other(v)) {
                edgeTo[v].addResidualFlowTo(v, bottle);
            }

            value += bottle;
        }
    }

    private boolean hasAugmentingPath(FlowNetwork G, int s, int t) {
        edgeTo = new FlowEdge[V];
        marked = new boolean[V];

        Queue<Integer> queue = new LinkedList<Integer>();
        queue.add(s);
        marked[s] = true;

        while (!queue.isEmpty() && !marked[t]) {
            int v = queue.remove();
            for (FlowEdge e : G.adj(v)) {
                int w = e.other(v);
                if (e.residualCapacityTo(w) > 0.0 && !marked[w]) {
                    edgeTo[w] = e;
                    marked[w] = true;
                    queue.add(w);
                }
            }
        }
        return marked[t];
    }

    public double value() { return value; }

```

```
58      public boolean inCut(int v) {
59          if (v < 0 || v >= V) {
60              throw new IllegalArgumentException("Vertex out of bounds");
61          }
62          return marked[v];
63      }
64  }
```

Listing 7.3: Edmonds–Karp maximum flow solver `FordFulkerson.java`.

7.6 Modeling Reductions and Extensions

The standard maximum flow problem requires a single source s, a single sink t, and simple upper bounds $0 \le f(e) \le c(e)$ on directed edges. In practical applications, problem descriptions frequently introduce complicating side constraints: multiple suppliers, node capacity limits, lower bounds, or bipartite matching structures. We now demonstrate how these variants can be transformed into standard single-source, single-sink maximum flow instances.

7.6.1 Multiple Sources and Multiple Sinks

Suppose a network has a set of k source vertices $S = \{s_0, s_1, \dots, s_{k-1}\}$ with supplies $S_i \ge 0$, and a set of ℓ sink vertices $T = \{t_0, t_1, \dots, t_{\ell-1}\}$ with demands $D_j \ge 0$.

Proposition 7.12 *Supersource and Supersink Transformation*

A multi-source, multi-sink flow problem is equivalent to a standard single-source, single-sink maximum flow problem on an augmented network $G' = (V \cup \{s^*, t^*\}, E')$ where:

1. A new *supersource* s^* is added, with a directed edge (s^*, s_i) of capacity $c(s^*, s_i) = S_i$ to each original source $s_i \in S$.
2. A new *supersink* t^* is added, with a directed edge (t_j, t^*) of capacity $c(t_j, t^*) = D_j$ from each original sink $t_j \in T$.

Any feasible flow from s^* to t^* in G' satisfies original conservation laws at all internal vertices, and the capacity constraints on edges from s^* and into t^* guarantee that individual supplies and demands are respected.

7.6.2 Vertex Capacities and Node Splitting

In telecommunication routing or railway yards, vertices themselves often represent physical switches or junction stations with bounded throughput capacities $c_V(v) < \infty$. The flow passing through vertex v must satisfy:

$$\sum_{u\colon (u,v)\in E} f(u,v) \le c_V(v). \tag{7.42}$$

Proposition 7.13 *Node Splitting Technique*

A network $G = (V, E)$ with vertex capacities $c_V \colon V \to \mathbb{R}_{\ge 0}$ is transformed into an equivalent network $G' = (V', E')$ with edge capacities only by splitting each vertex $v \in V$ into two vertices v_{in} and v_{out}:

1. For each $v \in V$, create an internal directed edge $(v_{\text{in}}, v_{\text{out}})$ with capacity $c'(v_{\text{in}}, v_{\text{out}}) = c_V(v)$.
2. For each original directed edge $(u, v) \in E$, replace it with directed edge $(u_{\text{out}}, v_{\text{in}})$ with capacity $c'(u_{\text{out}}, v_{\text{in}}) = c(u, v)$.
3. Set source $s' = s_{\text{out}}$ and sink $t' = t_{\text{in}}$.

Flow conservation across $(v_{\text{in}}, v_{\text{out}})$ enforces the vertex throughput constraint (7.42) exactly.

7.6.3 Lower Bounds and Edge Demands

In many distribution systems, contracts impose minimum delivery quantities: each edge $e \in E$ has both a lower bound $d(e) \geq 0$ and an upper bound $c(e) \geq d(e)$, requiring:

$$d(u, v) \leq f(u, v) \leq c(u, v), \quad \forall (u, v) \in E. \tag{7.43}$$

Unlike standard flow problems where $f \equiv 0$ is always feasible, determining whether a feasible flow exists under positive lower bounds is non-trivial.

Proposition 7.14 *Feasible Flow with Demands Reduction*

Let $G = (V, E)$ have edge lower bounds $d(e)$ and capacities $c(e)$. Define the vertex demand imbalance $b(v)$ as the net minimum flow required to leave vertex v:

$$b(v) = \sum_{w\colon (v,w)\in E} d(v, w) - \sum_{u\colon (u,v)\in E} d(u, v), \quad \forall v \in V. \tag{7.44}$$

Construct an auxiliary network $G' = (V \cup \{s', t'\}, E')$:

1. Add feedback edge (t, s) with capacity $c'(t, s) = \infty$ and lower bound 0 (converting s-t flow into a circulation).
2. For each original edge $(u, v) \in E$, set shifted capacity $c'(u, v) = c(u, v) - d(u, v) \geq 0$.
3. For each vertex $v \in V$:
   - If $b(v) < 0$ (flow surplus), add edge (s', v) with capacity $c'(s', v) = -b(v)$.
   - If $b(v) > 0$ (flow deficit), add edge (v, t') with capacity $c'(v, t') = b(v)$.

Then G admits a feasible flow satisfying (7.43) if and only if the maximum flow in G' from s' to t' saturates all edges leaving s'. If feasible, the original flow is recovered by setting $f(u, v) = f'(u, v) + d(u, v)$.

7.6.4 Bipartite Matching and Domain Filtering

As introduced in Section 7.1.4, the *Maximum Bipartite Matching Problem* on an undirected graph $G = (U \cup V, E)$ seeks a subset of edges $M \subseteq E$ of maximum cardinality such that no two edges in M share a common endpoint. By orienting all edges from U to V with capacity 1, connecting supersource s to each $u \in U$ with capacity 1, and connecting each $v \in V$ to supersink t with capacity 1, the maximum flow value equals $|M^*|$.

Beyond computing a single maximum matching, network flows play a central role in *Constraint Programming* (CP; see Chapter 9). In 1994, Jean-Charles Régin introduced

a filtering algorithm for the global constraint AllDifferent$(x_0, \ldots, x_{n-1})$ that achieves Generalized Arc Consistency (GAC) in $\mathcal{O}(n^{2.5})$ time [Rég94].

Régin's algorithm operates on the bipartite value graph between variables and domain values:

1. Compute a maximum matching M in the bipartite graph via maximum flow. If $|M| < n$, the constraint is unsatisfiable.
2. Construct the residual graph G_M oriented with respect to matching M (matching edges oriented from V to U, non-matching edges from U to V).
3. Compute the *Strongly Connected Components* (SCCs) of G_M in linear time $\mathcal{O}(n+m)$ using Tarjan's algorithm [Tar75].
4. An edge (u, v) can belong to *some* maximum matching if and only if it belongs to M, or its endpoints reside in the same SCC of G_M, or it belongs to an alternating path starting from an unmatched vertex.
5. Any edge (u, v) violating this condition can never participate in any valid assignment and is pruned from the domain of variable u.

7.7 Minimum-Cost Flow

7.7.1 Mathematical Formulation

The Maximum Flow Problem maximizes the quantity of commodity routed through a network. In many industrial applications, transporting commodity incurs monetary, physical, or environmental costs. The *Minimum-Cost Flow Problem* (MCF) seeks to route a required target flow volume $B \in \mathbb{R}_{\geq 0}$ from source s to sink t at minimum total cost.

Let $a(u, v)$ denote the cost per unit of flow routed along edge $(u, v) \in E$. The problem is formulated as:

$$\begin{aligned} \text{minimize} \quad & \sum_{(u,v)\in E} a(u,v)\, f(u,v) \\ \text{subject to} \quad & \sum_{w\colon (v,w)\in E} f(v,w) - \sum_{u\colon (u,v)\in E} f(u,v) = \begin{cases} B & \text{if } v = s, \\ -B & \text{if } v = t, \\ 0 & \forall v \in V \setminus \{s,t\}, \end{cases} \\ & 0 \leq f(u,v) \leq c(u,v), \quad \forall (u,v) \in E. \end{aligned} \tag{7.45}$$

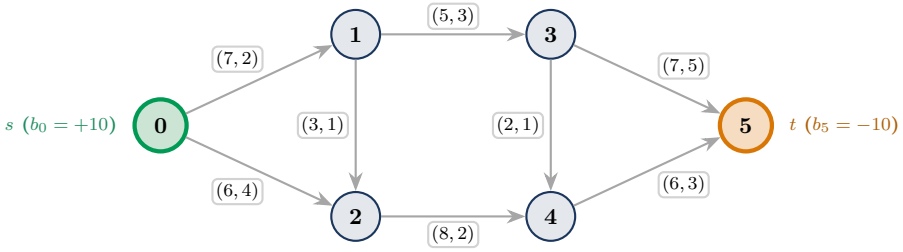
7.7.2 Residual Costs and Negative Cycles

When routing flow through a residual network G_f, each residual edge is assigned a modified unit cost $a_f(u, v)$:

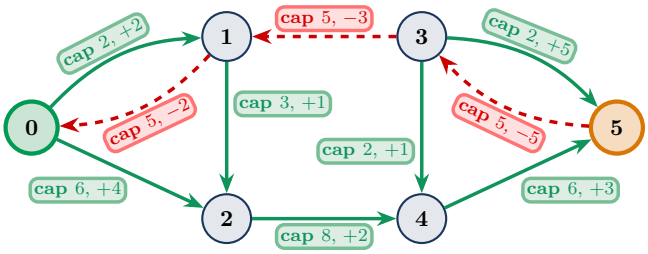
- For a forward residual edge $(u, v) \in E_f$, pushing flow increases the objective by $a(u, v)$:
$$a_f(u, v) = a(u, v). \tag{7.46}$$
- For a backward residual edge $(v, u) \in E_f$, pushing flow *cancels* flow previously sent along (u, v), refunding its unit cost:
$$a_f(v, u) = -a(u, v). \tag{7.47}$$

The presence of negative edge costs in G_f (Figure 7.5) leads to Klein's cycle-canceling optimality criterion [Kle67]:

(a) Minimum-Cost Network: Labels $(c_e, a_e) = (\textbf{capacity}, \textbf{unit cost})$, Target Flow $B = 10$



(b) Residual Network G_f after routing 5 units along $0 \to 1 \to 3 \to 5$ (unit path cost: $2 + 3 + 5 = 10$)



Arc Legend: → **Forward arc**: $c_f = c - f$, cost $+a$ ⇢ **Backward arc**: $c_f = f$, cost $-a$ (*refund*)
Successive Shortest Path: Bellman–Ford finds minimum-cost augmenting paths in G_f with negative costs.

Figure 7.5: The Minimum-Cost Flow Problem. (a) A network with capacities and unit costs (c_e, a_e) where $B = 10$ units of flow must be routed from $s = 0$ to $t = 5$. (b) Residual network G_f after routing 5 units along path $0 \to 1 \to 3 \to 5$. Backward residual arcs emerge with negative unit costs $a_f(v, u) = -a(u, v)$, representing the cost refund gained by canceling previously routed flow.

Theorem 7.15 *Negative Cycle Optimality Condition*

A feasible flow f of value B is a minimum-cost flow if and only if its residual graph G_f contains no directed cycle of negative total cost.

Proof. If G_f contains a directed cycle C with $\sum_{e \in C} a_f(e) < 0$, pushing $\delta = \min_{e \in C} c_f(e) > 0$ units around C preserves all conservation equations and flow value B while reducing the total cost by $\delta \cdot |\sum_{e \in C} a_f(e)|$. Hence f cannot be optimal. Conversely, if f' is an optimal flow of value B and f is non-optimal, the difference $f' - f$ decomposes into a set of cycles in G_f, at least one of which must have negative cost. □

7.7.3 The Successive Shortest Path Algorithm

Rather than canceling cycles from an existing feasible flow of value B, the *Successive Shortest Path* (SSP) algorithm starts with an initial flow of zero ($f \equiv 0$) and iteratively routes flow along shortest paths in G_f from s to t until the target demand B is met.

Algorithm 11: Successive Shortest Path Algorithm

Input: Network $G = (V, E)$, source s, sink t, capacities c, unit costs a, target flow B
Output: Minimum-cost flow f^*

```
Initialize f(u, v) ← 0 for all (u, v) ∈ E;
while v(f) < B do
    Find a shortest path P from s to t in G_f with respect to edge costs a_f;
    if no directed s-t path exists in G_f then
        return Target flow B is infeasible;
    δ ← min {B − v(f), min_{e∈P} c_f(e)};
    Augment flow along P by δ via (7.16);
return f;
```

Because backward edges in G_f have negative costs ($-a(u, v)$), Dijkstra's algorithm cannot be applied directly without modification. Instead, the *Moore–Bellman–Ford algorithm* is employed to find shortest paths in $\mathcal{O}(|V| \cdot |E|)$ time per augmentation.

Theorem 7.16 *Complexity of Successive Shortest Path*

If edge capacities and target demand B are integers, the Successive Shortest Path algorithm performs at most B unit augmentations, terminating in $\mathcal{O}(B \cdot |V| \cdot |E|)$ time.

Node Potentials and Dijkstra Acceleration. To avoid executing the $\mathcal{O}(|V||E|)$ Moore–Bellman–Ford routine at every augmentation, modern solvers maintain dual vertex potentials $\pi : V \to \mathbb{R}$. The *reduced cost* of edge (u, v) is defined as:

$$a^{\pi}(u, v) = a(u, v) + \pi(u) - \pi(v). \tag{7.48}$$

If potentials are updated at each iteration using shortest-path distances, reduced costs satisfy $a_f^{\pi}(u, v) \geq 0$ for all $e \in E_f$, enabling Dijkstra's algorithm with Fibonacci heaps to run in $\mathcal{O}(|E| + |V| \log |V|)$ time per augmentation [AMO93]. Furthermore, by applying capacity scaling to the target demands, polynomial-time bounds are achieved.

7.8 Chapter Summary and Exercises

7.8.1 Key Conceptual Takeaways

Table 7.2 synthesizes the core mathematical principles and algorithmic foundations established in this chapter.

Concept	Key Result and Algorithmic Significance
s-t Cut Capacity	Asymmetric definition: only forward edges $A \to B$ count towards capacity $\text{cap}(A, B)$.
Flow Conservation	Inflow equals outflow at all intermediate nodes; net flow across any cut equals $v(f)$.
Max-Flow Min-Cut	Equivalence: $\max v(f) = \min \text{cap}(A, B)$; dual of Max-Flow is the Min-Cut continuous LP.
Total Unimodularity	Vertex-edge incidence matrix is TUM; continuous Simplex guarantees integer flows.
Residual Graph G_f	Encodes available forward capacity $(c - f)$ and reversible backward flow (f).
Pathological DFS	Arbitrary path choices cause pseudo-polynomial running time $\mathcal{O}(\|V\|\|E\|U)$.
Capacity Scaling	Halving Δ filters small bottlenecks, achieving weakly polynomial $\mathcal{O}(\|E\|^2 \log U)$ time.
Edmonds–Karp	BFS shortest augmenting paths guarantee strongly polynomial $\mathcal{O}(\|V\| \cdot \|E\|^2)$ time.
Minimum-Cost Flow	Solved by successive shortest paths; backward edges refund cost $(a_f = -a)$.

Table 7.2: Foundational concepts in network flow optimization.

7.8.2 Exercises

Exercise 7.1 *Multi-Source Multi-Sink LP Formulation*

Consider a distribution network $G = (V, E)$ with two production plants s_0, s_1 with maximum supply capacities 40 and 50 units, and three retail distribution centers t_0, t_1, t_2 with demands 25, 35, and 30 units.

1. Formulate the maximum flow problem on this network as a continuous Linear Program in standard canonical form.
2. Construct the equivalent single-source single-sink network G' using supersource s^* and supersink t^*, specifying all node indices and edge capacities.
3. Formulate the dual linear program and state the complementary slackness conditions governing optimal flows and shadow prices.

Exercise 7.2 *Step-by-Step Ford-Fulkerson Trace*

Consider a network with vertices $V = \{0, 1, 2, 3, 4, 5\}$, source $s = 0$, sink $t = 5$, and directed capacities:

$$c(0,1) = 10, \quad c(0,2) = 10, \quad c(1,2) = 2, \quad c(1,3) = 4, \quad c(1,4) = 8,$$

$$c(2,4) = 9, \quad c(3,5) = 10, \quad c(4,3) = 6, \quad c(4,5) = 10.$$

1. Apply the Edmonds–Karp algorithm (BFS path selection) starting from zero flow. At each iteration, specify:
   - The augmenting path discovered in G_f.

- The bottleneck capacity δ.
- The updated flow assignment across all edges.

2. Draw the final residual graph G_{f^*} upon termination.
3. Identify the set of vertices A reachable from $s = 0$ in G_{f^*}, and compute the capacity of the resulting cut $(A, V \setminus A)$.
4. Verify that the maximum flow value equals the minimum cut capacity.

Exercise 7.3 *Capacity Scaling Execution Trace*

On the network from Exercise 7.2:

1. Determine the initial scaling parameter $\Delta = 2^{\lfloor \log_2 U \rfloor}$.
2. Trace the execution of the Capacity Scaling Algorithm (Algorithm 10) through each phase ($\Delta = 8, 4, 2, 1$).
3. Count the total number of augmenting paths discovered. Compare this count with the number of augmentations performed by the Edmonds–Karp algorithm.

Exercise 7.4 *Exact Node Throughput Demands*

Suppose each vertex $v \in V \setminus \{s, t\}$ in a network has an exact prescribed transit throughput $D(v) > 0$, requiring:

$$\sum_{u\colon (u,v)\in E} f(u, v) = \sum_{w\colon (v,w)\in E} f(v, w) = D(v).$$

1. Apply the node splitting transformation (Theorem 7.13) to convert this problem into a network with edge bounds.
2. Explain how to verify whether a feasible flow satisfying all node demands exists using a single maximum flow computation.

Exercise 7.5 *Circulations with Lower Bounds*

Consider a directed graph $G = (V, E)$ where each edge e has a lower bound $d(e) = 2$ and an upper capacity $c(e) = 6$.

1. Show that setting all flows to zero ($f \equiv 0$) is infeasible.
2. Apply the reduction in Theorem 7.14 to construct the auxiliary network G'.
3. Prove that if the maximum flow in G' saturates all edges leaving auxiliary source s', then setting $f(e) = f'(e) + d(e)$ yields a feasible circulation in G.

Exercise 7.6 *Project Selection and Maximum Weight Closure*

A company considers a set of potential projects $P = \{0, 1, \ldots, n-1\}$. Each project $i \in P$ has a net financial yield $p_i \in \mathbb{R}$ (positive for profitable projects, negative for costly prerequisite tasks). Prerequisites are modeled as a directed acyclic graph (DAG): directed edge (i, j) signifies that project i cannot be undertaken unless project j is also completed. A subset $S \subseteq P$ is *closed* if for every $i \in S$ and every $(i, j) \in E$, $j \in S$. The

goal is to find a closed subset $S^* \subseteq P$ maximizing total profit $\sum_{i \in S} p_i$.

1. Construct a network $G' = (P \cup \{s, t\}, E')$:
   - For each project i with $p_i > 0$, add edge (s, i) with capacity $c(s, i) = p_i$.
   - For each project j with $p_j < 0$, add edge (j, t) with capacity $c(j, t) = -p_j$.
   - For each dependency $(i, j) \in E$, add edge (i, j) with capacity $c(i, j) = \infty$.
2. Prove that any finite-capacity s-t cut (A, B) in G' corresponds to a valid closed subset $S = A \setminus \{s\}$.
3. Prove that minimizing cut capacity $\text{cap}(A, B)$ is mathematically equivalent to maximizing the total profit $\sum_{i \in S} p_i$.

Exercise 7.7 *Successive Shortest Path for Minimum-Cost Flow*

Consider a network with vertices $V = \{0, 1, 2, 3\}$, source $s = 0$, sink $t = 3$, and target demand $B = 4$. Directed edges have capacity and unit cost pairs (c_e, a_e):

$$e_1 = (0, 1) \colon (3, 2), \quad e_2 = (0, 2) \colon (3, 5), \quad e_3 = (1, 2) \colon (2, 1),$$

$$e_4 = (1, 3) \colon (2, 6), \quad e_5 = (2, 3) \colon (3, 2).$$

1. Trace the Successive Shortest Path algorithm (Algorithm 11) step-by-step from zero flow until $B = 4$ units are delivered.
2. At each step, identify the shortest path in G_f, verify that no negative cost cycles are created, and compute the cumulative routing cost.
3. Verify that the final flow assignment minimizes the total cost function (7.45).

CHAPTER 8

Local Search and Metaheuristics

Exact optimization paradigms—such as Dynamic Programming, Branch-and-Bound, Integer Linear Programming, and Lagrangian Relaxation—provide mathematical certificates of global optimality. Yet, when problem instances scale to tens of thousands of decision variables or when strict real-time operational deadlines must be met, the exponential worst-case complexity of exact search often becomes prohibitive. In such regimes, practitioners turn to *heuristic* and *metaheuristic* algorithms, trading theoretical optimality guarantees for the ability to discover high-quality feasible solutions rapidly [AL03; HS04; Tai23].

This chapter develops the mathematical principles, neighborhood design methodology, and algorithmic engines of *Local Search*. We begin with *constructive heuristics*—greedy procedures, Beam Search, and the Pilot Method—which synthesize an initial feasible solution element by element. We then formalize *neighborhood structures* and iterative improvement strategies (first-improvement and steepest-descent pivot rules), analyzing the five fundamental design properties of a neighborhood: connectivity, low diameter, low ruggedness, small cardinality, and fast incremental (Δ) evaluation. Using the Traveling Salesman Problem (TSP) as a canonical testbed, we study classical fixed-depth neighborhoods (2-Opt, 3-Opt, and Or-Opt), problem reductions (transforming the Asymmetric TSP into a Symmetric TSP), and variable-depth neighborhood extensions including Filter-and-Fan and the celebrated *Lin–Kernighan* ejection chain heuristic. Next, we address the fundamental pathology of local search—entrapment in local optima and plateaus—through two complementary paradigms: *Large Neighborhood Search* (LNS) and *Tabu Search*. Finally, we examine the architecture of *Constraint-Based Local Search* (CBLS), illustrating the interplay between hard and soft constraints on the Sudoku puzzle and the `IFlat-IRelax` algorithm for cumulative resource scheduling.

8.1 Constructive Heuristics

Before an iterative improvement algorithm can modify a candidate solution, a starting solution must be synthesized. While uniform random generation is sometimes possible, constraints often make random sampling non-trivial, and the resulting solutions typically exhibit poor objective values. *Constructive heuristics* build a complete feasible solution incrementally, starting from an empty or trivial partial structure and adding one elementary component at each step.

8.1.1 The Greedy Paradigm

Let $E = \{e_0, e_1, \ldots, e_{m-1}\}$ denote a finite ground set of elementary components from which any feasible solution is assembled. For instance, in the Traveling Salesman Problem over vertices $V = \{0, 1, \ldots, n-1\}$, the elements of E may represent cities to be appended to a partial sequence or edges to be selected into a tour; in graph coloring, an element may represent the assignment of a color to a vertex.

Definition 8.1 *Greedy Constructive Heuristic*

A *greedy constructive heuristic* maintains a partial solution $s \subseteq E$ (initialized to a trivial base state, typically $s = \emptyset$) and a candidate set $R \subseteq E \setminus s$ of admissible extensions that can be added to s without violating feasibility. Equipped with an *incremental cost function* $c(s, e) \in \mathbb{R}$, the algorithm repeatedly selects the myopically best element

$$e' \in \arg\min_{e \in R} c(s, e), \tag{8.1}$$

augments the partial solution via $s \leftarrow s \cup \{e'\}$, and prunes from R all elements that have become incompatible with the updated partial solution s, until $R = \emptyset$ and s forms a complete solution.

The general architecture of a greedy constructive procedure is formalized in Algorithm 12. A defining characteristic of pure greedy construction is that it *never questions a past choice*: once an element e' is committed to s, it is never retracted.

Algorithm 12: General Greedy Constructive Heuristic

Input: Initial trivial partial solution s (typically $\emptyset$); ground set E of solution elements; incremental cost function $c(s, e)$
Output: Complete feasible solution s

$R \leftarrow E$; `// Elements that can be added to` s
while $R \neq \emptyset$ **do**
  Compute $c(s, e)$ for all $e \in R$;
  Select $e' \in \arg\min_{e \in R} c(s, e)$;
  $s \leftarrow s \cup \{e'\}$; `// Include` e' `in partial solution` s
  Remove from R all elements that can no longer be feasibly added to s;
return s;

8.1.2 Greedy Heuristics for the TSP

Even for a single optimization problem, the generic template of Algorithm 12 yields markedly different algorithms depending on how the ground set E, the incremental cost $c(s, e)$, and the feasibility filter on R are defined. Consider the symmetric Traveling Salesman Problem on n cities $V = \{0, 1, \ldots, n-1\}$ with pairwise distances $d(u, v) = d(v, u)$:

1. **Greedy on Edge Weight (Multi-Fragment Heuristic)**:
   - *Element* e: An undirected edge $e = \{u, v\} \in E$.
   - *Incremental cost*: The static edge weight $c(s, e) = d(u, v)$.

- *Admissibility filter*: Adding e to the partial edge set s must neither create a vertex of degree 3 nor close a premature cycle (subtour) of length strictly less than n.

This heuristic mimics Kruskal's minimum spanning tree algorithm, supplemented by degree-2 and subtour-elimination checks. It grows multiple disconnected path fragments simultaneously and merges them into a single Hamiltonian cycle (Figure 8.1a).

2. **Nearest Neighbor**:
   - *Initialization*: Start at an arbitrary city (e.g., $v_0 = 0$), setting $s = [0]$ and $R = V \setminus \{0\}$.
   - *Element e*: An unvisited city $j \in R$.
   - *Incremental cost*: The distance $c(s, j) = d(u, j)$ from the *last added city* u of the partial tour to candidate city j.
   - *Selection*: Append $j^* \in \arg\min_{j \in R} d(u, j)$ to the end of the path; once $R = \emptyset$, close the tour by returning to city 0.

   While Nearest Neighbor runs in $\mathcal{O}(n^2)$ time and produces well-connected initial segments, its myopia is severely punished near the end of the construction: isolated cities overlooked early in the process force long, expensive closing jumps (Figure 8.1b).
3. **Cheapest Insertion**:
   - *Initialization*: Form a 2-city degenerate subtour between two cities (e.g., city 0 and its nearest neighbor).
   - *Element e*: An unvisited city $k \in R$.
   - *Incremental cost*: The minimum *detour cost* required to insert city k between two consecutive cities (i, j) of the current partial subtour s:

$$c(s, k) = \min_{(i,j) \in s} \Delta(i, k, j), \quad \text{where } \Delta(i, k, j) = d(i, k) + d(k, j) - d(i, j). \tag{8.2}$$

   - *Selection*: Choose the unvisited city $k^* \in \arg\min_{k \in R} c(s, k)$ that minimizes the detour cost and insert it into the edge (i, j) achieving the minimum in (8.2) (Figure 8.1c).
4. **Farthest Insertion**:
   - *Initialization*: Start from a 2-city subtour connecting the two mutually farthest cities in V.
   - *Element e and Cost*: For each unvisited city $k \in R$, evaluate its minimum insertion detour $c(s, k)$ as in (8.2) (or, in a common geometric variant, its minimum distance $\min_{i \in s} d(i, k)$ to any city already in the subtour).
   - *Selection*: Choose the unvisited city $k^* \in \arg\max_{k \in R} c(s, k)$ with the *largest* incremental cost, and insert k^* into the current subtour at the position $(i^*, j^*) \in \arg\min_{(i,j) \in s} \Delta(i, k^*, j)$ that minimizes the detour (Figure 8.1d).

Why Farthest Insertion Outperforms Cheapest Insertion. At first glance, selecting the city with the *largest* incremental cost in Farthest Insertion appears counterintuitive for a minimization problem. Geometrically, however, Farthest Insertion prioritizes outliers: within the first few iterations, it constructs a coarse bounding polygon spanning the perimeter of the point cloud. Subsequent insertions merely refine the interior along existing edges where detours are small. Conversely, Cheapest Insertion builds a tight, localized cluster and must repeatedly stretch and deform its topology as distant cities are forced in late in the construction.

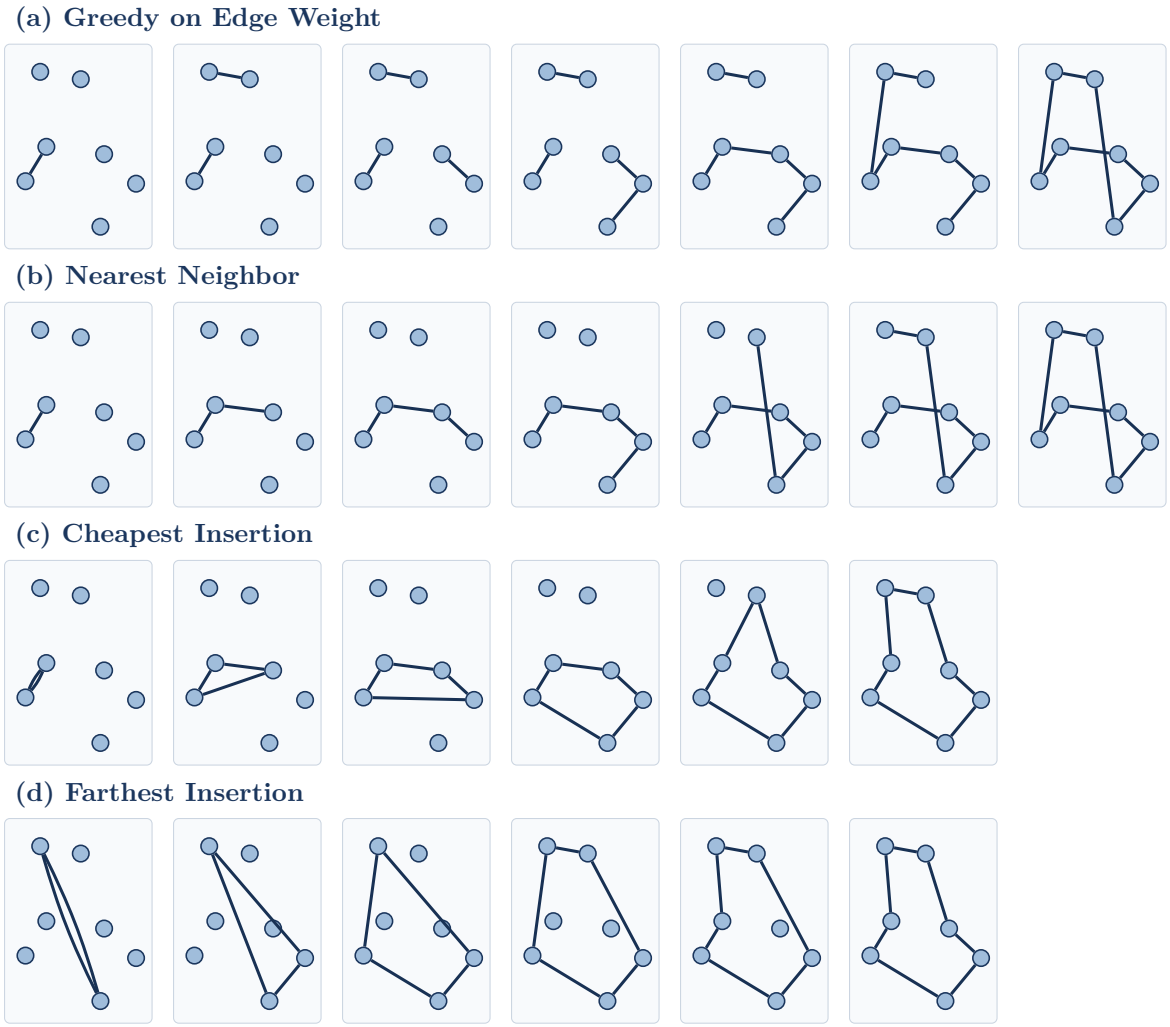


Figure 8.1: Step-by-step execution of four constructive greedy heuristics on a planar 7-city TSP instance ($V = \{0, \ldots, 6\}$). (a) Greedy on Edge Weight merges disjoint fragments while rejecting degree-3 and subtour edges. (b) Nearest Neighbor myopically extends the active endpoint, incurring long closing edges at the end. (c) Cheapest Insertion grows a localized cycle via minimum detours. (d) Farthest Insertion establishes the global outer hull first and inserts interior cities last.

Table 8.1 summarizes the structural properties and computational complexities of these four constructive TSP heuristics, and Listings 8.1 and 8.2 provide a clean Java implementation of the Nearest Neighbor and Farthest Insertion rules returning a `Candidate` tour object (whose full class definition, together with incremental 2-Opt methods, is given in Listing 8.3).

Heuristic	**Element** e	**Selection Rule**	**Time**	**Structural Behavior**
Edge Weight	Edge $\{u, v\}$	$\min d(u, v)$	$\mathcal{O}(n^2 \log n)$	Kruskal-like; builds disjoint paths, avoids degree 3 and subtours.
Nearest Neighbor	City $j \in R$	$\min d(u_{\text{last}}, j)$	$\mathcal{O}(n^2)$	Prim-like at one endpoint; fast but suffers from long final jumps.
Cheapest Insertion	City $k \in R$	$\min_k \min_{(i,j)} \Delta(i, k, j)$	$\mathcal{O}(n^2)$	Expands a local subtour smoothly; sensitive to late outliers.
Farthest Insertion	City $k \in R$	$\max_k \min_{(i,j)} \Delta(i, k, j)$	$\mathcal{O}(n^2)$	Captures global perimeter early; yields superior initial tours.

Table 8.1: Comparison of classical constructive greedy heuristics for the symmetric TSP.

```java
public class ConstructiveTSP {

    public static Candidate nearestNeighbor(
            TSPInstance tsp, int startCity) {
        int n = tsp.nCities();
        boolean[] visited = new boolean[n];
        int[] tour = new int[n];
        tour[0] = startCity;
        visited[startCity] = true;

        for (int step = 1; step < n; step++) {
            int curr = tour[step - 1], bestNext = -1;
            double bestDist = Double.POSITIVE_INFINITY;
            for (int j = 0; j < n; j++) {
                if (!visited[j]
                        && tsp.distance(curr, j) < bestDist) {
                    bestDist = tsp.distance(curr, j);
                    bestNext = j;
                }
            }
            tour[step] = bestNext;
            visited[bestNext] = true;
        }
        return new Candidate(tsp, tour);
    }
```

Listing 8.1: Java implementation of the Nearest Neighbor constructive heuristic for the TSP.

To observe how these constructive strategies behave at scale, Figure 8.2 plots the evolution of the cumulative objective value $f(s_k)$ across construction steps $k \in \{60, \ldots, 200\}$ on a 200-city planar Euclidean TSP instance with coordinates sampled uniformly in $[0, 1000]^2$, alongside two lookahead extensions of Nearest Neighbor—Beam Search and the Pilot Method—introduced next in Section 8.1.3. The trajectories reveal three distinct regimes:

```java
    public static Candidate farthestInsertion(TSPInstance tsp) {
        int n = tsp.nCities();
        boolean[] inTour = new boolean[n];
        List<Integer> subtour = new ArrayList<>(n);

        // Start with the two mutually farthest cities
        int u0 = 0, v0 = 1;
        double maxDist = -1.0;
        for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++)
                if (tsp.distance(i, j) > maxDist) {
                    maxDist = tsp.distance(i, j);
                    u0 = i; v0 = j;
                }
        subtour.add(u0); subtour.add(v0);
        inTour[u0] = true; inTour[v0] = true;

        while (subtour.size() < n) {
            int chosenCity = -1, chosenPos = -1;
            double maxMinDelta = Double.NEGATIVE_INFINITY;
            int m = subtour.size();
            for (int k = 0; k < n; k++) {
                if (inTour[k]) continue;
                double minDelta = Double.POSITIVE_INFINITY;
                int bestPosForK = -1;
                for (int idx = 0; idx < m; idx++) {
                    int i = subtour.get(idx);
                    int j = subtour.get((idx + 1) % m);
                    double delta = tsp.distance(i, k)
                        + tsp.distance(k, j) - tsp.distance(i, j);
                    if (delta < minDelta) {
                        minDelta = delta;
                        bestPosForK = idx + 1;
                    }
                }
                if (minDelta > maxMinDelta) {
                    maxMinDelta = minDelta;
                    chosenCity = k; chosenPos = bestPosForK;
                }
            }
            subtour.add(chosenPos, chosenCity);
            inTour[chosenCity] = true;
        }
        int[] tour = subtour.stream()
            .mapToInt(Integer::intValue).toArray();
        return new Candidate(tsp, tour);
    }
}
```

Listing 8.2: Java implementation of the Farthest Insertion constructive heuristic for the TSP.

- **Myopic Late-Stage Regret**: Greedy on Edge Weight and Nearest Neighbor maintain the lowest cumulative cost during the first 150 steps by greedily selecting short local edges. However, during the final 25 steps ($k \in [175, 200]$), both methods suffer a dramatic cost spike as they are forced to link isolated degree-1 path endpoints or forgotten cities across the plane, finishing at 12,145.9 and 12,563.3, respectively.
- **Lookahead Mitigation**: Augmenting Nearest Neighbor with Beam Search ($p = 5$) or the Pilot Method deliberately accepts slightly longer transitions during the first 160 steps in order to visit peripheral cities before they become isolated. This foresight dampens the late-stage spike, reducing the final tour length from 12,563.3 down to 12,028.5 (Beam Search) and 11,817.1 (Pilot Method).
- **Early Perimeter Stabilization**: Farthest Insertion exhibits the exact opposite profile. By step $k = 60$, its partial cost has already reached 9,290.0 because it spans the outer bounding hull of the 200-city cloud first, but the curve subsequently flattens out: for $k \geq 150$, the remaining interior cities lie almost directly on existing subtour edges and are absorbed with near-zero detour ($\Delta \approx 0$), achieving the best initial tour of cost 11,468.0.

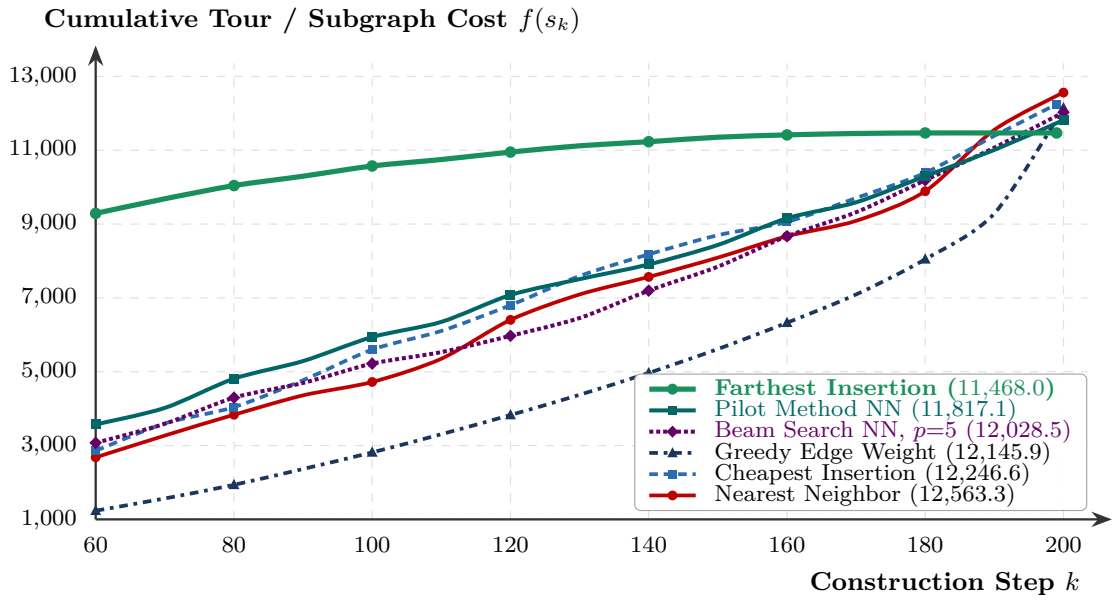


Figure 8.2: Evolution of the cumulative partial tour/subgraph cost $f(s_k)$ from construction step $k = 60$ to $k = 200$ for the four greedy initialization heuristics and two lookahead extensions (Beam Search and the Pilot Method applied to Nearest Neighbor) on a 200-city Euclidean TSP instance in $[0, 1000]^2$. Lookahead mitigates the late-stage cost spike of myopic Nearest Neighbor (12,563.3 → 12,028.5 → 11,817.1), while Farthest Insertion establishes the global perimeter early and plateaus to 11,468.0.

8.1.3 Beam Search and the Pilot Method

Because a standard greedy algorithm commits immediately to a single element e' based solely on the immediate incremental cost $c(s, e)$, it is inherently *myopic*: an attractive local step may trap the partial solution in a dead end or force exorbitant costs several steps later. Two general meta-strategies—Beam Search and the Pilot Method—inject lookahead into constructive heuristics while controlling combinatorial explosion (Figure 8.3).

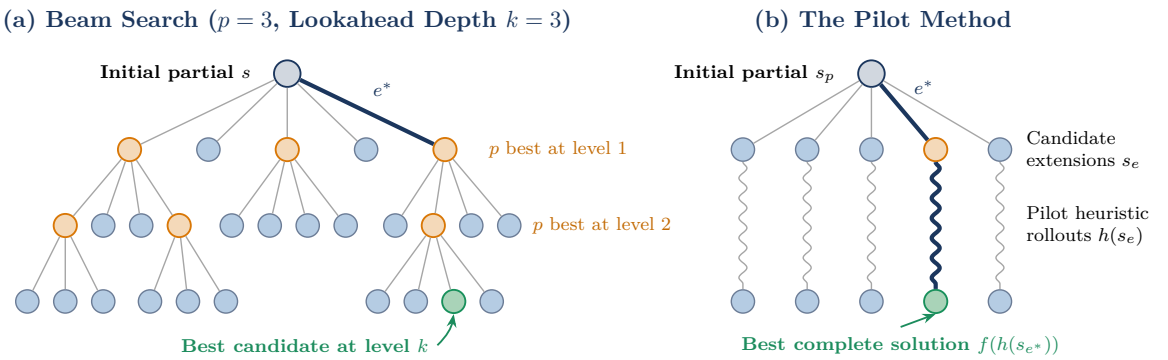


Figure 8.3: Lookahead extensions for constructive heuristics. (a) Beam Search expands a truncated breadth-first tree of width $p = 3$ up to lookahead depth k, committing to the root branch e^* that leads to the best level-k partial solution. (b) The Pilot Method probes each candidate extension s_e with a complete greedy rollout $h(s_e)$ and commits to the extension e^* yielding the best complete solution.

Beam Search

Beam Search (Figure 8.3a, Algorithm 13) generalizes the greedy construction of Algorithm 12 by replacing the single myopic choice with a truncated breadth-first lookahead parameterized by a *beam width* $p \in \mathbb{Z}_{\geq 1}$ and a *lookahead depth* $k \in \mathbb{Z}_{\geq 1}$:

- Starting from the current partial solution s_p with cumulative cost $F(s_p)$, level 1 generates all admissible one-element extensions $\mathcal{C}_1 = \{s_p \cup \{e\} \mid e \in R(s_p)\}$ and retains only the p lowest-cost partial candidates in the active beam $\mathcal{B}_1$, where $F(s \cup \{e\}) = F(s) + c(s, e)$.
- At each subsequent lookahead level $\ell \in \{2, \ldots, k\}$, every partial solution $s \in \mathcal{B}_{\ell-1}$ is expanded by all elements $e \in R(s)$ that can still be feasibly added to s, forming the candidate pool $\mathcal{C}_\ell$. To prevent exponential growth, $\mathcal{C}_\ell$ is immediately filtered to retain only the p best partial solutions overall ($\mathcal{B}_\ell \subseteq \mathcal{C}_\ell$ with $|\mathcal{B}_\ell| \leq p$).
- Once lookahead depth k is reached (or no further elements can be added), the algorithm identifies the best partial solution $\hat{s} \in \arg\min_{s \in \mathcal{B}_{\ell_{\max}}} F(s)$ at the last evaluated level, traces $\hat{s}$ back to the unique level-1 element $e^* \in R(s_p)$ that initiated its branch from s_p, and permanently commits to $s_p \leftarrow s_p \cup \{e^*\}$.

Note that setting $(p, k) = (1, 1)$ recovers the standard greedy algorithm (Algorithm 12), whereas setting $k = n$ propagates a single beam of width p all the way to complete solutions in a single pass. Because at most p partial solutions are expanded at each of the k lookahead levels, selecting each element e^* requires at most $\mathcal{O}(p \cdot k \cdot |E|)$ evaluations rather than $\mathcal{O}(|E|^k)$.

The Pilot Method

A subtle weakness of Beam Search is that comparing partial solutions of size $\ell < n$ using only the accumulated prefix cost $F(s)$ can be misleading: a partial tour may appear cheap simply because it has deferred visiting distant regions of the graph (which can be partially mitigated by augmenting $F(s)$ with a fast lower-bound estimate over the unvisited vertices, as done for the Beam Search curve in Figure 8.2). The *Pilot Method* resolves this evaluation bias directly by evaluating every candidate extension on the basis of a *complete* solution (Figure 8.3b, Algorithm 14):

1. Given the current partial solution s_p, tentatively append each admissible element $e \in R$

Algorithm 13: Beam Search Constructive Heuristic

Input: Trivial partial solution s_p (typically $\emptyset$); ground set E of solution elements; incremental cost function $c(s, e)$ inducing cumulative partial cost $F(s)$; beam width $p \geq 1$; lookahead depth $k \geq 1$

Output: Complete feasible solution s_p

while $R(s_p) \neq \emptyset$ **do**
  $\mathcal{C}_1 \leftarrow \{s_p \cup \{e\} \mid e \in R(s_p)\}$; `// All 1-step extensions of` s_p
  $\mathcal{B}_1 \leftarrow \text{Best}_p(\mathcal{C}_1, F)$; `// Keep` p `best candidates at level 1`
  $\ell_{\max} \leftarrow 1$;
  for $\ell = 2$ **to** k **do**
    $\mathcal{C}_\ell \leftarrow \bigcup_{s \in \mathcal{B}_{\ell-1}} \{s \cup \{e\} \mid e \in R(s)\}$;
    if $\mathcal{C}_\ell = \emptyset$ **then**
      break ; `// Complete solutions reached before depth` k
    $\mathcal{B}_\ell \leftarrow \text{Best}_p(\mathcal{C}_\ell, F)$; `// Keep` p `best candidates at level` ℓ
    $\ell_{\max} \leftarrow \ell$;
  Select $\hat{s} \in \arg\min_{s \in \mathcal{B}_{\ell_{\max}}} F(s)$; `// Best candidate at last evaluated level`
  Let $e^* \in R(s_p)$ be the level-1 element added to s_p on the branch leading to $\hat{s}$;
  $s_p \leftarrow s_p \cup \{e^*\}$; `// Commit` e^* `to partial solution` s_p
  Update $R(s_p)$ by removing elements that can no longer be feasibly added to s_p;
return s_p;

to form a candidate partial solution $s_e = s_p \cup \{e\}$.

2. From each s_e, launch a fast constructive *pilot heuristic* $h(s_e)$ (such as Nearest Neighbor or Cheapest Insertion) until a complete feasible solution $s = h(s_e)$ is obtained.
3. Evaluate the true objective $f(s)$ of each completed rollout, and permanently commit to the single extension $s_p \leftarrow s_{e^*}$ that produced the best complete solution.

If the underlying pilot heuristic $h(\cdot)$ runs in $\mathcal{O}(n^2)$ time and $|R| \leq n$ extensions are probed at each of the n construction steps, the Pilot Method requires $\mathcal{O}(n^4)$ time overall (which can be reduced to $\mathcal{O}(n^3)$ by restricting probes to a constant-size candidate list of nearest elements). As shown in Figure 8.2, guiding Nearest Neighbor with full pilot rollouts improves the final 200-city tour from 12,563.3 to 11,817.1, outperforming both Greedy on Edge Weight (12,145.9) and Beam Search (12,028.5).

8.2 Neighborhoods and Local Improvement

Once a complete feasible solution $s \in \mathcal{S}$ has been constructed, *Local Search* seeks to improve it iteratively by applying small, localized perturbations. Rather than maintaining an exponential search tree of partial states, local search maintains a single current state s and navigates through the solution space by moving to adjacent states.

Algorithm 14: The Pilot Constructive Method

Input: Trivial partial solution s_p; ground set E of solution elements; pilot heuristic $h(s_e)$ completing any partial solution s_e; objective function $f(s)$ to minimize
Output: Complete solution s^*

$R \leftarrow E$; `// Elements that can be added to` s_p
$f(s^*) \leftarrow \infty$;
while $R \neq \emptyset$ **do**
  $v \leftarrow \infty$;
  foreach $e \in R$ **do**
    Extend s_p with e to obtain partial solution $s_e = s_p \cup \{e\}$;
    Apply pilot heuristic $s \leftarrow h(s_e)$ to obtain a complete solution;
    if $f(s) \leq v$ **then**
      $v \leftarrow f(s)$;
      $s_r \leftarrow s_e$;
      if $f(s) < f(s^*)$ **then**
        $s^* \leftarrow s$; `// Record improved complete incumbent`
  $s_p \leftarrow s_r$; `// Commit best element to partial solution` s_p
  Remove from R all elements that can no longer be feasibly added to s_p;
return s^*;

8.2.1 Neighborhood Structures and Local Optima

Definition 8.2 *Neighborhood Structure and Moves*

Let $(\mathcal{S}, f)$ be a discrete optimization instance with feasible solution set $\mathcal{S}$ and objective (or fitness) function $f\colon \mathcal{S} \to \mathbb{R}$ to be minimized. A *neighborhood structure* is a mapping

$$\mathcal{N}\colon \mathcal{S} \to 2^{\mathcal{S}} \tag{8.3}$$

that associates with each candidate solution $s \in \mathcal{S}$ a subset $\mathcal{N}(s) \subseteq \mathcal{S}$ of *neighboring solutions* reachable from s in a single atomic transition.
In practice, $\mathcal{N}(s)$ is specified implicitly through a set of admissible *moves* $M(s)$. Applying a move $m \in M(s)$ to s, denoted $s' = s \oplus m$, transforms s into the neighbor s', so that

$$\mathcal{N}(s) = \{ s' \in \mathcal{S} \mid s' = s \oplus m,\ m \in M(s) \}. \tag{8.4}$$

The simplest local search scheme—known as *iterative improvement*, *hill climbing* (for maximization), or *descent* (for minimization)—repeatedly replaces the current solution s with a strictly improving neighbor $s' \in \mathcal{N}(s)$ until no such neighbor exists (Algorithm 15).

Definition 8.3 *Local and Global Optima*

Let $(\mathcal{S}, f)$ be a minimization problem equipped with neighborhood structure $\mathcal{N}$.

1. A solution $s^* \in \mathcal{S}$ is a *global optimum* if $f(s^*) \leq f(s)$ for all $s \in \mathcal{S}$.
2. A solution $\hat{s} \in \mathcal{S}$ is a *local optimum* with respect to $\mathcal{N}$ if no neighbor in $\mathcal{N}(\hat{s})$ is

Algorithm 15: Iterative Improvement (Descent Local Search)

Input: Initial feasible solution $s \in \mathcal{S}$; neighborhood structure $\mathcal{N}(\cdot)$; objective $f(\cdot)$ to minimize
Output: Locally optimal solution s

```
repeat
    if there exists a neighbor s' ∈ N(s) with f(s') < f(s) then
        s ← s';
until no neighbor s' ∈ N(s) satisfies f(s') < f(s);
return s;
```

strictly better than $\hat{s}$:

$$f(\hat{s}) \leq f(s'), \quad \forall s' \in \mathcal{N}(\hat{s}). \tag{8.5}$$

Every global optimum is trivially a local optimum for any neighborhood structure $\mathcal{N}$, whereas the converse rarely holds in **NP**-hard combinatorial problems. The quality of the local optimum returned by Algorithm 15 depends both on the initial solution s and on the richness of $\mathcal{N}$.

8.2.2 Pivot Rules: First vs. Best Improvement

When multiple improving moves exist in $\mathcal{N}(s)$, Algorithm 15 must specify a *pivot rule* to decide which neighbor $s' \in \mathcal{N}(s)$ to adopt:

- **First-Improvement Policy**: The neighborhood $\mathcal{N}(s)$ is scanned in a deterministic or randomized order. As soon as the first neighbor s' satisfying $f(s') < f(s)$ is encountered, the move $s \leftarrow s'$ is executed immediately, breaking the inner loop to begin scanning $\mathcal{N}(s')$ (Algorithm 16).
- **Best-Improvement (Steepest-Descent) Policy**: The entire neighborhood $\mathcal{N}(s)$ is evaluated exhaustively at every iteration, and the algorithm transitions to the neighbor achieving the largest cost reduction, $s \leftarrow \arg\min_{s' \in \mathcal{N}(s)} f(s')$ (Algorithm 17).

Algorithm 16: Single Pass of First-Improvement Policy

Input: Current solution s; neighborhood $\mathcal{N}(\cdot)$; objective $f(\cdot)$ to minimize
Output: Updated solution s

```
foreach s' ∈ N(s) do
    if f(s') < f(s) then
        s ← s' ;            // Move immediately and restart scan from new s
        break;
return s;
```

From a worst-case theoretical standpoint, because $\mathcal{S}$ is finite and every accepted move strictly decreases $f(s)$, both policies are guaranteed to terminate in at most $|\mathcal{S}|$ steps (and for integer-valued objectives, at most $f(s_0) - f(s^*)$ steps). In practice, this bound is vastly pessimistic. Empirically, on random two-dimensional Euclidean TSP instances using the 2-Opt neighborhood, Best-Improvement scales as $\Theta(n^{3.25})$ whereas First-Improvement scales

Algorithm 17: Iterative Improvement with Best-Improvement Policy

Input: Initial solution s; neighborhood $\mathcal{N}(\cdot)$; objective $f(\cdot)$ to minimize
Output: Locally optimal solution s

```
repeat
    end ← true;
    v_best ← ∞;
    foreach s' ∈ N(s) do
        if f(s') < v_best then
            v_best ← f(s');
            s_best ← s';
    if v_best < f(s) then
        s ← s_best ;                          // Commit steepest-descent move
        end ← false;
until end;
return s;
```

as $\Theta(n^{2.33})$ while converging to local optima of virtually identical quality. Because First-Improvement avoids scanning all $\Theta(n^2)$ candidates before making each early improvement, it is almost universally preferred for pure descent when neighborhoods are large.

8.2.3 Design Principles for Neighborhoods

Designing an effective neighborhood structure $\mathcal{N}$ requires balancing five fundamental properties [Tai23]:

1. **Connectivity**: The neighborhood graph $G_{\mathcal{N}} = (\mathcal{S}, \{(s, s') \mid s' \in \mathcal{N}(s)\})$ is *connected* (or weakly optimally connected) if and only if for every feasible solution $s \in \mathcal{S}$, there exists a finite directed path of moves $s = s_0 \to s_1 \to \cdots \to s_t = s^*$ leading to a globally optimal solution s^*.
    - *Why it matters*: If $\mathcal{N}$ is connected, one does not strictly require multi-start restarts to reach s^*, and ergodic stochastic metaheuristics (such as Simulated Annealing, which accepts any neighbor $s' \in \mathcal{N}(s)$ with positive probability) enjoy asymptotic convergence guarantees to the global optimum.
    - *How to prove it*: To prove that $\mathcal{N}$ is connected, one provides a constructive algorithm that transforms any arbitrary solution $s_1 \in \mathcal{S}$ into any target solution $s_2 \in \mathcal{S}$ using only valid moves in $\mathcal{N}$.
2. **Low Diameter**: The diameter of $G_{\mathcal{N}}$—the maximum number of elementary moves required to transform any solution s_1 into any s_2—should be small (typically $\mathcal{O}(n)$, as in 2-Opt for the TSP).
3. **Low Ruggedness (Smoothness)**: The objective value $f(s')$ of a neighbor should correlate strongly with $f(s)$, avoiding erratic jumps that shatter the fitness landscape into countless poor local optima. For example, 2-Opt exhibits low ruggedness on symmetric TSP instances because only two edges change weight, but high ruggedness on asymmetric TSP instances because reversing a subpath flips the direction of $\mathcal{O}(n)$ directed arcs.
4. **Small Size**: The cardinality $|\mathcal{N}(s)|$ should be small enough (e.g., $\mathcal{O}(n)$ or $\mathcal{O}(n^2)$) that scanning the neighborhood does not become a computational bottleneck.

5. **Fast Incremental (Δ) Evaluation**: Given $f(s)$ and a candidate move $m \in M(s)$, the objective difference
$$\Delta(s, m) = f(s \oplus m) - f(s) \tag{8.6}$$
must be computable in $\mathcal{O}(1)$ (or small sublinear) time by inspecting only the components modified by m, rather than re-evaluating $f(s \oplus m)$ from scratch in $\mathcal{O}(n)$ time.

The Fundamental Antagonism of Neighborhood Design. These five ideal properties are inherently conflicting. Enlarging a neighborhood (e.g., passing from 2-Opt to 3-Opt or 4-Opt) reduces ruggedness, eliminates many shallow local minima, and shrinks the graph diameter, but blows up the neighborhood size from $\mathcal{O}(n^2)$ to $\mathcal{O}(n^3)$ or $\mathcal{O}(n^4)$. Conversely, a tiny neighborhood is fast to scan and evaluate, but creates a rugged landscape with a large diameter and deep local traps.

8.3 Neighborhoods for the Traveling Salesman

We now instantiate these concepts on the Traveling Salesman Problem over n cities $V = \{0, 1, \ldots, n-1\}$. A candidate tour is represented as a permutation array $\texttt{tour}[0 \ldots n-1]$, where $\texttt{tour}[k]$ is the k-th city visited and the tour closes implicitly via the edge $(\texttt{tour}[n-1], \texttt{tour}[0])$. For any city $u \in V$, we denote by s_u the immediate successor of u along the oriented tour.

8.3.1 The 2-Opt Neighborhood

Introduced by Croes in 1958 [Cro58], the 2-*Opt move* is the foundational local search operator for the symmetric TSP.

Definition 8.4 *2-Opt Move and Neighborhood*

Given a tour s, a 2-*Opt move* specified by two non-adjacent vertices $[u, v]$ removes the two tour edges (u, s_u) and (v, s_v) and reconnects the two resulting paths into a single valid Hamiltonian cycle by inserting the edges (u, v) and (s_u, s_v). The set of valid 2-Opt moves is:
$$M_{\text{2-Opt}}(s) = \{[u, v] \mid u, v \in V,\ u \neq v,\ v \neq s_u,\ u \neq s_v\}\,. \tag{8.7}$$
Because there are n choices for u and $n-3$ non-adjacent choices for v, and unordered pairs $\{u, v\}$ define the same edge exchange, the 2-Opt neighborhood contains exactly $|\mathcal{N}_{\text{2-Opt}}(s)| = \frac{n(n-3)}{2} = \Theta(n^2)$ neighbors.

As illustrated in Figure 8.4a, disconnecting (u, s_u) and (v, s_v) splits the cycle into two subpaths: $s_v \rightsquigarrow u$ and $s_u \rightsquigarrow v$. Reconnecting u to v and s_u to s_v forces the intermediate subpath $s_u \rightsquigarrow v$ to be traversed in *reverse order*. In a permutation array representation $\texttt{tour}[0 \ldots n-1]$, if $u = \texttt{tour}[i]$ and $v = \texttt{tour}[j]$ with $0 \leq i < j < n$ (and $(i, j) \neq (0, n-1)$), applying $\texttt{twoOpt}(i, j)$ simply reverses the contiguous subarray $\texttt{tour}[i+1 \ldots j]$ in $\mathcal{O}(j-i) = \mathcal{O}(n)$ time! Meanwhile, for any symmetric distance matrix d, the cost variation $\Delta(i, j)$ is evaluated in $\mathcal{O}(1)$ time:
$$\Delta_{\text{2-Opt}}(u, v) = d(u, v) + d(s_u, s_v) - d(u, s_u) - d(v, s_v). \tag{8.8}$$

In the Euclidean plane, if two edges (u, s_u) and (v, s_v) intersect at a point P, the triangle inequality on $\triangle(u, P, v)$ and $\triangle(s_u, P, s_v)$ guarantees that $d(u, v) + d(s_u, s_v) < d(u, s_u) + d(v, s_v)$, so uncrossing them via a 2-Opt move strictly improves the tour. Consequently, any 2-Opt local optimum for a 2D Euclidean TSP is completely free of crossing edges. However, the converse is a classic conceptual pitfall:

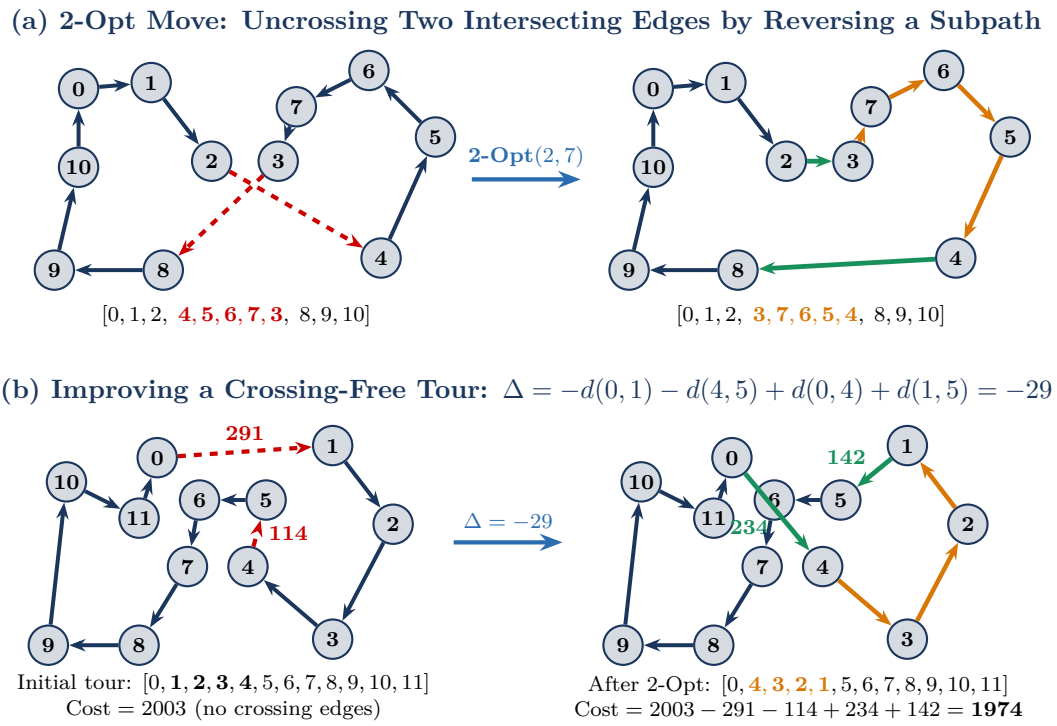


Figure 8.4: The 2-Opt neighborhood in action. (a) On an 11-city instance ($V = \{0, \ldots, 10\}$), removing intersecting edges $(2, 4)$ and $(3, 8)$ and adding $(2, 3)$ and $(4, 8)$ uncrosses the tour and reverses the subpath $[4, 5, 6, 7, 3] \to [3, 7, 6, 5, 4]$. (b) On a 12-city Euclidean instance ($V = \{0, \ldots, 11\}$) with no crossing edges, 2-Opt still improves the tour from 2003 to 1974 ($\Delta = -29$) by exchanging $(0, 1)$ and $(4, 5)$ for $(0, 4)$ and $(1, 5)$.

Example 8.1 *Improving a Crossing-Free Tour with 2-Opt*

Does the absence of crossing edges imply that a Euclidean TSP tour is 2-Opt optimal? Consider the 12-city planar Euclidean TSP instance ($V = \{0, 1, \ldots, 11\}$) depicted in Figure 8.4b, with the rounded Euclidean distance matrix given in Table 8.2. The initial tour

$$s = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]$$

has total cost $f(s) = 291+164+215+184+114+110+112+153+150+272+134+104 = 2003$ and contains *no crossing edges* in the plane.
Yet s is *not* a 2-Opt local minimum! Consider the 2-Opt move defined by vertices $u = 0$ ($s_0 = 1$) and $v = 4$ ($s_4 = 5$), which removes edges $(0, 1)$ of length 291 and $(4, 5)$ of length 114, and inserts edges $(0, 4)$ of length 234 and $(1, 5)$ of length 142. By (8.8), the incremental cost is:

$$\Delta_{\text{2-Opt}}(0, 4) = d(0, 4) + d(1, 5) - d(0, 1) - d(4, 5) = 234 + 142 - 291 - 114 = -29 < 0.$$

Reversing the subpath $[1, 2, 3, 4]$ yields the strictly better tour $s' = [0, 4, 3, 2, 1, 5, 6, 7, 8, 9, 10, 11]$ of total cost $f(s') = 2003 - 29 = 1974$. Thus, eliminating crossings is a *necessary* condition for 2-Opt optimality in the Euclidean plane, but it is not *sufficient*.

City	0	1	2	3	4	5	6	7	8	9	10	11
0	0	291	405	417	234	193	99	187	311	358	155	104
1	291	0	164	320	244	142	238	312	460	574	444	342
2	405	164	0	215	250	214	322	347	465	604	545	420
3	417	320	215	0	184	255	318	268	320	470	511	377
4	234	244	250	184	0	114	136	100	222	355	331	197
5	193	142	214	255	114	0	110	170	319	433	331	212
6	99	238	322	318	136	110	0	112	260	347	222	104
7	187	312	347	268	100	170	112	0	153	264	244	113
8	311	460	465	320	222	319	260	153	0	150	295	210
9	358	574	604	470	355	433	347	264	150	0	272	258
10	155	444	545	511	331	331	222	244	295	272	0	134
11	104	342	420	377	197	212	104	113	210	258	134	0

Table 8.2: Rounded Euclidean distance matrix for the 12-city TSP working example (Figure 8.4b), generated from planar coordinates $(210, 340)$, $(500, 360)$, $(600, 230)$, $(500, 40)$, $(360, 160)$, $(390, 270)$, $(280, 270)$, $(260, 160)$, $(180, 30)$, $(30, 30)$, $(60, 300)$, and $(180, 240)$.

We now establish the connectivity and diameter of the 2-Opt neighborhood.

Proposition 8.1 *Connectivity and Diameter of 2-Opt*

The 2-Opt neighborhood on the n-city TSP is connected, and its neighborhood graph has diameter $\mathcal{O}(n)$ (at most $n - 1$ moves are required to transform any tour s into any target tour s^*).

Proof. Represent both the current tour s and the target optimal tour s^* as permutation arrays of length n, rotated so that both start with the same city at index 0: $s[0] = s^*[0]$. We proceed inductively for index $k = 1, 2, \ldots, n - 2$, maintaining the invariant that the prefix $s[0 \ldots k - 1]$ matches $s^*[0 \ldots k - 1]$.

At step k, if $s[k] = s^*[k]$, no move is needed. Otherwise, locate the unique index $j > k$ where city $s^*[k]$ currently resides in s, so $s[j] = s^*[k]$. Applying the 2-Opt move $\texttt{twoOpt}(k-1, j)$ reverses the subarray $s[k \dots j]$, which places $s[j] = s^*[k]$ directly into position k while leaving the matched prefix $s[0 \dots k-1]$ untouched. After at most $n-2$ such reversals (plus at most one initial reversal if city $s^*[0]$ needed to be moved to index 0), s is identical to s^*. □

Example 8.2 *Transforming a Tour via 2-Opt Reversals*

Consider the 11-city tour from Figure 8.4a:

$$s = [0, 1, 2, 4, 5, 6, 7, 3, 8, 9, 10], \quad \text{and target } s^* = [2, 0, 1, 3, 6, 5, 4, 7, 8, 10, 9].$$

By sequentially reversing subarrays to match s^* from left to right:

1. Reverse $s[0 \dots 2]$ to bring 2 to index 0: $[2, 1, 0, 4, 5, 6, 7, 3, 8, 9, 10]$.
2. Reverse $s[1 \dots 2]$ to match prefix $[2, 0, 1]$: $[2, 0, 1, 4, 5, 6, 7, 3, 8, 9, 10]$.
3. Reverse $s[3 \dots 7]$ to bring 3 to index 3: $[2, 0, 1, 3, 7, 6, 5, 4, 8, 9, 10]$.
4. Reverse $s[4 \dots 7]$ to bring 4 to index 4, or directly reverse $s[4 \dots 7]$ into $[4, 5, 6, 7]$ and then reverse subsegments so $[6, 5, 4, 7]$ appears at indices $4 \dots 7$: specifically, reversing $s[4 \dots 5]$ gives $[2, 0, 1, 3, 6, 7, 5, 4, 8, 9, 10]$, reversing $s[5 \dots 6]$ gives $[2, 0, 1, 3, 6, 5, 7, 4, 8, 9, 10]$, and reversing $s[6 \dots 7]$ gives $[2, 0, 1, 3, 6, 5, 4, 7, 8, 9, 10]$.
5. Finally, reverse $s[9 \dots 10]$ to swap $(9, 10) \to (10, 9)$, reaching s^* in $7 \leq n-1$ moves.

Listing 8.3 and Listing 8.4 present the complete, self-contained Java implementation of the TSP `Candidate` tour representation (with $\mathcal{O}(1)$ incremental delta evaluation `twoOptDelta` and $\mathcal{O}(n)$ in-place array reversal `twoOpt`), along with the modular `LocalSearch` engine and the `FirstSelection` and `BestSelection` neighborhood strategies.

Constant-Time 2-Opt Move Application via Two-Level Trees. Although evaluating $\Delta_{\text{2-Opt}}$ takes $\mathcal{O}(1)$ time, reversing the subarray $\texttt{tour}[i+1 \dots j]$ in an array representation takes $\mathcal{O}(n)$ time. For massive TSP instances ($n \geq 10^5$), specialized data structures such as *two-level trees* (which partition the tour into $\Theta(\sqrt{n})$ doubly-linked segments equipped with a reverse-direction flag) or *splay trees* allow subpath reversals and successor queries to be performed in $\mathcal{O}(\sqrt{n})$ or amortized $\mathcal{O}(\log n)$ time (and $\mathcal{O}(1)$ for segment-level flips) [App+06; Tai23].

8.3.2 Higher-Order Moves: 3-Opt and Or-Opt

When a tour reaches a 2-Opt local minimum, we can enlarge the neighborhood by removing more than two edges simultaneously (Figures 8.5 and 8.6):

- **The 3-Opt Neighborhood**: A 3-Opt move deletes three mutually non-adjacent tour edges (i, s_i), (j, s_j), and (k, s_k), breaking the cycle into three directed subpaths $P_1 = (s_i \rightsquigarrow j)$, $P_2 = (s_j \rightsquigarrow k)$, and $P_3 = (s_k \rightsquigarrow i)$ initially traversed in the cyclic order $P_1 \to P_2 \to P_3 \to P_1$, and reconnects them into a single Hamiltonian cycle (Figure 8.5). Up to overall cycle orientation, there are $2^3 - 1 = 7$ distinct ways to reconnect the three segments into a single tour: three of them restore one of the deleted edges and degenerate to 2-Opt moves, while four are genuine 3-Opt exchanges (Figure 8.5b). Crucially, the specific reconnection

$$(i \to s_i),\ (j \to s_j),\ (k \to s_k) \quad \longrightarrow \quad (i \to s_j),\ (k \to s_i),\ (j \to s_k) \tag{8.9}$$

```java
public class Candidate {
    private int[] tour; // Permutation of cities 0..n-1
    private TSPInstance tsp;
    private double cost;

    public Candidate(TSPInstance tsp, int[] tour) {
        this.tsp = tsp;
        this.tour = tour.clone();
        this.cost = computeCost();
    }

    public double getCost() { return cost; }
    public int[] getTour() { return tour; }
    public int getSucc(int index) {
        return tour[(index + 1) % tour.length];
    }

    public double computeCost() {
        double sum = 0;
        for (int i = 0; i < tour.length; i++)
            sum += tsp.distance(tour[i], getSucc(i));
        return sum;
    }

    public double twoOptDelta(int index1, int index2) {
        int n = tour.length;
        int u = tour[index1], su = tour[(index1 + 1) % n];
        int v = tour[index2], sv = tour[(index2 + 1) % n];
        return tsp.distance(u, v) + tsp.distance(su, sv)
             - tsp.distance(u, su) - tsp.distance(v, sv);
    }

    public void twoOpt(int index1, int index2) {
        this.cost += twoOptDelta(index1, index2);
        int i = (Math.min(index1, index2) + 1) % tour.length;
        int j = Math.max(index1, index2);
        while (i < j) {
            int temp = tour[i];
            tour[i++] = tour[j];
            tour[j--] = temp;
        }
    }

    @Override
    public Candidate clone() {
        return new Candidate(tsp, this.tour);
    }
}
```

Listing 8.3: TSP candidate solution with $\mathcal{O}(1)$ incremental 2-Opt evaluation and in-place subpath reversal.

```java
public interface NeighborSelection {
    Candidate getNeighbor(Candidate candidate);
}

public class FirstSelection implements NeighborSelection {
    @Override
    public Candidate getNeighbor(Candidate candidate) {
        int n = candidate.getTour().length;
        for (int i = 0; i < n - 1; i++)
            for (int j = i + 2; j < n; j++) {
                if (i == 0 && j == n - 1) continue;
                if (candidate.twoOptDelta(i, j) < -1e-9) {
                    candidate.twoOpt(i, j);
                    return candidate;
                }
            }
        return candidate;
    }
}

public class BestSelection implements NeighborSelection {
    @Override
    public Candidate getNeighbor(Candidate candidate) {
        int n = candidate.getTour().length;
        int bestI = -1, bestJ = -1;
        double bestDelta = -1e-9;
        for (int i = 0; i < n - 1; i++)
            for (int j = i + 2; j < n; j++) {
                if (i == 0 && j == n - 1) continue;
                double delta = candidate.twoOptDelta(i, j);
                if (delta < bestDelta) {
                    bestDelta = delta;
                    bestI = i; bestJ = j;
                }
            }
        if (bestI != -1) candidate.twoOpt(bestI, bestJ);
        return candidate;
    }
}
```

Listing 8.4: First-Improvement and Best-Improvement 2-Opt neighborhood selectors.

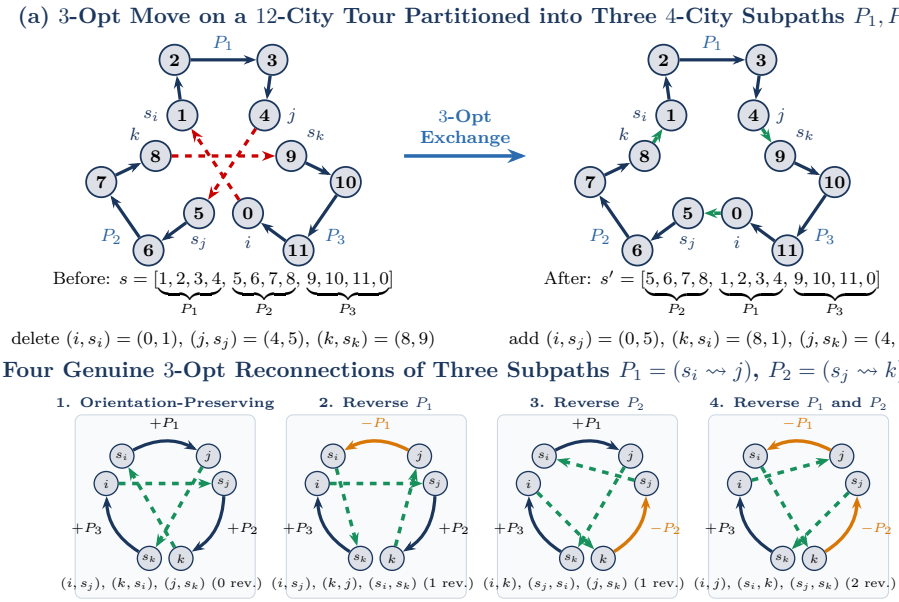


Figure 8.5: The 3-Opt neighborhood. (a) A 3-Opt move on a 12-city tour $s = [0, 1, \ldots, 11]$ where deleting $(i, s_i) = (0, 1)$, $(j, s_j) = (4, 5)$, and $(k, s_k) = (8, 9)$ splits the tour into three 4-city subpaths $P_1 = [1, 2, 3, 4]$, $P_2 = [5, 6, 7, 8]$, and $P_3 = [9, 10, 11, 0]$; reconnecting via $(i, s_j) = (0, 5)$, $(k, s_i) = (8, 1)$, and $(j, s_k) = (4, 9)$ eliminates all three crossings simultaneously. (b) The four genuine 3-Opt reconnections of P_1, P_2, and P_3 (one orientation-preserving move and three segment-reversing moves).

reorders the three subpaths into $P_2 \to P_1 \to P_3 \to P_2$ while *preserving the visiting orientation* of all three subpaths (Figure 8.5a and Case 1 of Figure 8.5b)! Because no segment is inverted, this move is directly applicable to the Asymmetric TSP (ATSP) with $\mathcal{O}(1)$ incremental cost evaluation. The drawback of full 3-Opt is its cubic neighborhood size $\Theta(n^3)$; while 3-Opt is still practical for medium-sized instances, exhaustive 4-Opt ($\Theta(n^4)$) almost never pays off computationally.

- **The Or-Opt Neighborhood**: Proposed by Ilhan Or in 1976 [Or76], the *Or-Opt neighborhood* restricts orientation-preserving 3-Opt moves (8.9) to those where the relocated subpath $P_1 = (s_i \rightsquigarrow j)$ consists of a short chain of $\ell \in \{1, 2, 3\}$ cities that are visited *consecutively in the current tour* s:
  - for $\ell = 1$, $j = s_i$, so the single city s_i is moved from between i and s_j and inserted between k and s_k (1-city insertion);
  - for $\ell = 2$, $j = s_{s_i}$, so the pair of consecutive tour cities $[s_i, j]$ is relocated together;
  - for $\ell = 3$, $j = s_{s_{s_i}}$, so the 3-city subpath $[s_i, s_{s_i}, j]$ is relocated together.

  Operationally, an Or-Opt move *unplugs* the ℓ-city subpath $s_i \rightsquigarrow j$ from its current tour position between i and s_j (closing the gap with the edge (i, s_j)) and *splices* it into another tour edge (k, s_k) (via the edges (k, s_i) and (j, s_k)), with $\mathcal{O}(1)$ incremental cost evaluation:

$$\Delta_{\text{Or-Opt}}(i,j,k) = \underbrace{\big[d(i,s_j) - d(i,s_i) - d(j,s_j)\big]}_{\text{unplugging } s_i \rightsquigarrow j \text{ between } i \text{ and } s_j} + \underbrace{\big[d(k,s_i) + d(j,s_k) - d(k,s_k)\big]}_{\text{splicing } s_i \rightsquigarrow j \text{ into edge } (k,s_k)}. \quad (8.10)$$

  For instance, in the 11-city tour $s = [0, 1, 2, \mathbf{3}, \mathbf{4}, \mathbf{5}, 6, 7, 8, 9, 10]$ of Figure 8.6, the three cities $[3, 4, 5]$ ($s_i = 3$ through $j = 5$, $\ell = 3$) are consecutive in the tour permutation

between $i = 2$ and $s_j = 6$, causing a long detour across the plane. Splicing the subpath $[3, 4, 5]$ into the bottom edge $(k, s_k) = (8, 9)$ produces the crossing-free tour $s' = [0, 1, 2, 6, 7, 8, \mathbf{3}, \mathbf{4}, \mathbf{5}, 9, 10]$ while keeping the internal order $3 \to 4 \to 5$ intact. In practice, Or-Opt is applied sequentially for $\ell = 3$, then $\ell = 2$, and finally $\ell = 1$: because j is uniquely fixed by i and ℓ, there are only n chains of length ℓ and $n - \ell - 1$ target insertion edges (k, s_k), reducing the neighborhood size from $\Theta(n^3)$ to $\Theta(n^2)$ while capturing the most impactful orientation-preserving 3-Opt moves.

- **Neighborhood Restriction (Candidate Lists)**: Even a quadratic $\Theta(n^2)$ neighborhood becomes expensive when $n \geq 10^4$. Because an optimal TSP tour almost exclusively uses short edges connecting geographically proximate cities, *neighborhood restriction* prunes the move set by precomputing, for each city $u \in V$, a sparse *candidate list* $\mathcal{C}(u)$ of promising incident edges. For planar Euclidean TSP instances, restricting candidate edges to the *Delaunay triangulation* (or the k-nearest neighbor graph with $k \in [15, 40]$) reduces the number of candidate edges from $\Theta(n^2)$ to $\Theta(n)$, accelerating each neighborhood scan to $\mathcal{O}(n)$ time with negligible loss in tour quality [App+06; Tai23].

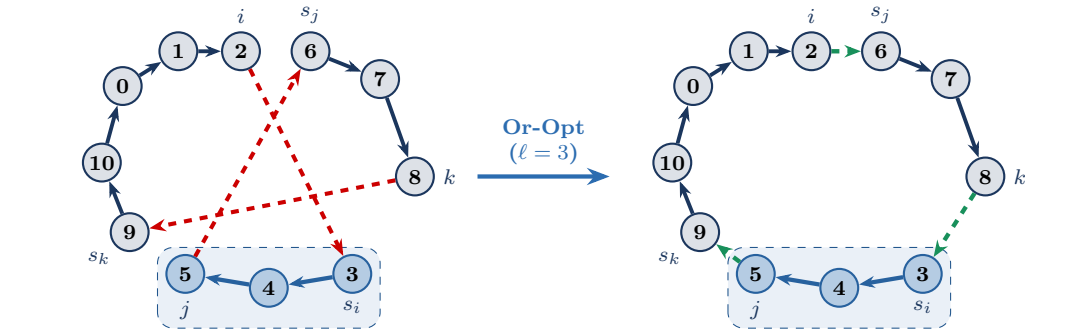


Figure 8.6: An Or-Opt move ($\ell = 3$) on an 11-city tour $s = [0, 1, 2, \mathbf{3}, \mathbf{4}, \mathbf{5}, 6, 7, 8, 9, 10]$. Deleting $(i, s_i) = (2, 3)$, $(j, s_j) = (5, 6)$, and $(k, s_k) = (8, 9)$ and reconnecting via $(i, s_j) = (2, 6)$, $(k, s_i) = (8, 3)$, and $(j, s_k) = (5, 9)$ unplugs the 3-city subpath $s_i \rightsquigarrow j$ ($[3, 4, 5]$) from between 2 and 6 and splices it between 8 and 9 without reversing any segment.

8.3.3 Reducing Asymmetric TSP to Symmetric TSP

In real-world urban routing, one-way streets, turn penalties, and traffic gradients produce an *asymmetric* distance matrix where $w_{u,v} \neq w_{v,u}$. As noted in Section 8.2.3, standard symmetric neighborhoods like 2-Opt perform poorly on the Asymmetric TSP (ATSP) because inverting a subpath reverses the traversal direction of $\mathcal{O}(n)$ arcs, invalidating $\mathcal{O}(1)$ incremental evaluation. Rather than redesigning specialized asymmetric neighborhoods, a classic reduction due to Jonker and Volgenant [JV83] transforms any n-city ATSP into an equivalent $2n$-city Symmetric TSP (STSP).

Theorem 8.2 *Jonker–Volgenant ATSP-to-STSP Reduction*

Let $G = (V, A, \mathbf{w})$ be a directed ATSP instance on n vertices $V = \{0, 1, \ldots, n-1\}$ with asymmetric arc weights $w_{u,v}$ for $u \neq v$. Construct an undirected bipartite graph $G' = (V \cup V', E', \mathbf{d})$ on $2n$ vertices, where $V = \{0, \ldots, n-1\}$ are the original nodes

and $V' = \{0', \ldots, (n-1)'\}$ are n twin copy nodes (Figure 8.7), with symmetric edge weights:

$$d(x, y) = \begin{cases} -M & \text{if } x = u \in V \text{ and } y = u' \in V' \text{ (twin coupling edge)}, \\ w_{u,v} & \text{if } x = u' \in V' \text{ and } y = v \in V \text{ with } u \neq v, \\ +\infty & \text{otherwise (between any } u, v \in V \text{ or any } u', v' \in V'), \end{cases} \tag{8.11}$$

where $M > 0$ is a sufficiently large constant (conceptually $+\infty$). Then there is a one-to-one correspondence between directed Hamiltonian cycles in G of cost C and undirected Hamiltonian cycles in G' of finite cost $C - nM$.

Proof. Because every twin edge $\{u, u'\}$ carries a large negative weight $-M$, any optimal undirected tour in G' must include all n coupling edges $\{u, u'\}$ for $u \in V$. Since no finite-weight edges exist between vertices within V or within V', the tour must alternate strictly between V and V'. Moreover, for any pair of distinct cities $u, v \in V$, the undirected tour in G' cannot select both $\{u', v\}$ and $\{v', u\}$ (unless $n = 2$), as combining them with $\{u, u'\}$ and $\{v, v'\}$ would close a 4-vertex subtour (u—u'—v—v'—u). Consequently, a consistent orientation of the undirected cycle visits each twin pair in the order $u \to u' \to v \to v' \ldots$, selecting exactly one directed transition $\{u', v\}$ of weight $w_{u,v}$ out of each u' and yielding a valid directed tour in G of cost C. □

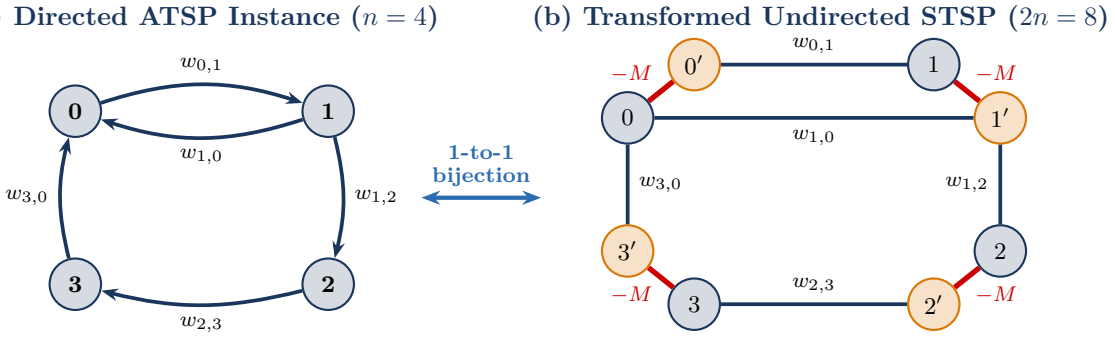


Figure 8.7: The Jonker–Volgenant reduction [JV83] transforming a 4-city Asymmetric TSP (a) into an 8-vertex Symmetric TSP (b). Each city $u \in \{0, 1, 2, 3\}$ is split into (u, u') linked by a heavy negative edge $-M$, while directed arc weight $w_{u,v}$ is assigned to the undirected edge $\{u', v\}$.

Example 8.3 *Urban Grid with One-Way Streets and LKH Resolution*

To see the Jonker–Volgenant reduction in action on a non-trivial routing network, consider the 7×7 urban street maze depicted in Figure 8.8a (rows $r \in \{0, \ldots, 6\}$ from top to bottom, columns $c \in \{0, \ldots, 6\}$ from left to right). Five square 1×1 city blocks together with one vertical 3×1 block ($c = 1, r \in \{3, 4, 5\}$) and one horizontal 1×3 block ($r = 5, c \in \{3, 4, 5\}$) partition the city into four horizontal one-way streets ($r \in \{0, 2, 4, 6\}$) and four vertical one-way avenues ($c \in \{0, 2, 4, 6\}$) that cross at sixteen street intersections $(r, c) \in \{0, 2, 4, 6\}^2$ (shaded in light blue):

- **Horizontal streets**: The two northern streets ($r = 0, 2$) run strictly **East** ($\rightarrow$), whereas the two southern streets ($r = 4, 6$) run strictly **West** ($\leftarrow$).
- **Vertical avenues**: The two western avenues ($c = 0, 2$) run strictly **North** ($\uparrow$), whereas the two eastern avenues ($c = 4, 6$) run strictly **South** ($\downarrow$).

A service vehicle must visit $n = 5$ checkpoints $V = \{0, 1, 2, 3, 4\}$ located at $0 = (2, 0)$, $1 = (2, 5)$, $2 = (4, 4)$, $3 = (6, 0)$, and $4 = (3, 2)$, and return to 0.
Computing all-pairs shortest directed paths by Breadth-First Search on the street graph yields the 5×5 asymmetric distance matrix $\boldsymbol{W}$ shown in Figure 8.8b, in which all 10 unordered pairs $\{u, v\}$ satisfy $w_{u,v} \neq w_{v,u}$:

- **Why Visual Perimeter and Greedy Heuristics Fail**: Starting from $0 = (2, 0)$ and driving east along $r = 2$, the closest checkpoint is $1 = (2, 5)$ ($w_{0,1} = 5$). A *Greedy Nearest-Neighbor* construction therefore commits to $0 \rightarrow 1$ (5), continues south-west to $2 = (4, 4)$ ($w_{1,2} = 5$), loops into the central checkpoint $4 = (3, 2)$ ($w_{2,4} = 3$), and is then trapped in the interior by the two long building blocks at $(4, 1)$ and $(5, 4)$: reaching $3 = (6, 0)$ from $4 = (3, 2)$ requires circling all the way to the east avenue $c = 6$ ($w_{4,3} = 15$), yielding the suboptimal tour $0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 0$ of cost $5 + 5 + 3 + 15 + 4 = 32$. Worse still, following the perimeter-plus-center index order $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$ costs $5 + 5 + 18 + 13 + 19 = 60$.
- **The Non-Planar Figure-Eight Optimum**: To discover the true global optimum, the vehicle must counter-intuitively *turn south at* $(2, 4)$ *before reaching* 1, visiting $2 = (4, 4)$ first ($w_{0,2} = 6$), looping west and north through $4 = (3, 2)$ ($w_{2,4} = 3$), and re-entering the eastbound street $r = 2$ to collect $1 = (2, 5)$ ($w_{4,1} = 4$) on a second pass before sweeping down the outer eastern avenue $c = 6$ to $3 = (6, 0)$ ($w_{1,3} = 11$) and north to 0 ($w_{3,0} = 4$). Notice that this trajectory crosses itself at intersection $(2, 4)$ in a figure-eight pattern—a configuration that can never be optimal in the Euclidean plane, yet achieves the unique minimum cost of $6 + 3 + 4 + 11 + 4 = \mathbf{28}$.

To find this non-trivial tour using a symmetric heuristic without modifying its move operators, we construct the 10-vertex Symmetric TSP on $V \cup V' = \{0, 1, 2, 3, 4, 0', 1', 2', 3', 4'\}$, shifting all incident weights by $+M$ (with $M = 25 > \max_{u,v} w_{u,v}$) so that every edge weight is non-negative:

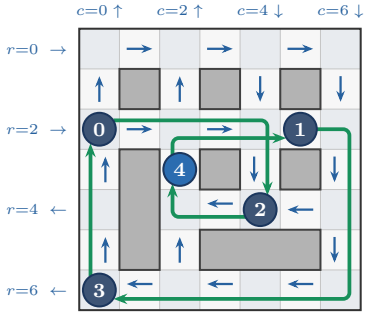
$$d(x, y) = \begin{cases} 0 & \text{if } \{x, y\} = \{u, u'\} \text{ for } u \in \{0, \ldots, 4\}, \\ M + w_{u,v} & \text{if } x = u' \in V' \text{ and } y = v \in V \text{ with } u \neq v, \\ +\infty & \text{otherwise.} \end{cases} \tag{8.12}$$

Any valid alternating tour in G' traverses all 5 zero-weight twin edges $\{u, u'\}$ and 5 cross edges $\{u', v\}$ of weight $M + w_{u,v}$, so its symmetric tour cost is $C_{\text{STSP}} = 5M + C_{\text{ATSP}} = 125 + C_{\text{ATSP}}$.
Starting from the naive index-order tour $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$ of cost 60 ($C_{\text{STSP}} = 185$), running the symmetric Lin–Kernighan heuristic (`LKH`, Section 8.4.2) on the 10×10 symmetric matrix $\boldsymbol{D}$ converges to the alternating 10-vertex cycle (Figure 8.8c):

$$(0\text{—}0'\text{—}2\text{—}2'\text{—}4\text{—}4'\text{—}1\text{—}1'\text{—}3\text{—}3'\text{—}0), \qquad C^*_{\text{STSP}} = 153.$$

Contracting each twin pair (u, u') extracts the globally optimal figure-eight directed tour $\mathbf{0} \to \mathbf{2} \to \mathbf{4} \to \mathbf{1} \to \mathbf{3} \to \mathbf{0}$ of cost $C^*_{\text{ATSP}} = 153 - 125 = \mathbf{28}$, whereas traversing the exact same cyclic sequence in the opposite orientation $(0 \to 3 \to 1 \to 4 \to 2 \to 0)$ would cost $16 + 9 + 8 + 5 + 22 = 60$.

(a) One-Way Street Maze (7×7)



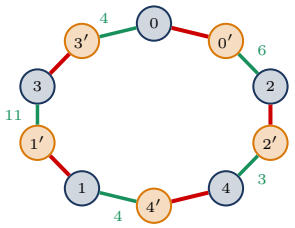
Optimal: $\mathbf{0} \to \mathbf{2} \to \mathbf{4} \to \mathbf{1} \to \mathbf{3} \to \mathbf{0}$ (**28**)
Greedy NN $(0 \to 1 \to 2 \to 4 \to 3 \to 0)$: 32

(b) Asymmetric Matrix W

W	0	1	2	3	4
0	0	5	**6**	16	9
1	15	0	5	**11**	8
2	22	7	0	18	**3**
3	**4**	9	10	0	13
4	19	**4**	5	15	0

Bold green: figure-eight optimum (28)
Perimeter $0 \to 1 \to 2 \to 3 \to 4 \to 0$: 60

(c) Reduced STSP ($2n = 10$)



Red: $\{u, u'\}$ $(-M)$
Green: $\{u', v\}$ $(28 - 5M)$

Figure 8.8: Solving an Asymmetric TSP via the Jonker–Volgenant reduction and symmetric Lin–Kernighan (Example 8.3). (a) A 7×7 urban one-way street maze with four horizontal streets ($r \in \{0, 2, 4, 6\}$), four vertical avenues ($c \in \{0, 2, 4, 6\}$), and extended building blocks at $(4, 1)$ and $(5, 4)$. Because of the one-way flows and interior blocks, the Greedy Nearest-Neighbor tour $(0 \to 1 \to 2 \to 4 \to 3 \to 0$, cost 32) is suboptimal, whereas the optimal tour $\mathbf{0} \to \mathbf{2} \to \mathbf{4} \to \mathbf{1} \to \mathbf{3} \to \mathbf{0}$ (green arrows, cost **28**) forms a self-crossing figure-eight loop through the central intersection $(2, 4)$. (b) The 5×5 asymmetric shortest-path distance matrix $\boldsymbol{W}$. (c) The reduced 10-vertex Symmetric TSP cycle found by `LKH`.

8.4 Neighborhood Extension and Ejection Chains

A simple neighborhood such as 2-Opt alters the structure of a solution only marginally and admits numerous local optima. Yet replacing 2-Opt with fixed higher-order k-Opt neighborhoods ($k \geq 4$) incurs a prohibitive $\mathcal{O}(n^k)$ enumeration cost. *Neighborhood extension* techniques resolve this dilemma by compounding a variable sequence of simple moves while pruning the search space so that exploration remains tractable.

8.4.1 Filter-and-Fan

Given a base move set $M(s)$ (such as 2-Opt moves), the k-step compound neighborhood is defined as:

$$\mathcal{N}_e(s) = \{s' \in \mathcal{S} \mid s' = s \oplus m_0 \oplus m_1 \oplus \cdots \oplus m_{k-1},\ m_\ell \in M\}\,. \tag{8.13}$$

Because $|\mathcal{N}_e(s)| = \mathcal{O}(|M(s)|^k)$ grows exponentially with depth k, the *Filter-and-Fan* method applies the Beam Search principle of Section 8.1.3 directly to neighborhood exploration (Figure 8.3a):

1. **Level 1 (Fan & Filter)**: Evaluate all neighbors in the base neighborhood $\mathcal{N}(s)$ and retain only the p best neighbors $\{s_0^{(1)}, \ldots, s_{p-1}^{(1)}\}$.

2. **Levels** $\ell = 2, \ldots, k$: From each of the p solutions at level $\ell - 1$, fan out by generating candidate moves (typically a restricted subset of η promising basic moves) and filter the resulting pool to keep only the p best compound solutions at level ℓ.
3. **Transition**: If the best solution visited across all k levels improves upon the starting solution s, adopt it as the new current state for the next iteration.

8.4.2 Lin–Kernighan and Ejection Chains

Can we design an algorithm that explores k-Opt moves *without fixing the depth k in advance*, dynamically adapting k during each move while maintaining low polynomial complexity? In 1973, Shen Lin and Brian W. Kernighan answered this question affirmatively with their landmark variable-depth heuristic [LK73], later refined by Keld Helsgaun (LKH) [Hel00].

The key structural insight of Lin and Kernighan is to restrict general k-Opt exchanges to *sequential k-Opt moves*—also known as *ejection chains*.

Definition 8.5 *Sequential k-Opt Move and Reference Structure*

A k-Opt move is *sequential* if the k deleted tour edges and the k added non-tour chords form a single alternating walk $(e_0^{\text{del}}, e_0^{\text{add}}, e_1^{\text{del}}, e_1^{\text{add}}, \ldots, e_{k-1}^{\text{del}}, e_{k-1}^{\text{add}})$ in the graph.
During the construction of an ejection chain starting from a tour s of length $f(s)$, the algorithm alternates between two intermediate topological states (Figure 8.9a):

1. A **Hamiltonian Path** $b \rightsquigarrow a$ of length L_{path}, obtained initially by ejecting a tour edge $[a, b]$ so that $L_{\text{path}} = f(s) - d(a, b)$. Closing this path with edge $[a, b]$ restores a valid tour of length $L_{\text{path}} + d(a, b)$.
2. A **Reference Structure** (a spanning *stem-and-cycle* or 1-tree with one degree-3 vertex d and one degree-1 endpoint a), obtained by identifying a vertex c along the path together with its path successor $d = \text{succ}(c)$ and inserting the chord $[b, d]$. The weight of this reference structure is $L_{\text{ref}} = L_{\text{path}} + d(b, d)$.

Example 8.4 *Tracing Reference Structures and Geometric Ejection Chains*

Let us first trace the abstract six-step progression in Figure 8.9a together with its weight invariants on a starting tour s of cost $f(s)$:

1. **Step 1 (Initial Tour s)**: In the successor-array representation, the directed tour visits $b \to \cdots \to c \to d \to \cdots \to a \to b$ with total cost $f(s)$, where $d = \text{succ}(c)$ and $b = \text{succ}(a)$.
2. **Step 2 (Path)**: Ejecting the tour edge $[a, b]$ opens the cycle into a directed Hamiltonian path $b \rightsquigarrow a$ of weight $L_{\text{path}} = f(s) - d(a, b)$.
3. **Step 3 (Reference Structure)**: Scanning along $b \rightsquigarrow a$, the algorithm identifies a vertex c (with $d = \text{succ}(c)$) such that $[c, d]$ has not yet been removed during the current chain and inserting the chord $[b, d]$ yields a stem-and-cycle *reference structure* strictly lighter than the initial tour:
$$L_{\text{ref}} = L_{\text{path}} + d(b, d) < f(s).$$
4. **Step 4 (New Path)**: Ejecting $[c, d]$ from the reference structure breaks the cycle at d and restores a spanning path of updated weight
$$L_{\text{path}} \leftarrow L_{\text{path}} + d(b, d) - d(c, d) < f(s).$$

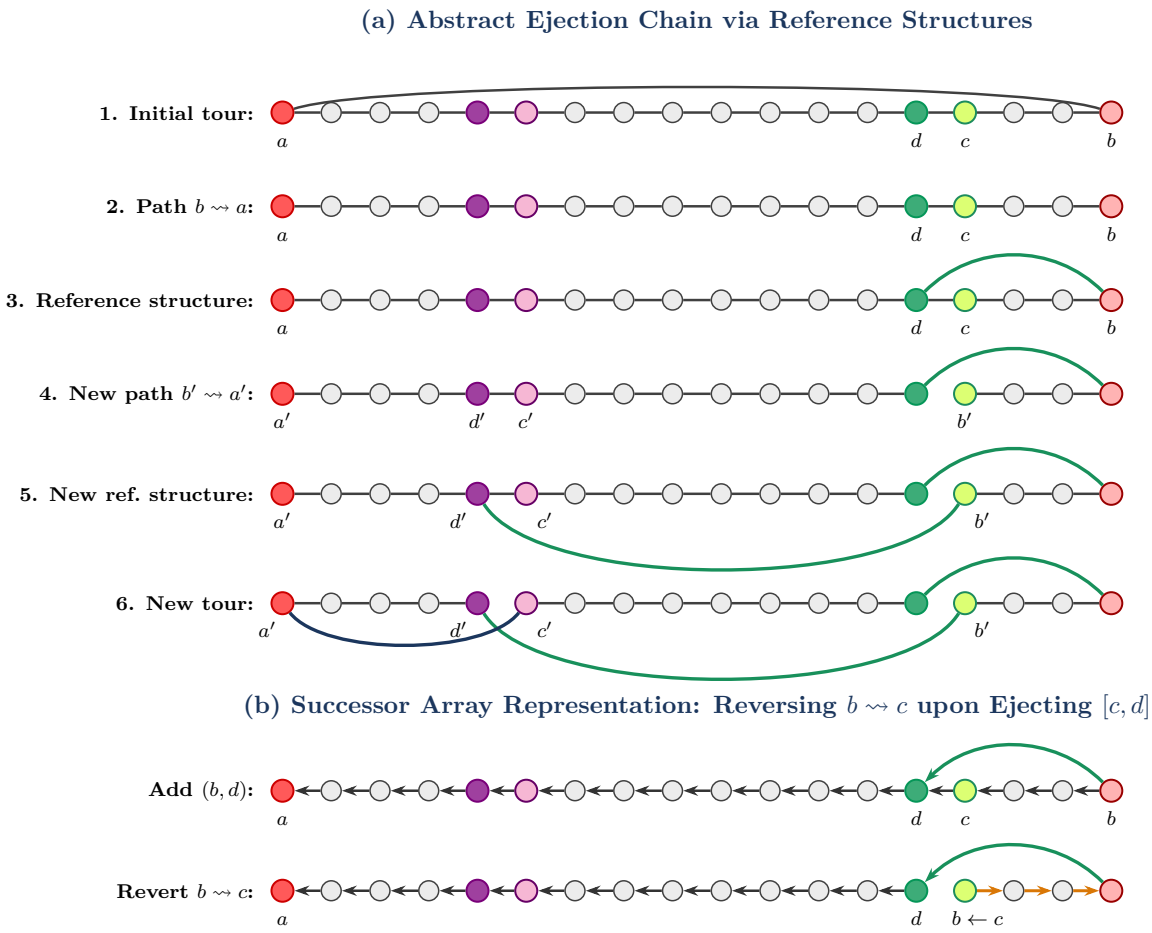


Figure 8.9: Abstract mechanics of the Lin–Kernighan Ejection Chain. (a) Six-step progression of a two-move ejection chain alternating between Hamiltonian paths and stem-and-cycle reference structures: (1) initial tour s, (2) path $b \rightsquigarrow a$ after ejecting $[a, b]$, (3) first reference structure formed by adding $[b, d]$, (4) new path $b' \rightsquigarrow a$ after ejecting $[c, d]$ and reversing $b \rightsquigarrow c$ ($b' \leftarrow c$), (5) second reference structure formed by adding $[b', d']$, and (6) improved tour obtained by ejecting $[c', d']$ and closing $[a', c']$. (b) In-place update on the successor array $\texttt{succ}[\cdot]$: inserting $\texttt{succ}[b] = d$ and reversing pointers along $b \rightsquigarrow c$.

As shown in Figure 8.9b, reversing the successor pointers along the prefix $b \rightsquigarrow c$ makes $b' \leftarrow c$ the new active endpoint of the directed path $b' \rightsquigarrow a$ while keeping the anchor $a' \leftarrow a$ fixed.

5. **Step 5 (New Reference Structure)**: If closing $[a', b']$ does not yet improve upon $f(s)$, the chain continues from b': scanning the updated path $b' \rightsquigarrow a'$ identifies a second pair (c', d') with $d' = \text{succ}(c')$ such that inserting $[b', d']$ forms a new reference structure of weight $L_{\text{path}} + d(b', d') < f(s)$.
6. **Step 6 (New Tour)**: Ejecting $[c', d']$ and reversing $b' \rightsquigarrow c'$ exposes c' as the new active endpoint. Checking the tour-closure condition

$$L_{\text{path}} + d(a', c') < f(s) \tag{8.14}$$

now succeeds: adding $[a', c']$ closes the Hamiltonian path into a strictly shorter tour!

Figure 8.10 illustrates how this abstract reference-structure mechanism operates geometrically over four iterations on a planar Euclidean tour containing three entangled crossings:

- **Iteration 1**: Removing $[a, b]$ opens *Path 1* ($b \rightsquigarrow a$). Closing $[a, b]$ immediately would merely recover the initial tour, so the algorithm scans the path and connects b to a nearby vertex d on the upper loop (*Ref. Struct. 1*). Ejecting the long crossing edge $[c, d]$ and reversing the subpath $b \rightsquigarrow c$ produces *Path 2*, where the upper loop is now untangled and the new endpoint $b \leftarrow c$ sits at the top of the middle crossing.
- **Iteration 2**: In *Path 2*, the active endpoint b is still far from a, so closing $[a, b]$ would not improve the tour. Instead, connecting b downward across the middle crossing to d forms *Ref. Struct. 2* of total weight $< f(s)$. Ejecting the long diagonal edge $[c, d]$ and reversing $b \rightsquigarrow c$ yields *Path 3*, resolving the middle crossing and shifting the active endpoint $b \leftarrow c$ to the bottom loop.
- **Iteration 3**: In *Path 3*, connecting b to d on the right branch forms *Ref. Struct. 3*. Ejecting the remaining crossing edge $[c, d]$ and reversing $b \rightsquigarrow c$ produces *Path 4*.
- **Iteration 4**: In *Path 4*, all three crossings have been eliminated and the new endpoint b lies directly adjacent to the fixed anchor a. The closure test $L_{\text{path}} + d(a, b) < f(s)$ now succeeds (*Close Path 4!*), completing a sequential 4-Opt move that simultaneously untangles the entire tour.

Successor Array Representation for LKH

To implement Algorithm 18 cleanly in Java, representing the tour via a *successor array* $\texttt{succ}[0 \ldots n-1]$—where $\texttt{succ}[u]$ stores the city visited immediately after city u—is particularly natural (Figure 8.9b). When edge $[c, d]$ (with $d = \texttt{succ}[c]$) is ejected and $[b, d]$ is inserted, all successor pointers along the subpath $b \rightsquigarrow c$ must be inverted so that the new path starts at c, traverses $c \rightsquigarrow b$ in reverse, jumps across the newly added chord $\texttt{succ}[b] = d$, and continues along $d \rightsquigarrow a$ to the fixed endpoint a.

Listings 8.5 and 8.6 present the complete Java implementation of the successor-array Lin–Kernighan heuristic (`LKH`). First, Listing 8.5 defines the tour-to-successor converters, the

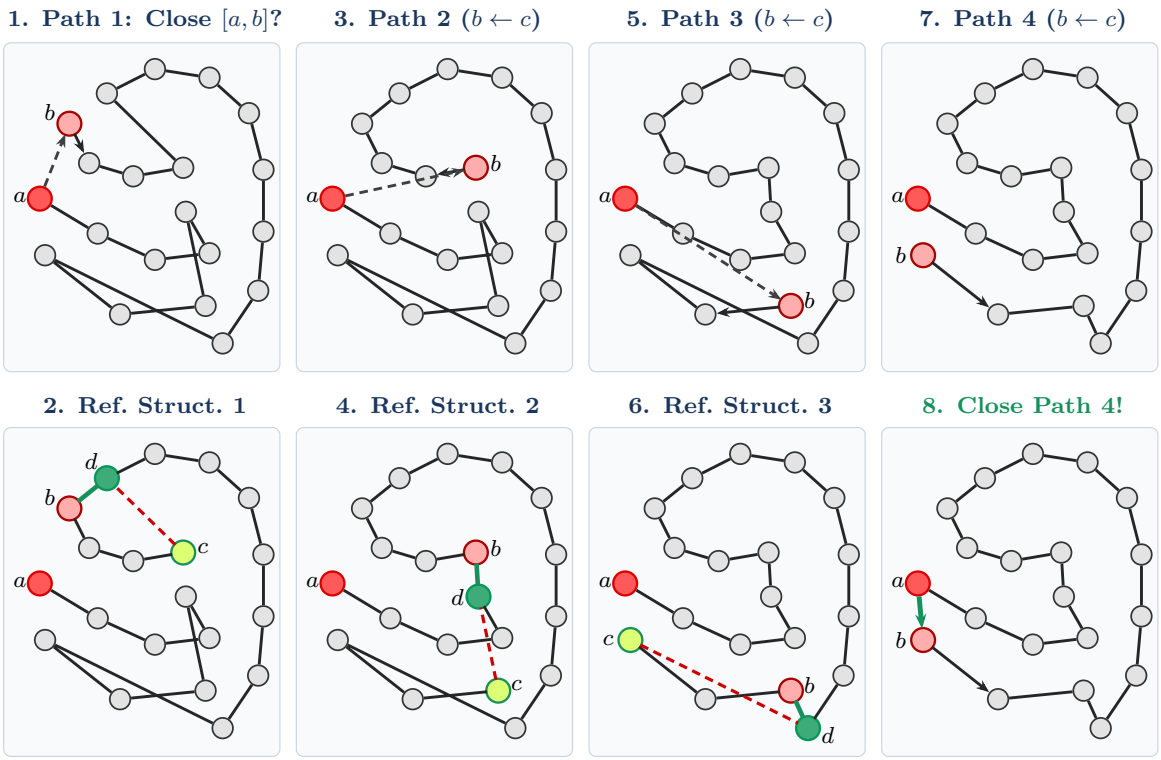


Figure 8.10: Step-by-step geometric execution of a four-iteration Lin–Kernighan ejection chain on a planar TSP instance. Each column corresponds to one iteration, alternating between a spanning Hamiltonian path $b \rightsquigarrow a$ (*top row*) and a stem-and-cycle reference structure obtained by inserting the green chord $[b, d]$ and ejecting the dashed red tour edge $[c, d]$ (*bottom row*), until closing $[a, b]$ in Iteration 4 (Step 8) yields a strictly shorter, crossing-free tour.

Algorithm 18: Lin–Kernighan Ejection Chain Heuristic for the TSP

Input: Initial TSP tour s of cost $f(s)$, symmetric distances $d : V \times V \to \mathbb{R}_{\geq 0}$
Output: Improved TSP tour s

repeat
  foreach *directed tour edge* $[a, b] \in s$ *with* $b = \mathrm{succ}_s(a)$ **do**
    $P \leftarrow s \setminus \{[a, b]\}$, $L_{\text{path}} \leftarrow f(s) - d(a, b)$; `// Open Hamiltonian path` $b \rightsquigarrow a$
    $E_{\text{del}} \leftarrow \{[a, b]\}$; `// Edges ejected in the current chain`
    while true **do**
      $C \leftarrow \{\, c \in V \setminus \{a, b\} \mid \mathrm{succ}_P(c) \neq a \ \wedge\ [c, \mathrm{succ}_P(c)] \notin E_{\text{del}} \,\}$;
      if $C = \emptyset$ **then**
        break
      $c^* \leftarrow \arg\min_{c \in C} \{\, L_{\text{path}} + d(b, \mathrm{succ}_P(c)) \,\}$, $d^* \leftarrow \mathrm{succ}_P(c^*)$;
      $L_{\text{ref}} \leftarrow L_{\text{path}} + d(b, d^*)$; `// Minimum reference-structure weight`
      if $L_{\text{ref}} < f(s)$ **then**
        $P \leftarrow (P \cup \{[b, d^*]\}) \setminus \{[c^*, d^*]\}$; `// Reverse subpath` $b \rightsquigarrow c^*$ `in` P
        $E_{\text{del}} \leftarrow E_{\text{del}} \cup \{[c^*, d^*]\}$;
        $L_{\text{path}} \leftarrow L_{\text{ref}} - d(c^*, d^*)$;
        $b \leftarrow c^*$; `// Update active path endpoint`
        if $L_{\text{path}} + d(a, b) < f(s)$ **then**
          $s \leftarrow P \cup \{[a, b]\}$, $f(s) \leftarrow L_{\text{path}} + d(a, b)$; `// Improved tour`
      else
        break ; `// Ejection failure (`$L_{\texttt{ref}} \geq f(s)$`): revert to` s
until *no edge* $[a, b] \in s$ *yields an improving ejection chain*;
return s;

in-place path-reversal routine `reversePath`, and the reference-structure search `findBestC`; next, Listing 8.6 implements the outer ejection-chain search loop `applyLKH`.

Experimental Comparison on 200-City TSP

To quantify the practical impact of neighborhood structure and variable-depth chaining, Figure 8.11 and Table 8.3 compare four local search neighborhoods—Or-Opt ($\ell \in \{3, 2, 1\}$), 2-Opt, 3-Opt, and Lin–Kernighan Ejection Chains—on the 200-city planar Euclidean TSP instance from Figure 8.2, all starting from the identical Nearest Neighbor initial tour of cost 12,563.3.

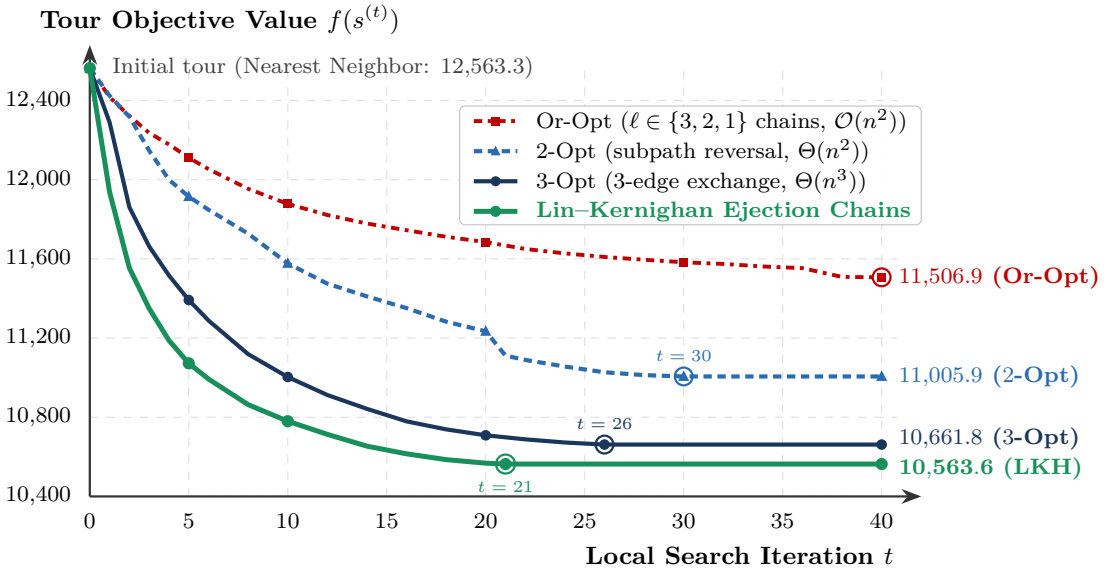


Figure 8.11: Evolution of the tour objective value $f(s^{(t)})$ across steepest-descent iterations $t \in \{0, \ldots, 40\}$ on the 200-city Euclidean TSP benchmark starting from the Nearest Neighbor tour (12,563.3). Hollow circles mark the iteration at which each neighborhood reaches its local minimum.

Neighborhood	Initial Cost	Final Cost	Gain (%)	Iterations	Step Complexity
Or-Opt ($\ell \in \{3, 2, 1\}$)	12,563.3	11,506.9	−8.41%	40	$\mathcal{O}(n^2)$
2-Opt (Subpath Reversal)	12,563.3	11,005.9	−12.40%	30	$\Theta(n^2)$
3-Opt (3-Edge Exchange)	12,563.3	10,661.8	−15.14%	26	$\Theta(n^3)$
Lin–Kernighan Ejection Chains	12,563.3	**10,563.6**	**−15.92%**	**21**	$\mathcal{O}(d_{\max} n^2)$

Table 8.3: Empirical performance of four TSP neighborhoods on the 200-city Euclidean benchmark starting from the Nearest Neighbor tour (12,563.3).

The trajectories in Figure 8.11 highlight a clear hierarchy:

1. **Or-Opt vs. 2-Opt**: Because Or-Opt relocates only short contiguous chains of up to 3 cities without inverting arbitrary subpaths, it cannot uncross wide global loops in a single step and stagnates at a local minimum of 11,506.9 after 40 iterations. By contrast, 2-Opt reverses subpaths of arbitrary length in $\Theta(n^2)$ time per pass, descending to 11,005.9 in 30 iterations.
2. **3-Opt vs. Lin–Kernighan**: Enlarging the fixed move set from 2-Opt to 3-Opt escapes the 2-Opt local optimum and reaches 10,661.8 in 26 iterations, at the price of a cubic

```java
public class LKH {
    private final TSPInstance instance;

    public LKH(TSPInstance instance) {
        this.instance = instance;
    }

    int[] tourToSucc(Candidate c) {
        int[] tour = c.getTour(), succ = new int[tour.length];
        for (int i = 0; i < tour.length; i++)
            succ[tour[i]] = tour[(i + 1) % tour.length];
        return succ;
    }

    void reversePath(int[] succ, int b, int c, int d) {
        int i = b, si = succ[b];
        succ[b] = d;
        while (i != c) {
            int temp = succ[si];
            succ[si] = i;
            i = si;
            si = temp;
        }
    }

    int[] succToTour(int[] succ) {
        int[] tour = new int[succ.length];
        for (int i = 0, cur = 0; i < succ.length; i++) {
            tour[i] = cur;
            cur = succ[cur];
        }
        return tour;
    }

    Pair<Integer, Double> findBestC(
            int[] succ, int a, int b, double length,
            double pathLength, int iter, int[][] tabu) {
        double[][] D = instance.distanceMatrix;
        int c = succ[b], bestC = c;
        double refStructCost = length;
        while (succ[c] != a) {
            int d = succ[c];
            double closed = pathLength - D[c][d]
                          + D[c][a] + D[b][d];
            if (closed < length) return new Pair<>(c, closed);
            if (tabu[c][d] != iter
                    && pathLength + D[b][d] < refStructCost) {
                refStructCost = pathLength + D[b][d];
                bestC = c;
            }
            c = d;
        }
        return new Pair<>(bestC, refStructCost);
    }
```

Listing 8.5: Successor-array representation converters, in-place path reversal, and reference-structure search for Lin–Kernighan (`LKH`).

```
    public Candidate applyLKH(Candidate candidate) {
        int n = instance.n;
        int[][] tabu = new int[n][n];
        int[] succ = tourToSucc(candidate);
        int[] tour = candidate.getTour();
        int iteration = 0;
        double length = candidate.getCost();
        boolean improvement = true;
        int a = 0;

        while (improvement) {
            improvement = false;
            iteration++;
            int b = succ[a];
            double[][] D = instance.distanceMatrix;
            double pathLength = length - D[a][b];
            boolean pathModified = true;

            while (pathModified) {
                pathModified = false;
                Pair<Integer, Double> res = findBestC(
                    succ, a, b, length,
                    pathLength, iteration, tabu);
                int c = res.getFirst();
                double newLength = res.getSecond();
                if (newLength < length) {
                    pathModified = true;
                    int d = succ[c];
                    tabu[c][d] = iteration;
                    tabu[d][c] = iteration;
                    pathLength += (D[b][d] - D[c][d]);
                    reversePath(succ, b, c, d);
                    b = c;
                    if (pathLength + D[a][b] < length) {
                        length = pathLength + D[a][b];
                        succ[a] = b;
                        improvement = true;
                        tour = succToTour(succ);
                    }
                }
            }
            succ = tourToSucc(new Candidate(instance, tour));
            a = succ[a];
        }
        return new Candidate(instance, tour);
    }
}
```

Listing 8.6: Main ejection-chain search loop `applyLKH` of the Lin–Kernighan heuristic.

$\Theta(n^3)$ neighborhood scan per iteration. Lin–Kernighan Ejection Chains strictly dominate all three fixed-order neighborhoods at every single iteration t: by chaining variable-depth sequential exchanges guided by the reference-structure bound, LKH converges in just 21 iterations to the lowest tour cost of 10,563.6 without incurring the $\Theta(n^3)$ exhaustive enumeration of 3-Opt.

8.5 Escaping Local Optima: Metaheuristics

Pure iterative improvement (Algorithm 15) terminates as soon as it encounters a local minimum $\hat{s}$ satisfying $f(\hat{s}) \leq f(s')$ for all $s' \in \mathcal{N}(\hat{s})$. Moreover, as illustrated in the fitness landscape of Figure 8.12a, real-world combinatorial landscapes are rife with two structural obstacles:

- **Local Minima**: Basins of attraction whose bottom $\hat{s}$ is strictly worse than the global optimum s^* ($f(\hat{s}) > f(s^*)$).
- **Plateaus**: Extended flat regions of the state space where all immediate neighbors have identical objective values ($\Delta f = 0$), depriving gradient-like descent of any directional signal.

To escape from a local minimum $\hat{s}$, an optimization algorithm has two fundamental avenues (Figure 8.12a):

1. **Enlarge the Neighborhood ($\mathcal{N}_1 \subset \mathcal{N}_2$)**: Transition to a much larger neighborhood structure $\mathcal{N}_2(\hat{s})$ whose radius exceeds the width of the local basin, so that a strictly improving state becomes directly reachable in a single compound step. Pushed to its logical extreme, this yields *Large Neighborhood Search* (LNS).
2. **Accept Objective Degradation (Metaheuristics)**: Retain the compact neighborhood $\mathcal{N}_1$, but permit the search trajectory to move uphill to worse solutions ($f(s') > f(s)$) under the guidance of a high-level control strategy—*a metaheuristic*—such as *Tabu Search* or *Simulated Annealing*.

8.5.1 Large Neighborhood Search

Introduced by Paul Shaw in 1998 [Sha98], *Large Neighborhood Search* (LNS) pushes the idea of neighborhood enlargement to an extreme through a *Destroy-and-Repair* architecture (Algorithm 19):

- **Destroy Operator** $d(s)$: Given the current complete solution s, the destroy method stochastically or heuristically relaxes (unassigns or removes) a subset of q decision variables, producing an incomplete partial solution $d(s)$.
- **Repair Operator** $r(\cdot)$: Given the partial solution $d(s)$, the repair method reconstructs a complete feasible solution $s' = r(d(s))$ by re-optimizing the unassigned variables—using a fast constructive insertion heuristic, Beam Search, or even an exact Branch-and-Bound / Constraint Programming solver over the restricted subproblem.
- **Acceptance Criterion** $a(s, s')$: Typically accepts improving or equal-cost solutions ($f(s') \leq f(s)$), though simulated-annealing style acceptance criteria can also be used.

Problem Geometry in LNS Destroy Operators. The effectiveness of LNS hinges critically on how the destroy operator $d(s)$ selects which components to relax. Consider the *Vehicle Routing Problem* (VRP) illustrated in Figure 8.12b, where a fleet of vehicles based at a central depot must serve a set of customers. If $d(s)$ removes a few isolated customers located far apart at opposite corners of the map, the repair operator $r(\cdot)$ has virtually no flexibility: each removed customer will simply be reinserted back into its original vehicle tour at its original position. Instead, an effective destroy

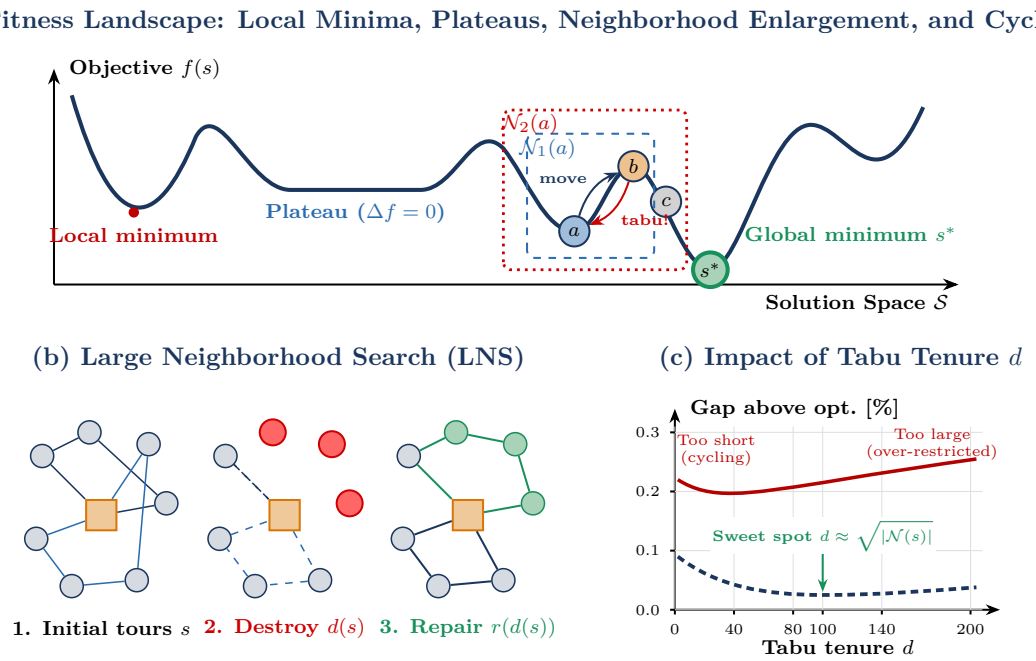


Figure 8.12: Escaping local optima. (a) A rugged fitness landscape featuring multiple local minima, a plateau, neighborhood enlargement ($\mathcal{N}_1 \subset \mathcal{N}_2$), and the cycling trap ($a \leftrightarrow b$) prevented by Tabu Search. (b) Large Neighborhood Search (LNS) on a Vehicle Routing Problem: geographically proximate customers across different vehicle tours are destroyed ($d(s)$) and reinserted ($r(d(s))$). (c) Empirical influence of Tabu tenure d on solution quality for a 100-city Euclidean TSP, exhibiting a sweet spot around $d \approx \sqrt{|\mathcal{N}(s)|}$.

Algorithm 19: Large Neighborhood Search (LNS)

Input: Initial feasible solution s; destroy operator $d(\cdot)$; repair operator $r(\cdot)$; acceptance criterion $a(\cdot,\cdot)$
Output: Best discovered solution s^*

$s^* \leftarrow s$;
repeat
  $s' \leftarrow r(d(s))$; `// Destroy part of` s `and rebuild via repair`
  if $a(s, s')$ **then**
    $s \leftarrow s'$;
  if $f(s') < f(s^*)$ **then**
    $s^* \leftarrow s'$;
until *a stopping criterion is satisfied*;
return s^*;

operator exploits the *geometry and relatedness* of the problem: it selects a cluster of customers that are *geographically close to one another* but currently assigned to *different vehicle routes*. Relaxing these adjacent cross-route customers simultaneously allows the repair procedure to exchange customers between vehicles and untangle overlapping routes.

8.5.2 Tabu Search

Rather than enlarging the neighborhood, what if we modify the Best-Improvement policy (Algorithm 17) so that when no improving neighbor exists, the algorithm still moves to the *least-degrading neighbor* $s' \in \arg\min_{s'' \in \mathcal{N}(s)} f(s'')$?

As shown in Figure 8.12a, this naive steepest-descent/mildest-ascent policy immediately falls into a *cycling trap*: starting from a local minimum a, the algorithm moves uphill to its best neighbor $b \in \mathcal{N}(a)$; at the very next iteration, since $a \in \mathcal{N}(b)$ has a strictly lower cost than b, the best move from b is to return immediately to a, oscillating forever ($a \to b \to a \to b \dots$).

Proposed by Fred Glover [Glo89; GL97], *Tabu Search* resolves the cycling problem by equipping local search with a *short-term memory*—the *tabu list*:

- Whenever a move $m^* \in M(s)$ is executed to transition from s to $s' = s \oplus m^*$, the *reverse move* $(m^*)^{-1}$ (or a structural attribute of s that was modified by m^*) is declared **tabu** (forbidden) for the next d iterations. The parameter $d \in \mathbb{Z}_{\geq 1}$ is called the *tabu tenure* (or *tabu duration*).
- At each iteration, the algorithm selects the best admissible move $m^* \in M(s)$ that is *not currently marked as tabu*, even if $f(s \oplus m^*) > f(s)$, and updates the incumbent s^* whenever $f(s) < f(s^*)$ (Algorithm 20).
- After d iterations have elapsed (*oblivion*), the prohibition on $(m^*)^{-1}$ expires, allowing the search to re-enter that region if warranted.

Algorithm 20: Elementary Tabu Search

Input: Initial solution s; set of moves M; objective $f(\cdot)$ to minimize; maximum iterations $I_{\max}$; tabu tenure d
Output: Best discovered solution s^*

$s^* \leftarrow s$;
for *iter* $= 0$ **to** $I_{\max} - 1$ **do**
  $v_{\text{best}} \leftarrow \infty$;
  foreach $m \in M$ *such that* m *(or* $s \oplus m$*) is not marked as tabu* **do**
    if $f(s \oplus m) < v_{best}$ **then**
      $v_{\text{best}} \leftarrow f(s \oplus m)$;
      $m^* \leftarrow m$;
  if $v_{best} < \infty$ **then**
    Mark reverse move $(m^*)^{-1}$ (or s) as tabu until iteration *iter* $+ d$;
    $s \leftarrow s \oplus m^*$;
    if $f(s) < f(s^*)$ **then**
      $s^* \leftarrow s$;
  else
    Error: Tabu tenure d is too large; no admissible move exists!
return s^*;

Example 8.5 *Tabu Search on the* 0-1 *Knapsack Problem*

Consider a 9-item 0-1 Knapsack maximization instance with 0-indexed decision variables $x_i \in \{0,1\}$ for $i \in \{0,1,\dots,8\}$ and capacity $C = 45$:

$$\begin{aligned} \text{maximize} \quad & r(\mathbf{x}) = 12x_0 + 10x_1 + 9x_2 + 7x_3 + 4x_4 + 8x_5 + 11x_6 + 6x_7 + 13x_8 \\ \text{subject to} \quad & v(\mathbf{x}) = 10x_0 + 12x_1 + 8x_2 + 7x_3 + 5x_4 + 13x_5 + 9x_6 + 6x_7 + 14x_8 \leq 45, \\ & x_i \in \{0,1\}, \quad \forall i \in \{0,1,\dots,8\}. \end{aligned} \tag{8.15}$$

Define the elementary move set $M(\mathbf{x})$ as flipping the selection status of a single item $i \in \{0,\dots,8\}$ ($x_i \leftarrow 1 - x_i$) provided the resulting total weight satisfies $v(\mathbf{x}') \leq 45$. Whenever variable x_i is flipped at iteration $t \in \{1,\dots,10\}$, flipping x_i again is forbidden for a tabu tenure of $d = 3$ iterations (so x_i becomes available again at iteration $t + d = t + 3$).

Table 8.4 traces 10 iterations starting from the empty knapsack $\mathbf{x}^{(0)} = \mathbf{0}$:

- **Iterations** 1–4 **(Greedy Ascent to Local Optimum)**: The algorithm successively inserts the highest-profit compatible items: x_8 (+13), x_0 (+12), x_6 (+11), and x_1 (+10), reaching $\mathbf{x}^{(4)} = (1,1,0,0,0,0,1,0,1)$ with profit $r = 46$ and weight $v = 45$ (full capacity). Because no further item can be added without exceeding $C = 45$, $\mathbf{x}^{(4)}$ is a *strict local optimum.*
- **Iteration** 5 **(Controlled Degradation)**: At $\mathbf{x}^{(4)}$, the only feasible moves are to *remove* an item currently in the knapsack (x_0, x_1, x_6, or x_8). Checking the tabu status array $(5,7,0,0,0,0,6,0,4)$ at iteration $t = 5$: x_0 is available ($5 \leq 5$), x_1 is tabu until 7, x_6 is tabu until 6, and x_8 is available ($4 \leq 5$). Removing x_0 would drop profit by 12 (to 34), whereas removing x_8 drops profit by 13 (to 33) and frees 14 units of capacity (Following the tie-breaking/selection schedule in Table 8.4, x_8 is dropped at $t = 5$, locking x_8 until iteration $5 + 3 = 8$).
- **Iterations** 6–7 **(Finding a Better Basin)**: With 14 units of freed capacity and x_8 tabu, the search inserts x_2 (+9, weight 8) at $t = 6$ and x_7 (+6, weight 6) at $t = 7$, discovering $\mathbf{x}^{(7)} = (1,1,1,0,0,0,1,1,0)$ with profit $r = 48 > 46$!
- **Iterations** 8–10 **(Reaching the Global Optimum)**: At $t = 8$, the full knapsack $\mathbf{x}^{(7)}$ must again drop a non-tabu item; since x_2 (tabu until 9) and x_7 (tabu until 10) are locked, it drops x_1 ($r = 38, v = 33$, locking x_1 until 11), then inserts x_3 (+7, weight 7) at $t = 9$ and x_4 (+4, weight 5) at $t = 10$, reaching the global optimum $\mathbf{x}^{(10)} = (1,0,1,1,1,0,1,1,0)$ with profit $r^* = 49$ and weight $v = 45$.

Tabu Search for the TSP and Tenure Calibration

When applying Tabu Search with the 2-Opt neighborhood to the TSP, storing complete tour permutations in a hash table is unnecessary and overly restrictive. Instead, we store *move attributes* in an $n \times n$ integer matrix `tabu`$[0\dots n-1][0\dots n-1]$: whenever a 2-Opt move removes an edge (u, s_u), we set `tabu`$[u][s_u] = \mathit{iteration} + d$, forbidding moves that interact with that pair until iteration $\mathit{iteration} + d$. This timestamp encoding enables $\mathcal{O}(1)$ tabu status queries (`iteration < tabu[u][v]`) without maintaining or shifting a dynamic queue (Listing 8.7).

How should one calibrate the tabu tenure d? As demonstrated empirically on a 100-city Euclidean TSP in Figure 8.12c:

```java
public class BestWithTabuSelection
        implements NeighborSelection {
    private int iteration = 0;
    private int tabuSize;
    private int[][] tabu;

    public BestWithTabuSelection(
            int tabuSize, TSPInstance tsp) {
        this.tabuSize = tabuSize;
        this.tabu = new int[tsp.n][tsp.n];
    }

    public void addTabu(int u, int v) {
        tabu[u][v] = iteration + tabuSize;
    }

    public boolean isTabu(int u, int v) {
        return iteration < tabu[u][v];
    }

    @Override
    public Candidate getNeighbor(Candidate candidate) {
        iteration++;
        int[] tour = candidate.getTour();
        int n = tour.length;
        int bestI = -1, bestJ = -1, succI = -1;
        double bestDelta = Double.MAX_VALUE;
        for (int i = 0; i < n - 1; i++)
            for (int j = i + 2; j < n; j++) {
                if (i == 0 && j == n - 1) continue;
                double delta = candidate.twoOptDelta(i, j);
                if (!isTabu(tour[i], tour[j])
                        && delta < bestDelta) {
                    bestDelta = delta;
                    bestI = i; bestJ = j;
                    succI = candidate.getSucc(i);
                }
            }
        if (bestI != -1) {
            addTabu(tour[bestI], succI);
            candidate.twoOpt(bestI, bestJ);
        }
        return candidate;
    }
}
```

Listing 8.7: Tabu Search 2-Opt selector using an $\mathcal{O}(1)$ timestamp expiration matrix.

Iter. t	**Flip**	**Solution** $\mathbf{x} = (x_0, \ldots, x_8)$	**Profit** r	**Weight** v	**Tabu Expiration Array** ($d = 3$)
0	—	$(0,0,0,0,0,0,0,0,0)$	0	0	$(0,0,0,0,0,0,0,0,0)$
1	x_8	$(0,0,0,0,0,0,0,0,1)$	13	14	$(0,0,0,0,0,0,0,0,\mathbf{4})$
2	x_0	$(1,0,0,0,0,0,0,0,1)$	25	24	$(\mathbf{5},0,0,0,0,0,0,0,4)$
3	x_6	$(1,0,0,0,0,0,1,0,1)$	36	33	$(5,0,0,0,0,0,\mathbf{6},0,4)$
4	x_1	$(1,1,0,0,0,0,1,0,1)$	**46**	45	$(5,\mathbf{7},0,0,0,0,6,0,4)$
5	x_8	$(1,1,0,0,0,0,1,0,0)$	33	31	$(5,7,0,0,0,0,6,0,\mathbf{8})$
6	x_2	$(1,1,1,0,0,0,1,0,0)$	42	39	$(5,7,\mathbf{9},0,0,0,6,0,8)$
7	x_7	$(1,1,1,0,0,0,1,1,0)$	**48**	45	$(5,7,9,0,0,0,6,\mathbf{10},8)$
8	x_1	$(1,0,1,0,0,0,1,1,0)$	38	33	$(5,\mathbf{11},9,0,0,0,6,10,8)$
9	x_3	$(1,0,1,1,0,0,1,1,0)$	45	40	$(5,11,9,\mathbf{12},0,0,6,10,8)$
10	x_4	$(1,0,1,1,1,0,1,1,0)$	**49**	45	$(5,11,9,12,\mathbf{13},0,6,10,8)$

Table 8.4: Trace of Tabu Search ($d = 3$) on the 9-item 0-1 Knapsack problem (Example 8.5). Entry k in the Tabu Expiration Array records the earliest iteration t at which variable x_k may be flipped again.

- If d is **too short** ($d \ll \sqrt{|\mathcal{N}(s)|}$), the memory expires before the search has escaped the basin of attraction, leading to cycling among a small group of solutions.
- If d is **too large** ($d \gg \sqrt{|\mathcal{N}(s)|}$), too many moves are simultaneously forbidden, over-constraining the search trajectory and preventing it from exploring high-quality neighbors.
- **Practitioner Rule of Thumb**: Choosing a tabu duration on the order of

$$d \approx \sqrt{|\mathcal{N}(s)|} \tag{8.16}$$

  (e.g., $d \approx n$ for an $\mathcal{O}(n^2)$ neighborhood like 2-Opt) consistently hits the empirical sweet spot between cycling prevention and search flexibility [Tai23].

Three classical enhancements further strengthen Tabu Search in practice:

1. **Aspiration Criterion**: If a tabu move $m \in M(s)$ yields a solution $s' = s \oplus m$ that is strictly better than the best-so-far incumbent s^* ($f(s') < f(s^*)$), there is zero risk of cycling—since $f(s')$ has never been visited before! The *aspiration criterion* overrides the tabu restriction and accepts m immediately.
2. **Strategic Oscillation and Randomized Tenure**: Rather than fixing a constant d, drawing the tabu duration uniformly at random in an interval $[d_{\min}, d_{\max}]$ at each step (or dynamically increasing d when repetitions are detected, as in *Reactive Tabu Search*) breaks periodic cycles effortlessly.
3. **Restarts and Frequency-Based Diversification**: Periodically applying a burst of random moves (or penalizing frequently executed moves using long-term frequency memory) drives the search into unexplored regions of $\mathcal{S}$.

8.6 Constraint-Based Local Search

In unconstrained or simple routing problems such as the TSP, maintaining feasibility across moves is straightforward: a 2-Opt exchange maps any valid tour to another valid tour. In richly constrained satisfaction and scheduling problems, however, moving from one feasible solution to another via small perturbations can be virtually impossible because feasible solutions are isolated islands separated by infeasible configurations.

8.6.1 Hard vs. Soft Constraints in CBLS

Pioneered by Pascal Van Hentenryck and Laurent Michel [VM05], *Constraint-Based Local Search* (CBLS) provides a declarative modeling and incremental evaluation architecture that separates *problem definition* from *search control*:

- **Hard Constraints**: Structural invariants that *cannot be violated* at any point during the search. Hard constraints are enforced by (1) the initial solution generator and (2) designing neighborhood moves that preserve them invariantly (e.g., value swaps within a domain-consistent partition).
- **Soft Constraints**: Constraints that *may be violated* by candidate states during the search. Each soft constraint $c \in \mathcal{C}_{\text{soft}}$ is equipped with a non-negative *violation function* $v_c(s) \geq 0$ measuring the degree to which s violates c, with $v_c(s) = 0$ if and only if c is satisfied.
- **CBLS Invariant Propagation and Δ-Queries**: The CBLS solver maintains a directed acyclic dependency graph of *invariants* tracking the total violation $V(s) = \sum_{c \in \mathcal{C}_{\text{soft}}} v_c(s)$. Crucially, the CBLS library automatically provides $\mathcal{O}(1)$ incremental delta evaluation queries—such as `violation.getSwapDelta(x, y)`—telling the search procedure how much the total violation would change if the values of decision variables `x` and `y` were exchanged.

8.6.2 Case Study: The Sudoku Puzzle

To see why naive local search struggles on tightly constrained satisfaction problems and how CBLS modeling overcomes this hurdle, consider the standard 9×9 Sudoku puzzle. The grid consists of 81 cells indexed $0 \ldots 80$ (with row $r \in \{0, \ldots, 8\}$ and column $c \in \{0, \ldots, 8\}$ corresponding to cell index $9r + c$), partitioned into nine 3×3 blocks. Some cells are pre-assigned fixed clue digits (*hints*), and every row, every column, and every 3×3 block must contain each of the 9 distinct symbols (e.g., $\{1, \ldots, 9\}$) exactly once.

Model 1: Global Value-Count Invariant

In our first model, we partition the constraints into:

- **Hard Constraints**: Fixed hint cells retain their initial values, and across the entire 9×9 grid, each symbol in $\{1, \ldots, 9\}$ occurs *exactly* 9 *times*.
- **Soft Constraints**: Each symbol occurs once in every row (AllDifferent on rows), once in every column (AllDifferent on columns), and once in every 3×3 block (AllDifferent on blocks).

If a symbol d appears $k > 1$ times in a row, column, or block, it contributes $k - 1$ to the violation of that unit (equivalently, each conflicting duplicate cell increments the violation counter for its row, column, and block). Because the global multiplicity of each digit is fixed to 9 at initialization, any *swap move* exchanging the values of two non-hint cells (a, b) preserves the hard constraint!

Model 2: Augmenting the Set of Hard Constraints

We can significantly shrink the search space and eliminate an entire family of soft constraints by strengthening the invariant (Figure 8.13a):

- **Hard Constraints**: Fixed hint cells retain their values, *and every row* $r \in \{0, \ldots, 8\}$ *is initialized as a valid permutation of* $\{1, \ldots, 9\}$.

- **Soft Constraints**: Only column AllDifferent constraints ($c \in \{0, \ldots, 8\}$) and 3×3 block AllDifferent constraints ($b \in \{0, \ldots, 8\}$) are posted as soft constraints in the `ConstraintSystem`.
- **Moves**: Restrict swap moves to pairs of non-hint cells (v_1, v_2) *located on the same row* r ($v_1 = 9r + i,\ v_2 = 9r + j$ with $0 \leq i < j < 9$).

Because swapping two cells within the same row preserves the permutation property of that row, all 9 row constraints remain permanently satisfied with zero violation! Furthermore, the neighborhood size drops from $\binom{81}{2} = 3240$ global swaps to at most $9 \times \binom{9}{2} = 324$ intra-row swaps.

Listing 8.8 and Listing 8.9 present the CBLS implementation of Model 2, while Figure 8.13b compares the empirical violation trajectories $V(s^{(t)})$ of four local search variants starting from the exact same initial row-permutation grid ($V(s^{(0)}) = 47$):

1. **Pure Greedy Descent**: Selecting only strictly improving swaps ($\Delta < 0$) rapidly drops the total violation from 47 to 12 in 20 iterations, at which point it becomes permanently trapped in a local minimum where every intra-row swap increases or maintains the violation ($\Delta \geq 0$).
2. **Tabu Search (Fixed Tenure $d = 28$, No Aspiration)**: Forbidding the reversal of recent swaps forces the solver to accept temporary uphill moves ($\Delta > 0$) and escape the greedy trap, reaching $V = 6$ at iteration $t = 30$. Without aspiration or randomized tenure, however, the trajectory subsequently oscillates between $V \in [6, 16]$.
3. **Tabu + Aspiration (Randomized Tenure $d \in [10, 24]$)**: Overriding the tabu status whenever a swap improves upon the best-so-far violation V^* and drawing the tenure uniformly at random breaks short cycles and drives the violation down to $V = 2$ at $t = 120$ (just one conflicting pair away from a valid grid), where it stagnates in a deep basin of attraction.
4. **Tabu + Aspiration + Restart ($R = 50$)**: Augmenting the solver with a diversification restart—applying 12 random intra-row swaps whenever V^* fails to improve for $R = 50$ consecutive iterations—triggers restarts at $t = 110$ (V jumps to 27) and $t = 160$ (V jumps to 22), escaping the stagnant basin and converging to a valid zero-violation Sudoku solution ($V = 0$) at iteration $t = 176$.

8.6.3 Cumulative Scheduling: IFlat-IRelax

As a second real-world application of local search to constrained optimization, consider the *Cumulative Scheduling Problem* (a core component of the Resource-Constrained Project Scheduling Problem, RCPSP). We are given n non-preemptive activities $i \in \{0, 1, \ldots, n-1\}$ (which cannot be interrupted once started) sharing a renewable resource of capacity $C \in \mathbb{Z}_{\geq 1}$. Each activity i has a fixed processing duration $d_i \in \mathbb{Z}_{\geq 1}$ and requires $r_i \in \mathbb{Z}_{\geq 1}$ units of the resource throughout its execution interval $[start_i, start_i + d_i)$. The goal is to find start times $start_i \geq 0$ minimizing the *makespan*

$$\text{minimize} \quad \max_{i \in \{0,\ldots,n-1\}} (start_i + d_i) \tag{8.17}$$

subject to the cumulative capacity constraint at every time instant $t \geq 0$:

$$\sum_{i:\ t \in [start_i,\ start_i + d_i)} r_i \leq C, \quad \forall t \geq 0. \tag{8.18}$$

Directly perturbing integer start times $start_i$ in local search is ineffective because shifting a single activity either violates capacity (8.18) or leaves dead time gaps. Instead, the *Iterative*

```
IntVarLS[] grid;
IntVarLS violation;
ConstraintSystem constraintSystem;
ArrayList<Pair> possibleSwaps;

SolverLS ls = makeSolver();
grid = makeIntVarArray(9 * 9,
    i -> makeIntVar(ls, init[i]));

ArrayList<Constraint> constraints = new ArrayList<>();
for (int k = 0; k < 9; k++) {
    final int i = k;
    Constraint allDiffCol = new AllDifferent(
        makeIntVarArray(9, j -> grid[j * 9 + i]));
    Constraint allDiffBlock = new AllDifferent(
        makeIntVarArray(9,
            j -> grid[blocks.get(i).get(j)]));
    constraints.add(allDiffBlock);
    constraints.add(allDiffCol);
}
constraintSystem = new ConstraintSystem(
    constraints.toArray(new Constraint[0]));
violation = constraintSystem.violation();

// Precompute intra-row swaps on non-hint cells
possibleSwaps = new ArrayList<>();
for (int r = 0; r < 9; r++) {
    for (int i = 0; i < 9; i++) {
        for (int j = i + 1; j < 9; j++) {
            int v1 = r * 9 + i;
            int v2 = r * 9 + j;
            if (problem[v1] == 0 && problem[v2] == 0) {
                possibleSwaps.add(new Pair(v1, v2));
            }
        }
    }
}
```

Listing 8.8: CBLS Sudoku model enforcing row permutations as hard constraints and column/block `AllDifferent` as soft constraints.

```java
public void solveWithTabu() {
    int iter = 0;
    int tabu = 20;
    int bestViol = violation.value();
    while (violation.value() > 0) {
        Pair best = bestSwapTabuAspiration(
            iter, bestViol);
        grid[best.a].swap(grid[best.b]);
        best.iter = iter + 10 + rand.nextInt(tabu);
        bestViol = Math.min(bestViol, violation.value());
        iter++;
    }
}

public Pair bestSwapTabuAspiration(
        int iter, int bestViol) {
    Pair bestSwap = null;
    int bestDelta = Integer.MAX_VALUE;
    int curViol = violation.value();
    for (Pair p : possibleSwaps) {
        int delta = violation.getSwapDelta(
            grid[p.a], grid[p.b]);
        boolean aspires = (curViol + delta < bestViol);
        if ((p.iter <= iter || aspires)
                && delta < bestDelta) {
            bestDelta = delta;
            bestSwap = p;
        }
    }
    return bestSwap;
}
```

Listing 8.9: Tabu Search with randomized tenure and aspiration criterion for CBLS Sudoku.

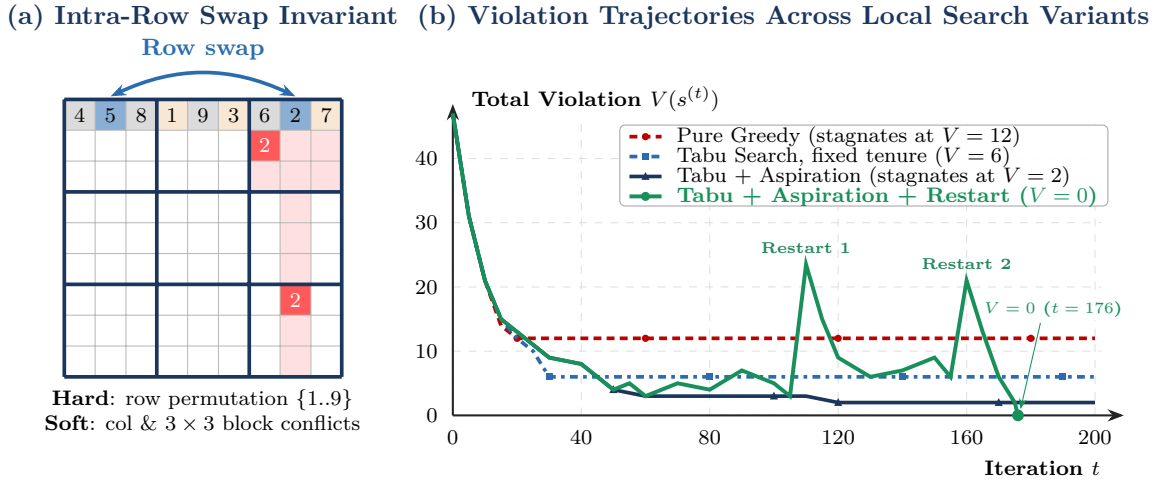


Figure 8.13: Constraint-Based Local Search on a 9×9 Sudoku puzzle. (a) Initializing each row $r \in \{0, \ldots, 8\}$ as a valid permutation of $\{1, \ldots, 9\}$ and restricting moves to intra-row swaps (v_1, v_2) enforces all row constraints as hard invariants, leaving only column and 3×3 block conflicts as soft constraints. (b) Evolution of the total violation $V(s^{(t)})$ across 200 iterations for Pure Greedy, Tabu Search, Tabu + Aspiration, and Tabu + Aspiration + Restart ($R = 50$), which reaches $V = 0$ at $t = 176$.

Flattening / Iterative Relaxation (`IFlat-IRelax`) algorithm [GLN05] represents a candidate schedule indirectly as a *precedence Directed Acyclic Graph* (DAG) $G = (V, E)$, where an arc $(i \to j) \in E$ enforces $start_j \geq start_i + d_i$. Given any precedence DAG G, the earliest-start schedule est_i for all activities is computed in $\mathcal{O}(|V| + |E|)$ time by dynamic programming (longest paths from dummy source t_0 in the topological order of G), and the makespan equals the length of the *critical path* in G.

The `IFlat-IRelax` algorithm alternates between two complementary phases:

1. **Flatten (`IFlat`)**: While the earliest-start schedule of G exhibits a resource peak exceeding capacity C at some time t ($\sum r_i > C$), select a pair of overlapping activities (i, j) contributing to the peak and insert a precedence constraint $(i \to j)$ into E (pushing j after i), updating earliest start times until the cumulative profile is completely flattened below C.
2. **Relax (`IRelax`)**: Because the makespan is determined solely by the critical path of G, removing non-critical precedence arcs cannot possibly reduce the makespan! Therefore, the relax phase identifies the critical path of G and randomly removes a subset of added precedence arcs along the critical path, allowing critical activities to shift left before re-invoking `IFlat`.

Example 8.6 *Trace of `IFlat-IRelax` on a 6-Activity Instance*

Consider the 6-activity cumulative scheduling instance ($i \in \{0, 1, \ldots, 5\}$) with resource capacity $C = 20$ and activity parameters given in Table 8.5, illustrated step by step in Figure 8.14:

1. **Current Flattened State**: Under the current precedence DAG $E = \{(t_0 \to 3), (t_0 \to 0), (3 \to 5), (5 \to 1), (3 \to 1), (0 \to 2), (0 \to 4)\}$, dynamic programming yields earliest start times $est_0 = 0$ (ends at 10), $est_3 = 0$ (ends at 13), $est_2 = 10$ (ends at 24), $est_4 = 10$ (ends at 31), $est_5 = 13$ (ends at 18), and $est_1 = 18$ (ends at 34). The resource load never exceeds $17 \leq 20$, and the makespan is 34, dictated by the *critical path* $t_0 \to 3 \to 5 \to 1$ ($13 + 5 + 16 = 34$).
2. **Relax Step**: To decrease the makespan, we must relax an arc on the critical path $t_0 \to 3 \to 5 \to 1$. Suppose we remove the precedence arc $(5 \to 1)$. Because $(3 \to 1)$ remains in the DAG, activity 1 shifts left to start immediately after activity 3 at $est_1 = 13$ (ending at $13 + 16 = 29$), dropping the provisional makespan to $\max(29, 31) = 31$ (now governed by path $t_0 \to 0 \to 4$ of length $10 + 21 = 31$).
3. **Flatten Step**: However, during the interval $t \in [13, 18)$, activities $4, 2, 5$, and 1 now execute concurrently, requiring a cumulative load of $r_4 + r_2 + r_5 + r_1 = 4 + 4 + 9 + 7 = 24 > C = 20$! To resolve this overload without re-imposing $(5 \to 1)$, the flatten phase posts a new precedence arc between two overlapping activities—for instance, $(2 \to 5)$ (delaying activity 5 of duration 5 to start at $t = 24$ and end at $29 \leq 31$), restoring feasibility across the entire horizon while preserving the improved makespan of 31.

Java Implementation and 50-Activity Benchmark

Listings 8.10 and 8.11 present the complete Java implementation of `IFlat-IRelax` in class `IFlatIRelaxScheduling`:

- In Listing 8.10, `propagateDAG()` performs a forward topological pass (Kahn's algorithm)

Activity i	**0**	**1**	**2**	**3**	**4**	**5**
Duration d_i	10	16	14	13	21	5
Consumption r_i	7	7	4	10	4	9

Table 8.5: Activity durations d_i and resource consumptions r_i for the 6-activity cumulative scheduling instance ($C = 20$, Figure 8.14).

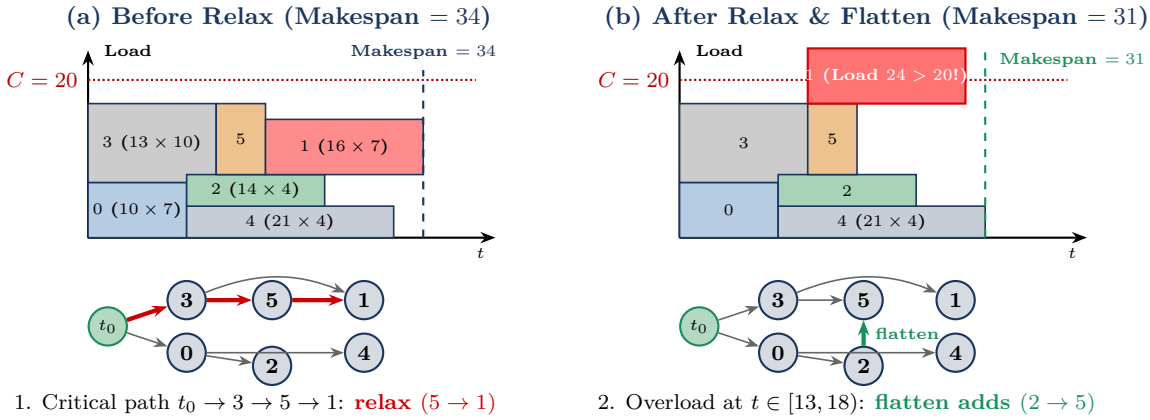


Figure 8.14: Step-by-step execution of `IFlat-IRelax` on the 6-activity instance of Table 8.5 ($C = 20$). *Left*: In the current feasible schedule of makespan 34, relaxing the critical arc $(5 \to 1)$ allows activity 1 to shift left to $t = 13$, reducing the provisional makespan to 31 but creating a resource peak of $24 > 20$ on $[13, 18)$. *Right*: The flatten phase resolves the peak by posting $(2 \to 5)$, shifting activity 5 to $[24, 29)$ and yielding a feasible schedule of makespan 31.

to compute earliest start times $\texttt{est}[u]$ and the overall `makespan`, followed by a backward pass in reverse topological order to compute latest completion times $\texttt{lct}[u]$. An added precedence arc $(u \to v)$ lies on a critical path if and only if it is tight ($\texttt{est}[u]+d_u = \texttt{est}[v]$) and both endpoints have zero slack ($\texttt{est}[u] + d_u = \texttt{lct}[u]$ and $\texttt{est}[v] + d_v = \texttt{lct}[v]$). The method `relax()` removes critical-path arcs with probability `relaxProb`.

- In Listing 8.11, `flatten()` scans activity start times (the only time instants where the step-function resource profile can increase) to locate the earliest overload time `peakTime` where $\sum r_i > C$, selects two concurrently active tasks (a, b), orders the earlier-completing task first $(a \to b)$, and re-propagates the DAG until the entire horizon satisfies (8.18).

Figure 8.15 illustrates the behavior of `IFlat-IRelax` on a 50-activity instance ($i \in \{0, \ldots, 49\}$, durations $d_i \in [4, 18]$, requirements $r_i \in [3, 12]$, capacity $C = 25$, and volume lower bound $\lceil \sum_{i=0}^{49} d_i r_i / C \rceil = 178$). If all 50 activities started at $t = 0$, the initial resource peak would reach $\sum_{i=0}^{49} r_i = 398 \gg 25$. The initial `IFlat` pass resolves all resource conflicts to produce a feasible schedule of makespan $C_{\max}^{(0)} = 259$, and 80 iterations of critical-path relaxation and re-flattening (Figure 8.15b) compact the schedule down to $C_{\max}^{*} = 217$ (Figure 8.15a), achieving an 81.9% average resource utilization across the entire scheduling horizon.

8.7 Chapter Summary and Exercises

```java
public class IFlatIRelaxScheduling {
    public record Arc(int from, int to) {}
    private final int n, capacity;
    private final int[] d, r, est, lct;
    private final boolean[][] adj;
    private final List<Arc> addedArcs = new ArrayList<>();
    private int makespan;

    public IFlatIRelaxScheduling(int[] d, int[] r, int cap) {
        this.n = d.length; this.capacity = cap;
        this.d = d.clone(); this.r = r.clone();
        this.est = new int[n]; this.lct = new int[n];
        this.adj = new boolean[n][n];
    }

    public boolean propagateDAG() {
        int[] inDeg = new int[n], topo = new int[n];
        for (int u = 0; u < n; u++)
            for (int v = 0; v < n; v++)
                if (adj[u][v]) inDeg[v]++;
        Arrays.fill(est, 0);
        Queue<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < n; i++)
            if (inDeg[i] == 0) q.add(i);
        int vis = 0; makespan = 0;
        while (!q.isEmpty()) {
            int u = q.poll(); topo[vis++] = u;
            int ectU = est[u] + d[u];
            makespan = Math.max(makespan, ectU);
            for (int v = 0; v < n; v++)
                if (adj[u][v]) {
                    est[v] = Math.max(est[v], ectU);
                    if (--inDeg[v] == 0) q.add(v);
                }
        }
        if (vis < n) return false;
        Arrays.fill(lct, makespan);
        for (int k = n - 1; k >= 0; k--)
            for (int v = 0, u = topo[k]; v < n; v++)
                if (adj[u][v])
                    lct[u] = Math.min(lct[u], lct[v] - d[v]);
        return true;
    }

    public void relax(double relaxProb, Random rng) {
        addedArcs.removeIf(arc -> {
            int u = arc.from(), v = arc.to();
            boolean crit = (est[u] + d[u] == est[v])
                && (est[u] + d[u] == lct[u])
                && (est[v] + d[v] == lct[v]);
            if (crit && rng.nextDouble() < relaxProb) {
                adj[u][v] = false; return true;
            }
            return false;
        });
    }
```

Listing 8.10: Precedence DAG propagation (`propagateDAG`) and critical-path relaxation (`relax`) in `IFlatIRelaxScheduling`.

```
    public void flatten(Random rng) {
        propagateDAG();
        while (true) {
            int peakTime = -1;
            for (int k = 0; k < n; k++) {
                int t = est[k], load = 0;
                for (int i = 0; i < n; i++)
                    if (est[i] <= t && t < est[i] + d[i])
                        load += r[i];
                if (load > capacity
                        && (peakTime == -1 || t < peakTime))
                    peakTime = t;
            }
            if (peakTime == -1) break;
            List<Integer> active = new ArrayList<>();
            for (int i = 0; i < n; i++)
                if (est[i] <= peakTime
                        && peakTime < est[i] + d[i])
                    active.add(i);
            int iA = rng.nextInt(active.size());
            int iB = (iA + 1 + rng.nextInt(active.size() - 1))
                   % active.size();
            int a = active.get(iA), b = active.get(iB);
            if (est[a] + d[a] > est[b] + d[b]) {
                int tmp = a; a = b; b = tmp;
            }
            if (!adj[a][b]) {
                adj[a][b] = true;
                addedArcs.add(new Arc(a, b));
                propagateDAG();
            }
        }
    }

    public int solve(
            int maxIter, double relaxProb, long seed) {
        Random rng = new Random(seed);
        int bestMakespan = Integer.MAX_VALUE;
        for (int iter = 0; iter < maxIter; iter++) {
            if (iter > 0) relax(relaxProb, rng);
            flatten(rng);
            bestMakespan = Math.min(bestMakespan, makespan);
        }
        return bestMakespan;
    }
}
```

Listing 8.11: Iterative peak flattening (`flatten`) and main search loop (`solve`) of `IFlatIRelaxScheduling`.

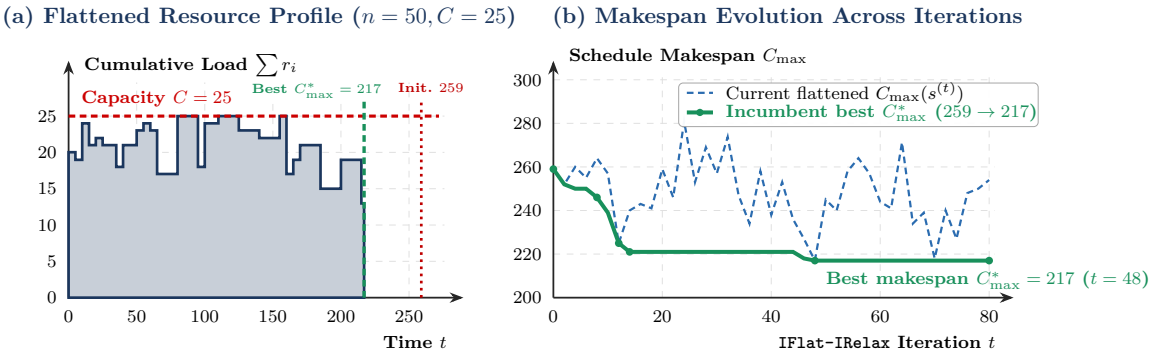


Figure 8.15: Experimental results of `IFlat-IRelax` on a 50-activity Cumulative Scheduling benchmark ($C = 25$, energy lower bound 178). (a) Cumulative resource profile $\sum r_i(t) \leq 25$ of the best flattened schedule ($C^*_{\max} = 217$). (b) Evolution of the current flattened makespan (faint spikes) and best-so-far makespan $C^*_{\max}$ across 80 iterations (259 → 217).

8.7.1 Key Conceptual Takeaways

In this chapter, we explored the foundational building blocks of heuristic and metaheuristic design. While we focused on trajectory-based methods, the broader landscape of local search also encompasses adaptive constructive learning (such as *Ant Colony Optimization* and *Greedy Randomized Adaptive Search Procedures*, GRASP), probabilistic acceptance schedules (*Simulated Annealing*, *Iterated Local Search*), population-based evolutionary methods (*Genetic* and *Memetic Algorithms*), and industrial CBLS/routing engines (such as *Comet*, *OscaR.cbls*, and *Google OR-Tools*). Table 8.6 synthesizes the core methods established in this chapter.

Method / Concept	Core Mechanism and Algorithmic Significance
Greedy Construction	Builds s element by element minimizing incremental cost $c(s, e)$ without backtracking.
Beam Search & Pilot	Mitigates greedy myopia via width-p breadth-first pruning or full heuristic rollouts $h(s_e)$.
First vs. Best Move	First-Improvement scales faster empirically ($\Theta(n^{2.33})$ vs. $\Theta(n^{3.25})$ on 2-Opt TSP).
Neighborhood Design	Balances connectivity, low diameter, low ruggedness, small size, and $\mathcal{O}(1)$ Δ-evaluation.
2-Opt, 3-Opt, Or-Opt	Edge-exchange TSP moves; Or-Opt relocates chains of 1–3 cities without reversing orientation.
ATSP-to-STSP Reduction	Jonker–Volgenant node doubling (u, u') with $-M$ edges maps directed tours to symmetric tours.
Filter-and-Fan & LKH	Explores compound / variable-depth k-Opt moves via Beam Search or sequential ejection chains.
Large Neighborhood (LNS)	Escapes deep local minima by destroying related components $d(s)$ and repairing $r(d(s))$.
Tabu Search	Escapes local minima via best non-tabu moves, short-term memory $d \approx \sqrt{\lvert\mathcal{N}(s)\rvert}$, and aspiration.
CBLS & `IFlat-IRelax`	Enforces hard invariants via moves, relaxes soft constraints into Δ-violations, or relaxes critical DAG arcs.

Table 8.6: Summary of constructive heuristics, neighborhood structures, and metaheuristics.

8.7.2 Exercises

Exercise 8.1 *Constructive TSP Heuristics Trace*

Consider the symmetric 6-city TSP sub-instance induced by cities $V = \{0, 1, 2, 3, 4, 5\}$ of the distance matrix in Table 8.2.

1. Execute the *Nearest Neighbor* heuristic starting from city 0. List the sequence of cities added, the incremental cost at each step, and the total cost of the resulting tour.
2. Execute the *Greedy on Edge Weight* (Multi-Fragment) heuristic on $V = \{0, \ldots, 5\}$. List all edges considered in non-decreasing order of weight, indicating for each edge whether it is accepted, rejected due to vertex degree 3, or rejected due to premature subtour closure.
3. Starting from the 2-city subtour $(0, 3, 0)$, trace one full iteration of *Cheapest Insertion* and one full iteration of *Farthest Insertion*, explicitly computing the detour cost $\Delta(i, k, j)$ for each candidate city $k \in \{1, 2, 4, 5\}$.

Exercise 8.2 *2-Opt Connectivity and Exact Reversal Sequence*

Let $V = \{0, 1, 2, 3, 4, 5, 6, 7\}$ be an 8-city TSP instance. Suppose the current tour is represented by the permutation array

$$s = [0, 5, 2, 7, 1, 6, 4, 3]$$

and the globally optimal tour is $s^* = [0, 1, 2, 3, 4, 5, 6, 7]$.

1. Apply the constructive algorithm from the proof of Theorem 8.1 to find a sequence of 2-Opt moves (subarray reversals) transforming s into s^*. Write down the tour array after each move.
2. Can you find a shorter sequence of 2-Opt moves transforming s into s^* than the prefix-by-prefix algorithm? Justify your answer.

Exercise 8.3 *Constant-Time Δ-Evaluation for 3-Opt and Or-Opt*

Let $\texttt{tour}[0 \ldots n-1]$ represent a tour in an Asymmetric TSP with directed distance matrix $D[u][v]$ (where $D[u][v] \neq D[v][u]$ in general).

1. Explain mathematically why the standard 2-Opt move $\texttt{twoOpt}(i, j)$ cannot be evaluated in $\mathcal{O}(1)$ time from D alone when the distance matrix is asymmetric. How could you precompute auxiliary prefix-sum arrays over the current tour so that $\Delta_{\text{2-Opt}}(i, j)$ can be queried in $\mathcal{O}(1)$ time during a single neighborhood scan?
2. Consider the orientation-preserving 3-Opt move (8.9) that replaces directed arcs (i, s_i), (j, s_j), and (k, s_k) with (i, s_j), (j, s_k), and (k, s_i). Write the exact $\mathcal{O}(1)$ formula for $\Delta_{\text{3-Opt}}(i, j, k)$ in terms of D and show that it remains valid for asymmetric instances.
3. Derive the $\mathcal{O}(1)$ incremental cost formula $\Delta_{\text{Or-Opt}}(i, \ell, p)$ for excising a contiguous chain of $\ell \in \{1, 2, 3\}$ cities starting after $\texttt{tour}[i]$ and reinserting it in the same orientation immediately after $\texttt{tour}[p]$.

Exercise 8.4 *Jonker–Volgenant ATSP-to-STSP Transformation*

Consider a 3-city Asymmetric TSP on $V = \{0, 1, 2\}$ with directed distance matrix:

$$\mathbf{W} = \begin{pmatrix} - & 10 & 25 \\ 40 & - & 15 \\ 20 & 35 & - \end{pmatrix}.$$

1. Evaluate the cost of the two possible directed Hamiltonian tours $(0 \to 1 \to 2 \to 0)$ and $(0 \to 2 \to 1 \to 0)$ in G.
2. Choose a finite penalty constant $M = 100$ and write out the complete 6×6 symmetric distance matrix $\mathbf{D}$ on vertices $\{0, 1, 2, 0', 1', 2'\}$ defined by Theorem 8.2.
3. Verify that the optimal undirected Hamiltonian tour in G' has cost $C^* - 3M$ and alternates properly between V and V'. What threshold value of M is strictly required in general to guarantee that all n twin edges $\{u, u'\}$ are selected in every optimal STSP tour?

Exercise 8.5 *Tabu Search with Aspiration Criterion*

Revisit the 9-item 0-1 Knapsack instance (8.15) from Example 8.5 with capacity $C = 45$ and tabu tenure $d = 3$.

1. Suppose we initialize the search at $\mathbf{x}^{(0)} = (1, 1, 0, 0, 1, 0, 0, 1, 0)$ (items $0, 1, 4, 7$ selected). Compute the initial profit $r(\mathbf{x}^{(0)})$ and weight $v(\mathbf{x}^{(0)})$.
2. Trace 6 iterations of Tabu Search ($d = 3$) starting from $\mathbf{x}^{(0)}$, recording at each step the flipped variable, the solution vector, r, v, and the Tabu Expiration Array.
3. Construct a small state configuration during the search where a move m is currently marked as tabu, yet satisfies the *aspiration criterion* $r(\mathbf{x} \oplus m) > r(\mathbf{x}^*)$. Modify Algorithm 20 and Listing 8.7 to incorporate this aspiration check.

Exercise 8.6 *CBLS Modeling and $\mathcal{O}(1)$ Swap Delta for n-Queens*

In the n-Queens problem, n queens must be placed on an $n \times n$ chessboard (rows and columns indexed $0 \dots n-1$) so that no two queens attack each other along the same row, column, or diagonal.

1. Formulate a CBLS model where variables $q_0, q_1, \dots, q_{n-1} \in \{0, \dots, n-1\}$ represent the row of the queen placed in column i. Which constraints are enforced automatically by this representation, which constraint can be enforced as a *hard constraint* by initializing $\mathbf{q}$ as a permutation of $\{0, \dots, n-1\}$ and restricting moves to swaps swap(i, j), and which constraints remain *soft*?
2. Express the soft diagonal constraints mathematically using the AllDifferent global constraint over linear expressions of q_i and i.
3. Design auxiliary frequency arrays storing the number of queens on each upward diagonal $(q_i + i)$ and downward diagonal $(q_i - i + n - 1)$, and write a Java method `getSwapDelta(int i, int j)` that computes the exact change in total diagonal violations in $\mathcal{O}(1)$ time without executing the swap.

Exercise 8.7 *Cumulative Scheduling and Critical Path Relaxation*

Consider the 6-activity cumulative scheduling instance in Table 8.5 with capacity $C = 20$ and the state depicted in the right panel of Figure 8.14 after posting the flattening arc $(2 \to 5)$.

1. Compute the exact earliest start time est_i and earliest completion time $ect_i = est_i + d_i$ for every activity $i \in \{0, \dots, 5\}$ in the updated DAG.
2. Verify that at every time $t \in [0, 31]$, the cumulative resource consumption satisfies $\sum r_i \leq 20$.
3. Identify the new critical path(s) in the updated DAG. Can any single precedence arc on the new critical path be relaxed to reduce the makespan below 31 without violating capacity $C = 20$? Prove your answer.

CHAPTER 9

Constraint Programming

In the preceding chapters, we explored mathematical optimization paradigms built upon specific structural foundations: Dynamic Programming exploits recursive Bellman decompositions over acyclic state graphs, Linear and Integer Linear Programming leverage polyhedral geometry and linear relaxations, Network Flow algorithms exploit graph conservation laws, and Local Search navigates combinatorial landscapes via neighborhood perturbations.

When transitioning from textbook abstractions to industrial decision-making, however, practitioners quickly encounter a sobering reality: **real-world discrete optimization problems are messy**. A pure Traveling Salesman Problem rarely exists outside academic benchmarks. In practical logistics, routing problems involve heterogeneous vehicle fleets, complex time windows, compartment incompatibilities, sequence-dependent loading rules, and driver rest regulations. Similarly, hospital nurse rostering requires satisfying intricate temporal patterns—such as limiting consecutive working days, guaranteeing minimum rest days after night-shift sequences, and forbidding isolated working days ("E–L–E" shift patterns)—alongside personalized employee preferences.

Whenever a problem features rich logical, combinatorial, or non-linear substructures that resist clean linear formulations or explodes the state space of dynamic programming, *Constraint Programming* (CP) emerges as the Swiss Army knife of discrete optimization [RVW06; MSV21]. Rather than forcing every constraint into a matrix of linear inequalities $\mathbf{Ax} \leq \mathbf{b}$, Constraint Programming treats constraints as active, heterogeneous algorithmic components. Much like interlocking building blocks, arbitrary constraints—whether arithmetic, logical, graph-theoretic, or global—cooperate through a shared domain store to prune impossible values and guide tree search.

This chapter introduces the foundational principles, modeling methodology, and solver architecture of Constraint Programming. Using the classic n-Queens problem as a pedagogical thread, we progress systematically through three algorithmic stages:

1. **Depth-First Search and Filtering** (`NQueensChecker`): A pure generate-and-test baseline that enumerates complete assignments and verifies constraints only at the leaves of the search tree.
2. **Depth-First Search and Prefix Pruning** (`NQueensPrune`): A backtracking procedure that checks constraints incrementally on partial decision prefixes to cut off infeasible subtrees early.
3. **A Generic Constraint Solver** (`TinyCSP`): A reusable, declarative CP engine encapsulating finite-domain variables, bitset domains, constraint propagation via a fixpoint loop, and backtracking search with state restoration.

We then extend the solver with bound-consistent arithmetic and relational propagators (`LessOrEqual`, `Sum`, and `KnightMove`), analyze empirical performance profiles to motivate data-driven propagation queues, contrast CP and Mixed-Integer Programming (MIP) formulations on the Traveling Salesman Problem, and apply our solver to three classic combinatorial benchmarks: the *Magic Square*, *Killer Sudoku*, and *Knight's Tour* problems.

9.1 The Declarative Programming Paradigm

In a celebrated reflection on the foundations of artificial intelligence, Eugene Freuder articulated the vision driving Constraint Programming [Fre97]:

> *"Constraint programming represents one of the closest approaches computer science has yet made to the Holy Grail of programming: the user states the problem, the computer solves it."*

Stating a problem without specifying step-by-step instructions for how to compute its solution is the hallmark of *declarative programming*.

Definition 9.1 *Declarative Programming for Combinatorial Problems*

Declarative programming is a programming paradigm that expresses the logic and mathematical properties of a computation without prescribing its explicit control flow. In the context of constrained combinatorial problems, declarative modeling means specifying the decision variables, their candidate value sets (domains), and the structural properties (constraints and objective function) that a valid solution must satisfy, delegating the algorithmic search for such a solution to a generic *solver*.

In the constraint programming literature, this separation of concerns is captured by the foundational equation:

$$\textbf{Constraint Programming} = \textbf{Model} + \textbf{Search}. \tag{9.1}$$

The two components play complementary roles:

- **The Model**: A declarative specification written through a high-level user API. The modeler declares the decision variables, their initial domains, and the collection of constraints linking them. For instance, in a high-level modeling language, placing $n = 8$ non-attacking queens on a chessboard is stated concisely as:

$$\begin{aligned} &\text{range } R = \{0, \dots, 7\}; \\ &\text{var}\{\mathbb{Z}\}\ q[R] \in R; \\ &\forall i \in R,\ \forall j \in R \text{ with } i < j : \\ &\qquad q[i] \neq q[j], \quad q[i] \neq q[j] + (j - i), \quad q[i] \neq q[j] - (j - i). \end{aligned} \tag{9.2}$$

- **The Search**: The algorithmic engine executed by the solver to discover an assignment of values to variables that satisfies all constraints (and optimizes an objective function, if present), typically by exploring a tree of branching decisions interleaved with constraint inference.

To master Constraint Programming, a computer scientist must understand both sides of this coin: *how to build* the internal inference and search engine of a CP solver, and *how to use* its modeling abstractions effectively to solve complex combinatorial problems. We

begin by formalizing the mathematical object at the heart of every CP solver: the Constraint Satisfaction Problem [Mac77; Lau78; RVW06].

Definition 9.2 *Constraint Satisfaction Problem (CSP)*

A *Constraint Satisfaction Problem* is a triple $\mathcal{P} = \langle X, \mathcal{D}, \mathcal{C} \rangle$ where:

1. $X = \{x_0, x_1, \ldots, x_{n-1}\}$ is a finite tuple of n *decision variables*.
2. $\mathcal{D} = (D(x_0), D(x_1), \ldots, D(x_{n-1}))$ is a family of finite sets called *domains*, where each variable $x_i \in X$ must take its value from $D(x_i) \subseteq \mathbb{Z}$.
3. $\mathcal{C} = \{c_0, c_1, \ldots, c_{m-1}\}$ is a finite set of *constraints*. Each constraint $c \in \mathcal{C}$ is defined over a subset of variables $\text{scope}(c) = (x_{i_0}, \ldots, x_{i_{k-1}}) \subseteq X$ of arity $k = |\,\text{scope}(c)|$, and specifies a relation $\text{sol}(c) \subseteq D(x_{i_0}) \times \cdots \times D(x_{i_{k-1}})$ of admissible value tuples.

A *complete assignment* $(v_0, v_1, \ldots, v_{n-1}) \in D(x_0) \times \cdots \times D(x_{n-1})$ is a *solution* to $\mathcal{P}$ if and only if its projection onto $\text{scope}(c)$ belongs to $\text{sol}(c)$ for every constraint $c \in \mathcal{C}$.

When a CSP is augmented with an objective function $f : D(x_0) \times \cdots \times D(x_{n-1}) \to \mathbb{R}$ to be minimized or maximized over the set of all feasible solutions, it is called a *Constraint Optimization Problem* (COP).

9.2 Modeling the n-Queens Problem

To illustrate how modeling choices profoundly affect the size of the search space and the structure of constraints, consider the classic *n-Queens problem*.

Definition 9.3 *The n-Queens Problem*

Given an $n \times n$ chessboard with rows and columns indexed in $\{0, 1, \ldots, n-1\}$, place n queens on the board so that no two queens threaten each other. According to the movement rules of chess, no two queens may share the same row, the same column, or the same upward or downward diagonal.

Suppose we wish to formulate the n-Queens problem (for example, with $n = 8$) as a Constraint Satisfaction Problem. How should we choose the decision variables X and their domains $\mathcal{D}$? Let us compare two natural representations, depicted in Figure 9.1.

9.2.1 Model 1: The Boolean Matrix Representation

A first, naive approach introduces a Boolean decision variable $x_{r,c} \in \{0, 1\}$ (or $\{\texttt{false}, \texttt{true}\}$) for every cell $(r, c) \in \{0, \ldots, n-1\}^2$ of the chessboard, indicating whether a queen is present in row r and column c (Figure 9.1a).

While conceptually straightforward, this representation suffers from two major drawbacks:

1. **Explicit Constraints on All Three Axes**: Because any combination of cells can be set to 1, the model must explicitly enforce that the total number of queens is n, and that no two queens share the same *row*, the same *column*, or the same *diagonal*.
2. **Astronomical Search Space**: With n^2 binary variables, the unconstrained Cartesian product of the domains has size:

$$|\mathcal{D}_{\text{bool}}| = 2^{n^2}. \tag{9.3}$$

For $n = 8$, this yields $2^{64} \approx 1.84 \times 10^{19}$ candidate configurations.

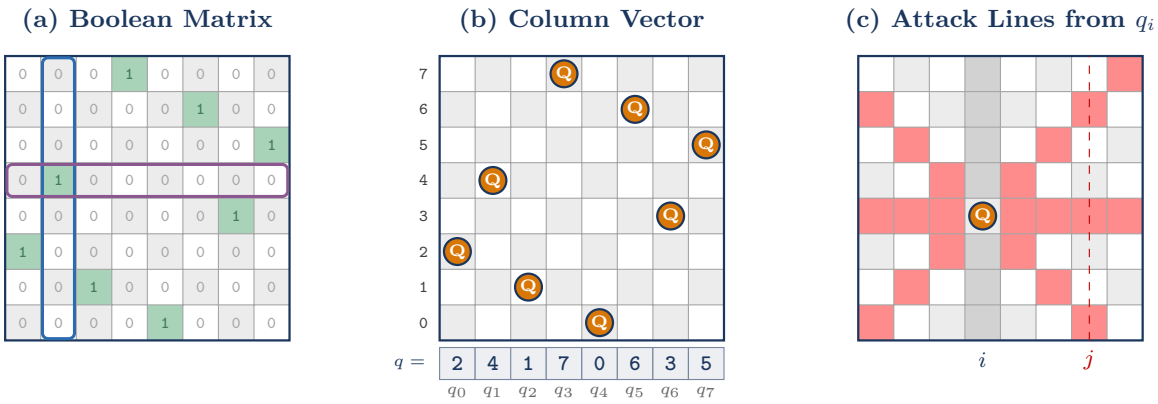


Figure 9.1: Alternative modeling representations for the 8-Queens problem on an 8×8 board. (a) The Boolean matrix model introduces a binary variable $x_{r,c} \in \{0, 1\}$ for each cell $(r, c) \in \{0, \ldots, 7\}^2$, yielding 2^{64} candidate states and requiring explicit row, column, and diagonal constraints. (b) The column decision vector model assigns a single row index $q_i \in \{0, \ldots, 7\}$ to each column i, reducing the search space to $8^8 = 2^{24}$ states and satisfying the column constraint by construction. (c) Pairwise row and diagonal attack lines emanating from queen q_i.

9.2.2 Model 2: The Column Decision Vector Representation

We can dramatically improve the formulation by exploiting a structural invariant of the problem: since n queens must be placed on n columns and no two queens may share a column, *every valid solution places exactly one queen in each column.*

Consequently, we associate a single integer decision variable q_i with each column $i \in \{0, 1, \ldots, n-1\}$, representing the *row index* in which the queen of column i is placed (Figure 9.1b):

$$D(q_i) = \{0, 1, \ldots, n-1\}, \quad \forall i \in \{0, 1, \ldots, n-1\}. \tag{9.4}$$

For example, the assignment vector $\mathbf{q} = (2, 4, 1, 7, 0, 6, 3, 5)$ places the queen of column 0 in row 2, the queen of column 1 in row 4, and so forth, yielding the valid 8-Queens configuration illustrated in Figure 9.1b.

This column-vector representation provides two decisive advantages:

1. **Elimination of Column Constraints**: Because each column i has exactly one variable q_i, it is impossible to place two queens in the same column. Only *two* types of constraints remain to be tested: no two queens may share the same row or the same diagonal (Figure 9.1c). Formally, for every pair of distinct columns $0 \leq i < j < n$:

$$q_i \neq q_j \qquad \text{(no shared row)}, \tag{9.5}$$
$$|q_j - q_i| \neq j - i \qquad \text{(no shared diagonal)}. \tag{9.6}$$

   Note that the absolute-value condition $|q_j - q_i| \neq j - i$ decomposes into two linear disequalities with constant offsets:

$$q_i \neq q_j + (i-j) \quad \text{(upward diagonal)} \qquad \text{and} \qquad q_i \neq q_j + (j-i) \quad \text{(downward diagonal)}. \tag{9.7}$$

2. **Massive State Space Reduction**: The Cartesian product of the domains now contains

$$|\mathcal{D}_{\text{col}}| = n^n \tag{9.8}$$

configurations. For $n = 8$, $8^8 = (2^3)^8 = 2^{24} \approx 1.67 \times 10^7$ states—a reduction by a factor of $2^{40} \approx 1.1 \times 10^{12}$ compared to the Boolean matrix model!

9.3 Approach 1: Generate and Filter (DFS + Filter)

Having selected the column vector model $q_0, \ldots, q_{n-1} \in \{0, \ldots, n-1\}$, how can we discover all valid solutions? The simplest conceivable architecture decouples the problem into two sequential stages (Figure 9.2a):

1. **Generator**: A recursive Depth-First Search (DFS) enumerates all n^n complete candidate assignments $(q_0, \ldots, q_{n-1}) \in \{0, \ldots, n-1\}^n$.
2. **Filter (Checker)**: Whenever a complete assignment is reached at a leaf of the recursion tree (`idx` $= n$), a boolean predicate `constraintsSatisfied()` tests whether all pairwise row and diagonal constraints (9.5)–(9.6) hold.

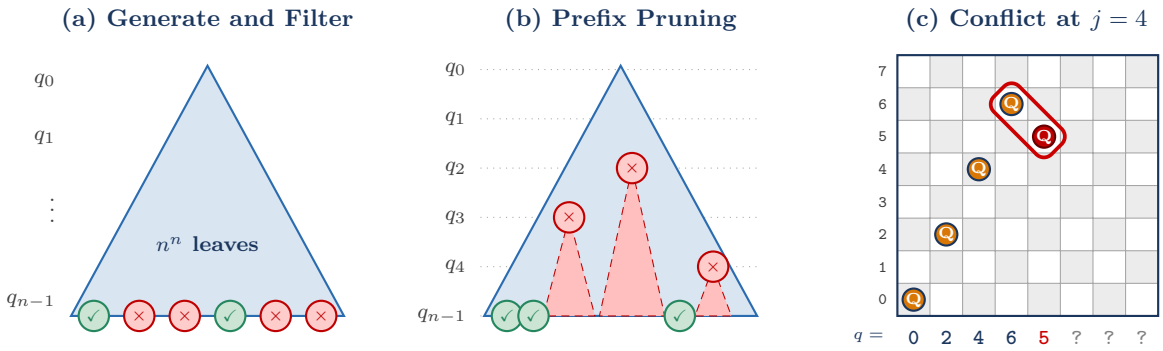


Figure 9.2: Comparison between (a) Generate-and-Filter (`NQueensChecker`), which explores the entire n-ary recursion tree of n^n leaves and verifies constraints only when all decisions are finished, and (b) Prefix Pruning (`NQueensPrune`), which checks constraints on the partial decision prefix $(q_0, \ldots, q_j)$ at every internal node and prunes inconsistent subtrees immediately. (c) An example of prefix pruning at column $j = 4$: placing a queen at $q_4 = 5$ attacks the previously placed queen at $q_3 = 6$ along the downward diagonal ($|5 - 6| = 1 = 4 - 3$), cutting off all $8^3 = 512$ completions of the prefix $(0, 2, 4, 6, 5)$.

9.3.1 Inversion of Control via the Hollywood Principle

When designing a search procedure that may discover zero, one, or thousands of solutions, we should avoid hardcoding what the solver does when a solution is found (such as printing to the console or storing in a specific collection). Instead, we apply the software engineering pattern known as the *Hollywood Principle* ("Don't call us, we'll call you"): the caller passes a callback closure—in Java, an instance of the functional interface `Consumer<int[]>`—that the search engine invokes whenever a valid solution is discovered.

Listing 9.1 presents the complete implementation of `NQueensChecker`. At depth `idx` $= n$, if `constraintsSatisfied()` returns `true`, the solver passes a defensive copy `Arrays.copyOf(q, n)` of the decision array to `onSolution.accept(...)`.

With this callback design, collecting all solutions to the 8-Queens problem requires only a three-line client program:

```
NQueensChecker q = new NQueensChecker(8);
ArrayList<int[]> solutions = new ArrayList<>();
```

```java
import java.util.Arrays;
import java.util.function.Consumer;

public class NQueensChecker {
    int[] q;
    int n = 0;

    public NQueensChecker(int n) {
        this.n = n;
        q = new int[n];
    }

    public void dfs(Consumer<int[]> onSolution) {
        dfs(0, onSolution);
    }

    private void dfs(int idx,
                     Consumer<int[]> onSolution) {
        if (idx == n) {
            if (constraintsSatisfied()) {
                onSolution.accept(Arrays.copyOf(q, n));
            }
        } else {
            for (int i = 0; i < n; i++) {
                q[idx] = i;
                dfs(idx + 1, onSolution);
            }
        }
    }

    public boolean constraintsSatisfied() {
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                // No two queens on the same row
                if (q[i] == q[j]) return false;
                // No two queens on the same diagonal
                if (Math.abs(q[j] - q[i]) == j - i) {
                    return false;
                }
            }
        }
        return true;
    }
}
```

Listing 9.1: Generate-and-filter implementation for the n-Queens problem (`NQueensChecker.java`).

```
q.dfs(solution -> solutions.add(solution));
```

Notice that the generate-and-filter architecture is already somewhat generic: to solve another problem over n variables with domain $\{0, \ldots, n-1\}$, one can declare the predicate `constraintsSatisfied()` as an abstract method and override it in a subclass. However, its computational complexity is prohibitive: the recursion visits every node of a complete n-ary tree of height n, resulting in

$$\sum_{k=0}^{n} n^k = \frac{n^{n+1} - 1}{n - 1} = \Theta(n^n) \tag{9.9}$$

recursive calls, regardless of how early a conflict occurs! For instance, even when the first two queens are placed on the exact same row ($q_0 = 0$ and $q_1 = 0$), `NQueensChecker` blindly enumerates all n^{n-2} placements for the remaining queens $q_2, \ldots, q_{n-1}$ before discovering at the leaves that $q_0 = q_1$.

9.4 Approach 2: Prefix Pruning (DFS + Prune)

To eliminate the wasted work of `NQueensChecker`, we move the feasibility test *inside* the search tree (Figure 9.2b). Whenever the recursive search assigns a candidate row $q_j = i$ to column j, the prefix of decisions $(q_0, q_1, \ldots, q_j)$ is already fixed. We can immediately verify whether the newly placed queen q_j conflicts with any previously placed queen $q_0, \ldots, q_{j-1}$ (Figure 9.2c).

Specifically, the overloaded method `constraintsSatisfied(int j)` checks only the pairs (i, j) with $0 \le i < j$:

$$\forall i \in \{0, \ldots, j-1\}: \quad q_i \neq q_j \quad \wedge \quad |q_j - q_i| \neq j - i. \tag{9.10}$$

If any conflict is detected, the recursive call `dfs(idx + 1, onSolution)` is skipped, pruning the entire subtree of n^{n-1-j} completions beneath that node. Conversely, if the search reaches depth `idx` $= n$, every prefix $(q_0, \ldots, q_j)$ for $j \in \{0, \ldots, n-1\}$ has already been validated, so the complete vector **q** is guaranteed to be a valid solution without any further testing. Listing 9.2 gives the resulting `NQueensPrune` implementation.

9.4.1 Limitations of Hardcoded Prefix Pruning

While `NQueensPrune` reduces the search tree by orders of magnitude, it suffers from three fundamental architectural limitations that prevent it from serving as a general-purpose optimization tool:

1. **Passive Backward Checking vs. Active Forward Propagation**: In `NQueensPrune`, placing a queen at $q_0 = 0$ does not proactively remove row 0 or the diagonal cells from the domains of future columns $q_1, \ldots, q_{n-1}$. Instead, the algorithm still tries every value $i \in \{0, \ldots, n-1\}$ at level `idx` and relies on backward checks against $i <$ `idx` to reject invalid values one by one. Because it maintains no explicit domains for future variables, it cannot detect a dead end in a future column $k >$ `idx` until the recursion actually reaches level k.
2. **Rigid Static Variable Ordering**: The recursion is locked to a single depth index `idx` that marches strictly from 0 to $n-1$, relying on overwriting `q[idx]` upon backtracking. It cannot dynamically select the most constrained unfixed variable next.

```java
import java.util.Arrays;
import java.util.function.Consumer;

public class NQueensPrune {
    int[] q;
    int n = 0;

    public NQueensPrune(int n) {
        this.n = n;
        q = new int[n];
    }

    public void dfs(Consumer<int[]> onSolution) {
        dfs(0, onSolution);
    }

    private void dfs(int idx,
                     Consumer<int[]> onSolution) {
        if (idx == n) {
            onSolution.accept(Arrays.copyOf(q, n));
        } else {
            for (int i = 0; i < n; i++) {
                q[idx] = i;
                if (constraintsSatisfied(idx)) {
                    dfs(idx + 1, onSolution);
                }
            }
        }
    }

    public boolean constraintsSatisfied(int j) {
        for (int i = 0; i < j; i++) {
            // No two queens on the same row
            if (q[i] == q[j]) return false;
            // No two queens on the same diagonal
            if (Math.abs(q[j] - q[i]) == j - i) {
                return false;
            }
        }
        return true;
    }
}
```

Listing 9.2: Backtracking search with prefix pruning (`NQueensPrune.java`).

3. **Zero Reusability**: The decision variables, domain bounds, and conflict checks are hardcoded into a single monolithic loop. None of the code in `NQueensPrune` can be reused to solve even a closely related problem such as Sudoku or Magic Square.

To overcome these limitations, we now design `TinyCSP`: a generic, reusable constraint programming solver that separates declarative modeling from the underlying propagation and search engines.

9.5 Architecture of a Generic Solver: `TinyCSP`

Before examining the internal mechanics of `TinyCSP`, let us look at how a user models and solves the n-Queens problem using its declarative API (Listing 9.3).

```java
int n = 8;
TinyCSP csp = new TinyCSP();
Variable[] q = new Variable[n];

// 1. Create decision variables with domain {0..n-1}
for (int i = 0; i < n; i++) {
    q[i] = csp.makeVariable(n);
}

// 2. Post pairwise row and diagonal constraints
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        // Queens q[i] and q[j] not on ...
        csp.notEqual(q[i], q[j], 0);     // same row
        csp.notEqual(q[i], q[j], i - j); // left diag
        csp.notEqual(q[i], q[j], j - i); // right diag
    }
}

// 3. Execute Depth-First Search to collect solutions
ArrayList<int[]> solutions = new ArrayList<>();
csp.dfs(() -> {
    int[] solution = Arrays.stream(q)
        .mapToInt(x -> x.dom.min()).toArray();
    solutions.add(solution);
});
```

Listing 9.3: Declarative model of the n-Queens problem using `TinyCSP` (`NQueens.java`).

Notice how Listing 9.3 mirrors the declarative mathematical specification (9.2): the user creates an array of `Variable` objects, posts `notEqual` constraints with appropriate offsets (0, $i-j$, and $j-i$), and invokes `csp.dfs(...)`. No problem-specific search loop or conflict checker is written by the user.

9.5.1 Computational Paradigm: Search and Store

How does `TinyCSP` execute this model under the hood? Every constraint programming solver is organized around two interacting subsystems, illustrated in Figure 9.3:

1. **The Constraint Store**: Maintains the *Domain Store*—the current domain $D(x)$ of every variable $x \in X$—together with the active *constraints* $c_0, c_1, \ldots, c_{m-1} \in \mathcal{C}$. Each constraint c encapsulates a dedicated filtering algorithm (called a *propagator*) that

inspects the domains of its variables and removes values that cannot participate in any feasible solution of c.

2. **The Search Engine**: Explores a tree of branching decisions (for example, $x_0 = 0$ on the left branch vs. $x_0 \neq 0$ on the right branch). Whenever the search engine imposes a branching decision on the domain store, it triggers the propagation engine of the constraint store. The constraint store replies with either **Success** (returning the tightened domains after all propagation has stabilized) or **Failure** (signaling that some variable's domain has become empty, forcing the search engine to backtrack).

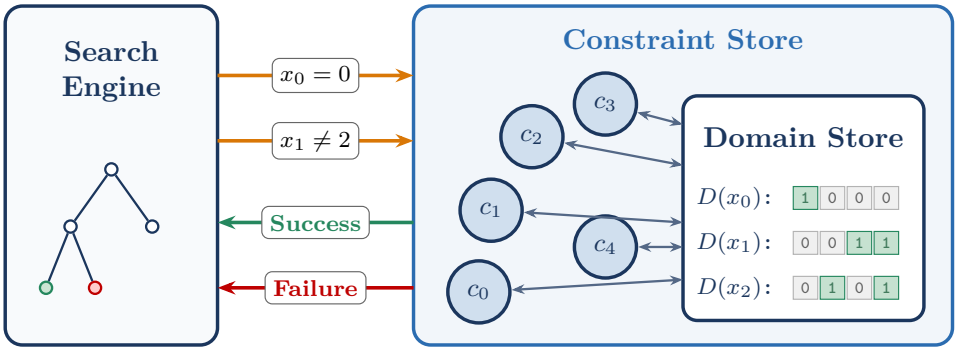


Figure 9.3: Computational architecture of a Constraint Programming solver. The Search Engine explores a depth-first branching tree and posts decisions (such as $x_0 = 0$ or $x_1 \neq 2$) to the Constraint Store. Inside the Constraint Store, propagators $c_0, \ldots, c_4$ read and prune the bitset domains $D(x_0), \ldots, D(x_{n-1})$ in the shared Domain Store until reaching a fixpoint (**Success**) or emptying a domain (**Failure**).

9.5.2 The Fixpoint Propagation Engine

What does an individual constraint $c \in \mathcal{C}$ do inside the constraint store? It performs two intertwined tasks:

- **Feasibility Checking**: Determines whether the constraint can still be satisfied given the current domains of its variables. If no valid tuple remains, or if pruning removes the last remaining value from a variable's domain ($D(x) = \emptyset$), the constraint signals an *inconsistency* (failure).
- **Domain Pruning (Filtering)**: Removes from the domains of scope(c) values that cannot belong to any feasible assignment of c.

Because different constraints share variables, a chain reaction occurs: when constraint c_0 prunes a value from $D(x)$, that domain reduction may enable another constraint c_1 (whose scope also contains x) to prune a value from $D(y)$, which in turn may allow c_2 (or even c_0 again) to deduce further reductions.

The core of any CP solver is its *propagation engine*, which orchestrates this chain reaction until no constraint can deduce any further domain reductions. Mathematically, this is a *fixpoint algorithm*, summarized in Algorithm 21.

Let us now examine the Java implementation of each component of this architecture: `Domain`, `Variable`, `Constraint`, `NotEqual`, and `TinyCSP`.

Algorithm 21: Fixpoint Propagation Engine (`fixPoint()`)

Input: A CSP $\langle X, \mathcal{D}, \mathcal{C} \rangle$ with current domain store $\mathcal{D}$
Output: `Success` with greatest fixpoint domains $\mathcal{D}$, or `Failure` if an inconsistency is detected

```
repeat
    foreach constraint c ∈ C do
        if c is infeasible given the domain store D then
            return Failure;
        Apply the pruning algorithm (c.propagate()) associated with c to D;
until no constraint c ∈ C removes any value from the domain of its variables;
return Success;
```

9.5.3 Domain Representation with `java.util.BitSet`

A finite domain $D(x) \subseteq \{0, 1, \ldots, n-1\}$ must support fast queries for its cardinality ($|D(x)|$), its minimum value ($\min D(x)$), and its maximum value ($\max D(x)$), as well as fast value removal ($D(x) \leftarrow D(x) \setminus \{v\}$), value fixing ($D(x) \leftarrow \{v\}$), and cloning for state restoration.

In `TinyCSP`, the `Domain` class (Listing 9.4) represents $D(x)$ using a `java.util.BitSet`, where bit $v \in \{0, \ldots, n-1\}$ is set to 1 (`true`) if and only if $v \in D(x)$:

- `size()` delegates to `values.cardinality()`, which counts the number of set bits using hardware population-count instructions.
- `isFixed()` checks whether `size()` $== 1$.
- `min()` and `max()` invoke `nextSetBit(0)` and `previousSetBit(L - 1)` on `values` (where $L =$ `values.length()`) to locate the lowest and highest set bits in $\mathcal{O}(n/64)$ word operations.
- `remove(int v)` clears bit v if present; if the domain becomes empty (`size()` $== 0$), it immediately throws the unchecked exception `TinyCSP.Inconsistency` to unwind the current propagation and trigger backtracking.
- `fix(int v)` verifies that $v \in D(x)$ (throwing `TinyCSP.Inconsistency` otherwise), clears all other bits, and sets bit v.

9.5.4 Variables and the `NotEqual` Propagator

A `Variable` is a lightweight wrapper around a `Domain` instance and a reference to its parent `TinyCSP` solver, while `Constraint` is an abstract base class declaring a single method `boolean propagate()`, which returns `true` if and only if at least one value was removed from the domain of any variable in its scope (Listing 9.5).

Consider the binary disequality constraint with constant offset `offset` $\in \mathbb{Z}$:

$$x \neq y + \texttt{offset}. \tag{9.11}$$

When can (9.11) prune a value from $D(x)$ or $D(y)$? As long as both $D(x)$ and $D(y)$ contain two or more values, no value in $D(y)$ is ruled out by x alone, because for any $v_y \in D(y)$ there remains at least one supporting value $v_x \in D(x)$ with $v_x \neq v_y + \texttt{offset}$. However, as soon as one of the two variables becomes **fixed** (i.e., its domain is reduced to a singleton), the forbidden value for the other variable is uniquely determined:

```java
import java.util.BitSet;
import java.util.Iterator;

public class Domain implements Iterable<Integer> {
    private BitSet values;

    public Domain(int n) {
        values = new BitSet(n);
        values.set(0, n);
    }

    private Domain(BitSet dom) {
        this.values = dom;
    }

    public boolean isFixed() { return size() == 1; }
    public int size() { return values.cardinality(); }
    public int min() { return values.nextSetBit(0); }

    public int max() {
        return values.previousSetBit(values.length() - 1);
    }

    public int value() {
        if (isFixed()) return values.nextSetBit(0);
        throw new TinyCSP.Inconsistency();
    }

    public boolean remove(int v) {
        if (0 <= v && v < values.length() && values.get(v)) {
            values.clear(v);
            if (size() == 0)
                throw new TinyCSP.Inconsistency();
            return true;
        }
        return false;
    }

    public boolean fix(int v) {
        if (!values.get(v))
            throw new TinyCSP.Inconsistency();
        boolean wasFixed = isFixed();
        values.clear();
        values.set(v);
        return !wasFixed;
    }

    @Override
    public Domain clone() {
        return new Domain((BitSet) values.clone());
    }

    @Override
    public Iterator<Integer> iterator() {
        return values.stream().iterator();
    }
}
```

Listing 9.4: BitSet-based finite domain implementation (`Domain.java`).

- If $D(x) = \{v_x\}$ (so `x.dom.isFixed()` holds with $v_x =$ `x.dom.min()`), then y cannot take the value $v_x -$ `offset`, and we invoke `y.dom.remove(x.dom.min() - offset)`.
- Symmetrically, if $D(y) = \{v_y\}$, then x cannot take the value $v_y +$ `offset`, and we invoke `x.dom.remove(y.dom.min() + offset)`.

Notice what happens if both x and y are fixed to conflicting values $v_x = v_y +$ `offset`: the call `y.dom.remove`(v_y) removes the sole remaining element from $D(y)$, causing `y.dom.size()` $==$ 0 and throwing `TinyCSP.Inconsistency` automatically!

9.5.5 State Management and Search in `TinyCSP`

During tree search, both branching decisions and `fixPoint()` propagation destructively remove values from variable domains. When a branch fails (throwing `Inconsistency`) or finishes exploring its subtree, the solver must **restore** all domains to the exact state they held before the branching decision was made.

`TinyCSP` implements state restoration via a clean *copy-on-branch* backup mechanism, illustrated step-by-step in Figure 9.4:

1. **Clone Domains**: Before branching at a parent search node N_p, `backupDomains()` iterates over all variables in the CSP and stores a cloned copy `x.dom.clone()` of each variable's `BitSet` domain in an `ArrayList<Domain> backup`.
2. **Left Branch ($y = v$) and Fixpoint**: The search selects the first unfixed variable y and its smallest available value $v = \min D(y)$. Inside a `try` block, it fixes `y.dom.fix(v)`, calls `fixPoint()` to propagate the consequences across all constraints at child node N_L, and recursively calls `dfs(closure)`.
3. **Restore Domains on Backtrack**: Whether the left subtree returns normally or aborts early by throwing `Inconsistency`, `restoreDomains(backup)` restores each variable's domain reference `variables.get(i).dom = backup.get(i)` at N_p.
4. **Right Branch ($y \neq v$) and Fixpoint**: After taking a fresh backup, the search explores the complementary decision `y.dom.remove(v)` at child node N_R, runs `fixPoint()`, and recurses on `dfs(closure)`, restoring the domains once more upon completion.

Listings 9.6 and 9.7 present the complete implementation of `TinyCSP`, organized into its constraint store and propagation methods (Listing 9.6) and its state-restoration and depth-first search methods (Listing 9.7).

Binary Branching vs. d-Way Branching. Notice a subtle difference between the branching structure of `NQueensPrune` (Listing 9.2) and `TinyCSP.dfs` (Listing 9.7). Whereas `NQueensPrune` uses a `for` loop to create an n-way branch ($q_{\texttt{idx}} = 0$, $q_{\texttt{idx}} = 1$, …, $q_{\texttt{idx}} = n - 1$), `TinyCSP` performs *binary branching*: the left branch imposes $y = v$ and the right branch imposes $y \neq v$ (`y.dom.remove(v)`). Binary branching is strictly more powerful when combined with constraint propagation: removing v from $D(y)$ on the right branch immediately triggers `fixPoint()`, which may prune additional values from $D(y)$ or other variables before the next value of y is even tried! Indeed, if $D(y)$ had only two values $\{v, w\}$ left, removing v on the right branch automatically fixes $y = w$ during `fixPoint()` without requiring another branching level.

9.6 Bound-Consistent Propagators

To solve problems beyond n-Queens—such as Magic Squares, Killer Sudoku, and the Knight's Tour—we must enrich `TinyCSP` with additional domain operations and constraint propagators.

In Constraint Programming, filtering algorithms are classified by the level of *local consistency* they enforce [Mac77; RVW06]:

```java
public class Variable {
    public Domain dom;
    private final TinyCSP csp;

    public Variable(TinyCSP csp, int n) {
        this.csp = csp;
        this.dom = new Domain(n);
    }

    public TinyCSP getCsp() { return csp; }
}

public abstract class Constraint {
    /**
     * Propagates the constraint and returns true
     * if at least one domain value was removed.
     */
    abstract boolean propagate();
}

public class NotEqual extends Constraint {
    Variable x, y;
    int offset;

    public NotEqual(Variable x, Variable y, int offset) {
        this.x = x;
        this.y = y;
        this.offset = offset;
    }

    public NotEqual(Variable x, Variable y) {
        this(x, y, 0);
    }

    @Override
    boolean propagate() {
        if (x.dom.isFixed()) {
            return y.dom.remove(x.dom.min() - offset);
        }
        if (y.dom.isFixed()) {
            return x.dom.remove(y.dom.min() + offset);
        }
        return false;
    }
}
```

Listing 9.5: The `Variable`, `Constraint`, and `NotEqual` classes.

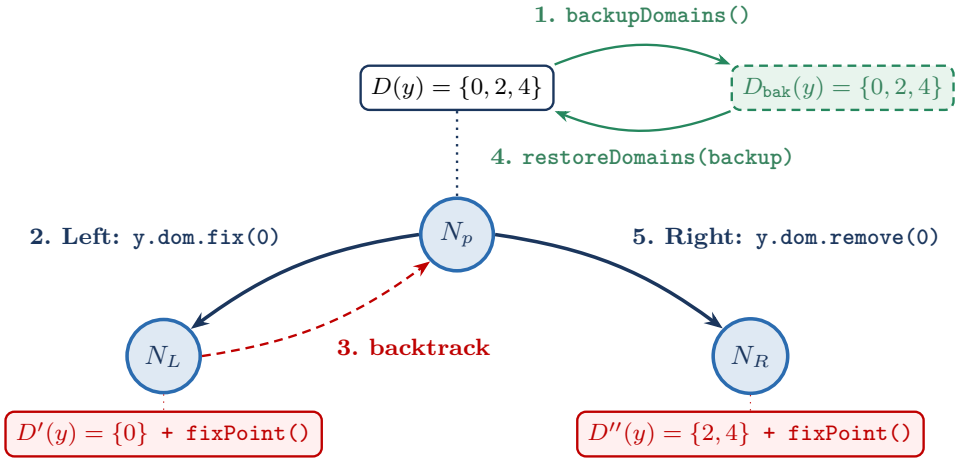


Figure 9.4: Binary branching and copy-on-branch state restoration during depth-first search in `TinyCSP`. (1) At parent node N_p, the active domain $D(y) = \{0, 2, 4\}$ is cloned into a backup list $D_{\text{bak}}(y)$ via `backupDomains()`. (2) On the left branch to child node N_L, `y.dom.fix(0)` fixes $y = 0$ and triggers `fixPoint()`. (3–4) Upon backtracking from N_L to N_p, `restoreDomains(backup)` restores $D(y)$ from $D_{\text{bak}}(y)$. (5) On the right branch to child node N_R, `y.dom.remove(0)` removes 0 from $D(y)$ and triggers `fixPoint()`.

```
public class TinyCSP {
    public int nRecur = 0;
    List<Constraint> constraints = new LinkedList<>();
    List<Variable> variables = new LinkedList<>();

    public Variable makeVariable(int domSize) {
        Variable x = new Variable(this, domSize);
        variables.add(x);
        return x;
    }

    public void add(Constraint c) {
        constraints.add(c);
        fixPoint();
    }

    public void notEqual(Variable x, Variable y,
                         int offset) {
        add(new NotEqual(x, y, offset));
    }

    public void fixPoint() {
        boolean fix = false;
        while (!fix) {
            fix = true;
            for (Constraint c : constraints) {
                fix &= !c.propagate();
            }
        }
    }

    public static class Inconsistency
            extends RuntimeException {}
```

Listing 9.6: Constraint store and fixpoint propagation in `TinyCSP` (`TinyCSP.java`, Part 1).

```
    private ArrayList<Domain> backupDomains() {
        ArrayList<Domain> backup = new ArrayList<>();
        for (Variable x : variables) {
            backup.add(x.dom.clone());
        }
        return backup;
    }

    private void restoreDomains(
            ArrayList<Domain> backup) {
        for (int i = 0; i < variables.size(); i++) {
            variables.get(i).dom = backup.get(i);
        }
    }

    Optional<Variable> firstNotFixed() {
        return variables.stream()
            .filter(x -> !x.dom.isFixed()).findFirst();
    }

    public void dfs(Runnable closure) {
        nRecur += 1;
        Optional<Variable> notFixed = firstNotFixed();
        if (!notFixed.isPresent()) {
            closure.run();
        } else {
            Variable y = notFixed.get();
            int v = y.dom.min();
            ArrayList<Domain> backup = backupDomains();
            // Left branch: y = v
            try {
                y.dom.fix(v);
                fixPoint();
                dfs(closure);
            } catch (Inconsistency ignored) {}
            restoreDomains(backup);
            backup = backupDomains();
            // Right branch: y != v
            try {
                y.dom.remove(v);
                fixPoint();
                dfs(closure);
            } catch (Inconsistency ignored) {}
            restoreDomains(backup);
        }
    }
}
```

Listing 9.7: Domain backup, restoration, and binary branching DFS in `TinyCSP` (`TinyCSP.java`, Part 2).

Definition 9.4 *Domain Consistency vs. Bound Consistency*

Let c be a constraint with scope $\text{scope}(c) = (x_0, \ldots, x_{k-1})$ and relation $\text{sol}(c)$.

1. **Domain Consistency** (also called *Generalized Arc Consistency*, GAC): A constraint c is domain-consistent with respect to domains $\mathcal{D}$ if and only if for every variable $x_i \in \text{scope}(c)$ and every value $v_i \in D(x_i)$, there exists a valid tuple $(v_0, \ldots, v_i, \ldots, v_{k-1}) \in \text{sol}(c)$ (called a *support*) such that $v_j \in D(x_j)$ for all $j \in \{0, \ldots, k-1\}$.
2. **Bound($\mathbb{Z}$) Consistency**: For arithmetic constraints over integer intervals, enforcing full domain consistency can be computationally expensive (or even **NP**-hard, as for linear equality $\sum_i x_i = y$ which generalizes Subset-Sum). A constraint c is *bound-consistent* if for every variable $x_i \in \text{scope}(c)$, both its lower bound $\underline{x}_i = \min D(x_i)$ and its upper bound $\overline{x}_i = \max D(x_i)$ belong to a valid tuple $(v_0, \ldots, v_{k-1}) \in \text{sol}(c)$ satisfying $v_j \in [\underline{x}_j, \overline{x}_j] \cap \mathbb{Z}$ for all $j \neq i$.

9.6.1 Bound Pruning Operations on `Domain`

To support bound-consistent propagators, we add two range-removal methods to `Domain` (Listing 9.8):

- `removeBelow(int v)`: Removes all values strictly less than v (i.e., clears all bits in $[0, \min(v, L))$ where $L =$ `values.length()`), enforcing the lower bound $x \geq v$.
- `removeAbove(int v)`: Removes all values strictly greater than v (i.e., clears all bits in $[\max(v+1, 0), L)$), enforcing the upper bound $x \leq v$.

Both methods throw `TinyCSP.Inconsistency` if the domain becomes empty and return `true` if and only if the domain size strictly decreased.

```
1  public boolean removeBelow(int v) {
2      int oldSize = size();
3      int maxIndex = Math.min(v, values.length());
4      if (v <= 0) return false;
5      values.clear(0, maxIndex);
6      int newSize = size();
7      if (newSize == 0) throw new TinyCSP.Inconsistency();
8      return oldSize != newSize;
9  }
10
11 public boolean removeAbove(int v) {
12     int oldSize = size();
13     if (v >= values.length()) return false;
14     int minIndex = Math.max(v + 1, 0);
15     values.clear(minIndex, values.length());
16     int newSize = size();
17     if (newSize == 0) throw new TinyCSP.Inconsistency();
18     return oldSize != newSize;
19 }
```

Listing 9.8: Bound-pruning operations on `Domain` (`Domain.java`).

9.6.2 The `LessOrEqual` Constraint ($x \leq y$)

Consider the binary inequality constraint $x \leq y$. A value $v_x \in D(x)$ has a valid support $v_y \in D(y)$ with $v_x \leq v_y$ if and only if $v_x \leq \max D(y)$. Symmetrically, a value $v_y \in D(y)$ has a valid support $v_x \in D(x)$ with $v_x \leq v_y$ if and only if $v_y \geq \min D(x)$.

Thus, the bound-consistent filtering rules for $x \leq y$ are simply:

$$D(x) \leftarrow \{v \in D(x) \mid v \leq \max D(y)\} \qquad \text{and} \qquad D(y) \leftarrow \{v \in D(y) \mid v \geq \min D(x)\}. \tag{9.12}$$

Notice that for $x \leq y$, bound pruning (9.12) actually achieves full *domain consistency* without ever inspecting interior holes in $D(x)$ or $D(y)$! Listing 9.9 gives the implementation of `LessOrEqual`.

```
1  public class LessOrEqual extends Constraint {
2      Variable x, y;
3
4      public LessOrEqual(Variable x, Variable y) {
5          this.x = x;
6          this.y = y;
7      }
8
9      @Override
10     boolean propagate() {
11         boolean changed = x.dom.removeAbove(y.dom.max());
12         changed = y.dom.removeBelow(x.dom.min()) || changed;
13         return changed;
14     }
15 }
```

Listing 9.9: Propagator for the inequality constraint $x \leq y$ (`LessOrEqual.java`).

Avoiding Short-Circuit Evaluation Pitfalls in Propagators. Look closely at line 12 of Listing 9.9:

```
changed = y.dom.removeBelow(x.dom.min()) || changed;
```

A frequent mistake among students is to write `changed || y.dom.removeBelow(x.dom.min())` on the right-hand side. In Java, the logical OR operator `||` is *short-circuiting*: when `changed` is already `true` (because `removeAbove` pruned a value from x), Java will **skip evaluating the right-hand side** altogether! Placing the method call `y.dom.removeBelow(...)` on the *left* of `||` (or using the non-short-circuiting bitwise OR assignment `changed |= ...`) guarantees that both pruning operations are always executed.

9.6.3 The Bound-Consistent `Sum` Constraint

Consider the linear equality constraint linking an array of variables $\mathbf{x} = (x_0, \ldots, x_{k-1})$ to a target sum variable y:

$$\sum_{i=0}^{k-1} x_i = y. \tag{9.13}$$

Let $\underline{x}_i = \min D(x_i)$ and $\overline{x}_i = \max D(x_i)$ denote the current lower and upper bounds of each term x_i, and define the aggregate minimum and maximum sums:

$$S_{\min} = \sum_{i=0}^{k-1} \underline{x}_i \qquad \text{and} \qquad S_{\max} = \sum_{i=0}^{k-1} \overline{x}_i. \tag{9.14}$$

First, since every x_i lies in $[\underline{x}_i, \overline{x}_i]$, their sum y must lie in $[S_{\min}, S_{\max}]$:

$$\min D(y) \geq S_{\min} \qquad \text{and} \qquad \max D(y) \leq S_{\max}. \tag{9.15}$$

Second, rewrite (9.13) to isolate an individual variable x_i:

$$x_i = y - \sum_{j \neq i} x_j. \tag{9.16}$$

To find the largest value that x_i can possibly take, we maximize y ($\overline{y} = \max D(y)$) and minimize every other term x_j ($\underline{x}_j$ for $j \neq i$). Observing that $\sum_{j \neq i} \underline{x}_j = S_{\min} - \underline{x}_i$, we obtain the upper bound on x_i:

$$x_i \leq \overline{y} - \sum_{j \neq i} \underline{x}_j = \overline{y} - (S_{\min} - \underline{x}_i). \tag{9.17}$$

Symmetrically, to find the smallest value that x_i can take, we minimize y ($\underline{y} = \min D(y)$) and maximize every other term x_j ($\overline{x}_j$ for $j \neq i$), where $\sum_{j \neq i} \overline{x}_j = S_{\max} - \overline{x}_i$:

$$x_i \geq \underline{y} - \sum_{j \neq i} \overline{x}_j = \underline{y} - (S_{\max} - \overline{x}_i). \tag{9.18}$$

By precomputing $S_{\min}$ and $S_{\max}$ in a single pass, all k variables $x_0, \ldots, x_{k-1}$ are updated in $\mathcal{O}(k)$ bound operations per propagation call (Listing 9.10).

9.7 Performance and Data-Driven Propagation

Let us now evaluate the empirical performance of our three n-Queens solvers:

1. `NQueensChecker` (DFS + Filter at the leaves),
2. `NQueensPrune` (DFS + Prefix Pruning), and
3. `NQueensTinyCSP` (Generic `TinyCSP` solver with binary `NotEqual` propagation).

To compare backtracking algorithms rigorously, we measure two complementary metrics:

- **Number of Search Nodes**: The total number of recursive invocations of `dfs(...)`, which reflects the mathematical strength of the pruning/filtering independently of implementation overhead.
- **Wall-Clock Execution Time (ms)**: The total CPU time required to enumerate all solutions, which captures both tree size and the per-node computational cost.

Table 9.1 reports the benchmark results across $n \in \{8, 9, 10\}$ for `NQueensChecker` and $n \in \{12, 13, 14, 15\}$ for `NQueensPrune` and `NQueensTinyCSP`.

Analyzing Table 9.1 reveals two striking lessons:

1. **Pruning vs. Filtering**: `NQueensChecker` already requires 1.11×10^{10} nodes and over 101 seconds at $n = 10$ (matching the geometric series $\sum_{k=0}^{10} 10^k = 11{,}111{,}111{,}111$). By pruning inconsistent prefixes early, both `NQueensPrune` and `NQueensTinyCSP` effortlessly scale to $n = 15$, where there are over 2.27 million valid solutions.
2. **Fewer Search Nodes, Yet Slower Wall-Clock Time in `TinyCSP`**: Compare `NQueensPrune` and `NQueensTinyCSP` at $n = 15$. Thanks to active forward domain propagation, `NQueensTinyCSP` explores **nearly** $10\times$ **fewer search nodes** than `NQueensPrune` (17.5 million vs. 171.1 million nodes). Yet `NQueensTinyCSP` takes roughly $15.8\times$ **more wall-clock time** (477.3 seconds vs. 30.1 seconds)! Why is each node in `TinyCSP` over $150\times$ more expensive than in `NQueensPrune`?

```java
public class Sum extends Constraint {
    private final Variable[] x;
    private final Variable y;
    int[] min;
    int[] max;

    public Sum(Variable[] x, Variable y) {
        this.x = x;
        this.y = y;
        this.min = new int[x.length];
        this.max = new int[x.length];
    }

    public Sum(Variable[] x, int y) {
        this(x, x[0].getCsp().makeVariable(y + 1));
        this.y.dom.fix(y);
    }

    @Override
    boolean propagate() {
        int minSum = 0;
        int maxSum = 0;
        boolean changed;
        for (int i = 0; i < x.length; i++) {
            min[i] = x[i].dom.min();
            max[i] = x[i].dom.max();
            minSum += min[i];
            maxSum += max[i];
        }
        // 1. Update bounds of y based on x
        if (minSum == maxSum) {
            changed = y.dom.fix(minSum);
        } else {
            changed = y.dom.removeBelow(minSum);
            changed = y.dom.removeAbove(maxSum) || changed;
        }
        // 2. Update bounds of each x[i] based on y and x
        for (int i = 0; i < x.length; i++) {
            if (!x[i].dom.isFixed()) {
                int newMax = y.dom.max() - (minSum - min[i]);
                int newMin = y.dom.min() - (maxSum - max[i]);
                changed = x[i].dom.removeAbove(newMax)
                        || changed;
                changed = x[i].dom.removeBelow(newMin)
                        || changed;
            }
        }
        return changed;
    }
}
```

Listing 9.10: Bound-consistent propagator for $\sum_{i=0}^{k-1} x_i = y$ (`Sum.java`).

Table 9.1: Empirical performance comparison on the n-Queens benchmark across the three approaches: `NQueensChecker` (Generate-and-Filter), `NQueensPrune` (DFS + Prefix Pruning), and `NQueensTinyCSP` (Generic `TinyCSP` solver).

		`NQueensChecker`		`NQueensPrune`		`NQueensTinyCSP`	
n	# Solutions	Nodes	Time (ms)	Nodes	Time (ms)	Nodes	Time (ms)
8	92	19,173,961	167	—	—	—	—
9	352	435,848,050	4,526	—	—	—	—
10	724	11,111,111,111	101,497	—	—	—	—
12	14,200	—	—	856,189	130	102,531	2,439
13	73,712	—	—	4,674,890	690	546,529	11,999
14	365,596	—	—	27,358,553	4,550	2,934,559	72,753
15	2,279,184	—	—	171,129,072	30,138	17,543,706	477,324

9.7.1 Profiling `TinyCSP`: Where Does the Time Go?

When a solver runs slower than expected, a computer scientist does not guess—they run a CPU and memory profiler (such as VisualVM). Profiling `NQueensTinyCSP` exposes three primary sources of overhead:

1. **The Naive Fixpoint Loop (`fixPoint()` and `NotEqual.propagate()`)**: In the n-Queens model, there are $3\binom{n}{2} = \frac{3n(n-1)}{2}$ binary `NotEqual` constraints (315 constraints for $n = 15$). Every time `fixPoint()` runs, its `for (Constraint c : constraints)` loop blindly invokes `c.propagate()` on *every single constraint in the model*, repeating the entire scan until a full pass produces zero changes.
2. **Exception Stack-Trace Construction (`Inconsistency.<init>`)**: In Java, throwing a standard `RuntimeException` walks the call stack to build a full stack trace (`fillInStackTrace()`). Because `Inconsistency` is thrown at every failed branch (millions of times during search), constructing stack traces consumes nearly 40% of the time spent inside `fixPoint()`! (In production Java CP solvers, failure exceptions either override `fillInStackTrace()` to be a no-op or reuse a single pre-allocated static singleton instance.)
3. **Heap Allocation and Garbage Collection in `backupDomains()`**: At every node of the search tree, `backupDomains()` allocates a new `ArrayList` and clones n `BitSet` objects on the heap, creating massive garbage collection pressure (manifesting as a classic "sawtooth" heap profile).

9.7.2 From Naive Fixpoint to Data-Driven Propagation

Let us examine the algorithmic inefficiency of the naive fixpoint loop more closely. Let $\mathcal{F}_c(\mathcal{D})$ denote the domain store obtained after applying the filtering algorithm of constraint c to $\mathcal{D}$, and let $\mathcal{F}_{\mathcal{C}}(\mathcal{D})$ denote the result of applying all constraints $c \in \mathcal{C}$. The naive `fixPoint()` procedure (Algorithm 21) computes the greatest fixpoint domain $\mathcal{D}^*$ by repeatedly evaluating $\mathcal{D}^p \leftarrow \mathcal{F}_{\mathcal{C}}(\mathcal{D})$ over *all* $c \in \mathcal{C}$ until $\mathcal{D}^p = \mathcal{D}$.

Now ask a simple question: **if no domain of any variable in the scope of constraint c has changed since the last time c was executed, is it worth executing c again?**

Clearly not! Because constraint propagators are deterministic and idempotent (or contracting), re-running `c.propagate()` when $\mathcal{D}|_{\text{scope}(c)}$ is unchanged is guaranteed to do nothing.

This observation leads directly to the *data-driven (event-driven) fixpoint algorithm* used by modern CP solvers, formalized in Algorithm 22 [Mac77; RVW06; MSV21]. Instead of scanning

all constraints in $\mathcal{C}$, the solver maintains a propagation queue $Q \subseteq \mathcal{C}$ of active constraints. When a constraint c is dequeued and executed via $\mathcal{D}' \leftarrow \mathcal{F}_c(\mathcal{D})$, we identify the subset of variables $V \subseteq \text{scope}(c)$ whose domains were actually modified:

$$V = \{x \in \text{scope}(c) \mid D'(x) \neq D(x)\}. \tag{9.19}$$

Only the constraints $c' \in \mathcal{C}$ whose scope intersects V ($\text{scope}(c') \cap V \neq \emptyset$) are enqueued into Q.

Algorithm 22: Data-Driven Fixpoint Propagation Algorithm

Input: A CSP $\langle X, \mathcal{D}^0, \mathcal{C} \rangle$ with initial domains $\mathcal{D}^0$
Output: The greatest fixpoint domain store $\mathcal{D}$ of the filtering algorithms for $\mathcal{C}$

$Q \leftarrow \mathcal{C}$; `// Initialize queue with all constraints (or recently posted ones)`
$\mathcal{D} \leftarrow \mathcal{D}^0$;
while $|Q| > 0$ **do**
  $c \leftarrow \text{dequeue}(Q)$;
  $\mathcal{D}' \leftarrow \mathcal{F}_c(\mathcal{D})$; `// Execute propagator of` c `(aborts if any` $D'(x) = \emptyset$`)`
  $V \leftarrow \{x \in \text{scope}(c) \mid D'(x) \neq D(x)\}$; `// Variables modified by` c
  if $|V| > 0$ **then**
    $\text{enqueue}(Q, \{c' \in \mathcal{C} \mid |\text{scope}(c') \cap V| > 0\})$;
  $\mathcal{D} \leftarrow \mathcal{D}'$;
return $\mathcal{D}$;

The Road to Industrial-Strength CP Solvers (`MiniCP`). Transforming `TinyCSP` into a high-performance constraint programming solver such as `MiniCP` [MSV21] rests on three architectural pillars:

1. **Fine-Grained Event-Driven Propagation**: Replacing the global scan in `fixPoint()` with variable-level observer lists that wake up subscribed constraints on specific domain events (*value removal*, *bound change*, or *variable fixing*).
2. **Reversible Trail-Based State Restoration**: Eliminating object cloning (`backupDomains()`) entirely by using a *trail* (undo stack) and *sparse-set* domain data structures that restore domains in $\mathcal{O}(1)$ time upon backtracking without allocating a single object on the heap.
3. **First-Class Branching Schemes and Global Constraints**: Supporting dynamic variable-ordering heuristics (such as *first-fail*, which branches on the variable with the smallest domain) and specialized polynomial-time filtering algorithms for global constraints (AllDifferent, Element, Circuit, Cumulative, and Table).

9.8 Modeling Expressiveness: CP vs. MIP on the TSP

To appreciate the modeling expressiveness of Constraint Programming, let us compare how Mixed-Integer Linear Programming (MIP, studied in Chapter 5) and Constraint Programming formulate the Asymmetric Traveling Salesman Problem (TSP) on a directed graph $G = (V, E)$ with n vertices $V = \{0, 1, \ldots, n-1\}$ and pairwise travel distances $d_{i,j}$ (Figure 9.5a).

9.8.1 The Mixed-Integer Programming Formulation

Because linear programming requires all constraints and the objective function to be linear combinations of decision variables, the standard Dantzig–Fulkerson–Johnson MIP formulation [DFJ54] must introduce n^2 binary indicator variables $x_{i,j} \in \{0, 1\}$ for every pair

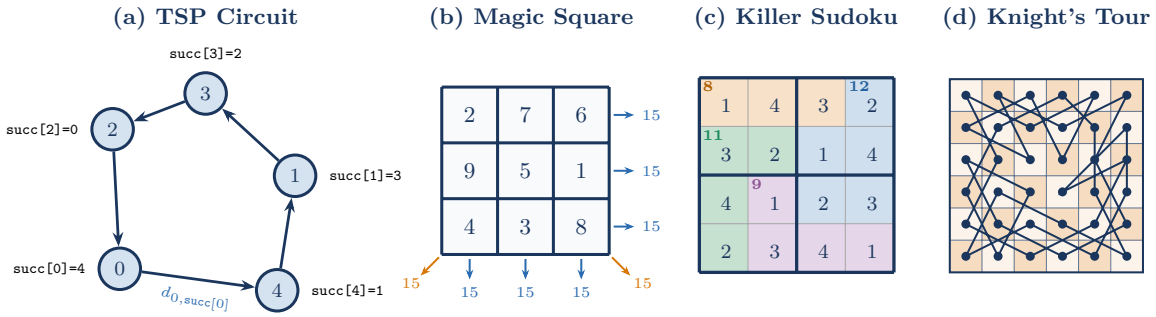


Figure 9.5: Expressive CP modeling and classic combinatorial applications. (a) In the CP formulation of the 5-city TSP, each city $i \in \{0, \dots, 4\}$ has a single successor variable $\mathtt{succ}[i]$; the global constraint Circuit($\mathtt{succ}$) enforces a single Hamiltonian tour, while variable-indexed array lookup $d_{i,\mathtt{succ}[i]}$ (Element) computes the tour length directly. (b) A 3×3 Magic Square where all rows, columns, and diagonals sum to $M_3 = 15$. (c) A 4×4 Killer Sudoku combining row, column, and 2×2 block disequalities with colored cage sum constraints. (d) A closed 6×6 Knight's Tour visiting all 36 squares in a single cycle.

$(i, j) \in V^2$, alongside exponentially many subtour elimination constraints:

$$\begin{aligned} \text{minimize} \quad & \sum_{i \in V} \sum_{j \in V} d_{i,j} x_{i,j} \\ \text{subject to} \quad & \sum_{j \in V \setminus \{i\}} x_{i,j} = 1, \quad \sum_{j \in V \setminus \{i\}} x_{j,i} = 1, \quad \forall i \in V, \\ & \sum_{i \in S} \sum_{j \in S \setminus \{i\}} x_{i,j} \leq |S| - 1, \qquad \forall S \subset V,\ \emptyset \neq S \neq V, \\ & x_{i,j} \in \{0, 1\}, \qquad \forall i, j \in V. \end{aligned} \tag{9.20}$$

9.8.2 The Constraint Programming Formulation

In Constraint Programming, we are not restricted to binary indicator variables or linear expressions. Instead, we introduce only n integer decision variables $\mathtt{succ}[0], \dots, \mathtt{succ}[n-1]$, where $\mathtt{succ}[i]$ denotes the immediate successor of city i along the tour (Figure 9.5a):

$$\begin{aligned} \text{minimize} \quad & \sum_{i \in V} d_{i,\mathtt{succ}[i]} \\ \text{subject to} \quad & \textsc{Circuit}(\mathtt{succ}), \\ & \mathtt{succ}[i] \in \{0, \dots, i-1, i+1, \dots, n-1\}, \quad \forall i \in V. \end{aligned} \tag{9.21}$$

Look closely at the two remarkable features of the CP formulation (9.21):

1. **Indexing an Array with a Decision Variable (Element)**: In the objective function $\sum_{i \in V} d_{i,\mathtt{succ}[i]}$, the constant distance matrix row $d_{i,\cdot}$ is indexed directly by the *decision variable* $\mathtt{succ}[i]$! In CP, expressions of the form $z = C[y]$—where $C = [c_0, \dots, c_{n-1}]$ is an array of constants (or variables) and y is an integer decision variable—are supported natively by the Element constraint. Its domain-consistent propagator maintains the bidirectional relation:

$$D(z) \leftarrow D(z) \cap \{C[v] \mid v \in D(y)\} \qquad \text{and} \qquad D(y) \leftarrow \{v \in D(y) \mid C[v] \in D(z)\}. \tag{9.22}$$

2. **The Global Circuit Constraint**: Instead of $2^n - 2$ linear subtour elimination inequalities, the single global constraint Circuit(`succ`) enforces both that all successors are pairwise distinct (AllDifferent(`succ`)) and that the directed edges $(i, \texttt{succ}[i])$ form a single Hamiltonian cycle of length n without premature subtours.

9.9 Combinatorial Applications in `TinyCSP`

We now demonstrate how the building blocks of `TinyCSP`—`Variable`, `Domain`, `NotEqual`, `LessOrEqual`, and `Sum`—combine to solve three classic combinatorial puzzles depicted in Figure 9.5b–d: the Magic Square, Killer Sudoku, and the Knight's Tour.

9.9.1 Magic Square and Symmetry Breaking

A *magic square* of order n is an $n \times n$ grid filled with the distinct integers $\{1, 2, \ldots, n^2\}$ such that every row, every column, and both main diagonals sum to the same value (Figure 9.5b). Some cells (i, j) may be pre-assigned to given initial values by the problem instance.

Deriving the Magic Constant

Although the problem definition only requires that all rows, columns, and diagonals sum to the *same* number M_n, we can deduce the exact value of M_n a priori (*implied knowledge*). Because every integer in $\{1, 2, \ldots, n^2\}$ appears exactly once in the grid, the sum of all n^2 cells is:

$$\sum_{k=1}^{n^2} k = \frac{n^2(n^2+1)}{2}. \tag{9.23}$$

Since the grid is partitioned into n rows of equal sum M_n, dividing the total sum by n yields the *magic constant*:

$$M_n = \frac{n(n^2+1)}{2}. \tag{9.24}$$

For $n = 3$, $M_3 = 3(10)/2 = 15$; for $n = 4$, $M_4 = 4(17)/2 = 34$. Fixing the target sum variable to M_n before search starts drastically strengthens the initial bound propagation of the `Sum` constraints!

Declarative Model in `TinyCSP`

We create an $n \times n$ matrix of variables `val[i][j]`, each initialized with domain $\{0, \ldots, n^2\}$ from which 0 is removed via `val[i][j].dom.remove(0)`, leaving $D(\texttt{val}[i][j]) = \{1, \ldots, n^2\}$. Any pre-filled clue at (i, j) is fixed via `val[i][j].dom.fix(instance.value(i, j))`. We then post:

1. $\binom{n^2}{2}$ pairwise `notEqual` constraints across all n^2 variables.
2. $2n + 2$ `sum` constraints equating each of the n rows, n columns, and 2 diagonals to M_n.

Listing 9.11 shows the complete `MagicSquareSolver` implementation.

Symmetry Breaking in Magic Squares. An empty $n \times n$ magic square exhibits the eight-fold geometric symmetry of the dihedral group D_4 (four rotations by $0°, 90°, 180°, 270°$ and four reflections across horizontal, vertical, and diagonal axes). Consequently, every essentially distinct magic square appears 8 times in the search tree! In Constraint Programming, we can eliminate symmetric duplicates by posting *symmetry-breaking constraints* using `LessOrEqual` among the four corner cells:

$$\begin{aligned} \texttt{val}[0][0] < \texttt{val}[0][n-1], \quad & \texttt{val}[0][0] < \texttt{val}[n-1][0], \\ \texttt{val}[0][0] < \texttt{val}[n-1][n-1], \quad & \texttt{val}[0][n-1] < \texttt{val}[n-1][0]. \end{aligned} \tag{9.25}$$

```java
public List<MagicSquareInstance.Solution> solve() {
    List<MagicSquareInstance.Solution> listSol =
            new ArrayList<>();
    int n = instance.n(), maxVal = n * n;
    Variable[][] val = new Variable[n][n];
    Variable sum = csp.makeVariable(getMagicConstant() + 1);
    sum.dom.fix(getMagicConstant());
    Variable[] allVars = new Variable[maxVal];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            val[i][j] = csp.makeVariable(maxVal + 1);
            val[i][j].dom.remove(0);
            if (instance.isValue(i, j))
                val[i][j].dom.fix(instance.value(i, j));
            allVars[i * n + j] = val[i][j];
        }
    }
    // 1. All-different via pairwise NotEqual
    for (int i = 0; i < maxVal; i++)
        for (int j = i + 1; j < maxVal; j++)
            csp.notEqual(allVars[i], allVars[j]);
    // 2. Row, column, and diagonal Sum constraints
    Variable[] diag1 = new Variable[n];
    Variable[] diag2 = new Variable[n];
    for (int i = 0; i < n; i++) {
        csp.sum(val[i], sum); // row i
        Variable[] col = new Variable[n];
        for (int j = 0; j < n; j++) col[j] = val[j][i];
        csp.sum(col, sum);    // column i
        diag1[i] = val[i][i];
        diag2[i] = val[n - 1 - i][i];
    }
    csp.sum(diag1, sum);
    csp.sum(diag2, sum);
    csp.dfs(() -> {
        int[][] sol = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                sol[i][j] = val[i][j].dom.value();
        listSol.add(new MagicSquareInstance.Solution(sol));
    });
    return listSol;
}

public int getMagicConstant() {
    int n = instance.n();
    return n * (n * n + 1) / 2;
}
```

Listing 9.11: Solving the Magic Square problem in `TinyCSP` (`MagicSquareSolver.java`).

The first three inequalities force the smallest of the four corners into the top-left cell $(0, 0)$ (breaking the 4 rotational symmetries), while the fourth inequality orders the two adjacent corners (breaking the diagonal reflection symmetry), shrinking the search space by a factor of 8.

9.9.2 The Killer Sudoku Problem

Killer Sudoku (Figure 9.5c) is a variation of Sudoku on an $n \times n$ grid (where $n = b^2$ is a perfect square, such as $n = 4$ or $n = 9$) partitioned into $b \times b$ sub-squares and a collection of arbitrary contiguous regions called *groups* (or *cages*). A valid solution assigns a digit in $\{1, \dots, n\}$ to every cell $(i, j) \in \{0, \dots, n-1\}^2$ such that:

1. Every row, every column, and every $b \times b$ sub-square contains the digits $\{1, \dots, n\}$ without repetition.
2. For every cage group $g = (S_g, C_g)$ with cell coordinates $C_g \subseteq \{0, \dots, n-1\}^2$ and target sum S_g, the digits in the cells of C_g sum to S_g:

$$\sum_{(i,j) \in C_g} \texttt{val}[i][j] = S_g. \tag{9.26}$$

Using `TinyCSP`, modeling Killer Sudoku is a direct composition of our existing constraints (Listing 9.12): pairwise `notEqual` constraints enforce the Latin-square and $b \times b$ block rules, while a `sum` constraint is posted for each group $g \in$ `instance.groups()`.

9.9.3 Knight's Tour and Custom Constraints

A *closed Knight's Tour* on an $n \times n$ chessboard is a sequence of n^2 legal knight moves that visits every square of the board exactly once and returns to the starting square (Figure 9.5d). Flattening the $n \times n$ board coordinates $(r, c) \in \{0, \dots, n-1\}^2$ into a single integer position $p = r \cdot n + c \in \{0, 1, \dots, n^2 - 1\}$ (with inverse mapping $r = \lfloor p/n \rfloor$ and $c = p \bmod n$), we model the tour using an array of n^2 variables $x_0, x_1, \dots, x_{n^2-1}$, where $x_t \in \{0, \dots, n^2 - 1\}$ denotes the board position occupied by the knight at time step $t \in \{0, \dots, n^2 - 1\}$.

How do we enforce that consecutive positions x_t and $x_{(t+1) \bmod n^2}$ are connected by a valid L-shaped knight jump $(\Delta r, \Delta c) \in \{(\pm 1, \pm 2), (\pm 2, \pm 1)\}$? This illustrates one of the greatest strengths of Constraint Programming: **whenever a problem features a custom structural relation, we can simply write a dedicated `Constraint` subclass!**

Given a constraint `KnightMove`(x, y, n), let $N_n(u) \subseteq \{0, \dots, n^2 - 1\}$ denote the set of board squares reachable from square u via a single legal knight move. For a value $v \in D(y)$ to be supported by some origin square $u \in D(x)$, v must belong to the union of reachable squares from $D(x)$:

$$D(y) \leftarrow D(y) \cap \bigcup_{u \in D(x)} N_n(u). \tag{9.27}$$

Listing 9.13 presents the implementation of `KnightMove` and the complete `KnightTourSolver`.

With `KnightMove` implemented, solving the closed Knight's Tour requires only posting pairwise `notEqual` constraints over $x_0, \dots, x_{n^2-1}$ and linking consecutive time steps via `new KnightMove(x[t], x[(t + 1) % (n * n)], n)`:

```
for (int i = 0; i < n * n; i++) {
    for (int j = i + 1; j < n * n; j++) {
        csp.notEqual(x[i], x[j], 0);
    }
    csp.add(new KnightMove(x[i], x[(i + 1) % (n * n)], n));
    if (instance.isMove(i)) x[i].dom.fix(instance.move(i));
}
```

```java
int n = instance.n();
Variable[][] val = new Variable[n][n];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        val[i][j] = csp.makeVariable(n + 1);
        val[i][j].dom.remove(0);
        if (instance.isValue(i, j)) {
            val[i][j].dom.fix(instance.value(i, j));
        }
    }
}
// 1. Pairwise NotEqual on rows and columns
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            csp.notEqual(val[k][i], val[k][j]); // row k
            csp.notEqual(val[i][k], val[j][k]); // col k
        }
    }
}
// 2. Pairwise NotEqual on sub-squares of size b x b
int b = (int) Math.sqrt(n);
for (int rOff = 0; rOff < n; rOff += b) {
    for (int cOff = 0; cOff < n; cOff += b) {
        Variable[] sq = new Variable[n];
        int cnt = 0;
        for (int i = rOff; i < rOff + b; i++)
            for (int j = cOff; j < cOff + b; j++)
                sq[cnt++] = val[i][j];
        for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++)
                csp.notEqual(sq[i], sq[j]);
    }
}
// 3. Cage Sum constraints on each KillerSudokuGroup
for (KillerSudokuGroup g : instance.groups()) {
    Variable[] gVars = new Variable[g.size()];
    int cnt = 0;
    for (Coordinate c : g.values()) {
        gVars[cnt++] = val[c.i()][c.j()];
    }
    csp.sum(gVars, g.sum());
}
```

Listing 9.12: Core constraint model for Killer Sudoku (`KillerSudokuSolver.java`).

```java
public class KnightMove extends Constraint {
    private final Variable x, y;
    private final int n;

    public KnightMove(Variable x, Variable y, int n) {
        this.x = x;
        this.y = y;
        this.n = n;
    }

    @Override
    boolean propagate() {
        boolean changed = false;
        HashSet<Integer> reachable = new HashSet<>();
        for (int u : x.dom) {
            reachable.addAll(accessible(u));
        }
        for (int v : y.dom) {
            if (!reachable.contains(v)) {
                y.dom.remove(v);
                changed = true;
            }
        }
        return changed;
    }

    Set<Integer> accessible(int u) {
        Set<Integer> res = new HashSet<>();
        int r = u / n, c = u % n;
        int[] dr = {1, 2, -1, -2, 1, 2, -1, -2};
        int[] dc = {2, 1, 2, 1, -2, -1, -2, -1};
        for (int k = 0; k < 8; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (0 <= nr && nr < n && 0 <= nc && nc < n) {
                res.add(nr * n + nc);
            }
        }
        return res;
    }
}
```

Listing 9.13: The `KnightMove` propagator and `KnightTourSolver`.

9.10 Exercises

Exercise 9.1 *6-Queens Problem and the First-Fail Branching Strategy*

In the 6-Queens problem on a 6×6 board, let $q_0, q_1, \ldots, q_5 \in \{0, 1, 2, 3, 4, 5\}$ denote the column position of the queen in each row $r \in \{0, \ldots, 5\}$. When selecting the next unfixed variable to branch on, the classic *first-fail strategy* selects the unfixed variable with the *smallest remaining domain size* $|D(q_r)|$ (breaking ties by choosing the smallest row index r).
Consider the three partial assignments shown below:

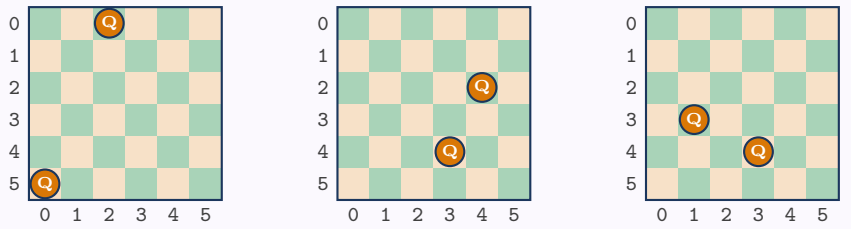
(a) $q_0 = 2$ and $q_5 = 0$,

(b) $q_2 = 4$ and $q_4 = 3$,

(c) $q_3 = 1$ and $q_4 = 3$.

For each partial assignment, determine the exact domains $D(q_r)$ of all unfixed variables after running `fixPoint()` with pairwise `NotEqual` constraints, and state which variable q_r will be chosen next by the first-fail strategy.

(a) $q_0 = 2,\ q_5 = 0$ **(b)** $q_2 = 4,\ q_4 = 3$ **(c)** $q_3 = 1,\ q_4 = 3$



Exercise 9.2 *The* Element *Constraint: Bound vs. Domain Consistency*

The Element constraint has the form $x = C[y]$, where $C = [c_0, c_1, \ldots, c_{k-1}]$ is a 0-indexed array of integer constants and x, y are integer decision variables. Suppose $D(x) = \{0, 1, 2, 3, 4, 5\}$, $D(y) = \{0, 1, 2, 3, 4, 5\}$, and the array is

$$C = [2, 4, 5, 1, 2, 5].$$

1. How do the domains $D(x)$ and $D(y)$ evolve after imposing the constraint $x = C[y]$ under full domain consistency (9.22)?
2. Suppose value 5 is subsequently removed from $D(x)$. What are the updated domains $D(x)$ and $D(y)$ after propagation?
3. For each of the four domain pairs $(D(x), D(y))$ listed in the table below (with respect to $x = C[y]$ and $C = [2, 4, 5, 1, 2, 5]$), determine whether *domain consistency* and/or *bound consistency* is achieved:

Case	$D(x)$	$D(y)$
1	$\{0, 1, 2, 3, 4, 5\}$	$\{0, 1, 2, 3, 4, 5\}$
2	$\{1, 2, 3, 4, 5\}$	$\{0, 2, 3\}$
3	$\{2, 5\}$	$\{0, 2, 5\}$
4	$\{2, 3, 4, 5\}$	$\{0, 1, 2, 3\}$

Exercise 9.3 *The AllDifferent Global Constraint and Hall's Condition*

Given four variables x_0, x_1, x_2, x_3 with domains:

$$D(x_0) = \{1, 3\}, \quad D(x_1) = \{1, 3\}, \quad D(x_2) = \{1, 3, 8\}, \quad D(x_3) = \{8, 9\},$$

linked by the global constraint AllDifferent(x_0, x_1, x_2, x_3) [Rég94]:

1. Explain why a decomposition into $\binom{4}{2} = 6$ binary `NotEqual` constraints ($x_i \neq x_j$ for $0 \leq i < j \leq 3$) in `TinyCSP` fails to remove any value from $D(x_0), D(x_1), D(x_2), D(x_3)$.
2. Justify mathematically why value 8 *must* be removed from $D(x_3)$ (and values $1, 3$ from $D(x_2)$) to achieve domain consistency on AllDifferent(x_0, x_1, x_2, x_3).

Exercise 9.4 *Bidirectional Filtering and Alternative Modeling for the Knight's Tour*

1. Examine the `KnightMove(x, y, n)` propagator in Listing 9.13. Notice that it filters $D(y)$ based on $D(x)$, but does not filter $D(x)$ based on $D(y)$. Because a knight jump is symmetric ($v \in N_n(u) \iff u \in N_n(v)$), show how to make `KnightMove.propagate()` bidirectional so that it achieves full domain consistency on both x and y in a single call.
2. In Section 9.9.3, we modeled the Knight's Tour with time-step variables x_t representing the board square visited at step $t \in \{0, \ldots, n^2 - 1\}$. Formulate an alternative CP model using successor variables $\texttt{succ}[u] \in \{0, \ldots, n^2 - 1\}$ for each square $u \in \{0, \ldots, n^2 - 1\}$ (analogous to the TSP model in Section 9.8). How are the initial domain of $\texttt{succ}[u]$ and the global tour constraint defined in this alternative model, and why is its initial domain size per variable at most 8 instead of n^2?

CHAPTER 10

Benchmarking Optimization Algorithms

Throughout this book, we have developed a rich repertoire of exact and heuristic paradigms for discrete optimization—Dynamic Programming, Branch-and-Bound, Linear and Mixed-Integer Linear Programming, Lagrangian Relaxation, Network Flows, Local Search, and Constraint Programming. Even within a single paradigm, a practitioner routinely faces a multitude of design choices: which variable-ordering heuristic should guide a Branch-and-Bound tree? Which relaxation or propagator strength offers the best trade-off between filtering power and per-node overhead? Which metaheuristic neighborhood yields the fastest convergence on large-scale routing instances?

Whereas asymptotic worst-case analysis ($\mathcal{O}$-notation) provides indispensable theoretical guarantees, it rarely predicts which of two **NP**-hard optimization solvers will perform better in practice. Worst-case bounds are dominated by pathological instances that may never arise in an application domain, and they ignore large constant factors, cache locality, and the cascading effects of domain pruning or cutting planes. Consequently, the empirical evaluation of optimization software through rigorous *benchmarking* is a cornerstone of modern computational operations research.

Yet comparing algorithms across a suite of benchmark instances is fraught with statistical and methodological pitfalls. Naive summary statistics—such as the arithmetic mean of runtimes, average speedups relative to a baseline, or simple counts of solved instances—can easily distort experimental findings or even reverse the apparent ranking of competing solvers depending on arbitrary normalization choices. This chapter establishes a rigorous methodology for evaluating and comparing optimization algorithms, covering cumulative runtime distributions (*cactus plots*), relative slowdown distributions (*performance profiles*), *optimality gap profiles*, anytime evaluation via the *primal integral*, distributional visualizations, and the mathematical invariance of the *geometric mean*.

10.1 The Benchmarking Problem

Suppose we have a set of candidate algorithms, solvers, or parameter configurations that we wish to compare on a given optimization problem. To evaluate them empirically, we assemble a finite collection of benchmark instances that is assumed to be representative of the unknown future instances the solver will encounter in deployment.

Definition 10.1 *Benchmarking Setup and Runtime Matrix*

Let $\mathcal{P} = \{0, 1, \ldots, n_p - 1\}$ denote a finite set of $n_p = |\mathcal{P}|$ benchmark problem instances, and let $\mathcal{S} = \{0, 1, \ldots, n_s - 1\}$ denote a finite set of $n_s = |\mathcal{S}|$ solvers (or algorithmic strategies). Running each solver $s \in \mathcal{S}$ on each instance $p \in \mathcal{P}$ under a fixed computational environment and a maximum time limit (timeout) $t_{\max} > 0$ yields a performance measurement $t_{p,s} \in (0, t_{\max}] \cup \{\text{TO}\}$, where $t_{p,s}$ denotes the computing time required by solver s to solve instance p to proven optimality, and TO indicates that solver s timed out at $t_{\max}$ without completing the proof.

When designing and analyzing a benchmark campaign, we seek to answer three fundamental questions:

1. **Generalizable Ranking**: Provided that the benchmark suite $\mathcal{P}$ is sufficiently diverse and representative of the target application, which solver performs best overall?
2. **Relative Speedup Quantification**: By what factor is one solver faster or slower than its competitors across the instance distribution, and how sensitive is its performance to instance characteristics?
3. **Win-Rate and Robustness Estimation**: What is the empirical probability that a given solver is the fastest on an unseen instance, and what fraction of instances can it solve reliably within a given time budget?

Example 10.1 *A Three-Solver Benchmark Across Heterogeneous Instances*

To ground our discussion, consider $n_s = 3$ solvers, denoted $\mathcal{S} = \{A, B, C\}$, evaluated on $n_p = 6$ problem instances $\mathcal{P} = \{p_0, p_1, p_2, p_3, p_4, p_5\}$. Just as an off-road vehicle (A), a formula race car (B), and a rally car (C) excel on different terrains—highway tarmac (p_0, p_1), mixed asphalt-and-gravel courses (p_2, p_3), and rugged forest trails (p_4, p_5)—discrete optimization solvers frequently exhibit specialized strengths across structural families of instances. Table 10.1 reports the completion times $t_{p,s}$ (in seconds) of the three solvers across the six instances.

Inspecting Table 10.1, we see that each solver is strictly the fastest on exactly two of the six instances: Solver B wins on p_0 and p_1, Solver C wins on p_2 and p_3, and Solver A wins on p_4 and p_5. Which solver is the best? As we shall see next, the answer depends critically on how we aggregate the matrix $(t_{p,s})$.

Table 10.1: Completion times $t_{p,s}$ (in seconds) for three solvers $\mathcal{S} = \{A, B, C\}$ across $n_p = 6$ benchmark instances $\mathcal{P} = \{p_0, \ldots, p_5\}$ spanning three structural families.

Instance p	**Instance Family**	**Solver** A	**Solver** B	**Solver** C
p_0	Family 0 (Structured)	100	**10**	15
p_1	Family 0 (Structured)	30	**7**	12
p_2	Family 1 (Mixed)	40	45	**35**
p_3	Family 1 (Mixed)	50	55	**40**
p_4	Family 2 (Unstructured)	**10**	30	15
p_5	Family 2 (Unstructured)	**20**	50	25
Total Time $\sum_{p=0}^{5} t_{p,s}$		250	197	**142**
Arithmetic Mean $\bar{t}_s$		41.67	32.83	**23.67**
Instances Won		2 (33.3%)	2 (33.3%)	2 (33.3%)

10.2 Pitfalls of Naive Aggregations

Before introducing distribution-based benchmarking tools, let us examine why the two most common single-number summaries—the arithmetic mean of runtimes and the raw count of solved instances—often fail to provide a trustworthy comparison.

10.2.1 Proposal 1: Arithmetic Mean of Runtimes

A natural first impulse is to rank solvers by their average computing time across all benchmark instances:

$$\bar{t}_s = \frac{1}{n_p} \sum_{p=0}^{n_p-1} t_{p,s}. \tag{10.1}$$

In Table 10.1, this yields $\bar{t}_C = 23.67$ s $< \bar{t}_B = 32.83$ s $< \bar{t}_A = 41.67$ s, placing Solver A last by a wide margin. However, relying on (10.1) suffers from two severe methodological flaws:

1. **Dominance of the Hardest Instances**: Because discrete optimization runtimes frequently span several orders of magnitude (e.g., from 0.01 s on easy instances to 3,600 s on hard ones), the sum $\sum_{p=0}^{n_p-1} t_{p,s}$ is overwhelmingly dictated by the few most expensive instances in $\mathcal{P}$. In Table 10.1, Solver A's single slow run on p_0 ($t_{0,A} = 100$ s) accounts for 40% of its total runtime across all six instances, completely masking the fact that A is the fastest solver on p_4 and p_5 and competitive on p_2 and p_3.
2. **Survivorship Bias When Discarding Unsolved Instances**: In realistic benchmarks, solvers often time out ($t_{p,s} = \text{TO}$) on difficult instances. How should timeouts be incorporated into (10.1)? If we simply *discard* unsolved instances and average over the subset of instances that each solver managed to finish, we introduce a perverse **survivorship bias** that penalizes the most robust solvers! Suppose Solver S_0 solves 9 easy instances in 1 s each and times out at $t_{\max} = 1{,}000$ s on a tenth hard instance p_9, whereas Solver S_1 solves the same 9 easy instances in 1 s each *and also* solves the hard instance p_9 in 900 s. Averaging only over solved instances gives $\bar{t}_{S_0}^{\text{solved}} = 9/9 = 1.0$ s versus $\bar{t}_{S_1}^{\text{solved}} = 909/10 = 90.9$ s—making the strictly superior solver S_1 look ninety times slower! Conversely, if we replace timeouts by a constant penalty $k \cdot t_{\max}$ (known as the *PAR-k score*, such as PAR-2 or PAR-10 in SAT competitions), the resulting average becomes heavily dependent on the arbitrary choice of $t_{\max}$ and k.

10.2.2 Proposal 2: Counting Solved Instances

To avoid the distortion of extreme runtimes and timeouts, a second common proposal ranks solvers solely by the number (or percentage) of instances solved within the time limit $t_{\max}$:

$$N_{\text{solved}}(s) = \big|\{p \in \mathcal{P} : t_{p,s} \leq t_{\max}\}\big|. \tag{10.2}$$

While $N_{\text{solved}}(s)$ is a useful headline indicator of robustness, it is far too coarse-grained on its own:

- **Loss of Speed Resolution**: In Table 10.1, for any timeout $t_{\max} \geq 100$ s, all three solvers achieve $N_{\text{solved}}(A) = N_{\text{solved}}(B) = N_{\text{solved}}(C) = 6$, rendering them indistinguishable despite large differences in runtime. Even when timeouts occur, a solver that finishes in 0.1 s is treated identically to one that finishes at $0.99\,t_{\max}$.
- **Loss of Instance-Wise Ordering**: Counting solved instances at a single threshold $t_{\max}$ discards all information about which solver was faster on each individual instance $p \in \mathcal{P}$.

10.3 Cactus Plots

The main defect of counting solved instances at a single timeout $t_{\max}$ is that $t_{\max}$ is just one arbitrary point along the time axis. Instead of evaluating (10.2) only at $\tau = t_{\max}$, we can plot the fraction of solved instances as a function of a variable time budget $\tau \in [0, t_{\max}]$. This cumulative distribution curve is known as a *cactus plot* (or *cumulative runtime distribution*).

Definition 10.2 *Cactus Plot Function*

Given a benchmark set $\mathcal{P}$ of size $n_p = |\mathcal{P}|$ and a solver $s \in \mathcal{S}$, the *cactus plot function* $\kappa_s : [0, t_{\max}] \to [0, 1]$ measures the fraction of instances solved by s within time threshold τ:

$$\kappa_s(\tau) = \frac{1}{n_p} \big| \{ p \in \mathcal{P} : t_{p,s} \leq \tau \} \big|. \tag{10.3}$$

Equivalently, $n_p \cdot \kappa_s(\tau)$ gives the cumulative number of instances solved within τ seconds, and $100 \cdot \kappa_s(\tau)$ gives the percentage of solved instances.

For each solver $s \in \mathcal{S}$, $\kappa_s(\tau)$ is a right-continuous, non-decreasing step function that starts at $\kappa_s(0) = 0$ and jumps by $1/n_p$ (or k/n_p in case of ties) at each sorted completion time $t_{(0),s} \leq t_{(1),s} \leq \cdots \leq t_{(n_p-1),s}$.

Orientation Convention in Cactus Plots. In operations research and mathematical programming, $\kappa_s(\tau)$ is most commonly plotted with the runtime threshold τ on the horizontal axis and the cumulative fraction (or percentage) of solved instances $\kappa_s(\tau)$ on the vertical axis, as in Figure 10.1a, so that higher and further-left curves indicate better performance. In the satisfiability (SAT) and constraint programming competition communities, the axes are often transposed—placing the cumulative number of solved instances on the horizontal axis and the runtime τ on the vertical axis—so that the steeply rising curves of competing solvers fan upward and outward like the arms of a desert cactus, which is the historical origin of the name *cactus plot*. Both orientations encode the exact same sorted runtime data.

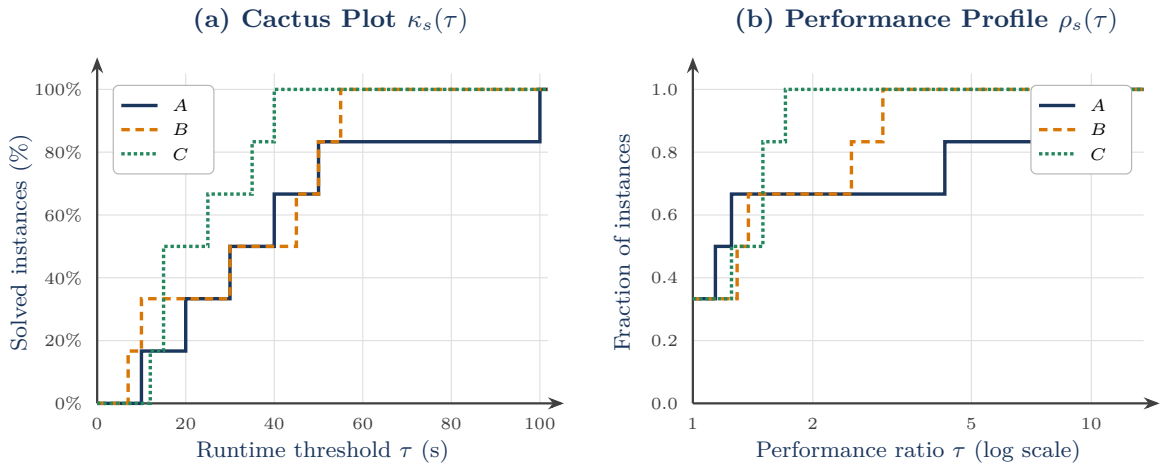


Figure 10.1: Comparison of Solvers A, B, C on the 6-instance benchmark of Table 10.1. (a) The Cactus Plot $\kappa_s(\tau)$ displays the percentage of instances solved within an absolute runtime threshold $\tau \in [0, 100]$ s. Solver B leads for $\tau \in [7, 12)$ s, after which Solver C dominates and solves 100% of the instances by $\tau = 40$ s. (b) The Performance Profile $\rho_s(\tau)$ plots the fraction of instances on which each solver is within a factor τ of the virtual best runtime t_p^*. At $\tau = 1$, all three solvers tie with $\rho_s(1) = 2/6 \approx 0.33$.

10.3.1 Strengths and Limitations of Cactus Plots

Figure 10.1a displays the cactus plots $\kappa_A(\tau)$, $\kappa_B(\tau)$, and $\kappa_C(\tau)$ for the runtime data of Table 10.1:

- **Fine-Grained Temporal Resolution**: Unlike a single solved-instance count at $t_{\max}$, the cactus plot immediately reveals how performance evolves with the time budget. For tight budgets $\tau \in [7, 12)$ s, Solver B solves the most instances ($2/6 = 33.3\%$ at $\tau = 10$ s). For medium and large budgets $\tau \geq 15$ s, Solver C's curve lies above both A and B, reaching 100% completion at $\tau = 40$ s, compared to $\tau = 55$ s for B and $\tau = 100$ s for A.
- **Fundamental Limitation—Loss of Instance-Wise Pairing**: Because each curve $\kappa_s(\tau)$ is constructed by sorting the column $(t_{p,s})_{p\in\mathcal{P}}$ *independently* for each solver s, the pairing between $t_{p,A}$, $t_{p,B}$, and $t_{p,C}$ on the same instance p is completely lost! Looking at Figure 10.1a alone, it is impossible to tell whether Solver A is ever faster than Solver C on any individual instance (in fact, A beats both B and C on p_4 and p_5!), nor can one distinguish an instance that took 40 s because all solvers found it intrinsically large from an instance that took 40 s because a specific solver struggled on an otherwise 1 s problem.

10.4 Performance Profiles

To overcome both the scale distortion of hard instances and the loss of instance-wise pairing in cactus plots, Dolan and Moré [DM02] introduced *performance profiles*. The key idea is to normalize each solver's runtime $t_{p,s}$ on instance p by the best runtime achieved on that exact same instance across all competing solvers in $\mathcal{S}$.

10.4.1 The Virtual Best Solver and Performance Ratios

Definition 10.3 *Virtual Best Solver and Performance Ratio*

Given a benchmark set $\mathcal{P}$ and a solver portfolio $\mathcal{S}$, the *virtual best solver* (VBS) is the idealized oracle that selects, for each instance $p \in \mathcal{P}$, the fastest solver in $\mathcal{S}$, achieving completion time:

$$t_p^* = \min_{s\in\mathcal{S}} t_{p,s}. \tag{10.4}$$

The *performance ratio* $r_{p,s} \geq 1$ of solver $s \in \mathcal{S}$ on instance $p \in \mathcal{P}$ is the ratio of its runtime to the virtual best runtime on instance p:

$$r_{p,s} = \frac{t_{p,s}}{\min_{s'\in\mathcal{S}} t_{p,s'}} = \frac{t_{p,s}}{t_p^*}. \tag{10.5}$$

If solver s fails to solve instance p within the timeout $t_{\max}$ (whereas at least one solver in $\mathcal{S}$ succeeds), we assign $r_{p,s} = r_M$, where $r_M > \max\{r_{p,s} : p \in \mathcal{P},\ s \in \mathcal{S} \text{ solved}\}$ is a fixed penalty constant representing failure (or $r_{p,s} = +\infty$).

By construction, $r_{p,s} \geq 1$ for every $p \in \mathcal{P}$ and $s \in \mathcal{S}$, and $r_{p,s} = 1$ if and only if solver s is the fastest (or tied for fastest) solver on instance p. Moreover, $r_{p,s}$ is dimensionless and scale-invariant: an easy instance where $t_{p,A} = 0.2$ s and $t_p^* = 0.1$ s yields the exact same performance ratio ($r_{p,A} = 2.0$) as a large instance where $t_{p',A} = 2{,}000$ s and $t_{p'}^* = 1{,}000$ s.

Table 10.2 computes the virtual best runtimes t_p^* and the performance ratios $r_{p,A}, r_{p,B}, r_{p,C}$ for our initial benchmark from Table 10.1.

Table 10.2: Virtual best runtime $t_p^* = \min_{s \in \mathcal{S}} t_{p,s}$ and dimensionless performance ratios $r_{p,s} = t_{p,s}/t_p^*$ for the runtime matrix of Table 10.1.

Instance p	$t_{p,A}$	$t_{p,B}$	$t_{p,C}$	**Virtual Best** t_p^*	$r_{p,A}$	$r_{p,B}$	$r_{p,C}$
p_0	100	10	15	10	10.00	**1.00**	1.50
p_1	30	7	12	7	4.29	**1.00**	1.71
p_2	40	45	35	35	1.14	1.29	**1.00**
p_3	50	55	40	40	1.25	1.38	**1.00**
p_4	10	30	15	10	**1.00**	3.00	1.50
p_5	20	50	25	20	**1.00**	2.50	1.25

10.4.2 From Performance Ratios to the Cumulative Profile $\rho_s(\tau)$

How should we visualize the matrix of performance ratios $(r_{p,s})$? Plotting a bar chart of $r_{p,s}$ indexed by the raw instance identifier $p \in \{0, \ldots, n_p - 1\}$ becomes unreadable as soon as n_p exceeds a few dozen instances, and the arbitrary numbering of instances obscures the overall distribution.

Instead, if we sort the ratios $(r_{p,s})_{p \in \mathcal{P}}$ of solver s in non-decreasing order (Figure 10.2a), the right envelope of the sorted bars forms a monotone staircase. Interpreting this staircase as an empirical cumulative distribution function yields the *performance profile* of solver s.

Definition 10.4 *Performance Profile*

The *performance profile* of solver $s \in \mathcal{S}$ is the cumulative distribution function $\rho_s : [1, \infty) \to [0, 1]$ of its performance ratio across the benchmark set $\mathcal{P}$:

$$\rho_s(\tau) = \frac{1}{n_p} \big| \{ p \in \mathcal{P} : r_{p,s} \leq \tau \} \big|, \quad \forall \tau \geq 1. \tag{10.6}$$

Thus, $\rho_s(\tau)$ is the empirical probability that solver s solves an instance $p \in \mathcal{P}$ within a factor τ of the fastest solver in $\mathcal{S}$ on that instance.

10.4.3 Step-by-Step Interpretation of Performance Profiles

Table 10.3: Runtimes $t_{p,s}$, virtual best t_p^*, and performance ratios $r_{p,s}$ illustrated in Figure 10.2. Bold entries indicate $r_{p,s} = 1.00$.

Instance p	$t_{p,A}$	$t_{p,B}$	$t_{p,C}$	**Virtual Best** t_p^*	$r_{p,A}$	$r_{p,B}$	$r_{p,C}$
p_0	20	10	15	10	2.00	**1.00**	1.50
p_1	21	7	12	7	3.00	**1.00**	1.71
p_2	40	45	35	35	1.14	1.29	**1.00**
p_3	50	55	40	40	1.25	1.38	**1.00**
p_4	10	30	15	10	**1.00**	3.00	1.50
p_5	20	50	25	20	**1.00**	2.50	1.25

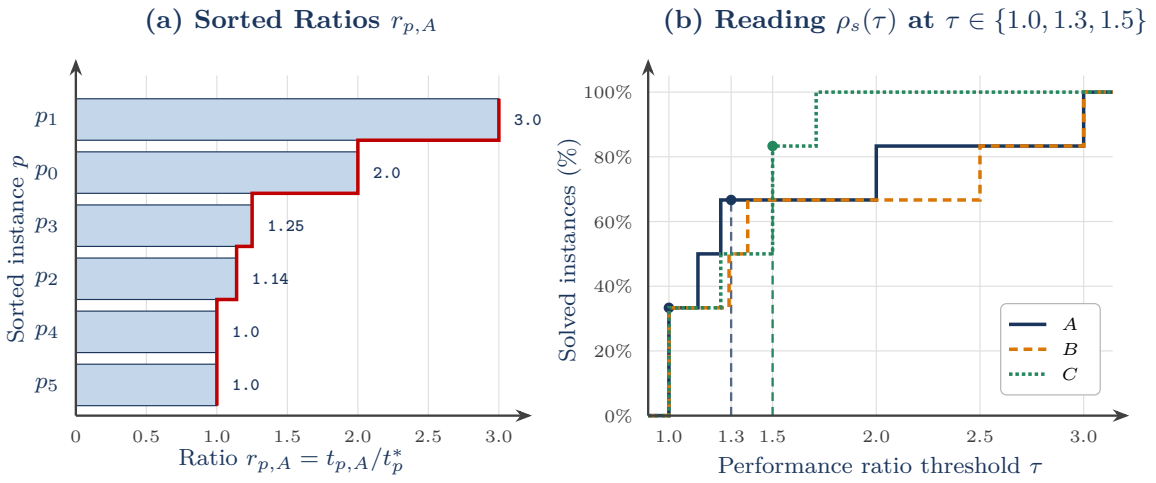


Figure 10.2: Construction and interpretation of a Performance Profile on the dataset of Table 10.3. (a) Sorting the performance ratios $r_{p,A}$ of Solver A from smallest ($r_{5,A} = r_{4,A} = 1.0$) to largest ($r_{1,A} = 3.0$) produces a monotone step envelope (highlighted in red). (b) Plotting the resulting cumulative step functions $\rho_A(\tau)$, $\rho_B(\tau)$, and $\rho_C(\tau)$ allows direct visual readings at any slowdown threshold τ: at $\tau = 1.0$, all three solvers tie with $\rho_s(1) = 33.3\%$; at $\tau = 1.3$, Solver A leads with $\rho_A(1.3) = 66.7\%$; at $\tau = 1.5$, Solver C leads with $\rho_C(1.5) = 83.3\%$.

Example 10.2 *Reading a Performance Profile Across Slowdown Thresholds*

To see how performance profiles reveal nuanced trade-offs between competing algorithms, consider the six-instance benchmark in Table 10.3 (where Solver A takes 20 s and 21 s on p_0 and p_1), plotted in Figure 10.2b.
By scanning Figure 10.2b from left ($\tau = 1$) to right ($\tau = 3$), we extract three complementary insights:

1. **At $\tau = 1.0$ (Win Probability)**: The vertical intercept $\rho_s(1.0)$ counts the fraction of instances where $r_{p,s} = 1.0$, i.e., where solver s is the strictly fastest solver (or tied for fastest). Here, A wins on $\{p_4, p_5\}$, B wins on $\{p_0, p_1\}$, and C wins on $\{p_2, p_3\}$, so all three curves start at the exact same height:
$$\rho_A(1.0) = \rho_B(1.0) = \rho_C(1.0) = \frac{2}{6} \approx 33.3\%.$$
2. **At $\tau = 1.3$ (Near-Optimal Competitiveness)**: If a user is willing to tolerate a slowdown of at most 30% relative to the virtual best solver ($r_{p,s} \leq 1.3$), which solver is best? Counting ratios ≤ 1.3 in Table 10.3 shows that Solver A qualifies on four instances (p_2, p_3, p_4, p_5 with ratios $1.14, 1.25, 1.00, 1.00$), whereas B qualifies on three (p_0, p_1, p_2) and C qualifies on three (p_2, p_3, p_5). Thus:
$$\rho_A(1.3) = \frac{4}{6} \approx 66.7\% > \rho_B(1.3) = \rho_C(1.3) = \frac{3}{6} = 50.0\%.$$
3. **At $\tau = 1.5$ and Beyond (Worst-Case Slowdown & Robustness)**: If we allow a slowdown factor of $\tau = 1.5$, Solver C solves five out of six instances (p_0, p_2, p_3, p_4, p_5), achieving $\rho_C(1.5) = 5/6 \approx 83.3\%$, and reaches $\rho_C(1.71) = 100\%$. Solver C is never more than $1.71\times$ slower than the virtual best on any instance, whereas both A and B suffer $3.0\times$ slowdowns on their unfavorable instance families.

10.4.4 Linear vs. Logarithmic Scale ($\log_2 \tau$) and Timeout Penalties

In large-scale optimization benchmarks—such as the COPS nonlinear programming test set analyzed by Dolan and Moré [DM02] or the MIPLIB mixed-integer programming library [Gle+21]—performance ratios $r_{p,s}$ frequently span several orders of magnitude, from $\tau = 1$ up to $\tau = 10^3$ or more. Plotting $\rho_s(\tau)$ on a linear horizontal axis over a narrow range such as $\tau \in [1, 10]$ (Figure 10.3a) can hide critical long-tail behavior.

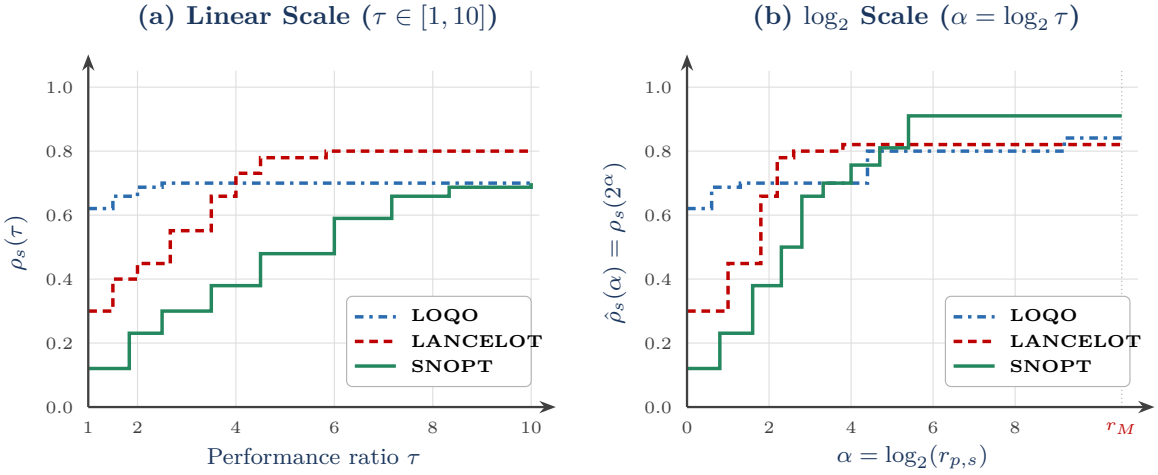


Figure 10.3: Effect of horizontal scaling on performance profiles, modeled after the COPS benchmark study of Dolan and Moré [DM02] comparing LOQO, LANCELOT, and SNOPT. (a) On a linear scale truncated at $\tau = 10$, LOQO has the highest probability of being the fastest solver ($\rho_{\text{LOQO}}(1) \approx 0.62$), LANCELOT overtakes LOQO for $\tau \geq 4$, and SNOPT appears third best—though its curve is still climbing at $\tau = 10$. (b) Plotting $\hat{\rho}_s(\alpha) = \rho_s(2^\alpha)$ against $\alpha = \log_2(r_{p,s})$ up to the timeout penalty r_M reveals that for larger time budgets ($\alpha \geq 5$, i.e., $\tau \geq 32$), SNOPT overtakes both competitors and achieves the highest overall robustness ($\approx 91\%$ of all instances solved).

To capture both fine-grained differences near $\tau = 1$ and asymptotic robustness across large slowdown factors on a single chart, Dolan and Moré recommend plotting the base-2 logarithm of the performance ratio on the horizontal axis (Figure 10.3b):

$$\hat{\rho}_s(\alpha) = \frac{1}{n_p} \left| \{ p \in \mathcal{P} : \log_2(r_{p,s}) \leq \alpha \} \right| = \rho_s(2^\alpha), \quad \alpha \geq 0. \tag{10.7}$$

Notice how the two views in Figure 10.3 complement one another:

- At $\alpha = 0$ ($\tau = 2^0 = 1$), $\hat{\rho}_{\text{LOQO}}(0) \approx 0.62$ shows that LOQO is the fastest solver on 62% of the benchmark instances.
- For moderate slowdown budgets $\tau \in [4, 16]$ ($\alpha \in [2, 4]$), LANCELOT solves more instances than LOQO ($\approx 80\%$ vs. 70%).
- On the linear plot (Figure 10.3a), SNOPT appears inferior over $\tau \in [1, 10]$ because it has larger startup overhead, yet its curve is still rising at $\tau = 10$. Extending the view via the $\log_2$ scale (Figure 10.3b) reveals that by $\alpha = 5.4$ ($\tau \approx 42$), SNOPT surpasses both LANCELOT and LOQO, solving over 90% of the test suite before reaching the timeout threshold r_M!

Table 10.4 summarizes the structural differences between Cactus Plots and Performance Profiles.

Table 10.4: Comparison of Cactus Plots ($\kappa_s(\tau)$) and Performance Profiles ($\rho_s(\tau)$).

Property	**Cactus Plot** $\kappa_s(\tau)$	**Performance Profile** $\rho_s(\tau)$
Definition	$\frac{1}{n_p}\lvert\{p \in \mathcal{P} : t_{p,s} \leq \tau\}\rvert$	$\frac{1}{n_p}\lvert\{p \in \mathcal{P} : r_{p,s} \leq \tau\}\rvert$
Horizontal axis τ	Absolute completion time (s)	Relative slowdown $t_{p,s}/t_p^* \geq 1$
Left intercept	$\kappa_s(0) = 0$	$\rho_s(1) =$ fraction won by s
Right asymptote	$\kappa_s(t_{\max}) =$ fraction solved	$\lim_{\tau \to r_M^-} \rho_s(\tau) =$ fraction solved
Instance pairing	Lost (columns sorted independently)	Preserved via virtual best t_p^*
Independence	Depends *only* on solver s	Depends on portfolio $\mathcal{S}$ via t_p^*

10.5 Optimality Gaps and Gap Profiles

Up to this point, we have evaluated solvers primarily by the time $t_{p,s}$ required to prove optimality. However, on challenging discrete optimization problems, exact solvers (such as Mixed-Integer Programming or Branch-and-Bound engines) frequently reach the time limit $t_{\max}$ without closing the search tree. When two solvers both time out at $t_{\max}$ on an instance p, recording only $t_{p,s} =$ TO discards vital information: one solver may have discovered a near-optimal solution with a 1% optimality gap, whereas the other may still be stuck with a poor 35% gap!

10.5.1 Primal Bounds, Dual Bounds, and Relative Optimality Gap

Consider a minimization problem on instance $p \in \mathcal{P}$ with optimal objective value z^*. During the execution of an exact Branch-and-Bound or MIP solver over time $t \in [0, t_{\max}]$, the solver maintains two monotonic quantities (Figure 10.4a):

- **The Primal Bound (Incumbent)** $\bar{z}(t)$: The objective value of the best feasible solution found up to time t (non-increasing step function, $\bar{z}(t) \geq z^*$).
- **The Dual Bound (Lower Bound)** $\underline{z}(t)$: The smallest lower bound across all remaining open subproblems in the search tree at time t (non-decreasing step function, $\underline{z}(t) \leq z^*$).

Definition 10.5 *Relative Optimality Gap*

Let $\bar{z}$ denote the best feasible objective value found by a solver on a minimization instance, and let z^* denote the optimal objective value. The *relative optimality gap* $\gamma \geq 0$ is defined as:

$$\gamma = \frac{|\bar{z} - z^*|}{|\bar{z}|}. \tag{10.8}$$

When the true optimal value z^* is unknown (for instance, when the solver times out before proving optimality), commercial and academic MIP solvers replace z^* with the best proven lower bound $\underline{z} \leq z^*$ to report the *proven optimality gap* (typically expressed as a percentage $100 \cdot \gamma$):

$$\gamma = \frac{|\bar{z} - \underline{z}|}{|\bar{z}|}. \tag{10.9}$$

When optimality is proven ($\underline{z} = \bar{z} = z^*$), the optimality gap reaches $\gamma = 0$.

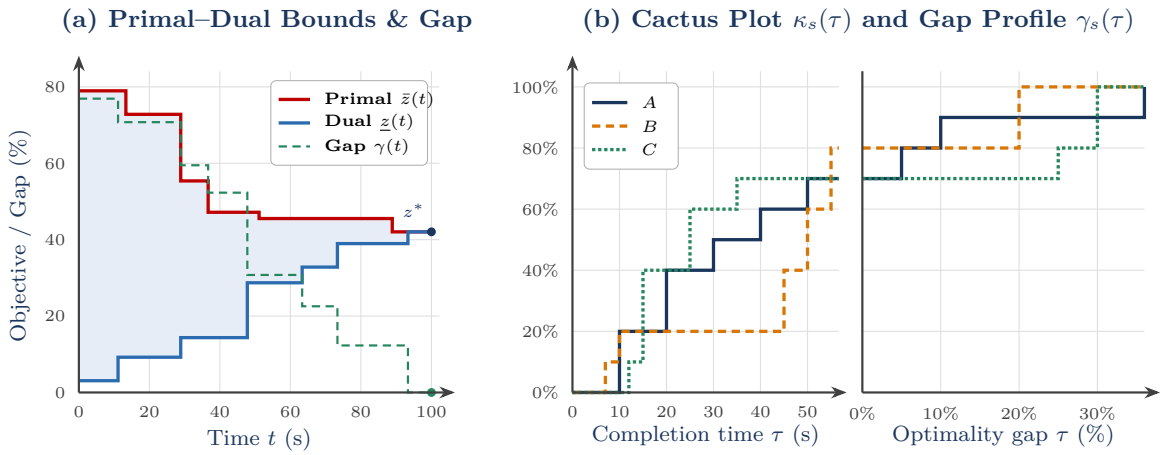


Figure 10.4: (a) Evolution of the primal bound $\bar{z}(t)$, dual bound $\underline{z}(t)$, and relative optimality gap $\gamma(t)$ during a Branch-and-Bound solve; optimality is proven at $t = 100$ s when $\bar{z}(t) = \underline{z}(t) = z^*$ and $\gamma(t) = 0\%$. (b) Joint evaluation of completion times and timeout optimality gaps for the 10-instance benchmark of Table 10.5. The left panel shows the Cactus Plot $\kappa_s(\tau)$ over $\tau \in [0, 55]$ s; the right panel continues seamlessly from $\gamma_s(0) = \kappa_s(t_{\max})$ (70% for A, C and 80% for B) to display the Optimality Gap Profile $\gamma_s(\tau)$ over $\tau \in [0\%, 36\%]$.

10.5.2 Joint Evaluation via Cactus Plots and Gap Profiles

When benchmarking exact optimization solvers under a timeout $t_{\max}$, each run (p, s) produces a pair $(t_{p,s}, \gamma_{p,s})$:

- If solver s proves optimality on instance p within the time limit, then $t_{p,s} \leq t_{\max}$ and its final optimality gap is $\gamma_{p,s} = 0\%$.
- If solver s hits the time limit on instance p, then $t_{p,s} = \text{TO}$ and $\gamma_{p,s} > 0\%$ records the remaining optimality gap at $t_{\max}$.

Definition 10.6 *Optimality Gap Profile*

Given the optimality gaps $\gamma_{p,s} \geq 0$ at termination (or timeout $t_{\max}$) for each instance $p \in \mathcal{P}$ and solver $s \in \mathcal{S}$, the *optimality gap profile* $\gamma_s : [0, \infty) \to [0, 1]$ is the empirical cumulative distribution function of the final gaps:

$$\gamma_s(\tau) = \frac{1}{n_p} \big|\{p \in \mathcal{P} : \gamma_{p,s} \leq \tau\}\big|, \quad \forall \tau \geq 0. \tag{10.10}$$

Observe the mathematical link between the Cactus Plot κ_s at the timeout $t_{\max}$ and the Optimality Gap Profile γ_s at $\tau = 0$: because an instance has $\gamma_{p,s} = 0$ if and only if it is solved to proven optimality within $t_{\max}$, we have:

$$\gamma_s(0) = \kappa_s(t_{\max}). \tag{10.11}$$

Consequently, placing the Cactus Plot $\kappa_s(\tau)$ for $\tau \in [0, t_{\max}]$ and the Gap Profile $\gamma_s(\tau)$ for $\tau \in [0, \gamma_{\max}]$ side-by-side with a shared vertical axis (Figure 10.4b) provides a unified, continuous view of both completion speed and timeout solution quality!

Table 10.5: Completion times $t_{p,s}$ (in seconds, or TO if timed out at $t_{\max} = 55$ s) and final optimality gaps $\gamma_{p,s}$ (in %) across $n_p = 10$ benchmark instances.

	Solver A		**Solver** B		**Solver** C	
Instance p	**Time (s)**	**Gap** γ **(%)**	**Time (s)**	**Gap** γ **(%)**	**Time (s)**	**Gap** γ **(%)**
p_0	TO	5	**10**	0	15	0
p_1	30	0	**7**	0	12	0
p_2	40	0	45	0	**35**	0
p_3	TO	36	**55**	0	TO	30
p_4	**10**	0	TO	20	15	0
p_5	**20**	0	50	0	25	0
p_6	TO	10	**45**	0	TO	25
p_7	**50**	0	55	0	TO	30
p_8	**10**	0	TO	20	15	0
p_9	**20**	0	50	0	25	0
Solved ($\gamma = 0\%$)	7/10 (70%)		**8/10 (80%)**		7/10 (70%)	

Example 10.3 *Joint Runtime and Optimality Gap Benchmark*

Consider $n_s = 3$ solvers $\mathcal{S} = \{A, B, C\}$ evaluated across $n_p = 10$ instances $\mathcal{P} = \{p_0, \ldots, p_9\}$ with a timeout of $t_{\max} = 55$ s, as reported in Table 10.5.
Examining the linked plots in Figure 10.4b reveals how the three solvers behave across both regimes:

- **Completion Time Regime (Left Panel)**: For budgets $\tau \in [15, 45]$ s, Solver C solves the largest fraction of instances (70% by $\tau = 35$ s). Near the timeout $\tau = 55$ s, Solver B catches up and solves 80% of the instances, whereas A and C each solve 70%.
- **Optimality Gap Regime (Right Panel)**: Starting at $\gamma_A(0) = \gamma_C(0) = 70\%$ and $\gamma_B(0) = 80\%$, we see that on two of its three timed-out instances (p_0 and p_6), Solver A has small gaps (5% and 10%), so $\gamma_A(\tau)$ jumps to 80% at $\tau = 5\%$ and to 90% at $\tau = 10\%$—overtaking Solver B for gap tolerances $\tau \in [10\%, 20\%)$! By $\tau = 20\%$, Solver B reaches $\gamma_B(20\%) = 100\%$, whereas Solver C's three timed-out instances all have large gaps (25% and 30%).

10.6 Anytime Behavior and Primal Integral

In many industrial applications—such as real-time vehicle dispatching, online production scheduling, or interactive planning—a solver may be terminated early by the user or by a strict real-time deadline long before proving optimality. Moreover, for pure heuristic and metaheuristic algorithms (such as Simulated Annealing, Tabu Search, or Large Neighborhood Search from Chapter 8), no dual bound is computed at all; the solver simply produces a sequence of improving incumbent solutions over time.

An optimization algorithm exhibits good *anytime behavior* when it discovers high-quality feasible solutions rapidly and continues to improve them steadily throughout its execution, regardless of when the search is stopped.

Consider the three incumbent objective trajectories A, B, and C depicted in Figure 10.5a:

- **Algorithm** A eventually finds the best solution just before the time limit, but spends 80% of the run with a very poor incumbent. If stopped early, A returns a terrible

solution.

- **Algorithm** C improves very rapidly at the beginning of the search, quickly delivering a good solution, but plateaus early at a slightly higher objective value than A and B.
- **Algorithm** B achieves a strong compromise between early descent and final solution quality.

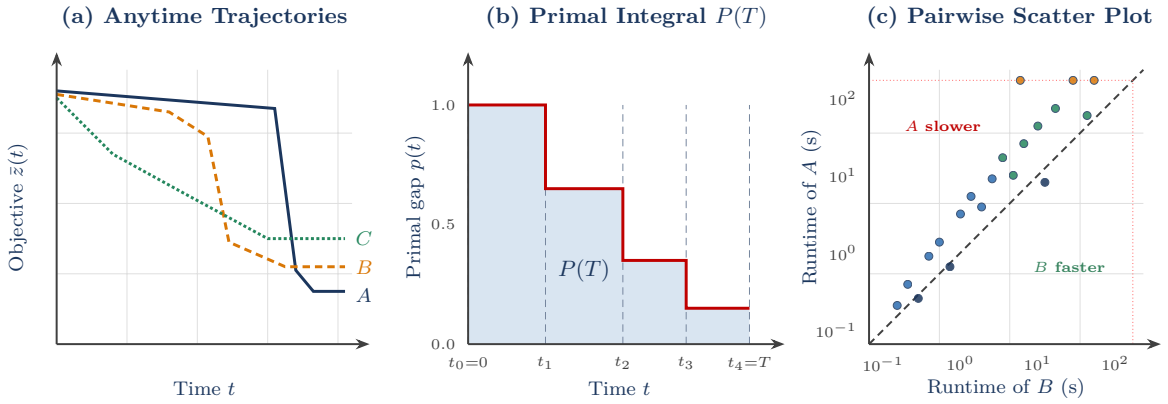


Figure 10.5: (a) Anytime incumbent trajectories $\bar{z}(t)$ for three minimization algorithms A, B, C: Algorithm C descends earliest, whereas Algorithm A finds the best final value only near the end of the time horizon. (b) Berthold's primal gap function $p(t) \in [0,1]$ and its shaded area, the Primal Integral $P(T) = \sum_{k=0}^{K-1} p(t_k)(t_{k+1} - t_k)$, across incumbent update times $t_0 = 0 < t_1 < t_2 < t_3 < t_4 = T$. (c) Pairwise log-log scatter plot comparing the runtime of Algorithm A (vertical axis) against Algorithm B (horizontal axis); points above the diagonal $y = x$ indicate instances where B is faster than A.

10.6.1 Berthold's Primal Gap and Primal Integral

To quantify anytime performance into a single, mathematically sound metric that accounts for *both* the quality of intermediate solutions and the exact times at which they are discovered, Berthold [Ber13] introduced the *primal integral*.

First, to ensure that the instantaneous gap is always bounded in $[0,1]$—even when no feasible solution has yet been found, or when the objective value can be zero or change sign—we normalize the incumbent $\bar{z}$ against the optimal (or best known) objective value z^* for the instance.

Definition 10.7 *Normalized Primal Gap*

Let $\bar{z} \in \mathbb{R}$ be the objective value of a feasible solution for a minimization problem, and let $z^* \in \mathbb{R}$ be the optimal (or best known) objective value for that problem instance. The *normalized primal gap* $\gamma(\bar{z}) \in [0,1]$ is defined as:

$$\gamma(\bar{z}) = \begin{cases} 0, & \text{if } |z^*| = |\bar{z}| = 0, \\ 1, & \text{if } z^* \cdot \bar{z} < 0, \\ \dfrac{|z^* - \bar{z}|}{\max\{|z^*|, |\bar{z}|\}}, & \text{otherwise.} \end{cases} \tag{10.12}$$

Note that whenever z^* and $\bar{z}$ share the same sign (so $z^* \cdot \bar{z} \geq 0$ and not both zero), we

have $0 \leq |z^* - \bar{z}| \leq \max\{|z^*|, |\bar{z}|\}$, which guarantees $\gamma(\bar{z}) \in [0,1]$, with $\gamma(\bar{z}) = 0$ if and only if $\bar{z} = z^*$.

Definition 10.8 *Primal Gap Function and Primal Integral*

Let $t_{\max} > 0$ be the time limit for a solver run on a given instance. The solver's *primal gap function* $p : [0, t_{\max}] \to [0,1]$ at time $t \in [0, t_{\max}]$ is defined as:

$$p(t) = \begin{cases} 1, & \text{if no feasible incumbent has been found up to time } t, \\ \gamma(\bar{z}(t)), & \text{where } \bar{z}(t) \text{ is the incumbent objective value at time } t. \end{cases} \tag{10.13}$$

For any time horizon $T \in [0, t_{\max}]$, let $0 = t_0 < t_1 < \cdots < t_{K-1} < t_K = T$ denote the sequence of time points at which a new incumbent solution is discovered (augmented with the endpoints $t_0 = 0$ and $t_K = T$). The *primal integral* $P(T)$ of the run over $[0, T]$ is the area under the step function $p(t)$:

$$P(T) = \int_0^T p(t)\, dt = \sum_{k=0}^{K-1} p(t_k) \cdot (t_{k+1} - t_k). \tag{10.14}$$

As illustrated in Figure 10.5b, $p(t)$ is a non-increasing staircase function on $[0, T]$, and $P(T) \in [0, T]$ is simply the sum of the areas of K rectangles. A smaller primal integral $P(t_{\max})$ (or normalized average primal gap $P(t_{\max})/t_{\max} \in [0,1]$) indicates better anytime performance: the solver either finds feasible solutions earlier, finds solutions with smaller primal gaps, or both.

10.7 Distributional and Pairwise Plots

When diagnosing *why* one algorithmic variant outperforms another—for example, comparing search speed in nodes per second, number of backtracks, or optimality gaps across multiple problem classes—two additional visualization techniques provide invaluable diagnostic detail:

1. **SinaPlots Over Box Plots**: Traditional box-and-whisker plots compress an entire empirical distribution into five numbers (median, quartiles, and whiskers), completely obscuring whether the underlying observations are unimodal, bimodal, or clustered around specific instance subclasses. A *SinaPlot* [Sid+18; Del25] plots every individual instance observation as a point whose horizontal jitter is constrained by the local kernel density estimate (violin contour) of the sample. This truthfully reveals both the exact sample size and multi-modal clustering across algorithmic variants without hiding outliers.
2. **Pairwise Log-Log Scatter Plots**: When comparing two specific algorithms A and B head-to-head across a benchmark set $\mathcal{P}$ (Figure 10.5c), each instance $p \in \mathcal{P}$ can be plotted directly as a point $(m_{p,B}, m_{p,A})$ in the plane, where $m_{p,s}$ is the metric of interest (such as runtime on a logarithmic scale, number of backtracks, or optimality gap in %), optionally colored by an instance feature such as the number of variables $|X|$. Every point above the main diagonal $y = x$ corresponds to an instance where B is strictly better (smaller runtime or gap) than A; every point below $y = x$ favors A; and points lying on the top or right boundary lines immediately highlight instances where one solver timed out while the other succeeded.

10.8 Aggregating Normalized Results

In tables of experimental results, authors frequently normalize the performance of competing algorithms relative to a reference baseline solver and then report a single summary row at the bottom of the table. We conclude this chapter with a classic, sobering lesson due to Fleming and Wallace [FW86]: **computing the arithmetic mean of normalized benchmark results is mathematically invalid and can reverse the apparent winner depending on which column is chosen as the reference!**

10.8.1 The Paradox of the Arithmetic Mean of Normalized Results

Suppose we measure the time required to reach optimality for three algorithms $\mathcal{S} = \{R, M, Z\}$ across $n_p = 5$ benchmark instances $\mathcal{P} = \{p_0, p_1, p_2, p_3, p_4\}$. The raw runtimes $t_{p,s}$ are identical in both panels of Table 10.6.

Table 10.6: The paradox of the arithmetic mean of normalized results [FW86]. Using the *exact same raw runtimes* $t_{p,s}$ for algorithms R, M, Z across instances $p_0, \ldots, p_4$, normalizing with respect to R (top) makes R the apparent winner by the arithmetic mean ($\mathbf{1.00} < 1.01 < 1.07$), whereas normalizing with respect to M (bottom) makes M the apparent winner and claims R is 32% slower ($1.32 > \mathbf{1.00} < 1.08$)! In contrast, the geometric mean preserves the exact same ranking ($Z < M < R$, with Z winning at **0.84** and **0.99**) under both normalizations.

	Raw Runtimes $t_{p,s}$			Normalized wrt Algo R ($t_{p,s}/t_{p,R}$)		
Instance	**Algo R**	**Algo M**	**Algo Z**	**Algo R**	**Algo M**	**Algo Z**
p_0 (E)	417	244	134	1.00	0.59	0.32
p_1 (F)	83	70	70	1.00	0.84	0.84
p_2 (H)	66	153	135	1.00	2.32	2.05
p_3 (I)	39,449	33,527	66,000	1.00	0.85	1.67
p_4 (K)	772	368	369	1.00	0.48	0.48
Arithmetic Mean				**1.00**	1.01	1.07
Geometric Mean				1.00	0.86	**0.84**

	Raw Runtimes $t_{p,s}$			Normalized wrt Algo M ($t_{p,s}/t_{p,M}$)		
Instance	**Algo R**	**Algo M**	**Algo Z**	**Algo R**	**Algo M**	**Algo Z**
p_0 (E)	417	244	134	1.71	1.00	0.55
p_1 (F)	83	70	70	1.19	1.00	1.00
p_2 (H)	66	153	135	0.43	1.00	0.88
p_3 (I)	39,449	33,527	66,000	1.18	1.00	1.97
p_4 (K)	772	368	369	2.10	1.00	1.00
Arithmetic Mean				1.32	**1.00**	1.08
Geometric Mean				1.17	1.00	**0.99**

Look closely at what happens when we take the arithmetic mean of the normalized columns in Table 10.6:

1. **Normalizing with respect to R (top half)**: For each instance $p \in \{p_0, \ldots, p_4\}$, we divide every entry in row p by $t_{p,R}$. The arithmetic means of the normalized columns are:

$$\mathrm{AM}(R/R) = 1.00, \qquad \mathrm{AM}(M/R) = 1.01, \qquad \mathrm{AM}(Z/R) = 1.07.$$

   From these numbers, an unsuspecting analyst would conclude that **Algorithm R is the winner**, and that M is 1% slower than R while Z is 7% slower than R!

2. **Normalizing with respect to M (bottom half)**: Now take the *exact same dataset* and normalize each row with respect to $t_{p,M}$ instead. The arithmetic means of the normalized columns become:

$$\mathrm{AM}(R/M) = 1.32, \qquad \mathrm{AM}(M/M) = 1.00, \qquad \mathrm{AM}(Z/M) = 1.08.$$

Now **Algorithm M is the winner**, and Algorithm R appears to be 32% slower than M!

Why does the arithmetic mean produce such a self-contradictory result? Because ratios are *multiplicative* rather than additive: a $2\times$ slowdown ($r = 2.0$) on one instance and a $2\times$ speedup ($r = 0.5$) on another instance should cancel each other out to unity ($2.0 \times 0.5 = 1.0^2$), yet their arithmetic mean is $\frac{1}{2}(2.0 + 0.5) = 1.25 > 1.0$! By Jensen's inequality applied to the strictly convex function $f(x) = 1/x$ on $(0, \infty)$, whenever the ratios $r_p = t_{p,A}/t_{p,B}$ are not all identical, the arithmetic mean of (r_p) and the arithmetic mean of $(1/r_p)$ strictly exceed 1 when multiplied together, systematically biasing the arithmetic mean against whichever solver is NOT chosen as the denominator.

10.8.2 The Solution: The Geometric Mean

The unique aggregation operator that preserves relative ratios under arbitrary changes of the reference baseline is the *geometric mean*.

Definition 10.9 *Geometric Mean and Shifted Geometric Mean*

Given n_p positive measurements $x_0, x_1, \ldots, x_{n_p-1} > 0$, their *geometric mean* is defined as:

$$\mathrm{GM}(x_0, \ldots, x_{n_p-1}) = \left(\prod_{p=0}^{n_p-1} x_p \right)^{1/n_p} = \exp\left(\frac{1}{n_p} \sum_{p=0}^{n_p-1} \ln x_p \right). \tag{10.15}$$

When measurements $x_p \geq 0$ can be zero or very close to zero (e.g., sub-millisecond runtimes or zero branch-and-bound nodes), the *shifted geometric mean* with shift parameter $\sigma > 0$ (commonly $\sigma = 1$ s or $\sigma = 10$ s for time, and $\sigma = 100$ for search tree nodes [Gle+21]) is defined as:

$$\mathrm{SGM}_\sigma(x_0, \ldots, x_{n_p-1}) = \left(\prod_{p=0}^{n_p-1} (x_p + \sigma) \right)^{1/n_p} - \sigma = \exp\left(\frac{1}{n_p} \sum_{p=0}^{n_p-1} \ln(x_p + \sigma) \right) - \sigma. \tag{10.16}$$

Theorem 10.1 *Invariance of Normalized Geometric Means*

Let $t_{p,s} > 0$ be positive performance measurements for $p \in \mathcal{P} = \{0, \ldots, n_p - 1\}$ and $s \in \mathcal{S}$. For any two solvers $A, B \in \mathcal{S}$ and any reference solver $\mathrm{Ref} \in \mathcal{S}$, the ratio of the geometric means of their normalized measurements is independent of Ref and equals the geometric mean of their direct pairwise ratios:

$$\frac{\mathrm{GM}(t_{\cdot,A}/t_{\cdot,\mathrm{Ref}})}{\mathrm{GM}(t_{\cdot,B}/t_{\cdot,\mathrm{Ref}})} = \left(\prod_{p=0}^{n_p-1} \frac{t_{p,A}}{t_{p,B}} \right)^{1/n_p} = \frac{\mathrm{GM}(t_{\cdot,A})}{\mathrm{GM}(t_{\cdot,B})}. \tag{10.17}$$

Consequently, the ranking of all solvers in $\mathcal{S}$ under the geometric mean is strictly invariant to the choice of reference solver Ref (or even to whether the measurements

are normalized at all before taking the geometric mean).

Proof. By Definition 10.9 and the multiplicative property of exponents:

$$\frac{\mathrm{GM}(t_{\cdot,A}/t_{\cdot,\mathrm{Ref}})}{\mathrm{GM}(t_{\cdot,B}/t_{\cdot,\mathrm{Ref}})} = \frac{\left(\prod_{p=0}^{n_p-1} \frac{t_{p,A}}{t_{p,\mathrm{Ref}}}\right)^{1/n_p}}{\left(\prod_{p=0}^{n_p-1} \frac{t_{p,B}}{t_{p,\mathrm{Ref}}}\right)^{1/n_p}} = \left(\prod_{p=0}^{n_p-1} \frac{t_{p,A}/t_{p,\mathrm{Ref}}}{t_{p,B}/t_{p,\mathrm{Ref}}}\right)^{1/n_p}$$
$$= \left(\prod_{p=0}^{n_p-1} \frac{t_{p,A}}{t_{p,B}}\right)^{1/n_p} = \frac{\mathrm{GM}(t_{\cdot,A})}{\mathrm{GM}(t_{\cdot,B})}. \quad \square$$

Returning to Table 10.6, when we aggregate using the geometric mean:

- Normalizing with respect to R gives $\mathrm{GM} = (1.00, 0.86, \mathbf{0.84})$.
- Normalizing with respect to M gives $\mathrm{GM} = (1.17, 1.00, \mathbf{0.99})$.

Notice that $1/0.86 \approx 1.17$ and $0.84/0.86 \approx 0.98 \approx 0.99$ (up to two-decimal rounding of the table entries): under both normalizations, **Algorithm Z is the true winner**, followed by M, with R in third place!

10.9 Computing Benchmarking Metrics in Java

To facilitate reproducible experimental evaluation in Java, Listings 10.1 and 10.2 provide a self-contained utility class `BenchmarkMetrics` implementing the core metrics developed in this chapter: the Dolan–Moré performance ratio matrix and performance profile $\rho_s(\tau)$, the cactus plot $\kappa_s(\tau)$, Berthold's primal gap $\gamma(\bar{z}, z^*)$ and primal integral $P(T)$, and the shifted geometric mean SGM_σ. Note that computing products $\prod_{p=0}^{n_p-1} x_p$ directly in floating-point arithmetic can easily overflow or underflow when n_p is large; therefore, `shiftedGeometricMean()` always accumulates $\frac{1}{n_p}\sum_{p=0}^{n_p-1} \ln(x_p + \sigma)$ in logarithmic space before applying `Math.exp(...)`.

10.10 Exercises

Exercise 10.1 *Cactus Plot vs. Performance Profile Construction*

Consider $n_s = 3$ solvers $\mathcal{S} = \{S_0, S_1, S_2\}$ evaluated on $n_p = 5$ instances $\mathcal{P} = \{p_0, p_1, p_2, p_3, p_4\}$ with timeout $t_{\max} = 100$ s and completion times (in seconds):

Solver	p_0	p_1	p_2	p_3	p_4
S_0	2	4	20	90	10
S_1	4	2	10	45	20
S_2	3	3	15	60	15

1. Compute the arithmetic mean $\bar{t}_s$ and the geometric mean GM_s of the completion times for each solver $s \in \mathcal{S}$. Why does p_3 dominate the arithmetic mean?
2. Determine the jump points and heights of the cactus plot functions $\kappa_{S_0}(\tau)$, $\kappa_{S_1}(\tau)$, and $\kappa_{S_2}(\tau)$ over $\tau \in [0, 100]$.
3. Compute the virtual best runtime t_p^* for each instance $p \in \mathcal{P}$ and the performance ratio matrix $(r_{p,s})$.

```java
public final class BenchmarkMetrics {

    public static double[][] performanceRatios(
            double[][] times, double timeout, double rM) {
        int nP = times.length, nS = times[0].length;
        double[][] ratios = new double[nP][nS];
        for (int p = 0; p < nP; p++) {
            double minT = Double.POSITIVE_INFINITY;
            for (int s = 0; s < nS; s++) {
                if (times[p][s] <= timeout) {
                    minT = Math.min(minT, times[p][s]);
                }
            }
            for (int s = 0; s < nS; s++) {
                ratios[p][s] = (times[p][s] <= timeout
                    && minT < Double.POSITIVE_INFINITY)
                    ? times[p][s] / minT : rM;
            }
        }
        return ratios;
    }

    public static double profileValue(
            double[][] values, int s, double tau) {
        int count = 0, nP = values.length;
        for (int p = 0; p < nP; p++) {
            if (values[p][s] <= tau) count++;
        }
        return (double) count / nP;
    }
```

Listing 10.1: Computation of performance ratios and empirical profile values (`BenchmarkMetrics.java`, Part 1).

```
    public static double primalGap(
            double zBar, double zStar) {
        if (zBar == 0.0 && zStar == 0.0) return 0.0;
        if (zBar * zStar < 0.0) return 1.0;
        double denom = Math.max(
            Math.abs(zBar), Math.abs(zStar));
        return Math.abs(zStar - zBar) / denom;
    }

    public static double primalIntegral(
            double[] times, double[] zBars,
            double zStar, double horizonT) {
        double integral = 0.0, prevT = 0.0, pGap = 1.0;
        for (int k = 0; k < times.length; k++) {
            double t = Math.min(times[k], horizonT);
            if (t > prevT) {
                integral += pGap * (t - prevT);
                prevT = t;
            }
            pGap = primalGap(zBars[k], zStar);
            if (times[k] >= horizonT) break;
        }
        if (horizonT > prevT) {
            integral += pGap * (horizonT - prevT);
        }
        return integral;
    }

    public static double shiftedGeometricMean(
            double[] x, double shift) {
        double logSum = 0.0;
        for (int p = 0; p < x.length; p++) {
            logSum += Math.log(x[p] + shift);
        }
        return Math.exp(logSum / x.length) - shift;
    }
}
```

Listing 10.2: Computation of primal gap, primal integral, and shifted geometric mean (`BenchmarkMetrics.java`, Part 2).

4. Give the exact values of the performance profiles $\rho_{S_0}(\tau)$, $\rho_{S_1}(\tau)$, and $\rho_{S_2}(\tau)$ at $\tau = 1.0$, $\tau = 1.5$, and $\tau = 2.0$. Which solver has the highest win probability at $\tau = 1.0$, and which solver has the smallest worst-case slowdown $\max_{p\in\mathcal{P}} r_{p,s}$?

Exercise 10.2 *Comparing Primal Heuristics via the Primal Integral*

Suppose we run two anytime heuristics H_0 and H_1 on a minimization problem instance whose optimal objective value is $z^* = 20$ over a time limit of $t_{\max} = 100$ s:

- Heuristic H_0 finds an initial feasible solution with objective $\bar{z} = 80$ at $t = 10$ s, improves it to $\bar{z} = 40$ at $t = 30$ s, and reaches $\bar{z} = 25$ at $t = 80$ s (no further improvements until $t_{\max} = 100$ s).
- Heuristic H_1 finds its first feasible solution at $t = 40$ s with objective $\bar{z} = 50$, and improves it directly to the optimal value $\bar{z} = 20$ at $t = 60$ s.

1. Using Definitions 10.7 and 10.8, write down the piecewise constant primal gap functions $p_{H_0}(t)$ and $p_{H_1}(t)$ on $[0, 100]$.
2. Compute the primal integrals $P_{H_0}(50)$ and $P_{H_1}(50)$ at the intermediate horizon $T = 50$ s, as well as $P_{H_0}(100)$ and $P_{H_1}(100)$ at the full horizon $T = 100$ s.
3. Determine the exact crossover time $T^* \in (60, 100)$ at which $P_{H_1}(T^*) = P_{H_0}(T^*)$.

Exercise 10.3 *Jensen's Inequality and the Arithmetic Mean Fallacy*

Let $a_0, \dots, a_{n_p-1} > 0$ and $b_0, \dots, b_{n_p-1} > 0$ be the positive runtimes of two solvers A and B across n_p instances, and let $r_p = a_p/b_p > 0$ denote the normalized runtime of A relative to B on instance $p \in \{0, \dots, n_p - 1\}$.

1. Using the Cauchy–Schwarz inequality (or Jensen's inequality for the strictly convex function $f(x) = 1/x$ on $(0, \infty)$), prove that:

$$\left(\frac{1}{n_p}\sum_{p=0}^{n_p-1}\frac{a_p}{b_p}\right)\cdot\left(\frac{1}{n_p}\sum_{p=0}^{n_p-1}\frac{b_p}{a_p}\right) \geq 1,$$

   with equality if and only if $a_0/b_0 = a_1/b_1 = \cdots = a_{n_p-1}/b_{n_p-1}$.
2. Construct an explicit 2-instance example ($n_p = 2$) where $\mathrm{GM}(a_0/b_0, a_1/b_1) = 1$ (the two solvers are perfectly symmetric), yet normalizing with respect to B gives an arithmetic mean of 1.25 (making A look 25% slower than B), and normalizing with respect to A also gives an arithmetic mean of 1.25 (making B look 25% slower than A).

Exercise 10.4 *Sensitivity to Uncompetitive Solvers in Performance Profiles*

Unlike cactus plots, a solver's performance profile $\rho_s(\tau)$ depends on the entire portfolio $\mathcal{S}$ through the virtual best runtime $t_p^* = \min_{s'\in\mathcal{S}} t_{p,s'}$.

1. Consider $n_p = 4$ instances $\mathcal{P} = \{p_0, p_1, p_2, p_3\}$ and two solvers S_0, S_1 with runtimes:

$$t_{\cdot,S_0} = (10, 10, 20, 20), \qquad t_{\cdot,S_1} = (15, 15, 10, 10).$$

Compute the performance profiles $\rho_{S_0}(\tau)$ and $\rho_{S_1}(\tau)$ for the two-solver portfolio $\mathcal{S} = \{S_0, S_1\}$ at $\tau \in \{1.0, 1.5, 2.0\}$. Which solver appears better at $\tau = 1.5$?

2. Now suppose a third solver S_2 is added to the benchmark with runtimes $t_{\cdot,S_2} = (5, 5, 100, 100)$. Recompute the virtual best runtimes t_p^* and the performance profiles $\rho_{S_0}(\tau)$ and $\rho_{S_1}(\tau)$ for the expanded portfolio $\mathcal{S}' = \{S_0, S_1, S_2\}$ at $\tau = 1.5$. Explain why adding S_2 alters the relative ordering of $\rho_{S_0}(1.5)$ and $\rho_{S_1}(1.5)$, and discuss why pairwise scatter plots or pairwise geometric means are useful sanity checks alongside multi-solver performance profiles.

APPENDIX A

Kruskal's Algorithm and Disjoint Sets

The *Minimum Spanning Tree* (MST) problem is one of the most fundamental and historically significant problems in combinatorial optimization. First formulated by Otakar Borůvka in 1926 to optimize the electrification network of Southern Moravia, the MST problem asks for a tree of minimum total edge weight connecting all vertices of a weighted undirected graph.

In this textbook, minimum spanning trees play a critical role as combinatorial relaxations for **NP**-hard problems, most notably in establishing the 1-tree lower bounds for the Traveling Salesperson Problem (Chapter 3) and in providing the baseline subproblems for Lagrangian relaxation (Chapter 6). Among the classical algorithms for the MST problem, *Kruskal's algorithm*, introduced by Joseph Kruskal in 1956 [Kru56], is particularly versatile because its greedy edge selection directly accommodates pre-committed (mandatory) edges and forbidden (excluded) edges.

This appendix provides a self-contained, mathematically rigorous treatment of Kruskal's algorithm, the foundational Cut Property, the Disjoint-Set (Union-Find) data structure, and an idiomatic Java implementation.

A.1 The Minimum Spanning Tree Problem

Let $G = (V, E)$ be a connected, undirected graph with vertex set $V = \{0, 1, \ldots, n-1\}$, edge set E, and a weight function $w : E \to \mathbb{R}$ assigning a real weight $w(e)$ to each edge $e = \{u, v\} \in E$. We denote $n = |V|$ and $m = |E|$.

Definition A.1 *Spanning Tree*

A subgraph $T = (V, E_T)$ of G is a *spanning tree* of G if:

1. T contains all vertices of G (it is *spanning*).
2. T is connected.
3. T contains no simple cycles (it is *acyclic*).

Equivalently, T is a connected subgraph spanning all n vertices with exactly $n-1$ edges: $|E_T| = n - 1$.

The total weight of a spanning tree $T = (V, E_T)$ is the sum of the weights of its constituent edges:

$$w(T) = \sum_{e \in E_T} w(e). \tag{A.1}$$

Definition A.2 *Minimum Spanning Tree (MST)*

A spanning tree T^* of G is a *Minimum Spanning Tree* (MST) if its total weight is minimal among all spanning trees of G:

$$w(T^*) = \min\{w(T) : T \text{ is a spanning tree of } G\}. \tag{A.2}$$

If all edge weights in G are distinct, the minimum spanning tree is unique. If some edges have identical weights, multiple minimum spanning trees may exist with the same minimum total weight.

A.2 The Cut Property and Greedy Optimality

The design of efficient algorithms for the MST problem relies on a fundamental combinatorial property of graphs known as the *Cut Property*.

Definition A.3 *Cut and Crossing Edges*

Let $G = (V, E)$ be a graph.

1. A *cut* $(S, V \setminus S)$ is a partition of the vertex set V into two non-empty, disjoint subsets S and $V \setminus S$.
2. An edge $e = \{u, v\} \in E$ is said to *cross* the cut $(S, V \setminus S)$ if one endpoint lies in S and the other lies in $V \setminus S$ (without loss of generality, $u \in S$ and $v \in V \setminus S$). We denote the set of crossing edges by $\delta(S)$.
3. A set of edges $A \subseteq E$ *respects* the cut $(S, V \setminus S)$ if no edge in A crosses the cut: $A \cap \delta(S) = \emptyset$.
4. An edge $e^* \in \delta(S)$ is a *light edge* crossing the cut $(S, V \setminus S)$ if its weight is minimum among all edges crossing the cut:

$$w(e^*) = \min_{e \in \delta(S)} w(e). \tag{A.3}$$

Theorem A.1 *The Cut Property*

Let $G = (V, E)$ be a connected, weighted undirected graph. Let $A \subset E$ be a subset of edges that is contained in some Minimum Spanning Tree of G. Let $(S, V \setminus S)$ be any cut of G that respects A. If e^* is a light edge crossing $(S, V \setminus S)$, then:

$$A \cup \{e^*\} \text{ is also contained in some Minimum Spanning Tree of } G. \tag{A.4}$$

Proof. Let $T = (V, E_T)$ be an MST of G that contains A ($A \subseteq E_T$). If $e^* \in E_T$, then $A \cup \{e^*\} \subseteq E_T$, and the claim holds immediately.

Now assume $e^* \notin E_T$. Because T is a spanning tree, adding $e^* = \{u, v\}$ to T creates a unique simple cycle C in $T \cup \{e^*\}$. Since e^* crosses the cut $(S, V \setminus S)$, its endpoints u and v lie on opposite sides of the cut ($u \in S$ and $v \in V \setminus S$). Therefore, tracing cycle C from u to v along the path in T requires crossing the cut at least once in the opposite direction. Thus, there exists at least one edge $e' = \{x, y\} \in E_T$ on cycle C that also crosses the cut: $e' \in E_T \cap \delta(S)$ with $e' \neq e^*$.

Because the cut $(S, V \setminus S)$ respects A, no edge of A crosses the cut. Consequently, e' cannot belong to A ($e' \notin A$). We construct a new spanning subgraph:

$$T' = (E_T \setminus \{e'\}) \cup \{e^*\}. \tag{A.5}$$

Removing e' breaks the cycle C, leaving T' acyclic and connected with exactly $n-1$ edges; thus, T' is a valid spanning tree of G. Furthermore, since $e' \notin A$, we have $A \subseteq E_T \setminus \{e'\}$, which implies $A \cup \{e^*\} \subseteq E_{T'}$.

The weight of the new tree is:

$$w(T') = w(T) - w(e') + w(e^*). \tag{A.6}$$

Since e^* is a light edge crossing the cut $(S, V \setminus S)$ and e' also crosses this cut, we have $w(e^*) \leq w(e')$, which yields:

$$w(T') \leq w(T). \tag{A.7}$$

Because T is an MST, $w(T)$ is minimal, forcing $w(T') = w(T)$. Hence, T' is also an MST of G. Since $A \cup \{e^*\} \subseteq E_{T'}$, the edge e^* can be safely added to A. □

The Cut Property establishes that greedy, locally optimal edge choices on crossing cuts never compromise global optimality.

A.3 Kruskal's Algorithm and Invariants

Kruskal's algorithm [Kru56] maintains an acyclic forest $F = (V, E_F)$ initially consisting of n isolated vertices ($E_F = \emptyset$). The algorithm examines the edges of G in non-decreasing order of weight and greedily adds an edge to E_F if and only if it connects two previously disconnected components.

Algorithm 23: Kruskal's Minimum Spanning Tree Algorithm.

Input: Connected weighted graph $G = (V, E, w)$ with $V = \{0, 1, \ldots, n-1\}$
Output: Minimum Spanning Tree edges E_F

$E_F \leftarrow \emptyset$;
// Initialize empty forest
Sort E in non-decreasing order of weight: $w(e_0) \leq w(e_1) \leq \cdots \leq w(e_{m-1})$;
Initialize Disjoint-Set $\mathcal{D}$ with n singleton sets $\{0\}, \{1\}, \ldots, \{n-1\}$;
for $k \leftarrow 0$ **to** $m-1$ **do**
    $e_k = \{u, v\} \leftarrow E[k]$;
    if $\mathcal{D}.find(u) \neq \mathcal{D}.find(v)$ **then**
        $E_F \leftarrow E_F \cup \{e_k\}$;
        // Add safe edge to forest
        $\mathcal{D}$.union(u, v);
        // Merge the two connected components
        if $|E_F| == n-1$ **then**
            break;
            // Tree is complete
return E_F

Theorem A.2 *Correctness of Kruskal's Algorithm*

Kruskal's algorithm terminates with a Minimum Spanning Tree of G.

Proof. We proceed by induction on the size of the edge set E_F.

- **Base Case**: Before the loop begins, $E_F = \emptyset$, which is trivially a subset of any MST.
- **Inductive Step**: Suppose E_F is a subset of some MST. When Kruskal's algorithm selects edge $e = \{u, v\}$ to add to E_F, vertices u and v belong to distinct connected components in the forest (V, E_F). Let $S \subset V$ be the set of vertices in the connected component containing u. Because $v \notin S$, edge e crosses the cut $(S, V \setminus S)$.

  By construction, no edge currently in E_F crosses $(S, V \setminus S)$, because S is an entire connected component of (V, E_F). Thus, the cut $(S, V \setminus S)$ respects E_F. Furthermore, since edges are considered in non-decreasing order of weight, any edge crossing $(S, V \setminus S)$ that was examined previously must have formed a cycle within S or within $V \setminus S$, meaning it did not cross the cut. Therefore, e is a light edge crossing $(S, V \setminus S)$. By the Cut Property (Theorem A.1), $E_F \cup \{e\}$ is also contained in some MST.

When $|E_F| = n-1$, the forest is connected and acyclic, and hence constitutes a valid MST. □

A.4 The Disjoint-Set Data Structure

At line 6 of Algorithm 23, Kruskal's algorithm must determine whether the two endpoints u and v of candidate edge $e = \{u, v\}$ belong to the same connected component. Performing a depth-first or breadth-first search at every edge would require $\mathcal{O}(n)$ time per edge, resulting in an impractical $\mathcal{O}(m \cdot n)$ total running time.

The *Disjoint-Set* (or *Union-Find*) data structure maintains a collection of disjoint dynamic sets $\mathcal{S} = \{S_0, S_1, \dots\}$ partitioning the universe $V = \{0, 1, \dots, n-1\}$. Each set is identified by a unique *representative* element (the root of a directed tree). The data structure supports three fundamental operations:

- `makeSet`(i): Creates a new set containing the single element i, which serves as its own representative.
- `find`(i): Returns the canonical representative of the set containing element i.
- `union`(i, j): Merges the set containing i with the set containing j into a single set.

A.4.1 Union by Rank and Path Compression

A naive parent-pointer tree implementation where `parent[i]` points to i's parent can degrade into a linear chain of length n, yielding an undesirable $\mathcal{O}(n)$ time per `find`. To guarantee near-constant amortized performance, two complementary optimizations are essential:

1. **Union by Rank**: Maintain an integer array `rank[i]`, which represents an upper bound on the height of the subtree rooted at i. When merging two sets with roots r_u and r_v:
   - If `rank`$[r_u] <$ `rank`$[r_v]$, attach r_u as a child of r_v (the rank of r_v does not change).
   - If `rank`$[r_u] >$ `rank`$[r_v]$, attach r_v as a child of r_u.
   - If `rank`$[r_u] ==$ `rank`$[r_v]$, arbitrarily make r_v a child of r_u and increment `rank`$[r_u]$ by 1.

   Union by rank ensures that the height of any tree on k elements never exceeds $\lfloor \log_2 k \rfloor$.
2. **Path Compression**: During a call to `find`(i), make every node along the path from i to the root point directly to the root. While path compression does not change the tree during the traversal to the root, it flattens the tree during the recursion unwinding, so subsequent lookups along that branch take $\mathcal{O}(1)$ time.

Theorem A.3 *Complexity of Disjoint-Set with Both Heuristics*

A sequence of m operations on a Disjoint-Set data structure of n elements utilizing both union by rank and path compression executes in:

$$\mathcal{O}(m \cdot \alpha(n)) \text{ time}, \tag{A.8}$$

where $\alpha(n)$ denotes the extremely slow-growing inverse Ackermann function [Tar75].

The value of $\alpha(n)$ satisfies $\alpha(n) \leq 4$ for all $n \leq 10^{80}$ (a number vastly exceeding the estimated number of atoms in the observable universe). For all practical computational purposes, $\alpha(n)$ is a small constant, rendering each Union-Find operation effectively $\mathcal{O}(1)$ amortized.

A.5 Java Implementation

Listing A.1 provides a clean, production-grade implementation of the `UnionFind` class with path compression and union by rank.

```
public class UnionFind {
    private final int[] parent;
    private final int[] rank;
    private int count; // Number of disjoint sets

    /**
     * Initializes Union-Find on {0, 1, ..., n-1}.
     * @param n Universe size.
     */
    public UnionFind(int n) {
        this.parent = new int[n];
        this.rank = new int[n];
        this.count = n;
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            rank[i] = 0;
        }
    }

    /**
     * Finds canonical root of element i
     * using path compression.
     */
    public int find(int i) {
        if (parent[i] != i) {
            parent[i] = find(parent[i]); // Path compression
        }
        return parent[i];
    }

    /**
     * Merges sets containing i and j using union by rank.
     * @return true if merged; false if already connected.
     */
    public boolean union(int i, int j) {
        int rootI = find(i);
        int rootJ = find(j);
        if (rootI == rootJ) {
```

```
            // Already in same component (cycle detected)
            return false;
        }
        // Union by rank: attach smaller under larger
        if (rank[rootI] < rank[rootJ]) {
            parent[rootI] = rootJ;
        } else if (rank[rootI] > rank[rootJ]) {
            parent[rootJ] = rootI;
        } else {
            parent[rootJ] = rootI;
            rank[rootI]++;
        }
        count--;
        return true;
    }

    /** Returns true if i and j belong to the same set. */
    public boolean connected(int i, int j) {
        return find(i) == find(j);
    }

    /** Returns current number of disjoint components. */
    public int getCount() {
        return count;
    }
}
```

Listing A.1: The Disjoint-Set (Union-Find) data structure in Java.

Listing A.2 implements Kruskal's algorithm using the `UnionFind` structure. Crucially, we also provide the method `computeConditionedMST`, which pre-inserts mandatory edges E_{fixed} before processing free edges, directly supporting the TSP Branch and Bound bounding procedure introduced in Section 3.3.3.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class KruskalMST {

    public static class Edge implements Comparable<Edge> {
        public final int u;
        public final int v;
        public final double weight;

        public Edge(int u, int v, double weight) {
            this.u = u;
            this.v = v;
            this.weight = weight;
        }

        @Override
        public int compareTo(Edge other) {
            return Double.compare(
                    this.weight, other.weight);
        }
    }

    /**
     * Standard Kruskal MST computation on undirected graph.
     * @param n Number of vertices {0, 1, ..., n-1}.
```

```
28       * @param edges Available candidate edges.
29       * @return List of n - 1 MST edges, or empty list.
30       */
31      public static List<Edge> computeMST(
32              int n, List<Edge> edges) {
33          List<Edge> sortedEdges = new ArrayList<>(edges);
34          Collections.sort(sortedEdges);
35
36          UnionFind uf = new UnionFind(n);
37          List<Edge> mst = new ArrayList<>();
38
39          for (Edge e : sortedEdges) {
40              if (uf.union(e.u, e.v)) {
41                  mst.add(e);
42                  if (mst.size() == n - 1) {
43                      break;
44                  }
45              }
46          }
47          return mst;
48      }
49
50      /**
51       * Computes a Conditioned MST with mandatory edges.
52       * Used for bounding in TSP Branch and Bound.
53       *
54       * @param n Number of vertices {0, 1, ..., n-1}.
55       * @param freeEdges Undecided edges for selection.
56       * @param mandatoryEdges Pre-committed edges.
57       * @return List of n - 1 edges, or null if infeasible.
58       */
59      public static List<Edge> computeConditionedMST(
60              int n,
61              List<Edge> freeEdges,
62              List<Edge> mandatoryEdges) {
63          UnionFind uf = new UnionFind(n);
64          List<Edge> mst = new ArrayList<>();
65
66          // Phase 1: Pre-insert mandatory edges
67          for (Edge e : mandatoryEdges) {
68              if (!uf.union(e.u, e.v)) {
69                  // Premature cycle: infeasible!
70                  return null;
71              }
72              mst.add(e);
73          }
74
75          // Phase 2: Greedily complete the spanning forest
76          List<Edge> sortedFree = new ArrayList<>(freeEdges);
77          Collections.sort(sortedFree);
78
79          for (Edge e : sortedFree) {
80              if (uf.union(e.u, e.v)) {
81                  mst.add(e);
82                  if (mst.size() == n - 1) {
83                      break;
84                  }
85              }
86          }
87
88          // Must connect all n vertices with n - 1 edges
```

```
89            return (mst.size() == n - 1) ? mst : null;
90        }
91    }
```

Listing A.2: Kruskal's algorithm and conditioned MST computation in Java.

A.6 Complexity and Comparison with Prim

Theorem A.4 *Asymptotic Complexity of Kruskal's Algorithm*

For a graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ edges:

1. **Sorting Time**: Sorting the m edges by weight requires $\mathcal{O}(m \log m)$ time. Since $m \leq n^2$, we have $\log m \leq 2 \log n$, so sorting takes $\mathcal{O}(m \log n)$ time.
2. **Union-Find Operations**: The algorithm performs n initializations (`makeSet`), at most $2m$ calls to `find`, and exactly $n - 1$ calls to `union`. By Theorem A.3, these execute in $\mathcal{O}(m \cdot \alpha(n))$ time.
3. **Total Time Complexity**: $\mathcal{O}(m \log n + m \cdot \alpha(n)) = \mathcal{O}(m \log n)$.
4. **Space Complexity**: $\mathcal{O}(n + m)$ to store the edge list and the `parent` and `rank` arrays.

A.6.1 Kruskal vs. Prim: Trade-Offs

While Kruskal's algorithm grows a global spanning forest across the entire graph, *Prim's algorithm* grows a single tree outwardly from an arbitrary seed vertex, always adding the cheapest edge connecting the growing tree to a remaining unvisited vertex. Table A.1 compares their computational trade-offs.

Table A.1: Comparison of Minimum Spanning Tree Algorithms.

Algorithm	Data Structure	Time Complexity	Best Suited For
Kruskal	Array + Union-Find	$\mathcal{O}(m \log n)$	Sparse graphs, conditioned MSTs (E_{fixed})
Prim	Binary Heap	$\mathcal{O}(m \log n)$	Sparse to moderately dense graphs
Prim	Fibonacci Heap	$\mathcal{O}(m + n \log n)$	Theoretical optimum for dense graphs
Prim	Adjacency Matrix	$\mathcal{O}(n^2)$	Complete/dense graphs ($m = \Theta(n^2)$)

Why Kruskal is Indispensable in Branch and Bound. For unconstrained complete graphs where $m = \frac{n(n-1)}{2}$, Prim's algorithm with an adjacency matrix runs in $\mathcal{O}(n^2)$ time, which is asymptotically faster than Kruskal's $\mathcal{O}(n^2 \log n)$ edge-sorting overhead.

However, within a Branch and Bound search tree for the TSP (Chapter 3):

- **Handling Pre-Committed Decisions**: Kruskal's algorithm integrates fixed edges E_{fixed} immediately: by pre-merging all mandatory edges in the Union-Find structure, any cycle formed by E_{fixed} on fewer than n vertices immediately reveals an invalid premature subtour in $\mathcal{O}(1)$ amortized time.
- **Re-using Sorted Edges**: The candidate edges E can be sorted once at the root node in $\mathcal{O}(m \log n)$ time. At child nodes, Kruskal can simply iterate over the already-sorted edge list, skipping edges marked in E_{excluded}, which reduces the per-node cost to near-linear $\mathcal{O}(m \cdot \alpha(n))$ time!

This structural synergy makes Kruskal's algorithm with Disjoint-Set Union one of the most effective building blocks in discrete optimization.

References

[AL03] Emile Aarts and Jan Karel Lenstra. *Local Search in Combinatorial Optimization*. Princeton University Press, 2003 (cit. on p. 185).

[AMO93] Ravindra K Ahuja, Thomas L Magnanti, and James B Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993 (cit. on pp. 140, 181).

[App+06] David L Applegate et al. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006 (cit. on pp. 3, 155, 200, 204).

[Bel57] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957 (cit. on p. 11).

[Ber13] Timo Berthold. "Measuring the Impact of Primal Heuristics". In: *Operations Research Letters* 41.6 (2013), pp. 611–614 (cit. on p. 276).

[BT97] Dimitris Bertsimas and John N Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, 1997 (cit. on p. 84).

[Bla77] Robert G Bland. "New Finite Pivoting Rules for the Simplex Method". In: *Mathematics of Operations Research* 2.1 (1977), pp. 34–44 (cit. on p. 111).

[Cro58] Georges A Croes. "A Method for Solving Traveling-Salesman Problems". In: *Operations Research* 6.6 (1958), pp. 791–812 (cit. on p. 197).

[DFJ54] George Dantzig, Ray Fulkerson, and Selmer Johnson. "Solution of a Large-Scale Traveling-Salesman Problem". In: *Journal of the Operations Research Society of America* 2.4 (1954), pp. 393–410 (cit. on pp. 56, 152, 256).

[Dan57] George B Dantzig. "Discrete-Variable Extremum Problems". In: *Operations Research* 5.2 (1957), pp. 266–277 (cit. on pp. 34, 47, 51).

[Dan63] George B Dantzig. *Linear Programming and Extensions*. Princeton University Press, 1963 (cit. on p. 81).

[Del25] Augustin Delecluse. "Sequence Variables and Search Heuristics for Vehicle Routing Problems in CP". PhD thesis. UCLouvain, 2025 (cit. on p. 277).

[Dij59] Edsger W Dijkstra. "A Note on Two Problems in Connexion with Graphs". In: *Numerische Mathematik* 1.1 (1959), pp. 269–271 (cit. on p. 30).

[Din70] Efim A Dinitz. "Algorithm for Solution of a Problem of Maximum Flow in a Network with Power Estimation". In: *Soviet Mathematics Doklady* 11 (1970), pp. 1277–1280 (cit. on p. 172).

[DM02] Elizabeth D Dolan and Jorge J Moré. "Benchmarking Optimization Software with Performance Profiles". In: *Mathematical Programming* 91.2 (2002), pp. 201–213 (cit. on pp. 269, 272).

[Edm71] Jack Edmonds. "Matroids and the Greedy Algorithm". In: *Mathematical Programming* 1.1 (1971), pp. 127–136 (cit. on p. 154).

[EK72] Jack Edmonds and Richard M Karp. "Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems". In: *Journal of the ACM* 19.2 (1972), pp. 248–264 (cit. on p. 172).

[Eve63] Hugh Everett. "Generalized Lagrange Multiplier Method for Solving Problems of Optimum Allocation of Resources". In: *Operations Research* 11.3 (1963), pp. 399–417 (cit. on p. 137).

[Fis81] Marshall L Fisher. "The Lagrangian Relaxation Method for Solving Integer Programming Problems". In: *Management Science* 27.1 (1981), pp. 1–18 (cit. on p. 137).

[FW86] Philip J Fleming and John J Wallace. "How Not to Lie with Statistics: The Correct Way to Summarize Benchmark Results". In: *Communications of the ACM* 29.3 (1986), pp. 218–221 (cit. on p. 278).

[FF56] Lester R Ford and Delbert R Fulkerson. "Maximal Flow through a Network". In: *Canadian Journal of Mathematics* 8 (1956), pp. 399–404 (cit. on pp. 159, 164).

[FF62] Lester R Ford and Delbert R Fulkerson. *Flows in Networks*. Princeton University Press, 1962 (cit. on p. 159).

[Fre97] Eugene C Freuder. "In Pursuit of the Holy Grail". In: *Constraints* 2.1 (1997), pp. 57–61 (cit. on p. 236).

[GJ79] Michael R Garey and David S Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, 1979 (cit. on pp. 1, 11).

[Geo74] Arthur M Geoffrion. "Lagrangian Relaxation for Integer Programming". In: *Mathematical Programming Study* 2 (1974), pp. 82–114 (cit. on pp. 137, 154).

[Gle+21] Ambros Gleixner et al. "MIPLIB 2017: Data-Driven Compilation of the 6th Mixed-Integer Programming Library". In: *Mathematical Programming Computation* 13 (2021), pp. 443–490 (cit. on pp. 272, 279).

[Glo89] Fred Glover. "Tabu Search—Part I". In: *ORSA Journal on Computing* 1.3 (1989), pp. 190–206 (cit. on p. 218).

[GL97] Fred Glover and Manuel Laguna. *Tabu Search*. Kluwer Academic Publishers, 1997 (cit. on p. 218).

[GLN05] Daniel Godard, Philippe Laborie, and Wim Nuijten. "Randomized Large Neighborhood Search for Cumulative Scheduling". In: *Proceedings of the Fifteenth International Conference on Automated Planning and Scheduling (ICAPS)*. 2005, pp. 81–89 (cit. on p. 226).

[Gom58] Ralph E Gomory. "Outline of an Algorithm for Integer Solutions to Linear Programs". In: *Bulletin of the American Mathematical Society* 64.5 (1958), pp. 275–278 (cit. on p. 132).

[HZ07] Eric A Hansen and Rong Zhou. "Anytime Heuristic Search". In: *Journal of Artificial Intelligence Research* 28 (2007), pp. 267–297 (cit. on p. 37).

[HR55] Theodore E Harris and Frank S Ross. *Fundamentals of a Method for Evaluating Rail Net Capacities*. Tech. rep. RM-1573. Santa Monica, CA: The RAND Corporation, 1955 (cit. on p. 159).

[HNR68] Peter E Hart, Nils J Nilsson, and Bertram Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107 (cit. on pp. 31, 32).

[HK70] Michael Held and Richard M Karp. “The Traveling-Salesman Problem and Minimum Spanning Trees”. In: *Operations Research* 18.6 (1970), pp. 1138–1162 (cit. on pp. 61, 137, 145, 154, 155).

[HK71] Michael Held and Richard M Karp. “The Traveling-Salesman Problem and Minimum Spanning Trees: Part II”. In: *Mathematical Programming* 1.1 (1971), pp. 6–25 (cit. on pp. 137, 145, 149, 154, 155).

[Hel00] Keld Helsgaun. “An Effective Implementation of the Lin–Kernighan Traveling Salesman Heuristic”. In: *European Journal of Operational Research* 126.1 (2000), pp. 106–130 (cit. on p. 208).

[HS04] Holger H Hoos and Thomas Stützle. *Stochastic Local Search: Foundations and Applications*. Morgan Kaufmann, 2004 (cit. on p. 185).

[JM97] David S Johnson and Lyle A McGeoch. “The Traveling Salesman Problem: A Case Study in Local Optimization”. In: *Local Search in Combinatorial Optimization*. Ed. by Emile Aarts and Jan Karel Lenstra. John Wiley & Sons, 1997, pp. 215–310 (cit. on p. 155).

[JV87] Roy Jonker and Anton Volgenant. “A Shortest Augmenting Path Algorithm for Dense and Sparse Linear Assignment Problems”. In: *Computing* 38.4 (1987), pp. 325–340 (cit. on p. 58).

[JV83] Roy Jonker and Ton Volgenant. “Transforming Asymmetric into Symmetric Traveling Salesman Problems”. In: *Operations Research Letters* 2.4 (1983), pp. 161–163 (cit. on pp. 204, 205).

[Kar84] Narendra Karmarkar. “A New Polynomial-Time Algorithm for Linear Programming”. In: *Combinatorica* 4.4 (1984), pp. 373–395 (cit. on p. 111).

[Kar72] Richard M Karp. “Reducibility among Combinatorial Problems”. In: *Complexity of Computer Computations*. Springer, 1972, pp. 85–103 (cit. on p. 12).

[Kha79] Leonid G Khachiyan. “A Polynomial Algorithm in Linear Programming”. In: *Soviet Mathematics Doklady* 20 (1979), pp. 191–194 (cit. on p. 111).

[KM72] Victor Klee and George J Minty. “How Good is the Simplex Algorithm?” In: *Inequalities III*. Academic Press, 1972, pp. 159–175 (cit. on p. 111).

[Kle67] Morton Klein. “A Primal Method for Minimal Cost Flows with Applications to the Assignment and Transportation Problems”. In: *Management Science* 14.3 (1967), pp. 205–220 (cit. on p. 179).

[Kru56] Joseph B Kruskal. “On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem”. In: *Proceedings of the American Mathematical Society* 7.1 (1956), pp. 48–50 (cit. on pp. 285, 287).

[Kuh55] Harold W Kuhn. “The Hungarian Method for the Assignment Problem”. In: *Naval Research Logistics Quarterly* 2.1-2 (1955), pp. 83–97 (cit. on p. 58).

[LD60] Ailsa H Land and Alison G Doig. “An Automatic Method of Solving Discrete Programming Problems”. In: *Econometrica* 28.3 (1960), pp. 497–520 (cit. on p. 47).

[Lau78] Jean-Louis Laurière. “A Language and a Program for Stating and Solving Combinatorial Problems”. In: *Artificial Intelligence* 10.1 (1978), pp. 29–127 (cit. on p. 237).

[Lem54] Carlton E Lemke. “The Dual Method of Solving the Linear Programming Problem”. In: *Naval Research Logistics Quarterly* 1.1 (1954), pp. 36–47 (cit. on p. 124).

[LK73] Shen Lin and Brian W Kernighan. "An Effective Heuristic Algorithm for the Traveling-Salesman Problem". In: *Operations Research* 21.2 (1973), pp. 498–516 (cit. on p. 208).

[Lit+63] John DC Little et al. "An Algorithm for the Traveling Salesman Problem". In: *Operations Research* 11.6 (1963), pp. 972–989 (cit. on p. 47).

[Mac77] Alan K Mackworth. "Consistency in Networks of Relations". In: *Artificial Intelligence* 8.1 (1977), pp. 99–118 (cit. on pp. 237, 247, 255).

[MSV21] Laurent Michel, Pierre Schaus, and Pascal Van Hentenryck. "MiniCP: A Lightweight Solver for Constraint Programming". In: *Mathematical Programming Computation* 13.1 (2021), pp. 133–184 (cit. on pp. 235, 255, 256).

[Or76] Ilhan Or. "Traveling Salesman-Type Combinatorial Problems and Their Relation to the Logistics of Blood Banking". PhD thesis. Department of Industrial Engineering and Management Science, Northwestern University, 1976 (cit. on p. 203).

[Poh70] Ira Pohl. "Heuristic Search Viewed as Path Finding in a Graph". In: *Artificial Intelligence* 1.3-4 (1970), pp. 193–204 (cit. on p. 36).

[Pol87] Boris T Polyak. *Introduction to Optimization*. Optimization Software, New York, 1987 (cit. on p. 149).

[Rég94] Jean-Charles Régin. "A Filtering Algorithm for Constraints of Difference in CSPs". In: *AAAI* 94 (1994), pp. 362–367 (cit. on pp. 179, 264).

[RVW06] Francesca Rossi, Peter Van Beek, and Toby Walsh. *Handbook of Constraint Programming*. Elsevier, 2006 (cit. on pp. 235, 237, 247, 255).

[SW11] Robert Sedgewick and Kevin Wayne. *Algorithms*. 4th. Addison-Wesley Professional, 2011 (cit. on pp. 10, 82).

[Sha98] Paul Shaw. "Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems". In: *Principles and Practice of Constraint Programming—CP98* (1998), pp. 417–431 (cit. on p. 216).

[Sho85] Naum Zuselevich Shor. *Minimization Methods for Non-Differentiable Functions*. Springer-Verlag, 1985 (cit. on pp. 143, 144).

[Sid+18] Nikos Sidiropoulos et al. "SinaPlot: An Enhanced Chart for Simple and Truthful Representation of Single Observations over Multiple Classes". In: *Journal of Computational and Graphical Statistics* 27.3 (2018), pp. 673–676 (cit. on p. 277).

[Ste+14] Roni Stern et al. "Max is More than Min: Solving Maximization Problems with Heuristic Search". In: *Proceedings of the Seventh Annual Symposium on Combinatorial Search (SoCS)*. 2014, pp. 148–156 (cit. on p. 35).

[Sti45] George J Stigler. "The Cost of Subsistence". In: *Journal of Farm Economics* 27.2 (1945), pp. 303–314 (cit. on p. 112).

[Tai23] Éric D Taillard. *Design of Heuristic Algorithms for Hard Optimization: With Python Codes for the Travelling Salesman Problem*. Springer, 2023 (cit. on pp. 185, 196, 200, 204, 221).

[Tar75] Robert Endre Tarjan. "Efficiency of a Good But Not Linear Set Union Algorithm". In: *Journal of the ACM* 22.2 (1975), pp. 215–225 (cit. on pp. 179, 289).

[VJ97] Christine L Valenzuela and Antonia J Jones. "Estimating the Held-Karp Lower Bound for the Geometric TSP". In: *European Journal of Operational Research* 102.1 (1997), pp. 157–175 (cit. on pp. 149, 150, 155).

[VM05] Pascal Van Hentenryck and Laurent Michel. *Constraint-Based Local Search*. MIT Press, 2005 (cit. on p. 222).

[Wol20] Laurence A Wolsey. *Integer Programming*. 2nd. John Wiley & Sons, 2020 (cit. on p. 72).

Index

A^* algorithm, 31
admissible algorithm, 32
admissible heuristic, 32
anytime algorithm, 275
Anytime Weighted A^* algorithm, 37
arc consistency, 251

benchmarking, 265
bound consistency, 251

cactus plot, 268
Circuit constraint, 258
consistent heuristic, 33
constraint, 237
Constraint Optimization Problem (COP), 237
Constraint Satisfaction Problem (CSP), 237

declarative programming, 236
Dijkstra's algorithm, 30
domain, 237
domain consistency, 251

Element constraint, 257
evaluation function, 31

first-fail heuristic, 263
fixpoint algorithm, 244
    data-driven, 255

geometric mean, 279
    shifted geometric mean, 279

heuristic function, 31
    consistent, 33
Hollywood Principle, 239

informed search, 30

Killer Sudoku, 260
Knight's Tour problem, 260

linear programming relaxation
    Knapsack, 34
local consistency, 247

magic constant, 258
Magic Square problem, 258

n-Queens problem, 237

optimality gap, 273
    proven MIP gap, 273
optimality gap profile, 274

PAR-k score, 267
performance profile, 269, 270
performance ratio, 269
primal gap, 276
primal gap function, 277
primal integral, 276, 277
propagation engine, 244
propagator, 243

SinaPlot, 277
survivorship bias, 267
symmetry breaking, 258

virtual best solver (VBS), 269

Weighted A^* algorithm, 36